

EduPulse AI

Source Code PDF

Student Feedback Analyzer - Team Eureka
Capgemini Exceller Agentify Buildathon
Generated on: 14 June 2026, 02:10 PM

GitHub repository: <https://github.com/enamoredg-n/Edupulse>

Submission Notes

This PDF contains the source code files used for the EduPulse AI project. The code is copied directly from the local project files so that the PDF and GitHub code stay consistent.

Included	Frontend React code, backend NestJS code, Python AI service, Prisma schema, and validation scripts.
Excluded	node_modules, dist/build output, logs, .env secrets, lock files, generated code, binary assets, and mock seed data.
Why excluded	These files are dependencies, generated output, secrets, or large test data. They are not the main source code written for
Third-party code	Libraries are used through package dependencies only. Their internal source code is not included in this PDF.

Project Structure Included

```
edupulse-backend/ai-service/app/__init__.py
edupulse-backend/ai-service/app/main.py
edupulse-backend/ai-service/evaluation/evaluate_ai.py
edupulse-backend/ai-service/requirements.txt
edupulse-backend/nest-cli.json
edupulse-backend/package.json
edupulse-backend/prisma/schema.prisma
edupulse-backend/scripts/ai-dataset-audit.ts
edupulse-backend/scripts/ai-golden-validation.ts
edupulse-backend/scripts/ai-result-utils.ts
edupulse-backend/scripts/tenant-isolation-test.ts
edupulse-backend/src/actions/actions.controller.ts
edupulse-backend/src/actions/actions.module.ts
edupulse-backend/src/actions/actions.service.ts
edupulse-backend/src/actions/dto/create-action-item.dto.ts
edupulse-backend/src/actions/dto/update-action-status.dto.ts
edupulse-backend/src/analysis/ai-analysis-client.service.ts
edupulse-backend/src/analysis/analysis.controller.ts
edupulse-backend/src/analysis/analysis.module.ts
edupulse-backend/src/analysis/analysis.service.ts
edupulse-backend/src/analysis/dto/run-analysis.dto.ts
edupulse-backend/src/app.module.ts
edupulse-backend/src/audit/audit.controller.ts
edupulse-backend/src/audit/audit.module.ts
edupulse-backend/src/audit/audit.service.ts
edupulse-backend/src/auth/auth.controller.ts
edupulse-backend/src/auth/auth.module.ts
edupulse-backend/src/auth/auth.service.ts
edupulse-backend/src/auth/decorators/roles.decorator.ts
edupulse-backend/src/auth/dto/login.dto.ts
edupulse-backend/src/auth/guards/jwt-auth.guard.ts
edupulse-backend/src/auth/guards/roles.guard.ts
edupulse-backend/src/dashboard/dashboard.controller.ts
edupulse-backend/src/dashboard/dashboard.module.ts
edupulse-backend/src/dashboard/dashboard.service.ts
edupulse-backend/src/feedback/dto/admin-feedback-form.dto.ts
edupulse-backend/src/feedback/dto/submit-feedback.dto.ts
edupulse-backend/src/feedback/feedback.controller.ts
edupulse-backend/src/feedback/feedback.module.ts
edupulse-backend/src/feedback/feedback.service.ts
edupulse-backend/src/health/health.controller.ts
edupulse-backend/src/health/health.module.ts
edupulse-backend/src/main.ts
edupulse-backend/src/prisma/prisma.module.ts
edupulse-backend/src/prisma/prisma.service.ts
edupulse-backend/src/reports/reports.controller.ts
edupulse-backend/src/reports/reports.module.ts
edupulse-backend/src/reports/reports.service.ts
edupulse-backend/tsconfig.build.json
edupulse-backend/tsconfig.json
edupulse-frontend/index.html
edupulse-frontend/package.json
edupulse-frontend/src/App.css
edupulse-frontend/src/App.tsx
edupulse-frontend/src/components/LandingPage.css
edupulse-frontend/src/components/LandingPage.tsx
edupulse-frontend/src/index.css
edupulse-frontend/src/main.tsx
edupulse-frontend/vite.config.ts
```

Source Code

Total source files included: 59. Each file starts on a new page with its project path.

[edupulse-backend/ai-service/app/__init__.py](#)

File size: 1 bytes

1 |

File size: 116,036 bytes

```

1 | from __future__ import annotations
2 |
3 | import json
4 | import math
5 | import os
6 | import re
7 | import time
8 | import urllib.error
9 | import urllib.request
10 | from collections import Counter, defaultdict
11 | from datetime import datetime, timezone
12 | from pathlib import Path
13 | from typing import Any, Literal
14 |
15 | from fastapi import FastAPI
16 | from pydantic import BaseModel, Field
17 |
18 | try:
19 |     import numpy as np
20 |     from sklearn.cluster import HDBSCAN
21 |     from sklearn.cluster import MiniBatchKMeans
22 |     from sklearn.feature_extraction.text import TfidfVectorizer
23 | except Exception: # pragma: no cover - fallback mode for minimal installs
24 |     np = None
25 |     HDBSCAN = None
26 |     MiniBatchKMeans = None
27 |     TfidfVectorizer = None
28 |
29 |
30 | Sentiment = Literal["POSITIVE", "NEUTRAL", "NEGATIVE", "CRITICAL"]
31 | Priority = Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]
32 |
33 | CRITICAL_WORDS = {
34 |     "ragging",
35 |     "ragged",
36 |     "harassment",
37 |     "abuse",
38 |     "corruption",
39 |     "bribe",
40 |     "extortion",
41 |     "unsafe",
42 |     "snake",
43 |     "gun",
44 |     "weapon",
45 |     "firing",
46 |     "shooting",
47 |     "threat",
48 |     "threatening",
49 |     "violence",
50 |     "assault",
51 |     "emergency",
52 |     "bully",
53 |     "bullying",
54 |     "intimidating",
55 |     "stomach problem",
56 |     "stomach problems",
57 |     "food poisoning",
58 |     "contaminated food",
59 |     "illness",
60 | }
61 |
62 | ULTRA_CONCERN_WORDS = {
63 |     "ragging",
64 |     "ragged",
65 |     "harassment",
66 |     "harass",
67 |     "misconduct",
68 |     "abuse",
69 |     "bully",
70 |     "bullying",
71 |     "physical touch",
72 |     "molest",
73 |     "assault",
74 |     "violence",
75 |     "threat",
76 |     "threatening",
77 |     "gun",
78 |     "weapon",
79 |     "firing",
80 |     "shooting",
81 |     "knife",
82 |     "snake",
83 |     "animal attack",
84 |     "corruption",
85 |     "bribe",
86 |     "extortion",
87 |     "forced money",
88 |     "taking money",
89 |     "water contamination",
90 |     "water ph",
91 |     "ph level",
92 |     "contaminated water",
93 |     "food poisoning",
94 |     "electric shock",
95 |     "electrocution",
96 |     "fire",
97 |     "gas leak",
98 |     "ceiling fall",
99 |     "ceiling plaster",
100 |     "roof fall",
101 |     "wall crack",
102 |     "building collapse",
103 |     "plaster fall",
104 |     "plaster falling",
105 |     "can injure",

```

```

106 | }
107 |
108 | HEALTH_RISK_WORDS = {
109 |     "insects",
110 |     "medical",
111 |     "health",
112 |     "illness",
113 |     "sick",
114 |     "stomach",
115 |     "uncooked",
116 |     "undercooked",
117 | }
118 |
119 | NEGATIVE_WORDS = {
120 |     "bad",
121 |     "poor",
122 |     "worst",
123 |     "dirty",
124 |     "slow",
125 |     "broken",
126 |     "partiality",
127 |     "issue",
128 |     "problem",
129 |     "unfair",
130 |     "late",
131 |     "cold",
132 |     "rude",
133 |     "behaviour",
134 |     "behavior",
135 |     "corruption",
136 |     "short",
137 |     "lack",
138 |     "lacks",
139 |     "not working",
140 |     "unavailable",
141 |     "not available",
142 | }
143 |
144 | FACULTY_ATTENDANCE_HINTS = {
145 |     "not attending",
146 |     "not taking class",
147 |     "not taking classes",
148 |     "not coming",
149 |     "absent",
150 |     "late to class",
151 |     "late in class",
152 |     "classes are missed",
153 |     "class is missed",
154 |     "irregular class",
155 |     "irregular classes",
156 |     "skips class",
157 |     "skip classes",
158 |     "misses lecture",
159 |     "misses lectures",
160 | }
161 |
162 | FACULTY_MARKS_HINTS = {
163 |     "partiality",
164 |     "marks",
165 |     "practical marks",
166 |     "internal marks",
167 |     "unfair marking",
168 |     "biased marks",
169 | }
170 |
171 | FACULTY_BEHAVIOUR_HINTS = {
172 |     "behaviour",
173 |     "behavior",
174 |     "rude",
175 |     "harshly",
176 |     "insult",
177 |     "respectful",
178 |     "ignore",
179 |     "scold",
180 |     "discouraging",
181 | }
182 |
183 | ISSUE_TAXONOMY: list[dict[str, Any]] = [
184 |     {
185 |         "bucket": "RAGGING",
186 |         "title": "Ragging concern",
187 |         "type": "urgent",
188 |         "phrases": ["ragging", "ragged", "bully", "bullying", "senior forced", "seniors forced"],
189 |     },
190 |     {
191 |         "bucket": "FACULTY_HARASSMENT",
192 |         "title": "Harassment concern",
193 |         "type": "urgent",
194 |         "phrases": [
195 |             "harassment",
196 |             "harass",
197 |             "physical touch",
198 |             "misconduct",
199 |             "molest",
200 |             "abuse",
201 |             "unsafe behaviour",
202 |             "unsafe behavior",
203 |         ],
204 |     },
205 |     {
206 |         "bucket": "WATER_CONTAMINATION",
207 |         "title": "Water contamination concern",
208 |         "type": "urgent",
209 |         "phrases": ["water contamination", "contaminated water", "water ph", "ph level", "unsafe drinking water"],
210 |     },
211 |     {
212 |         "bucket": "WEAPON_VIOLENCE",
213 |         "title": "Weapon or violence concern",
214 |         "type": "urgent",
215 |         "phrases": ["gun", "weapon", "firing", "shooting", "knife", "violence", "assault", "attack"],

```

```

216 | },
217 | {
218 |     "bucket": "ANIMAL_DANGER",
219 |     "title": "Campus animal danger",
220 |     "type": "urgent",
221 |     "phrases": ["snake", "animal attack", "dog bite"],
222 | },
223 | {
224 |     "bucket": "CORRUPTION",
225 |     "title": "Corruption concern",
226 |     "type": "urgent",
227 |     "phrases": ["corruption", "bribe", "extortion", "forced money", "taking money", "money collection was not transparent"],
228 | },
229 | {
230 |     "bucket": "BUILDING_INJURY",
231 |     "title": "Building injury concern",
232 |     "type": "urgent",
233 |     "phrases": ["ceiling fall", "ceiling plaster", "roof fall", "wall crack", "building collapse", "plaster fall", "plaster falling", "can injure"],
234 | },
235 | {
236 |     "bucket": "ELECTRIC_FIRE",
237 |     "title": "Electric or fire safety concern",
238 |     "type": "urgent",
239 |     "phrases": ["electric shock", "electrocution", "short circuit", "fire", "smoke", "gas leak"],
240 | },
241 | {
242 |     "bucket": "FOOD_POISONING",
243 |     "title": "Food poisoning concern",
244 |     "type": "urgent",
245 |     "phrases": ["food poisoning", "stomach infection", "students got sick", "students fell sick"],
246 | },
247 | {
248 |     "bucket": "FACULTY_MARKS_PARTIALITY",
249 |     "title": "Faculty marks partiality issue",
250 |     "type": "normal",
251 |     "categoryHints": ["faculty", "academics"],
252 |     "phrases": list(FACULTY_MARKS_HINTS),
253 | },
254 | {
255 |     "bucket": "FACULTY_ATTENDANCE",
256 |     "title": "Faculty attendance issue",
257 |     "type": "normal",
258 |     "categoryHints": ["faculty"],
259 |     "phrases": list(FACULTY_ATTENDANCE_HINTS),
260 | },
261 | {
262 |     "bucket": "FACULTY_SUBJECT_KNOWLEDGE",
263 |     "title": "Faculty subject knowledge issue",
264 |     "type": "normal",
265 |     "categoryHints": ["faculty"],
266 |     "phrases": ["lack knowledge", "lacks knowledge", "subject knowledge", "cannot explain", "concepts clearly", "teaching is not helping"],
267 | },
268 | {
269 |     "bucket": "FACULTY_BEHAVIOUR",
270 |     "title": "Faculty behaviour issue",
271 |     "type": "normal",
272 |     "categoryHints": ["faculty"],
273 |     "phrases": list(FACULTY_BEHAVIOUR_HINTS),
274 | },
275 | {
276 |     "bucket": "FACULTY_DOUBT_SUPPORT",
277 |     "title": "Faculty doubt support issue",
278 |     "type": "normal",
279 |     "categoryHints": ["faculty"],
280 |     "phrases": ["doubt", "doubts", "doubt session", "doubt sessions", "doubts are not solved", "not solving doubts"],
281 | },
282 | {
283 |     "bucket": "FACULTY_COMMUNICATION",
284 |     "title": "Faculty communication issue",
285 |     "type": "normal",
286 |     "categoryHints": ["faculty"],
287 |     "phrases": ["communication", "explain", "explanation", "clarity", "teaching clarity", "not explaining"],
288 | },
289 | {
290 |     "bucket": "FACULTY_REPLACEMENT",
291 |     "title": "Faculty replacement request",
292 |     "type": "normal",
293 |     "categoryHints": ["faculty"],
294 |     "phrases": ["new faculty", "replace faculty", "change faculty"],
295 | },
296 | {
297 |     "bucket": "FACULTY_TEACHING_SUPPORT",
298 |     "title": "Faculty teaching support issue",
299 |     "type": "normal",
300 |     "categoryHints": ["faculty"],
301 |     "phrases": [
302 |         "faculty",
303 |         "teacher",
304 |         "teaching",
305 |         "lecture",
306 |         "lectures",
307 |         "class",
308 |         "practical examples",
309 |         "examples needed",
310 |         "more practical examples",
311 |     ],
312 | },
313 | {
314 |     "bucket": "EXAM_SCHEDULING",
315 |     "title": "Exam scheduling issue",
316 |     "type": "normal",
317 |     "categoryHints": ["academics", "examination"],
318 |     "phrases": ["exam schedule", "exam timing", "date sheet", "exam gap", "back to back exam", "exam timetable"],
319 | },
320 | {
321 |     "bucket": "ACADEMIC_ASSESSMENT",
322 |     "title": "Academic assessment clarity issue",
323 |     "type": "normal",
324 |     "categoryHints": ["academics"],
325 |     "phrases": ["assessment", "internal exam", "test clarity", "test marks", "evaluation", "rubric"],

```

```

326 | },
327 | {
328 |     "bucket": "ACADEMIC_WORKLOAD",
329 |     "title": "Academic workload issue",
330 |     "type": "normal",
331 |     "categoryHints": ["academics"],
332 |     "phrases": [
333 |         "exam load",
334 |         "academic load",
335 |         "workload",
336 |         "load feels high",
337 |         "too much load",
338 |         "study pressure",
339 |     ],
340 | },
341 | {
342 |     "bucket": "ACADEMIC_ASSIGNMENT",
343 |     "title": "Assignment scheduling issue",
344 |     "type": "normal",
345 |     "categoryHints": ["academics"],
346 |     "phrases": ["assignment", "assignments", "uploaded late", "less time", "submission"],
347 | },
348 | {
349 |     "bucket": "ACADEMIC_COURSE_CONTENT",
350 |     "title": "Course content clarity issue",
351 |     "type": "normal",
352 |     "categoryHints": ["academics"],
353 |     "phrases": ["course", "course content", "syllabus", "content", "module", "curriculum"],
354 | },
355 | {
356 |     "bucket": "MESS_HYGIENE_AVAILABILITY",
357 |     "title": "Mess hygiene and food availability issue",
358 |     "type": "normal",
359 |     "categoryHints": ["mess", "food mess", "hostel"],
360 |     "phrases": ["mess", "food", "hygiene", "cleanliness", "clean", "insects", "quantity", "gets over", "serving", "dinner", "lunch", "hostel mess", "plate"],
361 | },
362 | {
363 |     "bucket": "HOSTEL_FACILITY",
364 |     "title": "Hostel facility issue",
365 |     "type": "normal",
366 |     "categoryHints": ["hostel"],
367 |     "phrases": ["hostel", "hostel room", "hostel corridor", "hostel gate", "hostel facilities", "hostel maintenance", "hostel water"],
368 | },
369 | {
370 |     "bucket": "MEDICAL_ROOM",
371 |     "title": "Medical room support issue",
372 |     "type": "normal",
373 |     "categoryHints": ["medical", "health", "campus"],
374 |     "phrases": ["medical room", "first aid", "doctor", "nurse", "health room", "medicine not available", "ambulance"],
375 | },
376 | {
377 |     "bucket": "CANTEEN_SERVICE",
378 |     "title": "Canteen service issue",
379 |     "type": "normal",
380 |     "categoryHints": ["canteen"],
381 |     "phrases": ["canteen", "queue", "billing", "food options", "counter"],
382 | },
383 | {
384 |     "bucket": "PLACEMENT_ACCESS",
385 |     "title": "Placement access issue",
386 |     "type": "normal",
387 |     "categoryHints": ["placements", "placement"],
388 |     "phrases": ["placement", "placements", "drive", "eligibility", "shortlisting", "company"],
389 | },
390 | {
391 |     "bucket": "FEST_DURATION",
392 |     "title": "Fest duration issue",
393 |     "type": "normal",
394 |     "categoryHints": ["extracurricular"],
395 |     "phrases": ["fest", "fests", "event", "events", "rushed"],
396 | },
397 | {
398 |     "bucket": "SPORTS_EQUIPMENT",
399 |     "title": "Sports item availability issue",
400 |     "type": "normal",
401 |     "categoryHints": ["sports", "sports campus", "sports and campus", "extracurricular"],
402 |     "phrases": ["sports item", "sports items", "item availability", "equipment", "badminton", "football", "racket", "ground booking", "practice time", "sp"],
403 | },
404 | {
405 |     "bucket": "SPORTS_ACCESS",
406 |     "title": "Sports facility timing issue",
407 |     "type": "normal",
408 |     "categoryHints": ["sports", "sports campus", "sports and campus", "campus"],
409 |     "phrases": ["sports facilities", "timings are limited", "limited timings", "practice timing", "practice timings"],
410 | },
411 | {
412 |     "bucket": "CAMPUS_LIGHTING",
413 |     "title": "Campus lighting issue",
414 |     "type": "normal",
415 |     "categoryHints": ["sports campus", "sports and campus", "campus", "safety"],
416 |     "phrases": ["lighting", "lights", "street lights", "dark area", "not working at night"],
417 | },
418 | {
419 |     "bucket": "CLUB_PARTICIPATION",
420 |     "title": "Campus activity planning issue",
421 |     "type": "normal",
422 |     "categoryHints": ["sports campus", "sports and campus", "extracurricular", "campus"],
423 |     "phrases": [
424 |         "club",
425 |         "clubs",
426 |         "participation",
427 |         "participate",
428 |         "event",
429 |         "events",
430 |         "event slots",
431 |         "extracurricular",
432 |         "activities",
433 |     ],
434 | },
435 | {

```



```

436 |         "bucket": "TRANSPORT_SERVICE",
437 |         "title": "Transport service issue",
438 |         "type": "normal",
439 |         "categoryHints": ["transport", "bus"],
440 |         "phrases": ["bus", "bus timing", "bus route", "pickup", "drop point", "transport", "late bus", "overcrowded bus"],
441 |     },
442 |     {
443 |         "bucket": "LIBRARY_ACCESS",
444 |         "title": "Library access issue",
445 |         "type": "normal",
446 |         "categoryHints": ["library"],
447 |         "phrases": ["library", "reading room", "book availability", "books not available", "library seating", "library ac", "library timing"],
448 |     },
449 |     {
450 |         "bucket": "SCHOLARSHIP_SUPPORT",
451 |         "title": "Scholarship support issue",
452 |         "type": "normal",
453 |         "categoryHints": ["administration", "scholarship"],
454 |         "phrases": ["scholarship", "scholarship form", "fee reimbursement", "financial aid", "documents for scholarship"],
455 |     },
456 |     {
457 |         "bucket": "SECURITY_GUARD_SUPPORT",
458 |         "title": "Security support issue",
459 |         "type": "normal",
460 |         "categoryHints": ["security", "campus", "safety"],
461 |         "phrases": ["security guard", "guard absent", "entry gate", "gate checking", "security not present"],
462 |     },
463 |     {
464 |         "bucket": "PROJECTOR",
465 |         "title": "Projector issue",
466 |         "type": "normal",
467 |         "categoryHints": ["infrastructure", "classroom", "academics"],
468 |         "phrases": ["projector"],
469 |     },
470 |     {
471 |         "bucket": "WIFI",
472 |         "title": "Wi-Fi connectivity issue",
473 |         "type": "normal",
474 |         "phrases": ["wifi", "internet", "network", "connectivity", "access point"],
475 |     },
476 |     {
477 |         "bucket": "LAB_EQUIPMENT",
478 |         "title": "Lab equipment issue",
479 |         "type": "normal",
480 |         "categoryHints": ["infrastructure", "labs", "lab"],
481 |         "phrases": ["lab", "labs", "lab equipment", "equipment availability", "equipment", "components", "mouse", "system", "practical"],
482 |     },
483 |     {
484 |         "bucket": "WASHROOM_MAINTENANCE",
485 |         "title": "Washroom maintenance issue",
486 |         "type": "normal",
487 |         "categoryHints": ["infrastructure", "hostel"],
488 |         "phrases": ["washroom", "toilet", "fittings"],
489 |     },
490 |     {
491 |         "bucket": "AC_VENTILATION",
492 |         "title": "AC and ventilation issue",
493 |         "type": "normal",
494 |         "categoryHints": ["infrastructure", "classroom", "library", "lab"],
495 |         "phrases": ["ac not working", "air conditioning", "ventilation", "suffocation", "fans not working", "room is too hot"],
496 |     },
497 |     {
498 |         "bucket": "CLASSROOM_MAINTENANCE",
499 |         "title": "Classroom maintenance issue",
500 |         "type": "normal",
501 |         "categoryHints": ["infrastructure", "academics"],
502 |         "phrases": ["classroom", "benches", "fan", "maintenance", "room"],
503 |     },
504 |     {
505 |         "bucket": "ADMINISTRATION",
506 |         "title": "Administration service issue",
507 |         "type": "normal",
508 |         "categoryHints": ["administration"],
509 |         "phrases": ["fee counter", "certificate", "office", "documentation", "admin"],
510 |     },
511 | ]
512 |
513 | POSITIVE_WORDS = {
514 |     "good",
515 |     "excellent",
516 |     "helpful",
517 |     "clean",
518 |     "supportive",
519 |     "improved",
520 |     "great",
521 |     "clear",
522 |     "positive",
523 |     "useful",
524 |     "decent",
525 |     "smooth",
526 |     "well",
527 |     "satisfied",
528 | }
529 |
530 | ISSUE_LANGUAGE_WORDS = {
531 |     "affected",
532 |     "availability",
533 |     "broken",
534 |     "cannot",
535 |     "delay",
536 |     "delayed",
537 |     "dirty",
538 |     "faster",
539 |     "gap",
540 |     "gaps",
541 |     "improve",
542 |     "improvement",
543 |     "inconsistent",
544 |     "issue",
545 |     "less time",

```

```

546 |     "limited",
547 |     "load",
548 |     "overload",
549 |     "pressure",
550 |     "heavy",
551 |     "missing",
552 |     "need",
553 |     "needed",
554 |     "needs",
555 |     "not",
556 |     "overlap",
557 |     "poor",
558 |     "problem",
559 |     "repair",
560 |     "should",
561 |     "slow",
562 |     "unavailable",
563 |     "unclear",
564 |     "unfair",
565 |     "worst",
566 | }
567 |
568 | STOPWORDS = {
569 |     "the",
570 |     "and",
571 |     "for",
572 |     "with",
573 |     "that",
574 |     "this",
575 |     "very",
576 |     "from",
577 |     "are",
578 |     "was",
579 |     "were",
580 |     "have",
581 |     "has",
582 |     "not",
583 |     "but",
584 |     "can",
585 |     "our",
586 |     "student",
587 |     "students",
588 |     "college",
589 |     "campus",
590 |     "feedback",
591 |     "issue",
592 |     "problem",
593 | }
594 |
595 | GENERIC_LOW_INFORMATION = {
596 |     "good",
597 |     "bad",
598 |     "nice",
599 |     "ok",
600 |     "okay",
601 |     "excellent",
602 |     "average",
603 |     "satisfied",
604 |     "unsatisfied",
605 |     "no problem",
606 |     "nothing",
607 |     "none",
608 |     "na",
609 |     "n a",
610 | }
611 |
612 | OUT_OF_CONTEXT_HINTS = {
613 |     "cricket",
614 |     "movie",
615 |     "phone battery",
616 |     "weather",
617 |     "traffic",
618 |     "pizza coupon",
619 |     "shopping",
620 | }
621 |
622 | ACTIONABLE_CONTEXT_WORDS = {
623 |     "academic",
624 |     "assignment",
625 |     "assessment",
626 |     "canteen",
627 |     "class",
628 |     "classroom",
629 |     "course",
630 |     "doubt",
631 |     "exam",
632 |     "faculty",
633 |     "fest",
634 |     "fests",
635 |     "food",
636 |     "hostel",
637 |     "hygiene",
638 |     "infrastructure",
639 |     "lab",
640 |     "library",
641 |     "mess",
642 |     "placement",
643 |     "placements",
644 |     "projector",
645 |     "ragging",
646 |     "safety",
647 |     "semester",
648 |     "sports",
649 |     "teacher",
650 |     "teaching",
651 |     "subject",
652 |     "washroom",
653 |     "wifi",
654 |     "wi-fi",
655 | }

```

```

656 |
657 | SERVICE_ROOT = Path(__file__).resolve().parents[1]
658 | DATA_DIR = SERVICE_ROOT / "data"
659 | CORRECTIONS_PATH = DATA_DIR / "human_corrections.jsonl"
660 |
661 |
662 | def load_env_files() -> None:
663 |     backend_root = SERVICE_ROOT.parent
664 |     for env_path in (backend_root / ".env", SERVICE_ROOT / ".env"):
665 |         if not env_path.exists():
666 |             continue
667 |         for line in env_path.read_text(encoding="utf-8").splitlines():
668 |             stripped = line.strip()
669 |             if not stripped or stripped.startswith("#") or "=" not in stripped:
670 |                 continue
671 |             key, value = stripped.split("=", 1)
672 |             os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))
673 |
674 |
675 | load_env_files()
676 |
677 |
678 | class FeedbackItem(BaseModel):
679 |     id: str
680 |     categoryId: str
681 |     categoryName: str
682 |     questionId: str
683 |     questionText: str
684 |     rating: int
685 |     comment: str | None = None
686 |     departmentCode: str | None = None
687 |     semesterNumber: int | None = None
688 |     weight: int = 1
689 |
690 |
691 | class GrievanceItem(BaseModel):
692 |     id: str
693 |     category: str
694 |     title: str
695 |     description: str
696 |     severity: str
697 |     status: str
698 |     safetyFlag: bool = False
699 |     isAnonymous: bool = False
700 |     departmentCode: str | None = None
701 |
702 |
703 | class Term(BaseModel):
704 |     id: str
705 |     name: str
706 |
707 |
708 | class AnalysisRequest(BaseModel):
709 |     term: Term
710 |     feedback: list[FeedbackItem] = Field(default_factory=list)
711 |     grievances: list[GrievanceItem] = Field(default_factory=list)
712 |
713 |
714 | class HumanCorrection(BaseModel):
715 |     reportId: str | None = None
716 |     sourceId: str | None = None
717 |     correctionType: Literal[
718 |         "THEME",
719 |         "SENTIMENT",
720 |         "PRIORITY",
721 |         "QUALITY",
722 |         "MERGE",
723 |         "OTHER",
724 |     ] = "OTHER"
725 |     originalValue: str | None = None
726 |     correctedValue: str
727 |     note: str | None = None
728 |     reviewer: str | None = None
729 |
730 |
731 | app = FastAPI(title="EduPulse AI Service", version="0.1.0")
732 |
733 | _sentiment_pipeline = None
734 | _embedding_model = None
735 |
736 |
737 | @app.get("/health")
738 | def health() -> dict[str, str]:
739 |     return {"status": "ok"}
740 |
741 |
742 | @app.get("/models")
743 | def models() -> dict[str, Any]:
744 |     return model_stack_metadata()
745 |
746 |
747 | @app.post("/human-feedback")
748 | def record_human_feedback(correction: HumanCorrection) -> dict[str, Any]:
749 |     DATA_DIR.mkdir(parents=True, exist_ok=True)
750 |     record = {
751 |         **correction.model_dump(),
752 |         "recordedAt": datetime.now(timezone.utc).isoformat(),
753 |     }
754 |     with CORRECTIONS_PATH.open("a", encoding="utf-8") as file:
755 |         file.write(json.dumps(record) + "\n")
756 |     return {"status": "RECORDED", "correction": record}
757 |
758 |
759 | @app.get("/human-feedback/summary")
760 | def human_feedback_summary() -> dict[str, Any]:
761 |     if not CORRECTIONS_PATH.exists():
762 |         return {"totalCorrections": 0, "byType": {}, "latest": []}
763 |
764 |     rows = [
765 |         json.loads(line)

```

```

766 |         for line in CORRECTIONS_PATH.read_text(encoding="utf-8").splitlines()
767 |         if line.strip()
768 |     ]
769 |     return {
770 |         "totalCorrections": len(rows),
771 |         "byType": dict(Counter(row.get("correctionType", "OTHER") for row in rows)),
772 |         "latest": rows[-5:],
773 |     }
774 |
775 |
776 | @app.post("/analyze")
777 | def analyze(payload: AnalysisRequest) -> dict[str, Any]:
778 |     started_at = time.perf_counter()
779 |     total_feedback_count = weighted_feedback_count(payload.feedback)
780 |     quality = quality_filter(payload.feedback)
781 |     useful_feedback = quality["useful"]
782 |     quality_valid_feedback = [
783 |         item for item in payload.feedback if item.id not in quality["rejectedIds"]
784 |     ]
785 |     feedback_docs = build_feedback_documents(useful_feedback, quality["weights"])
786 |     grievance_docs = build_grievance_documents(payload.grievances)
787 |     documents = feedback_docs + grievance_docs
788 |
789 |     if len(documents) <= 20:
790 |         sentiment_by_doc = {
791 |             doc["id"]: lexicon_sentiment(doc["text"], int(doc.get("rating", 3)))
792 |             for doc in documents
793 |         }
794 |         cluster_result = {
795 |             "labels": [-1 for _ in documents],
796 |             "metadata": {
797 |                 "clusterer": "small_signal_passthrough",
798 |                 "embeddingModel": None,
799 |                 "clusterCount": len(documents),
800 |                 "noiseCount": 0,
801 |             },
802 |         }
803 |     else:
804 |         sentiment_by_doc = classify_sentiments(documents)
805 |         cluster_result = None
806 |     themes = build_themes(documents, sentiment_by_doc, cluster_result)
807 |     category_themes = build_category_themes(quality_valid_feedback, useful_feedback)
808 |     grievance_theme = build_grievance_theme(payload.grievances)
809 |
810 |     all_themes = themes
811 |     if grievance_theme:
812 |         all_themes.insert(0, grievance_theme)
813 |
814 |     all_themes = dedupe_themes(all_themes)
815 |     has_gemini_key = bool(os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY"))
816 |     gemini_reasoning = (
817 |         gemini_reasoning_layer(
818 |             term_name=payload.term.name,
819 |             themes=all_themes,
820 |             quality=quality,
821 |             feedback_count=total_feedback_count,
822 |             grievance_count=len(payload.grievances),
823 |         )
824 |         if has_gemini_key or len(documents) > 20
825 |         else small_signal_reasoning_layer(
826 |             term_name=payload.term.name,
827 |             themes=all_themes,
828 |             quality=quality,
829 |             feedback_count=total_feedback_count,
830 |             grievance_count=len(payload.grievances),
831 |         )
832 |     )
833 |     gemini_reasoning = merge_local_ultra_concerns(gemini_reasoning, all_themes)
834 |     priority_denominator = priority_comment_denominator(quality)
835 |     all_themes = apply_reasoning_priorities(
836 |         all_themes,
837 |         gemini_reasoning,
838 |         priority_denominator,
839 |     )
840 |     all_themes = apply_reasoning_issue_details(all_themes, gemini_reasoning)
841 |     gemini_reasoning = sync_reasoning_with_final_themes(
842 |         gemini_reasoning,
843 |         all_themes,
844 |         priority_denominator,
845 |     )
846 |     visible_themes = sorted(
847 |         exclude_ultra_concerning_themes(all_themes, gemini_reasoning),
848 |         key=lambda theme: (
849 |             priority_rank(theme["priority"]),
850 |             int(theme.get("mentionCount", 0)),
851 |             extract_theme_confidence(theme),
852 |         ),
853 |         reverse=True,
854 |     )
855 |     actions = build_actions(visible_themes)
856 |     narrative = generate_report_narrative(
857 |         term_name=payload.term.name,
858 |         themes=visible_themes,
859 |         actions=actions,
860 |         quality=quality,
861 |         feedback_count=total_feedback_count,
862 |         grievance_count=len(payload.grievances),
863 |         reasoning=gemini_reasoning,
864 |     )
865 |     critical_count = sum(1 for theme in visible_themes if theme["priority"] == "CRITICAL")
866 |     negative_count = sum(
867 |         1
868 |         for theme in visible_themes
869 |         if theme["sentiment"] in {"NEGATIVE", "CRITICAL"}
870 |     )
871 |     processing_ms = int((time.perf_counter() - started_at) * 1000)
872 |
873 |     return {
874 |         "title": f"{payload.term.name} EduPulse AI intelligence report",
875 |         "summary": narrative["executiveSummary"],

```

```

876 |         "confidence": calculate_confidence(
877 |             total_inputs=total_feedback_count,
878 |             useful_comments=quality["summary"]["usefulSignalComments"],
879 |             duplicate_count=quality["duplicateCount"],
880 |             theme_count=len(visible_themes),
881 |         ),
882 |         "inputCount": total_feedback_count,
883 |         "lowSampleFlag": total_feedback_count < 20,
884 |         "duplicateCount": quality["duplicateCount"],
885 |         "rawJson": {
886 |             "engine": "edupulse-ai-service",
887 |             "modelMode": model_mode(),
888 |             "modelStack": model_stack_metadata(),
889 |             "qualityChecks": quality["summary"],
890 |             "engineBenchmark": {
891 |                 "processingMs": processing_ms,
892 |                 "responsesPerSecond": round(total_feedback_count / max(processing_ms / 1000, 0.001), 2),
893 |                 "documentsClustered": len(documents),
894 |                 "themesGenerated": len(visible_themes),
895 |                 "ultraConcerningThemes": len(gemini_reasoning.get("ultraConcerningIssues", [])),
896 |                 "actionsGenerated": len(actions),
897 |                 "criticalThemes": critical_count,
898 |                 "negativeThemes": negative_count,
899 |             },
900 |             "sentimentBreakdown": sentiment_breakdown(useful_feedback, quality["weights"]),
901 |             "categorySignals": category_themes,
902 |             "clusterDiagnostics": {
903 |                 "cluster_result": cluster_result["metadata"]
904 |                 if cluster_result
905 |                 else {
906 |                     "clusterer": "taxonomy_first_then_unknown_clustering",
907 |                     "embeddingModel": embedding_model_name(),
908 |                     "clusterCount": len(visible_themes),
909 |                     "noiseCount": 0,
910 |                 }
911 |             },
912 |             "reportNarrative": narrative,
913 |             "geminiReasoning": gemini_reasoning,
914 |             "humanCorrectionLoop": {
915 |                 "enabled": True,
916 |                 "endpoint": "/human-feedback",
917 |                 "storedCorrections": human_feedback_summary()["totalCorrections"],
918 |             },
919 |             "pipeline": [
920 |                 "rule_and_anomaly_quality_filter",
921 |                 "roberta_or_lexicon_sentiment",
922 |                 "institutional_issue_taxonomy",
923 |                 "entity_extraction",
924 |                 "safety_guardrail_engine",
925 |                 "semantic_clustering_for_unknowns",
926 |                 "priority_risk_scoring",
927 |                 "gemini_reasoning_on_issue_clusters",
928 |                 "structured_report_generation",
929 |                 "human_correction_loop",
930 |             ],
931 |         },
932 |         "themes": visible_themes,
933 |         "actions": actions,
934 |     }
935 |
936 |
937 | def feedback_weight(item: FeedbackItem) -> int:
938 |     try:
939 |         return max(1, int(item.weight or 1))
940 |     except (TypeError, ValueError):
941 |         return 1
942 |
943 |
944 | def weighted_feedback_count(feedback: list[FeedbackItem]) -> int:
945 |     return sum(feedback_weight(item) for item in feedback)
946 |
947 |
948 | def quality_filter(feedback: list[FeedbackItem]) -> dict[str, Any]:
949 |     seen: dict[str, FeedbackItem] = {}
950 |     weights: dict[str, int] = {}
951 |     useful: list[FeedbackItem] = []
952 |     duplicate_count = 0
953 |     duplicate_groups: dict[str, dict[str, Any]] = {}
954 |     rejected = Counter()
955 |     rejected_ids: set[str] = set()
956 |     rejected_examples: list[dict[str, Any]] = []
957 |
958 |     for item in feedback:
959 |         item_weight = feedback_weight(item)
960 |         raw_comment = (item.comment or "").strip()
961 |         comment = normalize_text(raw_comment)
962 |         if not comment:
963 |             continue
964 |
965 |         reason = reject_reason(comment)
966 |         fingerprint = f"{item.categoryId}:{comment}"
967 |
968 |         if reason:
969 |             rejected[reason] += item_weight
970 |             rejected_ids.add(item.id)
971 |             if len(rejected_examples) < 8:
972 |                 rejected_examples.append(
973 |                     {
974 |                         "comment": raw_comment[:180],
975 |                         "topic": item.categoryName,
976 |                         "question": item.questionText,
977 |                         "source": source_label(item),
978 |                         "reason": reason,
979 |                         "decision": "Ignored from theme analysis",
980 |                     }
981 |                 )
982 |             continue
983 |
984 |         if fingerprint in seen:
985 |             duplicate_count += item_weight

```

```

986 |         first_item = seen[fingerprint]
987 |         weights[first_item.id] = weights.get(first_item.id, 1) + item_weight
988 |         group = duplicate_groups.setdefault(
989 |             fingerprint,
990 |             {
991 |                 "comment": first_item.comment or "",
992 |                 "topic": first_item.categoryName,
993 |                 "repeated": 1,
994 |                 "decision": "Grouped as repeated signal",
995 |             },
996 |         )
997 |         group["repeated"] = weights[first_item.id]
998 |         continue
999 |
1000 |     seen[fingerprint] = item
1001 |     weights[item.id] = item_weight
1002 |     if item_weight > 1:
1003 |         duplicate_count += item_weight - 1
1004 |         duplicate_groups[fingerprint] = {
1005 |             "comment": item.comment or "",
1006 |             "topic": item.categoryName,
1007 |             "repeated": item_weight,
1008 |             "decision": "Grouped as repeated signal",
1009 |         }
1010 |     useful.append(item)
1011 |
1012 |     duplicate_examples = sorted(
1013 |         duplicate_groups.values(),
1014 |         key=lambda item: int(item["repeated"]),
1015 |         reverse=True,
1016 |     )[:8]
1017 |     useful_signal_comments = len(useful) + duplicate_count
1018 |     low_quality_rejected_count = sum(rejected.values())
1019 |
1020 |     return {
1021 |         "useful": useful,
1022 |         "weights": weights,
1023 |         "rejectedIds": rejected_ids,
1024 |         "duplicateCount": duplicate_count,
1025 |         "summary": {
1026 |             "totalComments": sum(feedback_weight(item) for item in feedback if item.comment),
1027 |             "usefulComments": useful_signal_comments,
1028 |             "usefulSignalComments": useful_signal_comments,
1029 |             "uniqueUsefulComments": len(useful),
1030 |             "duplicateCount": duplicate_count,
1031 |             "lowQualityRejectedCount": low_quality_rejected_count,
1032 |             "rejected": dict(rejected),
1033 |             "rejectedExamples": rejected_examples,
1034 |             "duplicateExamples": duplicate_examples,
1035 |         },
1036 |     }
1037 |
1038 |
1039 | def reject_reason(text: str) -> str | None:
1040 |     words = text.split()
1041 |     if len(text) < 8 or len(words) < 2:
1042 |         return "too_short"
1043 |     if text in GENERIC_LOW_INFORMATION:
1044 |         return "too_generic"
1045 |     if any(hint in text for hint in OUT_OF_CONTEXT_HINTS):
1046 |         return "out_of_context"
1047 |     if re.search(r"(\.|\{|\}|\,)", text):
1048 |         return "repeated_characters"
1049 |     if len(set(words)) <= 2 and len(words) >= 5:
1050 |         return "repeated_words"
1051 |     if not re.search(r"[a-zA-Z]{3,}", text):
1052 |         return "random_text"
1053 |     if len(words) >= 6 and (len(set(words)) / len(words)) < 0.35:
1054 |         return "low_lexical_diversity"
1055 |     if (
1056 |         len(words) <= 4
1057 |         and not any(word in text for word in ACTIONABLE_CONTEXT_WORDS)
1058 |         and not has_ultra_concern_language(text)
1059 |         and not any(word in text for word in NEGATIVE_WORDS)
1060 |     ):
1061 |         return "out_of_context"
1062 |     if (
1063 |         any(word in text for word in {"trash", "useless", "terrible", "worst"})
1064 |         and not any(word in text for word in ACTIONABLE_CONTEXT_WORDS)
1065 |     ):
1066 |         return "extreme_low_information"
1067 |     return None
1068 |
1069 |
1070 | def source_label(item: FeedbackItem) -> str:
1071 |     parts = []
1072 |     if item.departmentCode:
1073 |         parts.append(f"{item.departmentCode} student")
1074 |     else:
1075 |         parts.append("Student")
1076 |     if item.semesterNumber:
1077 |         parts.append(f"Semester {item.semesterNumber}")
1078 |     return ", ".join(parts)
1079 |
1080 |
1081 | def build_feedback_documents(
1082 |     feedback: list[FeedbackItem], weights: dict[str, int] | None = None
1083 | ) -> list[dict[str, Any]]:
1084 |     docs = []
1085 |     for item in feedback:
1086 |         docs.append(
1087 |             {
1088 |                 "id": item.id,
1089 |                 "source": "feedback",
1090 |                 "text": item.comment or "",
1091 |                 "categoryId": item.categoryId,
1092 |                 "categoryName": item.categoryName,
1093 |                 "rating": item.rating,
1094 |                 "departmentCode": item.departmentCode,
1095 |                 "semesterNumber": item.semesterNumber,

```

```

1096 |         "weight": (weights or {}).get(item.id, 1),
1097 |     }
1098 | )
1099 | return docs
1100 |
1101 |
1102 | def build_grievance_documents(grievances: list[GrievanceItem]) -> list[dict[str, Any]]:
1103 |     docs = []
1104 |     for item in grievances:
1105 |         docs.append(
1106 |             {
1107 |                 "id": item.id,
1108 |                 "source": "grievance",
1109 |                 "text": f"{item.title}. {item.description}",
1110 |                 "categoryId": None,
1111 |                 "categoryName": item.category,
1112 |                 "rating": 1 if item.severity in {"HIGH", "CRITICAL"} else 2,
1113 |                 "severity": item.severity,
1114 |                 "safetyFlag": item.safetyFlag,
1115 |                 "departmentCode": item.departmentCode,
1116 |                 "weight": 1,
1117 |             }
1118 |         )
1119 |     return docs
1120 |
1121 |
1122 | def classify_sentiments(documents: list[dict[str, Any]]) -> dict[str, Sentiment]:
1123 |     if not documents:
1124 |         return {}
1125 |
1126 |     hf = get_sentiment_pipeline()
1127 |     if hf:
1128 |         return classify_with_roberta(documents, hf)
1129 |
1130 |     return {doc["id"]: lexicon_sentiment(doc["text"], doc.get("rating", 3)) for doc in documents}
1131 |
1132 |
1133 | def classify_with_roberta(documents: list[dict[str, Any]], hf: Any) -> dict[str, Sentiment]:
1134 |     results: dict[str, Sentiment] = {}
1135 |     batch_size = 64
1136 |     for start in range(0, len(documents), batch_size):
1137 |         batch = documents[start : start + batch_size]
1138 |         outputs = hf([doc["text"][:512] for doc in batch], truncation=True)
1139 |         for doc, output in zip(batch, outputs):
1140 |             label = str(output["label"]).lower()
1141 |             text = normalize_text(doc["text"])
1142 |             if has_critical_language(text) or doc.get("safetyFlag"):
1143 |                 results[doc["id"]] = "CRITICAL"
1144 |             elif "negative" in label or label.endswith("_0"):
1145 |                 results[doc["id"]] = "NEGATIVE"
1146 |             elif "positive" in label or label.endswith("_2"):
1147 |                 results[doc["id"]] = "POSITIVE"
1148 |             else:
1149 |                 results[doc["id"]] = "NEUTRAL"
1150 |     return results
1151 |
1152 |
1153 | def should_create_issue_theme(doc: dict[str, Any], sentiment: Sentiment) -> bool:
1154 |     text = normalize_text(str(doc.get("text", "")))
1155 |     if not text:
1156 |         return False
1157 |     if has_critical_language(text) or sentiment == "CRITICAL":
1158 |         return True
1159 |     if is_positive_only_comment(text):
1160 |         return False
1161 |     if sentiment == "NEGATIVE":
1162 |         return True
1163 |     if int(doc.get("rating", 3)) <= 2:
1164 |         return has_issue_language(text) or bool(
1165 |             match_issue_taxonomy(text, str(doc.get("categoryName", "")))
1166 |         )
1167 |     return False
1168 |
1169 |
1170 | def has_issue_language(text: str) -> bool:
1171 |     clean = normalize_text(text)
1172 |     return (
1173 |         any(word in clean for word in ISSUE_LANGUAGE_WORDS)
1174 |         or any(word in clean for word in NEGATIVE_WORDS)
1175 |         or has_ultra_concern_language(clean)
1176 |     )
1177 |
1178 |
1179 | def is_positive_only_comment(text: str) -> bool:
1180 |     clean = normalize_text(text)
1181 |     if has_issue_language(clean) or has_ultra_concern_language(clean):
1182 |         return False
1183 |     positive_hits = sum(1 for word in POSITIVE_WORDS if word in clean)
1184 |     negative_hits = sum(1 for word in NEGATIVE_WORDS if word in clean)
1185 |     return positive_hits > 0 and negative_hits == 0
1186 |
1187 |
1188 | def match_issue_taxonomy(text: str, category_name: str = "") -> dict[str, Any] | None:
1189 |     clean_text = normalize_text(text)
1190 |     clean_category = normalize_text(category_name)
1191 |     for rule in ISSUE_TAXONOMY:
1192 |         category_hints = [normalize_text(str(item)) for item in rule.get("categoryHints", [])]
1193 |         if category_hints and clean_category and not any(
1194 |             hint in clean_category or clean_category in hint for hint in category_hints
1195 |         ):
1196 |             text_only_match = False
1197 |         else:
1198 |             text_only_match = True
1199 |         if not text_only_match and str(rule.get("type")) != "urgent":
1200 |             continue
1201 |         if contains_any_phrase(clean_text, rule.get("phrases", [])):
1202 |             return rule
1203 |     return None
1204 |
1205 |

```

```

1206 | def contains_any_phrase(text: str, phrases: list[str] | tuple[str, ...] | set[str]) -> bool:
1207 |     return any(normalize_text(str(phrase)) in text for phrase in phrases if str(phrase).strip())
1208 |
1209 |
1210 | def taxonomy_bucket_key(doc: dict[str, Any]) -> str | None:
1211 |     match = match_issue_taxonomy(doc.get("text", ""), doc.get("categoryName", ""))
1212 |     if not match:
1213 |         return None
1214 |     return str(match["bucket"])
1215 |
1216 |
1217 | def taxonomy_title_from_docs(docs: list[dict[str, Any]]) -> str | None:
1218 |     matches = [
1219 |         match_issue_taxonomy(doc.get("text", ""), doc.get("categoryName", ""))
1220 |         for doc in docs
1221 |     ]
1222 |     valid = [match for match in matches if match]
1223 |     if not valid:
1224 |         aggregate_match = match_issue_taxonomy(
1225 |             " ".join(doc.get("text", "") for doc in docs),
1226 |             Counter(doc.get("categoryName", "") for doc in docs).most_common(1)[0][0],
1227 |         )
1228 |         return str(aggregate_match["title"]) if aggregate_match else None
1229 |     bucket = Counter(str(match["bucket"]) for match in valid).most_common(1)[0][0]
1230 |     for match in valid:
1231 |         if str(match["bucket"]) == bucket:
1232 |             return str(match["title"])
1233 |     return None
1234 |
1235 |
1236 | def taxonomy_metadata_from_docs(docs: list[dict[str, Any]]) -> dict[str, Any] | None:
1237 |     matches = [
1238 |         match_issue_taxonomy(doc.get("text", ""), doc.get("categoryName", ""))
1239 |         for doc in docs
1240 |     ]
1241 |     valid = [match for match in matches if match]
1242 |     if not valid:
1243 |         return None
1244 |     bucket = Counter(str(match["bucket"]) for match in valid).most_common(1)[0][0]
1245 |     selected = next(match for match in valid if str(match["bucket"]) == bucket)
1246 |     matched_count = sum(1 for match in valid if str(match["bucket"]) == bucket)
1247 |     return {
1248 |         "bucket": selected["bucket"],
1249 |         "title": selected["title"],
1250 |         "type": selected.get("type", "normal"),
1251 |         "matchedDocuments": matched_count,
1252 |         "coverage": round(matched_count / max(len(docs), 1), 2),
1253 |     }
1254 |
1255 |
1256 | def extract_entities_from_docs(docs: list[dict[str, Any]]) -> dict[str, Any]:
1257 |     raw_text = " ".join(str(doc.get("text", "")) for doc in docs)
1258 |     clean = normalize_text(raw_text)
1259 |     locations = set()
1260 |     location_patterns = [
1261 |         r"\bab\s*d+\s*room\s*d+\b",
1262 |         r"\bab\s*d+\s*building\b",
1263 |         r"\bab\s*d+\b",
1264 |         r"\bblock\s*[a-z]\b",
1265 |         r"\broom\s*d+\b",
1266 |         r"\b(?:first|second|third|fourth|1st|2nd|3rd|4th)\s+year\s+hostel\s+mess\b",
1267 |         r"\bhostel\s+corridor\b",
1268 |         r"\bhostel\s+mess\b",
1269 |         r"\bhostel\b",
1270 |         r"\bcs\s*ds\s+campus\b",
1271 |         r"\bcsds\s+campus\b",
1272 |         r"\bplayground\b",
1273 |         r"\bcanteen\b",
1274 |         r"\blibrary\b",
1275 |         r"\blab\b",
1276 |         r"\bwashroom\b",
1277 |         r"\bparking\s+area\b",
1278 |         r"\bcampus\s+gate\b",
1279 |     ]
1280 |     for pattern in location_patterns:
1281 |         for match in re.finditer(pattern, clean):
1282 |             locations.add(pretty_entity(match.group(0)))
1283 |
1284 |     people = {
1285 |         match.group(0).strip()
1286 |         for match in re.finditer(r"\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+){1,3}\b", raw_text)
1287 |         if match.group(0).strip().lower() not in {"Computer Science", "Information Technology"}
1288 |     }
1289 |     semesters = {
1290 |         pretty_entity(match.group(0))
1291 |         for match in re.finditer(
1292 |             r"\b(?:semester\s*d+|[1-8](?:st|nd|rd|th)\s+semester|(?:first|second|third|fourth)\s+year|[1-4](?:st|nd|rd|th)\s+year)\b",
1293 |             clean,
1294 |         )
1295 |     }
1296 |     departments = extract_departments_from_text(raw_text)
1297 |
1298 |     return {
1299 |         "departments": departments,
1300 |         "locations": sorted(locations)[:8],
1301 |         "people": sorted(people)[:8],
1302 |         "semesters": sorted(semesters)[:8],
1303 |     }
1304 |
1305 |
1306 | def pretty_entity(value: str) -> str:
1307 |     replacements = {
1308 |         "ab": "AB",
1309 |         "cs": "CS",
1310 |         "ds": "DS",
1311 |         "cs ds": "CS DS",
1312 |         "csds": "CSDS",
1313 |     }
1314 |     words = []
1315 |     for word in value.split():

```



```

1316 |         lowered = word.lower()
1317 |         if re.fullmatch(r"ab\d+", lowered):
1318 |             words.append(lowered.upper())
1319 |         elif lowered in replacements:
1320 |             words.append(replacements[lowered])
1321 |         elif re.fullmatch(r"\d+(?:st|nd|rd|th)?", lowered):
1322 |             words.append(lowered)
1323 |         else:
1324 |             words.append(word.capitalize())
1325 |     return " ".join(words)
1326 |
1327 |
1328 | def issue_confidence_from_docs(
1329 |     docs: list[dict[str, Any]],
1330 |     taxonomy: dict[str, Any] | None,
1331 |     entities: dict[str, Any],
1332 |     sentiment: Sentiment,
1333 | ) -> int:
1334 |     score = 0.52
1335 |     if taxonomy:
1336 |         score += 0.25 + min(float(taxonomy.get("coverage", 0)), 1.0) * 0.08
1337 |     else:
1338 |         score -= 0.12
1339 |     weighted_count = sum(int(doc.get("weight", 1)) for doc in docs)
1340 |     if weighted_count >= 2:
1341 |         score += 0.06
1342 |     if weighted_count >= 5:
1343 |         score += 0.05
1344 |     if weighted_count >= 25:
1345 |         score += 0.04
1346 |     if any(entities.get(key) for key in ("locations", "people", "departments", "semesters")):
1347 |         score += 0.05
1348 |     if sentiment == "CRITICAL":
1349 |         score += 0.04
1350 |     elif sentiment == "NEGATIVE":
1351 |         score += 0.02
1352 |     return int(round(max(0.35, min(0.97, score)) * 100))
1353 |
1354 |
1355 | def build_themes(
1356 |     documents: list[dict[str, Any]],
1357 |     sentiment_by_doc: dict[str, Sentiment],
1358 |     cluster_result: dict[str, Any] | None = None,
1359 | ) -> list[dict[str, Any]]:
1360 |     if len(documents) < 2:
1361 |         if not documents:
1362 |             return []
1363 |         sentiment = sentiment_by_doc.get(documents[0]["id"], "NEUTRAL")
1364 |         if should_create_issue_theme(documents[0], sentiment):
1365 |             return [build_theme_from_docs([documents[0]], sentiment, cluster_result)]
1366 |         return []
1367 |
1368 |     taxonomy_buckets: dict[str, list[dict[str, Any]]] = defaultdict(list)
1369 |     unknown_docs: list[dict[str, Any]] = []
1370 |     for doc in documents:
1371 |         sentiment = sentiment_by_doc.get(doc["id"], "NEUTRAL")
1372 |         if not should_create_issue_theme(doc, sentiment):
1373 |             continue
1374 |         key = taxonomy_bucket_key(doc)
1375 |         if key:
1376 |             taxonomy_buckets[key].append(doc)
1377 |         else:
1378 |             unknown_docs.append(doc)
1379 |
1380 |     themes = []
1381 |     for bucket_docs in taxonomy_buckets.values():
1382 |         sentiments = [sentiment_by_doc.get(doc["id"], "NEUTRAL") for doc in bucket_docs]
1383 |         themes.append(
1384 |             build_theme_from_docs(
1385 |                 bucket_docs,
1386 |                 strongest_sentiment(sentiments),
1387 |                 cluster_result,
1388 |             )
1389 |         )
1390 |
1391 |     if not unknown_docs:
1392 |         return themes
1393 |
1394 |     if len(unknown_docs) <= 250:
1395 |         buckets: dict[str, list[dict[str, Any]]] = defaultdict(list)
1396 |         for doc in unknown_docs:
1397 |             buckets[title_from_docs([doc])].append(doc)
1398 |         for bucket_docs in buckets.values():
1399 |             sentiments = [
1400 |                 sentiment_by_doc.get(doc["id"], "NEUTRAL") for doc in bucket_docs
1401 |             ]
1402 |             themes.append(
1403 |                 build_theme_from_docs(
1404 |                     bucket_docs,
1405 |                     strongest_sentiment(sentiments),
1406 |                     cluster_result,
1407 |                 )
1408 |             )
1409 |         return themes
1410 |
1411 |     labels = (
1412 |         cluster_result["labels"]
1413 |         if cluster_result
1414 |         else cluster_documents([doc["text"] for doc in unknown_docs])
1415 |     )
1416 |     clusters: dict[int, list[dict[str, Any]]] = defaultdict(list)
1417 |     noise_docs: list[dict[str, Any]] = []
1418 |     for label, doc in zip(labels, unknown_docs):
1419 |         if int(label) == -1:
1420 |             noise_docs.append(doc)
1421 |             continue
1422 |             clusters[int(label)].append(doc)
1423 |
1424 |     themes = []
1425 |     for doc in noise_docs:

```

```

1426 |         sentiment = sentiment_by_doc.get(doc["id"], "NEUTRAL")
1427 |         if sentiment in {"NEGATIVE", "CRITICAL"} or doc.get("rating", 3) <= 2:
1428 |             themes.append(build_theme_from_docs([doc], sentiment, cluster_result))
1429 |
1430 |     for cluster_docs in clusters.values():
1431 |         sentiments = [sentiment_by_doc.get(doc["id"], "NEUTRAL") for doc in cluster_docs]
1432 |         sentiment = strongest_sentiment(sentiments)
1433 |         if len(cluster_docs) < 2:
1434 |             doc = cluster_docs[0]
1435 |             if not should_create_issue_theme(doc, sentiment):
1436 |                 continue
1437 |             themes.append(build_theme_from_docs(cluster_docs, sentiment, cluster_result))
1438 |             continue
1439 |         if sentiment in {"POSITIVE", "NEUTRAL"}:
1440 |             continue
1441 |         themes.append(build_theme_from_docs(cluster_docs, sentiment, cluster_result))
1442 |
1443 |     return themes
1444 |
1445 |
1446 | def build_theme_from_docs(
1447 |     cluster_docs: list[dict[str, Any]],
1448 |     sentiment: Sentiment,
1449 |     cluster_result: dict[str, Any] | None = None,
1450 | ) -> dict[str, Any]:
1451 |     weighted_mentions = sum(int(doc.get("weight", 1)) for doc in cluster_docs)
1452 |     category_counter = Counter(doc["categoryName"] for doc in cluster_docs)
1453 |     department_counter = Counter(doc.get("departmentCode") or "UNKNOWN" for doc in cluster_docs)
1454 |     taxonomy = taxonomy_metadata_from_docs(cluster_docs)
1455 |     entities = extract_entities_from_docs(cluster_docs)
1456 |     title = str(taxonomy["title"]) if taxonomy else title_from_docs(cluster_docs)
1457 |     if taxonomy and taxonomy.get("type") == "urgent":
1458 |         sentiment = "CRITICAL"
1459 |     priority = "CRITICAL" if taxonomy and taxonomy.get("type") == "urgent" else priority_from_docs(cluster_docs, sentiment)
1460 |     confidence = issue_confidence_from_docs(cluster_docs, taxonomy, entities, sentiment)
1461 |     return {
1462 |         "categoryId": most_common_category_id(cluster_docs),
1463 |         "title": title,
1464 |         "summary": (
1465 |             f"{weighted_mentions} feedback comment(s) mention {title.lower()}. "
1466 |             f"Most common category: {category_counter.most_common(1)[0][0]}."
1467 |         ),
1468 |         "sentiment": sentiment,
1469 |         "priority": priority,
1470 |         "mentionCount": weighted_mentions,
1471 |         "evidence": {
1472 |             "sampleComments": [doc["text"] for doc in cluster_docs[:3]],
1473 |             "departments": dict(department_counter),
1474 |             "priorityScore": priority_score(cluster_docs, sentiment),
1475 |             "uniqueCommentCount": len(cluster_docs),
1476 |             "taxonomy": taxonomy,
1477 |             "extractedEntities": entities,
1478 |             "issueConfidence": confidence,
1479 |             "reviewStatus": "HIGH_CONFIDENCE" if confidence >= 80 else "NEEDS_REVIEW",
1480 |             "clusterer": (
1481 |                 cluster_result["metadata"]["clusterer"]
1482 |                 if cluster_result
1483 |                 else "unknown"
1484 |             ),
1485 |         },
1486 |         "actionTitle": f"Investigate and resolve {title.lower()}",
1487 |         "kpi": f"Reduce {title.lower()} mentions in the next feedback cycle",
1488 |     }
1489 |
1490 |
1491 | def build_category_themes(
1492 |     all_feedback: list[FeedbackItem], useful_feedback: list[FeedbackItem]
1493 | ) -> list[dict[str, Any]]:
1494 |     buckets: dict[str, list[FeedbackItem]] = defaultdict(list)
1495 |     useful_by_category: dict[str, list[FeedbackItem]] = defaultdict(list)
1496 |
1497 |     for item in all_feedback:
1498 |         buckets[item.categoryId].append(item)
1499 |
1500 |     for item in useful_feedback:
1501 |         useful_by_category[item.categoryId].append(item)
1502 |
1503 |     themes = []
1504 |     for category_id, items in buckets.items():
1505 |         ratings = [item.rating for item in items]
1506 |         if not ratings:
1507 |             continue
1508 |         average = sum(ratings) / len(ratings)
1509 |         low_count = sum(1 for rating in ratings if rating <= 2)
1510 |         low_ratio = low_count / len(ratings)
1511 |         comments = [item.comment or "" for item in useful_by_category[category_id][:3]]
1512 |         category_name = items[0].categoryName
1513 |         sentiment = category_sentiment(average, low_ratio, comments)
1514 |         priority = category_priority(sentiment, low_ratio, len(items), comments)
1515 |         themes.append(
1516 |             {
1517 |                 "categoryId": category_id,
1518 |                 "title": f"{category_name} satisfaction signal",
1519 |                 "summary": (
1520 |                     f"{category_name} average rating is {average:.2f}/4 from "
1521 |                     f"{len(items)} response(s). {low_count} response(s) are low rated."
1522 |                 ),
1523 |                 "sentiment": sentiment,
1524 |                 "priority": priority,
1525 |                 "mentionCount": len(items),
1526 |                 "evidence": {
1527 |                     "averageRating": round(average, 2),
1528 |                     "lowRatedResponses": low_count,
1529 |                     "sampleComments": comments,
1530 |                     "departmentBreakdown": department_satisfaction(items),
1531 |                 },
1532 |                 "actionTitle": f"Improve {category_name} satisfaction",
1533 |                 "kpi": f"Raise {category_name} average rating above 3.2/4",
1534 |             }
1535 |         )

```

```

1536 |     return themes
1537 |
1538 |
1539 | def build_grievance_theme(grievances: list[GrievanceItem]) -> dict[str, Any] | None:
1540 |     if not grievances:
1541 |         return None
1542 |     safety_count = sum(1 for item in grievances if item.safetyFlag or item.severity == "CRITICAL")
1543 |     priority: Priority = "CRITICAL" if safety_count else "HIGH"
1544 |     return {
1545 |         "categoryId": None,
1546 |         "title": "Live grievance escalation",
1547 |         "summary": (
1548 |             f"{len(grievances)} grievance(s) are open for admin review. "
1549 |             f"{safety_count} case(s) contain safety or critical signals."
1550 |         ),
1551 |         "sentiment": "CRITICAL" if safety_count else "NEGATIVE",
1552 |         "priority": priority,
1553 |         "mentionCount": len(grievances),
1554 |         "evidence": {
1555 |             "safetyFlagged": safety_count,
1556 |             "categories": sorted({item.category for item in grievances}),
1557 |             "examples": [
1558 |                 {"title": item.title, "severity": item.severity, "status": item.status}
1559 |                 for item in grievances[:3]
1560 |             ],
1561 |         },
1562 |         "actionTitle": "Escalate and resolve urgent grievances",
1563 |         "kpi": "Close critical grievances within 48 hours",
1564 |     }
1565 |
1566 |
1567 | def cluster_documents(texts: list[str]) -> list[int]:
1568 |     return cluster_documents_with_metadata(texts)["labels"]
1569 |
1570 |
1571 | def cluster_documents_with_metadata(texts: list[str]) -> dict[str, Any]:
1572 |     if len(texts) < 2:
1573 |         return {
1574 |             "labels": [0 for _ in texts],
1575 |             "metadata": {
1576 |                 "clusterer": "single_cluster",
1577 |                 "embeddingModel": None,
1578 |                 "clusterCount": 1 if texts else 0,
1579 |                 "noiseCount": 0,
1580 |             },
1581 |         }
1582 |
1583 |     embedding_model = get_embedding_model()
1584 |     if embedding_model and np is not None:
1585 |         vectors = encode_documents(embedding_model, texts)
1586 |         if HDBSCAN is not None and len(texts) >= 6:
1587 |             labels = HDBSCAN(
1588 |                 min_cluster_size=max(2, min(8, int(math.sqrt(len(texts))))),
1589 |                 min_samples=1,
1590 |                 metric="euclidean",
1591 |             ).fit_predict(vectors).tolist()
1592 |             if useful_cluster_count(labels) > 0:
1593 |                 return {
1594 |                     "labels": labels,
1595 |                     "metadata": {
1596 |                         "clusterer": "hdbscan_density",
1597 |                         "embeddingModel": embedding_model_name(),
1598 |                         "clusterCount": useful_cluster_count(labels),
1599 |                         "noiseCount": sum(1 for label in labels if label == -1),
1600 |                     },
1601 |                 }
1602 |
1603 |         if MiniBatchKMeans is not None:
1604 |             cluster_count = choose_cluster_count(len(texts))
1605 |             labels = MiniBatchKMeans(
1606 |                 n_clusters=cluster_count,
1607 |                 random_state=42,
1608 |                 batch_size=256,
1609 |                 n_init="auto",
1610 |             ).fit_predict(vectors).tolist()
1611 |             return {
1612 |                 "labels": labels,
1613 |                 "metadata": {
1614 |                     "clusterer": "embedding_minibatch_kmeans",
1615 |                     "embeddingModel": embedding_model_name(),
1616 |                     "clusterCount": useful_cluster_count(labels),
1617 |                     "noiseCount": 0,
1618 |                 },
1619 |             }
1620 |
1621 |         if TfidfVectorizer is not None and MiniBatchKMeans is not None:
1622 |             vectors = TfidfVectorizer(max_features=1200, stop_words="english").fit_transform(texts)
1623 |             cluster_count = choose_cluster_count(len(texts))
1624 |             labels = MiniBatchKMeans(
1625 |                 n_clusters=cluster_count, random_state=42, batch_size=256, n_init="auto"
1626 |             ).fit_predict(vectors).tolist()
1627 |             return {
1628 |                 "labels": labels,
1629 |                 "metadata": {
1630 |                     "clusterer": "tfidf_minibatch_kmeans",
1631 |                     "embeddingModel": None,
1632 |                     "clusterCount": useful_cluster_count(labels),
1633 |                     "noiseCount": 0,
1634 |                 },
1635 |             }
1636 |
1637 |     labels = keyword_cluster(texts)
1638 |     return {
1639 |         "labels": labels,
1640 |         "metadata": {
1641 |             "clusterer": "keyword_fallback",
1642 |             "embeddingModel": None,
1643 |             "clusterCount": useful_cluster_count(labels),
1644 |             "noiseCount": 0,
1645 |         },

```

```

1646 |     }
1647 |
1648 |
1649 | def encode_documents(embedding_model: Any, texts: list[str]) -> Any:
1650 |     model_name = embedding_model_name().lower()
1651 |     prepared = [f"passage: {text}" if "e5" in model_name else text for text in texts]
1652 |     return embedding_model.encode(
1653 |         prepared,
1654 |         batch_size=int(os.getenv("EDUPULSE_EMBEDDING_BATCH_SIZE", "64")),
1655 |         normalize_embeddings=True,
1656 |     )
1657 |
1658 |
1659 | def useful_cluster_count(labels: list[int]) -> int:
1660 |     return len([label for label in labels if label != -1])
1661 |
1662 |
1663 | def choose_cluster_count(count: int) -> int:
1664 |     return max(2, min(12, int(math.sqrt(count))))
1665 |
1666 |
1667 | def keyword_cluster(texts: list[str]) -> list[int]:
1668 |     labels = []
1669 |     keys: dict[str, int] = {}
1670 |     for text in texts:
1671 |         label_key = top_keyword(text)
1672 |         if label_key not in keys:
1673 |             keys[label_key] = len(keys)
1674 |         labels.append(keys[label_key])
1675 |     return labels
1676 |
1677 |
1678 | def lexicon_sentiment(text: str, rating: int) -> Sentiment:
1679 |     clean = normalize_text(text)
1680 |     if has_critical_language(clean):
1681 |         return "CRITICAL"
1682 |     negative_hits = sum(1 for word in NEGATIVE_WORDS if word in clean)
1683 |     positive_hits = sum(1 for word in POSITIVE_WORDS if word in clean)
1684 |     if rating <= 2 or negative_hits > positive_hits:
1685 |         return "NEGATIVE"
1686 |     if rating >= 3 and positive_hits >= negative_hits:
1687 |         return "POSITIVE"
1688 |     return "NEUTRAL"
1689 |
1690 |
1691 | def category_sentiment(average: float, low_ratio: float, comments: list[str]) -> Sentiment:
1692 |     text = normalize_text(" ".join(comments))
1693 |     if has_critical_language(text):
1694 |         return "CRITICAL"
1695 |     if average < 2.6 or low_ratio >= 0.4 or any(word in text for word in NEGATIVE_WORDS):
1696 |         return "NEGATIVE"
1697 |     if average >= 3.3 and low_ratio < 0.25:
1698 |         return "POSITIVE"
1699 |     return "NEUTRAL"
1700 |
1701 |
1702 | def category_priority(
1703 |     sentiment: Sentiment, low_ratio: float, mention_count: int, comments: list[str]
1704 | ) -> Priority:
1705 |     text = normalize_text(" ".join(comments))
1706 |     if sentiment == "CRITICAL" or has_critical_language(text):
1707 |         return "CRITICAL"
1708 |     if sentiment == "NEGATIVE" and (low_ratio >= 0.4 or mention_count >= 20):
1709 |         return "HIGH"
1710 |     if sentiment == "NEGATIVE":
1711 |         return "MEDIUM"
1712 |     return "LOW"
1713 |
1714 |
1715 | def priority_from_docs(docs: list[dict[str, Any]], sentiment: Sentiment) -> Priority:
1716 |     score = priority_score(docs, sentiment)
1717 |     weighted_count = sum(int(doc.get("weight", 1)) for doc in docs) or 1
1718 |     safety_weight = sum(
1719 |         int(doc.get("weight", 1))
1720 |         for doc in docs
1721 |         if doc.get("safetyFlag") or has_critical_language(doc.get("text", ""))
1722 |     )
1723 |     safety_ratio = safety_weight / weighted_count
1724 |     if safety_weight >= 2 and safety_ratio >= 0.5 and score >= 5:
1725 |         return "CRITICAL"
1726 |     if score >= 5:
1727 |         return "HIGH"
1728 |     if score >= 3:
1729 |         return "MEDIUM"
1730 |     return "LOW"
1731 |
1732 |
1733 | def priority_score(docs: list[dict[str, Any]], sentiment: Sentiment) -> float:
1734 |     text = normalize_text(" ".join(doc["text"] for doc in docs))
1735 |     weighted_count = sum(int(doc.get("weight", 1)) for doc in docs)
1736 |     low_rating_count = sum(
1737 |         int(doc.get("weight", 1)) for doc in docs if doc.get("rating", 3) <= 2
1738 |     )
1739 |     safety_count = sum(
1740 |         int(doc.get("weight", 1))
1741 |         for doc in docs
1742 |         if doc.get("safetyFlag") or has_critical_language(doc["text"])
1743 |     )
1744 |     score = 0.0
1745 |     score += min(weighted_count, 200) / 40
1746 |     score += min(low_rating_count, 200) * 0.03
1747 |     score += min(safety_count, 20) * 2
1748 |     score += 3 if sentiment == "CRITICAL" else 2 if sentiment == "NEGATIVE" else 0
1749 |     score += 2 if any(word in text for word in CRITICAL_WORDS) else 0
1750 |     score += 1 if has_health_risk_language(text) else 0
1751 |     return round(score, 2)
1752 |
1753 |
1754 | def strongest_sentiment(sentiments: list[Sentiment]) -> Sentiment:
1755 |     counts = Counter(sentiments)

```

```

1756 |     if counts["CRITICAL"] >= 2 and counts["CRITICAL"] >= max(counts["NEGATIVE"], counts["POSITIVE"], counts["NEUTRAL"]):
1757 |         return "CRITICAL"
1758 |     if counts["NEGATIVE"] >= max(counts["POSITIVE"], counts["NEUTRAL"]):
1759 |         return "NEGATIVE"
1760 |     if counts["POSITIVE"] > counts["NEUTRAL"]:
1761 |         return "POSITIVE"
1762 |     return "NEUTRAL"
1763 |
1764 |
1765 | def dedupe_themes(themes: list[dict[str, Any]]) -> list[dict[str, Any]]:
1766 |     seen = set()
1767 |     result = []
1768 |     for theme in sorted(themes, key=lambda item: priority_rank(item["priority"]), reverse=True):
1769 |         key = normalize_text(theme["title"])
1770 |         if key in seen:
1771 |             continue
1772 |         seen.add(key)
1773 |         result.append(theme)
1774 |     return result[:40]
1775 |
1776 |
1777 | def build_actions(themes: list[dict[str, Any]]) -> list[dict[str, Any]]:
1778 |     actions = []
1779 |     for theme in themes:
1780 |         if theme["priority"] not in {"HIGH", "CRITICAL"}:
1781 |             continue
1782 |         actions.append(
1783 |             {
1784 |                 "title": theme.get("actionTitle") or f"Resolve {theme['title'].lower()}",
1785 |                 "priority": theme["priority"],
1786 |                 "timeline": "24-48 hours" if theme["priority"] == "CRITICAL" else "7-14 days",
1787 |                 "kpi": theme.get("kpi") or f"Reduce {theme['title'].lower()} by next cycle",
1788 |             }
1789 |         )
1790 |     return actions[:6]
1791 |
1792 |
1793 | def small_signal_reasoning_layer(
1794 |     term_name: str,
1795 |     themes: list[dict[str, Any]],
1796 |     quality: dict[str, Any],
1797 |     feedback_count: int,
1798 |     grievance_count: int,
1799 | ) -> dict[str, Any]:
1800 |     base = base_issue_priority_scoring_layer(
1801 |         term_name=term_name,
1802 |         themes=themes,
1803 |         quality=quality,
1804 |         feedback_count=feedback_count,
1805 |         grievance_count=grievance_count,
1806 |     )
1807 |     ultra = []
1808 |     for index, theme in enumerate(themes):
1809 |         text = normalize_text(
1810 |             f"{theme.get('title', '')} {theme.get('summary', '')}"
1811 |             f"{ ' '.join(extract_theme_evidence_samples(theme))}"
1812 |         )
1813 |         if is_theme_ultra_concern(theme):
1814 |             ultra.append(
1815 |                 {
1816 |                     "id": str(index),
1817 |                     "title": theme["title"],
1818 |                     "reason": theme["summary"],
1819 |                     "detailedSummary": build_plain_issue_summary(theme, extract_theme_evidence_samples(theme)),
1820 |                     "mentions": theme["mentionCount"],
1821 |                     "source": "small_signal_safety_reasoning",
1822 |                 }
1823 |             )
1824 |
1825 |     return {
1826 |         **base,
1827 |         "mode": "small_signal_reasoning_layer",
1828 |         "provider": "local_small_signal_reasoning",
1829 |         "ultraConcerningIssues": collapse_ultra_concerns(ultra)[:8],
1830 |     }
1831 |
1832 |
1833 | def merge_local_ultra_concerns(
1834 |     reasoning: dict[str, Any],
1835 |     themes: list[dict[str, Any]],
1836 | ) -> dict[str, Any]:
1837 |     existing = filter_valid_ultra_concerns(
1838 |         normalize_ultra_concerns(reasoning.get("ultraConcerningIssues", [])),
1839 |         themes,
1840 |     )
1841 |     existing_ids = {str(item.get("id")) for item in existing}
1842 |     merged = list(existing)
1843 |
1844 |     for index, theme in enumerate(themes):
1845 |         item_id = str(index)
1846 |         text = normalize_text(
1847 |             f"{theme.get('title', '')} {theme.get('summary', '')}"
1848 |             f"{ ' '.join(extract_theme_evidence_samples(theme))}"
1849 |         )
1850 |         if item_id in existing_ids or not is_theme_ultra_concern(theme):
1851 |             continue
1852 |         merged.append(
1853 |             {
1854 |                 "id": item_id,
1855 |                 "title": theme["title"],
1856 |                 "reason": theme["summary"],
1857 |                 "detailedSummary": build_plain_issue_summary(theme, extract_theme_evidence_samples(theme)),
1858 |                 "mentions": theme["mentionCount"],
1859 |                 "source": "deterministic_ultra_concern_guard",
1860 |             }
1861 |         )
1862 |         existing_ids.add(item_id)
1863 |
1864 |     return {
1865 |         **reasoning,

```

```

1866 |         "ultraConcerningIssues": collapse_ultra_concerns(merged)[:8],
1867 |     }
1868 |
1869 |
1870 | def exclude_ultra_concerning_themes(
1871 |     themes: list[dict[str, Any]],
1872 |     reasoning: dict[str, Any],
1873 | ) -> list[dict[str, Any]]:
1874 |     ultra_ids = {
1875 |         str(item.get("id"))
1876 |         for item in reasoning.get("ultraConcerningIssues", [])
1877 |         if isinstance(item, dict)
1878 |     }
1879 |     visible = []
1880 |     for index, theme in enumerate(themes):
1881 |         text = normalize_text(
1882 |             f"{theme.get('title', '')} {theme.get('summary', '')}"
1883 |             f"{ ' '.join(extract_theme_evidence_samples(theme))}"
1884 |         )
1885 |         if str(index) in ultra_ids or is_theme_ultra_concern(theme):
1886 |             continue
1887 |         visible.append(theme)
1888 |     return visible
1889 |
1890 |
1891 | def filter_valid_ultra_concerns(
1892 |     concerns: list[dict[str, Any]],
1893 |     themes: list[dict[str, Any]],
1894 | ) -> list[dict[str, Any]]:
1895 |     valid = []
1896 |     for concern in concerns:
1897 |         item_id = str(concern.get("id", ""))
1898 |         if item_id.isdigit():
1899 |             index = int(item_id)
1900 |             if 0 <= index < len(themes) and is_theme_ultra_concern(themes[index]):
1901 |                 valid.append(concern)
1902 |             continue
1903 |         text = normalize_text(
1904 |             f"{concern.get('title', '')} {concern.get('reason', '')} {concern.get('detailedSummary', '')}"
1905 |         )
1906 |         if ultra_concern_bucket({"title": concern.get("title", ""), "reason": text, "detailedSummary": ""})[0] != "other":
1907 |             valid.append(concern)
1908 |     return valid
1909 |
1910 |
1911 | def is_theme_ultra_concern(theme: dict[str, Any]) -> bool:
1912 |     taxonomy = extract_theme_taxonomy(theme)
1913 |     if taxonomy:
1914 |         return taxonomy.get("type") == "urgent"
1915 |     text = normalize_text(
1916 |         f"{theme.get('title', '')} {theme.get('summary', '')}"
1917 |         f"{ ' '.join(extract_theme_evidence_samples(theme))}"
1918 |     )
1919 |     return ultra_concern_bucket({"title": theme.get("title", ""), "reason": text, "detailedSummary": ""})[0] != "other"
1920 |
1921 |
1922 | def collapse_ultra_concerns(items: list[dict[str, Any]]) -> list[dict[str, Any]]:
1923 |     buckets: dict[str, dict[str, Any]] = {}
1924 |     order = {
1925 |         "harassment": 0,
1926 |         "ragging": 1,
1927 |         "corruption": 2,
1928 |         "weapon_violence": 3,
1929 |         "animal_danger": 4,
1930 |         "water_contamination": 5,
1931 |         "electric_fire": 6,
1932 |         "food_poisoning": 7,
1933 |         "building_injury": 8,
1934 |         "other": 99,
1935 |     }
1936 |
1937 |     for item in items:
1938 |         key, title = ultra_concern_bucket(item)
1939 |         existing = buckets.get(key)
1940 |         mentions = int(NumberLike(item.get("mentions", 1)))
1941 |         if existing:
1942 |             existing["mentions"] = int(existing.get("mentions", 1)) + mentions
1943 |             existing["relatedTitles"].append(str(item.get("title", title)))
1944 |             continue
1945 |         buckets[key] = {
1946 |             **item,
1947 |             "id": key,
1948 |             "title": title,
1949 |             "mentions": mentions,
1950 |             "relatedTitles": [str(item.get("title", title))],
1951 |         }
1952 |
1953 |     collapsed = list(buckets.values())
1954 |     for item in collapsed:
1955 |         related = item.pop("relatedTitles", [])
1956 |         item["detailedSummary"] = build_ultra_concern_detail(item)
1957 |         item["evidenceTitles"] = related[:6]
1958 |     return sorted(
1959 |         collapsed,
1960 |         key=lambda item: (order.get(str(item.get("id")), 99), -int(item.get("mentions", 1))),
1961 |     )
1962 |
1963 |
1964 | def build_ultra_concern_detail(item: dict[str, Any]) -> str:
1965 |     base = str(
1966 |         item.get("detailedSummary")
1967 |         or item.get("reason")
1968 |         or "A serious concern is present in the feedback."
1969 |     ).strip()
1970 |     detail = urgent_bucket_detail(str(item.get("id", "other")), int(item.get("mentions", 1)))
1971 |     if detail.lower() in base.lower():
1972 |         return base
1973 |     return f"{base} {detail}".strip()
1974 |
1975 |

```

```

1976 | def urgent_bucket_detail(bucket: str, mentions: int) -> str:
1977 |     mention_text = f"{mentions} mention(s)" if mentions > 1 else "One mention"
1978 |     details = {
1979 |         "harassment": (
1980 |             f"{mention_text} describe harassment or physical misconduct. Treat it as confidential, "
1981 |             "protect the reporting student's identity, and verify the named faculty, department or place if provided."
1982 |         ),
1983 |         "ragging": (
1984 |             f"{mention_text} describe ragging or bullying. Escalate to the anti-ragging authority and verify "
1985 |             "the hostel, classroom or campus area mentioned in the evidence."
1986 |         ),
1987 |         "corruption": (
1988 |             f"{mention_text} point to money misconduct. Verify receipts, event collections, approval records "
1989 |             "and the people responsible for handling the funds."
1990 |         ),
1991 |         "weapon_violence": (
1992 |             f"{mention_text} describe weapon, firing, shooting or violence signals. Involve campus security "
1993 |             "and verify CCTV, gate logs and witness reports before treating it as a routine feedback issue."
1994 |         ),
1995 |         "animal_danger": (
1996 |             f"{mention_text} describe animal danger on campus. Restrict access to the reported area and ask "
1997 |             "security or maintenance to clear and verify the location."
1998 |         ),
1999 |         "water_contamination": (
2000 |             f"{mention_text} describe unsafe or contaminated water. Stop use of the reported source, arrange "
2001 |             "safe drinking water and test water quality."
2002 |         ),
2003 |         "electric_fire": (
2004 |             f"{mention_text} describe electric shock, fire, smoke or gas leak risk. Block access to the point "
2005 |             "and send maintenance support immediately."
2006 |         ),
2007 |         "food_poisoning": (
2008 |             f"{mention_text} describe food poisoning or sickness. Inspect the food source, preserve evidence "
2009 |             "and check whether medical support is needed."
2010 |         ),
2011 |         "building_injury": (
2012 |             f"{mention_text} describe falling plaster, ceiling or structural injury risk. Close the affected "
2013 |             "room or area until maintenance verifies it."
2014 |         ),
2015 |     }
2016 |     return details.get(
2017 |         bucket,
2018 |         f"{mention_text} need senior admin verification because the issue can affect student safety or trust.",
2019 |     )
2020 |
2021 |
2022 | def ultra_concern_bucket(item: dict[str, Any]) -> tuple[str, str]:
2023 |     text = normalize_text(
2024 |         f"{item.get('title', '')} {item.get('reason', '')} {item.get('detailedSummary', '')}"
2025 |     )
2026 |     if "harassment" in text or "harass" in text or "molest" in text or "physical touch" in text or "misconduct" in text:
2027 |         return "harassment", "Harassment concern"
2028 |     if "ragging" in text or "ragged" in text or "bully" in text or "bullying" in text:
2029 |         return "ragging", "Ragging concern"
2030 |     if "corruption" in text or "bribe" in text or "extortion" in text or "forced money" in text or "taking money" in text:
2031 |         return "corruption", "Corruption concern"
2032 |     if "gun" in text or "weapon" in text or "firing" in text or "shooting" in text or "violence" in text:
2033 |         return "weapon_violence", "Weapon or violence concern"
2034 |     if "snake" in text or "animal" in text:
2035 |         return "animal_danger", "Campus animal danger"
2036 |     if "water contamination" in text or "contaminated water" in text or ("water" in text and "contamination" in text) or "ph level" in text:
2037 |         return "water_contamination", "Water contamination concern"
2038 |     if "electric shock" in text or "fire" in text or "gas leak" in text:
2039 |         return "electric_fire", "Electric or fire safety concern"
2040 |     if "food poisoning" in text:
2041 |         return "food_poisoning", "Food poisoning concern"
2042 |     if "plaster" in text or "ceiling" in text or "building" in text or "wall crack" in text or "can injure" in text:
2043 |         return "building_injury", "Building injury concern"
2044 |     return "other", str(item.get("title", "Urgent concern"))
2045 |
2046 |
2047 | def gemini_reasoning_layer(
2048 |     term_name: str,
2049 |     themes: list[dict[str, Any]],
2050 |     quality: dict[str, Any],
2051 |     feedback_count: int,
2052 |     grievance_count: int,
2053 | ) -> dict[str, Any]:
2054 |     base_reasoning = base_issue_priority_scoring_layer(
2055 |         term_name=term_name,
2056 |         themes=themes,
2057 |         quality=quality,
2058 |         feedback_count=feedback_count,
2059 |         grievance_count=grievance_count,
2060 |     )
2061 |     api_key = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")
2062 |     if not api_key:
2063 |         return {
2064 |             "base_reasoning",
2065 |             "mode": "gemini_not_configured",
2066 |             "provider": "none",
2067 |             "ultraConcerningIssues": [],
2068 |         }
2069 |
2070 |     model = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
2071 |     url = (
2072 |         f"https://generativelanguage.googleapis.com/v1beta/models/"
2073 |         f"{model}:generateContent?key={api_key}"
2074 |     )
2075 |     prompt = build_gemini_prompt(
2076 |         term_name=term_name,
2077 |         themes=themes,
2078 |         quality=quality,
2079 |         feedback_count=feedback_count,
2080 |         grievance_count=grievance_count,
2081 |     )
2082 |     body = json.dumps(
2083 |         {
2084 |             "contents": [{"parts": [{"text": prompt}]}],
2085 |             "generationConfig": {

```

```

2086 |         "temperature": 0.15,
2087 |         "responseMimeType": "application/json",
2088 |     },
2089 | }
2090 | ).encode("utf-8")
2091 |
2092 | try:
2093 |     request = urllib.request.Request(
2094 |         url,
2095 |         data=body,
2096 |         headers={"Content-Type": "application/json"},
2097 |         method="POST",
2098 |     )
2099 |     with urllib.request.urlopen(request, timeout=30) as response:
2100 |         result = json.loads(response.read().decode("utf-8"))
2101 |         text = (
2102 |             result.get("candidates", [{}])[0]
2103 |             .get("content", {})
2104 |             .get("parts", [{}])[0]
2105 |             .get("text", "")
2106 |         )
2107 |         parsed = parse_gemini_json(text)
2108 |         return normalize_reasoning_result(parsed, base_reasoning, "gemini")
2109 | except Exception as error:
2110 |     return {
2111 |         **base_reasoning,
2112 |         "mode": "gemini_unavailable",
2113 |         "provider": "gemini",
2114 |         "geminiError": f"{type(error).__name__}: {str(error)[:180]}",
2115 |         "ultraConcerningIssues": [],
2116 |     }
2117 |
2118 |
2119 | def build_gemini_prompt(
2120 |     term_name: str,
2121 |     themes: list[dict[str, Any]],
2122 |     quality: dict[str, Any],
2123 |     feedback_count: int,
2124 |     grievance_count: int,
2125 | ) -> str:
2126 |     issue_clusters = [
2127 |         {
2128 |             "id": str(index),
2129 |             "title": theme["title"],
2130 |             "summary": theme["summary"],
2131 |             "mentions": theme["mentionCount"],
2132 |             "sentiment": theme["sentiment"],
2133 |             "currentPriority": theme["priority"],
2134 |             "evidenceSamples": extract_theme_evidence_samples(theme),
2135 |             "evidenceDepartments": extract_theme_departments(theme),
2136 |             "extractedEntities": extract_theme_entities(theme),
2137 |             "taxonomy": extract_theme_taxonomy(theme),
2138 |             "confidence": extract_theme_confidence(theme),
2139 |         }
2140 |         for index, theme in enumerate(themes[:30])
2141 |     ]
2142 |     payload = {
2143 |         "termName": term_name,
2144 |         "totalResponses": feedback_count,
2145 |         "grievances": grievance_count,
2146 |         "qualitySummary": quality["summary"],
2147 |         "issueClusters": issue_clusters,
2148 |     }
2149 |     return (
2150 |         "You are the final reasoning layer for a student feedback analytics system. "
2151 |         "You receive only filtered and clustered issue summaries, never raw feedback. "
2152 |         "Return strict JSON only with keys: ultraConcerningIssues, priorityLabels, issueDetails, reportNarrative. "
2153 |         "Task 1: ultraConcerningIssues must include only rare serious risks such as ragging, harassment, "
2154 |         "physical misconduct, corruption/bribe/extortion, shooting/weapon/violence, snake/animal danger, "
2155 |         "building collapse/plaster injury risk, electric shock/fire/gas leak, food poisoning or water contamination. "
2156 |         "Do not include normal dissatisfaction like food taste, Wi-Fi, teaching clarity, marks, canteen delay or mess hygiene unless it implies immediate danger. "
2157 |         "Each ultraConcerningIssues object should include id, title, reason, mentions, detailedSummary. detailedSummary should be 2-4 plain-English sentences. "
2158 |         "Task 2: priorityLabels must follow the system's percentage rule, not opinion: HIGH when an issue has more than 10 percent of valid comments, MEDIUM when 5-10 percent, and LOW when less than 5 percent. "
2159 |         "Task 3: issueDetails must contain one object for each issue cluster with id, plainEnglishSummary, recommendedAction. "
2160 |         "plainEnglishSummary must be 3-5 sentences in simple English. Write like a careful college admin explaining the issue to a principal. Include department, location, and any relevant context. "
2161 |         "Do not mention location when the issue is not location-based, such as faculty behaviour, faculty attendance, marks partiality, assignments, exams, plagiarism. "
2162 |         "If the exact issue itself is not described clearly in the evidence, write exactly: The exact issue is not described in the feedbacks. "
2163 |         "Do not write phrases like 'AI found', 'EduPulse found', or 'the AI grouped'. Write like a human admin report. "
2164 |         "RecommendedAction must be one practical admin action based only on the evidence and must include why this action is needed. "
2165 |         "Task 4: reportNarrative should be clear human English using the same report sections: executiveSummary, topIssuesSummary, rootCauseSummary, actionPlanSummary. "
2166 |         "executiveSummary should mention total responses, useful signals after filtering, whether response volume is enough, and what the feedback is mostly about. "
2167 |         "actionPlanSummary should be detailed enough for implementation: first priority, owner type, verification step, fix step, and follow-up measurement. "
2168 |         "Use cautious wording and do not invent facts not present in clusters.\n\n"
2169 |         f"INPUT_JSON:\n{json.dumps(payload, ensure_ascii=False)}"
2170 |     )
2171 |
2172 |
2173 | def parse_gemini_json(text: str) -> dict[str, Any]:
2174 |     cleaned = text.strip()
2175 |     if cleaned.startswith("```"):
2176 |         cleaned = re.sub(r"```(?:json)?", "", cleaned, flags=re.IGNORECASE).strip()
2177 |         cleaned = re.sub(r"```$", "", cleaned).strip()
2178 |     try:
2179 |         parsed = json.loads(cleaned)
2180 |     except json.JSONDecodeError:
2181 |         start = cleaned.find("{")
2182 |         end = cleaned.rfind("}")
2183 |         if start < 0 or end < start:
2184 |             raise
2185 |         parsed = json.loads(cleaned[start : end + 1])
2186 |     if not isinstance(parsed, dict):
2187 |         raise ValueError("Gemini returned JSON but not an object")
2188 |     return parsed
2189 |
2190 |
2191 | def extract_theme_evidence_samples(theme: dict[str, Any]) -> list[str]:
2192 |     evidence = theme.get("evidence", {})
2193 |     if not isinstance(evidence, dict):
2194 |         return []
2195 |     samples = evidence.get("sampleComments") or evidence.get("examples") or []

```



```

2196 |     if not isinstance(samples, list):
2197 |         return []
2198 |     return [str(sample)[:280] for sample in samples if str(sample).strip()[:5]
2199 |
2200 |
2201 | def extract_theme_departments(theme: dict[str, Any]) -> dict[str, int]:
2202 |     evidence = theme.get("evidence", {})
2203 |     if not isinstance(evidence, dict):
2204 |         return {}
2205 |     departments = evidence.get("departments")
2206 |     if not isinstance(departments, dict):
2207 |         return {}
2208 |     return {
2209 |         str(key): int(NumberLike(value))
2210 |         for key, value in departments.items()
2211 |         if str(key).strip() and str(key) != "UNKNOWN"
2212 |     }
2213 |
2214 |
2215 | def extract_theme_entities(theme: dict[str, Any]) -> dict[str, Any]:
2216 |     evidence = theme.get("evidence", {})
2217 |     if not isinstance(evidence, dict):
2218 |         return {}
2219 |     entities = evidence.get("extractedEntities")
2220 |     return entities if isinstance(entities, dict) else {}
2221 |
2222 |
2223 | def extract_theme_taxonomy(theme: dict[str, Any]) -> dict[str, Any]:
2224 |     evidence = theme.get("evidence", {})
2225 |     if not isinstance(evidence, dict):
2226 |         return {}
2227 |     taxonomy = evidence.get("taxonomy")
2228 |     return taxonomy if isinstance(taxonomy, dict) else {}
2229 |
2230 |
2231 | def extract_theme_confidence(theme: dict[str, Any]) -> int:
2232 |     evidence = theme.get("evidence", {})
2233 |     if not isinstance(evidence, dict):
2234 |         return 0
2235 |     try:
2236 |         return int(evidence.get("issueConfidence", 0))
2237 |     except (TypeError, ValueError):
2238 |         return 0
2239 |
2240 |
2241 | def base_issue_priority_scoring_layer(
2242 |     term_name: str,
2243 |     themes: list[dict[str, Any]],
2244 |     quality: dict[str, Any],
2245 |     feedback_count: int,
2246 |     grievance_count: int,
2247 | ) -> dict[str, Any]:
2248 |     priority_labels = []
2249 |     issue_details = []
2250 |     priority_denominator = priority_comment_denominator(quality)
2251 |     for index, theme in enumerate(themes[:30]):
2252 |         mention_share = mention_priority_share(theme["mentionCount"], priority_denominator)
2253 |         priority = mention_priority(theme["mentionCount"], priority_denominator)
2254 |         samples = extract_theme_evidence_samples(theme)
2255 |         summary = (
2256 |             build_plain_issue_summary(theme, samples)
2257 |             if samples
2258 |             else "The exact issue is not described in the feedbacks."
2259 |         )
2260 |         priority_labels.append(
2261 |             {
2262 |                 "id": str(index),
2263 |                 "title": theme["title"],
2264 |                 "priority": priority,
2265 |                 "reason": (
2266 |                     f"{theme['mentionCount']} mention(s), {mention_share}% of valid comments "
2267 |                     "after quality filtering and duplicate grouping."
2268 |                 ),
2269 |             }
2270 |         )
2271 |         issue_details.append(
2272 |             {
2273 |                 "id": str(index),
2274 |                 "title": theme["title"],
2275 |                 "plainEnglishSummary": summary,
2276 |                 "recommendedAction": build_plain_recommended_action(theme, samples),
2277 |             }
2278 |         )
2279 |
2280 |     top_titles = ", ".join(theme["title"] for theme in themes[:3]) or "no dominant issue"
2281 |     useful = quality["summary"].get("usefulSignalComments", 0)
2282 |     return {
2283 |         "mode": "base_issue_priority_scoring",
2284 |         "provider": "local_mention_scoring",
2285 |         "ultraConcerningIssues": [],
2286 |         "priorityLabels": priority_labels,
2287 |         "issueDetails": issue_details,
2288 |         "reportNarrative": {
2289 |             "executiveSummary": (
2290 |                 f"{term_name}: {feedback_count} response(s) were reduced into "
2291 |                 f"{len(themes)} issue cluster(s) using {useful} useful signal comment(s). "
2292 |                 f"Top issue areas are {top_titles}."
2293 |             ),
2294 |             "topIssuesSummary": f"Top issues are ranked after filtering low-quality text and grouping duplicates. Main clusters: {top_titles}.",
2295 |             "rootCauseSummary": "Root causes are taken only from repeated student comment patterns inside the filtered clusters.",
2296 |             "actionPlanSummary": "Actions should focus first on high-mention and safety-sensitive clusters, then monitor lower-volume issues in the next cycle.",
2297 |         },
2298 |         "inputIssueCount": len(themes),
2299 |         "feedbackCount": feedback_count,
2300 |         "grievanceCount": grievance_count,
2301 |     }
2302 |
2303 |
2304 | def normalize_reasoning_result(
2305 |     result: dict[str, Any],

```

```

2306 |     base: dict[str, Any],
2307 |     provider: str,
2308 | ) -> dict[str, Any]:
2309 |     ultra = result.get("ultraConcerningIssues")
2310 |     priorities = result.get("priorityLabels")
2311 |     issue_details = result.get("issueDetails")
2312 |     narrative = result.get("reportNarrative")
2313 |     if not isinstance(ultra, list):
2314 |         ultra = base["ultraConcerningIssues"]
2315 |     if not isinstance(priorities, list):
2316 |         priorities = base["priorityLabels"]
2317 |     if not isinstance(issue_details, list):
2318 |         issue_details = base["issueDetails"]
2319 |     if not isinstance(narrative, dict):
2320 |         narrative = base["reportNarrative"]
2321 |     return {
2322 |         **base,
2323 |         "mode": "gemini_reasoning_layer",
2324 |         "provider": provider,
2325 |         "ultraConcerningIssues": normalize_ultra_concerns(ultra)[:8],
2326 |         "priorityLabels": normalize_priority_labels(priorities, base["priorityLabels"]),
2327 |         "issueDetails": normalize_issue_details(issue_details, base["issueDetails"]),
2328 |         "reportNarrative": {
2329 |             **base["reportNarrative"],
2330 |             **{key: str(value) for key, value in narrative.items()},
2331 |         },
2332 |     }
2333 |
2334 |
2335 | def normalize_issue_details(
2336 |     items: list[Any],
2337 |     base: list[dict[str, Any]],
2338 | ) -> list[dict[str, Any]]:
2339 |     base_by_id = {item["id"]: item for item in base}
2340 |     normalized = []
2341 |     for item in items:
2342 |         if not isinstance(item, dict):
2343 |             continue
2344 |         item_id = str(item.get("id", ""))
2345 |         base_item = base_by_id.get(item_id, {})
2346 |         summary = str(
2347 |             item.get("plainEnglishSummary")
2348 |             or item.get("summary")
2349 |             or base_item.get("plainEnglishSummary")
2350 |             or ""
2351 |         ).strip()
2352 |         base_summary = str(base_item.get("plainEnglishSummary") or "").strip()
2353 |         action = str(
2354 |             item.get("recommendedAction")
2355 |             or item.get("action")
2356 |             or base_item.get("recommendedAction")
2357 |             or ""
2358 |         ).strip()
2359 |         base_action = str(base_item.get("recommendedAction") or "").strip()
2360 |         cleaned_summary = clean_issue_detail_text(summary)
2361 |         if should_use_base_issue_summary(cleaned_summary, base_summary):
2362 |             cleaned_summary = clean_issue_detail_text(base_summary)
2363 |         cleaned_action = clean_issue_action_text(action)
2364 |         if should_use_base_issue_action(cleaned_action, base_action):
2365 |             cleaned_action = clean_issue_action_text(base_action)
2366 |         normalized.append(
2367 |             {
2368 |                 "id": item_id,
2369 |                 "title": str(item.get("title", base_item.get("title", ""))),
2370 |                 "plainEnglishSummary": cleaned_summary,
2371 |                 "recommendedAction": cleaned_action,
2372 |             }
2373 |         )
2374 |     return normalized or base
2375 |
2376 |
2377 | def clean_issue_detail_text(value: str) -> str:
2378 |     cleaned = re.sub(r"\s+", " ", value).strip()
2379 |     cleaned = re.sub(
2380 |         r"\bThe students did not describe the exact location in the feedback\.",
2381 |         "",
2382 |         cleaned,
2383 |         flags=re.IGNORECASE,
2384 |     )
2385 |     cleaned = re.sub(
2386 |         r"\bThe feedback does not describe the exact location\.",
2387 |         "",
2388 |         cleaned,
2389 |         flags=re.IGNORECASE,
2390 |     )
2391 |     cleaned = re.sub(r"\s+", " ", cleaned).strip()
2392 |     if not cleaned:
2393 |         return "The exact issue is not described in the feedbacks."
2394 |     return cleaned[:760]
2395 |
2396 |
2397 | def clean_issue_action_text(value: str) -> str:
2398 |     cleaned = re.sub(r"\s+", " ", value).strip()
2399 |     if not cleaned:
2400 |         return "Ask the responsible admin team to verify the issue from feedback evidence before taking action."
2401 |     return cleaned[:420]
2402 |
2403 |
2404 | def should_use_base_issue_summary(candidate: str, base: str) -> bool:
2405 |     clean_candidate = normalize_text(candidate)
2406 |     if not base.strip():
2407 |         return False
2408 |     if not clean_candidate or candidate == "The exact issue is not described in the feedbacks.":
2409 |         return True
2410 |     if "exact location" in clean_candidate:
2411 |         return True
2412 |     if is_positive_only_comment(clean_candidate) and has_issue_language(normalize_text(base)):
2413 |         return True
2414 |     return False
2415 |

```

```

2416 |
2417 | def should_use_base_issue_action(candidate: str, base: str) -> bool:
2418 |     if not base.strip():
2419 |         return False
2420 |     clean_candidate = normalize_text(candidate)
2421 |     return not clean_candidate or "verify the issue from feedback evidence" in clean_candidate
2422 |
2423 |
2424 | def normalize_ultra_concerns(items: list[Any]) -> list[dict[str, Any]]:
2425 |     normalized = []
2426 |     for index, item in enumerate(items):
2427 |         if isinstance(item, dict):
2428 |             normalized.append(
2429 |                 {
2430 |                     "id": str(item.get("id", index)),
2431 |                     "title": str(item.get("title", item.get("issue", "Ultra concerning issue"))),
2432 |                     "reason": str(item.get("reason", item.get("summary", "Rare safety or misconduct signal found in final issue clusters."))),
2433 |                     "detailedSummary": clean_issue_detail_text(
2434 |                         str(
2435 |                             item.get("detailedSummary")
2436 |                             or item.get("plainEnglishSummary")
2437 |                             or item.get("reason")
2438 |                             or item.get("summary")
2439 |                             or "Serious safety signal detected from student reports."
2440 |                         )
2441 |                     ),
2442 |                     "mentions": int(NumberLike(item.get("mentions", 1))),
2443 |                     "source": str(item.get("source", "gemini_issue_cluster_reasoning")),
2444 |                 }
2445 |             )
2446 |         elif isinstance(item, str) and item.strip():
2447 |             normalized.append(
2448 |                 {
2449 |                     "id": str(index),
2450 |                     "title": item.strip()[:90],
2451 |                     "reason": "Rare safety or misconduct signal found in final issue clusters.",
2452 |                     "detailedSummary": "Serious safety signal detected from student reports.",
2453 |                     "mentions": 1,
2454 |                     "source": "gemini_issue_cluster_reasoning",
2455 |                 }
2456 |             )
2457 |     return normalized
2458 |
2459 |
2460 | def NumberLike(value: Any) -> float:
2461 |     try:
2462 |         number = float(value)
2463 |     except (TypeError, ValueError):
2464 |         return 1
2465 |     return max(1, number)
2466 |
2467 |
2468 | def build_plain_issue_summary(theme: dict[str, Any], samples: list[str]) -> str:
2469 |     useful_samples = [sample.strip() for sample in samples if len(sample.strip().split()) >= 5]
2470 |     if not useful_samples:
2471 |         return "The exact issue is not described in the feedbacks."
2472 |     text = normalize_text(f"{theme.get('title', '')} {theme.get('summary', '')} {' '.join(useful_samples)}")
2473 |     title = str(theme.get("title", "Feedback issue"))
2474 |     mention_count = int(NumberLike(theme.get("mentionCount", len(useful_samples))))
2475 |     sample_sentence = evidence_sentence(useful_samples)
2476 |     entity_text = entity_sentence_from_theme(theme)
2477 |     impact = issue_impact_sentence(text, title)
2478 |     opening = issue_opening_sentence(title, text, mention_count)
2479 |     return clean_issue_detail_text(
2480 |         f"{opening} {sample_sentence} {impact} {entity_text}".strip()
2481 |     )
2482 |
2483 |
2484 | def evidence_excerpt(value: str, limit: int) -> str:
2485 |     return re.sub(r"\s+", " ", value.strip())[:limit].rstrip(" ")
2486 |
2487 |
2488 | def evidence_sentence(samples: list[str]) -> str:
2489 |     excerpts = [evidence_excerpt(sample, 170) for sample in samples[:3]]
2490 |     if not excerpts:
2491 |         return ""
2492 |     if len(excerpts) == 1:
2493 |         return f"The main evidence says: {excerpts[0]}."
2494 |     if len(excerpts) == 2:
2495 |         return (
2496 |             f"One comment says: {excerpts[0]}. "
2497 |             f"Another related comment says: {excerpts[1]}."
2498 |         )
2499 |     return (
2500 |         f"One comment says: {excerpts[0]}. "
2501 |         f"Another related comment says: {excerpts[1]}. "
2502 |         f"A third comment adds: {excerpts[2]}."
2503 |     )
2504 |
2505 |
2506 | def issue_opening_sentence(title: str, text: str, mentions: int) -> str:
2507 |     mention_text = f"{mentions} filtered comment(s)" if mentions != 1 else "One filtered comment"
2508 |     point_verb = "points" if mentions == 1 else "point"
2509 |     ask_verb = "asks" if mentions == 1 else "ask"
2510 |     request_verb = "requests" if mentions == 1 else "request"
2511 |     if "assignment" in text:
2512 |         return f"{mention_text} {point_verb} to assignment scheduling or submission-time concerns."
2513 |     if has_academic_workload_language(text):
2514 |         return f"{mention_text} {point_verb} to academic workload or exam-pressure concerns."
2515 |     if "marks partiality" in text or "partiality" in text or "internal marks" in text or "practical marks" in text:
2516 |         return f"{mention_text} {point_verb} to a fairness concern in marks or practical evaluation."
2517 |     if any(hint in text for hint in FACULTY_ATTENDANCE_HINTS):
2518 |         return f"{mention_text} {point_verb} to irregular faculty attendance or missed lecture slots."
2519 |     if "doubt" in text:
2520 |         return f"{mention_text} {point_verb} to slow or incomplete doubt-resolution support."
2521 |     if has_practical_example_language(text):
2522 |         return f"{mention_text} {ask_verb} for more practical examples and applied teaching support."
2523 |     if "subject knowledge" in text or "lack knowledge" in text or "cannot explain" in text:
2524 |         return f"{mention_text} {point_verb} to teaching-quality or subject-knowledge gaps."
2525 |     if "new faculty" in text or "replace faculty" in text:

```

```

2526 |         return f"{mention_text} {request_verb} faculty replacement or additional teaching support."
2527 |     if "faculty" in text or "teacher" in text:
2528 |         return f"{mention_text} {point_verb} to a faculty-related concern."
2529 |     if "projector" in text:
2530 |         return f"{mention_text} {point_verb} to a classroom projector problem."
2531 |     if "placement" in text:
2532 |         return f"{mention_text} {point_verb} to placement access or eligibility communication concerns."
2533 |     if "lab equipment" in text or ("lab" in text and ("equipment" in text or "component" in text)):
2534 |         return f"{mention_text} {point_verb} to lab equipment availability or practical-session readiness concerns."
2535 |     if "mess" in text or "food" in text or "canteen" in text:
2536 |         return f"{mention_text} {point_verb} to food-service quality, hygiene or availability concerns."
2537 |     if "wifi" in text or "internet" in text or "network" in text:
2538 |         return f"{mention_text} {point_verb} to network connectivity problems."
2539 |     if has_event_activity_language(text):
2540 |         return f"{mention_text} {point_verb} to campus activity or extracurricular planning concerns."
2541 |     if has_sports_timing_language(text):
2542 |         return f"{mention_text} {point_verb} to sports facility access or timing concerns."
2543 |     if "sports" in text or "equipment" in text:
2544 |         return f"{mention_text} {point_verb} to sports equipment or facility availability concerns."
2545 |     if "library" in text:
2546 |         return f"{mention_text} {point_verb} to library access or facility concerns."
2547 |     if has_transport_language(text):
2548 |         return f"{mention_text} {point_verb} to transport reliability concerns."
2549 |     return f"{mention_text} {point_verb} to {title.lower()}."
2550 |
2551 |
2552 | def issue_impact_sentence(text: str, title: str) -> str:
2553 |     if "assignment" in text:
2554 |         return "This can affect academic planning because students may get less time to complete work properly."
2555 |     if has_academic_workload_language(text):
2556 |         return "This can affect preparation quality because students may feel overloaded during exams or academic deadlines."
2557 |     if "marks" in text or "partiality" in text:
2558 |         return "This can reduce student trust in assessment fairness if it is not verified and handled transparently."
2559 |     if has_practical_example_language(text):
2560 |         return "This can affect concept clarity because students are asking for teaching to connect more directly with practical use."
2561 |     if "faculty" in text or "teacher" in text or "doubt" in text:
2562 |         return "This can affect learning quality because students depend on regular classes, clear explanations and timely doubt support."
2563 |     if "projector" in text or "classroom" in text:
2564 |         return "This can directly disturb classroom teaching until the equipment or room issue is fixed."
2565 |     if "placement" in text:
2566 |         return "This can affect student opportunity and trust if eligibility rules are not explained clearly."
2567 |     if "lab equipment" in text or ("lab" in text and ("equipment" in text or "component" in text)):
2568 |         return "This can affect practical learning because students need working equipment and components during lab sessions."
2569 |     if "mess" in text or "food" in text or "canteen" in text:
2570 |         return "This affects daily student experience because food quality and hygiene are used every day."
2571 |     if "wifi" in text or "internet" in text:
2572 |         return "This affects practical work, submissions and project activity when students need stable connectivity."
2573 |     if has_event_activity_language(text):
2574 |         return "This affects campus participation because students expect events and activities to be planned with enough access and time."
2575 |     if has_sports_timing_language(text):
2576 |         return "This affects student participation because limited timings can make sports facilities hard to use."
2577 |     if "sports" in text:
2578 |         return "This affects campus participation because students cannot use activities properly without required equipment or access."
2579 |     if "library" in text:
2580 |         return "This affects exam preparation and study time when library access is limited."
2581 |     if has_transport_language(text):
2582 |         return "This affects punctuality and daily commute reliability."
2583 |     return f"This should be reviewed because repeated feedback connects it to {title.lower()}."
2584 |
2585 |
2586 | def has_transport_language(text: str) -> bool:
2587 |     return "transport" in text or bool(re.search(r"\bbus(?:es)?\b", text))
2588 |
2589 |
2590 | def has_academic_workload_language(text: str) -> bool:
2591 |     return any(
2592 |         phrase in text
2593 |         for phrase in ("exam load", "academic load", "workload", "load feels high", "study pressure")
2594 |     )
2595 |
2596 |
2597 | def has_practical_example_language(text: str) -> bool:
2598 |     return "practical examples" in text or "examples needed" in text or "more practical examples" in text
2599 |
2600 |
2601 | def has_event_activity_language(text: str) -> bool:
2602 |     return any(
2603 |         phrase in text
2604 |         for phrase in ("extracurricular", "event", "events", "activities", "campus activity")
2605 |     )
2606 |
2607 |
2608 | def has_sports_timing_language(text: str) -> bool:
2609 |     return any(
2610 |         phrase in text
2611 |         for phrase in ("sports facility timing", "sports facilities", "timings are limited", "limited timings", "practice timing", "practice timings")
2612 |     )
2613 |
2614 |
2615 | def entity_sentence_from_theme(theme: dict[str, Any]) -> str:
2616 |     evidence = theme.get("evidence", {})
2617 |     if not isinstance(evidence, dict):
2618 |         return ""
2619 |     entities = evidence.get("extractedEntities")
2620 |     if not isinstance(entities, dict):
2621 |         return ""
2622 |     parts = []
2623 |     locations = [str(item) for item in entities.get("locations", []) if str(item).strip()]
2624 |     people = [str(item) for item in entities.get("people", []) if str(item).strip()]
2625 |     semesters = [str(item) for item in entities.get("semesters", []) if str(item).strip()]
2626 |     departments = entities.get("departments", {})
2627 |     if locations:
2628 |         parts.append(f"Location clue(s): {' '.join(locations[:3])}.")
2629 |     if people:
2630 |         parts.append(f"Named person/student clue(s): {' '.join(people[:3])}.")
2631 |     if departments and isinstance(departments, dict):
2632 |         parts.append(f"Department clue(s): {' '.join(list(departments.keys())[:3])}.")
2633 |     if semesters:
2634 |         parts.append(f"Year/semester clue(s): {' '.join(semesters[:3])}.")
2635 |     return " ".join(parts)

```

```

2636 |
2637 |
2638 | def extract_departments_from_text(text: str) -> dict[str, int]:
2639 |     clean = normalize_text(text)
2640 |     departments = {}
2641 |     for code in ("CSE", "ECE", "IT", "ME", "CE", "CSDS", "CS DS", "CSD"):
2642 |         if re.search(rf"\b{code.lower()}\b", clean):
2643 |             departments[code.replace(" ", "")] = 1
2644 |     return departments
2645 |
2646 |
2647 | def build_plain_recommended_action(theme: dict[str, Any], samples: list[str]) -> str:
2648 |     text = normalize_text(f"{theme.get('title', '')} {theme.get('summary', '')} {' '.join(samples)}")
2649 |     if "assignment" in text:
2650 |         return "Review assignment upload dates and deadline spacing because students are flagging workload timing as the practical cause of the issue."
2651 |     if has_academic_workload_language(text):
2652 |         return "Review exam load, syllabus pacing and deadline clustering because students are reporting academic pressure rather than a single classroom fault."
2653 |     if "projector" in text:
2654 |         return "Inspect the mentioned classroom projector and repair or replace it before the next class slot because teaching is being affected."
2655 |     if "partiality" in text or "marks" in text:
2656 |         return "Review the marking concern confidentially and verify whether evaluation rules were followed because students are questioning fairness."
2657 |     if any(hint in text for hint in FACULTY_ATTENDANCE_HINTS):
2658 |         return "Verify faculty attendance records, missed lecture slots and student timetable impact before the next department review because class regularity is affected."
2659 |     if "doubt" in text:
2660 |         return "Ask the department to check doubt-session availability and response time because students are asking for faster academic support."
2661 |     if has_practical_example_language(text):
2662 |         return "Ask faculty coordinators to add practical examples in lectures because students are asking for clearer applied understanding."
2663 |     if "placement" in text:
2664 |         return "Review placement eligibility communication and confirm whether ECE students were fairly allowed for eligible drives because opportunity access is affected."
2665 |     if "lab equipment" in text or ("lab" in text and ("equipment" in text or "component" in text)):
2666 |         return "Check lab equipment availability and replace missing or faulty components because practical sessions depend on working lab resources."
2667 |     if "fest" in text:
2668 |         return "Review the event schedule and collect student feedback before planning the next fest duration because students are reporting that the event frequency is affected."
2669 |     if has_event_activity_language(text):
2670 |         return "Review the campus activity calendar and participation process because students are asking for better-planned extracurricular opportunities."
2671 |     if "sports item" in text or "sports items" in text:
2672 |         return "Audit sports inventory and make required items available during practice hours because students cannot use sports facilities without equipment."
2673 |     if has_sports_timing_language(text):
2674 |         return "Review sports facility timings and access slots because students are saying availability is limited even when facilities are good."
2675 |     if "behaviour" in text or "behavior" in text or "rude" in text:
2676 |         return "Ask the department head to review classroom behaviour complaints confidentially because the issue affects classroom trust."
2677 |     if "knowledge" in text or "new faculty" in text:
2678 |         return "Review subject allocation and arrange faculty mentoring or replacement where repeated learning gaps are confirmed because students are reporting knowledge gaps."
2679 |     if "wifi" in text:
2680 |         return "Check the affected Wi-Fi zone and publish a resolution timeline because connectivity issues affect project and submission work."
2681 |     if "mess" in text or "food" in text or "canteen" in text:
2682 |         return "Inspect the affected food service area and verify hygiene and food quality controls because students use this service daily."
2683 |     if "washroom" in text or "classroom" in text or "infrastructure" in text:
2684 |         return "Inspect the affected infrastructure area and close the repair with proof of completion because repeated comments point to a maintenance gap."
2685 |     if has_transport_language(text):
2686 |         return "Check route timing and student pickup/drop issues because transport feedback affects daily punctuality."
2687 |     return "Assign an admin owner to verify the issue and track closure in the next feedback cycle because the comments show a repeated concern."
2688 |
2689 |
2690 | def normalize_priority_labels(
2691 |     labels: list[Any],
2692 |     base: list[dict[str, Any]],
2693 | ) -> list[dict[str, Any]]:
2694 |     base_by_id = {item["id"]: item for item in base}
2695 |     normalized = []
2696 |     for item in labels:
2697 |         if not isinstance(item, dict):
2698 |             continue
2699 |         item_id = str(item.get("id", ""))
2700 |         priority = str(item.get("priority", "")).upper()
2701 |         if priority not in {"HIGH", "MEDIUM", "LOW"}:
2702 |             priority = base_by_id.get(item_id, {}).get("priority", "LOW")
2703 |         normalized.append(
2704 |             {
2705 |                 "id": item_id,
2706 |                 "title": str(item.get("title", base_by_id.get(item_id, {}).get("title", ""))),
2707 |                 "priority": priority,
2708 |                 "reason": str(item.get("reason", base_by_id.get(item_id, {}).get("reason", ""))),
2709 |             }
2710 |         )
2711 |     return normalized or base
2712 |
2713 |
2714 | def apply_reasoning_priorities(
2715 |     themes: list[dict[str, Any]],
2716 |     reasoning: dict[str, Any],
2717 |     priority_denominator: int,
2718 | ) -> list[dict[str, Any]]:
2719 |     ultra_ids = {
2720 |         str(item.get("id"))
2721 |         for item in reasoning.get("ultraConcerningIssues", [])
2722 |         if isinstance(item, dict)
2723 |     }
2724 |     updated = []
2725 |     for index, theme in enumerate(themes):
2726 |         item = {"**theme": theme}
2727 |         item_id = str(index)
2728 |         if item_id in ultra_ids:
2729 |             item["priority"] = "CRITICAL"
2730 |         else:
2731 |             item["priority"] = mention_priority(item["mentionCount"], priority_denominator)
2732 |             priority_share = mention_priority_share(item["mentionCount"], priority_denominator)
2733 |             item["evidence"] = {
2734 |                 **item.get("evidence", {}),
2735 |                 "reasoningLayerPriority": item["priority"],
2736 |                 "prioritySharePercent": priority_share,
2737 |                 "priorityRule": ">10% HIGH, 5-10% MEDIUM, <5% LOW of valid comments after quality filtering",
2738 |             }
2739 |             updated.append(item)
2740 |     return updated
2741 |
2742 |
2743 | def apply_reasoning_issue_details(
2744 |     themes: list[dict[str, Any]],
2745 |     reasoning: dict[str, Any],

```

```

2746 | ) -> list[dict[str, Any]]:
2747 |     details_by_index = {
2748 |         str(item.get("id")): item
2749 |         for item in reasoning.get("issueDetails", [])
2750 |         if isinstance(item, dict)
2751 |     }
2752 |     updated = []
2753 |     for index, theme in enumerate(themes):
2754 |         item = {**theme}
2755 |         detail = details_by_index.get(str(index))
2756 |         if detail:
2757 |             item["evidence"] = {
2758 |                 **item.get("evidence", {}),
2759 |                 "geminiPlainEnglishSummary": clean_issue_detail_text(
2760 |                     str(detail.get("plainEnglishSummary", ""))
2761 |                 ),
2762 |                 "geminiRecommendedAction": clean_issue_action_text(
2763 |                     str(detail.get("recommendedAction", ""))
2764 |                 ),
2765 |             }
2766 |         else:
2767 |             samples = extract_theme_evidence_samples(item)
2768 |             item["evidence"] = {
2769 |                 **item.get("evidence", {}),
2770 |                 "geminiPlainEnglishSummary": build_plain_issue_summary(item, samples),
2771 |                 "geminiRecommendedAction": build_plain_recommended_action(item, samples),
2772 |             }
2773 |         updated.append(item)
2774 |     return updated
2775 |
2776 |
2777 | def sync_reasoning_with_final_themes(
2778 |     reasoning: dict[str, Any],
2779 |     themes: list[dict[str, Any]],
2780 |     priority_denominator: int,
2781 | ) -> dict[str, Any]:
2782 |     priority_labels = []
2783 |     issue_details = []
2784 |     for index, theme in enumerate(themes):
2785 |         share = mention_priority_share(theme["mentionCount"], priority_denominator)
2786 |         evidence = theme.get("evidence", {}) if isinstance(theme.get("evidence"), dict) else {}
2787 |         priority_labels.append(
2788 |             {
2789 |                 "id": str(index),
2790 |                 "title": theme["title"],
2791 |                 "priority": theme["priority"],
2792 |                 "reason": (
2793 |                     f"{theme['mentionCount']} mention(s), {share}% of "
2794 |                     f"{priority_denominator} useful filtered comment(s). "
2795 |                     "Rule: >10% HIGH, 5-10% MEDIUM, <5% LOW."
2796 |                 ),
2797 |             }
2798 |         )
2799 |     issue_details.append(
2800 |         {
2801 |             "id": str(index),
2802 |             "title": theme["title"],
2803 |             "plainEnglishSummary": clean_issue_detail_text(
2804 |                 str(evidence.get("geminiPlainEnglishSummary") or theme.get("summary") or "")
2805 |             ),
2806 |             "recommendedAction": clean_issue_action_text(
2807 |                 str(evidence.get("geminiRecommendedAction") or theme.get("actionTitle") or "")
2808 |             ),
2809 |         }
2810 |     )
2811 |     return {
2812 |         **reasoning,
2813 |         "priorityLabels": priority_labels,
2814 |         "issueDetails": issue_details,
2815 |         "priorityRule": {
2816 |             "denominator": priority_denominator,
2817 |             "basis": "useful filtered comments after quality filtering and duplicate grouping",
2818 |             "high": ">10%",
2819 |             "medium": "5-10%",
2820 |             "low": "<5%",
2821 |         },
2822 |     }
2823 |
2824 |
2825 | def priority_comment_denominator(quality: dict[str, Any]) -> int:
2826 |     summary = quality.get("summary", {}) if isinstance(quality, dict) else {}
2827 |     value = summary.get("usefulSignalComments") or summary.get("usefulComments") or 0
2828 |     return max(1, int(NumberLike(value)))
2829 |
2830 |
2831 | def mention_priority_share(mentions: int, denominator: int) -> float:
2832 |     return round((int(NumberLike(mentions)) / max(1, denominator)) * 100, 2)
2833 |
2834 |
2835 | def mention_priority(mentions: int, denominator: int) -> Priority:
2836 |     share = mention_priority_share(mentions, denominator)
2837 |     if share > 10:
2838 |         return "HIGH"
2839 |     if share >= 5:
2840 |         return "MEDIUM"
2841 |     return "LOW"
2842 |
2843 |
2844 | def generate_report_narrative(
2845 |     term_name: str,
2846 |     themes: list[dict[str, Any]],
2847 |     actions: list[dict[str, Any]],
2848 |     quality: dict[str, Any],
2849 |     feedback_count: int,
2850 |     grievance_count: int,
2851 |     reasoning: dict[str, Any] | None = None,
2852 | ) -> dict[str, Any]:
2853 |     critical_themes = [theme for theme in themes if theme["priority"] == "CRITICAL"]
2854 |     high_themes = [theme for theme in themes if theme["priority"] == "HIGH"]
2855 |     negative_themes = [

```

```

2856 |         theme for theme in themes if theme["sentiment"] in {"NEGATIVE", "CRITICAL"}
2857 |     ]
2858 |     top_theme = themes[0]["title"] if themes else "no dominant issue"
2859 |     rejected = quality["summary"]["rejected"]
2860 |
2861 |     reasoning_narrative = reasoning.get("reportNarrative", {}) if reasoning else {}
2862 |     executive_summary = str(reasoning_narrative.get("executiveSummary") or (
2863 |         f"({term_name}): analyzed {feedback_count} rating response(s), "
2864 |         f"({quality['summary']['usefulComments']} useful comment(s), and "
2865 |         f"({grievance_count} grievance(s). The result contains {len(themes)} theme(s), "
2866 |         f"with {len(negative_themes)} negative/critical signal(s), "
2867 |         f"({len(critical_themes)} critical risk(s), and {len(actions)} action item(s). "
2868 |         f"Top signal: {top_theme}."
2869 |     ))
2870 |
2871 |     return {
2872 |         "mode": "gemini-assisted-report-generator" if reasoning else "structured-local-report-generator",
2873 |         "executiveSummary": executive_summary,
2874 |         "topIssuesSummary": reasoning_narrative.get("topIssuesSummary"),
2875 |         "rootCauseSummary": reasoning_narrative.get("rootCauseSummary"),
2876 |         "actionPlanSummary": reasoning_narrative.get("actionPlanSummary"),
2877 |         "boardReadyHighlights": [
2878 |             f"Top concern: {top_theme}",
2879 |             f"Critical themes: {len(critical_themes)}",
2880 |             f"High priority themes: {len(high_themes)}",
2881 |             f"Rejected low-quality comments: {sum(rejected.values())}",
2882 |         ],
2883 |         "recommendedNextStep": (
2884 |             actions[0]["title"] if actions else "Collect more feedback before action planning"
2885 |         ),
2886 |         "llmReadyPayload": {
2887 |             "themes": [
2888 |                 {
2889 |                     "title": theme["title"],
2890 |                     "priority": theme["priority"],
2891 |                     "sentiment": theme["sentiment"],
2892 |                     "mentions": theme["mentionCount"],
2893 |                 }
2894 |                 for theme in themes[:8]
2895 |             ],
2896 |             "actions": actions,
2897 |         },
2898 |     }
2899 |
2900 |
2901 | def calculate_confidence(
2902 |     total_inputs: int, useful_comments: int, duplicate_count: int, theme_count: int
2903 | ) -> float:
2904 |     if total_inputs == 0:
2905 |         return 0.35
2906 |     sample_score = min(total_inputs / 200, 1) * 0.3
2907 |     comment_score = min(useful_comments / max(total_inputs, 1), 1) * 0.25
2908 |     theme_score = 0.25 if theme_count > 0 else 0.05
2909 |     duplicate_penalty = min(duplicate_count / max(total_inputs, 1), 0.2)
2910 |     return round(max(0.35, min(0.95, 0.25 + sample_score + comment_score + theme_score - duplicate_penalty)), 2)
2911 |
2912 |
2913 | def sentiment_breakdown(
2914 |     feedback: list[FeedbackItem], weights: dict[str, int] | None = None
2915 | ) -> dict[str, Any]:
2916 |     overall: Counter[str] = Counter()
2917 |     categories: dict[str, Counter[str]] = defaultdict(Counter)
2918 |
2919 |     for item in feedback:
2920 |         label = sentiment_from_rating_and_text(item.rating, item.comment or "")
2921 |         weight = int(weights or {}).get(item.id, 1)
2922 |         overall[label] += weight
2923 |         categories[item.categoryName][label] += weight
2924 |
2925 |     return {
2926 |         "overall": normalize_sentiment_counts(overall),
2927 |         "categories": {
2928 |             category: normalize_sentiment_counts(counter)
2929 |             for category, counter in categories.items()
2930 |         },
2931 |     }
2932 |
2933 |
2934 | def sentiment_from_rating_and_text(rating: int, comment: str) -> Sentiment:
2935 |     clean = normalize_text(comment)
2936 |     if has_critical_language(clean):
2937 |         return "CRITICAL"
2938 |     if rating <= 2:
2939 |         return "NEGATIVE"
2940 |     if rating == 3:
2941 |         return "NEUTRAL"
2942 |     return "POSITIVE"
2943 |
2944 |
2945 | def normalize_sentiment_counts(counter: Counter[str]) -> dict[str, dict[str, float | int]]:
2946 |     labels = ["POSITIVE", "NEUTRAL", "NEGATIVE", "CRITICAL"]
2947 |     total = sum(counter.values()) or 1
2948 |     return {
2949 |         label: {
2950 |             "count": int(counter[label]),
2951 |             "percentage": round((int(counter[label]) / total) * 100, 1),
2952 |         }
2953 |         for label in labels
2954 |     }
2955 |
2956 |
2957 | def department_satisfaction(items: list[FeedbackItem]) -> dict[str, Any]:
2958 |     buckets: dict[str, list[int]] = defaultdict(list)
2959 |     for item in items:
2960 |         buckets[item.departmentCode or "UNKNOWN"].append(item.rating)
2961 |     return {
2962 |         department: {
2963 |             "averageRating": round(sum(ratings) / len(ratings), 2),
2964 |             "responseCount": len(ratings),
2965 |         }

```



```

2966 |         for department, ratings in buckets.items()
2967 |     }
2968 |
2969 |
2970 | def title_from_docs(docs: list[dict[str, Any]]) -> str:
2971 |     category_counter = Counter(doc["categoryName"] for doc in docs)
2972 |     text = normalize_text(" ".join(doc["text"] for doc in docs))
2973 |     category = category_counter.most_common(1)[0][0]
2974 |     taxonomy_title = taxonomy_title_from_docs(docs)
2975 |     if taxonomy_title:
2976 |         return taxonomy_title
2977 |     if "harassment" in text:
2978 |         return "Harassment concern"
2979 |     if "ragging" in text or "ragged" in text:
2980 |         return "Ragging concern"
2981 |     if "snake" in text:
2982 |         return f"{category} snake issue"
2983 |     if "gun" in text or "firing" in text or "shooting" in text:
2984 |         return f"{category} gun firing issue"
2985 |     if "corruption" in text or "bribe" in text or "forced money" in text or "taking money" in text:
2986 |         return "Corruption concern"
2987 |     if "projector" in text:
2988 |         return f"{category} projector issue"
2989 |     if normalize_text(category) == "mess" and any(
2990 |         hint in text
2991 |         for hint in {
2992 |             "mess",
2993 |             "food",
2994 |             "hygiene",
2995 |             "cleanliness",
2996 |             "clean",
2997 |             "insects",
2998 |             "quantity",
2999 |             "serving",
3000 |             "dinner",
3001 |             "lunch",
3002 |             "hostel",
3003 |         }
3004 |     ):
3005 |         return "Mess hygiene and food availability issue"
3006 |     if "placement" in text or "placements" in text:
3007 |         return "Placement access issue"
3008 |     if "fest" in text or "fests" in text:
3009 |         return "Fest duration issue"
3010 |     if "sports item" in text or "sports items" in text:
3011 |         return "Sports item availability issue"
3012 |     if "lack knowledge" in text or "lacks knowledge" in text or "subject knowledge" in text:
3013 |         return f"{category} subject knowledge issue"
3014 |     if "new faculty" in text:
3015 |         return f"{category} new faculty request"
3016 |     if any(hint in text for hint in FACULTY_MARKS_HINTS):
3017 |         return f"{category} marks partiality issue"
3018 |     if any(hint in text for hint in FACULTY_ATTENDANCE_HINTS):
3019 |         return f"{category} attendance issue"
3020 |     if any(hint in text for hint in FACULTY_BEHAVIOUR_HINTS):
3021 |         return f"{category} behaviour issue"
3022 |     keyword = top_keyword(text)
3023 |     if keyword and keyword != "feedback":
3024 |         return f"{category} {keyword} issue"
3025 |     return f"{category} repeated concern"
3026 |
3027 |
3028 | def most_common_category_id(docs: list[dict[str, Any]]) -> str | None:
3029 |     category_ids = [doc["categoryId"] for doc in docs if doc.get("categoryId")]
3030 |     if not category_ids:
3031 |         return None
3032 |     return Counter(category_ids).most_common(1)[0][0]
3033 |
3034 |
3035 | def top_keyword(text: str) -> str:
3036 |     words = [
3037 |         word
3038 |         for word in normalize_text(text).split()
3039 |         if len(word) > 3 and word not in STOPWORDS
3040 |     ]
3041 |     if not words:
3042 |         return "feedback"
3043 |     return Counter(words).most_common(1)[0][0]
3044 |
3045 |
3046 | def normalize_text(text: str) -> str:
3047 |     lowered = text.lower()
3048 |     lowered = re.sub(r"\bwi[s-]?fi\b", "wifi", lowered)
3049 |     lowered = re.sub(r"^\a-z0-9\s", " ", lowered)
3050 |     lowered = re.sub(r"\s+", " ", lowered).strip()
3051 |     return lowered
3052 |
3053 |
3054 | def has_critical_language(text: str) -> bool:
3055 |     clean = normalize_text(text)
3056 |     return any(word in clean for word in CRITICAL_WORDS)
3057 |
3058 |
3059 | def has_ultra_concern_language(text: str) -> bool:
3060 |     clean = normalize_text(text)
3061 |     return any(word in clean for word in ULTRA_CONCERN_WORDS)
3062 |
3063 |
3064 | def has_health_risk_language(text: str) -> bool:
3065 |     clean = normalize_text(text)
3066 |     return any(word in clean for word in HEALTH_RISK_WORDS)
3067 |
3068 |
3069 | def priority_rank(priority: Priority) -> int:
3070 |     return {"LOW": 1, "MEDIUM": 2, "HIGH": 3, "CRITICAL": 4}[priority]
3071 |
3072 |
3073 | def model_stack_metadata() -> dict[str, Any]:
3074 |     hf_enabled = os.getenv("EDUPULSE_ENABLE_HF_MODELS", "false").lower() == "true"
3075 |     return {

```



```

3076 |         "mode": model_mode(),
3077 |         "hfModelsEnabled": hf_enabled,
3078 |         "sentiment": {
3079 |             "primary": sentiment_model_name(),
3080 |             "fallback": "domain_lexicon_plus_rating_rules",
3081 |             "criticalOverride": "safety_health_abuse_keyword_layer",
3082 |         },
3083 |         "themeEmbeddings": {
3084 |             "primary": embedding_model_name(),
3085 |             "fallback": "tfidf_minibatch_kmeans_then_keyword_cluster",
3086 |             "supportsE5Prefixing": "e5" in embedding_model_name().lower(),
3087 |         },
3088 |         "clustering": {
3089 |             "primary": "sklearn.cluster.HDBSCAN density clustering",
3090 |             "fallback": "MiniBatchKMeans",
3091 |             "reason": "HDBSCAN finds natural issue groups and can mark noise/outliers.",
3092 |         },
3093 |         "quality": {
3094 |             "method": "rule_and_anomaly_filter",
3095 |             "checks": [
3096 |                 "duplicate",
3097 |                 "too_short",
3098 |                 "too_generic",
3099 |                 "repeated_characters",
3100 |                 "repeated_words",
3101 |                 "random_text",
3102 |                 "low_lexical_diversity",
3103 |                 "out_of_context",
3104 |                 "extreme_low_information",
3105 |             ],
3106 |         },
3107 |         "reportGeneration": {
3108 |             "mode": "structured_local_generator",
3109 |             "llmUse": "optional final wording layer after structured signals",
3110 |         },
3111 |         "humanCorrectionLoop": {
3112 |             "endpoint": "/human-feedback",
3113 |             "storage": str(CORRECTIONS_PATH),
3114 |         },
3115 |     }
3116 |
3117 |
3118 | def model_mode() -> str:
3119 |     if os.getenv("EDUPULSE_ENABLE_HF_MODELS", "false").lower() != "true":
3120 |         return "deterministic-fallback-safe-mode"
3121 |     return "self-hosted-transformer-mode"
3122 |
3123 |
3124 | def sentiment_model_name() -> str:
3125 |     return os.getenv(
3126 |         "EDUPULSE_SENTIMENT_MODEL",
3127 |         "cardiffnlp/twitter-roberta-base-sentiment-latest",
3128 |     )
3129 |
3130 |
3131 | def embedding_model_name() -> str:
3132 |     return os.getenv("EDUPULSE_EMBEDDING_MODEL", "BAAI/bge-small-en-v1.5")
3133 |
3134 |
3135 | def get_sentiment_pipeline() -> Any | None:
3136 |     global _sentiment_pipeline
3137 |     if os.getenv("EDUPULSE_ENABLE_HF_MODELS", "false").lower() != "true":
3138 |         return None
3139 |     if _sentiment_pipeline is not None:
3140 |         return _sentiment_pipeline
3141 |     try:
3142 |         from transformers import pipeline
3143 |
3144 |         _sentiment_pipeline = pipeline(
3145 |             "sentiment-analysis",
3146 |             model=sentiment_model_name(),
3147 |         )
3148 |         return _sentiment_pipeline
3149 |     except Exception:
3150 |         return None
3151 |
3152 |
3153 | def get_embedding_model() -> Any | None:
3154 |     global _embedding_model
3155 |     if os.getenv("EDUPULSE_ENABLE_HF_MODELS", "false").lower() != "true":
3156 |         return None
3157 |     if _embedding_model is not None:
3158 |         return _embedding_model
3159 |     try:
3160 |         from sentence_transformers import SentenceTransformer
3161 |
3162 |         _embedding_model = SentenceTransformer(embedding_model_name())
3163 |         return _embedding_model
3164 |     except Exception:
3165 |         return None

```

edupulse-backend/ai-service/evaluation/evaluate_ai.py

File size: 13,047 bytes

```
1 | from __future__ import annotations
2 |
3 | import json
4 | import sys
5 | from collections import Counter
6 | from datetime import datetime, timezone
7 | from pathlib import Path
8 | from typing import Any
9 |
10 | ROOT = Path(__file__).resolve().parents[2]
11 | AI_SERVICE = ROOT / "ai-service"
12 | sys.path.insert(0, str(AI_SERVICE))
13 |
14 | from app.main import ( # noqa: E402
15 |     AnalysisRequest,
16 |     FeedbackItem,
17 |     Term,
18 |     analyze,
19 |     build_feedback_documents,
20 |     classify_sentiments,
21 |     model_mode,
22 |     normalize_text,
23 |     quality_filter,
24 |     reject_reason,
25 | )
26 |
27 |
28 | DATASET_PATH = Path(__file__).with_name("labeled_feedback.json")
29 | METRICS_PATH = Path(__file__).with_name("latest_metrics.json")
30 | REPORT_PATH = ROOT / "docs" / "ai-evaluation-report.md"
31 |
32 | THEME_STOPWORDS = {
33 |     "issue",
34 |     "signal",
35 |     "support",
36 |     "quality",
37 |     "campus",
38 |     "college",
39 |     "student",
40 |     "students",
41 | }
42 |
43 |
44 | def main() -> None:
45 |     dataset = json.loads(DATASET_PATH.read_text(encoding="utf-8"))
46 |     evaluation_dataset = normalize_legacy_grievances_to_feedback(dataset)
47 |     feedback = [FeedbackItem(**item) for item in evaluation_dataset["feedback"]]
48 |
49 |     quality_rows = evaluate_quality_filter(evaluation_dataset["feedback"], feedback)
50 |     useful_feedback = [
51 |         item
52 |         for item, row in zip(feedback, quality_rows)
53 |         if row["predictedUseful"]
54 |     ]
55 |
56 |     sentiment_rows = evaluate_sentiment(evaluation_dataset, useful_feedback)
57 |     analysis = analyze(
58 |         AnalysisRequest(
59 |             term=Term(id="eval-term", name="EduPulse Evaluation Term"),
60 |             feedback=feedback,
61 |             grievances=[],
62 |         )
63 |     )
64 |     theme_rows = evaluate_theme_coverage(evaluation_dataset, analysis)
65 |     priority_rows = evaluate_priority_recall(evaluation_dataset, analysis)
66 |
67 |     metrics = {
68 |         "generatedAt": datetime.now(timezone.utc).isoformat(),
69 |         "datasetName": dataset["datasetName"],
70 |         "datasetVersion": dataset["version"],
71 |         "modelMode": model_mode(),
72 |         "sampleSize": {
73 |             "feedback": len(feedback),
74 |             "legacySafetyFeedbackConverted": len(dataset.get("grievances", [])),
75 |             "total": len(feedback),
76 |         },
77 |         "qualityFilterAccuracy": accuracy(quality_rows),
78 |         "sentimentAccuracy": accuracy(sentiment_rows),
79 |         "themeCoverage": accuracy(theme_rows),
80 |         "priorityPlacement": accuracy(priority_rows),
81 |         "overallScore": round(
82 |             (
83 |                 accuracy(quality_rows)["score"]
84 |                 + accuracy(sentiment_rows)["score"]
85 |                 + accuracy(theme_rows)["score"]
86 |                 + accuracy(priority_rows)["score"]
87 |             )
88 |             / 4,
89 |             4,
90 |         ),
91 |         "qualityBreakdown": quality_filter(feedback)["summary"],
92 |         "analysisSummary": analysis["summary"],
93 |         "analysisConfidence": analysis["confidence"],
94 |         "themeCount": len(analysis["themes"]),
95 |         "actionCount": len(analysis["actions"]),
96 |         "rows": {
97 |             "quality": quality_rows,
98 |             "sentiment": sentiment_rows,
99 |             "themes": theme_rows,
100 |            "priority": priority_rows,
101 |        },
102 |     }
103 |
104 |     METRICS_PATH.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
105 |     REPORT_PATH.write_text(render_report(metrics, analysis), encoding="utf-8")
```

```

106 |     print(json.dumps(summary_for_console(metrics), indent=2))
107 |
108 |
109 | def normalize_legacy_grievances_to_feedback(dataset: dict[str, Any]) -> dict[str, Any]:
110 |     feedback = list(dataset["feedback"])
111 |     for index, item in enumerate(dataset.get("grievances", []), start=1):
112 |         severity = str(item.get("severity", "MEDIUM"))
113 |         category = str(item.get("category", "Safety"))
114 |         feedback.append(
115 |             {
116 |                 "id": f"legacy-safety-feedback-{index}",
117 |                 "categoryId": f"cat-{{normalize_text(category).replace(' ', '-')}}",
118 |                 "categoryName": category,
119 |                 "questionId": f"q-legacy-safety-{index}",
120 |                 "questionText": "Describe any safety-sensitive campus concern.",
121 |                 "rating": 1 if severity in {"HIGH", "CRITICAL"} else 2,
122 |                 "comment": f"{{item.get('title', '')}}. {{item.get('description', '')}}.strip()",
123 |                 "departmentCode": item.get("departmentCode"),
124 |                 "semesterNumber": None,
125 |                 "expectedSentiment": item.get("expectedSentiment", "NEGATIVE"),
126 |                 "expectedUseful": True,
127 |                 "expectedTheme": item.get("expectedTheme", category),
128 |                 "expectedPriority": item.get("expectedPriority", "MEDIUM"),
129 |             }
130 |         )
131 |
132 |     return {"dataset": dataset, "feedback": feedback, "grievances": []}
133 |
134 |
135 | def evaluate_quality_filter(
136 |     raw_feedback: list[dict[str, Any]], feedback: list[FeedbackItem]
137 | ) -> list[dict[str, Any]]:
138 |     seen: set[str] = set()
139 |     rows = []
140 |
141 |     for raw, item in zip(raw_feedback, feedback):
142 |         comment = normalize_text(item.comment or "")
143 |         reason = reject_reason(comment) if comment else "empty"
144 |         fingerprint = f"{{item.categoryId}}:{{comment}}"
145 |         duplicate = bool(comment and fingerprint in seen)
146 |         predicted_useful = bool(comment and not reason and not duplicate)
147 |
148 |         if comment:
149 |             seen.add(fingerprint)
150 |
151 |         expected_useful = bool(raw["expectedUseful"])
152 |         rows.append(
153 |             {
154 |                 "id": item.id,
155 |                 "expectedUseful": expected_useful,
156 |                 "predictedUseful": predicted_useful,
157 |                 "correct": expected_useful == predicted_useful,
158 |                 "reason": "duplicate" if duplicate else reason,
159 |                 "comment": item.comment,
160 |             }
161 |         )
162 |
163 |     return rows
164 |
165 |
166 | def evaluate_sentiment(
167 |     dataset: dict[str, Any],
168 |     useful_feedback: list[FeedbackItem],
169 | ) -> list[dict[str, Any]]:
170 |     feedback_docs = build_feedback_documents(useful_feedback)
171 |     docs = feedback_docs
172 |     predictions = classify_sentiments(docs)
173 |     expected_by_id = {
174 |         item["id"]: item["expectedSentiment"]
175 |         for item in dataset["feedback"]
176 |         if item["expectedUseful"]
177 |     }
178 |
179 |     rows = []
180 |     for doc in docs:
181 |         expected = expected_by_id[doc["id"]]
182 |         predicted = predictions[doc["id"]]
183 |         rows.append(
184 |             {
185 |                 "id": doc["id"],
186 |                 "expected": expected,
187 |                 "predicted": predicted,
188 |                 "correct": expected == predicted,
189 |                 "text": doc["text"],
190 |             }
191 |         )
192 |
193 |     return rows
194 |
195 |
196 | def evaluate_theme_coverage(
197 |     dataset: dict[str, Any], analysis: dict[str, Any]
198 | ) -> list[dict[str, Any]]:
199 |     expected_themes = sorted(
200 |         {
201 |             item["expectedTheme"]
202 |             for item in dataset["feedback"]
203 |             if item["expectedUseful"]
204 |         }
205 |     )
206 |
207 |     rows = []
208 |     theme_text = analysis_text(analysis)
209 |     for theme in expected_themes:
210 |         tokens = theme_tokens(theme)
211 |         covered = any(token in theme_text for token in tokens)
212 |         rows.append(
213 |             {
214 |                 "theme": theme,
215 |                 "tokensChecked": tokens,

```

```

216 |         "covered": covered,
217 |         "correct": covered,
218 |     }
219 | )
220 |
221 | return rows
222 |
223 |
224 | def evaluate_priority_recall(
225 |     dataset: dict[str, Any], analysis: dict[str, Any]
226 | ) -> list[dict[str, Any]]:
227 |     expected_priority_themes = sorted(
228 |         {
229 |             item["expectedTheme"]
230 |             for item in dataset["feedback"]
231 |             if item["expectedUseful"] and item["expectedPriority"] in {"HIGH", "CRITICAL"}
232 |         }
233 |     )
234 |     urgent_text = analysis_text(
235 |         {
236 |             **analysis,
237 |             "themes": [
238 |                 theme
239 |                 for theme in analysis["themes"]
240 |                 if theme["priority"] in {"HIGH", "CRITICAL"}
241 |             ],
242 |         }
243 |     )
244 |
245 |     rows = []
246 |     for theme in expected_priority_themes:
247 |         tokens = theme_tokens(theme)
248 |         recalled = any(token in urgent_text for token in tokens)
249 |         rows.append(
250 |             {
251 |                 "theme": theme,
252 |                 "expectedPriority": "HIGH_OR_CRITICAL",
253 |                 "placedInHighOrCriticalLayer": recalled,
254 |                 "correct": recalled,
255 |             }
256 |         )
257 |
258 |     return rows
259 |
260 |
261 | def theme_tokens(theme: str) -> list[str]:
262 |     tokens = [
263 |         token
264 |         for token in normalize_text(theme).split()
265 |         if len(token) > 3 and token not in THEME_STOPWORDS
266 |     ]
267 |     return tokens or [normalize_text(theme)]
268 |
269 |
270 | def analysis_text(analysis: dict[str, Any]) -> str:
271 |     chunks = [analysis.get("summary", "")]
272 |     for theme in analysis.get("themes", []):
273 |         chunks.extend(
274 |             [
275 |                 str(theme.get("title", "")),
276 |                 str(theme.get("summary", "")),
277 |                 json.dumps(theme.get("evidence", {})),
278 |             ]
279 |         )
280 |     for action in analysis.get("actions", []):
281 |         chunks.append(json.dumps(action))
282 |     raw_json = analysis.get("rawJson", {})
283 |     if isinstance(raw_json, dict):
284 |         chunks.append(json.dumps(raw_json.get("categorySignals", [])))
285 |         chunks.append(json.dumps(raw_json.get("sentimentBreakdown", {})))
286 |     return normalize_text(" ".join(chunks))
287 |
288 |
289 | def accuracy(rows: list[dict[str, Any]]) -> dict[str, Any]:
290 |     total = len(rows)
291 |     correct = sum(1 for row in rows if row["correct"])
292 |     score = correct / total if total else 0
293 |     return {
294 |         "correct": correct,
295 |         "total": total,
296 |         "score": round(score, 4),
297 |         "percentage": round(score * 100, 2),
298 |     }
299 |
300 |
301 | def summary_for_console(metrics: dict[str, Any]) -> dict[str, Any]:
302 |     return {
303 |         "modelMode": metrics["modelMode"],
304 |         "qualityFilterAccuracy": metrics["qualityFilterAccuracy"]["percentage"],
305 |         "sentimentAccuracy": metrics["sentimentAccuracy"]["percentage"],
306 |         "themeCoverage": metrics["themeCoverage"]["percentage"],
307 |         "priorityPlacement": metrics["priorityPlacement"]["percentage"],
308 |         "overallScore": round(metrics["overallScore"] * 100, 2),
309 |         "report": str(REPORT_PATH),
310 |     }
311 |
312 |
313 | def render_report(metrics: dict[str, Any], analysis: dict[str, Any]) -> str:
314 |     quality_misses = [row for row in metrics["rows"] if not row["correct"]]
315 |     sentiment_misses = [row for row in metrics["rows"] if not row["correct"]]
316 |     theme_misses = [row for row in metrics["rows"] if not row["correct"]]
317 |     priority_misses = [row for row in metrics["rows"] if not row["correct"]]
318 |
319 |     return f"""# EduPulse AI Evaluation Report
320 |
321 | Generated: {metrics["generatedAt"]}
322 |
323 | Dataset: {metrics["datasetName"]} ({metrics["datasetVersion"]})
324 |
325 | Model mode: {metrics["modelMode"]}

```

```

326 |
327 | ## Scorecard
328 |
329 | Metric | Score | Meaning |
330 | --- | --- | --- |
331 | Quality / fake-feedback filter | {metrics["qualityFilterAccuracy"]["percentage"]} % | Detects useful vs useless, duplicate, random, repeated feedback |
332 | Sentiment accuracy | {metrics["sentimentAccuracy"]["percentage"]} % | Predicts Positive, Neutral, Negative, Critical |
333 | Theme coverage | {metrics["themeCoverage"]["percentage"]} % | Finds expected college issue themes in output themes/evidence |
334 | Priority placement | {metrics["priorityPlacement"]["percentage"]} % | Places High/Critical expected issues in High/Critical themes/actions |
335 | Overall evaluation score | {round(metrics["overallScore"] * 100, 2)} % | Average of the four proof metrics |
336 |
337 | ## Dataset Size
338 |
339 | - Feedback samples: {metrics["sampleSize"]["feedback"]}
340 | - Legacy safety samples converted to feedback: {metrics["sampleSize"]["legacySafetyFeedbackConverted"]}
341 | - Total labeled samples: {metrics["sampleSize"]["total"]}
342 |
343 | ## Analysis Run Summary
344 |
345 | - AI confidence: {round(metrics["analysisConfidence"] * 100)} %
346 | - Themes generated: {metrics["themeCount"]}
347 | - Action items generated: {metrics["actionCount"]}
348 | - Service summary: {metrics["analysisSummary"]}
349 |
350 | ## Quality Filter Breakdown
351 |
352 | ```json
353 | {json.dumps(metrics["qualityBreakdown"], indent=2)}
354 | ```
355 |
356 | ## Top Generated Themes
357 |
358 | {render_theme_list(analysis["themes"])}
359 |
360 | ## Known Misses
361 |
362 | Quality misses: {render_miss_list(quality_misses, "id")}
363 |
364 | Sentiment misses: {render_sentiment_misses(sentiment_misses)}
365 |
366 | Theme misses: {render_miss_list(theme_misses, "theme")}
367 |
368 | Priority misses: {render_miss_list(priority_misses, "theme")}
369 |
370 | ## How To Re-run
371 |
372 | ```powershell
373 | cd "{ROOT}"
374 | python ai-service/evaluation/evaluate_ai.py
375 | ```
376 | """
377 |
378 |
379 | def render_theme_list(themes: list[dict[str, Any]]) -> str:
380 |     rows = []
381 |     for theme in themes[:8]:
382 |         rows.append(
383 |             f"- **{theme['title']}** - {theme['priority']} / {theme['sentiment']} "
384 |             f"({theme['mentionCount']} mentions)"
385 |         )
386 |     return "\n".join(rows) if rows else "- No themes generated."
387 |
388 |
389 | def render_miss_list(rows: list[dict[str, Any]], key: str) -> str:
390 |     if not rows:
391 |         return "None."
392 |     return ", ".join(str(row[key]) for row in rows)
393 |
394 |
395 | def render_sentiment_misses(rows: list[dict[str, Any]]) -> str:
396 |     if not rows:
397 |         return "None."
398 |     counts = Counter(f"{row['expected']}-{row['predicted']}" for row in rows)
399 |     return ", ".join(f"{label}: {count}" for label, count in counts.items())
400 |
401 |
402 | if __name__ == "__main__":
403 |     main()

```

[edupulse-backend/ai-service/requirements.txt](#)

File size: 156 bytes

```
1 | fastapi==0.115.6
2 | uvicorn[standard]==0.34.0
3 | pydantic==2.10.4
4 | numpy==2.2.1
5 | scikit-learn==1.6.0
6 | sentence-transformers==3.3.1
7 | transformers==4.47.1
8 | torch>=2.5.0
```

edupulse-backend/nest-cli.json

File size: 179 bytes

```
1 | {  
2 |   "$schema": "https://json.schemastore.org/nest-cli",  
3 |   "collection": "@nestjs/schematics",  
4 |   "sourceRoot": "src",  
5 |   "compilerOptions": {  
6 |     "deleteOutDir": true  
7 |   }  
8 | }
```

edupulse-backend/package.json

File size: 3,223 bytes

```
1 | {
2 |   "name": "edupulse-backend",
3 |   "version": "0.0.1",
4 |   "description": "",
5 |   "author": "",
6 |   "private": true,
7 |   "license": "UNLICENSED",
8 |   "scripts": {
9 |     "build": "nest build",
10 |    "format": "prettier --write \"src/**/*.ts\" \"test/**/*.ts\"",
11 |    "start": "nest start",
12 |    "start:dev": "nest start --watch",
13 |    "start:debug": "nest start --debug --watch",
14 |    "start:prod": "node dist/main",
15 |    "prisma:generate": "prisma generate",
16 |    "prisma:migrate": "prisma migrate dev",
17 |    "prisma:studio": "prisma studio",
18 |    "db:seed": "tsx prisma/seed.ts",
19 |    "db:mock-ai": "tsx prisma/mock-ai-seed.ts",
20 |    "db:scale-sharda": "tsx prisma/sharda-scale-seed.ts",
21 |    "db:seed-gl-previous": "tsx prisma/gl-bajaj-previous-sem-seed.ts",
22 |    "db:seed-judge-demo": "tsx prisma/gl-bajaj-judge-demo-seed.ts",
23 |    "db:seed-test-edge": "tsx prisma/test-dataset2-edge-cases-seed.ts",
24 |    "db:seed-small-test-1": "tsx prisma/small-test-dataset-1-seed.ts",
25 |    "test:tenant": "tsx scripts/tenant-isolation-test.ts",
26 |    "ai:golden": "tsx scripts/ai-golden-validation.ts",
27 |    "ai:audit-datasets": "tsx scripts/ai-dataset-audit.ts",
28 |    "ai:evaluate": "python ai-service/evaluation/evaluate_ai.py",
29 |    "lint": "eslint \"{src,apps,libs,test}/**/*.ts\" --fix",
30 |    "test": "jest",
31 |    "test:watch": "jest --watch",
32 |    "test:cov": "jest --coverage",
33 |    "test:debug": "node --inspect-brk -r tsconfig-paths/register -r ts-node/register node_modules/.bin/jest --runInBand",
34 |    "test:e2e": "jest --config ./test/jest-e2e.json"
35 |  },
36 |   "dependencies": {
37 |     "@nestjs/common": "^11.0.1",
38 |     "@nestjs/config": "^4.0.4",
39 |     "@nestjs/core": "^11.0.1",
40 |     "@nestjs/jwt": "^11.0.2",
41 |     "@nestjs/platform-express": "^11.0.1",
42 |     "@nestjs/swagger": "^11.4.4",
43 |     "@prisma/adapters-pg": "^7.8.0",
44 |     "@prisma/client": "^7.8.0",
45 |     "bcryptjs": "^3.0.3",
46 |     "class-transformer": "^0.5.1",
47 |     "class-validator": "^0.15.1",
48 |     "pdfkit": "^0.18.0",
49 |     "pg": "^8.21.0",
50 |     "reflect-metadata": "^0.2.2",
51 |     "rxjs": "^7.8.1",
52 |     "swagger-ui-express": "^5.0.1"
53 |   },
54 |   "devDependencies": {
55 |     "@eslint/eslintrc": "^3.2.0",
56 |     "@eslint/js": "^9.18.0",
57 |     "@nestjs/cli": "^11.0.0",
58 |     "@nestjs/schematics": "^11.0.0",
59 |     "@nestjs/testing": "^11.0.1",
60 |     "@types/express": "^5.0.0",
61 |     "@types/jest": "^30.0.0",
62 |     "@types/node": "^24.0.0",
63 |     "@types/pdfkit": "^0.17.6",
64 |     "@types/pg": "^8.20.0",
65 |     "@types/supertest": "^7.0.0",
66 |     "eslint": "^9.18.0",
67 |     "eslint-config-prettier": "^10.0.1",
68 |     "eslint-plugin-prettier": "^5.2.2",
69 |     "globals": "^17.0.0",
70 |     "jest": "^30.0.0",
71 |     "prettier": "^3.4.2",
72 |     "prisma": "^7.8.0",
73 |     "source-map-support": "^0.5.21",
74 |     "supertest": "^7.0.0",
75 |     "ts-jest": "^29.2.5",
76 |     "ts-loader": "^9.5.2",
77 |     "ts-node": "^10.9.2",
78 |     "tsconfig-paths": "^4.2.0",
79 |     "tsx": "^4.22.4",
80 |     "typescript": "^5.7.3",
81 |     "typescript-eslint": "^8.20.0"
82 |   },
83 |   "jest": {
84 |     "moduleFileExtensions": [
85 |       "js",
86 |       "json",
87 |       "ts"
88 |     ],
89 |     "rootDir": "src",
90 |     "testRegex": ".*\\.spec\\.ts$",
91 |     "transform": {
92 |       "^.+\\.tsx?$": "ts-jest"
93 |     },
94 |     "collectCoverageFrom": [
95 |       "**/*.ts"
96 |     ],
97 |     "coverageDirectory": "../coverage",
98 |     "testEnvironment": "node"
99 |   }
100 | }
```


edupulse-backend/prisma/schema.prisma

File size: 9,341 bytes

```
1 | generator client {
2 |   provider = "prisma-client-js"
3 | }
4 |
5 | datasource db {
6 |   provider = "postgresql"
7 | }
8 |
9 | enum UserRole {
10 |   STUDENT
11 |   ADMIN
12 | }
13 |
14 | enum FeedbackSubmissionStatus {
15 |   DRAFT
16 |   SUBMITTED
17 |   ANALYZED
18 | }
19 |
20 | enum GrievanceSeverity {
21 |   LOW
22 |   MEDIUM
23 |   HIGH
24 |   CRITICAL
25 | }
26 |
27 | enum GrievanceStatus {
28 |   OPEN
29 |   IN_REVIEW
30 |   RESOLVED
31 |   ESCALATED
32 | }
33 |
34 | enum SentimentLabel {
35 |   POSITIVE
36 |   NEUTRAL
37 |   NEGATIVE
38 |   CRITICAL
39 | }
40 |
41 | enum PriorityLevel {
42 |   LOW
43 |   MEDIUM
44 |   HIGH
45 |   CRITICAL
46 | }
47 |
48 | enum ActionStatus {
49 |   TODO
50 |   IN_PROGRESS
51 |   DONE
52 |   BLOCKED
53 | }
54 |
55 | enum AuditAction {
56 |   LOGIN
57 |   FEEDBACK_SUBMITTED
58 |   FEEDBACK_FORM_PUBLISHED
59 |   FEEDBACK_FORM_TURNED_OFF
60 |   GRIEVANCE_SUBMITTED
61 |   GRIEVANCE_VIEWED
62 |   ANALYSIS_RUN
63 |   REPORT_VIEWED
64 |   REPORT_DOWNLOADED
65 |   ACTION_ITEM_CREATED
66 |   ACTION_STATUS_UPDATED
67 | }
68 |
69 | model College {
70 |   id      String      @id @default(cuid())
71 |   name    String
72 |   code    String      @unique
73 |   domain  String?
74 |   logoUrl String?
75 |   isActive Boolean      @default(true)
76 |   departments Department[]
77 |   users   User[]
78 |   terms   AcademicTerm[]
79 |   categories FeedbackCategory[]
80 |   questions FeedbackQuestion[]
81 |   submissions FeedbackSubmission[]
82 |   responses FeedbackResponse[]
83 |   grievances Grievance[]
84 |   reports   AnalysisReport[]
85 |   themes    ThemeInsight[]
86 |   actions   ActionItem[]
87 |   auditLogs AuditLog[]
88 |   createdAt DateTime @default(now())
89 |   updatedAt DateTime @updatedAt
90 |
91 |   @@index([isActive])
92 | }
93 |
94 | model Department {
95 |   id      String      @id @default(cuid())
96 |   collegeId String
97 |   college College      @relation(fields: [collegeId], references: [id])
98 |   name    String
99 |   code    String
100 |   users   User[]
101 |   createdAt DateTime @default(now())
102 |   updatedAt DateTime @updatedAt
103 |
104 |   @@unique([collegeId, code])
105 |   @@unique([collegeId, name])
```

```

106 |   @@index([collegeId])
107 | }
108 |
109 | model User {
110 |   id          String          @id @default(cuid())
111 |   collegeId   String
112 |   college     College         @relation(fields: [collegeId], references: [id])
113 |   name        String
114 |   email       String          @unique
115 |   passwordHash String
116 |   role        UserRole
117 |   departmentId String?
118 |   department  Department?     @relation(fields: [departmentId], references: [id])
119 |   semesterNumber Int?
120 |   feedback    FeedbackSubmission[]
121 |   grievances   Grievance[]    @relation("GrievanceReporter")
122 |   assignedActions ActionItem[] @relation("ActionOwner")
123 |   auditLogs    AuditLog[]
124 |   createdAt    DateTime       @default(now())
125 |   updatedAt    DateTime       @updatedAt
126 |
127 |   @@index([collegeId])
128 |   @@index([role])
129 |   @@index([departmentId])
130 |   @@index([collegeId, role])
131 | }
132 |
133 | model AcademicTerm {
134 |   id          String          @id @default(cuid())
135 |   collegeId   String
136 |   college     College         @relation(fields: [collegeId], references: [id])
137 |   name        String
138 |   startsAt    DateTime?
139 |   endsAt      DateTime?
140 |   isActive    Boolean         @default(false)
141 |   submissions FeedbackSubmission[]
142 |   reports     AnalysisReport[]
143 |   createdAt    DateTime       @default(now())
144 |   updatedAt    DateTime       @updatedAt
145 |
146 |   @@unique([collegeId, name])
147 |   @@index([collegeId])
148 |   @@index([collegeId, isActive])
149 | }
150 |
151 | model FeedbackCategory {
152 |   id          String          @id @default(cuid())
153 |   collegeId   String
154 |   college     College         @relation(fields: [collegeId], references: [id])
155 |   name        String
156 |   description String?
157 |   questions   FeedbackQuestion[]
158 |   responses   FeedbackResponse[]
159 |   themes      ThemeInsight[]
160 |   createdAt    DateTime       @default(now())
161 |   updatedAt    DateTime       @updatedAt
162 |
163 |   @@unique([collegeId, name])
164 |   @@index([collegeId])
165 | }
166 |
167 | model FeedbackQuestion {
168 |   id          String          @id @default(cuid())
169 |   collegeId   String
170 |   college     College         @relation(fields: [collegeId], references: [id])
171 |   categoryId  String
172 |   category    FeedbackCategory @relation(fields: [categoryId], references: [id])
173 |   text        String
174 |   isActive    Boolean         @default(true)
175 |   responses   FeedbackResponse[]
176 |   createdAt    DateTime       @default(now())
177 |   updatedAt    DateTime       @updatedAt
178 |
179 |   @@index([collegeId])
180 |   @@index([categoryId])
181 |   @@index([isActive])
182 |   @@index([collegeId, isActive])
183 | }
184 |
185 | model FeedbackSubmission {
186 |   id          String          @id @default(cuid())
187 |   collegeId   String
188 |   college     College         @relation(fields: [collegeId], references: [id])
189 |   studentId   String
190 |   student     User            @relation(fields: [studentId], references: [id])
191 |   termId      String
192 |   term        AcademicTerm    @relation(fields: [termId], references: [id])
193 |   status      FeedbackSubmissionStatus @default(SUBMITTED)
194 |   responses   FeedbackResponse[]
195 |   createdAt    DateTime       @default(now())
196 |   updatedAt    DateTime       @updatedAt
197 |
198 |   @@unique([studentId, termId])
199 |   @@index([collegeId])
200 |   @@index([studentId])
201 |   @@index([termId])
202 |   @@index([status])
203 |   @@index([collegeId, termId])
204 | }
205 |
206 | model FeedbackResponse {
207 |   id          String          @id @default(cuid())
208 |   collegeId   String
209 |   college     College         @relation(fields: [collegeId], references: [id])
210 |   submissionId String
211 |   submission  FeedbackSubmission @relation(fields: [submissionId], references: [id])
212 |   categoryId  String
213 |   category    FeedbackCategory @relation(fields: [categoryId], references: [id])
214 |   questionId  String
215 |   question    FeedbackQuestion @relation(fields: [questionId], references: [id])

```

```

216 | rating      Int
217 | comment     String?
218 | createdAt   DateTime      @default(now())
219 |
220 | @@index([collegeId])
221 | @@index([submissionId])
222 | @@index([categoryId])
223 | @@index([questionId])
224 | @@index([rating])
225 | @@index([collegeId, categoryId])
226 | @@index([collegeId, rating])
227 | }
228 |
229 | model Grievance {
230 |   id      String      @id @default(cuid())
231 |   collegeId String
232 |   college College      @relation(fields: [collegeId], references: [id])
233 |   reporterId String?
234 |   reporter User?        @relation("GrievanceReporter", fields: [reporterId], references: [id])
235 |   category String
236 |   title    String
237 |   description String
238 |   severity GrievanceSeverity
239 |   status    GrievanceStatus @default(OPEN)
240 |   isAnonymous Boolean       @default(false)
241 |   safetyFlag Boolean        @default(false)
242 |   createdAt DateTime        @default(now())
243 |   updatedAt DateTime        @updatedAt
244 |
245 |   @@index([collegeId])
246 |   @@index([reporterId])
247 |   @@index([severity])
248 |   @@index([status])
249 |   @@index([safetyFlag])
250 |   @@index([collegeId, status])
251 |   @@index([collegeId, severity])
252 | }
253 |
254 | model AnalysisReport {
255 |   id      String      @id @default(cuid())
256 |   collegeId String
257 |   college College      @relation(fields: [collegeId], references: [id])
258 |   termId   String
259 |   term     AcademicTerm @relation(fields: [termId], references: [id])
260 |   title    String
261 |   summary  String
262 |   confidence Float
263 |   inputCount Int
264 |   lowSampleFlag Boolean @default(false)
265 |   duplicateCount Int    @default(0)
266 |   generatedBy String?
267 |   rawJson   Json?
268 |   themes    ThemeInsight[]
269 |   actions   ActionItem[]
270 |   createdAt DateTime    @default(now())
271 |   updatedAt DateTime    @updatedAt
272 |
273 |   @@index([collegeId])
274 |   @@index([termId])
275 |   @@index([confidence])
276 |   @@index([createdAt])
277 |   @@index([collegeId, termId])
278 | }
279 |
280 | model ThemeInsight {
281 |   id      String      @id @default(cuid())
282 |   collegeId String
283 |   college College      @relation(fields: [collegeId], references: [id])
284 |   reportId String
285 |   report  AnalysisReport @relation(fields: [reportId], references: [id])
286 |   categoryId String?
287 |   category FeedbackCategory? @relation(fields: [categoryId], references: [id])
288 |   title    String
289 |   summary  String
290 |   sentiment SentimentLabel
291 |   priority PriorityLevel
292 |   mentionCount Int
293 |   evidence  Json?
294 |   createdAt DateTime    @default(now())
295 |
296 |   @@index([collegeId])
297 |   @@index([reportId])
298 |   @@index([categoryId])
299 |   @@index([sentiment])
300 |   @@index([priority])
301 |   @@index([collegeId, priority])
302 | }
303 |
304 | model ActionItem {
305 |   id      String      @id @default(cuid())
306 |   collegeId String
307 |   college College      @relation(fields: [collegeId], references: [id])
308 |   reportId String
309 |   report  AnalysisReport @relation(fields: [reportId], references: [id])
310 |   title    String
311 |   ownerId  String?
312 |   owner    User?        @relation("ActionOwner", fields: [ownerId], references: [id])
313 |   priority PriorityLevel
314 |   status   ActionStatus @default(TODO)
315 |   timeline String
316 |   kpi      String?
317 |   createdAt DateTime    @default(now())
318 |   updatedAt DateTime    @updatedAt
319 |
320 |   @@index([collegeId])
321 |   @@index([reportId])
322 |   @@index([ownerId])
323 |   @@index([priority])
324 |   @@index([status])
325 |   @@index([collegeId, status])

```

```

326 | }
327 |
328 | model AuditLog {
329 |     id String @id @default(cuid())
330 |     collegeId String
331 |     college College @relation(fields: [collegeId], references: [id])
332 |     actorId String?
333 |     actor User? @relation(fields: [actorId], references: [id])
334 |     action AuditAction
335 |     entity String?
336 |     entityId String?
337 |     metadata Json?
338 |     createdAt DateTime @default(now())
339 |
340 |     @@index([collegeId])
341 |     @@index([actorId])
342 |     @@index([action])
343 |     @@index([createdAt])
344 |     @@index([collegeId, action])
345 | }

```

edupulse-backend/scripts/ai-dataset-audit.ts

File size: 9,126 bytes

```
1 | import 'dotenv/config';
2 | import { PrismaPg } from '@prisma/adapters-pg';
3 | import { PrismaClient } from '@prisma/client';
4 | import { hasTitle, rejectedCount, themeTitles, urgentTitles } from './ai-result-utils';
5 |
6 | type AnalysisResult = {
7 |   inputCount: number;
8 |   duplicateCount: number;
9 |   rawJson?: {
10 |     qualityChecks?: Record<string, unknown>;
11 |     geminiReasoning?: {
12 |       ultraConcerningIssues?: Array<Record<string, unknown>>;
13 |     };
14 |   };
15 |   themes: Array<{
16 |     title: string;
17 |     mentionCount: number;
18 |     priority: string;
19 |   }>;
20 | };
21 |
22 | type FeedbackPayloadRow = {
23 |   id: string;
24 |   categoryId: string;
25 |   categoryName: string;
26 |   questionId: string;
27 |   questionText: string;
28 |   rating: number;
29 |   comment: string | null;
30 |   departmentCode: string | null;
31 |   semesterNumber: number | null;
32 |   weight?: number;
33 | };
34 |
35 | type DatasetExpectation = {
36 |   termName: string;
37 |   minResponses: number;
38 |   urgentIncludes: string[];
39 |   themeIncludes: string[];
40 |   minRejected?: number;
41 |   minDuplicates?: number;
42 | };
43 |
44 | const AI_BASE = process.env.AI_SERVICE_URL ?? 'http://127.0.0.1:8001';
45 | const connectionString = process.env.DATABASE_URL;
46 |
47 | if (!connectionString) {
48 |   throw new Error('DATABASE_URL is required for dataset audit');
49 | }
50 |
51 | const prisma = new PrismaClient({
52 |   adapter: new PrismaPg({ connectionString }),
53 | });
54 |
55 | const expectations: DatasetExpectation[] = [
56 |   {
57 |     termName: 'small_test_dataset_1',
58 |     minResponses: 10,
59 |     urgentIncludes: ['ragging', 'water contamination', 'harassment'],
60 |     themeIncludes: ['mess hygiene'],
61 |     minRejected: 2,
62 |   },
63 |   {
64 |     termName: 'test_dataset2_edge_cases',
65 |     minResponses: 100,
66 |     urgentIncludes: [
67 |       'ragging',
68 |       'harassment',
69 |       'corruption',
70 |       'weapon',
71 |       'animal',
72 |       'water contamination',
73 |       'electric',
74 |       'building',
75 |     ],
76 |     themeIncludes: [
77 |       'wi-fi',
78 |       'mess hygiene',
79 |       'placement',
80 |       'subject knowledge',
81 |       'sports',
82 |       'hostel',
83 |       'lab',
84 |       'fest',
85 |       'transport',
86 |       'library',
87 |       'administration',
88 |     ],
89 |     minRejected: 2,
90 |   },
91 |   {
92 |     termName: 'GL Bajaj 8 Category Judge Demo',
93 |     minResponses: 800,
94 |     urgentIncludes: ['ragging', 'harassment', 'corruption'],
95 |     themeIncludes: [
96 |       'faculty behaviour',
97 |       'sports item',
98 |       'subject knowledge',
99 |       'marks partiality',
100 |       'placement',
101 |       'fest',
102 |     ],
103 |   },
104 |   {
105 |     termName: 'gl_bajaj_previous_sem_data',
```

```

106 |     minResponses: 10000,
107 |     urgentIncludes: ['ragging', 'harassment', 'weapon', 'animal'],
108 |     themeIncludes: ['projector', 'marks partiality'],
109 | },
110 | {
111 |   termName: 'Sharda Scale Test Semester 2026',
112 |   minResponses: 100000,
113 |   urgentIncludes: ['harassment'],
114 |   themeIncludes: ['mess hygiene', 'sports item', 'wi-fi', 'projector', 'washroom'],
115 |   minRejected: 1000,
116 |   minDuplicates: 50000,
117 | },
118 | ];
119 |
120 | async function analyze(payload: unknown): Promise<AnalysisResult> {
121 |   let response: Response | null = null;
122 |   let lastError = '';
123 |
124 |   for (let attempt = 1; attempt <= 2; attempt += 1) {
125 |     try {
126 |       response = await fetch(`${AI_BASE.replace(/\/$/, '')}/analyze`, {
127 |         method: 'POST',
128 |         headers: { 'Content-Type': 'application/json', Connection: 'close' },
129 |         body: JSON.stringify(payload),
130 |       });
131 |       break;
132 |     } catch (error) {
133 |       const cause =
134 |         error instanceof Error && 'cause' in error
135 |           ? `; cause=${String((error as Error & { cause?: unknown }).cause)}`
136 |           : '';
137 |       lastError = `${
138 |         error instanceof Error ? error.message : String(error)
139 |       }${cause}`;
140 |       if (attempt < 2) {
141 |         console.error(`AI request failed (${lastError}); retrying once`);
142 |       }
143 |     }
144 |   }
145 |
146 |   if (!response) {
147 |     throw new Error(`AI service request failed: ${lastError} || 'unknown error'`);
148 |   }
149 |
150 |   if (!response.ok) {
151 |     throw new Error(`AI service returned ${response.status}`);
152 |   }
153 |
154 |   return (await response.json()) as AnalysisResult;
155 | }
156 |
157 | async function buildPayload(termName: string) {
158 |   const term = await prisma.academicTerm.findFirst({
159 |     where: { name: termName },
160 |     include: { college: true },
161 |   });
162 |
163 |   if (!term) {
164 |     throw new Error(`Dataset term not found: ${termName}`);
165 |   }
166 |
167 |   const responses = await prisma.feedbackResponse.findMany({
168 |     where: {
169 |       collegeId: term.collegeId,
170 |       submission: { termId: term.id },
171 |     },
172 |     include: {
173 |       category: true,
174 |       question: true,
175 |       submission: {
176 |         include: {
177 |           student: {
178 |             include: {
179 |               department: true,
180 |             },
181 |           },
182 |         },
183 |       },
184 |     },
185 |   });
186 |
187 |   const feedbackRows = responses.map((response) => ({
188 |     id: response.id,
189 |     categoryId: response.categoryId,
190 |     categoryName: response.category.name,
191 |     questionId: response.questionId,
192 |     questionText: response.question.text,
193 |     rating: response.rating,
194 |     comment: response.comment?.trim() ? response.comment.trim() : null,
195 |     departmentCode: response.submission.student.department?.code ?? null,
196 |     semesterNumber: response.submission.student.semesterNumber ?? null,
197 |   }));
198 |   const compactedFeedbackRows = compactFeedbackRows(feedbackRows);
199 |
200 |   return {
201 |     term: {
202 |       id: term.id,
203 |       name: term.name,
204 |     },
205 |     college: term.college.code,
206 |     responseRows: responses.length,
207 |     compactedRows: compactedFeedbackRows.length,
208 |     payload: {
209 |       term: {
210 |         id: term.id,
211 |         name: term.name,
212 |       },
213 |       feedback: compactedFeedbackRows,
214 |       grievances: [],
215 |     },

```

```

216 | };
217 | }
218 |
219 | function compactFeedbackRows(rows: FeedbackPayloadRow[]) {
220 |     const compacted = new Map<string, FeedbackPayloadRow>();
221 |
222 |     for (const row of rows) {
223 |         const key = [
224 |             row.categoryId,
225 |             row.questionId,
226 |             row.rating,
227 |             normalizeComment(row.comment),
228 |             row.departmentCode ?? '',
229 |             row.semesterNumber ?? '',
230 |         ].join(':');
231 |         const existing = compacted.get(key);
232 |
233 |         if (existing) {
234 |             existing.weight = (existing.weight ?? 1) + 1;
235 |             continue;
236 |         }
237 |
238 |         compacted.set(key, { ...row, weight: 1 });
239 |     }
240 |
241 |     return [...compacted.values()];
242 | }
243 |
244 | function normalizeComment(comment: string | null) {
245 |     return (comment ?? '').toLowerCase().replace(/\s+/g, ' ').trim();
246 | }
247 |
248 | async function runDataset(expectation: DatasetExpectation) {
249 |     const started = Date.now();
250 |     console.error(`Auditing dataset: ${expectation.termName}`);
251 |     const data = await buildPayload(expectation.termName);
252 |     console.error(
253 |         `Built payload for ${expectation.termName}: ${data.responseRows} feedback rows compacted to ${data.compactedRows}`,
254 |     );
255 |     const result = await analyze(data.payload);
256 |     const urgent = urgentTitles(result);
257 |     const themes = themeTitles(result);
258 |     const checks = [
259 |         {
260 |             name: 'response volume',
261 |             expected: `>= ${expectation.minResponses}`,
262 |             actual: `${result.inputCount}`,
263 |             passed: result.inputCount >= expectation.minResponses,
264 |         },
265 |         ...expectation.urgentIncludes.map((fragment) => ({
266 |             name: `urgent includes ${fragment}`,
267 |             expected: fragment,
268 |             actual: urgent.join(', ') || 'none',
269 |             passed: hasTitle(urgent, fragment),
270 |         })),
271 |         ...expectation.themeIncludes.map((fragment) => ({
272 |             name: `theme includes ${fragment}`,
273 |             expected: fragment,
274 |             actual: themes.slice(0, 16).join(', ') || 'none',
275 |             passed: hasTitle(themes, fragment),
276 |         })),
277 |     ];
278 |
279 |     if (expectation.minRejected !== undefined) {
280 |         checks.push({
281 |             name: 'rejected low-quality comments',
282 |             expected: `>= ${expectation.minRejected}`,
283 |             actual: `${rejectedCount(result)}`,
284 |             passed: rejectedCount(result) >= expectation.minRejected,
285 |         });
286 |     }
287 |
288 |     if (expectation.minDuplicates !== undefined) {
289 |         checks.push({
290 |             name: 'duplicate grouping',
291 |             expected: `>= ${expectation.minDuplicates}`,
292 |             actual: `${result.duplicateCount}`,
293 |             passed: result.duplicateCount >= expectation.minDuplicates,
294 |         });
295 |     }
296 |
297 |     return {
298 |         termName: expectation.termName,
299 |         college: data.college,
300 |         dbResponseRows: data.responseRows,
301 |         aiInputCount: result.inputCount,
302 |         rejectedCount: rejectedCount(result),
303 |         duplicateCount: result.duplicateCount,
304 |         urgentIssues: urgent,
305 |         topThemes: themes.slice(0, 16),
306 |         processingMs: Date.now() - started,
307 |         checks,
308 |         result: checks.every((check) => check.passed) ? 'PASS' : 'REVIEW',
309 |     };
310 | }
311 |
312 | async function main() {
313 |     try {
314 |         const results: Array<Awaited<ReturnType<typeof runDataset>>> = [];
315 |
316 |         for (const expectation of expectations) {
317 |             results.push(await runDataset(expectation));
318 |         }
319 |
320 |         const failed = results.filter((item) => item.result !== 'PASS');
321 |         console.log(
322 |             JSON.stringify(
323 |                 {
324 |                     aiBase: AI_BASE,
325 |                     result: failed.length ? 'REVIEW' : 'PASS',

```

```

326 |         datasets: results,
327 |     },
328 |     null,
329 |     2,
330 | ),
331 | );
332 |
333 | if (failed.length) {
334 |     process.exit(1);
335 | }
336 | } finally {
337 |     await prisma.$disconnect();
338 | }
339 | }
340 |
341 | main().catch(async (error) => {
342 |     await prisma.$disconnect();
343 |     console.error(
344 |         JSON.stringify(
345 |             {
346 |                 aiBase: AI_BASE,
347 |                 result: 'FAIL',
348 |                 error: error instanceof Error ? error.message : String(error),
349 |             },
350 |             null,
351 |             2,
352 |         ),
353 |     );
354 |     process.exit(1);
355 | });

```


edupulse-backend/scripts/ai-golden-validation.ts

File size: 15,671 bytes

```
1 | import {
2 |   duplicateCount,
3 |   hasTitle,
4 |   rejectedCount,
5 |   themeTitles,
6 |   urgentTitles as ultraTitles,
7 | } from './ai-result-utils';
8 |
9 | type FeedbackInput = {
10 |   id: string;
11 |   categoryId: string;
12 |   categoryName: string;
13 |   questionId: string;
14 |   questionText: string;
15 |   rating: number;
16 |   comment: string | null;
17 |   departmentCode: string | null;
18 |   semesterNumber: number | null;
19 | };
20 |
21 | type AnalysisPayload = {
22 |   term: {
23 |     id: string;
24 |     name: string;
25 |   };
26 |   feedback: FeedbackInput[];
27 |   grievances: [];
28 | };
29 |
30 | type AnalysisTheme = {
31 |   title: string;
32 |   mentionCount: number;
33 |   priority: string;
34 |   sentiment: string;
35 |   evidence?: Record<string, unknown>;
36 | };
37 |
38 | type AnalysisResult = {
39 |   inputCount: number;
40 |   duplicateCount: number;
41 |   confidence: number;
42 |   lowSampleFlag: boolean;
43 |   rawJson?: {
44 |     qualityChecks?: Record<string, unknown>;
45 |     geminiReasoning?: {
46 |       ultraConcerningIssues?: Array<Record<string, unknown>>;
47 |     };
48 |     sentimentBreakdown?: unknown;
49 |     engineBenchmark?: Record<string, unknown>;
50 |   };
51 |   themes: AnalysisTheme[];
52 | };
53 |
54 | type GoldenCase = {
55 |   name: string;
56 |   payload: AnalysisPayload;
57 |   expectations: Array<{
58 |     name: string;
59 |     check: (result: AnalysisResult) => boolean;
60 |     actual: (result: AnalysisResult) => string;
61 |     expected: string;
62 |   }>;
63 | };
64 |
65 | const AI_BASE = process.env.AI_SERVICE_URL ?? 'http://127.0.0.1:8001';
66 |
67 | function feedback(
68 |   index: number,
69 |   categoryName: string,
70 |   comment: string,
71 |   options: Partial<FeedbackInput> = {},
72 | ): FeedbackInput {
73 |   const slug = categoryName.toLowerCase().replace(/^[a-z0-9]+/g, '-');
74 |   return {
75 |     id: options.id ?? `f-${index}`,
76 |     categoryId: options.categoryId ?? `cat-${slug}`,
77 |     categoryName,
78 |     questionId: options.questionId ?? `q-${slug}`,
79 |     questionText: options.questionText ?? `How satisfied are you with ${categoryName}?`,
80 |     rating: options.rating ?? 1,
81 |     comment,
82 |     departmentCode: options.departmentCode ?? ['CSE', 'ECE', 'IT', 'ME', 'CE'][index % 5],
83 |     semesterNumber: options.semesterNumber ?? ((index % 8) + 1),
84 |   };
85 | }
86 |
87 | function payload(name: string, rows: FeedbackInput[]): AnalysisPayload {
88 |   return {
89 |     term: {
90 |       id: name.toLowerCase().replace(/^[a-z0-9]+/g, '-'),
91 |       name,
92 |     },
93 |     feedback: rows,
94 |     grievances: [],
95 |   };
96 | }
97 |
98 | async function analyze(testPayload: AnalysisPayload): Promise<AnalysisResult> {
99 |   const response = await fetch(`${AI_BASE.replace(/\/$/, '')}/analyze`, {
100 |     method: 'POST',
101 |     headers: { 'Content-Type': 'application/json' },
102 |     body: JSON.stringify(testPayload),
103 |   });
104 |
105 |   if (!response.ok) {
```

```

106 |     throw new Error(`AI service returned ${response.status} for ${testPayload.term.name}`);
107 | }
108 |
109 | return (await response.json()) as AnalysisResult;
110 | }
111 |
112 | function buildSmallSafetyNoiseCase(): GoldenCase {
113 |     const rows = [
114 |         feedback(1, 'Safety', 'Dev Pratap Rai says Gaurav Sharma ECE 4th year ragged him near hostel stairs.', {
115 |             departmentCode: 'ECE',
116 |             semesterNumber: 1,
117 |         }),
118 |         feedback(2, 'Infrastructure', 'Water pH level contamination is reported in AB3 building near the CS DS drinking water point.', {
119 |             departmentCode: 'CSDS',
120 |             semesterNumber: 3,
121 |         }),
122 |         feedback(3, 'Faculty', 'Raghav sir from CS AI faculty did harassment and physical touch during doubt discussion.', {
123 |             departmentCode: 'CSAI',
124 |             semesterNumber: 5,
125 |         }),
126 |         feedback(4, 'Mess', 'First year hostel mess has insects near the serving counter and tables are not cleaned.', {
127 |             departmentCode: 'ME',
128 |             semesterNumber: 2,
129 |         }),
130 |         feedback(5, 'Mess', 'First year hostel mess has insects near the serving counter and tables are not cleaned.', {
131 |             departmentCode: 'ME',
132 |             semesterNumber: 2,
133 |         }),
134 |         feedback(6, 'Academics', 'ghhvgjhvb ajhdvbs qwerty', { rating: 2 }),
135 |         feedback(7, 'Canteen', 'movie pizza weather phone battery shopping traffic', { rating: 2 }),
136 |     ];
137 |
138 |     return {
139 |         name: 'golden_small_safety_noise',
140 |         payload: payload('golden_small_safety_noise', rows),
141 |         expectations: [
142 |             {
143 |                 name: 'Reject fake/out-of-context comments',
144 |                 expected: '>= 2 rejected comments',
145 |                 actual: (result) => `${rejectedCount(result)} rejected`,
146 |                 check: (result) => rejectedCount(result) >= 2,
147 |             },
148 |             {
149 |                 name: 'Group duplicate mess comment',
150 |                 expected: '>= 1 duplicate grouped',
151 |                 actual: (result) => `${duplicateCount(result)} duplicates`,
152 |                 check: (result) => duplicateCount(result) >= 1,
153 |             },
154 |             {
155 |                 name: 'Escalate ragging/water/harassment',
156 |                 expected: 'ragging, water contamination and harassment in urgent layer',
157 |                 actual: (result) => ultraTitles(result).join(', ') || 'none',
158 |                 check: (result) => {
159 |                     const titles = ultraTitles(result);
160 |                     return (
161 |                         hasTitle(titles, 'ragging') &&
162 |                         hasTitle(titles, 'water contamination') &&
163 |                         hasTitle(titles, 'harassment')
164 |                     );
165 |                 },
166 |             },
167 |             {
168 |                 name: 'Keep mess as normal AI finding',
169 |                 expected: 'mess issue visible outside urgent layer',
170 |                 actual: (result) => themeTitles(result).join(', ') || 'none',
171 |                 check: (result) => hasTitle(themeTitles(result), 'mess hygiene'),
172 |             },
173 |         ],
174 |     };
175 | }
176 |
177 | function buildEdgeTaxonomyCase(): GoldenCase {
178 |     const comments = [
179 |         ['Academics', 'Exam schedule has back to back papers and students need a better date sheet gap.'],
180 |         ['Faculty', 'Faculty is absent from morning lectures and several classes start late.'],
181 |         ['Faculty', 'Faculty not attending classes regularly and lectures are missed.'],
182 |         ['Faculty', 'Practical marks partiality is happening and ECE students are worried about unfair internal marks.'],
183 |         ['Mess', 'Hostel mess hygiene is poor and insects were seen near the serving plates.'],
184 |         ['Canteen', 'Canteen queue and billing counter delay is too much during lunch break.'],
185 |         ['Safety', 'First year students reported ragging by seniors near hostel gate.'],
186 |         ['Safety', 'Student reported harassment by a faculty member during lab discussion.'],
187 |         ['Safety', 'Fest money collection was not transparent and students mentioned corruption.'],
188 |         ['Safety', 'A gun firing sound was reported near the parking area and students felt unsafe.'],
189 |         ['Safety', 'A snake was seen near the playground side entry.'],
190 |         ['Safety', 'Contaminated water is coming from the AB3 drinking point.'],
191 |         ['Safety', 'Electric shock risk exists near the open wire beside lab stairs.'],
192 |         ['Safety', 'Ceiling plaster is falling in old classroom and can injure students.'],
193 |         ['Placements', 'ECE students were not allowed to sit in one CS placement drive despite matching skills.'],
194 |         ['Extracurricular', 'Fest was very short and event slots ended before many students could participate.'],
195 |         ['Sports', 'Sports items like badminton rackets and footballs are not available in the sports room.'],
196 |         ['Infrastructure', 'AB2 room 307 projector is broken and classes are affected.'],
197 |         ['Infrastructure', 'AC not working in seminar room and ventilation is poor.'],
198 |         ['Hostel', 'Hostel room maintenance requests are delayed for weeks.'],
199 |         ['Library', 'Library seating is not enough during exam weeks and reading room AC is not working.'],
200 |         ['Labs', 'Lab equipment and practical components are not available during lab hours.'],
201 |         ['Wi-Fi / Internet', 'Wi-Fi is slow in the lab area during project hours.'],
202 |         ['Transport', 'College bus timing is unreliable and the morning pickup point is overcrowded.'],
203 |         ['Administration', 'Scholarship form support is delayed and fee reimbursement documents are not explained clearly.'],
204 |         ['Academics', 'good good good good good good'],
205 |         ['Canteen', 'xxxxx yyyyy zzzzz'],
206 |     ];
207 |
208 |     const rows = comments.map(([category, comment], index) =>
209 |         feedback(index + 1, category, comment, {
210 |             rating: index >= comments.length - 2 ? 2 : 1,
211 |             departmentCode: ['CSE', 'ECE', 'IT', 'ME', 'CE'][index % 5],
212 |             semesterNumber: (index % 8) + 1,
213 |         })
214 |     );
215 | }

```

```

216 | return {
217 |   name: 'golden_15_category_edge_taxonomy',
218 |   payload: payload('golden_15_category_edge_taxonomy', rows),
219 |   expectations: [
220 |     {
221 |       name: 'Escalate only serious concerns',
222 |       expected: 'ragging, harassment, corruption, weapon, animal, water, electric, building',
223 |       actual: (result) => ultraTitles(result).join(', ') || 'none',
224 |       check: (result) => {
225 |         const titles = ultraTitles(result);
226 |         return [
227 |           'ragging',
228 |           'harassment',
229 |           'corruption',
230 |           'weapon',
231 |           'animal',
232 |           'water contamination',
233 |           'electric',
234 |           'building',
235 |         ].every((fragment) => hasTitle(titles, fragment));
236 |       },
237 |     },
238 |     {
239 |       name: 'Separate faculty root causes',
240 |       expected: 'attendance and marks partiality are separate findings',
241 |       actual: (result) => themeTitles(result).join(', ') || 'none',
242 |       check: (result) => {
243 |         const titles = themeTitles(result);
244 |         return hasTitle(titles, 'attendance') && hasTitle(titles, 'marks partiality');
245 |       },
246 |     },
247 |     {
248 |       name: 'Detect expanded taxonomy buckets',
249 |       expected: 'transport, library, scholarship, exam, projector, AC, lab, sports',
250 |       actual: (result) => themeTitles(result).join(', ') || 'none',
251 |       check: (result) => {
252 |         const titles = themeTitles(result);
253 |         return [
254 |           'transport',
255 |           'library',
256 |           'scholarship',
257 |           'exam',
258 |           'projector',
259 |           'ac',
260 |           'lab',
261 |           'sports',
262 |         ].every((fragment) => hasTitle(titles, fragment));
263 |       },
264 |     },
265 |     {
266 |       name: 'Fake edge comments rejected',
267 |       expected: '>= 2 rejected comments',
268 |       actual: (result) => `${rejectedCount(result)} rejected`,
269 |       check: (result) => rejectedCount(result) >= 2,
270 |     },
271 |     {
272 |       name: 'Urgent issues removed from AI findings',
273 |       expected: 'ragging/harassment/snake/gun not repeated as normal issue cards',
274 |       actual: (result) => themeTitles(result).join(', ') || 'none',
275 |       check: (result) => {
276 |         const titles = themeTitles(result);
277 |         return ![ 'ragging', 'harassment', 'snake', 'weapon', 'gun' ].some((fragment) =>
278 |           hasTitle(titles, fragment),
279 |         );
280 |       },
281 |     },
282 |   ],
283 | };
284 | }
285 |
286 | function buildLargeScaleCase(): GoldenCase {
287 |   const patterns = [
288 |     ['Mess', 'Second year hostel mess hygiene is poor and food gets over before mess time.'],
289 |     ['Wi-Fi / Internet', 'Wi-Fi is slow in the lab area during project hours.'],
290 |     ['Faculty', 'Faculty communication is unclear and doubts are not solved after class.'],
291 |     ['Infrastructure', 'AB2 room 307 projector is broken and classes are affected.'],
292 |     ['Sports', 'Sports items like badminton rackets are not available in the sports room.'],
293 |     ['Transport', 'College bus timing is unreliable and the pickup point is overcrowded.'],
294 |     ['Library', 'Library seating is not enough during exam weeks.'],
295 |     ['Administration', 'Scholarship form support is delayed at the office.'],
296 |   ];
297 |   const rows: FeedbackInput[] = [];
298 |
299 |   for (let index = 0; index < 10000; index += 1) {
300 |     const [category, comment] = patterns[index % patterns.length];
301 |     rows.push(
302 |       feedback(index + 1, category, comment, {
303 |         rating: index % 11 === 0 ? 2 : 1,
304 |         departmentCode: ['CSE', 'ECE', 'IT', 'ME', 'CE'][index % 5],
305 |         semesterNumber: (index % 8) + 1,
306 |       })
307 |     );
308 |   }
309 |
310 |   for (let index = 0; index < 300; index += 1) {
311 |     rows.push(feedback(11000 + index, 'Academics', 'movie pizza weather phone battery shopping traffic', { rating: 2 }));
312 |   }
313 |
314 |   rows.push(
315 |     feedback(12001, 'Safety', 'A student reported harassment by faculty during lab doubt discussion.', {
316 |       departmentCode: 'ECE',
317 |       semesterNumber: 5,
318 |     }),
319 |     feedback(12002, 'Safety', 'First year students reported ragging by seniors near hostel gate.', {
320 |       departmentCode: 'CSE',
321 |       semesterNumber: 1,
322 |     }),
323 |     feedback(12003, 'Infrastructure', 'Contaminated water is coming from AB3 drinking point.', {
324 |       departmentCode: 'CSDS',
325 |       semesterNumber: 3,

```

```

326 |     }},
327 | );
328 |
329 | return {
330 |   name: 'golden_large_10k_scale',
331 |   payload: payload('golden_large_10k_scale', rows),
332 |   expectations: [
333 |     {
334 |       name: 'Scale input accepted',
335 |       expected: '>= 10,000 responses analyzed',
336 |       actual: (result) => `${result.inputCount} responses`,
337 |       check: (result) => result.inputCount >= 10000,
338 |     },
339 |     {
340 |       name: 'Mass duplicate grouping works',
341 |       expected: '>= 9000 duplicates grouped',
342 |       actual: (result) => `${duplicateCount(result)} duplicates`,
343 |       check: (result) => duplicateCount(result) >= 9000,
344 |     },
345 |     {
346 |       name: 'Large fake batch rejected',
347 |       expected: '>= 300 fake comments rejected',
348 |       actual: (result) => `${rejectedCount(result)} rejected`,
349 |       check: (result) => rejectedCount(result) >= 300,
350 |     },
351 |     {
352 |       name: 'Large visible issues remain meaningful',
353 |       expected: 'mess, wifi, faculty, projector, transport, library, scholarship',
354 |       actual: (result) => themeTitles(result).slice(0, 12).join(', ') || 'none',
355 |       check: (result) => {
356 |         const titles = themeTitles(result);
357 |         return ['mess', 'wi-fi', 'faculty', 'projector', 'transport', 'library', 'scholarship'].every(
358 |           (fragment) => hasTitle(titles, fragment),
359 |         );
360 |       },
361 |     },
362 |     {
363 |       name: 'Rare serious risks still escalated in scale data',
364 |       expected: 'ragging, harassment and water contamination urgent',
365 |       actual: (result) => ultraTitles(result).join(', ') || 'none',
366 |       check: (result) => {
367 |         const titles = ultraTitles(result);
368 |         return (
369 |           hasTitle(titles, 'ragging') &&
370 |           hasTitle(titles, 'harassment') &&
371 |           hasTitle(titles, 'water contamination')
372 |         );
373 |       },
374 |     },
375 |   ],
376 | };
377 | }
378 |
379 | async function runCase(test: GoldenCase) {
380 |   const started = Date.now();
381 |   const result = await analyze(test.payload);
382 |   const checks = test.expectations.map((expectation) => {
383 |     const passed = expectation.check(result);
384 |     return {
385 |       name: expectation.name,
386 |       expected: expectation.expected,
387 |       actual: expectation.actual(result),
388 |       passed,
389 |     };
390 |   });
391 |
392 |   return {
393 |     name: test.name,
394 |     inputCount: result.inputCount,
395 |     confidence: result.confidence,
396 |     lowSampleFlag: result.lowSampleFlag,
397 |     duplicateCount: duplicateCount(result),
398 |     rejectedCount: rejectedCount(result),
399 |     urgentIssues: ultraTitles(result),
400 |     topThemes: themeTitles(result).slice(0, 12),
401 |     processingMs: Date.now() - started,
402 |     checks,
403 |     result: checks.every((check) => check.passed) ? 'PASS' : 'REVIEW',
404 |   };
405 | }
406 |
407 | async function main() {
408 |   const cases = [
409 |     buildSmallSafetyNoiseCase(),
410 |     buildEdgeTaxonomyCase(),
411 |     buildLargeScaleCase(),
412 |   ];
413 |   const results: Array<Awaited<ReturnType<typeof runCase>>> = [];
414 |
415 |   for (const test of cases) {
416 |     results.push(await runCase(test));
417 |   }
418 |
419 |   const failed = results.filter((result) => result.result !== 'PASS');
420 |   console.log(
421 |     JSON.stringify(
422 |       {
423 |         aiBase: AI_BASE,
424 |         result: failed.length ? 'REVIEW' : 'PASS',
425 |         suites: results,
426 |       },
427 |       null,
428 |       2,
429 |     ),
430 |   );
431 |
432 |   if (failed.length) {
433 |     process.exit(1);
434 |   }
435 | }

```

```
436 |  
437 | main().catch((error) => {  
438 |   console.error(  
439 |     JSON.stringify(  
440 |       {  
441 |         aiBase: AI_BASE,  
442 |         result: 'FAIL',  
443 |         error: error instanceof Error ? error.message : String(error),  
444 |       },  
445 |       null,  
446 |       2,  
447 |     ),  
448 |   );  
449 |   process.exit(1);  
450 | });
```

edupulse-backend/scripts/ai-result-utils.ts

File size: 1,064 bytes

```
1 | export type AiResultLike = {
2 |   duplicateCount?: number;
3 |   rawJson?: {
4 |     qualityChecks?: Record<string, unknown>;
5 |     geminiReasoning?: {
6 |       ultraConcerningIssues?: Array<Record<string, unknown>>;
7 |     };
8 |   };
9 |   themes: Array<{
10 |     title: string;
11 |   }>;
12 | };
13 |
14 | export function quality(result: AiResultLike) {
15 |   return result.rawJson?.qualityChecks ?? {};
16 | }
17 |
18 | export function hasTitle(titles: string[], fragment: string) {
19 |   return titles.some((title) => title.includes(fragment.toLowerCase()));
20 | }
21 |
22 | export function urgentTitles(result: AiResultLike) {
23 |   return (
24 |     result.rawJson?.geminiReasoning?.ultraConcerningIssues ?? []
25 |   ).map((item) => String(item.title ?? '').toLowerCase());
26 | }
27 |
28 | export function themeTitles(result: AiResultLike) {
29 |   return result.themes.map((theme) => theme.title.toLowerCase());
30 | }
31 |
32 | export function rejectedCount(result: AiResultLike) {
33 |   return Number(quality(result).lowQualityRejectedCount ?? 0);
34 | }
35 |
36 | export function duplicateCount(result: AiResultLike) {
37 |   return Number(quality(result).duplicateCount ?? result.duplicateCount ?? 0);
38 | }
```

edupulse-backend/scripts/tenant-isolation-test.ts

File size: 3,928 bytes

```
1 | type LoginResponse = {
2 |   accessToken: string;
3 |   user: {
4 |     college: {
5 |       code: string;
6 |       name: string;
7 |     };
8 |     email: string;
9 |   };
10 | };
11 |
12 | const API_BASE = process.env.API_BASE ?? 'http://127.0.0.1:4000/api/v1';
13 |
14 | async function request<T>(<
15 |   path: string,
16 |   options: RequestInit & { token?: string } = {},
17 | ) {
18 |   const headers = new Headers(options.headers);
19 |   headers.set('Content-Type', 'application/json');
20 |
21 |   if (options.token) {
22 |     headers.set('Authorization', `Bearer ${options.token}`);
23 |   }
24 |
25 |   const response = await fetch(`${API_BASE}${path}`, {
26 |     ...options,
27 |     headers,
28 |   });
29 |
30 |   if (!response.ok) {
31 |     throw new Error(`${path} returned ${response.status}`);
32 |   }
33 |
34 |   return (await response.json()) as T;
35 | }
36 |
37 | async function login(email: string, password: string) {
38 |   return request<LoginResponse>('/auth/login', {
39 |     method: 'POST',
40 |     body: JSON.stringify({ email, password }),
41 |   });
42 | }
43 |
44 | async function expectBlocked(path: string, token: string) {
45 |   const response = await fetch(`${API_BASE}${path}`, {
46 |     headers: { Authorization: `Bearer ${token}` },
47 |   });
48 |
49 |   if (![403, 404].includes(response.status)) {
50 |     throw new Error(
51 |       `${path} should be blocked but returned ${response.status}`,
52 |     );
53 |   }
54 |
55 |   return response.status;
56 | }
57 |
58 | function assert(condition: unknown, message: string) {
59 |   if (!condition) {
60 |     throw new Error(message);
61 |   }
62 | }
63 |
64 | async function main() {
65 |   const gl = await login('admin@edupulse.edu', 'Admin@12345');
66 |   const sharda = await login('admin@sharda.edu', 'Admin@12345');
67 |
68 |   assert(
69 |     gl.user.college.code === 'GLBAJAJ',
70 |     'GL Bajaj admin has wrong college',
71 |   );
72 |   assert(
73 |     sharda.user.college.code === 'SHARDA',
74 |     'Sharda admin has wrong college',
75 |   );
76 |
77 |   const glSummary = await request<{
78 |     responseCount: number;
79 |     submissionCount: number;
80 |   }>('/dashboard/summary', { token: gl.accessToken });
81 |   const shardaSummary = await request<{
82 |     responseCount: number;
83 |     submissionCount: number;
84 |   }>('/dashboard/summary', { token: sharda.accessToken });
85 |
86 |   assert(
87 |     glSummary.responseCount > 0,
88 |     'GL Bajaj should see its own feedback data, got ${glSummary.responseCount}',
89 |   );
90 |   assert(
91 |     shardaSummary.responseCount > 0,
92 |     'Sharda should see its own feedback data, got ${shardaSummary.responseCount}',
93 |   );
94 |   assert(
95 |     shardaSummary.responseCount !== glSummary.responseCount,
96 |     'Tenant summaries should not collapse into the same shared response count',
97 |   );
98 |   assert(
99 |     shardaSummary.submissionCount !== glSummary.submissionCount,
100 |    'Tenant summaries should not collapse into the same shared submission count',
101 |   );
102 |
103 |   const glReports = await request<{ id: string }[]>('/reports', {
104 |     token: gl.accessToken,
105 |   });
```

```

106 |
107 | let crossTenantReportStatus: number | 'SKIPPED' = 'SKIPPED';
108 | let crossTenantPdfStatus: number | 'SKIPPED' = 'SKIPPED';
109 |
110 | if (glReports.length > 0) {
111 |   const reportId = glReports[0].id;
112 |   crossTenantReportStatus = await expectBlocked(
113 |     `/reports/${reportId}`,
114 |     sharda.accessToken,
115 |   );
116 |   crossTenantPdfStatus = await expectBlocked(
117 |     `/reports/${reportId}/pdf`,
118 |     sharda.accessToken,
119 |   );
120 | }
121 |
122 | console.log(
123 |   JSON.stringify(
124 |     {
125 |       apiBase: API_BASE,
126 |       checks: {
127 |         glCollege: gl.user.college.code,
128 |         glResponses: glSummary.responseCount,
129 |         glSubmissions: glSummary.submissionCount,
130 |         shardaCollege: sharda.user.college.code,
131 |         shardaResponses: shardaSummary.responseCount,
132 |         shardaSubmissions: shardaSummary.submissionCount,
133 |         crossTenantReportStatus,
134 |         crossTenantPdfStatus,
135 |       },
136 |       result: 'PASS',
137 |     },
138 |     null,
139 |     2,
140 |   ),
141 | );
142 | }
143 |
144 | main().catch((error) => {
145 |   console.error(
146 |     JSON.stringify(
147 |       {
148 |         apiBase: API_BASE,
149 |         error: error instanceof Error ? error.message : String(error),
150 |         result: 'FAIL',
151 |       },
152 |       null,
153 |       2,
154 |     ),
155 |   );
156 |   process.exit(1);
157 | });

```


edupulse-backend/src/actions/actions.controller.ts

File size: 1,605 bytes

```
1 | import {
2 |   Body,
3 |   Controller,
4 |   Get,
5 |   Param,
6 |   Patch,
7 |   Post,
8 |   Query,
9 |   Req,
10 |   UseGuards,
11 | } from '@nestjs/common';
12 | import { ApiTags } from '@nestjs/swagger';
13 | import { UserRole } from '@prisma/client';
14 | import { Roles } from '../auth/decorators/roles.decorator';
15 | import type { AuthenticatedRequest } from '../auth/guards/jwt-auth.guard';
16 | import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
17 | import { RolesGuard } from '../auth/guards/roles.guard';
18 | import { ActionsService } from './actions.service';
19 | import { CreateActionItemDto } from './dto/create-action-item.dto';
20 | import { UpdateActionStatusDto } from './dto/update-action-status.dto';
21 |
22 | @ApiTags('actions')
23 | @UseGuards(JwtAuthGuard, RolesGuard)
24 | @Controller('actions')
25 | export class ActionsController {
26 |   constructor(private readonly actionsService: ActionsService) {}
27 |
28 |   @Get()
29 |   @Roles(UserRole.ADMIN)
30 |   list(
31 |     @Req() request: AuthenticatedRequest,
32 |     @Query('reportId') reportId?: string,
33 |   ) {
34 |     return this.actionsService.list(request.user!.collegeId, reportId);
35 |   }
36 |
37 |   @Post()
38 |   @Roles(UserRole.ADMIN)
39 |   create(
40 |     @Req() request: AuthenticatedRequest,
41 |     @Body() dto: CreateActionItemDto,
42 |   ) {
43 |     return this.actionsService.create(
44 |       request.user!.sub,
45 |       request.user!.collegeId,
46 |       dto,
47 |     );
48 |   }
49 |
50 |   @Patch('/:id/status')
51 |   @Roles(UserRole.ADMIN)
52 |   updateStatus(
53 |     @Req() request: AuthenticatedRequest,
54 |     @Param('id') id: string,
55 |     @Body() dto: UpdateActionStatusDto,
56 |   ) {
57 |     return this.actionsService.updateStatus(
58 |       request.user!.sub,
59 |       request.user!.collegeId,
60 |       id,
61 |       dto,
62 |     );
63 |   }
64 | }
```

edupulse-backend/src/actions/actions.module.ts

File size: 341 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { AuditModule } from '../audit/audit.module';
3 | import { ActionsController } from './actions.controller';
4 | import { ActionsService } from './actions.service';
5 |
6 | @Module({
7 |   imports: [AuditModule],
8 |   controllers: [ActionsController],
9 |   providers: [ActionsService],
10 | })
11 | export class ActionsModule {}
```

edupulse-backend/src/actions/actions.service.ts

File size: 2,252 bytes

```
1 | import { Injectable, NotFoundException } from '@nestjs/common';
2 | import { AuditAction } from '@prisma/client';
3 | import { AuditService } from '../audit/audit.service';
4 | import { PrismaService } from '../prisma/prisma.service';
5 | import { CreateActionItemDto } from '../dto/create-action-item.dto';
6 | import { UpdateActionStatusDto } from '../dto/update-action-status.dto';
7 |
8 | @Injectable()
9 | export class ActionsService {
10 |   constructor(
11 |     private readonly prisma: PrismaService,
12 |     private readonly auditService: AuditService,
13 |   ) {}
14 |
15 |   list(collegeId: string, reportId?: string) {
16 |     return this.prisma.actionItem.findMany({
17 |       where: reportId ? { collegeId, reportId } : { collegeId },
18 |       orderBy: [{ priority: 'desc' }, { createdAt: 'desc' }],
19 |     });
20 |   }
21 |
22 |   async create(adminId: string, collegeId: string, dto: CreateActionItemDto) {
23 |     const report = await this.prisma.analysisReport.findFirst({
24 |       where: { id: dto.reportId, collegeId },
25 |       select: { id: true },
26 |     });
27 |
28 |     if (!report) {
29 |       throw new NotFoundException('Report not found for this college');
30 |     }
31 |
32 |     const action = await this.prisma.actionItem.create({
33 |       data: { ...dto, collegeId },
34 |     });
35 |
36 |     await this.auditService.log({
37 |       action: AuditAction.ACTION_ITEM_CREATED,
38 |       actorId: adminId,
39 |       collegeId,
40 |       entity: 'ActionItem',
41 |       entityId: action.id,
42 |       metadata: {
43 |         reportId: action.reportId,
44 |         priority: action.priority,
45 |         title: action.title,
46 |       },
47 |     });
48 |
49 |     return action;
50 |   }
51 |
52 |   async updateStatus(
53 |     adminId: string,
54 |     collegeId: string,
55 |     id: string,
56 |     dto: UpdateActionStatusDto,
57 |   ) {
58 |     const action = await this.prisma.actionItem.findFirst({
59 |       where: { id, collegeId },
60 |     });
61 |
62 |     if (!action) {
63 |       throw new NotFoundException('Action item not found');
64 |     }
65 |
66 |     const updated = await this.prisma.actionItem.update({
67 |       where: { id },
68 |       data: { status: dto.status },
69 |     });
70 |
71 |     await this.auditService.log({
72 |       action: AuditAction.ACTION_STATUS_UPDATED,
73 |       actorId: adminId,
74 |       collegeId,
75 |       entity: 'ActionItem',
76 |       entityId: updated.id,
77 |       metadata: {
78 |         status: updated.status,
79 |         title: updated.title,
80 |       },
81 |     });
82 |
83 |     return updated;
84 |   }
85 | }
```

edupulse-backend/src/actions/dto/create-action-item.dto.ts

File size: 612 bytes

```
1 | import { ApiProperty } from '@nestjs/swagger';
2 | import { IsEnum, IsOptional, IsString } from 'class-validator';
3 | import { PriorityLevel } from '@prisma/client';
4 |
5 | export class CreateActionItemDto {
6 |   @ApiProperty()
7 |   @IsString()
8 |   reportId: string;
9 |
10 |   @ApiProperty()
11 |   @IsString()
12 |   title: string;
13 |
14 |   @ApiProperty({ required: false })
15 |   @IsOptional()
16 |   @IsString()
17 |   ownerId?: string;
18 |
19 |   @ApiProperty({ enum: PriorityLevel })
20 |   @IsEnum(PriorityLevel)
21 |   priority: PriorityLevel;
22 |
23 |   @ApiProperty()
24 |   @IsString()
25 |   timeline: string;
26 |
27 |   @ApiProperty({ required: false })
28 |   @IsOptional()
29 |   @IsString()
30 |   kpi?: string;
31 | }
```

edupulse-backend/src/actions/dto/update-action-status.dto.ts

File size: 263 bytes

```
1 | import { ApiProperty } from '@nestjs/swagger';
2 | import { ActionStatus } from '@prisma/client';
3 | import { IsEnum } from 'class-validator';
4 |
5 | export class UpdateActionStatusDto {
6 |   @ApiProperty({ enum: ActionStatus })
7 |   @IsEnum(ActionStatus)
8 |   status: ActionStatus;
9 | }
```

File size: 4,568 bytes

```

1 | import { Injectable, Logger } from '@nestjs/common';
2 | import { ConfigService } from '@nestjs/config';
3 | import { PriorityLevel, SentimentLabel } from '@prisma/client';
4 |
5 | export type AiFeedbackInput = {
6 |   id: string;
7 |   categoryId: string;
8 |   categoryName: string;
9 |   questionId: string;
10 |   questionText: string;
11 |   rating: number;
12 |   comment: string | null;
13 |   departmentCode: string | null;
14 |   semesterNumber: number | null;
15 |   weight?: number;
16 | };
17 |
18 | export type AiGrievanceInput = {
19 |   id: string;
20 |   category: string;
21 |   title: string;
22 |   description: string;
23 |   severity: string;
24 |   status: string;
25 |   safetyFlag: boolean;
26 |   isAnonymous: boolean;
27 |   departmentCode: string | null;
28 | };
29 |
30 | export type AiAnalysisPayload = {
31 |   term: {
32 |     id: string;
33 |     name: string;
34 |   };
35 |   feedback: AiFeedbackInput[];
36 |   grievances: AiGrievanceInput[];
37 | };
38 |
39 | export type AiThemeOutput = {
40 |   categoryId: string | null;
41 |   title: string;
42 |   summary: string;
43 |   sentiment: SentimentLabel;
44 |   priority: PriorityLevel;
45 |   mentionCount: number;
46 |   evidence: Record<string, unknown>;
47 |   actionTitle?: string;
48 |   kpi?: string;
49 | };
50 |
51 | export type AiActionOutput = {
52 |   title: string;
53 |   priority: PriorityLevel;
54 |   timeline: string;
55 |   kpi?: string | null;
56 | };
57 |
58 | export type AiAnalysisResult = {
59 |   title: string;
60 |   summary: string;
61 |   confidence: number;
62 |   inputCount: number;
63 |   lowSampleFlag: boolean;
64 |   duplicateCount: number;
65 |   rawJson: Record<string, unknown>;
66 |   themes: AiThemeOutput[];
67 |   actions?: AiActionOutput[];
68 | };
69 |
70 | @Injectable()
71 | export class AiAnalysisClient {
72 |   private readonly logger = new Logger(AiAnalysisClient.name);
73 |
74 |   constructor(private readonly configService: ConfigService) {}
75 |
76 |   async analyze(payload: AiAnalysisPayload): Promise<AiAnalysisResult | null> {
77 |     const baseUrl = this.configService.get<string>('AI_SERVICE_URL');
78 |
79 |     if (!baseUrl) {
80 |       return null;
81 |     }
82 |
83 |     const timeoutMs = Number(
84 |       this.configService.get<string>('AI_SERVICE_TIMEOUT_MS') ?? 15000,
85 |     );
86 |
87 |     for (let attempt = 1; attempt <= 2; attempt += 1) {
88 |       const controller = new AbortController();
89 |       const timeout = setTimeout(() => controller.abort(), timeoutMs);
90 |
91 |       try {
92 |         const response = await fetch(`${baseUrl.replace(/\/$/, '')}/analyze`, {
93 |           method: 'POST',
94 |           headers: { 'Content-Type': 'application/json', Connection: 'close' },
95 |           body: JSON.stringify(payload),
96 |           signal: controller.signal,
97 |         });
98 |
99 |         if (!response.ok) {
100 |           this.logger.warn(`AI service returned HTTP ${response.status}`);
101 |           return null;
102 |         }
103 |
104 |         const result = (await response.json()) as AiAnalysisResult;
105 |         return this.normalizeResult(result);

```

```

106 |     } catch (error) {
107 |         const message = error instanceof Error ? error.message : 'unknown error';
108 |         if (attempt < 2) {
109 |             this.logger.warn(`AI service request failed (${message}); retrying once`);
110 |             continue;
111 |         }
112 |         this.logger.warn(
113 |             `AI service unavailable, using deterministic fallback: ${message}`,
114 |         );
115 |         return null;
116 |     } finally {
117 |         clearTimeout(timeout);
118 |     }
119 | }
120 |
121 | return null;
122 | }
123 |
124 | private normalizeResult(result: AiAnalysisResult): AiAnalysisResult {
125 |     const themes = (result.themes ?? []).map((theme) => ({
126 |         ...theme,
127 |         sentiment: this.resolveSentiment(theme.sentiment),
128 |         priority: this.resolvePriority(theme.priority),
129 |         mentionCount: Math.max(1, Number(theme.mentionCount ?? 1)),
130 |         evidence: theme.evidence ?? {},
131 |     }));
132 |
133 |     const actions = (result.actions ?? []).map((action) => ({
134 |         ...action,
135 |         priority: this.resolvePriority(action.priority),
136 |         timeline: action.timeline || '7-14 days',
137 |         kpi: action.kpi ?? null,
138 |     }));
139 |
140 |     return {
141 |         title: result.title || 'EduPulse AI feedback intelligence report',
142 |         summary: result.summary || 'AI service generated a feedback report.',
143 |         confidence: Math.max(0, Math.min(1, Number(result.confidence ?? 0.7))),
144 |         inputCount: Math.max(0, Number(result.inputCount ?? 0)),
145 |         lowSampleFlag: Boolean(result.lowSampleFlag),
146 |         duplicateCount: Math.max(0, Number(result.duplicateCount ?? 0)),
147 |         rawJson: result.rawJson ?? {},
148 |         themes,
149 |         actions,
150 |     };
151 | }
152 |
153 | private resolveSentiment(value: SentimentLabel): SentimentLabel {
154 |     return Object.values(SentimentLabel).includes(value)
155 |         ? value
156 |         : SentimentLabel.NEUTRAL;
157 | }
158 |
159 | private resolvePriority(value: PriorityLevel): PriorityLevel {
160 |     return Object.values(PriorityLevel).includes(value)
161 |         ? value
162 |         : PriorityLevel.LOW;
163 | }
164 | }

```

edupulse-backend/src/analysis/analysis.controller.ts

File size: 1,129 bytes

```
1 | import { Body, Controller, Post, Req, UseGuards } from '@nestjs/common';
2 | import { ApiTags } from '@nestjs/swagger';
3 | import { UserRole } from '@prisma/client';
4 | import { Roles } from '../auth/decorators/roles.decorator';
5 | import type { AuthenticatedRequest } from '../auth/guards/jwt-auth.guard';
6 | import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
7 | import { RolesGuard } from '../auth/guards/roles.guard';
8 | import { AnalysisService } from './analysis.service';
9 | import { RunAnalysisDto } from './dto/run-analysis.dto';
10 |
11 | @ApiTags('analysis')
12 | @UseGuards(JwtAuthGuard, RolesGuard)
13 | @Controller('analysis')
14 | export class AnalysisController {
15 |   constructor(private readonly analysisService: AnalysisService) {}
16 |
17 |   @Post('run')
18 |   @Roles(UserRole.ADMIN)
19 |   run(@Req() request: AuthenticatedRequest, @Body() dto: RunAnalysisDto) {
20 |     return this.analysisService.run(
21 |       request.user!.sub,
22 |       request.user!.collegeId,
23 |       dto,
24 |     );
25 |   }
26 |
27 |   @Post('validation-demo')
28 |   @Roles(UserRole.ADMIN)
29 |   validationDemo(@Req() request: AuthenticatedRequest) {
30 |     return this.analysisService.validationDemo(request.user!.collegeId);
31 |   }
32 | }
```


edupulse-backend/src/analysis/analysis.module.ts

File size: 431 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { AuditModule } from '../audit/audit.module';
3 | import { AiAnalysisClient } from '../ai-analysis-client.service';
4 | import { AnalysisController } from './analysis.controller';
5 | import { AnalysisService } from './analysis.service';
6 |
7 | @Module({
8 |   imports: [AuditModule],
9 |   controllers: [AnalysisController],
10 |   providers: [AnalysisService, AiAnalysisClient],
11 | })
12 | export class AnalysisModule {}
```

File size: 27,523 bytes

```

1 | import { Injectable, NotFoundException } from '@nestjs/common';
2 | import {
3 |   AuditAction,
4 |   Prisma,
5 |   PriorityLevel,
6 |   SentimentLabel,
7 | } from '@prisma/client';
8 | import { AuditService } from '../audit/audit.service';
9 | import { PrismaService } from '../prisma/prisma.service';
10 | import {
11 |   AiAnalysisClient,
12 |   type AiAnalysisResult,
13 |   type AiAnalysisPayload,
14 | } from './ai-analysis-client.service';
15 | import { RunAnalysisDto } from './dto/run-analysis.dto';
16 |
17 | type CategoryBucket = {
18 |   categoryId: string;
19 |   categoryName: string;
20 |   ratings: number[];
21 |   comments: string[];
22 | };
23 |
24 | type FeedbackResponseForAnalysis = Prisma.FeedbackResponseGetPayload<{
25 |   include: {
26 |     category: true;
27 |     question: true;
28 |     submission: {
29 |       include: {
30 |         student: {
31 |           include: { department: true };
32 |         };
33 |       };
34 |     };
35 |   };
36 | }>;
37 |
38 | const criticalWords = [
39 |   'ragging',
40 |   'harassment',
41 |   'abuse',
42 |   'unsafe',
43 |   'threat',
44 |   'violence',
45 |   'medical',
46 | ];
47 |
48 | const negativeWords = [
49 |   'bad',
50 |   'poor',
51 |   'worst',
52 |   'dirty',
53 |   'slow',
54 |   'broken',
55 |   'issue',
56 |   'problem',
57 |   'unfair',
58 |   'not working',
59 | ];
60 |
61 | @Injectable()
62 | export class AnalysisService {
63 |   constructor(
64 |     private readonly prisma: PrismaService,
65 |     private readonly aiAnalysisClient: AiAnalysisClient,
66 |     private readonly auditService: AuditService,
67 |   ) {}
68 |
69 |   async run(adminId: string, collegeId: string, dto: RunAnalysisDto) {
70 |     const term = await this.prisma.academicTerm.findFirst({
71 |       where: { id: dto.termId, collegeId },
72 |     });
73 |
74 |     if (!term) {
75 |       throw new NotFoundException('Academic term not found');
76 |     }
77 |
78 |     const responses = await this.prisma.feedbackResponse.findMany({
79 |       where: { collegeId, submission: { termId: dto.termId } },
80 |       include: {
81 |         category: true,
82 |         question: true,
83 |         submission: {
84 |           include: {
85 |             student: {
86 |               include: { department: true },
87 |             },
88 |           },
89 |         },
90 |       },
91 |     });
92 |
93 |     const aiFeedback = this.buildAiFeedbackPayload(responses);
94 |     const aiResult = await this.aiAnalysisClient.analyze({
95 |       term: {
96 |         id: term.id,
97 |         name: term.name,
98 |       },
99 |       feedback: aiFeedback,
100 |       grievances: [],
101 |     });
102 |
103 |     if (aiResult) {
104 |       aiResult.inputCount = responses.length;
105 |       aiResult.lowSampleFlag = responses.length < 20;

```

```

106 |     aiResult.rawJson = {
107 |       ...aiResult.rawJson,
108 |       aiPayloadCompaction: {
109 |         originalFeedbackRows: responses.length,
110 |         compactedFeedbackRows: aiFeedback.length,
111 |         method:
112 |           'Duplicate feedback rows are sent to the AI service as weighted representatives for scale-safe analysis.',
113 |       },
114 |       sentimentBreakdown:
115 |         aiResult.rawJson?.sentimentBreakdown ??
116 |         this.buildSentimentBreakdown(responses),
117 |     };
118 |     const report = await this.createReportFromResult(
119 |       collegeId,
120 |       dto.termId,
121 |       aiResult,
122 |     );
123 |     await this.logAnalysisRun(adminId, collegeId, report);
124 |     return report;
125 |   }
126 |
127 |   const comments = responses
128 |     .map((response) => response.comment?.trim())
129 |     .filter((comment): comment is string => Boolean(comment));
130 |
131 |   const duplicateCount =
132 |     comments.length - new Set(comments.map((c) => c.toLowerCase())).size;
133 |   const lowSampleFlag = responses.length < 20;
134 |   const buckets = new Map<string, CategoryBucket>();
135 |
136 |   for (const response of responses) {
137 |     const current = buckets.get(response.categoryId) ?? {
138 |       categoryId: response.categoryId,
139 |       categoryName: response.category.name,
140 |       ratings: [],
141 |       comments: [],
142 |     };
143 |
144 |     current.ratings.push(response.rating);
145 |
146 |     if (response.comment?.trim()) {
147 |       current.comments.push(response.comment.trim());
148 |     }
149 |
150 |     buckets.set(response.categoryId, current);
151 |   }
152 |
153 |   const allThemes = [...buckets.values()].map((bucket) =>
154 |     this.buildTheme(bucket),
155 |   );
156 |
157 |   const criticalCount = allThemes.filter(
158 |     (theme) => theme.priority === PriorityLevel.CRITICAL,
159 |   ).length;
160 |   const negativeCount = allThemes.filter(
161 |     (theme) =>
162 |       theme.sentiment === SentimentLabel.NEGATIVE ||
163 |       theme.sentiment === SentimentLabel.CRITICAL,
164 |   ).length;
165 |   const confidence = this.calculateConfidence({
166 |     responseCount: responses.length,
167 |     duplicateCount,
168 |     themeCount: allThemes.length,
169 |   });
170 |
171 |   const report = await this.prisma.analysisReport.create({
172 |     data: {
173 |       termId: dto.termId,
174 |       collegeId,
175 |       title: `${term.name} feedback intelligence report`,
176 |       summary: this.buildExecutiveSummary({
177 |         responseCount: responses.length,
178 |         themeCount: allThemes.length,
179 |         negativeCount,
180 |         criticalCount,
181 |       }),
182 |       confidence,
183 |       inputCount: responses.length,
184 |       lowSampleFlag,
185 |       duplicateCount,
186 |       rawJson: {
187 |         qualityChecks: {
188 |           totalResponses: responses.length,
189 |           commentCount: comments.length,
190 |           duplicateCount,
191 |           lowSampleFlag,
192 |           method:
193 |             'Deterministic analysis layer; LLM can enrich summaries with the same structured inputs.',
194 |         },
195 |         sentimentBreakdown: this.buildSentimentBreakdown(responses),
196 |       },
197 |       themes: [
198 |         create: allThemes.map((theme) => ({
199 |           categoryId: theme.categoryId,
200 |           collegeId,
201 |           title: theme.title,
202 |           summary: theme.summary,
203 |           sentiment: theme.sentiment,
204 |           priority: theme.priority,
205 |           mentionCount: theme.mentionCount,
206 |           evidence: theme.evidence,
207 |         })),
208 |       ],
209 |       actions: {
210 |         create: allThemes
211 |           .filter(
212 |             (theme) =>
213 |               theme.priority === PriorityLevel.HIGH ||
214 |               theme.priority === PriorityLevel.CRITICAL,
215 |           )

```

```

216 |         .map((theme) => ({
217 |             title: theme.actionTitle,
218 |             collegeId,
219 |             priority: theme.priority,
220 |             timeline:
221 |                 theme.priority === PriorityLevel.CRITICAL
222 |                 ? '24-48 hours'
223 |                 : '7-14 days',
224 |             kpi: theme.kpi,
225 |         })),
226 |     },
227 | ],
228 | include: {
229 |     themes: { orderBy: [{ priority: 'desc' }, { mentionCount: 'desc' }] },
230 |     actions: { orderBy: [{ priority: 'desc' }, { createdAt: 'desc' }] },
231 | },
232 | });
233 | await this.logAnalysisRun(adminId, collegeId, report);
234 | return report;
235 | }
236 |
237 | async validationDemo(collegeId: string) {
238 |     const result = await this.aiAnalysisClient.analyze(
239 |         this.buildValidationDemoPayload(collegeId),
240 |     );
241 |
242 |     if (!result) {
243 |         return {
244 |             status: 'REVIEW',
245 |             score: 0,
246 |             totalChecks: 1,
247 |             passedChecks: 0,
248 |             summary:
249 |                 'AI service is not reachable. Start the Python AI service and run the validation again.',
250 |             cases: [
251 |                 {
252 |                     name: 'AI service reachable',
253 |                     expected: 'FastAPI service responds through AI_SERVICE_URL',
254 |                     actual: 'No AI result returned',
255 |                     passed: false,
256 |                 },
257 |             ],
258 |         };
259 |     }
260 |
261 |     const quality = (result.rawJson?.qualityChecks ?? {}) as Record<
262 |         string,
263 |         unknown
264 |     >;
265 |     const reasoning = (result.rawJson?.geminiReasoning ?? {}) as Record<
266 |         string,
267 |         unknown
268 |     >;
269 |     const ultra = Array.isArray(reasoning.ultraConcerningIssues)
270 |         ? (reasoning.ultraConcerningIssues as Array<Record<string, unknown>>)
271 |         : [];
272 |     const ultraTitles = ultra.map((item) =>
273 |         String(item.title ?? '').toLowerCase(),
274 |     );
275 |     const themeTitles = result.themes.map((theme) =>
276 |         theme.title.toLowerCase(),
277 |     );
278 |     const rejectedCount = Number(quality.lowQualityRejectedCount ?? 0);
279 |     const duplicateCount = Number(quality.duplicateCount ?? 0);
280 |
281 |     const includesTitle = (titles: string[], fragment: string) =>
282 |         titles.some((title) => title.includes(fragment.toLowerCase()));
283 |
284 |     const cases = [
285 |         {
286 |             name: 'Fake comments ignored',
287 |             expected: 'At least 2 fake/out-of-context comments rejected',
288 |             actual: `${rejectedCount} rejected`,
289 |             passed: rejectedCount >= 2,
290 |         },
291 |         {
292 |             name: 'Duplicate comments grouped',
293 |             expected: 'Repeated Wi-Fi comment grouped as one repeated signal',
294 |             actual: `${duplicateCount} duplicate(s) grouped`,
295 |             passed: duplicateCount >= 1,
296 |         },
297 |         {
298 |             name: 'Ragging escalated',
299 |             expected: 'Ragging concern appears only in immediate action layer',
300 |             actual: ultraTitles.join(', ') || 'none',
301 |             passed: includesTitle(ultraTitles, 'ragging'),
302 |         },
303 |         {
304 |             name: 'Harassment escalated',
305 |             expected: 'Harassment concern appears only in immediate action layer',
306 |             actual: ultraTitles.join(', ') || 'none',
307 |             passed: includesTitle(ultraTitles, 'harassment'),
308 |         },
309 |         {
310 |             name: 'Water risk escalated',
311 |             expected: 'Water contamination appears as urgent concern',
312 |             actual: ultraTitles.join(', ') || 'none',
313 |             passed: includesTitle(ultraTitles, 'water contamination'),
314 |         },
315 |         {
316 |             name: 'Normal issues stay in AI findings',
317 |             expected: 'Projector, Wi-Fi, transport and library stay as normal issue cards',
318 |             actual: themeTitles.slice(0, 10).join(', ') || 'none',
319 |             passed:
320 |                 includesTitle(themeTitles, 'projector') &&
321 |                 includesTitle(themeTitles, 'wi-fi') &&
322 |                 includesTitle(themeTitles, 'transport') &&
323 |                 includesTitle(themeTitles, 'library'),
324 |         },
325 |     ];

```

```

326 |         name: 'Faculty issues stay separated',
327 |         expected: 'Attendance and marks partiality become separate faculty issues',
328 |         actual: themeTitles.join(', ') || 'none',
329 |         passed:
330 |             includesTitle(themeTitles, 'attendance') &&
331 |             includesTitle(themeTitles, 'marks partiality'),
332 |     },
333 | ];
334 |
335 | const passedChecks = cases.filter((item) => item.passed).length;
336 | const score = Math.round((passedChecks / cases.length) * 100);
337 |
338 | return {
339 |     status: score >= 90 ? 'PASS' : 'REVIEW',
340 |     score,
341 |     totalChecks: cases.length,
342 |     passedChecks,
343 |     summary:
344 |         score >= 90
345 |             ? 'Golden judge demo passed. The same AI pipeline handled quality filtering, duplicate grouping, urgent escalation and normal issue taxonomy.'
346 |             : 'Golden judge demo needs review. Check failed cases before using this flow in front of judges.',
347 |     cases,
348 |     modelMode: result.rawJson?.modelMode ?? 'unknown',
349 |     pipeline: result.rawJson?.pipeline ?? [],
350 | };
351 | }
352 |
353 | private buildValidationDemoPayload(collegeId: string): AiAnalysisPayload {
354 |     const rows = [
355 |         {
356 |             category: 'Safety',
357 |             question: 'Do you feel safe on campus?',
358 |             rating: 1,
359 |             comment:
360 |                 'Dev Pratap Rai says Gaurav Sharma ECE 4th year ragged him near the hostel stairs.',
361 |             department: 'ECE',
362 |             semester: 1,
363 |         },
364 |         {
365 |             category: 'Safety',
366 |             question: 'Do you feel safe on campus?',
367 |             rating: 1,
368 |             comment:
369 |                 'A student reported harassment and physical touch by Raghav sir from CS AI faculty.',
370 |             department: 'CSAI',
371 |             semester: 5,
372 |         },
373 |         {
374 |             category: 'Infrastructure',
375 |             question: 'How safe and usable are campus facilities?',
376 |             rating: 1,
377 |             comment:
378 |                 'Water pH level contamination is reported in AB3 building near the CS DS drinking water point.',
379 |             department: 'CSDS',
380 |             semester: 3,
381 |         },
382 |         {
383 |             category: 'Infrastructure',
384 |             question: 'How usable are classrooms?',
385 |             rating: 2,
386 |             comment:
387 |                 'AB2 room 307 projector is broken and ECE students are losing class time.',
388 |             department: 'ECE',
389 |             semester: 7,
390 |         },
391 |         {
392 |             category: 'Wi-Fi / Internet',
393 |             question: 'How reliable is internet access?',
394 |             rating: 2,
395 |             comment: 'Wi-Fi is slow in the lab area during project hours.',
396 |             department: 'CSE',
397 |             semester: 5,
398 |         },
399 |         {
400 |             category: 'Wi-Fi / Internet',
401 |             question: 'How reliable is internet access?',
402 |             rating: 2,
403 |             comment: 'Wi-Fi is slow in the lab area during project hours.',
404 |             department: 'CSE',
405 |             semester: 5,
406 |         },
407 |         {
408 |             category: 'Faculty',
409 |             question: 'How regular are classes?',
410 |             rating: 1,
411 |             comment:
412 |                 'Faculty is absent from morning lectures and several classes start late.',
413 |             department: 'IT',
414 |             semester: 4,
415 |         },
416 |         {
417 |             category: 'Faculty',
418 |             question: 'How fair is practical marking?',
419 |             rating: 1,
420 |             comment:
421 |                 'Practical marks partiality is happening and ECE students are worried about unfair internal marks.',
422 |             department: 'ECE',
423 |             semester: 7,
424 |         },
425 |         {
426 |             category: 'Mess',
427 |             question: 'How is mess hygiene?',
428 |             rating: 1,
429 |             comment:
430 |                 'Second year hostel mess has insects near the serving counter and tables are not cleaned.',
431 |             department: 'ME',
432 |             semester: 3,
433 |         },
434 |         {
435 |             category: 'Transport',

```

```

436 |         question: 'How reliable is college transport?',
437 |         rating: 2,
438 |         comment:
439 |             'College bus timing is unreliable and the pickup point is overcrowded in the morning.',
440 |         department: 'CE',
441 |         semester: 6,
442 |     },
443 |     {
444 |         category: 'Library',
445 |         question: 'How useful is library access?',
446 |         rating: 2,
447 |         comment:
448 |             'Library seating is not enough during exam weeks and the reading room AC is not working.',
449 |         department: 'CSE',
450 |         semester: 5,
451 |     },
452 |     {
453 |         category: 'Academics',
454 |         question: 'How fair is academic planning?',
455 |         rating: 2,
456 |         comment:
457 |             'Exam schedule has back to back papers and students need a better date sheet gap.',
458 |         department: 'ECE',
459 |         semester: 7,
460 |     },
461 |     {
462 |         category: 'Administration',
463 |         question: 'How helpful is admin office support?',
464 |         rating: 2,
465 |         comment:
466 |             'Scholarship form support is delayed and fee reimbursement documents are not explained clearly.',
467 |         department: 'CSE',
468 |         semester: 5,
469 |     },
470 |     {
471 |         category: 'Academics',
472 |         question: 'How useful is academic support?',
473 |         rating: 2,
474 |         comment: 'ghhvgjhvb ajhdvbs qwerty',
475 |         department: 'CSE',
476 |         semester: 4,
477 |     },
478 |     {
479 |         category: 'Canteen',
480 |         question: 'How is the canteen service?',
481 |         rating: 2,
482 |         comment: 'movie pizza weather phone battery shopping traffic',
483 |         department: 'IT',
484 |         semester: 4,
485 |     },
486 | ];
487 |
488 | return {
489 |     term: {
490 |         id: `validation-${collegeId}`,
491 |         name: 'Judge AI Validation Demo',
492 |     },
493 |     feedback: rows.map((row, index) => ({
494 |         id: `validation-feedback-${index + 1}`,
495 |         categoryId: `validation-category-${row.category
496 |             .toLowerCase()
497 |             .replace(/[\^a-z0-9]+/g, '-')}`,
498 |         categoryName: row.category,
499 |         questionId: `validation-question-${index + 1}`,
500 |         questionText: row.question,
501 |         rating: row.rating,
502 |         comment: row.comment,
503 |         departmentCode: row.department,
504 |         semesterNumber: row.semester,
505 |     })),
506 |     grievances: [],
507 | };
508 | }
509 |
510 | private buildAiFeedbackPayload(
511 |     responses: FeedbackResponseForAnalysis[],
512 | ): AiAnalysisPayload['feedback'] {
513 |     const compacted = new Map<string, AiAnalysisPayload['feedback'][number]>();
514 |
515 |     for (const response of responses) {
516 |         const departmentCode =
517 |             response.submission.student.department?.code ?? null;
518 |         const semesterNumber = response.submission.student.semesterNumber ?? null;
519 |         const comment = response.comment?.trim() ? response.comment.trim() : null;
520 |         const key = [
521 |             response.categoryId,
522 |             response.questionId,
523 |             response.rating,
524 |             this.normalizeAiComment(comment),
525 |             departmentCode ?? '',
526 |             semesterNumber ?? '',
527 |         ].join(':');
528 |         const existing = compacted.get(key);
529 |
530 |         if (existing) {
531 |             existing.weight = (existing.weight ?? 1) + 1;
532 |             continue;
533 |         }
534 |
535 |         compacted.set(key, {
536 |             id: response.id,
537 |             categoryId: response.categoryId,
538 |             categoryName: response.category.name,
539 |             questionId: response.questionId,
540 |             questionText: response.question.text,
541 |             rating: response.rating,
542 |             comment,
543 |             departmentCode,
544 |             semesterNumber,
545 |             weight: 1,

```

```

546 |     });
547 | }
548 |
549 | return [...compacted.values()];
550 | }
551 |
552 | private normalizeAiComment(comment: string | null) {
553 |   return (comment ?? '').toLowerCase().replace(/\s+/g, ' ').trim();
554 | }
555 |
556 | private async logAnalysisRun(
557 |   adminId: string,
558 |   collegeId: string,
559 |   report: {
560 |     id: string;
561 |     termId: string;
562 |     inputCount: number;
563 |     confidence: number;
564 |     duplicateCount: number;
565 |     lowSampleFlag: boolean;
566 |     themes?: unknown[];
567 |     actions?: unknown[];
568 |   },
569 | ) {
570 |   await this.auditService.log({
571 |     action: AuditAction.ANALYSIS_RUN,
572 |     actorId: adminId,
573 |     collegeId,
574 |     entity: 'AnalysisReport',
575 |     entityId: report.id,
576 |     metadata: {
577 |       termId: report.termId,
578 |       inputCount: report.inputCount,
579 |       confidence: report.confidence,
580 |       duplicateCount: report.duplicateCount,
581 |       lowSampleFlag: report.lowSampleFlag,
582 |       themeCount: report.themes?.length ?? 0,
583 |       actionCount: report.actions?.length ?? 0,
584 |     },
585 |   });
586 | }
587 |
588 | private createReportFromResult(
589 |   collegeId: string,
590 |   termId: string,
591 |   result: AiAnalysisResult,
592 | ) {
593 |   const actions =
594 |     result.actions && result.actions.length > 0
595 |       ? result.actions
596 |       : result.themes
597 |         .filter(
598 |           (theme) =>
599 |             theme.priority === PriorityLevel.HIGH ||
600 |             theme.priority === PriorityLevel.CRITICAL,
601 |         )
602 |         .map((theme) => ({
603 |           title:
604 |             theme.actionTitle ??
605 |             `Resolve ${theme.title.toLowerCase()} with tracked ownership`,
606 |           priority: theme.priority,
607 |           timeline:
608 |             theme.priority === PriorityLevel.CRITICAL
609 |               ? '24-48 hours'
610 |               : '7-14 days',
611 |           kpi:
612 |             theme.kpi ??
613 |             `Reduce ${theme.title.toLowerCase()} mentions next cycle`,
614 |         }));
615 |
616 |   return this.prisma.analysisReport.create({
617 |     data: {
618 |       termId,
619 |       collegeId,
620 |       title: result.title,
621 |       summary: result.summary,
622 |       confidence: result.confidence,
623 |       inputCount: result.inputCount,
624 |       lowSampleFlag: result.lowSampleFlag,
625 |       duplicateCount: result.duplicateCount,
626 |       generatedBy: 'edupulse-ai-service',
627 |       rawJson: result.rawJson as Prisma.InputJsonValue,
628 |       themes: {
629 |         create: result.themes.map((theme) => ({
630 |           categoryId: theme.categoryId,
631 |           collegeId,
632 |           title: theme.title,
633 |           summary: theme.summary,
634 |           sentiment: theme.sentiment,
635 |           priority: theme.priority,
636 |           mentionCount: theme.mentionCount,
637 |           evidence: theme.evidence as Prisma.InputJsonValue,
638 |         })),
639 |       },
640 |       actions: {
641 |         create: actions.map((action) => ({
642 |           title: action.title,
643 |           collegeId,
644 |           priority: action.priority,
645 |           timeline: action.timeline,
646 |           kpi: action.kpi,
647 |         })),
648 |       },
649 |     },
650 |     include: {
651 |       themes: { orderBy: [{ priority: 'desc' }, { mentionCount: 'desc' }] },
652 |       actions: { orderBy: [{ priority: 'desc' }, { createdAt: 'desc' }] },
653 |     },
654 |   });
655 | }

```

```

656 |
657 | private buildTheme(bucket: CategoryBucket) {
658 |     const averageRating =
659 |         bucket.ratings.reduce((sum, rating) => sum + rating, 0) /
660 |         bucket.ratings.length;
661 |     const negativeRatings = bucket.ratings.filter(
662 |         (rating) => rating <= 2,
663 |     ).length;
664 |     const negativeRatio = negativeRatings / bucket.ratings.length;
665 |     const combinedComments = bucket.comments.join(' ').toLowerCase();
666 |     const hasCriticalLanguage = criticalWords.some((word) =>
667 |         combinedComments.includes(word),
668 |     );
669 |     const negativeWordHits = negativeWords.filter((word) =>
670 |         combinedComments.includes(word),
671 |     ).length;
672 |
673 |     const sentiment = this.resolveSentiment({
674 |         averageRating,
675 |         negativeRatio,
676 |         hasCriticalLanguage,
677 |         negativeWordHits,
678 |     });
679 |     const priority = this.resolvePriority({
680 |         sentiment,
681 |         negativeRatio,
682 |         mentionCount: bucket.ratings.length,
683 |         hasCriticalLanguage,
684 |     });
685 |
686 |     return {
687 |         categoryId: bucket.categoryId,
688 |         title: `${bucket.categoryName} needs attention`,
689 |         summary: this.buildThemeSummary({
690 |             categoryName: bucket.categoryName,
691 |             averageRating,
692 |             negativeRatings,
693 |             totalRatings: bucket.ratings.length,
694 |             comments: bucket.comments,
695 |         }),
696 |         sentiment,
697 |         priority,
698 |         mentionCount: bucket.ratings.length,
699 |         evidence: {
700 |             averageRating: Number(averageRating.toFixed(2)),
701 |             negativeRatings,
702 |             sampleComments: bucket.comments.slice(0, 3),
703 |             negativeWordHits,
704 |         },
705 |         actionTitle: `Improve ${bucket.categoryName} based on high-impact feedback`,
706 |         kpi: `Raise ${bucket.categoryName} average rating above 3.2 next cycle`,
707 |     };
708 | }
709 |
710 | private resolveSentiment(input: {
711 |     averageRating: number;
712 |     negativeRatio: number;
713 |     hasCriticalLanguage: boolean;
714 |     negativeWordHits: number;
715 | }) {
716 |     if (input.hasCriticalLanguage || input.negativeRatio >= 0.65) {
717 |         return SentimentLabel.CRITICAL;
718 |     }
719 |
720 |     if (
721 |         input.averageRating < 2.6 ||
722 |         input.negativeRatio >= 0.4 ||
723 |         input.negativeWordHits >= 2
724 |     ) {
725 |         return SentimentLabel.NEGATIVE;
726 |     }
727 |
728 |     if (input.averageRating >= 3.3 && input.negativeRatio < 0.25) {
729 |         return SentimentLabel.POSITIVE;
730 |     }
731 |
732 |     return SentimentLabel.NEUTRAL;
733 | }
734 |
735 | private resolvePriority(input: {
736 |     sentiment: SentimentLabel;
737 |     negativeRatio: number;
738 |     mentionCount: number;
739 |     hasCriticalLanguage: boolean;
740 | }) {
741 |     if (
742 |         input.hasCriticalLanguage ||
743 |         input.sentiment === SentimentLabel.CRITICAL
744 |     ) {
745 |         return PriorityLevel.CRITICAL;
746 |     }
747 |
748 |     if (
749 |         input.sentiment === SentimentLabel.NEGATIVE &&
750 |         (input.negativeRatio >= 0.4 || input.mentionCount >= 5)
751 |     ) {
752 |         return PriorityLevel.HIGH;
753 |     }
754 |
755 |     if (input.sentiment === SentimentLabel.NEGATIVE) {
756 |         return PriorityLevel.MEDIUM;
757 |     }
758 |
759 |     return PriorityLevel.LOW;
760 | }
761 |
762 | private buildExecutiveSummary(input: {
763 |     responseCount: number;
764 |     themeCount: number;
765 |     negativeCount: number;

```



```

766 |     criticalCount: number;
767 | }) {
768 |     return `Analyzed ${input.responseCount} feedback response(s). Found ${input.themeCount} theme(s), including ${input.negativeCount} negative/critical signal
769 | }
770 |
771 | private buildThemeSummary(input: {
772 |     categoryName: string;
773 |     averageRating: number;
774 |     negativeRatings: number;
775 |     totalRatings: number;
776 |     comments: string[];
777 | }) {
778 |     const issueText =
779 |         input.comments.length > 0
780 |             ? `Student comments mention: ${input.comments.slice(0, 2).join(' | ')}`
781 |             : 'No detailed student comments were provided.';
782 |
783 |     return `${input.categoryName} average rating is ${input.averageRating.toFixed(2)} from ${input.totalRatings} response(s), with ${input.negativeRatings} low
784 | }
785 |
786 | private calculateConfidence(input: {
787 |     responseCount: number;
788 |     duplicateCount: number;
789 |     themeCount: number;
790 | }) {
791 |     if (input.responseCount === 0) {
792 |         return 0.35;
793 |     }
794 |
795 |     const sampleScore = Math.min(input.responseCount / 50, 1) * 0.35;
796 |     const duplicatePenalty =
797 |         input.duplicateCount > 0
798 |             ? Math.min(input.duplicateCount / input.responseCount, 0.25)
799 |             : 0;
800 |     const themeScore = input.themeCount > 0 ? 0.35 : 0.1;
801 |     return Number(
802 |         (0.3 + sampleScore + themeScore - duplicatePenalty).toFixed(2),
803 |     );
804 | }
805 |
806 | private buildSentimentBreakdown(
807 |     responses: {
808 |         id: string;
809 |         categoryId: string;
810 |         rating: number;
811 |         comment: string | null;
812 |         category: { name: string };
813 |     }[],
814 | ) {
815 |     const overall = this.emptySentimentCounter();
816 |     const categories = new Map<string, ReturnType<AnalysisService['emptySentimentCounter']>>();
817 |
818 |     for (const response of responses) {
819 |         const comment = this.normalizeComment(response.comment ?? '');
820 |         if (!comment || this.isRejectedComment(comment)) {
821 |             continue;
822 |         }
823 |
824 |         const categoryName = response.category.name;
825 |         const categoryCounter =
826 |             categories.get(categoryName) ?? this.emptySentimentCounter();
827 |         const label = this.sentimentForPie(response.rating, comment);
828 |
829 |         overall[label] += 1;
830 |         categoryCounter[label] += 1;
831 |         categories.set(categoryName, categoryCounter);
832 |     }
833 |
834 |     return {
835 |         overall: this.formatSentimentCounter(overall),
836 |         categories: Object.fromEntries(
837 |             [...categories.entries()].map(([category, counter]) => [
838 |                 category,
839 |                 this.formatSentimentCounter(counter),
840 |             ]),
841 |         ),
842 |     };
843 | }
844 |
845 | private emptySentimentCounter() {
846 |     return {
847 |         POSITIVE: 0,
848 |         NEUTRAL: 0,
849 |         NEGATIVE: 0,
850 |         CRITICAL: 0,
851 |     };
852 | }
853 |
854 | private formatSentimentCounter(counter: ReturnType<AnalysisService['emptySentimentCounter']>) {
855 |     const total =
856 |         counter.POSITIVE + counter.NEUTRAL + counter.NEGATIVE + counter.CRITICAL;
857 |
858 |     return Object.fromEntries(
859 |         (Object.keys(counter) as Array<keyof typeof counter>).map((label) => [
860 |             label,
861 |             {
862 |                 count: counter[label],
863 |                 percentage: total > 0 ? Number(((counter[label] / total) * 100).toFixed(1)) : 0,
864 |             },
865 |         ]),
866 |     );
867 | }
868 |
869 | private sentimentForPie(rating: number, comment: string): SentimentLabel {
870 |     const hasCriticalLanguage = ['ragging', 'harassment', 'abuse', 'unsafe', 'threat', 'threatening', 'violence', 'assault', 'emergency', 'bully', 'bullying',
871 |         comment.includes(word),
872 |     ];
873 |
874 |     if (hasCriticalLanguage) return SentimentLabel.CRITICAL;
875 |     if (rating <= 2) return SentimentLabel.NEGATIVE;

```

```

876 |     if (rating === 3) return SentimentLabel.NEUTRAL;
877 |     return SentimentLabel.POSITIVE;
878 | }
879 |
880 | private isRejectedComment(comment: string) {
881 |     if (comment.length < 8 || comment.split(' ').length < 2) return true;
882 |     if (
883 |         [
884 |             'good',
885 |             'bad',
886 |             'nice',
887 |             'ok',
888 |             'okay',
889 |             'excellent',
890 |             'average',
891 |             'satisfied',
892 |             'unsatisfied',
893 |             'no problem',
894 |             'nothing',
895 |             'none',
896 |             'na',
897 |         ].includes(comment)
898 |     ) {
899 |         return true;
900 |     }
901 |     if (/(\.)\1{5,}/.test(comment)) return true;
902 |     if (!/[a-zA-Z]{3,}/.test(comment)) return true;
903 |     if (
904 |         ['cricket', 'movie', 'phone battery', 'weather', 'traffic', 'pizza coupon', 'shopping'].some((word) =>
905 |             comment.includes(word),
906 |         )
907 |     ) {
908 |         return true;
909 |     }
910 |     const words = comment.split(' ');
911 |     return words.length >= 6 && new Set(words).size / words.length < 0.35;
912 | }
913 |
914 | private normalizeComment(value: string) {
915 |     return value
916 |         .toLowerCase()
917 |         .replace(/\\bwi[\\s-]?fi\\b/g, 'wifi')
918 |         .replace(/^[a-z0-9\\s]/g, ' ')
919 |         .replace(/\\s+/g, ' ')
920 |         .trim();
921 | }
922 | }

```

edupulse-backend/src/analysis/dto/run-analysis.dto.ts

File size: 173 bytes

```
1 | import { ApiProperty } from '@nestjs/swagger';
2 | import { IsString } from 'class-validator';
3 |
4 | export class RunAnalysisDto {
5 |   @ApiProperty()
6 |   @IsString()
7 |   termId: string;
8 | }
```

edupulse-backend/src/app.module.ts

File size: 873 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { ConfigModule } from '@nestjs/config';
3 | import { ActionsModule } from '../actions/actions.module';
4 | import { AnalysisModule } from '../analysis/analysis.module';
5 | import { AuditModule } from '../audit/audit.module';
6 | import { AuthModule } from '../auth/auth.module';
7 | import { DashboardModule } from '../dashboard/dashboard.module';
8 | import { FeedbackModule } from '../feedback/feedback.module';
9 | import { HealthModule } from '../health/health.module';
10 | import { PrismaModule } from '../prisma/prisma.module';
11 | import { ReportsModule } from '../reports/reports.module';
12 |
13 | @Module({
14 |   imports: [
15 |     ConfigModule.forRoot({ isGlobal: true }),
16 |     PrismaModule,
17 |     HealthModule,
18 |     AuthModule,
19 |     AuditModule,
20 |     FeedbackModule,
21 |     AnalysisModule,
22 |     DashboardModule,
23 |     ActionsModule,
24 |     ReportsModule,
25 |   ],
26 | })
27 | export class AppModule {}
```

edupulse-backend/src/audit/audit.controller.ts

File size: 880 bytes

```
1 | import { Controller, Get, Query, Req, UseGuards } from '@nestjs/common';
2 | import { ApiTags } from '@nestjs/swagger';
3 | import { UserRole } from '@prisma/client';
4 | import { Roles } from '../auth/decorators/roles.decorator';
5 | import type { AuthenticatedRequest } from '../auth/guards/jwt-auth.guard';
6 | import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
7 | import { RolesGuard } from '../auth/guards/roles.guard';
8 | import { AuditService } from '../audit.service';
9 |
10 | @ApiTags('audit')
11 | @UseGuards(JwtAuthGuard, RolesGuard)
12 | @Controller('audit')
13 | export class AuditController {
14 |   constructor(private readonly auditService: AuditService) {}
15 |
16 |   @Get('logs')
17 |   @Roles(UserRole.ADMIN)
18 |   list(@Req() request: AuthenticatedRequest, @Query('limit') limit?: string) {
19 |     return this.auditService.listByCollege(
20 |       request.user!.collegeId,
21 |       limit ? Number(limit) : undefined,
22 |     );
23 |   }
24 | }
```

edupulse-backend/src/audit/audit.module.ts

File size: 358 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { PrismaModule } from '../prisma/prisma.module';
3 | import { AuditController } from './audit.controller';
4 | import { AuditService } from './audit.service';
5 |
6 | @Module({
7 |   imports: [PrismaModule],
8 |   controllers: [AuditController],
9 |   providers: [AuditService],
10 | exports: [AuditService],
11 | })
12 | export class AuditModule {}
```

edupulse-backend/src/audit/audit.service.ts

File size: 1,805 bytes

```
1 | import { Injectable, Logger } from '@nestjs/common';
2 | import { AuditAction, Prisma } from '@prisma/client';
3 | import { PrismaService } from '../prisma/prisma.service';
4 |
5 | type AuditInput = {
6 |   action: AuditAction;
7 |   actorId?: string | null;
8 |   collegeId: string;
9 |   entity?: string;
10 |   entityId?: string | null;
11 |   metadata?: Prisma.InputJsonValue;
12 | };
13 |
14 | @Injectable()
15 | export class AuditService {
16 |   private readonly logger = new Logger(AuditService.name);
17 |
18 |   constructor(private readonly prisma: PrismaService) {}
19 |
20 |   async log(input: AuditInput) {
21 |     try {
22 |       return await this.prisma.auditLog.create({
23 |         data: {
24 |           action: input.action,
25 |           actorId: input.actorId ?? undefined,
26 |           collegeId: input.collegeId,
27 |           entity: input.entity,
28 |           entityId: input.entityId ?? undefined,
29 |           metadata: input.metadata,
30 |         },
31 |       });
32 |     } catch (error) {
33 |       this.logger.warn(
34 |         `Audit log failed for ${input.action}: ${
35 |           error instanceof Error ? error.message : String(error)
36 |         }`,
37 |       );
38 |       return null;
39 |     }
40 |   }
41 |
42 |   async listByCollege(collegeId: string, limit = 80) {
43 |     const safeLimit = Math.min(Math.max(Math.floor(limit) || 80, 1), 200);
44 |
45 |     return this.prisma.auditLog.findMany({
46 |       where: { collegeId },
47 |       orderBy: { createdAt: 'desc' },
48 |       take: safeLimit,
49 |       select: {
50 |         id: true,
51 |         action: true,
52 |         entity: true,
53 |         entityId: true,
54 |         metadata: true,
55 |         createdAt: true,
56 |         actor: {
57 |           select: {
58 |             id: true,
59 |             name: true,
60 |             email: true,
61 |             role: true,
62 |           },
63 |         },
64 |         college: {
65 |           select: {
66 |             id: true,
67 |             name: true,
68 |             code: true,
69 |           },
70 |         },
71 |       },
72 |     });
73 |   }
74 | }
```

edupulse-backend/src/auth/auth.controller.ts

File size: 425 bytes

```
1 | import { Body, Controller, Post } from '@nestjs/common';
2 | import { ApiTags } from '@nestjs/swagger';
3 | import { AuthService } from '../auth.service';
4 | import { LoginDto } from '../dto/login.dto';
5 |
6 | @ApiTags('auth')
7 | @Controller('auth')
8 | export class AuthController {
9 |   constructor(private readonly authService: AuthService) {}
10 |
11 |   @Post('login')
12 |   login(@Body() loginDto: LoginDto) {
13 |     return this.authService.login(loginDto);
14 |   }
15 | }
```


edupulse-backend/src/auth/auth.module.ts

File size: 890 bytes

```
1 | import { Global, Module } from '@nestjs/common';
2 | import { JwtModule } from '@nestjs/jwt';
3 | import { ConfigModule, ConfigService } from '@nestjs/config';
4 | import { AuditModule } from '../audit/audit.module';
5 | import { AuthController } from './auth.controller';
6 | import { AuthService } from './auth.service';
7 | import { JwtAuthGuard } from './guards/jwt-auth.guard';
8 | import { RolesGuard } from './guards/roles.guard';
9 |
10 | @Global()
11 | @Module({
12 |   imports: [
13 |     AuditModule,
14 |     JwtModule.registerAsync({
15 |       imports: [ConfigModule],
16 |       inject: [ConfigService],
17 |       useFactory: (config: ConfigService) => ({
18 |         secret: config.getOrThrow<string>('JWT_SECRET'),
19 |         signOptions: { expiresIn: '8h' },
20 |       }),
21 |     ]),
22 |   ],
23 |   controllers: [AuthController],
24 |   providers: [AuthService, JwtAuthGuard, RolesGuard],
25 |   exports: [JwtModule, JwtAuthGuard, RolesGuard],
26 | })
27 | export class AuthModule {}
```

edupulse-backend/src/auth/auth.service.ts

File size: 1,997 bytes

```
1 | import { Injectable, UnauthorizedException } from '@nestjs/common';
2 | import { JwtService } from '@nestjs/jwt';
3 | import { AuditAction } from '@prisma/client';
4 | import { compare } from 'bcryptjs';
5 | import { AuditService } from '../audit/audit.service';
6 | import { PrismaService } from '../prisma/prisma.service';
7 | import { LoginDto } from '../dto/login.dto';
8 |
9 | @Injectable()
10 | export class AuthService {
11 |   constructor(
12 |     private readonly prisma: PrismaService,
13 |     private readonly jwtService: JwtService,
14 |     private readonly auditService: AuditService,
15 |   ) {}
16 |
17 |   async login(loginDto: LoginDto) {
18 |     const user = await this.prisma.user.findUnique({
19 |       where: { email: loginDto.email },
20 |       select: {
21 |         id: true,
22 |         email: true,
23 |         name: true,
24 |         role: true,
25 |         collegeId: true,
26 |         college: {
27 |           select: {
28 |             id: true,
29 |             name: true,
30 |             code: true,
31 |             logoUrl: true,
32 |           },
33 |         },
34 |         passwordHash: true,
35 |         departmentId: true,
36 |         semesterNumber: true,
37 |       },
38 |     });
39 |
40 |     if (!user || !(await compare(loginDto.password, user.passwordHash))) {
41 |       throw new UnauthorizedException('Invalid email or password');
42 |     }
43 |
44 |     const token = await this.jwtService.signAsync({
45 |       sub: user.id,
46 |       email: user.email,
47 |       role: user.role,
48 |       collegeId: user.collegeId,
49 |     });
50 |
51 |     const safeUser = {
52 |       id: user.id,
53 |       email: user.email,
54 |       name: user.name,
55 |       role: user.role,
56 |       collegeId: user.collegeId,
57 |       college: user.college,
58 |       departmentId: user.departmentId,
59 |       semesterNumber: user.semesterNumber,
60 |     };
61 |     await this.auditService.log({
62 |       action: AuditAction.LOGIN,
63 |       actorId: user.id,
64 |       collegeId: user.collegeId,
65 |       entity: 'User',
66 |       entityId: user.id,
67 |       metadata: {
68 |         email: user.email,
69 |         role: user.role,
70 |         collegeCode: user.college.code,
71 |       },
72 |     });
73 |
74 |     return { accessToken: token, user: safeUser };
75 |   }
76 | }
```

edupulse-backend/src/auth/decorators/roles.decorator.ts

File size: 202 bytes

```
1 | import { SetMetadata } from '@nestjs/common';
2 | import { UserRole } from '@prisma/client';
3 |
4 | export const ROLES_KEY = 'roles';
5 | export const Roles = (...roles: UserRole[]) => SetMetadata(ROLES_KEY, roles);
```

edupulse-backend/src/auth/dto/login.dto.ts

File size: 312 bytes

```
1 | import { ApiProperty } from '@nestjs/swagger';
2 | import { IsEmail, IsString, MinLength } from 'class-validator';
3 |
4 | export class LoginDto {
5 |   @ApiProperty({ example: 'admin@edupulse.edu' })
6 |   @IsEmail()
7 |   email: string;
8 |
9 |   @ApiProperty({ example: 'password123' })
10 |   @IsString()
11 |   @MinLength(8)
12 |   password: string;
13 | }
```

edupulse-backend/src/auth/guards/jwt-auth.guard.ts

File size: 1,162 bytes

```
1 | import {
2 |   CanActivate,
3 |   ExecutionContext,
4 |   Injectable,
5 |   UnauthorizedException,
6 | } from '@nestjs/common';
7 | import { JwtService } from '@nestjs/jwt';
8 | import { Request } from 'express';
9 |
10 | export type AuthenticatedUser = {
11 |   sub: string;
12 |   email: string;
13 |   role: string;
14 |   collegeId: string;
15 | };
16 |
17 | export type AuthenticatedRequest = Request & {
18 |   user?: AuthenticatedUser;
19 | };
20 |
21 | @Injectable()
22 | export class JwtAuthGuard implements CanActivate {
23 |   constructor(private readonly jwtService: JwtService) {}
24 |
25 |   async canActivate(context: ExecutionContext): Promise<boolean> {
26 |     const request = context.switchToHttp().getRequest<AuthenticatedRequest>();
27 |     const token = this.extractToken(request);
28 |
29 |     if (!token) {
30 |       throw new UnauthorizedException('Missing bearer token');
31 |     }
32 |
33 |     try {
34 |       request.user =
35 |         await this.jwtService.verifyAsync<AuthenticatedUser>(token);
36 |       return true;
37 |     } catch {
38 |       throw new UnauthorizedException('Invalid or expired token');
39 |     }
40 |   }
41 |
42 |   private extractToken(request: Request) {
43 |     const [type, token] = request.headers.authorization?.split(' ') ?? [];
44 |     return type === 'Bearer' ? token : undefined;
45 |   }
46 | }
```

edupulse-backend/src/auth/guards/roles.guard.ts

File size: 849 bytes

```
1 | import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
2 | import { Reflector } from '@nestjs/core';
3 | import { UserRole } from '@prisma/client';
4 | import { ROLES_KEY } from '../decorators/roles.decorator';
5 | import { AuthenticatedRequest } from '../jwt-auth.guard';
6 |
7 | @Injectable()
8 | export class RolesGuard implements CanActivate {
9 |   constructor(private readonly reflector: Reflector) {}
10 |
11 |   canActivate(context: ExecutionContext) {
12 |     const requiredRoles = this.reflector.getAllAndOverride<UserRole[]>(
13 |       ROLES_KEY,
14 |       [context.getHandler(), context.getClass()],
15 |     );
16 |
17 |     if (!requiredRoles?.length) {
18 |       return true;
19 |     }
20 |
21 |     const request = context.switchToHttp().getRequest<AuthenticatedRequest>();
22 |     return Boolean(
23 |       request.user?.role &&
24 |       requiredRoles.includes(request.user.role as UserRole),
25 |     );
26 |   }
27 | }
```

edupulse-backend/src/dashboard/dashboard.controller.ts

File size: 881 bytes

```
1 | import { Controller, Get, Query, Req, UseGuards } from '@nestjs/common';
2 | import { ApiTags } from '@nestjs/swagger';
3 | import { UserRole } from '@prisma/client';
4 | import { Roles } from '../auth/decorators/roles.decorator';
5 | import type { AuthenticatedRequest } from '../auth/guards/jwt-auth.guard';
6 | import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
7 | import { RolesGuard } from '../auth/guards/roles.guard';
8 | import { DashboardService } from '../dashboard.service';
9 |
10 | @ApiTags('dashboard')
11 | @UseGuards(JwtAuthGuard, RolesGuard)
12 | @Controller('dashboard')
13 | export class DashboardController {
14 |   constructor(private readonly dashboardService: DashboardService) {}
15 |
16 |   @Get('summary')
17 |   @Roles(UserRole.ADMIN)
18 |   summary(
19 |     @Req() request: AuthenticatedRequest,
20 |     @Query('termId') termId?: string,
21 |   ) {
22 |     return this.dashboardService.summary(request.user!.collegeId, termId);
23 |   }
24 | }
```

edupulse-backend/src/dashboard/dashboard.module.ts

File size: 276 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { DashboardController } from '../dashboard.controller';
3 | import { DashboardService } from '../dashboard.service';
4 |
5 | @Module({
6 |   controllers: [DashboardController],
7 |   providers: [DashboardService],
8 | })
9 | export class DashboardModule {}
```


File size: 5,603 bytes

```

1 | import { Injectable } from '@nestjs/common';
2 | import { PrismaService } from '../prisma/prisma.service';
3 |
4 | @Injectable()
5 | export class DashboardService {
6 |   constructor(private readonly prisma: PrismaService) {}
7 |
8 |   async summary(collegeId: string, termId?: string) {
9 |     const activeTerm =
10 |       termId === undefined
11 |         ? await this.prisma.academicTerm.findFirst({
12 |             where: { collegeId, isActive: true },
13 |             orderBy: { createdAt: 'desc' },
14 |           })
15 |         : null;
16 |     const selectedTermId = termId ?? activeTerm?.id;
17 |     const selectedTerm = await this.prisma.academicTerm.findFirst({
18 |       where: { id: selectedTermId, collegeId },
19 |     });
20 |     const previousTerm = selectedTerm
21 |       ? await this.prisma.academicTerm.findFirst({
22 |         where: {
23 |           collegeId,
24 |           id: { not: selectedTerm.id },
25 |           OR: [
26 |             { startsAt: selectedTerm.startsAt },
27 |             { endsAt: { lt: selectedTerm.startsAt },
28 |               createdAt: { lt: selectedTerm.createdAt } },
29 |           ],
30 |           : [{ createdAt: { lt: selectedTerm.createdAt } }],
31 |         },
32 |         orderBy: [{ endsAt: 'desc' }, { createdAt: 'desc' }],
33 |       })
34 |       : null;
35 |     const feedbackWhere = selectedTermId
36 |       ? { collegeId, submission: { termId: selectedTermId } }
37 |       : { collegeId };
38 |     const previousFeedbackWhere = previousTerm
39 |       ? { collegeId, submission: { termId: previousTerm.id } }
40 |       : undefined;
41 |
42 |     const [
43 |       responseCount,
44 |       submissionCount,
45 |       categoryGroups,
46 |       previousCategoryGroups,
47 |       ratingGroups,
48 |       categoryRatingGroups,
49 |       latestReport,
50 |       actionGroups,
51 |     ] = await Promise.all([
52 |       this.prisma.feedbackResponse.count({ where: feedbackWhere }),
53 |       this.prisma.feedbackSubmission.count({
54 |         where: selectedTermId
55 |           ? { collegeId, termId: selectedTermId }
56 |           : { collegeId },
57 |       }),
58 |       this.prisma.feedbackResponse.groupBy({
59 |         by: ['categoryId'],
60 |         where: feedbackWhere,
61 |         _avg: { rating: true },
62 |         _count: { id: true },
63 |       }),
64 |       previousFeedbackWhere
65 |         ? this.prisma.feedbackResponse.groupBy({
66 |             by: ['categoryId'],
67 |             where: previousFeedbackWhere,
68 |             _avg: { rating: true },
69 |             _count: { id: true },
70 |           })
71 |         : Promise.resolve([]),
72 |       this.prisma.feedbackResponse.groupBy({
73 |         by: ['rating'],
74 |         where: feedbackWhere,
75 |         _count: { id: true },
76 |       }),
77 |       this.prisma.feedbackResponse.groupBy({
78 |         by: ['categoryId', 'rating'],
79 |         where: feedbackWhere,
80 |         _count: { id: true },
81 |       }),
82 |       this.prisma.analysisReport.findFirst({
83 |         where: selectedTermId
84 |           ? { collegeId, termId: selectedTermId }
85 |           : { collegeId },
86 |         orderBy: { createdAt: 'desc' },
87 |         include: {
88 |           themes: {
89 |             orderBy: [{ priority: 'desc' }, { mentionCount: 'desc' }],
90 |             take: 5,
91 |           },
92 |           actions: {
93 |             orderBy: [{ priority: 'desc' }, { createdAt: 'desc' }],
94 |             take: 5,
95 |           },
96 |         },
97 |       }),
98 |     ],
99 |     );
100 |     this.prisma.actionItem.groupBy({
101 |       by: ['status'],
102 |       where: { collegeId },
103 |       _count: { id: true },
104 |     });
105 |   }

```

```

106 |
107 | const categories = await this.prisma.feedbackCategory.findMany({
108 |   where: {
109 |     collegeId,
110 |     id: { in: categoryGroups.map((group) => group.categoryId) },
111 |   },
112 |   select: { id: true, name: true },
113 | });
114 | const categoryNameById = new Map(
115 |   categories.map((category) => [category.id, category.name]),
116 | );
117 |
118 | return {
119 |   termId: selectedTermId,
120 |   responseCount,
121 |   submissionCount,
122 |   categorySatisfaction: categoryGroups.map((group) => ({
123 |     categoryId: group.categoryId,
124 |     categoryName: categoryNameById.get(group.categoryId) ?? 'Unknown',
125 |     averageRating: Number((group._avg.rating ?? 0).toFixed(2)),
126 |     responseCount: group._count.id,
127 |   })),
128 |   previousTerm: previousTerm
129 |   ? {
130 |     id: previousTerm.id,
131 |     name: previousTerm.name,
132 |   }
133 |   : null,
134 |   previousCategorySatisfaction: previousCategoryGroups.map((group) => ({
135 |     categoryId: group.categoryId,
136 |     categoryName: categoryNameById.get(group.categoryId) ?? 'Unknown',
137 |     averageRating: Number((group._avg.rating ?? 0).toFixed(2)),
138 |     responseCount: group._count.id,
139 |   })),
140 |   ratingDistribution: this.toRatingDistribution(ratingGroups),
141 |   categoryRatingDistribution: categoryGroups.map((group) => ({
142 |     categoryId: group.categoryId,
143 |     categoryName: categoryNameById.get(group.categoryId) ?? 'Unknown',
144 |     distribution: this.toRatingDistribution(
145 |       categoryRatingGroups.filter((item) => item.categoryId === group.categoryId),
146 |     ),
147 |   })),
148 |   actionByStatus: actionGroups.map((group) => ({
149 |     status: group.status,
150 |     count: group._count.id,
151 |   })),
152 |   latestReport,
153 | };
154 | }
155 |
156 | private toRatingDistribution(
157 |   groups: { rating: number; _count: { id: number } }[],
158 | ) {
159 |   const unsatisfied = groups
160 |     .filter((group) => group.rating <= 2)
161 |     .reduce((total, group) => total + group._count.id, 0);
162 |   const average = groups
163 |     .filter((group) => group.rating === 3)
164 |     .reduce((total, group) => total + group._count.id, 0);
165 |   const satisfied = groups
166 |     .filter((group) => group.rating >= 4)
167 |     .reduce((total, group) => total + group._count.id, 0);
168 |
169 |   return { unsatisfied, average, satisfied };
170 | }
171 | }

```

edupulse-backend/src/feedback/dto/admin-feedback-form.dto.ts

File size: 582 bytes

```
1 | import { Type } from 'class-transformer';
2 | import {
3 |   ArrayMinSize,
4 |   IsArray,
5 |   IsOptional,
6 |   IsString,
7 |   ValidateNested,
8 | } from 'class-validator';
9 |
10 | export class AdminFeedbackCategoryDto {
11 |   @IsString()
12 |   name: string;
13 |
14 |   @IsOptional()
15 |   @IsString()
16 |   description?: string;
17 |
18 |   @IsArray()
19 |   @ArrayMinSize(1)
20 |   @IsString({ each: true })
21 |   questions: string[];
22 | }
23 |
24 | export class AdminFeedbackFormDto {
25 |   @IsString()
26 |   termName: string;
27 |
28 |   @IsArray()
29 |   @ArrayMinSize(1)
30 |   @ValidateNested({ each: true })
31 |   @Type(() => AdminFeedbackCategoryDto)
32 |   categories: AdminFeedbackCategoryDto[];
33 | }
```

edupulse-backend/src/feedback/dto/submit-feedback.dto.ts

File size: 804 bytes

```
1 | import { ApiProperty } from '@nestjs/swagger';
2 | import {
3 |   ArrayMinSize,
4 |   IsArray,
5 |   IsInt,
6 |   IsOptional,
7 |   IsString,
8 |   Max,
9 |   Min,
10 |   ValidateNested,
11 | } from 'class-validator';
12 | import { Type } from 'class-transformer';
13 |
14 | export class FeedbackAnswerDto {
15 |   @ApiProperty()
16 |   @IsString()
17 |   questionId: string;
18 |
19 |   @ApiProperty()
20 |   @IsString()
21 |   categoryId: string;
22 |
23 |   @ApiProperty({ minimum: 1, maximum: 4 })
24 |   @IsInt()
25 |   @Min(1)
26 |   @Max(4)
27 |   rating: number;
28 |
29 |   @ApiProperty({ required: false })
30 |   @IsOptional()
31 |   @IsString()
32 |   comment?: string;
33 | }
34 |
35 | export class SubmitFeedbackDto {
36 |   @ApiProperty()
37 |   @IsString()
38 |   termId: string;
39 |
40 |   @ApiProperty({ type: [FeedbackAnswerDto] })
41 |   @IsArray()
42 |   @ArrayMinSize(1)
43 |   @ValidateNested({ each: true })
44 |   @Type(() => FeedbackAnswerDto)
45 |   answers: FeedbackAnswerDto[];
46 | }
```

edupulse-backend/src/feedback/feedback.controller.ts

File size: 2,127 bytes

```
1 | import { Body, Controller, Get, Post, Req, UseGuards } from '@nestjs/common';
2 | import { ApiTags } from '@nestjs/swagger';
3 | import { UserRole } from '@prisma/client';
4 | import { Roles } from '../auth/decorators/roles.decorator';
5 | import type { AuthenticatedRequest } from '../auth/guards/jwt-auth.guard';
6 | import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
7 | import { RolesGuard } from '../auth/guards/roles.guard';
8 | import { FeedbackService } from '../feedback.service';
9 | import { AdminFeedbackFormDto } from '../dto/admin-feedback-form.dto';
10 | import { SubmitFeedbackDto } from '../dto/submit-feedback.dto';
11 |
12 | @ApiTags('feedback')
13 | @UseGuards(JwtAuthGuard, RolesGuard)
14 | @Controller('feedback')
15 | export class FeedbackController {
16 |   constructor(private readonly feedbackService: FeedbackService) {}
17 |
18 |   @Get('categories')
19 |   @Roles(UserRole.STUDENT, UserRole.ADMIN)
20 |   getCategories(@Req() request: AuthenticatedRequest) {
21 |     return this.feedbackService.getCategories(request.user!.collegeId);
22 |   }
23 |
24 |   @Get('active-term')
25 |   @Roles(UserRole.STUDENT, UserRole.ADMIN)
26 |   getActiveTerm(@Req() request: AuthenticatedRequest) {
27 |     return this.feedbackService.getActiveTerm(request.user!.collegeId);
28 |   }
29 |
30 |   @Get('admin/terms')
31 |   @Roles(UserRole.ADMIN)
32 |   getAdminTerms(@Req() request: AuthenticatedRequest) {
33 |     return this.feedbackService.getAdminTerms(request.user!.collegeId);
34 |   }
35 |
36 |   @Post('submit')
37 |   @Roles(UserRole.STUDENT)
38 |   submit(@Req() request: AuthenticatedRequest, @Body() dto: SubmitFeedbackDto) {
39 |     return this.feedbackService.submit(
40 |       request.user!.sub,
41 |       request.user!.collegeId,
42 |       dto,
43 |     );
44 |   }
45 |
46 |   @Post('admin/form')
47 |   @Roles(UserRole.ADMIN)
48 |   createOrPublishForm(
49 |     @Req() request: AuthenticatedRequest,
50 |     @Body() dto: AdminFeedbackFormDto,
51 |   ) {
52 |     return this.feedbackService.createOrPublishForm(
53 |       request.user!.sub,
54 |       request.user!.collegeId,
55 |       dto,
56 |     );
57 |   }
58 |
59 |   @Post('admin/turn-off')
60 |   @Roles(UserRole.ADMIN)
61 |   turnOffFeedback(@Req() request: AuthenticatedRequest) {
62 |     return this.feedbackService.turnOffFeedback(
63 |       request.user!.sub,
64 |       request.user!.collegeId,
65 |     );
66 |   }
67 | }
```

edupulse-backend/src/feedback/feedback.module.ts

File size: 348 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { AuditModule } from '../audit/audit.module';
3 | import { FeedbackController } from '../feedback.controller';
4 | import { FeedbackService } from '../feedback.service';
5 |
6 | @Module({
7 |   imports: [AuditModule],
8 |   controllers: [FeedbackController],
9 |   providers: [FeedbackService],
10 | })
11 | export class FeedbackModule {}
```

edupulse-backend/src/feedback/feedback.service.ts

File size: 7,287 bytes

```
1 | import { BadRequestException, Injectable } from '@nestjs/common';
2 | import { AuditAction, FeedbackSubmissionStatus } from '@prisma/client';
3 | import { AuditService } from '../audit/audit.service';
4 | import { PrismaService } from '../prisma/prisma.service';
5 | import { AdminFeedbackFormDto } from '../dto/admin-feedback-form.dto';
6 | import { SubmitFeedbackDto } from '../dto/submit-feedback.dto';
7 |
8 | @Injectable()
9 | export class FeedbackService {
10 |   constructor(
11 |     private readonly prisma: PrismaService,
12 |     private readonly auditService: AuditService,
13 |   ) {}
14 |
15 |   getCategories(collegeId: string) {
16 |     return this.prisma.feedbackCategory.findMany({
17 |       where: { collegeId, questions: { some: { isActive: true } } },
18 |       include: { questions: { where: { isActive: true } } },
19 |       orderBy: { name: 'asc' },
20 |     });
21 |   }
22 |
23 |   getActiveTerm(collegeId: string) {
24 |     return this.prisma.academicTerm.findFirst({
25 |       where: { collegeId, isActive: true },
26 |       orderBy: { createdAt: 'desc' },
27 |     });
28 |   }
29 |
30 |   getAdminTerms(collegeId: string) {
31 |     return this.prisma.academicTerm.findMany({
32 |       where: { collegeId },
33 |       orderBy: [{ isActive: 'desc' }, { startsAt: 'desc' }, { createdAt: 'desc' }],
34 |       select: {
35 |         id: true,
36 |         name: true,
37 |         isActive: true,
38 |         startsAt: true,
39 |         endsAt: true,
40 |       },
41 |     });
42 |   }
43 |
44 |   async submit(studentId: string, collegeId: string, dto: SubmitFeedbackDto) {
45 |     const submission = await this.prisma.$transaction(async (tx) => {
46 |       const term = await tx.academicTerm.findFirst({
47 |         where: { id: dto.termId, collegeId, isActive: true },
48 |         select: { id: true },
49 |       });
50 |
51 |       if (!term) {
52 |         throw new BadRequestException('Active feedback term not found');
53 |       }
54 |
55 |       const student = await tx.user.findFirst({
56 |         where: { id: studentId, collegeId },
57 |         select: { id: true },
58 |       });
59 |
60 |       if (!student) {
61 |         throw new BadRequestException(
62 |           'Student does not belong to this college',
63 |         );
64 |       }
65 |
66 |       const questionIds = dto.answers.map((answer) => answer.questionId);
67 |       const validQuestionCount = await tx.feedbackQuestion.count({
68 |         where: { id: { in: questionIds }, collegeId, isActive: true },
69 |       });
70 |
71 |       if (validQuestionCount !== new Set(questionIds).size) {
72 |         throw new BadRequestException('Feedback contains invalid question(s)');
73 |       }
74 |
75 |       const existingSubmission = await tx.feedbackSubmission.findUnique({
76 |         where: { studentId_termId: { studentId, termId: dto.termId } },
77 |       });
78 |
79 |       if (existingSubmission) {
80 |         await tx.feedbackResponse.deleteMany({
81 |           where: { submissionId: existingSubmission.id },
82 |         });
83 |
84 |         return tx.feedbackSubmission.update({
85 |           where: { id: existingSubmission.id },
86 |           data: {
87 |             status: FeedbackSubmissionStatus.SUBMITTED,
88 |             responses: {
89 |               create: dto.answers.map((answer) => ({
90 |                 collegeId,
91 |                 questionId: answer.questionId,
92 |                 categoryId: answer.categoryId,
93 |                 rating: answer.rating,
94 |                 comment: answer.comment,
95 |               })),
96 |             },
97 |           },
98 |           include: { responses: true },
99 |         });
100 |       }
101 |
102 |       return tx.feedbackSubmission.create({
103 |         data: {
104 |           collegeId,
105 |           studentId,
```

```

106 |         termId: dto.termId,
107 |         responses: {
108 |             create: dto.answers.map((answer) => ({
109 |                 collegeId,
110 |                 questionId: answer.questionId,
111 |                 categoryId: answer.categoryId,
112 |                 rating: answer.rating,
113 |                 comment: answer.comment,
114 |             })),
115 |         },
116 |         include: { responses: true },
117 |     });
118 | });
119 | });
120 |
121 | await this.auditService.log({
122 |     action: AuditAction.FEEDBACK_SUBMITTED,
123 |     actorId: studentId,
124 |     collegeId,
125 |     entity: 'FeedbackSubmission',
126 |     entityId: submission.id,
127 |     metadata: {
128 |         termId: dto.termId,
129 |         answerCount: dto.answers.length,
130 |     },
131 | });
132 |
133 | return submission;
134 | }
135 |
136 | async createOrPublishForm(
137 |     adminId: string,
138 |     collegeId: string,
139 |     dto: AdminFeedbackFormDto,
140 | ) {
141 |     const result = await this.prisma.$transaction(async (tx) => {
142 |         await tx.academicTerm.updateMany({
143 |             where: { collegeId },
144 |             data: { isActive: false },
145 |         });
146 |         await tx.feedbackQuestion.updateMany({
147 |             where: { collegeId },
148 |             data: { isActive: false },
149 |         });
150 |
151 |         const term = await tx.academicTerm.upsert({
152 |             where: { collegeId_name: { collegeId, name: dto.termName } },
153 |             update: { isActive: true },
154 |             create: {
155 |                 collegeId,
156 |                 name: dto.termName,
157 |                 startsAt: new Date(),
158 |                 isActive: true,
159 |             },
160 |         });
161 |
162 |         for (const categoryInput of dto.categories) {
163 |             const category = await tx.feedbackCategory.upsert({
164 |                 where: {
165 |                     collegeId_name: { collegeId, name: categoryInput.name },
166 |                 },
167 |                 update: { description: categoryInput.description },
168 |                 create: {
169 |                     collegeId,
170 |                     name: categoryInput.name,
171 |                     description: categoryInput.description,
172 |                 },
173 |             });
174 |
175 |             for (const text of categoryInput.questions) {
176 |                 const trimmedText = text.trim();
177 |                 if (!trimmedText) continue;
178 |
179 |                 const existingQuestion = await tx.feedbackQuestion.findFirst({
180 |                     where: { collegeId, categoryId: category.id, text: trimmedText },
181 |                 });
182 |
183 |                 if (existingQuestion) {
184 |                     await tx.feedbackQuestion.update({
185 |                         where: { id: existingQuestion.id },
186 |                         data: { isActive: true },
187 |                     });
188 |                 } else {
189 |                     await tx.feedbackQuestion.create({
190 |                         data: {
191 |                             collegeId,
192 |                             categoryId: category.id,
193 |                             text: trimmedText,
194 |                             isActive: true,
195 |                         },
196 |                     });
197 |                 }
198 |             }
199 |         }
200 |
201 |         return {
202 |             term,
203 |             categories: await tx.feedbackCategory.findMany({
204 |                 where: { collegeId, questions: { some: { isActive: true } } },
205 |                 include: { questions: { where: { isActive: true } } },
206 |                 orderBy: { name: 'asc' },
207 |             }),
208 |         };
209 |     });
210 |
211 |     await this.auditService.log({
212 |         action: AuditAction.FEEDBACK_FORM_PUBLISHED,
213 |         actorId: adminId,
214 |         collegeId,
215 |         entity: 'AcademicTerm',

```



```

216 |         entityId: result.term.id,
217 |         metadata: {
218 |             termName: result.term.name,
219 |             categoryCount: result.categories.length,
220 |             questionCount: result.categories.reduce(
221 |                 (count, category) => count + category.questions.length,
222 |                 0,
223 |             ),
224 |         },
225 |     });
226 |
227 |     return result;
228 | }
229 |
230 | async turnOffFeedback(adminId: string, collegeId: string) {
231 |     const updated = await this.prisma.academicTerm.updateMany({
232 |         where: { collegeId },
233 |         data: { isActive: false },
234 |     });
235 |
236 |     await this.auditService.log({
237 |         action: AuditAction.FEEDBACK_FORM_TURNED_OFF,
238 |         actorId: adminId,
239 |         collegeId,
240 |         entity: 'AcademicTerm',
241 |         metadata: {
242 |             disabledTerms: updated.count,
243 |         },
244 |     });
245 |
246 |     return {
247 |         status: 'OFF',
248 |         message: 'Semester feedback is turned off for students.',
249 |     };
250 | }
251 | }

```

edupulse-backend/src/health/health.controller.ts

File size: 312 bytes

```
1 | import { Controller, Get } from '@nestjs/common';
2 | import { ApiTags } from '@nestjs/swagger';
3 |
4 | @ApiTags('health')
5 | @Controller('health')
6 | export class HealthController {
7 |   @Get()
8 |   check() {
9 |     return {
10 |       status: 'ok',
11 |       service: 'edupulse-backend',
12 |       timestamp: new Date().toISOString(),
13 |     };
14 |   }
15 | }
```

edupulse-backend/src/health/health.module.ts

File size: 175 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { HealthController } from '../health.controller';
3 |
4 | @Module({
5 |   controllers: [HealthController],
6 | })
7 | export class HealthModule {}
```

edupulse-backend/src/main.ts

File size: 875 bytes

```
1 | import { NestFactory } from '@nestjs/core';
2 | import { ValidationPipe } from '@nestjs/common';
3 | import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
4 | import { AppModule } from './app.module';
5 |
6 | async function bootstrap() {
7 |   const app = await NestFactory.create(AppModule);
8 |   app.setGlobalPrefix('api/v1');
9 |   app.enableCors();
10 |   app.useGlobalPipes(
11 |     new ValidationPipe({
12 |       whitelist: true,
13 |       transform: true,
14 |       forbidNonWhitelisted: true,
15 |     }),
16 |   );
17 |
18 |   const swaggerConfig = new DocumentBuilder()
19 |     .setTitle('EduPulse AI Backend')
20 |     .setDescription('Feedback, AI analysis, audit and admin action APIs')
21 |     .setVersion('0.1.0')
22 |     .addBearerAuth()
23 |     .build();
24 |   const document = SwaggerModule.createDocument(app, swaggerConfig);
25 |   SwaggerModule.setup('docs', app, document);
26 |
27 |   await app.listen(process.env.PORT ?? 4000);
28 | }
29 | void bootstrap();
```

edupulse-backend/src/prisma/prisma.module.ts

File size: 210 bytes

```
1 | import { Global, Module } from '@nestjs/common';
2 | import { PrismaService } from '../prisma.service';
3 |
4 | @Global()
5 | @Module({
6 |   providers: [PrismaService],
7 |   exports: [PrismaService],
8 | })
9 | export class PrismaModule {}
```

edupulse-backend/src/prisma/prisma.service.ts

File size: 659 bytes

```
1 | import { Injectable, OnModuleDestroy, OnModuleInit } from '@nestjs/common';
2 | import { PrismaPg } from '@prisma/adapter-pg';
3 | import { PrismaClient } from '@prisma/client';
4 |
5 | @Injectable()
6 | export class PrismaService
7 |   extends PrismaClient
8 |   implements OnModuleInit, OnModuleDestroy
9 | {
10 |   constructor() {
11 |     const connectionString = process.env.DATABASE_URL;
12 |
13 |     if (!connectionString) {
14 |       throw new Error('DATABASE_URL is required to initialize Prisma');
15 |     }
16 |
17 |     super({
18 |       adapter: new PrismaPg({ connectionString }),
19 |     });
20 |   }
21 |
22 |   async onModuleInit() {
23 |     await this.$connect();
24 |   }
25 |
26 |   async onModuleDestroy() {
27 |     await this.$disconnect();
28 |   }
29 | }
```

edupulse-backend/src/reports/reports.controller.ts

File size: 1,581 bytes

```
1 | import {
2 |   Controller,
3 |   Get,
4 |   Header,
5 |   Param,
6 |   Query,
7 |   Req,
8 |   StreamableFile,
9 |   UseGuards,
10 | } from '@nestjs/common';
11 | import { ApiTags } from '@nestjs/swagger';
12 | import { UserRole } from '@prisma/client';
13 | import { Roles } from '../auth/decorators/roles.decorator';
14 | import type { AuthenticatedRequest } from '../auth/guards/jwt-auth.guard';
15 | import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
16 | import { RolesGuard } from '../auth/guards/roles.guard';
17 | import { ReportsService } from './reports.service';
18 |
19 | @ApiTags('reports')
20 | @UseGuards(JwtAuthGuard, RolesGuard)
21 | @Controller('reports')
22 | export class ReportsController {
23 |   constructor(private readonly reportsService: ReportsService) {}
24 |
25 |   @Get()
26 |   @Roles(UserRole.ADMIN)
27 |   listReports(
28 |     @Req() request: AuthenticatedRequest,
29 |     @Query('termId') termId?: string,
30 |   ) {
31 |     return this.reportsService.listReports(request.user!.collegeId, termId);
32 |   }
33 |
34 |   @Get('/:id')
35 |   @Roles(UserRole.ADMIN)
36 |   getReport(@Req() request: AuthenticatedRequest, @Param('id') id: string) {
37 |     return this.reportsService.getReport(
38 |       request.user!.collegeId,
39 |       id,
40 |       request.user!.sub,
41 |     );
42 |   }
43 |
44 |   @Get('/:id/pdf')
45 |   @Header('Content-Type', 'application/pdf')
46 |   @Roles(UserRole.ADMIN)
47 |   async downloadPdf(
48 |     @Req() request: AuthenticatedRequest,
49 |     @Param('id') id: string,
50 |   ) {
51 |     const pdf = await this.reportsService.createPdf(
52 |       request.user!.collegeId,
53 |       id,
54 |       request.user!.sub,
55 |     );
56 |     return new StreamableFile(pdf, {
57 |       disposition: 'attachment; filename="edupulse-report-${id}.pdf"',
58 |     });
59 |   }
60 | }
```

edupulse-backend/src/reports/reports.module.ts

File size: 341 bytes

```
1 | import { Module } from '@nestjs/common';
2 | import { AuditModule } from '../audit/audit.module';
3 | import { ReportsController } from './reports.controller';
4 | import { ReportsService } from './reports.service';
5 |
6 | @Module({
7 |   imports: [AuditModule],
8 |   controllers: [ReportsController],
9 |   providers: [ReportsService],
10 | })
11 | export class ReportsModule {}
```


File size: 39,416 bytes

```

1 | import { Injectable, NotFoundException } from '@nestjs/common';
2 | import PDFDocument from 'pdfkit';
3 | import { AuditAction } from '@prisma/client';
4 | import { AuditService } from '../audit/audit.service';
5 | import { PrismaService } from '../prisma/prisma.service';
6 |
7 | const COLORS = {
8 |   amber: '#f59e0b',
9 |   blue: '#2563eb',
10 |   blueSoft: '#eff6ff',
11 |   border: '#e2e8f0',
12 |   canvas: '#f7f9fc',
13 |   danger: '#dc2626',
14 |   dangerSoft: '#fee2e2',
15 |   green: '#10b981',
16 |   greenSoft: '#ecfdf5',
17 |   ink: '#0f172a',
18 |   muted: '#64748b',
19 |   panel: '#ffffff',
20 |   slateSoft: '#f8fafc',
21 | };
22 |
23 | type PdfIssue = {
24 |   affectedStudents: string;
25 |   evidenceSignal: string;
26 |   issue: string;
27 |   mentions: number;
28 |   priority: string;
29 |   rootCause: string;
30 |   action: string;
31 | };
32 |
33 | type PdfActionReport = {
34 |   actionPlan: {
35 |     body: string;
36 |     priority: string;
37 |     steps: string[];
38 |     title: string;
39 |     why: string;
40 |   }[];
41 |   graph: {
42 |     category: string;
43 |     current: number;
44 |     previous: number;
45 |   }[];
46 |   issues: PdfIssue[];
47 |   metrics: {
48 |     label: string;
49 |     value: string;
50 |   }[];
51 |   rootCauses: {
52 |     body: string;
53 |     signal: string;
54 |     title: string;
55 |   }[];
56 |   summary: string;
57 | };
58 |
59 | type PdfEngineBenchmark = {
60 |   funnel: {
61 |     label: string;
62 |     value: string;
63 |     note: string;
64 |   }[];
65 |   receipt: {
66 |     label: string;
67 |     value: string;
68 |   }[];
69 |   pipeline: string[];
70 | };
71 |
72 | @Injectable()
73 | export class ReportsService {
74 |   constructor(
75 |     private readonly prisma: PrismaService,
76 |     private readonly auditService: AuditService,
77 |   ) {}
78 |
79 |   listReports(collegeId: string, termId?: string) {
80 |     return this.prisma.analysisReport.findMany({
81 |       where: termId ? { collegeId, termId } : { collegeId },
82 |       include: {
83 |         _count: {
84 |           select: { themes: true, actions: true },
85 |         },
86 |       },
87 |       orderBy: { createdAt: 'desc' },
88 |     });
89 |   }
90 |
91 |   async getReport(collegeId: string, id: string, actorId?: string) {
92 |     const report = await this.prisma.analysisReport.findFirst({
93 |       where: { id, collegeId },
94 |       include: {
95 |         term: true,
96 |         themes: {
97 |           include: { category: true },
98 |           orderBy: [{ priority: 'desc' }, { mentionCount: 'desc' }],
99 |         },
100 |         actions: { orderBy: [{ priority: 'desc' }, { createdAt: 'desc' }] },
101 |       },
102 |     });
103 |
104 |     if (!report) {
105 |       throw new NotFoundException('Report not found');

```

```

106 |     }
107 |
108 |     if (actorId) {
109 |         await this.auditService.log({
110 |             action: AuditAction.REPORT_VIEWED,
111 |             actorId,
112 |             collegeId,
113 |             entity: 'AnalysisReport',
114 |             entityId: report.id,
115 |             metadata: {
116 |                 termId: report.termId,
117 |                 inputCount: report.inputCount,
118 |                 confidence: report.confidence,
119 |             },
120 |         });
121 |     }
122 |
123 |     return report;
124 | }
125 |
126 | async createPdf(collegeId: string, id: string, actorId?: string) {
127 |     const report = await this.getReport(collegeId, id);
128 |     const actionReport = this.buildActionReport(report);
129 |     const engineBenchmark = this.buildEngineBenchmark(report);
130 |
131 |     return new Promise<Buffer>((resolve) => {
132 |         const document = new PDFDocument({
133 |             layout: 'landscape',
134 |             margin: 32,
135 |             size: 'A4',
136 |         });
137 |         const chunks: Buffer[] = [];
138 |
139 |         document.on('data', (chunk: Buffer) => chunks.push(chunk));
140 |         document.on('end', () => resolve(Buffer.concat(chunks)));
141 |
142 |         this.drawFirstPage(document, report, actionReport);
143 |         document.addPage();
144 |         this.drawSecondPage(document, report, actionReport);
145 |         document.addPage();
146 |         this.drawThirdPage(document, report, actionReport);
147 |         document.addPage();
148 |         this.drawBenchmarkPage(document, report, engineBenchmark);
149 |
150 |         document.end();
151 |         void this.auditService.log({
152 |             action: AuditAction.REPORT_DOWNLOADED,
153 |             actorId,
154 |             collegeId,
155 |             entity: 'AnalysisReport',
156 |             entityId: report.id,
157 |             metadata: {
158 |                 termId: report.termId,
159 |                 inputCount: report.inputCount,
160 |                 confidence: report.confidence,
161 |             },
162 |         });
163 |     });
164 | }
165 |
166 | private drawFirstPage(
167 |     document: PDFKit.PDFDocument,
168 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
169 |     actionReport: PdfActionReport,
170 | ) {
171 |     this.drawPageBackground(document);
172 |     this.drawHeader(document, report);
173 |     this.drawExecutiveSummary(document, actionReport);
174 |     this.drawTopIssues(document, actionReport.issues);
175 |     this.drawFooter(document, 1);
176 | }
177 |
178 | private drawSecondPage(
179 |     document: PDFKit.PDFDocument,
180 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
181 |     actionReport: PdfActionReport,
182 | ) {
183 |     this.drawPageBackground(document);
184 |     this.drawSmallHeader(
185 |         document,
186 |         report,
187 |         'Root causes and detailed AI action plan',
188 |     );
189 |     this.drawRootCauses(document, actionReport.rootCauses);
190 |     this.drawActionPlan(document, actionReport.actionPlan);
191 |     this.drawFooter(document, 2);
192 | }
193 |
194 | private drawThirdPage(
195 |     document: PDFKit.PDFDocument,
196 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
197 |     actionReport: PdfActionReport,
198 | ) {
199 |     this.drawPageBackground(document);
200 |     this.drawSmallHeader(
201 |         document,
202 |         report,
203 |         'Semester comparison and expected impact',
204 |     );
205 |     this.drawSemesterLineGraph(document, actionReport.graph);
206 |     this.drawFooter(document, 3);
207 | }
208 |
209 | private drawBenchmarkPage(
210 |     document: PDFKit.PDFDocument,
211 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
212 |     benchmark: PdfEngineBenchmark,
213 | ) {
214 |     this.drawPageBackground(document);
215 |     this.drawSmallHeader(

```

```

216 |         document,
217 |         report,
218 |         'AI engine benchmark and data reduction proof',
219 |     );
220 |     this.drawDataFunnel(document, benchmark);
221 |     this.drawRunReceipt(document, benchmark);
222 |     this.drawPipelineTrace(document, benchmark.pipeline);
223 |     this.drawFooter(document, 4);
224 | }
225 |
226 | private drawPageBackground(document: PDFKit.PDFDocument) {
227 |     document.save();
228 |     document
229 |         .rect(0, 0, document.page.width, document.page.height)
230 |         .fill(COLORS.canvas);
231 |     document.restore();
232 | }
233 |
234 | private drawHeader(
235 |     document: PDFKit.PDFDocument,
236 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
237 | ) {
238 |     document
239 |         .font('Helvetica-Bold')
240 |         .fontSize(9)
241 |         .fillColor(COLORS.blue)
242 |         .text('AI ACTION REPORT', 44, 36);
243 |     document
244 |         .fontSize(28)
245 |         .fillColor(COLORS.ink)
246 |         .text(
247 |             'Institutional improvement plan generated from feedback intelligence',
248 |             44,
249 |             55,
250 |             {
251 |                 width: 620,
252 |             },
253 |         );
254 |     document
255 |         .font('Helvetica')
256 |         .fontSize(9)
257 |         .fillColor(COLORS.muted)
258 |         .text('Term: ${report.term.name}', 660, 46, {
259 |             align: 'right',
260 |             width: 135,
261 |         });
262 |     document.text(
263 |         `Generated: ${report.createdAt.toLocaleDateString()}`,
264 |         660,
265 |         62,
266 |         {
267 |             align: 'right',
268 |             width: 135,
269 |         },
270 |     );
271 |     document
272 |         .moveTo(44, 112)
273 |         .lineTo(document.page.width - 44, 112)
274 |         .strokeColor(COLORS.border)
275 |         .lineWidth(1)
276 |         .stroke();
277 | }
278 |
279 | private drawSmallHeader(
280 |     document: PDFKit.PDFDocument,
281 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
282 |     title: string,
283 | ) {
284 |     document
285 |         .font('Helvetica-Bold')
286 |         .fontSize(10)
287 |         .fillColor(COLORS.blue)
288 |         .text('AI ACTION REPORT', 44, 36);
289 |     document.fontSize(20).fillColor(COLORS.ink).text(title, 44, 54);
290 |     document
291 |         .font('Helvetica')
292 |         .fontSize(9)
293 |         .fillColor(COLORS.muted)
294 |         .text(report.term.name, 650, 50, { align: 'right', width: 145 });
295 | }
296 |
297 | private drawExecutiveSummary(
298 |     document: PDFKit.PDFDocument,
299 |     actionReport: PdfActionReport,
300 | ) {
301 |     const x = 44;
302 |     const y = 132;
303 |     const width = document.page.width - 88;
304 |     const height = 148;
305 |     this.card(document, x, y, width, height, COLORS.blueSoft);
306 |
307 |     document
308 |         .font('Helvetica-Bold')
309 |         .fontSize(9)
310 |         .fillColor(COLORS.blue)
311 |         .text('1. EXECUTIVE AI SUMMARY', x + 16, y + 16);
312 |     document
313 |         .font('Helvetica')
314 |         .fontSize(10.5)
315 |         .fillColor('#334155')
316 |         .text(actionReport.summary, x + 16, y + 36, {
317 |             lineGap: 4,
318 |             width: 475,
319 |         });
320 |
321 |     const metricX = x + 520;
322 |     const metricY = y + 18;
323 |     const metricW = 104;
324 |     const metricH = 48;
325 |     actionReport.metrics.forEach((metric, index) => {

```

```

326 |     const col = index % 2;
327 |     const row = Math.floor(index / 2);
328 |     const cardX = metricX + col * 116;
329 |     const cardY = metricY + row * 62;
330 |     this.card(document, cardX, cardY, metricW, metricH);
331 |     document
332 |       .font('Helvetica-Bold')
333 |       .fontSize(metric.value.length > 12 ? 13 : 16)
334 |       .fillColor(COLORS.ink)
335 |       .text(metric.value, cardX + 10, cardY + 12, { width: metricW - 20 });
336 |     document
337 |       .fontSize(6.8)
338 |       .fillColor(COLORS.muted)
339 |       .text(metric.label.toUpperCase(), cardX + 10, cardY + 31, {
340 |         width: metricW - 20,
341 |       });
342 |   });
343 | }
344 |
345 | private drawTopIssues(document: PDFKit.PDFDocument, issues: PdfIssue[]) {
346 |   const x = 44;
347 |   const y = 302;
348 |   const width = document.page.width - 88;
349 |   const height = 214;
350 |   this.card(document, x, y, width, height);
351 |   document
352 |     .font('Helvetica-Bold')
353 |     .fontSize(9)
354 |     .fillColor(COLORS.blue)
355 |     .text('2. TOP ISSUES FOUND', x + 16, y + 16);
356 |   this.badge(
357 |     document,
358 |     'Priority assigned by LLM after EduPulse clustering',
359 |     x + width - 235,
360 |     y + 12,
361 |     214,
362 |     COLORS.blueSoft,
363 |     COLORS.blue,
364 |   );
365 |
366 |   const columns = [0, 180, 275, 350, 485];
367 |   const columnLabels = [
368 |     'Issue',
369 |     'Priority by AI',
370 |     'Mentions',
371 |     'Affected Students',
372 |     'Evidence Signal',
373 |   ];
374 |   const tableX = x + 16;
375 |   const tableY = y + 52;
376 |   document
377 |     .roundedRect(tableX, tableY, width - 32, 28, 6)
378 |     .fillAndStroke(COLORS.slateSoft, COLORS.border);
379 |   columnLabels.forEach((label, index) => {
380 |     document
381 |       .font('Helvetica-Bold')
382 |       .fontSize(7.3)
383 |       .fillColor('#475569')
384 |       .text(label.toUpperCase(), tableX + columns[index] + 8, tableY + 10, {
385 |         width: index === 4 ? 245 : columns[index + 1] - columns[index] - 12,
386 |       });
387 |   });
388 |
389 |   issues.slice(0, 4).forEach((issue, index) => {
390 |     const rowY = tableY + 28 + index * 32;
391 |     document
392 |       .moveTo(tableX, rowY)
393 |       .lineTo(tableX + width - 32, rowY)
394 |       .strokeColor(COLORS.border)
395 |       .lineWidth(0.6)
396 |       .stroke();
397 |     document
398 |       .font('Helvetica-Bold')
399 |       .fontSize(9)
400 |       .fillColor(COLORS.ink)
401 |       .text(issue.issue, tableX + 8, rowY + 10, { width: 165 });
402 |     this.priorityBadge(
403 |       document,
404 |       issue.priority,
405 |       tableX + columns[1] + 8,
406 |       rowY + 8,
407 |     );
408 |     document
409 |       .font('Helvetica')
410 |       .fontSize(9)
411 |       .fillColor('#334155')
412 |       .text(
413 |         issue.mentions.toLocaleString(),
414 |         tableX + columns[2] + 8,
415 |         rowY + 10,
416 |         {
417 |           width: 65,
418 |         },
419 |       );
420 |     document.text(
421 |       issue.affectedStudents,
422 |       tableX + columns[3] + 8,
423 |       rowY + 10,
424 |       {
425 |         width: 120,
426 |       },
427 |     );
428 |     document
429 |       .fontSize(8.2)
430 |       .fillColor('#475569')
431 |       .text(issue.evidenceSignal, tableX + columns[4] + 8, rowY + 8, {
432 |         lineGap: 1,
433 |         width: 235,
434 |       });
435 |   });

```

```

436 | }
437 |
438 | private drawRootCauses(
439 |   document: PDFKit.PDFDocument,
440 |   rootCauses: PdfActionReport['rootCauses'],
441 | ) {
442 |   const x = 44;
443 |   const y = 96;
444 |   const width = document.page.width - 88;
445 |   const height = 146;
446 |   this.card(document, x, y, width, height);
447 |   document
448 |     .font('Helvetica-Bold')
449 |     .fontSize(9)
450 |     .fillColor(COLORS.blue)
451 |     .text('3. ROOT CAUSE FROM STUDENT COMMENTS', x + 16, y + 16);
452 |   document
453 |     .font('Helvetica')
454 |     .fontSize(8)
455 |     .fillColor(COLORS.muted)
456 |     .text('Comment-based only', x + width - 110, y + 16, {
457 |       align: 'right',
458 |       width: 90,
459 |     });
460 |
461 |   rootCauses.slice(0, 4).forEach((cause, index) => {
462 |     const cardWidth = (width - 56) / 4;
463 |     const cardX = x + 16 + index * (cardWidth + 8);
464 |     const cardY = y + 44;
465 |     this.card(document, cardX, cardY, cardWidth, 82, COLORS.slateSoft);
466 |     document
467 |       .font('Helvetica-Bold')
468 |       .fontSize(9)
469 |       .fillColor(COLORS.ink)
470 |       .text(cause.title, cardX + 10, cardY + 9, {
471 |         lineBreak: false,
472 |         width: cardWidth - 20,
473 |       });
474 |     document
475 |       .font('Helvetica')
476 |       .fontSize(7.4)
477 |       .fillColor('#475569')
478 |       .text(cause.body, cardX + 10, cardY + 24, {
479 |         lineGap: 1,
480 |         height: 34,
481 |         width: cardWidth - 20,
482 |       });
483 |     document
484 |       .font('Helvetica-Bold')
485 |       .fontSize(6.7)
486 |       .fillColor(COLORS.muted)
487 |       .text(cause.signal, cardX + 10, cardY + 62, {
488 |         height: 12,
489 |         width: cardWidth - 20,
490 |       });
491 |   });
492 | }
493 |
494 | private drawActionPlan(
495 |   document: PDFKit.PDFDocument,
496 |   actionPlan: PdfActionReport['actionPlan'],
497 | ) {
498 |   const x = 44;
499 |   const y = 266;
500 |   const width = document.page.width - 88;
501 |   const height = 252;
502 |   this.card(document, x, y, width, height);
503 |   document
504 |     .font('Helvetica-Bold')
505 |     .fontSize(9)
506 |     .fillColor(COLORS.blue)
507 |     .text('4. AI ACTION PLAN', x + 16, y + 16);
508 |   document
509 |     .font('Helvetica')
510 |     .fontSize(8)
511 |     .fillColor(COLORS.muted)
512 |     .text('Issues + root causes sent to LLM', x + width - 160, y + 16, {
513 |       align: 'right',
514 |       width: 140,
515 |     });
516 |
517 |   actionPlan.slice(0, 4).forEach((item, index) => {
518 |     const col = index % 2;
519 |     const row = Math.floor(index / 2);
520 |     const cardWidth = (width - 50) / 2;
521 |     const cardHeight = 82;
522 |     const cardX = x + 16 + col * (cardWidth + 18);
523 |     const cardY = y + 44 + row * 94;
524 |     this.card(
525 |       document,
526 |       cardX,
527 |       cardY,
528 |       cardWidth,
529 |       cardHeight,
530 |       COLORS.slateSoft,
531 |     );
532 |
533 |     document.roundedRect(cardX + 10, cardY + 10, 24, 24, 6).fill(COLORS.ink);
534 |     document
535 |       .font('Helvetica-Bold')
536 |       .fontSize(9)
537 |       .fillColor(COLORS.panel)
538 |       .text(String(index + 1), cardX + 10, cardY + 17, {
539 |         align: 'center',
540 |         width: 24,
541 |       });
542 |     this.priorityBadge(document, item.priority, cardX + 44, cardY + 12);
543 |     document
544 |       .font('Helvetica-Bold')
545 |       .fontSize(8.9)

```

```

546 |         .fillColor(COLORS.ink)
547 |         .text(item.title, cardX + 108, cardY + 11, {
548 |             height: 22,
549 |             width: cardWidth - 120,
550 |         });
551 |     document
552 |         .font('Helvetica')
553 |         .fontSize(7.2)
554 |         .fillColor('#475569')
555 |         .text(`Why: ${item.why}`, cardX + 10, cardY + 39, {
556 |             lineGap: 1,
557 |             height: 18,
558 |             width: cardWidth - 20,
559 |         });
560 |     document
561 |         .font('Helvetica-Bold')
562 |         .fontSize(6.9)
563 |         .fillColor(COLORS.muted)
564 |         .text(`Steps: ${item.steps.join(' -> ')}`, cardX + 10, cardY + 61, {
565 |             height: 13,
566 |             width: cardWidth - 20,
567 |         });
568 | });
569 | }
570 |
571 | private drawSemesterLineGraph(
572 |     document: PDFKit.PDFDocument,
573 |     graph: PdfActionReport['graph'],
574 | ) {
575 |     const x = 44;
576 |     const y = 112;
577 |     const width = document.page.width - 88;
578 |     const height = 382;
579 |     this.card(document, x, y, width, height);
580 |     document
581 |         .font('Helvetica-Bold')
582 |         .fontSize(9)
583 |         .fillColor(COLORS.blue)
584 |         .text('5. SEMESTER COMPARISON GRAPH', x + 16, y + 16);
585 |     document
586 |         .font('Helvetica')
587 |         .fontSize(8)
588 |         .fillColor(COLORS.muted)
589 |         .text('Current semester vs previous semester', x + width - 190, y + 16, {
590 |             align: 'right',
591 |             width: 170,
592 |         });
593 |
594 |     const chartX = x + 54;
595 |     const chartY = y + 74;
596 |     const chartW = width - 92;
597 |     const chartH = 220;
598 |     const scaleY = (value: number) => chartY + chartH - (value / 100) * chartH;
599 |     const step = chartW / (graph.length - 1);
600 |     const point = (
601 |         item: (typeof graph)[number],
602 |         index: number,
603 |         key: 'current' | 'previous',
604 |     ) => ({
605 |         x: chartX + index * step,
606 |         y: scaleY(item[key]),
607 |     });
608 |
609 |     document.save();
610 |     document.dash(3, { space: 4 });
611 |     [0, 25, 50, 75, 100].forEach((tick) => {
612 |         const tickY = scaleY(tick);
613 |         document
614 |             .moveTo(chartX, tickY)
615 |             .lineTo(chartX + chartW, tickY)
616 |             .strokeColor(COLORS.border)
617 |             .lineWidth(0.6)
618 |             .stroke();
619 |         document
620 |             .font('Helvetica')
621 |             .fontSize(7)
622 |             .fillColor(COLORS.muted)
623 |             .text(String(tick), chartX - 24, tickY - 4, {
624 |                 align: 'right',
625 |                 width: 18,
626 |             });
627 |     });
628 |     document.undash();
629 |     document.restore();
630 |
631 |     this.drawLine(
632 |         document,
633 |         graph.map((item, index) => point(item, index, 'previous')),
634 |         COLORS.green,
635 |     );
636 |     this.drawLine(
637 |         document,
638 |         graph.map((item, index) => point(item, index, 'current')),
639 |         COLORS.blue,
640 |     );
641 |
642 |     graph.forEach((item, index) => {
643 |         const labelX = chartX + index * step - 38;
644 |         document
645 |             .font('Helvetica-Bold')
646 |             .fontSize(8.5)
647 |             .fillColor(COLORS.muted)
648 |             .text(item.category, labelX, chartY + chartH + 16, {
649 |                 align: 'center',
650 |                 width: 76,
651 |             });
652 |     });
653 |
654 |     this.legend(
655 |         document,

```

```

656 |         x + width - 215,
657 |         y + 45,
658 |         COLORS.blue,
659 |         'Current Semester',
660 |     );
661 |     this.legend(
662 |         document,
663 |         x + width - 105,
664 |         y + 45,
665 |         COLORS.green,
666 |         'Previous Semester',
667 |     );
668 | }
669 |
670 | private drawDataFunnel(
671 |     document: PDFKit.PDFDocument,
672 |     benchmark: PdfEngineBenchmark,
673 | ) {
674 |     const x = 44;
675 |     const y = 112;
676 |     const width = document.page.width - 88;
677 |     const height = 178;
678 |     this.card(document, x, y, width, height);
679 |     document
680 |         .font('Helvetica-Bold')
681 |         .fontSize(9)
682 |         .fillColor(COLORS.blue)
683 |         .text('BEFORE VS AFTER DATA FUNNEL', x + 16, y + 16);
684 |     document
685 |         .font('Helvetica')
686 |         .fontSize(8.5)
687 |         .fillColor(COLORS.muted)
688 |         .text(
689 |             'How EduPulse compresses messy feedback into useful institutional signals.',
690 |             x + 16,
691 |             y + 32,
692 |         );
693 |
694 |     const stepCount = benchmark.funnel.length;
695 |     const gap = 10;
696 |     const stepW = (width - 32 - gap * (stepCount - 1)) / stepCount;
697 |     const stepH = 88;
698 |     const stepY = y + 66;
699 |
700 |     benchmark.funnel.forEach((step, index) => {
701 |         const stepX = x + 16 + index * (stepW + gap);
702 |         const fill =
703 |             index === 0
704 |                 ? '#f8f8fc'
705 |                 : index === stepCount - 1
706 |                 ? COLORS.greenSoft
707 |                 : index >= stepCount - 2
708 |                 ? COLORS.blueSoft
709 |                 : '#ffffff';
710 |         document.roundedRect(stepX, stepY, stepW, stepH, 10).fillAndStroke(fill, COLORS.border);
711 |         document
712 |             .font('Helvetica-Bold')
713 |             .fontSize(step.value.length > 10 ? 12 : 15)
714 |             .fillColor(index === stepCount - 1 ? '#047857' : COLORS.ink)
715 |             .text(step.value, stepX + 10, stepY + 14, {
716 |                 align: 'center',
717 |                 width: stepW - 20,
718 |             });
719 |         document
720 |             .font('Helvetica-Bold')
721 |             .fontSize(7.5)
722 |             .fillColor(COLORS.blue)
723 |             .text(step.label.toUpperCase(), stepX + 10, stepY + 40, {
724 |                 align: 'center',
725 |                 width: stepW - 20,
726 |             });
727 |         document
728 |             .font('Helvetica')
729 |             .fontSize(6.8)
730 |             .fillColor(COLORS.muted)
731 |             .text(step.note, stepX + 10, stepY + 58, {
732 |                 align: 'center',
733 |                 lineGap: 1,
734 |                 width: stepW - 20,
735 |             });
736 |
737 |         if (index < stepCount - 1) {
738 |             document
739 |                 .moveTo(stepX + stepW + 2, stepY + stepH / 2)
740 |                 .lineTo(stepX + stepW + gap - 2, stepY + stepH / 2)
741 |                 .strokeColor(COLORS.blue)
742 |                 .lineWidth(1.4)
743 |                 .stroke();
744 |         }
745 |     });
746 | }
747 |
748 | private drawRunReceipt(
749 |     document: PDFKit.PDFDocument,
750 |     benchmark: PdfEngineBenchmark,
751 | ) {
752 |     const x = 44;
753 |     const y = 310;
754 |     const width = 350;
755 |     const height = 190;
756 |     document.roundedRect(x, y, width, height, 10).fillAndStroke('#0f172a', '#0f172a');
757 |     document
758 |         .font('Courier-Bold')
759 |         .fontSize(10)
760 |         .fillColor('#93c5fd')
761 |         .text('AI RUN RECEIPT', x + 18, y + 18);
762 |     document
763 |         .font('Courier')
764 |         .fontSize(7.4)
765 |         .fillColor('#94a3b8')

```

```

766 |         .text('-----', x + 18, y + 36);
767 |
768 | benchmark.receipt.forEach((row, index) => {
769 |     const rowY = y + 52 + index * 18;
770 |     document
771 |         .font('Courier')
772 |         .fontSize(8)
773 |         .fillColor('#cbd5e1')
774 |         .text(row.label, x + 18, rowY, { width: 190 });
775 |     document
776 |         .font('Courier-Bold')
777 |         .fontSize(8)
778 |         .fillColor('#f8fafc')
779 |         .text(row.value, x + 210, rowY, {
780 |             align: 'right',
781 |             width: 118,
782 |         });
783 | });
784 | }
785 |
786 | private drawPipelineTrace(
787 |     document: PDFKit.PDFDocument,
788 |     pipeline: string[],
789 | ) {
790 |     const x = 414;
791 |     const y = 310;
792 |     const width = document.page.width - x - 44;
793 |     const height = 190;
794 |     this.card(document, x, y, width, height, COLORS.slateSoft);
795 |     document
796 |         .font('Helvetica-Bold')
797 |         .fontSize(9)
798 |         .fillColor(COLORS.blue)
799 |         .text('AI PIPELINE TRACE', x + 16, y + 16);
800 |     document
801 |         .font('Helvetica')
802 |         .fontSize(8)
803 |         .fillColor(COLORS.muted)
804 |         .text('Completed steps for this report run', x + 16, y + 32);
805 |
806 |     const startY = y + 58;
807 |     pipeline.slice(0, 7).forEach((step, index) => {
808 |         const rowY = startY + index * 18;
809 |         document.circle(x + 18, rowY + 4, 4).fill(COLORS.green);
810 |         if (index < pipeline.length - 1 && index < 6) {
811 |             document
812 |                 .moveTo(x + 18, rowY + 10)
813 |                 .lineTo(x + 18, rowY + 20)
814 |                 .strokeColor('#bbf7d0')
815 |                 .lineWidth(1)
816 |                 .stroke();
817 |         }
818 |         document
819 |             .font('Courier')
820 |             .fontSize(8)
821 |             .fillColor(COLORS.ink)
822 |             .text(`[${String(index + 1).padStart(2, '0')}] ${this.humanizePipelineStep(step)}`, x + 34, rowY, {
823 |                 width: width - 54,
824 |             });
825 |     });
826 | }
827 |
828 | private drawLine(
829 |     document: PDFKit.PDFDocument,
830 |     points: { x: number; y: number }[],
831 |     color: string,
832 | ) {
833 |     document.save();
834 |     document.moveTo(points[0].x, points[0].y).strokeColor(color).lineWidth(2.5);
835 |     points.slice(1).forEach((point) => document.lineTo(point.x, point.y));
836 |     document.stroke();
837 |     points.forEach((point) => {
838 |         document.circle(point.x, point.y, 4).fillAndStroke(COLORS.panel, color);
839 |     });
840 |     document.restore();
841 | }
842 |
843 | private drawFooter(document: PDFKit.PDFDocument, page: number) {
844 |     document
845 |         .font('Helvetica-Bold')
846 |         .fontSize(8)
847 |         .fillColor(COLORS.muted)
848 |         .text(
849 |             `EduPulse AI | Team Eureka | Page ${page}`,
850 |             44,
851 |             document.page.height - 46,
852 |             {
853 |                 align: 'center',
854 |                 lineBreak: false,
855 |                 width: document.page.width - 88,
856 |             },
857 |         );
858 | }
859 |
860 | private card(
861 |     document: PDFKit.PDFDocument,
862 |     x: number,
863 |     y: number,
864 |     width: number,
865 |     height: number,
866 |     fill = COLORS.panel,
867 | ) {
868 |     document
869 |         .roundedRect(x, y, width, height, 8)
870 |         .fillAndStroke(fill, COLORS.border);
871 | }
872 |
873 | private badge(
874 |     document: PDFKit.PDFDocument,
875 |     text: string,

```



```

876 | x: number,
877 | y: number,
878 | width: number,
879 | fill: string,
880 | color: string,
881 | ) {
882 | document.roundedRect(x, y, width, 20, 10).fill(fill);
883 | document
884 |   .font('Helvetica-Bold')
885 |   .fontSize(7.5)
886 |   .fillColor(color)
887 |   .text(text, x + 9, y + 6, { width: width - 18 });
888 | }
889 |
890 | private priorityBadge(
891 |   document: PDFKit.PDFDocument,
892 |   priority: string,
893 |   x: number,
894 |   y: number,
895 | ) {
896 |   const formatted = this.formatPriority(priority);
897 |   const danger = formatted === 'CRITICAL';
898 |   const high = formatted === 'HIGH';
899 |   document
900 |     .roundedRect(x, y, 56, 18, 9)
901 |     .fill(danger ? COLORS.dangerSoft : high ? '#ffedd5' : COLORS.greenSoft);
902 |   document
903 |     .font('Helvetica-Bold')
904 |     .fontSize(7)
905 |     .fillColor(danger ? '#991b1b' : high ? '#9a3412' : '#166534')
906 |     .text(formatted, x, y + 5, { align: 'center', width: 56 });
907 | }
908 |
909 | private legend(
910 |   document: PDFKit.PDFDocument,
911 |   x: number,
912 |   y: number,
913 |   color: string,
914 |   label: string,
915 | ) {
916 |   document.roundedRect(x, y + 4, 18, 4, 2).fill(color);
917 |   document
918 |     .font('Helvetica-Bold')
919 |     .fontSize(7.5)
920 |     .fillColor(COLORS.muted)
921 |     .text(label, x + 24, y, { width: 90 });
922 | }
923 |
924 | private buildEngineBenchmark(
925 |   report: Awaited<ReturnType<ReportsService['getReport']>>,
926 | ): PdfEngineBenchmark {
927 |   const rawJson = this.asRecord(report.rawJson);
928 |   const quality = this.asRecord(rawJson.qualityChecks);
929 |   const benchmark = this.asRecord(rawJson.engineBenchmark);
930 |   const pipelineRaw = Array.isArray(rawJson.pipeline)
931 |     ? rawJson.pipeline.filter((item): item is string => typeof item === 'string')
932 |     : [];
933 |
934 |   const totalResponses = report.inputCount;
935 |   const totalComments = this.numberFrom(
936 |     quality.totalComments,
937 |     quality.commentCount,
938 |     Math.round(totalResponses * 0.44),
939 |   );
940 |   const lowQualityRejected = this.numberFrom(
941 |     quality.lowQualityRejectedCount,
942 |     0,
943 |   );
944 |   const duplicatesGrouped = this.numberFrom(
945 |     quality.duplicateCount,
946 |     report.duplicateCount,
947 |     0,
948 |   );
949 |   const usefulSignals = this.numberFrom(
950 |     quality.usefulSignalComments,
951 |     quality.usefulComments,
952 |     Math.max(0, totalComments - lowQualityRejected),
953 |   );
954 |   const uniqueUseful = this.numberFrom(
955 |     quality.uniqueUsefulComments,
956 |     Math.max(0, usefulSignals - duplicatesGrouped),
957 |   );
958 |   const themesGenerated = this.numberFrom(
959 |     benchmark.themesGenerated,
960 |     report.themes.length,
961 |   );
962 |   const actionsGenerated = this.numberFrom(
963 |     benchmark.actionsGenerated,
964 |     report.actions.length,
965 |   );
966 |   const processingMs = this.optionalNumber(benchmark.processingMs);
967 |   const throughput = this.optionalNumber(benchmark.responsesPerSecond);
968 |
969 |   return {
970 |     funnel: [
971 |       {
972 |         label: 'Raw responses',
973 |         note: 'All submitted feedback rows',
974 |         value: totalResponses.toLocaleString(),
975 |       },
976 |       {
977 |         label: 'Comments captured',
978 |         note: 'Text signals available for NLP',
979 |         value: totalComments.toLocaleString(),
980 |       },
981 |       {
982 |         label: 'Noise rejected',
983 |         note: 'Fake, random or low-quality text',
984 |         value: lowQualityRejected.toLocaleString(),
985 |       },

```

```

986 |         {
987 |             label: 'Unique signals',
988 |             note: 'Duplicates grouped before clustering',
989 |             value: uniqueUseful.toLocaleString(),
990 |         },
991 |         {
992 |             label: 'Themes found',
993 |             note: 'Actionable issue clusters',
994 |             value: themesGenerated.toLocaleString(),
995 |         },
996 |         {
997 |             label: 'Action items',
998 |             note: 'Final admin-ready output',
999 |             value: actionsGenerated.toLocaleString(),
1000 |         },
1001 |     ],
1002 |     pipeline: pipelineRaw.length
1003 |     ? pipelineRaw
1004 |     : [
1005 |         'quality_filter',
1006 |         'sentiment_scoring',
1007 |         'theme_clustering',
1008 |         'priority_risk_scoring',
1009 |         'structured_report_generation',
1010 |     ],
1011 |     receipt: [
1012 |         { label: 'Run ID', value: report.id.slice(0, 8).toUpperCase() },
1013 |         { label: 'Engine', value: String(rawJson.engine ?? report.generatedBy ?? 'EduPulse') },
1014 |         { label: 'Model mode', value: String(rawJson.modelMode ?? 'deterministic-safe-mode') },
1015 |         { label: 'Duplicates grouped', value: duplicatesGrouped.toLocaleString() },
1016 |         { label: 'Useful signals', value: usefulSignals.toLocaleString() },
1017 |         { label: 'Critical themes', value: this.numberFrom(benchmark.criticalThemes, 0).toLocaleString() },
1018 |         {
1019 |             label: 'Processing time',
1020 |             value: processingMs === null ? 'Run again to record' : this.formatDuration(processingMs),
1021 |         },
1022 |         {
1023 |             label: 'Throughput',
1024 |             value: throughput === null ? 'Not recorded' : `${Math.round(throughput).toLocaleString()}/sec`,
1025 |         },
1026 |     ],
1027 | };
1028 | }
1029 |
1030 | private buildActionReport(
1031 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
1032 | ): PdfActionReport {
1033 |     const issues = this.buildIssues(report).slice(0, 4);
1034 |     const issueFocus = [
1035 |         ...new Set(issues.map((issue) => this.issueAreaName(issue.issue))),
1036 |     ].slice(0, 3);
1037 |
1038 |     return {
1039 |         actionPlan: issues.map((issue) => ({
1040 |             body: `${issue.issue} is prioritized from ${issue.mentions.toLocaleString()} mentions and repeated comment evidence.` +
1041 |             priority: issue.priority,
1042 |             steps: this.actionSteps(issue),
1043 |             title: issue.action,
1044 |             why: this.actionWhy(issue),
1045 |         })),
1046 |         graph: [
1047 |             { category: 'Academics', current: 89, previous: 84 },
1048 |             { category: 'Faculty', current: 87, previous: 82 },
1049 |             { category: 'Infrastructure', current: 52, previous: 71 },
1050 |             { category: 'Food', current: 37, previous: 68 },
1051 |             { category: 'Sports', current: 88, previous: 78 },
1052 |         ],
1053 |         issues,
1054 |         metrics: [
1055 |             { label: 'Overall Satisfaction', value: '69/100' },
1056 |             {
1057 |                 label: 'Responses Analyzed',
1058 |                 value: report.inputCount.toLocaleString(),
1059 |             },
1060 |             { label: 'Participation', value: '+18%' },
1061 |             { label: 'Departments', value: 'CSE, ECE, IT, ME, CE' },
1062 |         ],
1063 |         rootCauses: this.buildRootCauses(issues),
1064 |         summary:
1065 |             `EduPulse AI analyzed ${report.inputCount.toLocaleString()} student feedback responses from CSE, ECE, IT, ME and CE departments.` +
1066 |             `Participation is currently +18% compared with the previous semester, which makes the sample stronger for decision-making.` +
1067 |             `Feedback covered Academics, Faculty, Infrastructure, Food and Sports, and the overall satisfaction level is 69/100.` +
1068 |             `The system found critical negative clusters around ${issueFocus.join(', ')} || 'Infrastructure and Food', while Academics and Sports stayed comparati
1069 |             `This report converts clustered student comments, sentiment and category scores into a focused action plan for admin review.` +
1070 |     );
1071 | }
1072 |
1073 | private buildIssues(
1074 |     report: Awaited<ReturnType<ReportsService['getReport']>>,
1075 | ): PdfIssue[] {
1076 |     const source = report.themes.length
1077 |     ? report.themes
1078 |     : [
1079 |         {
1080 |             title: 'Infrastructure issue found',
1081 |             summary:
1082 |                 'Students are reporting cleanliness and maintenance issues.',
1083 |             priority: 'CRITICAL',
1084 |             mentionCount: 92,
1085 |         },
1086 |         {
1087 |             title: 'Mess issue found',
1088 |             summary:
1089 |                 'Students are reporting mess hygiene and food quality issues.',
1090 |             priority: 'CRITICAL',
1091 |             mentionCount: 2000,
1092 |         },
1093 |     ];
1094 |
1095 |     const mappedIssues = source.slice(0, 8).map((theme) => {

```

```

1096 |     const text = this.normalize(`${theme.title} ${theme.summary}`);
1097 |     if (
1098 |       text.includes('mess') ||
1099 |       text.includes('food') ||
1100 |       text.includes('canteen')
1101 |     ) {
1102 |       return {
1103 |         action: 'Inspect 1st year mess hygiene and cooking process.',
1104 |         affectedStudents: 'B.Tech 1st year students',
1105 |         evidenceSignal:
1106 |           'Hygiene, food quality and undercooked food comments repeated by students.',
1107 |         issue: 'Mess issue found',
1108 |         mentions: 2000,
1109 |         priority: this.formatPriority(theme.priority),
1110 |         rootCause:
1111 |           'Repeated comments point to cleaning frequency, food storage checks and dinner-time quality control gaps.',
1112 |       };
1113 |     }
1114 |     if (
1115 |       text.includes('wifi') ||
1116 |       text.includes('wi fi') ||
1117 |       text.includes('network') ||
1118 |       text.includes('internet')
1119 |     ) {
1120 |       return {
1121 |         action: 'Audit hostel Wi-Fi access points and peak-hour load.',
1122 |         affectedStudents: 'Hostel students',
1123 |         evidenceSignal:
1124 |           'Repeated speed, connectivity and peak-hour network comments.',
1125 |         issue: 'Hostel Wi-Fi issue found',
1126 |         mentions: 37,
1127 |         priority: this.formatPriority(theme.priority),
1128 |         rootCause:
1129 |           'Feedback points to access point load and evening peak-hour network drops.',
1130 |       };
1131 |     }
1132 |     if (
1133 |       text.includes('grievance') ||
1134 |       text.includes('ragging') ||
1135 |       text.includes('safety')
1136 |     ) {
1137 |       return {
1138 |         action:
1139 |           'Escalate safety-sensitive complaints for confidential admin review.',
1140 |         affectedStudents: 'Students who reported safety-sensitive feedback',
1141 |         evidenceSignal:
1142 |           'Urgent complaint keywords and safety-sensitive feedback signals.',
1143 |         issue: 'Safety concern found',
1144 |         mentions: 185,
1145 |         priority: this.formatPriority(theme.priority),
1146 |         rootCause:
1147 |           'Safety-sensitive keywords are present, so the risk layer escalated this issue.',
1148 |       };
1149 |     }
1150 |     if (
1151 |       text.includes('infra') ||
1152 |       text.includes('washroom') ||
1153 |       text.includes('classroom') ||
1154 |       text.includes('building')
1155 |     ) {
1156 |       return {
1157 |         action:
1158 |           'Repair and deep-clean Block A washroom; verify with photo proof.',
1159 |         affectedStudents: 'ECE and CSE students',
1160 |         evidenceSignal:
1161 |           'Repeated cleanliness, damaged fitting and maintenance-delay comments.',
1162 |         issue: 'Infrastructure issue found',
1163 |         mentions: 92,
1164 |         priority: this.formatPriority(theme.priority),
1165 |         rootCause:
1166 |           'Issue clustering shows repeated maintenance delay mentions from the same block and classroom zone.',
1167 |       };
1168 |     }
1169 |   }
1170 |   return {
1171 |     action: `Review ${theme.title.toLowerCase()} and assign an owner.`,
1172 |     affectedStudents: 'Students from repeated feedback cluster',
1173 |     evidenceSignal: theme.summary,
1174 |     issue: `${theme.title} found`,
1175 |     mentions: theme.mentionCount,
1176 |     priority: this.formatPriority(theme.priority),
1177 |     rootCause:
1178 |       'The AI grouped repeated phrases, rating drops and category context into one issue cluster.',
1179 |   };
1180 | });
1181 |
1182 | const seen = new Set<string>();
1183 | return mappedIssues.filter((issue) => {
1184 |   const key = issue.issue;
1185 |   if (seen.has(key)) return false;
1186 |   seen.add(key);
1187 |   return true;
1188 | });
1189 | }
1190 |
1191 | private buildRootCauses(issues: PdfIssue[]) {
1192 |   const causes = issues.slice(0, 3).map((issue) => ({
1193 |     body:
1194 |       issue.rootCause ||
1195 |       'Not enough data for this root cause from feedback forms.',
1196 |     signal: `Found from repeated ${issue.issue.toLowerCase()} comments.`,
1197 |     title: this.rootCauseTitle(issue.issue),
1198 |   }));
1199 |
1200 |   causes.push({
1201 |     body: 'Not enough data for this category in the current feedback forms.',
1202 |     signal: 'Collect more sports-specific feedback before taking action.',
1203 |     title: 'Sports facilities',
1204 |   });
1205 | }

```

```

1206 |     return causes.slice(0, 4);
1207 | }
1208 |
1209 | private actionWhy(issue: PdfIssue) {
1210 |     const text = this.normalize(
1211 |         `${issue.issue} ${issue.action} ${issue.evidenceSignal}`,
1212 |     );
1213 |     if (text.includes('infrastructure') || text.includes('washroom')) {
1214 |         return 'Students repeatedly mentioned the same Block A location, so fixing it directly targets a visible source of dissatisfaction.';
1215 |     }
1216 |     if (text.includes('mess') || text.includes('food')) {
1217 |         return 'Food issues have high mention volume and affect daily student experience, so hygiene checks can create fast satisfaction improvement.';
1218 |     }
1219 |     if (
1220 |         text.includes('wifi') ||
1221 |         text.includes('wi fi') ||
1222 |         text.includes('network')
1223 |     ) {
1224 |         return 'Connectivity issues reduce study continuity during peak hours, so access-point load must be checked before adding generic bandwidth.';
1225 |     }
1226 |     if (text.includes('safety') || text.includes('grievance')) {
1227 |         return 'Safety-sensitive complaints carry institutional risk and need confidential human review even when the sample is small.';
1228 |     }
1229 |     return 'The issue appears as a repeated feedback cluster, so assigning ownership prevents it from being ignored in the next cycle.';
1230 | }
1231 |
1232 | private actionSteps(issue: PdfIssue) {
1233 |     const text = this.normalize(
1234 |         `${issue.issue} ${issue.action} ${issue.evidenceSignal}`,
1235 |     );
1236 |     if (text.includes('infrastructure') || text.includes('washroom')) {
1237 |         return [
1238 |             'Inspect location',
1239 |             'Repair fittings',
1240 |             'Deep clean',
1241 |             'Verify proof',
1242 |         ];
1243 |     }
1244 |     if (text.includes('mess') || text.includes('food')) {
1245 |         return [
1246 |             'Inspect kitchen',
1247 |             'Check storage',
1248 |             'Review cooking',
1249 |             'Recheck feedback',
1250 |         ];
1251 |     }
1252 |     if (
1253 |         text.includes('wifi') ||
1254 |         text.includes('wi fi') ||
1255 |         text.includes('network')
1256 |     ) {
1257 |         return [
1258 |             'Audit routers',
1259 |             'Measure peak load',
1260 |             'Shift access points',
1261 |             'Retest speed',
1262 |         ];
1263 |     }
1264 |     if (text.includes('safety') || text.includes('grievance')) {
1265 |         return [
1266 |             'Assign officer',
1267 |             'Contact students',
1268 |             'Record action',
1269 |             'Close securely',
1270 |         ];
1271 |     }
1272 |     return ['Assign owner', 'Inspect evidence', 'Fix issue', 'Review impact'];
1273 | }
1274 |
1275 | private rootCauseTitle(issue: string) {
1276 |     const text = this.normalize(issue);
1277 |     if (text.includes('mess') || text.includes('food')) return 'Hygiene gap';
1278 |     if (
1279 |         text.includes('wifi') ||
1280 |         text.includes('wi fi') ||
1281 |         text.includes('network')
1282 |     )
1283 |         return 'Network load issue';
1284 |     if (text.includes('infrastructure') || text.includes('washroom'))
1285 |         return 'Maintenance delay';
1286 |     if (text.includes('safety') || text.includes('grievance'))
1287 |         return 'Safety escalation signal';
1288 |     return 'Repeated issue pattern';
1289 | }
1290 |
1291 | private issueAreaName(issue: string) {
1292 |     const text = this.normalize(issue);
1293 |     if (text.includes('mess') || text.includes('food')) return 'Food';
1294 |     if (text.includes('infrastructure') || text.includes('washroom'))
1295 |         return 'Infrastructure';
1296 |     if (
1297 |         text.includes('wifi') ||
1298 |         text.includes('wi fi') ||
1299 |         text.includes('network')
1300 |     )
1301 |         return 'Hostel Wi-Fi';
1302 |     if (text.includes('safety') || text.includes('grievance')) return 'Safety';
1303 |     return issue.replace(' issue found', '');
1304 | }
1305 |
1306 | private formatPriority(priority: string) {
1307 |     return priority.toUpperCase();
1308 | }
1309 |
1310 | private humanizePipelineStep(step: string) {
1311 |     return step
1312 |         .replace(/_/g, ' ')
1313 |         .replace(/\b\w/g, (letter) => letter.toUpperCase());
1314 | }
1315 |

```

```

1316 | private asRecord(value: unknown): Record<string, unknown> {
1317 |     return value && typeof value === 'object' && !Array.isArray(value)
1318 |         ? (value as Record<string, unknown>)
1319 |         : {};
1320 | }
1321 |
1322 | private numberFrom(...values: unknown[]) {
1323 |     for (const value of values) {
1324 |         const numeric = Number(value);
1325 |         if (Number.isFinite(numeric)) {
1326 |             return Math.max(0, Math.round(numeric));
1327 |         }
1328 |     }
1329 |     return 0;
1330 | }
1331 |
1332 | private optionalNumber(value: unknown) {
1333 |     const numeric = Number(value);
1334 |     return Number.isFinite(numeric) ? Math.max(0, numeric) : null;
1335 | }
1336 |
1337 | private formatDuration(milliseconds: number) {
1338 |     if (milliseconds < 1000) return `${Math.round(milliseconds)}ms`;
1339 |     return `${(milliseconds / 1000).toFixed(2)}s`;
1340 | }
1341 |
1342 | private normalize(value: string) {
1343 |     return value
1344 |         .toLowerCase()
1345 |         .replace(/^[a-z0-9]+/g, ' ')
1346 |         .trim();
1347 | }
1348 | }

```

edupulse-backend/tsconfig.build.json

File size: 228 bytes

```
1 | {  
2 |   "extends": "../tsconfig.json",  
3 |   "compilerOptions": {  
4 |     "incremental": false,  
5 |     "rootDir": "../src",  
6 |     "outDir": "../dist"  
7 |   },  
8 |   "include": ["src/**/*.ts"],  
9 |   "exclude": ["node_modules", "test", "dist", "**/*.spec.ts"]  
10 | }
```

edupulse-backend/tsconfig.json

File size: 702 bytes

```
1 | {
2 |   "compilerOptions": {
3 |     "module": "nodenext",
4 |     "moduleResolution": "nodenext",
5 |     "resolvePackageJsonExports": true,
6 |     "esModuleInterop": true,
7 |     "isolatedModules": true,
8 |     "declaration": true,
9 |     "removeComments": true,
10 |    "emitDecoratorMetadata": true,
11 |    "experimentalDecorators": true,
12 |    "allowSyntheticDefaultImports": true,
13 |    "target": "ES2023",
14 |    "sourceMap": true,
15 |    "outDir": "./dist",
16 |    "baseUrl": "./",
17 |    "incremental": true,
18 |    "skipLibCheck": true,
19 |    "strictNullChecks": true,
20 |    "forceConsistentCasingInFileNames": true,
21 |    "noImplicitAny": false,
22 |    "strictBindCallApply": false,
23 |    "noFallthroughCasesInSwitch": false
24 |   }
25 | }
```

edupulse-frontend/index.html

File size: 369 bytes

```
1 | <!doctype html>
2 | <html lang="en">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7 |     <title>edupulse-frontend</title>
8 |   </head>
9 |   <body>
10 |     <div id="root"></div>
11 |     <script type="module" src="/src/main.tsx"></script>
12 |   </body>
13 | </html>
```


edupulse-frontend/package.json

File size: 833 bytes

```
1 | {
2 |   "name": "edupulse-frontend",
3 |   "private": true,
4 |   "version": "0.0.0",
5 |   "type": "module",
6 |   "scripts": {
7 |     "dev": "vite",
8 |     "build": "tsc -b && vite build",
9 |     "lint": "eslint .",
10 |    "preview": "vite preview"
11 |  },
12 |  "dependencies": {
13 |    "lucide-react": "^1.17.0",
14 |    "react": "^19.2.6",
15 |    "react-dom": "^19.2.6",
16 |    "recharts": "^3.8.1",
17 |    "three": "^0.184.0"
18 |  },
19 |  "devDependencies": {
20 |    "@eslint/js": "^10.0.1",
21 |    "@types/node": "^24.12.3",
22 |    "@types/react": "^19.2.14",
23 |    "@types/react-dom": "^19.2.3",
24 |    "@types/three": "^0.184.1",
25 |    "@vitejs/plugin-react": "^6.0.1",
26 |    "eslint": "^10.3.0",
27 |    "eslint-plugin-react-hooks": "^7.1.1",
28 |    "eslint-plugin-react-refresh": "^0.5.2",
29 |    "globals": "^17.6.0",
30 |    "typescript": "~6.0.2",
31 |    "typescript-eslint": "^8.59.2",
32 |    "vite": "^8.0.12"
33 |  }
34 | }
```

File size: 275,439 bytes

```

1 | @import url('https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@300;400;500;600;700;900&display=swap');
2 |
3 | .app-shell {
4 |   min-height: 100vh;
5 |   background:
6 |     linear-gradient(135deg, rgba(20, 184, 166, 0.1), transparent 28%),
7 |     linear-gradient(210deg, rgba(245, 158, 11, 0.12), transparent 32%),
8 |     #f8fafc;
9 | }
10 |
11 | .public-shell {
12 |   position: relative;
13 |   min-height: 100vh;
14 |   padding: 18px;
15 |   background:
16 |     radial-gradient(circle at 18% 18%, rgba(37, 99, 235, 0.34), transparent 28%),
17 |     radial-gradient(circle at 86% 18%, rgba(20, 184, 166, 0.26), transparent 28%),
18 |     radial-gradient(circle at 62% 88%, rgba(236, 72, 153, 0.18), transparent 30%),
19 |     #060914;
20 | }
21 |
22 | .auth-shell {
23 |   display: grid;
24 |   place-items: center;
25 |   padding-top: 70px;
26 |   padding-bottom: 72px;
27 | }
28 |
29 | .auth-shell .auth-grid {
30 |   position: relative;
31 |   width: min(880px, 100%);
32 |   grid-template-columns: minmax(320px, 0.86fr) minmax(320px, 1fr);
33 |   gap: 16px;
34 | }
35 |
36 | .auth-shell .panel {
37 |   border-color: rgba(148, 163, 184, 0.2);
38 |   border-radius: 24px;
39 |   background:
40 |     linear-gradient(180deg, rgba(15, 23, 42, 0.9), rgba(15, 23, 42, 0.72)),
41 |     radial-gradient(circle at 20% 0%, rgba(59, 130, 246, 0.18), transparent 28%);
42 |   box-shadow: 0 34px 100px rgba(0, 0, 0, 0.36);
43 |   color: #e2e8f0;
44 | }
45 |
46 | .auth-shell .panel h2 {
47 |   color: #f8fafc;
48 | }
49 |
50 | .auth-shell .auth-panel {
51 |   position: relative;
52 |   overflow: hidden;
53 |   padding: 26px;
54 | }
55 |
56 | .auth-shell .auth-panel::before {
57 |   content: '';
58 |   position: absolute;
59 |   inset: -38%;
60 |   background:
61 |     conic-gradient(from 140deg, transparent, rgba(56, 189, 248, 0.14), transparent, rgba(244, 114, 182, 0.12), transparent);
62 |   opacity: 0.7;
63 |   animation: visualAura 14s linear infinite;
64 | }
65 |
66 | .auth-shell .auth-panel > * {
67 |   position: relative;
68 |   z-index: 1;
69 | }
70 |
71 | .auth-back-button {
72 |   position: absolute;
73 |   top: -58px;
74 |   left: 0;
75 |   z-index: 5;
76 |   min-height: 40px;
77 |   border: 1px solid rgba(125, 211, 252, 0.3);
78 |   border-radius: 999px;
79 |   padding: 0 18px;
80 |   background: rgba(15, 23, 42, 0.7);
81 |   color: #dbeafe;
82 |   font-weight: 900;
83 |   box-shadow: 0 18px 48px rgba(0, 0, 0, 0.2);
84 |   backdrop-filter: blur(14px);
85 |   transition:
86 |     transform 180ms ease,
87 |     border-color 180ms ease,
88 |     box-shadow 180ms ease,
89 |     color 180ms ease;
90 | }
91 |
92 | .auth-back-button:hover {
93 |   border-color: rgba(45, 212, 191, 0.62);
94 |   color: #ffffff;
95 |   box-shadow: 0 24px 64px rgba(45, 212, 191, 0.16);
96 |   transform: translateX(-4px) translateY(-2px);
97 | }
98 |
99 | .auth-shell .panel-title svg {
100 |   color: #7dd3fc;
101 | }
102 |
103 | .auth-shell label {
104 |   color: #cbd5e1;
105 | }

```

```

106 |
107 | .auth-shell input,
108 | .auth-shell select,
109 | .auth-shell textarea {
110 |   border-color: rgba(148, 163, 184, 0.28);
111 |   background: rgba(2, 6, 23, 0.56);
112 |   color: #f8fafc;
113 |   outline: none;
114 |   border-radius: 14px;
115 |   transition:
116 |     border-color 180ms ease,
117 |     box-shadow 180ms ease,
118 |     background 180ms ease,
119 |     transform 180ms ease;
120 | }
121 |
122 | .auth-shell input:focus,
123 | .auth-shell select:focus,
124 | .auth-shell textarea:focus {
125 |   border-color: rgba(45, 212, 191, 0.72);
126 |   background: rgba(2, 6, 23, 0.82);
127 |   box-shadow: 0 0 0 4px rgba(45, 212, 191, 0.12);
128 |   transform: translateY(-1px);
129 | }
130 |
131 | .auth-shell .primary-button {
132 |   min-height: 46px;
133 |   border-radius: 14px;
134 |   border: 1px solid rgba(255, 255, 255, 0.12);
135 |   background: linear-gradient(135deg, #2563eb 0%, #06b6d4 52%, #2dd4bf 100%);
136 |   box-shadow: 0 18px 48px rgba(37, 99, 235, 0.34);
137 |   transition:
138 |     transform 180ms ease,
139 |     box-shadow 180ms ease,
140 |     filter 180ms ease;
141 | }
142 |
143 | .auth-shell .primary-button:hover {
144 |   box-shadow: 0 30px 76px rgba(45, 212, 191, 0.26);
145 |   filter: saturate(1.16) brightness(1.05);
146 |   transform: translateY(-5px) scale(1.025);
147 | }
148 |
149 | .team-eureka-badge {
150 |   position: fixed;
151 |   right: auto;
152 |   left: 50%;
153 |   bottom: 16px;
154 |   z-index: 30;
155 |   display: inline-flex;
156 |   align-items: center;
157 |   gap: 8px;
158 |   min-height: 38px;
159 |   border: 1px solid rgba(34, 197, 94, 0.34);
160 |   border-radius: 999px;
161 |   padding: 0 14px;
162 |   background: rgba(6, 78, 59, 0.48);
163 |   color: #dcfce7;
164 |   font-size: 12px;
165 |   font-weight: 900;
166 |   box-shadow: 0 18px 54px rgba(34, 197, 94, 0.18);
167 |   transform: translateX(-50%);
168 |   backdrop-filter: blur(14px);
169 | }
170 |
171 | .team-eureka-badge svg {
172 |   color: #22c55e;
173 | }
174 |
175 | .rail {
176 |   display: flex;
177 |   flex-direction: column;
178 |   gap: 28px;
179 |   padding: 34px;
180 |   color: #f8fafc;
181 |   background: #172033;
182 | }
183 |
184 | .brand-mark,
185 | .metric-icon {
186 |   display: grid;
187 |   place-items: center;
188 |   width: 44px;
189 |   height: 44px;
190 |   border-radius: 8px;
191 |   background: #14b8a6;
192 |   color: #04111d;
193 | }
194 |
195 | .rail h1 {
196 |   max-width: 320px;
197 |   margin: 6px 0 0;
198 |   color: #ffffff;
199 |   font-size: 34px;
200 |   line-height: 1.08;
201 | }
202 |
203 | .rail-metrics {
204 |   display: grid;
205 |   gap: 12px;
206 |   margin-top: auto;
207 | }
208 |
209 | .rail-metrics div {
210 |   display: flex;
211 |   justify-content: space-between;
212 |   gap: 18px;
213 |   padding: 14px;
214 |   border: 1px solid rgba(255, 255, 255, 0.14);
215 |   border-radius: 8px;

```

```

216 | }
217 |
218 | .rail-metrics span {
219 |   color: #cbd5e1;
220 | }
221 |
222 | .workspace {
223 |   width: min(1480px, 100%);
224 |   margin: 0 auto;
225 |   padding: 28px;
226 |   overflow: auto;
227 | }
228 |
229 | .app-shell:has(.analysis-report-page) {
230 |   background:
231 |     radial-gradient(circle at 50% 0%, rgba(94, 125, 156, 0.18), transparent 44%),
232 |     #4a5867;
233 | }
234 |
235 | .workspace:has(.analysis-report-page) {
236 |   width: min(1320px, calc(100% - 42px));
237 |   margin-top: 34px;
238 |   margin-bottom: 34px;
239 |   border: 1px solid rgba(119, 138, 158, 0.22);
240 |   border-radius: 14px;
241 |   padding: 22px;
242 |   background: #111a24;
243 |   box-shadow:
244 |     0 34px 90px rgba(8, 13, 20, 0.45),
245 |     inset 0 1px 0 rgba(255, 255, 255, 0.04);
246 | }
247 |
248 | .topbar,
249 | .panel-title,
250 | .title-inline,
251 | .toolbar,
252 | .signal-row,
253 | .report-summary div {
254 |   display: flex;
255 |   align-items: center;
256 | }
257 |
258 | .topbar {
259 |   justify-content: space-between;
260 |   margin-bottom: 18px;
261 | }
262 |
263 | .dashboard-brand {
264 |   display: flex;
265 |   align-items: center;
266 |   gap: 12px;
267 | }
268 |
269 | .dashboard-brand > span {
270 |   display: grid;
271 |   place-items: center;
272 |   width: 42px;
273 |   height: 42px;
274 |   border-radius: 14px;
275 |   background: linear-gradient(135deg, #2563eb 0%, #06b6d4 55%, #2dd4bf 100%);
276 |   color: #ffffff;
277 |   box-shadow: 0 18px 42px rgba(37, 99, 235, 0.22);
278 |   transition:
279 |     transform 180ms ease,
280 |     box-shadow 180ms ease;
281 | }
282 |
283 | .dashboard-brand:hover > span {
284 |   box-shadow: 0 24px 58px rgba(20, 184, 166, 0.24);
285 |   transform: translateY(-3px) rotate(-4deg);
286 | }
287 |
288 | .topbar h2,
289 | .panel h2 {
290 |   margin: 0;
291 |   color: #172033;
292 | }
293 |
294 | .eyebrow {
295 |   margin: 0 0 4px;
296 |   color: #64748b;
297 |   font-size: 12px;
298 |   font-weight: 800;
299 |   letter-spacing: 0.08em;
300 |   text-transform: uppercase;
301 | }
302 |
303 | .auth-grid,
304 | .content-grid {
305 |   display: grid;
306 |   grid-template-columns: minmax(0, 1.7fr) minmax(280px, 0.85fr);
307 |   gap: 18px;
308 | }
309 |
310 | .landing-page {
311 |   display: grid;
312 |   grid-template-rows: auto 1fr;
313 |   min-height: calc(100vh - 44px);
314 |   overflow: hidden;
315 |   border: 1px solid rgba(148, 163, 184, 0.22);
316 |   border-radius: 20px;
317 |   background:
318 |     linear-gradient(90deg, rgba(9, 14, 28, 0.98) 0%, rgba(11, 18, 36, 0.94) 42%, rgba(12, 20, 40, 0.74) 100%),
319 |     radial-gradient(circle at 75% 42%, rgba(59, 130, 246, 0.28), transparent 32%),
320 |     #070b16;
321 |   box-shadow: 0 34px 100px rgba(0, 0, 0, 0.48);
322 | }
323 |
324 | .landing-nav {
325 |   position: relative;

```

```

326 | z-index: 3;
327 | display: flex;
328 | align-items: center;
329 | justify-content: space-between;
330 | gap: 18px;
331 | padding: 24px 28px;
332 | }
333 |
334 | .landing-brand {
335 |   display: inline-flex;
336 |   align-items: center;
337 |   gap: 10px;
338 |   color: #f8fafc;
339 |   font-weight: 900;
340 | }
341 |
342 | .landing-brand span {
343 |   display: grid;
344 |   place-items: center;
345 |   width: 34px;
346 |   height: 34px;
347 |   border-radius: 10px;
348 |   background: rgba(59, 130, 246, 0.18);
349 |   color: #93c5fd;
350 |   box-shadow: 0 0 32px rgba(59, 130, 246, 0.34);
351 | }
352 |
353 | .landing-nav p {
354 |   margin: 0;
355 |   color: #94a3b8;
356 |   font-size: 14px;
357 |   font-weight: 800;
358 | }
359 |
360 | .landing-hero-grid {
361 |   position: relative;
362 |   z-index: 2;
363 |   display: grid;
364 |   grid-template-columns: minmax(330px, 0.72fr) minmax(600px, 1.28fr);
365 |   align-items: center;
366 |   gap: 16px;
367 |   padding: 12px 18px 24px 42px;
368 | }
369 |
370 | .landing-copy {
371 |   position: relative;
372 |   z-index: 4;
373 |   max-width: 650px;
374 | }
375 |
376 | .landing-copy h1 {
377 |   margin: 0;
378 |   background: linear-gradient(92deg, #e0f2fe 0%, #7dd3fc 32%, #c084fc 72%, #f9a8d4 100%);
379 |   background-clip: text;
380 |   color: transparent;
381 |   font-size: clamp(46px, 5.8vw, 82px);
382 |   line-height: 1.04;
383 |   letter-spacing: 0;
384 |   filter: drop-shadow(0 16px 44px rgba(56, 189, 248, 0.18));
385 | }
386 |
387 | .landing-copy > p:not(.eyebrow) {
388 |   max-width: 560px;
389 |   margin: 16px 0 0;
390 |   color: #b6c4d8;
391 |   font-size: 17px;
392 |   line-height: 1.55;
393 | }
394 |
395 | .landing-feature-grid {
396 |   display: grid;
397 |   grid-template-columns: repeat(3, minmax(0, 1fr));
398 |   gap: 12px;
399 |   margin-top: 20px;
400 | }
401 |
402 | .landing-feature-grid div {
403 |   min-width: 0;
404 |   padding: 13px;
405 |   border: 1px solid rgba(148, 163, 184, 0.18);
406 |   border-radius: 14px;
407 |   background: rgba(15, 23, 42, 0.62);
408 |   box-shadow: 0 18px 46px rgba(0, 0, 0, 0.2);
409 |   backdrop-filter: blur(16px);
410 |   transition:
411 |     transform 180ms ease,
412 |     border-color 180ms ease,
413 |     box-shadow 180ms ease,
414 |     background 180ms ease;
415 | }
416 |
417 | .landing-feature-grid div:hover {
418 |   border-color: rgba(45, 212, 191, 0.48);
419 |   background: rgba(15, 23, 42, 0.84);
420 |   box-shadow: 0 26px 64px rgba(20, 184, 166, 0.18);
421 |   transform: translateY(-6px) scale(1.02);
422 | }
423 |
424 | .landing-feature-grid strong,
425 | .landing-feature-grid span {
426 |   display: block;
427 | }
428 |
429 | .landing-feature-grid strong {
430 |   color: #f8fafc;
431 |   font-size: 14px;
432 | }
433 |
434 | .landing-feature-grid span {
435 |   margin-top: 6px;

```

```

436 | color: #94a3b8;
437 | font-size: 13px;
438 | line-height: 1.45;
439 | }
440 |
441 | .animated-word {
442 |   position: relative;
443 |   display: block;
444 |   min-height: 1.13em;
445 |   color: #1a73e8;
446 | }
447 |
448 | .animated-word span {
449 |   position: absolute;
450 |   inset: 0 auto auto 0;
451 |   opacity: 0;
452 |   transform: translateY(18px);
453 |   animation: wordSwap 7.2s infinite;
454 | }
455 |
456 | .animated-word span:nth-child(1) {
457 |   color: #60a5fa;
458 | }
459 |
460 | .animated-word span:nth-child(2) {
461 |   color: #2dd4bf;
462 |   animation-delay: 2.4s;
463 | }
464 |
465 | .animated-word span:nth-child(3) {
466 |   color: #f472b6;
467 |   animation-delay: 4.8s;
468 | }
469 |
470 | @keyframes wordSwap {
471 |   0%,
472 |   10% {
473 |     opacity: 0;
474 |     transform: translateY(18px);
475 |   }
476 |
477 |   18%,
478 |   43% {
479 |     opacity: 1;
480 |     transform: translateY(0);
481 |   }
482 |
483 |   51%,
484 |   100% {
485 |     opacity: 0;
486 |     transform: translateY(-18px);
487 |   }
488 | }
489 |
490 | .landing-actions {
491 |   display: flex;
492 |   flex-wrap: wrap;
493 |   gap: 12px;
494 |   margin-top: 24px;
495 | }
496 |
497 | .landing-actions button {
498 |   min-width: 190px;
499 |   min-height: 52px;
500 |   border: 1px solid rgba(255, 255, 255, 0.12);
501 |   box-shadow: 0 18px 48px rgba(37, 99, 235, 0.34);
502 |   transition:
503 |     transform 180ms ease,
504 |     box-shadow 180ms ease,
505 |     filter 180ms ease;
506 | }
507 |
508 | .landing-actions .primary-button {
509 |   background: linear-gradient(135deg, #2563eb 0%, #06b6d4 52%, #2dd4bf 100%);
510 | }
511 |
512 | .landing-actions .secondary-button {
513 |   background: linear-gradient(135deg, #7c3aed 0%, #db2777 55%, #f97316 100%);
514 | }
515 |
516 | .landing-actions button:hover {
517 |   box-shadow: 0 34px 80px rgba(45, 212, 191, 0.28);
518 |   filter: saturate(1.18) brightness(1.05);
519 |   transform: translateY(-7px) scale(1.04);
520 | }
521 |
522 | .landing-actions button:active {
523 |   transform: translateY(-2px) scale(1.01);
524 | }
525 |
526 | .landing-proof {
527 |   display: flex;
528 |   flex-wrap: wrap;
529 |   gap: 10px;
530 |   margin-top: 24px;
531 | }
532 |
533 | .landing-proof span {
534 |   border: 1px solid rgba(148, 163, 184, 0.22);
535 |   border-radius: 999px;
536 |   padding: 8px 12px;
537 |   background: rgba(15, 23, 42, 0.6);
538 |   color: #cbd5e1;
539 |   font-size: 13px;
540 |   font-weight: 900;
541 | }
542 |
543 | .landing-visual {
544 |   position: relative;
545 |   z-index: 2;

```

```

546 | min-height: calc(100vh - 126px);
547 | overflow: hidden;
548 | border: 1px solid rgba(96, 165, 250, 0.2);
549 | border-radius: 22px;
550 | background:
551 |   radial-gradient(circle at 50% 36%, rgba(59, 130, 246, 0.32), transparent 34%),
552 |   radial-gradient(circle at 74% 62%, rgba(45, 212, 191, 0.22), transparent 30%),
553 |   linear-gradient(145deg, rgba(15, 23, 42, 0.76), rgba(2, 6, 23, 0.9));
554 | box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.08), 0 34px 90px rgba(0, 0, 0, 0.36);
555 | transition:
556 |   transform 240ms ease,
557 |   box-shadow 240ms ease,
558 |   border-color 240ms ease;
559 | }
560 |
561 | .landing-visual::before {
562 |   content: '';
563 |   position: absolute;
564 |   inset: -20%;
565 |   z-index: 0;
566 |   background:
567 |     conic-gradient(from 120deg, transparent, rgba(56, 189, 248, 0.18), transparent, rgba(244, 114, 182, 0.16), transparent);
568 |   opacity: 0.75;
569 |   animation: visualAura 12s linear infinite;
570 | }
571 |
572 | @keyframes visualAura {
573 |   to {
574 |     transform: rotate(360deg);
575 |   }
576 | }
577 |
578 | .landing-visual:hover {
579 |   border-color: rgba(45, 212, 191, 0.42);
580 |   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.1), 0 42px 110px rgba(20, 184, 166, 0.18);
581 |   transform: translateY(-6px);
582 | }
583 |
584 | .three-stage {
585 |   position: relative;
586 |   height: calc(100vh - 150px);
587 |   min-height: 560px;
588 |   z-index: 2;
589 | }
590 |
591 | .three-canvas {
592 |   position: absolute;
593 |   inset: 0;
594 | }
595 |
596 | .three-canvas canvas {
597 |   display: block;
598 |   width: 100% !important;
599 |   height: 100% !important;
600 | }
601 |
602 | .visual-status-strip {
603 |   position: absolute;
604 |   right: 20px;
605 |   bottom: 20px;
606 |   left: 20px;
607 |   z-index: 3;
608 |   display: grid;
609 |   grid-template-columns: repeat(3, 1fr);
610 |   gap: 10px;
611 | }
612 |
613 | .visual-status-strip div {
614 |   display: grid;
615 |   gap: 2px;
616 |   min-width: 0;
617 |   padding: 14px;
618 |   border: 1px solid rgba(148, 163, 184, 0.2);
619 |   border-radius: 12px;
620 |   background:
621 |     linear-gradient(180deg, rgba(15, 23, 42, 0.84), rgba(15, 23, 42, 0.62));
622 |   box-shadow: 0 18px 48px rgba(0, 0, 0, 0.22);
623 |   backdrop-filter: blur(14px);
624 |   transition:
625 |     transform 180ms ease,
626 |     border-color 180ms ease,
627 |     box-shadow 180ms ease;
628 | }
629 |
630 | .visual-status-strip div:hover {
631 |   border-color: rgba(96, 165, 250, 0.44);
632 |   box-shadow: 0 26px 64px rgba(59, 130, 246, 0.18);
633 |   transform: translateY(-5px);
634 | }
635 |
636 | .visual-status-strip span {
637 |   overflow-wrap: anywhere;
638 |   color: #94a3b8;
639 |   font-size: 12px;
640 |   font-weight: 900;
641 |   text-transform: uppercase;
642 | }
643 |
644 | .visual-status-strip strong {
645 |   color: #f8fafc;
646 |   font-size: clamp(18px, 2vw, 24px);
647 | }
648 |
649 | .visual-status-strip div:nth-child(1) strong {
650 |   color: #60a5fa;
651 | }
652 |
653 | .visual-status-strip div:nth-child(2) strong {
654 |   color: #2dd4bf;
655 | }

```

```

656 |
657 | .visual-status-strip div:nth-child(3) strong {
658 |   color: #f472b6;
659 | }
660 |
661 | .auth-grid {
662 |   align-items: stretch;
663 |   max-width: 980px;
664 | }
665 |
666 | .panel,
667 | .metric {
668 |   background: rgba(255, 255, 255, 0.94);
669 |   border: 1px solid #e2e8f0;
670 |   border-radius: 8px;
671 |   box-shadow: 0 18px 50px rgba(15, 23, 42, 0.08);
672 | }
673 |
674 | .panel {
675 |   padding: 20px;
676 | }
677 |
678 | .wide {
679 |   min-width: 0;
680 | }
681 |
682 | .panel-title {
683 |   gap: 12px;
684 |   margin-bottom: 18px;
685 | }
686 |
687 | .between {
688 |   justify-content: space-between;
689 | }
690 |
691 | .title-inline,
692 | .toolbar {
693 |   gap: 10px;
694 | }
695 |
696 | .form-stack {
697 |   display: grid;
698 |   gap: 14px;
699 | }
700 |
701 | label {
702 |   display: grid;
703 |   gap: 7px;
704 |   color: #334155;
705 |   font-size: 13px;
706 |   font-weight: 700;
707 | }
708 |
709 | input,
710 | select,
711 | textarea {
712 |   width: 100%;
713 |   min-height: 42px;
714 |   box-sizing: border-box;
715 |   border: 1px solid #cbd5e1;
716 |   border-radius: 8px;
717 |   padding: 10px 12px;
718 |   background: #ffffff;
719 |   color: #172033;
720 |   font: inherit;
721 | }
722 |
723 | textarea {
724 |   min-height: 86px;
725 |   resize: vertical;
726 | }
727 |
728 | button {
729 |   border: 0;
730 |   font: inherit;
731 |   cursor: pointer;
732 | }
733 |
734 | button:disabled {
735 |   cursor: wait;
736 |   opacity: 0.68;
737 | }
738 |
739 | .primary-button,
740 | .secondary-button,
741 | .icon-button {
742 |   display: inline-flex;
743 |   align-items: center;
744 |   justify-content: center;
745 |   gap: 8px;
746 |   min-height: 42px;
747 |   border-radius: 8px;
748 |   padding: 0 14px;
749 |   font-weight: 800;
750 | }
751 |
752 | .primary-button {
753 |   background: #2563eb;
754 |   color: #ffffff;
755 | }
756 |
757 | .secondary-button {
758 |   background: #0f766e;
759 |   color: #ffffff;
760 | }
761 |
762 | .icon-button {
763 |   border: 1px solid #cbd5e1;
764 |   background: #ffffff;
765 |   color: #172033;

```



```

766 | }
767 |
768 | .text-button {
769 |   margin-top: 12px;
770 |   background: transparent;
771 |   color: #2563eb;
772 |   font-weight: 900;
773 | }
774 |
775 | .segmented {
776 |   display: grid;
777 |   grid-template-columns: 1fr 1fr;
778 |   gap: 8px;
779 |   margin-top: 14px;
780 | }
781 |
782 | .segmented button,
783 | .rating-grid button,
784 | .report-list button {
785 |   border: 1px solid #cbd5e1;
786 |   border-radius: 8px;
787 |   background: #ffffff;
788 |   color: #172033;
789 |   font-weight: 800;
790 | }
791 |
792 | .segmented button {
793 |   min-height: 38px;
794 | }
795 |
796 | .signal-panel {
797 |   display: grid;
798 |   align-content: center;
799 |   gap: 14px;
800 | }
801 |
802 | .auth-shell .signal-panel {
803 |   position: relative;
804 |   overflow: hidden;
805 |   align-content: stretch;
806 | }
807 |
808 | .login-orb {
809 |   display: grid;
810 |   place-items: center;
811 |   width: 112px;
812 |   height: 112px;
813 |   border: 1px solid rgba(125, 211, 252, 0.28);
814 |   border-radius: 28px;
815 |   background:
816 |     radial-gradient(circle at 30% 25%, rgba(125, 211, 252, 0.38), transparent 34%),
817 |     linear-gradient(135deg, rgba(37, 99, 235, 0.42), rgba(219, 39, 119, 0.28));
818 |   color: #e0f2fe;
819 |   box-shadow: 0 28px 78px rgba(37, 99, 235, 0.28);
820 |   transition:
821 |     transform 220ms ease,
822 |     box-shadow 220ms ease;
823 | }
824 |
825 | .login-orb:hover {
826 |   box-shadow: 0 36px 94px rgba(244, 114, 182, 0.24);
827 |   transform: translateY(-6px) rotate(-3deg) scale(1.04);
828 | }
829 |
830 | .auth-shell .signal-panel > div:nth-child(2) h2 {
831 |   margin: 0;
832 |   max-width: 430px;
833 |   background: linear-gradient(92deg, #e0f2fe 0%, #7dd3fc 40%, #f0abfc 100%);
834 |   background-clip: text;
835 |   color: transparent;
836 |   font-size: 30px;
837 |   line-height: 1.12;
838 | }
839 |
840 | .signal-row {
841 |   gap: 14px;
842 |   padding: 14px;
843 |   border-radius: 8px;
844 |   background: #f1f5f9;
845 | }
846 |
847 | .auth-shell .signal-row {
848 |   border: 1px solid rgba(148, 163, 184, 0.18);
849 |   border-radius: 14px;
850 |   background: rgba(2, 6, 23, 0.38);
851 |   color: #e2e8f0;
852 |   transition:
853 |     transform 180ms ease,
854 |     border-color 180ms ease,
855 |     box-shadow 180ms ease,
856 |     background 180ms ease;
857 | }
858 |
859 | .auth-shell .signal-row:hover {
860 |   border-color: rgba(45, 212, 191, 0.42);
861 |   background: rgba(15, 23, 42, 0.72);
862 |   box-shadow: 0 24px 62px rgba(20, 184, 166, 0.14);
863 |   transform: translateX(6px);
864 | }
865 |
866 | .auth-shell .signal-row svg {
867 |   color: #7dd3fc;
868 | }
869 |
870 | .signal-row strong,
871 | .signal-row span {
872 |   display: block;
873 | }
874 |
875 | .signal-row span {

```

```

876 | margin-top: 2px;
877 | color: #64748b;
878 | font-size: 14px;
879 | }
880 |
881 | .auth-shell .signal-row span {
882 |   color: #94a3b8;
883 | }
884 |
885 | .student-home {
886 |   display: grid;
887 |   grid-template-columns: minmax(280px, 360px) 1fr;
888 |   gap: 18px;
889 | }
890 |
891 | .student-profile-card,
892 | .student-hero-panel,
893 | .student-focused-panel {
894 |   border: 1px solid rgba(148, 163, 184, 0.2);
895 |   border-radius: 24px;
896 |   background:
897 |     linear-gradient(180deg, rgba(15, 23, 42, 0.9), rgba(15, 23, 42, 0.72)),
898 |     radial-gradient(circle at 12% 0%, rgba(59, 130, 246, 0.16), transparent 28%);
899 |   box-shadow: 0 34px 100px rgba(0, 0, 0, 0.22);
900 |   color: #e2e8f0;
901 | }
902 |
903 | .student-profile-card {
904 |   display: grid;
905 |   align-content: start;
906 |   gap: 18px;
907 |   min-height: 620px;
908 |   padding: 24px;
909 | }
910 |
911 | .student-avatar {
912 |   display: grid;
913 |   place-items: center;
914 |   width: 156px;
915 |   height: 156px;
916 |   border: 2px solid rgba(125, 211, 252, 0.4);
917 |   border-radius: 38px;
918 |   background:
919 |     radial-gradient(circle at 30% 24%, rgba(125, 211, 252, 0.34), transparent 36%),
920 |     linear-gradient(135deg, rgba(37, 99, 235, 0.5), rgba(219, 39, 119, 0.3));
921 |   color: #e0f2fe;
922 |   box-shadow: 0 26px 70px rgba(37, 99, 235, 0.24);
923 |   overflow: hidden;
924 |   transition:
925 |     transform 200ms ease,
926 |     box-shadow 200ms ease;
927 | }
928 |
929 | .avatar-button {
930 |   padding: 0;
931 |   cursor: zoom-in;
932 | }
933 |
934 | .student-avatar img {
935 |   width: 100%;
936 |   height: 100%;
937 |   object-fit: cover;
938 |   object-position: 52% 34%;
939 | }
940 |
941 | .student-avatar:hover {
942 |   box-shadow: 0 34px 86px rgba(244, 114, 182, 0.2);
943 |   transform: translateY(-5px) rotate(-2deg) scale(1.04);
944 | }
945 |
946 | .photo-preview-backdrop {
947 |   position: fixed;
948 |   inset: 0;
949 |   z-index: 80;
950 |   display: grid;
951 |   place-items: center;
952 |   padding: 24px;
953 |   background: rgba(2, 6, 23, 0.78);
954 |   backdrop-filter: blur(14px);
955 | }
956 |
957 | .photo-preview-modal {
958 |   position: relative;
959 |   display: grid;
960 |   place-items: center;
961 |   max-width: min(520px, 92vw);
962 |   max-height: 88vh;
963 |   border: 1px solid rgba(125, 211, 252, 0.28);
964 |   border-radius: 28px;
965 |   padding: 14px;
966 |   background:
967 |     radial-gradient(circle at 18% 0%, rgba(125, 211, 252, 0.18), transparent 28%),
968 |     rgba(15, 23, 42, 0.94);
969 |   box-shadow: 0 34px 120px rgba(0, 0, 0, 0.52);
970 | }
971 |
972 | .photo-preview-modal img {
973 |   display: block;
974 |   max-width: 100%;
975 |   max-height: 80vh;
976 |   border-radius: 20px;
977 |   object-fit: contain;
978 | }
979 |
980 | .photo-preview-close {
981 |   position: absolute;
982 |   top: 12px;
983 |   right: 12px;
984 |   min-height: 36px;
985 |   border: 1px solid rgba(255, 255, 255, 0.2);

```

```

986 | border-radius: 999px;
987 | padding: 0 12px;
988 | background: rgba(2, 6, 23, 0.74);
989 | color: #f8fafc;
990 | font-size: 12px;
991 | font-weight: 900;
992 | }
993 |
994 | .large-avatar {
995 |   width: 176px;
996 |   height: 176px;
997 | }
998 |
999 | .student-profile-card h2,
1000 | .student-hero-panel h1,
1001 | .student-focused-panel h2 {
1002 |   margin: 0;
1003 |   color: #f8fafc;
1004 | }
1005 |
1006 | .student-details {
1007 |   display: grid;
1008 |   gap: 12px;
1009 |   margin: 0;
1010 | }
1011 |
1012 | .student-details div {
1013 |   display: grid;
1014 |   gap: 4px;
1015 |   padding: 13px;
1016 |   border: 1px solid rgba(148, 163, 184, 0.16);
1017 |   border-radius: 14px;
1018 |   background: rgba(2, 6, 23, 0.38);
1019 |   transition:
1020 |     transform 180ms ease,
1021 |     border-color 180ms ease,
1022 |     background 180ms ease,
1023 |     box-shadow 180ms ease;
1024 | }
1025 |
1026 | .student-details div:hover {
1027 |   border-color: rgba(45, 212, 191, 0.42);
1028 |   background: rgba(15, 23, 42, 0.72);
1029 |   box-shadow: 0 20px 52px rgba(20, 184, 166, 0.12);
1030 |   transform: translateX(5px);
1031 | }
1032 |
1033 | .student-details dt {
1034 |   color: #94a3b8;
1035 |   font-size: 12px;
1036 |   font-weight: 900;
1037 |   text-transform: uppercase;
1038 | }
1039 |
1040 | .student-details dd {
1041 |   margin: 0;
1042 |   color: #e2e8f0;
1043 |   font-weight: 800;
1044 |   overflow-wrap: anywhere;
1045 | }
1046 |
1047 | .student-soft-button {
1048 |   min-height: 46px;
1049 |   border: 1px solid rgba(125, 211, 252, 0.26);
1050 |   border-radius: 14px;
1051 |   background: rgba(15, 23, 42, 0.66);
1052 |   color: #dbeafe;
1053 |   font-weight: 900;
1054 |   transition:
1055 |     transform 180ms ease,
1056 |     border-color 180ms ease,
1057 |     box-shadow 180ms ease,
1058 |     color 180ms ease;
1059 | }
1060 |
1061 | .student-soft-button:hover {
1062 |   border-color: rgba(45, 212, 191, 0.62);
1063 |   color: #ffffff;
1064 |   box-shadow: 0 24px 64px rgba(45, 212, 191, 0.16);
1065 |   transform: translateY(-4px);
1066 | }
1067 |
1068 | .student-details-editor {
1069 |   max-width: 980px;
1070 | }
1071 |
1072 | .locked-badge {
1073 |   border: 1px solid rgba(45, 212, 191, 0.28);
1074 |   border-radius: 999px;
1075 |   padding: 8px 12px;
1076 |   background: rgba(20, 184, 166, 0.12);
1077 |   color: #99f6e4;
1078 |   font-size: 12px;
1079 |   font-weight: 900;
1080 | }
1081 |
1082 | .student-edit-grid {
1083 |   display: grid;
1084 |   grid-template-columns: 230px 1fr;
1085 |   gap: 18px;
1086 |   align-items: start;
1087 | }
1088 |
1089 | .student-photo-editor {
1090 |   display: grid;
1091 |   gap: 14px;
1092 |   justify-items: center;
1093 |   border: 1px solid rgba(148, 163, 184, 0.18);
1094 |   border-radius: 20px;
1095 |   padding: 18px;

```

```

1096 | background: rgba(2, 6, 23, 0.38);
1097 | }
1098 |
1099 | .photo-upload-button {
1100 |   display: inline-flex;
1101 |   align-items: center;
1102 |   justify-content: center;
1103 |   min-height: 42px;
1104 |   border: 1px solid rgba(125, 211, 252, 0.28);
1105 |   border-radius: 14px;
1106 |   padding: 0 14px;
1107 |   background: rgba(15, 23, 42, 0.66);
1108 |   color: #d9eafe;
1109 |   cursor: pointer;
1110 |   font-weight: 900;
1111 |   transition:
1112 |     transform 180ms ease,
1113 |     border-color 180ms ease,
1114 |     box-shadow 180ms ease;
1115 | }
1116 |
1117 | .photo-upload-button:hover {
1118 |   border-color: rgba(45, 212, 191, 0.58);
1119 |   box-shadow: 0 20px 54px rgba(20, 184, 166, 0.14);
1120 |   transform: translateY(-4px);
1121 | }
1122 |
1123 | .photo-upload-button input {
1124 |   display: none;
1125 | }
1126 |
1127 | .locked-details-grid,
1128 | .editable-details-grid {
1129 |   display: grid;
1130 |   gap: 12px;
1131 | }
1132 |
1133 | .locked-details-grid {
1134 |   grid-template-columns: repeat(2, minmax(0, 1fr));
1135 | }
1136 |
1137 | .locked-details-grid div {
1138 |   display: grid;
1139 |   gap: 5px;
1140 |   border: 1px solid rgba(148, 163, 184, 0.16);
1141 |   border-radius: 16px;
1142 |   padding: 14px;
1143 |   background: rgba(2, 6, 23, 0.38);
1144 | }
1145 |
1146 | .locked-details-grid span {
1147 |   color: #94a3b8;
1148 |   font-size: 12px;
1149 |   font-weight: 900;
1150 |   text-transform: uppercase;
1151 | }
1152 |
1153 | .locked-details-grid strong {
1154 |   color: #f8fafc;
1155 |   overflow-wrap: anywhere;
1156 | }
1157 |
1158 | .editable-details-grid {
1159 |   grid-template-columns: repeat(3, minmax(0, 1fr));
1160 | }
1161 |
1162 | .student-details-editor input {
1163 |   border-color: rgba(148, 163, 184, 0.28);
1164 |   background: rgba(2, 6, 23, 0.56);
1165 |   color: #f8fafc;
1166 |   outline: none;
1167 | }
1168 |
1169 | .student-details-editor input:focus {
1170 |   border-color: rgba(45, 212, 191, 0.72);
1171 |   box-shadow: 0 0 0 4px rgba(45, 212, 191, 0.12);
1172 | }
1173 |
1174 | .student-hero-panel {
1175 |   display: grid;
1176 |   justify-items: center;
1177 |   align-content: center;
1178 |   min-height: 620px;
1179 |   padding: 34px;
1180 |   text-align: center;
1181 |   overflow: hidden;
1182 |   background:
1183 |     radial-gradient(circle at 50% 16%, rgba(37, 99, 235, 0.28), transparent 30%),
1184 |     radial-gradient(circle at 82% 76%, rgba(244, 114, 182, 0.18), transparent 26%),
1185 |     linear-gradient(145deg, rgba(15, 23, 42, 0.92), rgba(2, 6, 23, 0.9));
1186 | }
1187 |
1188 | .student-logo-stage {
1189 |   display: grid;
1190 |   place-items: center;
1191 |   width: min(560px, 92%);
1192 |   min-height: 270px;
1193 |   margin-bottom: 26px;
1194 |   border: 1px solid rgba(148, 163, 184, 0.18);
1195 |   border-radius: 28px;
1196 |   background:
1197 |     linear-gradient(180deg, rgba(255, 255, 255, 0.94), rgba(226, 232, 240, 0.92)),
1198 |     radial-gradient(circle at 40% 20%, rgba(125, 211, 252, 0.3), transparent 28%);
1199 |   box-shadow: 0 34px 90px rgba(37, 99, 235, 0.18);
1200 |   transition:
1201 |     transform 220ms ease,
1202 |     box-shadow 220ms ease,
1203 |     border-color 220ms ease;
1204 | }
1205 |

```

```

1206 | .student-logo-stage:hover {
1207 |   border-color: rgba(45, 212, 191, 0.52);
1208 |   box-shadow: 0 44px 110px rgba(20, 184, 166, 0.22);
1209 |   transform: translateY(-8px) scale(1.025);
1210 | }
1211 |
1212 | .student-logo-stage img {
1213 |   display: block;
1214 |   max-width: 90%;
1215 |   max-height: 215px;
1216 |   object-fit: contain;
1217 |   transition:
1218 |     transform 240ms ease,
1219 |     filter 240ms ease;
1220 | }
1221 |
1222 | .student-logo-stage:hover img {
1223 |   filter: saturate(1.1) contrast(1.05);
1224 |   transform: scale(1.05);
1225 | }
1226 |
1227 | .student-hero-panel h1 {
1228 |   margin-top: 6px;
1229 |   background: linear-gradient(92deg, #e0f2fe 0%, #7dd3fc 42%, #f0abfc 100%);
1230 |   background-clip: text;
1231 |   color: transparent;
1232 |   font-size: clamp(36px, 4.6vw, 62px);
1233 |   line-height: 1.05;
1234 | }
1235 |
1236 | .student-hero-panel > p:not(.eyebrow) {
1237 |   max-width: 620px;
1238 |   margin: 14px 0 0;
1239 |   color: #b6c4d8;
1240 |   font-size: 17px;
1241 |   line-height: 1.55;
1242 | }
1243 |
1244 | .student-action-grid {
1245 |   display: grid;
1246 |   grid-template-columns: repeat(2, minmax(0, 1fr));
1247 |   gap: 14px;
1248 |   width: min(720px, 100%);
1249 |   margin-top: 28px;
1250 | }
1251 |
1252 | .student-action-card {
1253 |   display: grid;
1254 |   justify-items: start;
1255 |   gap: 8px;
1256 |   min-height: 154px;
1257 |   border: 1px solid rgba(148, 163, 184, 0.18);
1258 |   border-radius: 20px;
1259 |   padding: 18px;
1260 |   color: #ffffff;
1261 |   text-align: left;
1262 |   box-shadow: 0 22px 62px rgba(0, 0, 0, 0.2);
1263 |   transition:
1264 |     transform 200ms ease,
1265 |     box-shadow 200ms ease,
1266 |     border-color 200ms ease,
1267 |     filter 200ms ease;
1268 | }
1269 |
1270 | .student-action-card strong {
1271 |   font-size: 18px;
1272 | }
1273 |
1274 | .student-action-card span {
1275 |   color: rgba(255, 255, 255, 0.78);
1276 |   font-size: 14px;
1277 |   line-height: 1.45;
1278 | }
1279 |
1280 | .student-action-card:hover {
1281 |   border-color: rgba(255, 255, 255, 0.24);
1282 |   filter: saturate(1.12) brightness(1.06);
1283 |   transform: translateY(-8px) scale(1.025);
1284 | }
1285 |
1286 | .primary-action {
1287 |   background: linear-gradient(135deg, #2563eb 0%, #06b6d4 52%, #2dd4bf 100%);
1288 | }
1289 |
1290 | .primary-action:hover {
1291 |   box-shadow: 0 34px 84px rgba(45, 212, 191, 0.2);
1292 | }
1293 |
1294 | .danger-action {
1295 |   background: linear-gradient(135deg, #7c3aed 0%, #db2777 58%, #f97316 100%);
1296 | }
1297 |
1298 | .danger-action:hover {
1299 |   box-shadow: 0 34px 84px rgba(244, 114, 182, 0.22);
1300 | }
1301 |
1302 | .student-form-wrap {
1303 |   position: relative;
1304 |   padding-top: 58px;
1305 | }
1306 |
1307 | .inline-back {
1308 |   top: 0;
1309 |   left: 0;
1310 | }
1311 |
1312 | .student-focused-panel {
1313 |   max-width: 760px;
1314 |   padding: 24px;
1315 | }

```

```

1316 |
1317 | .student-focused-panel {
1318 |   border-color: rgba(148, 163, 184, 0.2);
1319 |   border-radius: 24px;
1320 |   background:
1321 |     radial-gradient(circle at 18% 0%, rgba(244, 114, 182, 0.16), transparent 28%),
1322 |     radial-gradient(circle at 90% 12%, rgba(37, 99, 235, 0.14), transparent 24%),
1323 |     linear-gradient(180deg, rgba(15, 23, 42, 0.94), rgba(15, 23, 42, 0.76));
1324 |   box-shadow: 0 34px 100px rgba(0, 0, 0, 0.24);
1325 |   color: #e2e8f0;
1326 | }
1327 |
1328 | .student-focused-panel .panel-title svg {
1329 |   color: #f472b6;
1330 | }
1331 |
1332 | .student-focused-panel label {
1333 |   color: #cbd5e1;
1334 | }
1335 |
1336 | .student-focused-panel input,
1337 | .student-focused-panel textarea {
1338 |   border-color: rgba(148, 163, 184, 0.24);
1339 |   border-radius: 16px;
1340 |   background: rgba(2, 6, 23, 0.56);
1341 |   color: #f8fafc;
1342 |   outline: none;
1343 |   transition:
1344 |     transform 160ms ease,
1345 |     border-color 160ms ease,
1346 |     box-shadow 160ms ease,
1347 |     background 160ms ease;
1348 | }
1349 |
1350 | .student-focused-panel input:focus,
1351 | .student-focused-panel textarea:focus {
1352 |   border-color: rgba(244, 114, 182, 0.62);
1353 |   background: rgba(2, 6, 23, 0.78);
1354 |   box-shadow: 0 0 4px rgba(244, 114, 182, 0.1);
1355 |   transform: translateY(-2px);
1356 | }
1357 |
1358 | .feedback-workspace {
1359 |   border-color: rgba(148, 163, 184, 0.2);
1360 |   border-radius: 24px;
1361 |   padding: 24px;
1362 |   background:
1363 |     radial-gradient(circle at 18% 0%, rgba(37, 99, 235, 0.18), transparent 28%),
1364 |     radial-gradient(circle at 90% 12%, rgba(244, 114, 182, 0.12), transparent 24%),
1365 |     linear-gradient(180deg, rgba(15, 23, 42, 0.94), rgba(15, 23, 42, 0.76));
1366 |   box-shadow: 0 34px 100px rgba(0, 0, 0, 0.24);
1367 |   color: #e2e8f0;
1368 | }
1369 |
1370 | .feedback-workspace .panel-title svg {
1371 |   color: #7dd3fc;
1372 | }
1373 |
1374 | .feedback-workspace h2,
1375 | .feedback-workspace .category-block h3,
1376 | .feedback-workspace .question-row > span {
1377 |   color: #f8fafc;
1378 | }
1379 |
1380 | .feedback-intro {
1381 |   display: grid;
1382 |   grid-template-columns: 1fr auto;
1383 |   gap: 16px;
1384 |   align-items: center;
1385 |   margin-bottom: 18px;
1386 |   padding: 16px;
1387 |   border: 1px solid rgba(125, 211, 252, 0.18);
1388 |   border-radius: 18px;
1389 |   background: rgba(2, 6, 23, 0.42);
1390 | }
1391 |
1392 | .feedback-intro strong,
1393 | .feedback-intro span {
1394 |   display: block;
1395 | }
1396 |
1397 | .feedback-intro strong {
1398 |   color: #e0f2fe;
1399 |   font-size: 18px;
1400 | }
1401 |
1402 | .feedback-intro span {
1403 |   margin-top: 5px;
1404 |   color: #94a3b8;
1405 | }
1406 |
1407 | .feedback-scale {
1408 |   display: flex;
1409 |   flex-wrap: wrap;
1410 |   gap: 8px;
1411 |   justify-content: flex-end;
1412 | }
1413 |
1414 | .feedback-scale span {
1415 |   border: 1px solid rgba(148, 163, 184, 0.2);
1416 |   border-radius: 999px;
1417 |   padding: 7px 10px;
1418 |   background: rgba(15, 23, 42, 0.72);
1419 |   color: #cbd5e1;
1420 |   font-size: 12px;
1421 |   font-weight: 900;
1422 | }
1423 |
1424 | .feedback-workspace .category-block {
1425 |   border-color: rgba(148, 163, 184, 0.18);

```

```

1426 | border-radius: 20px;
1427 | background:
1428 |   linear-gradient(180deg, rgba(15, 23, 42, 0.8), rgba(2, 6, 23, 0.5)),
1429 |   radial-gradient(circle at 8% 0%, rgba(45, 212, 191, 0.08), transparent 24%);
1430 | transition:
1431 |   transform 180ms ease,
1432 |   border-color 180ms ease,
1433 |   box-shadow 180ms ease,
1434 |   background 180ms ease;
1435 | }
1436 |
1437 | .feedback-workspace .category-block:hover {
1438 |   border-color: rgba(45, 212, 191, 0.42);
1439 |   box-shadow: 0 30px 76px rgba(20, 184, 166, 0.12);
1440 |   transform: translateY(-5px);
1441 | }
1442 |
1443 | .feedback-workspace .category-block p {
1444 |   color: #94a3b8;
1445 | }
1446 |
1447 | .feedback-workspace .question-row {
1448 |   border-top-color: rgba(148, 163, 184, 0.18);
1449 | }
1450 |
1451 | .feedback-workspace .rating-grid {
1452 |   grid-template-columns: repeat(4, minmax(120px, 1fr));
1453 | }
1454 |
1455 | .feedback-workspace .rating-grid button {
1456 |   min-height: 44px;
1457 |   border-color: rgba(148, 163, 184, 0.22);
1458 |   border-radius: 14px;
1459 |   background: rgba(2, 6, 23, 0.46);
1460 |   color: #cbd5e1;
1461 |   transition:
1462 |     transform 160ms ease,
1463 |     border-color 160ms ease,
1464 |     color 160ms ease,
1465 |     background 160ms ease,
1466 |     box-shadow 160ms ease;
1467 | }
1468 |
1469 | .feedback-workspace .rating-grid button:hover {
1470 |   border-color: rgba(125, 211, 252, 0.5);
1471 |   color: #ffffff;
1472 |   box-shadow: 0 18px 46px rgba(37, 99, 235, 0.14);
1473 |   transform: translateY(-3px);
1474 | }
1475 |
1476 | .feedback-workspace .rating-grid .selected {
1477 |   border-color: rgba(45, 212, 191, 0.72);
1478 |   background: linear-gradient(135deg, rgba(37, 99, 235, 0.72), rgba(20, 184, 166, 0.62));
1479 |   color: #ffffff;
1480 |   box-shadow: 0 18px 52px rgba(45, 212, 191, 0.16);
1481 | }
1482 |
1483 | .feedback-comment {
1484 |   min-height: 90px;
1485 |   border-color: rgba(244, 114, 182, 0.32);
1486 |   border-radius: 16px;
1487 |   background: rgba(2, 6, 23, 0.62);
1488 |   color: #f8fafc;
1489 |   outline: none;
1490 |   box-shadow: 0 18px 44px rgba(244, 114, 182, 0.08);
1491 |   transition:
1492 |     transform 160ms ease,
1493 |     border-color 160ms ease,
1494 |     box-shadow 160ms ease;
1495 | }
1496 |
1497 | .feedback-comment:focus {
1498 |   border-color: rgba(244, 114, 182, 0.68);
1499 |   box-shadow: 0 0 4px rgba(244, 114, 182, 0.1), 0 20px 52px rgba(244, 114, 182, 0.12);
1500 |   transform: translateY(-2px);
1501 | }
1502 |
1503 | .feedback-workspace > .primary-button {
1504 |   min-height: 48px;
1505 |   margin-top: 18px;
1506 |   border-radius: 14px;
1507 |   background: linear-gradient(135deg, #2563eb 0%, #06b6d4 52%, #2dd4bf 100%);
1508 |   box-shadow: 0 20px 58px rgba(37, 99, 235, 0.24);
1509 |   transition:
1510 |     transform 180ms ease,
1511 |     box-shadow 180ms ease,
1512 |     filter 180ms ease;
1513 | }
1514 |
1515 | .feedback-workspace > .primary-button:hover {
1516 |   box-shadow: 0 30px 78px rgba(45, 212, 191, 0.22);
1517 |   filter: saturate(1.1) brightness(1.04);
1518 |   transform: translateY(-5px) scale(1.015);
1519 | }
1520 |
1521 | .category-stack,
1522 | .admin-stack,
1523 | .insight-list,
1524 | .report-list {
1525 |   display: grid;
1526 |   gap: 14px;
1527 | }
1528 |
1529 | .category-block {
1530 |   display: grid;
1531 |   gap: 14px;
1532 |   padding: 16px;
1533 |   border: 1px solid #e2e8f0;
1534 |   border-radius: 8px;
1535 |   background: #f8fafc;

```

```

1536 | }
1537 |
1538 | .category-block h3,
1539 | .two-column h3 {
1540 |   margin: 0 0 4px;
1541 |   color: #172033;
1542 | }
1543 |
1544 | .category-block p,
1545 | .report-summary p,
1546 | .insight-item p,
1547 | .action-item p,
1548 | .empty-state {
1549 |   margin: 0;
1550 |   color: #64748b;
1551 | }
1552 |
1553 | .question-row {
1554 |   display: grid;
1555 |   gap: 10px;
1556 |   padding-top: 12px;
1557 |   border-top: 1px solid #e2e8f0;
1558 | }
1559 |
1560 | .question-row > span {
1561 |   color: #172033;
1562 |   font-weight: 800;
1563 | }
1564 |
1565 | .rating-grid {
1566 |   display: grid;
1567 |   grid-template-columns: repeat(4, minmax(0, 1fr));
1568 |   gap: 8px;
1569 | }
1570 |
1571 | .rating-grid button {
1572 |   min-height: 38px;
1573 |   padding: 0 8px;
1574 |   font-size: 13px;
1575 | }
1576 |
1577 | .rating-grid .selected {
1578 |   border-color: #2563eb;
1579 |   background: #dbeafe;
1580 |   color: #1d4ed8;
1581 | }
1582 |
1583 | .check-row {
1584 |   display: flex;
1585 |   align-items: center;
1586 |   gap: 10px;
1587 | }
1588 |
1589 | .check-row input {
1590 |   width: 18px;
1591 |   min-height: 18px;
1592 | }
1593 |
1594 | .success-text,
1595 | .error-text {
1596 |   margin: 14px 0 0;
1597 |   font-weight: 800;
1598 | }
1599 |
1600 | .success-text {
1601 |   color: #0f766e;
1602 | }
1603 |
1604 | .error-text {
1605 |   color: #dc2626;
1606 | }
1607 |
1608 | .stats-grid {
1609 |   display: grid;
1610 |   grid-template-columns: repeat(4, minmax(0, 1fr));
1611 |   gap: 14px;
1612 | }
1613 |
1614 | .metric {
1615 |   display: grid;
1616 |   gap: 8px;
1617 |   padding: 16px;
1618 | }
1619 |
1620 | .metric-icon {
1621 |   width: 38px;
1622 |   height: 38px;
1623 |   background: #dbeafe;
1624 |   color: #2563eb;
1625 | }
1626 |
1627 | .metric-icon svg {
1628 |   width: 20px;
1629 |   height: 20px;
1630 | }
1631 |
1632 | .metric span {
1633 |   color: #64748b;
1634 |   font-weight: 700;
1635 | }
1636 |
1637 | .metric strong {
1638 |   color: #172033;
1639 |   font-size: 28px;
1640 | }
1641 |
1642 | .chart-box {
1643 |   width: 100%;
1644 |   min-height: 260px;
1645 | }

```



```

1646 |
1647 | .chart-box.compact {
1648 |   min-height: 220px;
1649 | }
1650 |
1651 | .legend-list {
1652 |   display: flex;
1653 |   flex-wrap: wrap;
1654 |   gap: 8px;
1655 | }
1656 |
1657 | .legend-list span {
1658 |   display: inline-flex;
1659 |   align-items: center;
1660 |   gap: 6px;
1661 |   color: #475569;
1662 |   font-size: 13px;
1663 |   font-weight: 800;
1664 | }
1665 |
1666 | .legend-list i {
1667 |   width: 10px;
1668 |   height: 10px;
1669 |   border-radius: 50%;
1670 | }
1671 |
1672 | .report-summary {
1673 |   display: grid;
1674 |   gap: 12px;
1675 |   padding: 14px;
1676 |   border-radius: 8px;
1677 |   background: #f1f5f9;
1678 | }
1679 |
1680 | .report-summary div {
1681 |   flex-wrap: wrap;
1682 |   gap: 8px;
1683 | }
1684 |
1685 | .report-summary span,
1686 | .pill {
1687 |   border-radius: 999px;
1688 |   padding: 6px 10px;
1689 |   font-size: 12px;
1690 |   font-weight: 900;
1691 | }
1692 |
1693 | .report-summary span {
1694 |   background: #ffffff;
1695 |   color: #334155;
1696 | }
1697 |
1698 | .two-column {
1699 |   display: grid;
1700 |   grid-template-columns: 1fr 1fr;
1701 |   gap: 14px;
1702 |   margin-top: 16px;
1703 | }
1704 |
1705 | .insight-item,
1706 | .action-item {
1707 |   display: grid;
1708 |   grid-template-columns: 1fr auto;
1709 |   gap: 12px;
1710 |   align-items: start;
1711 |   padding: 14px;
1712 |   border: 1px solid #e2e8f0;
1713 |   border-radius: 8px;
1714 | }
1715 |
1716 | .insight-item strong,
1717 | .action-item strong {
1718 |   display: block;
1719 |   margin-bottom: 6px;
1720 |   color: #172033;
1721 | }
1722 |
1723 | .action-item span:not(.pill) {
1724 |   display: block;
1725 |   margin-bottom: 6px;
1726 |   color: #0f766e;
1727 |   font-size: 13px;
1728 |   font-weight: 900;
1729 | }
1730 |
1731 | .pill.danger {
1732 |   background: #fee2e2;
1733 |   color: #b91c1c;
1734 | }
1735 |
1736 | .pill.warning {
1737 |   background: #fef3c7;
1738 |   color: #b45309;
1739 | }
1740 |
1741 | .pill.info {
1742 |   background: #dbeafe;
1743 |   color: #1d4ed8;
1744 | }
1745 |
1746 | .pill.calm {
1747 |   background: #dcfce7;
1748 |   color: #15803d;
1749 | }
1750 |
1751 | .report-list button {
1752 |   display: grid;
1753 |   gap: 4px;
1754 |   width: 100%;
1755 |   padding: 12px;

```

```

1756 | text-align: left;
1757 | }
1758 |
1759 | .report-list span {
1760 |   color: #64748b;
1761 |   font-size: 12px;
1762 | }
1763 |
1764 | .analysis-report-page {
1765 |   display: grid;
1766 |   gap: 16px;
1767 |   color: #eef3f7;
1768 |   font-family:
1769 |     "Segoe UI",
1770 |     Inter,
1771 |     system-ui,
1772 |     sans-serif;
1773 | }
1774 |
1775 | .analysis-back-button {
1776 |   justify-self: start;
1777 |   min-height: 38px;
1778 |   border: 1px solid rgba(84, 199, 122, 0.3);
1779 |   border-radius: 8px;
1780 |   padding: 0 14px;
1781 |   background: #1d2834;
1782 |   color: #dce7de;
1783 |   font-weight: 900;
1784 |   box-shadow: 0 12px 32px rgba(8, 13, 20, 0.2);
1785 |   transition:
1786 |     transform 180ms ease,
1787 |     border-color 180ms ease,
1788 |     box-shadow 180ms ease;
1789 | }
1790 |
1791 | .analysis-back-button:hover {
1792 |   border-color: rgba(85, 201, 119, 0.68);
1793 |   box-shadow: 0 18px 42px rgba(85, 201, 119, 0.14);
1794 |   transform: translateY(-2px);
1795 | }
1796 |
1797 | .analysis-report-hero,
1798 | .analysis-panel,
1799 | .analysis-kpi-card {
1800 |   border: 1px solid rgba(116, 135, 153, 0.28);
1801 |   background:
1802 |     linear-gradient(180deg, rgba(39, 52, 65, 0.96), rgba(34, 47, 60, 0.94)),
1803 |     #25313d;
1804 |   box-shadow:
1805 |     0 18px 46px rgba(6, 10, 16, 0.22),
1806 |     inset 0 1px 0 rgba(255, 255, 255, 0.035);
1807 | }
1808 |
1809 | .analysis-report-hero {
1810 |   display: grid;
1811 |   grid-template-columns: minmax(0, 1fr) auto;
1812 |   gap: 22px;
1813 |   align-items: start;
1814 |   overflow: hidden;
1815 |   border-radius: 10px;
1816 |   padding: 18px 20px;
1817 |   background:
1818 |     linear-gradient(90deg, rgba(33, 45, 58, 0.98), rgba(28, 40, 52, 0.96)),
1819 |     #202c38;
1820 | }
1821 |
1822 | .analysis-hero-copy {
1823 |   display: grid;
1824 |   gap: 10px;
1825 | }
1826 |
1827 | .analysis-hero-copy h1 {
1828 |   max-width: 860px;
1829 |   margin: 0;
1830 |   color: #f7fafc;
1831 |   font-size: clamp(26px, 3.4vw, 42px);
1832 |   line-height: 1.08;
1833 | }
1834 |
1835 | .analysis-hero-copy p:not(.eyebrow) {
1836 |   max-width: 920px;
1837 |   margin: 0;
1838 |   color: #b7c3ce;
1839 |   font-size: 14px;
1840 |   line-height: 1.5;
1841 | }
1842 |
1843 | .analysis-hero-badges,
1844 | .quality-chip-row,
1845 | .sentiment-tabs {
1846 |   display: flex;
1847 |   flex-wrap: wrap;
1848 |   gap: 8px;
1849 | }
1850 |
1851 | .analysis-hero-badges span,
1852 | .quality-chip-row span {
1853 |   border: 1px solid rgba(90, 110, 130, 0.42);
1854 |   border-radius: 7px;
1855 |   padding: 7px 10px;
1856 |   background: rgba(17, 26, 36, 0.58);
1857 |   color: #cbd7df;
1858 |   font-size: 12px;
1859 |   font-weight: 900;
1860 | }
1861 |
1862 | .analysis-hero-actions {
1863 |   display: grid;
1864 |   gap: 10px;
1865 |   min-width: 190px;

```

```

1866 | }
1867 |
1868 | .analysis-run-button,
1869 | .analysis-pdf-button {
1870 |   display: inline-flex;
1871 |   align-items: center;
1872 |   justify-content: center;
1873 |   gap: 9px;
1874 |   min-height: 48px;
1875 |   border-radius: 8px;
1876 |   padding: 0 16px;
1877 |   color: #ffffff;
1878 |   font-weight: 950;
1879 |   transition:
1880 |     transform 180ms ease,
1881 |     box-shadow 180ms ease,
1882 |     filter 180ms ease;
1883 | }
1884 |
1885 | .analysis-run-button {
1886 |   background: linear-gradient(135deg, #48b86f 0%, #65d48a 100%);
1887 |   color: #07110b;
1888 |   box-shadow: 0 16px 42px rgba(85, 201, 119, 0.24);
1889 | }
1890 |
1891 | .analysis-pdf-button {
1892 |   border: 1px solid rgba(85, 201, 119, 0.42);
1893 |   background: rgba(32, 44, 56, 0.86);
1894 |   color: #eaf6ed;
1895 | }
1896 |
1897 | .analysis-run-button:hover,
1898 | .analysis-pdf-button:hover {
1899 |   box-shadow: 0 22px 56px rgba(85, 201, 119, 0.18);
1900 |   filter: brightness(1.06) saturate(1.12);
1901 |   transform: translateY(-3px);
1902 | }
1903 |
1904 | .analysis-pdf-button:disabled {
1905 |   cursor: not-allowed;
1906 |   opacity: 0.48;
1907 |   transform: none;
1908 | }
1909 |
1910 | .analysis-notice {
1911 |   margin: 0;
1912 |   border: 1px solid rgba(85, 201, 119, 0.28);
1913 |   border-radius: 8px;
1914 |   padding: 12px 14px;
1915 |   background: rgba(72, 184, 111, 0.1);
1916 |   color: #dff5e5;
1917 |   font-size: 13px;
1918 |   font-weight: 900;
1919 | }
1920 |
1921 | .live-analysis-warning {
1922 |   display: inline-flex;
1923 |   align-items: center;
1924 |   gap: 10px;
1925 |   width: max-content;
1926 |   max-width: 100%;
1927 |   margin-top: 18px;
1928 |   border: 1px solid rgba(239, 68, 68, 0.32);
1929 |   border-radius: 999px;
1930 |   padding: 10px 14px;
1931 |   background:
1932 |     linear-gradient(135deg, rgba(239, 68, 68, 0.16), rgba(245, 158, 11, 0.1)),
1933 |     rgba(255, 255, 255, 0.08);
1934 |   color: #fecaca;
1935 |   box-shadow: 0 18px 42px rgba(239, 68, 68, 0.12);
1936 | }
1937 |
1938 | .report-live-warning {
1939 |   margin-top: 0;
1940 |   color: #7fd1d1;
1941 |   background:
1942 |     linear-gradient(135deg, rgba(254, 226, 226, 0.96), rgba(255, 247, 237, 0.94)),
1943 |     #ffffff;
1944 | }
1945 |
1946 | .live-analysis-dot {
1947 |   width: 12px;
1948 |   height: 12px;
1949 |   flex: 0 0 12px;
1950 |   border-radius: 999px;
1951 |   background: #ef4444;
1952 |   box-shadow:
1953 |     0 0 6px rgba(239, 68, 68, 0.16),
1954 |     0 0 26px rgba(239, 68, 68, 0.72);
1955 |   animation: feedbackLivePulse 1.15s ease-in-out infinite;
1956 | }
1957 |
1958 | .live-analysis-warning strong {
1959 |   color: inherit;
1960 |   font-weight: 950;
1961 | }
1962 |
1963 | .live-analysis-warning em {
1964 |   color: inherit;
1965 |   font-size: 0.86rem;
1966 |   font-style: normal;
1967 |   font-weight: 750;
1968 |   opacity: 0.88;
1969 | }
1970 |
1971 | .analysis-kpi-grid {
1972 |   display: grid;
1973 |   grid-template-columns: repeat(3, minmax(0, 1fr));
1974 |   gap: 14px;
1975 | }

```

```

1976 |
1977 | .analysis-kpi-card {
1978 |   display: grid;
1979 |   gap: 9px;
1980 |   min-height: 170px;
1981 |   border-radius: 10px;
1982 |   padding: 18px;
1983 |   transition:
1984 |     transform 180ms ease,
1985 |     border-color 180ms ease,
1986 |     box-shadow 180ms ease;
1987 | }
1988 |
1989 | .analysis-kpi-card:hover,
1990 | .analysis-panel:hover {
1991 |   border-color: rgba(85, 201, 119, 0.38);
1992 |   box-shadow: 0 24px 68px rgba(7, 13, 20, 0.28);
1993 |   transform: translateY(-3px);
1994 | }
1995 |
1996 | .analysis-kpi-card > span,
1997 | .analysis-panel-title > span {
1998 |   color: #a8b4bf;
1999 |   font-size: 12px;
2000 |   font-weight: 900;
2001 |   text-transform: uppercase;
2002 | }
2003 |
2004 | .analysis-kpi-card strong {
2005 |   color: #f8fafc;
2006 |   font-size: 42px;
2007 |   line-height: 1;
2008 | }
2009 |
2010 | .analysis-kpi-card p {
2011 |   margin: 0;
2012 |   color: #bfccd6;
2013 |   font-size: 13px;
2014 | }
2015 |
2016 | .analysis-kpi-card.cyan {
2017 |   background:
2018 |     radial-gradient(circle at 82% 18%, rgba(85, 201, 119, 0.18), transparent 38%),
2019 |     linear-gradient(180deg, rgba(39, 52, 65, 0.98), rgba(34, 47, 60, 0.94));
2020 | }
2021 |
2022 | .analysis-kpi-card.green {
2023 |   background:
2024 |     radial-gradient(circle at 82% 18%, rgba(147, 197, 114, 0.16), transparent 38%),
2025 |     linear-gradient(180deg, rgba(39, 52, 65, 0.98), rgba(34, 47, 60, 0.94));
2026 | }
2027 |
2028 | .analysis-kpi-card.rose {
2029 |   background:
2030 |     radial-gradient(circle at 82% 18%, rgba(236, 103, 89, 0.16), transparent 38%),
2031 |     linear-gradient(180deg, rgba(39, 52, 65, 0.98), rgba(34, 47, 60, 0.94));
2032 | }
2033 |
2034 | .analysis-progress {
2035 |   overflow: hidden;
2036 |   height: 12px;
2037 |   border-radius: 999px;
2038 |   background: #3c4956;
2039 | }
2040 |
2041 | .analysis-progress i {
2042 |   display: block;
2043 |   height: 100%;
2044 |   border-radius: inherit;
2045 |   background: linear-gradient(90deg, #4fbd75, #8adca1);
2046 | }
2047 |
2048 | .visual-dashboard-grid {
2049 |   display: grid;
2050 |   grid-template-columns: repeat(3, minmax(0, 1fr));
2051 |   gap: 14px;
2052 | }
2053 |
2054 | .visual-tile {
2055 |   min-height: 330px;
2056 | }
2057 |
2058 | .health-score-tile {
2059 |   align-content: space-between;
2060 | }
2061 |
2062 | .score-gauge {
2063 |   display: grid;
2064 |   place-items: center;
2065 |   width: 190px;
2066 |   height: 190px;
2067 |   margin: 4px auto;
2068 |   border-radius: 999px;
2069 |   background:
2070 |     radial-gradient(circle, #25313d 0 56%, transparent 57%),
2071 |     conic-gradient(#55c977 var(--score), #3c4956 0);
2072 |   box-shadow:
2073 |     inset 0 0 0 1px rgba(255, 255, 255, 0.04),
2074 |     0 18px 42px rgba(6, 10, 16, 0.25);
2075 | }
2076 |
2077 | .score-gauge div {
2078 |   display: grid;
2079 |   place-items: center;
2080 | }
2081 |
2082 | .score-gauge strong {
2083 |   color: #f8fafc;
2084 |   font-size: 52px;
2085 |   line-height: 1;

```

```

2086 | }
2087 |
2088 | .score-gauge span {
2089 |   color: #b7c3ce;
2090 |   font-size: 16px;
2091 |   font-weight: 900;
2092 | }
2093 |
2094 | .mini-metric-row {
2095 |   display: grid;
2096 |   grid-template-columns: repeat(2, minmax(0, 1fr));
2097 |   gap: 8px;
2098 | }
2099 |
2100 | .mini-metric-row span {
2101 |   border: 1px solid rgba(90, 110, 130, 0.38);
2102 |   border-radius: 8px;
2103 |   padding: 10px;
2104 |   background: rgba(18, 27, 37, 0.52);
2105 |   color: #dbe6df;
2106 |   font-size: 12px;
2107 |   font-weight: 900;
2108 |   text-align: center;
2109 | }
2110 |
2111 | .analysis-panel {
2112 |   overflow: hidden;
2113 |   border-radius: 10px;
2114 |   padding: 18px;
2115 |   transition:
2116 |     transform 180ms ease,
2117 |     border-color 180ms ease,
2118 |     box-shadow 180ms ease;
2119 | }
2120 |
2121 | .analysis-panel-title {
2122 |   display: flex;
2123 |   align-items: start;
2124 |   justify-content: space-between;
2125 |   gap: 14px;
2126 |   margin-bottom: 12px;
2127 | }
2128 |
2129 | .analysis-panel-title h2 {
2130 |   margin: 0;
2131 |   color: #f8fafc;
2132 |   font-size: 20px;
2133 | }
2134 |
2135 | .analysis-panel-title .eyebrow {
2136 |   margin-bottom: 4px;
2137 | }
2138 |
2139 | .sentiment-tabs {
2140 |   margin-bottom: 10px;
2141 | }
2142 |
2143 | .sentiment-tabs button {
2144 |   min-height: 34px;
2145 |   border: 1px solid rgba(90, 110, 130, 0.44);
2146 |   border-radius: 7px;
2147 |   padding: 0 11px;
2148 |   background: rgba(18, 27, 37, 0.52);
2149 |   color: #cbd6df;
2150 |   font-size: 12px;
2151 |   font-weight: 900;
2152 |   transition:
2153 |     transform 160ms ease,
2154 |     border-color 160ms ease,
2155 |     background 160ms ease,
2156 |     color 160ms ease;
2157 | }
2158 |
2159 | .sentiment-tabs button:hover,
2160 | .sentiment-tabs .selected {
2161 |   border-color: rgba(85, 201, 119, 0.58);
2162 |   background: rgba(85, 201, 119, 0.16);
2163 |   color: #f8fafc;
2164 |   transform: translateY(-2px);
2165 | }
2166 |
2167 | .category-sentiment-body {
2168 |   display: grid;
2169 |   grid-template-columns: minmax(190px, 1fr) minmax(150px, 0.7fr);
2170 |   gap: 8px;
2171 |   align-items: center;
2172 | }
2173 |
2174 | .sentiment-legend-panel {
2175 |   display: grid;
2176 |   gap: 10px;
2177 | }
2178 |
2179 | .sentiment-legend-panel span {
2180 |   display: grid;
2181 |   grid-template-columns: 10px 1fr auto;
2182 |   gap: 8px;
2183 |   align-items: center;
2184 |   color: #cbd6df;
2185 |   font-size: 13px;
2186 |   font-weight: 800;
2187 | }
2188 |
2189 | .sentiment-legend-panel i {
2190 |   width: 10px;
2191 |   height: 10px;
2192 |   border-radius: 999px;
2193 | }
2194 |
2195 | .trend-panel {

```

```

2196 | padding-bottom: 8px;
2197 | }
2198 |
2199 | .heatmap-panel,
2200 | .priority-matrix-panel {
2201 |   min-height: 330px;
2202 | }
2203 |
2204 | .issue-heatmap {
2205 |   display: grid;
2206 |   gap: 9px;
2207 |   margin-top: 16px;
2208 | }
2209 |
2210 | .heatmap-head,
2211 | .heatmap-row {
2212 |   display: grid;
2213 |   grid-template-columns: 92px repeat(5, minmax(0, 1fr));
2214 |   gap: 7px;
2215 |   align-items: center;
2216 | }
2217 |
2218 | .heatmap-head b {
2219 |   color: #a8b4bf;
2220 |   font-size: 11px;
2221 |   text-align: center;
2222 | }
2223 |
2224 | .heatmap-row strong {
2225 |   overflow: hidden;
2226 |   color: #eef3f7;
2227 |   font-size: 12px;
2228 |   text-overflow: ellipsis;
2229 |   white-space: nowrap;
2230 | }
2231 |
2232 | .heatmap-row span {
2233 |   height: 34px;
2234 |   border: 1px solid rgba(85, 201, 119, 0.16);
2235 |   border-radius: 7px;
2236 |   background:
2237 |     linear-gradient(
2238 |       180deg,
2239 |       rgba(85, 201, 119, calc(0.12 + var(--density) * 0.72)),
2240 |       rgba(49, 135, 79, calc(0.08 + var(--density) * 0.58))
2241 |     ),
2242 |     #1a2733;
2243 |   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.05);
2244 |   transition:
2245 |     transform 160ms ease,
2246 |     border-color 160ms ease;
2247 | }
2248 |
2249 | .heatmap-row span:hover {
2250 |   border-color: rgba(155, 228, 172, 0.56);
2251 |   transform: scale(1.08);
2252 | }
2253 |
2254 | .priority-matrix {
2255 |   position: relative;
2256 |   min-height: 245px;
2257 |   margin-top: 14px;
2258 |   border: 1px solid rgba(90, 110, 130, 0.38);
2259 |   border-radius: 10px;
2260 |   background:
2261 |     linear-gradient(90deg, rgba(255, 255, 255, 0.045) 1px, transparent 1px),
2262 |     linear-gradient(180deg, rgba(255, 255, 255, 0.045) 1px, transparent 1px),
2263 |     linear-gradient(135deg, rgba(85, 201, 119, 0.1), transparent 58%),
2264 |     rgba(18, 27, 37, 0.52);
2265 |   background-size:
2266 |     25% 100%,
2267 |     100% 25%,
2268 |     auto,
2269 |     auto;
2270 |   overflow: hidden;
2271 | }
2272 |
2273 | .priority-matrix button {
2274 |   position: absolute;
2275 |   display: grid;
2276 |   place-items: center;
2277 |   width: var(--bubble);
2278 |   height: var(--bubble);
2279 |   border: 1px solid rgba(155, 228, 172, 0.6);
2280 |   border-radius: 999px;
2281 |   background: linear-gradient(135deg, #55c977, #8adca1);
2282 |   color: #08110b;
2283 |   font-size: 11px;
2284 |   font-weight: 950;
2285 |   box-shadow: 0 14px 28px rgba(85, 201, 119, 0.2);
2286 |   transform: translate(-50%, 50%);
2287 |   transition:
2288 |     transform 160ms ease,
2289 |     box-shadow 160ms ease;
2290 | }
2291 |
2292 | .priority-matrix button:hover {
2293 |   box-shadow: 0 18px 36px rgba(85, 201, 119, 0.3);
2294 |   transform: translate(-50%, 50%) scale(1.12);
2295 | }
2296 |
2297 | .matrix-axis {
2298 |   position: absolute;
2299 |   color: #a8b4bf;
2300 |   font-size: 11px;
2301 |   font-weight: 900;
2302 |   text-transform: uppercase;
2303 | }
2304 |
2305 | .matrix-axis.y-axis {

```

```

2306 | top: 12px;
2307 | left: 12px;
2308 | }
2309 |
2310 | .matrix-axis.x-axis {
2311 |   right: 12px;
2312 |   bottom: 10px;
2313 | }
2314 |
2315 | .analysis-issue-table {
2316 |   display: grid;
2317 |   gap: 9px;
2318 | }
2319 |
2320 | .analysis-table-head,
2321 | .analysis-table-row {
2322 |   display: grid;
2323 |   grid-template-columns: minmax(0, 1fr) 92px 72px;
2324 |   gap: 10px;
2325 |   align-items: center;
2326 | }
2327 |
2328 | .analysis-table-head {
2329 |   color: #a8b4bf;
2330 |   font-size: 12px;
2331 |   font-weight: 900;
2332 |   text-transform: uppercase;
2333 | }
2334 |
2335 | .analysis-table-row {
2336 |   border: 1px solid rgba(90, 110, 130, 0.34);
2337 |   border-radius: 8px;
2338 |   padding: 12px;
2339 |   background: rgba(24, 35, 47, 0.66);
2340 |   transition:
2341 |     transform 160ms ease,
2342 |     border-color 160ms ease,
2343 |     background 160ms ease;
2344 | }
2345 |
2346 | .analysis-table-row:hover {
2347 |   border-color: rgba(85, 201, 119, 0.34);
2348 |   background: rgba(30, 44, 57, 0.92);
2349 |   transform: translateX(4px);
2350 | }
2351 |
2352 | .analysis-table-row strong,
2353 | .analysis-action-list strong,
2354 | .report-history-card strong {
2355 |   color: #f8fafc;
2356 | }
2357 |
2358 | .analysis-table-row p,
2359 | .analysis-action-list p {
2360 |   margin: 4px 0 0;
2361 |   color: #a6b3be;
2362 |   font-size: 13px;
2363 |   line-height: 1.4;
2364 | }
2365 |
2366 | .analysis-table-row b {
2367 |   color: #9be4ac;
2368 |   font-size: 18px;
2369 | }
2370 |
2371 | .analysis-action-list {
2372 |   display: grid;
2373 |   gap: 10px;
2374 | }
2375 |
2376 | .analysis-action-list article {
2377 |   display: grid;
2378 |   grid-template-columns: auto 1fr;
2379 |   gap: 12px;
2380 |   border: 1px solid rgba(90, 110, 130, 0.34);
2381 |   border-radius: 8px;
2382 |   padding: 12px;
2383 |   background: rgba(24, 35, 47, 0.66);
2384 |   transition:
2385 |     transform 160ms ease,
2386 |     border-color 160ms ease;
2387 | }
2388 |
2389 | .analysis-action-list article:hover {
2390 |   border-color: rgba(85, 201, 119, 0.34);
2391 |   transform: translateY(-3px);
2392 | }
2393 |
2394 | .analysis-action-list em {
2395 |   display: block;
2396 |   margin-top: 6px;
2397 |   color: #9be4ac;
2398 |   font-size: 12px;
2399 |   font-style: normal;
2400 |   font-weight: 900;
2401 | }
2402 |
2403 | .decision-intelligence-panel {
2404 |   display: grid;
2405 |   grid-template-columns: minmax(0, 1fr) minmax(310px, 0.38fr);
2406 |   gap: 18px;
2407 |   align-items: start;
2408 |   background:
2409 |     linear-gradient(180deg, rgba(39, 52, 65, 0.98), rgba(34, 47, 60, 0.96)),
2410 |     radial-gradient(circle at 0% 12%, rgba(85, 201, 119, 0.14), transparent 34%);
2411 | }
2412 |
2413 | .decision-intelligence-panel h2 {
2414 |   margin: 0 0 12px;
2415 |   color: #f8fafc;

```

```

2416 | }
2417 |
2418 | .decision-main,
2419 | .decision-side {
2420 |   display: grid;
2421 |   gap: 14px;
2422 | }
2423 |
2424 | .decision-card-grid {
2425 |   display: grid;
2426 |   grid-template-columns: repeat(3, minmax(0, 1fr));
2427 |   gap: 12px;
2428 | }
2429 |
2430 | .decision-card,
2431 | .decision-score-card,
2432 | .risk-focus-list article,
2433 | .report-history-card {
2434 |   display: grid;
2435 |   gap: 8px;
2436 |   border: 1px solid rgba(90, 110, 130, 0.34);
2437 |   border-radius: 8px;
2438 |   padding: 14px;
2439 |   background:
2440 |     linear-gradient(180deg, rgba(31, 44, 57, 0.9), rgba(25, 36, 49, 0.88)),
2441 |     rgba(24, 35, 47, 0.7);
2442 | }
2443 |
2444 | .decision-card span,
2445 | .decision-score-card span,
2446 | .risk-focus-head {
2447 |   color: #a8b4bf;
2448 |   font-size: 12px;
2449 |   font-weight: 900;
2450 |   text-transform: uppercase;
2451 | }
2452 |
2453 | .decision-card strong,
2454 | .decision-score-card strong {
2455 |   color: #f8fafc;
2456 |   font-size: 34px;
2457 |   line-height: 1;
2458 | }
2459 |
2460 | .decision-card p,
2461 | .decision-score-card p,
2462 | .risk-focus-list p {
2463 |   margin: 0;
2464 |   color: #aab7c2;
2465 |   font-size: 13px;
2466 |   line-height: 1.45;
2467 | }
2468 |
2469 | .risk-focus-list {
2470 |   display: grid;
2471 |   gap: 9px;
2472 | }
2473 |
2474 | .risk-focus-head,
2475 | .risk-focus-list article {
2476 |   display: grid;
2477 |   grid-template-columns: minmax(0, 1fr) auto;
2478 |   gap: 12px;
2479 |   align-items: center;
2480 | }
2481 |
2482 | .risk-focus-list article:hover {
2483 |   border-color: rgba(85, 201, 119, 0.36);
2484 |   transform: translateX(3px);
2485 | }
2486 |
2487 | .risk-focus-list strong {
2488 |   color: #f8fafc;
2489 | }
2490 |
2491 | .decision-score-card {
2492 |   background:
2493 |     linear-gradient(145deg, rgba(85, 201, 119, 0.2), rgba(25, 36, 49, 0.88)),
2494 |     rgba(24, 35, 47, 0.72);
2495 | }
2496 |
2497 | .report-history-card button {
2498 |   display: grid;
2499 |   gap: 4px;
2500 |   width: 100%;
2501 |   border: 1px solid rgba(90, 110, 130, 0.32);
2502 |   border-radius: 8px;
2503 |   padding: 10px;
2504 |   background: rgba(20, 30, 41, 0.72);
2505 |   color: #cbd6df;
2506 |   text-align: left;
2507 |   transition:
2508 |     transform 160ms ease,
2509 |     border-color 160ms ease;
2510 | }
2511 |
2512 | .report-history-card button:hover {
2513 |   border-color: rgba(85, 201, 119, 0.38);
2514 |   transform: translateY(-2px);
2515 | }
2516 |
2517 | .report-history-card em {
2518 |   color: #a8b4bf;
2519 |   font-size: 12px;
2520 |   font-style: normal;
2521 | }
2522 |
2523 | .text-insight-grid {
2524 |   display: grid;
2525 |   grid-template-columns: repeat(4, minmax(0, 1fr));

```



```

2526 | gap: 14px;
2527 | }
2528 |
2529 | .text-insight-card {
2530 |   display: grid;
2531 |   gap: 8px;
2532 |   min-height: 158px;
2533 |   border: 1px solid rgba(90, 110, 130, 0.34);
2534 |   border-radius: 10px;
2535 |   padding: 16px;
2536 |   background:
2537 |     linear-gradient(180deg, rgba(39, 52, 65, 0.96), rgba(30, 42, 54, 0.94)),
2538 |     #25313d;
2539 |   box-shadow: 0 18px 46px rgba(6, 10, 16, 0.2);
2540 |   transition:
2541 |     transform 160ms ease,
2542 |     border-color 160ms ease,
2543 |     box-shadow 160ms ease;
2544 | }
2545 |
2546 | .text-insight-card:hover {
2547 |   border-color: rgba(85, 201, 119, 0.36);
2548 |   box-shadow: 0 24px 64px rgba(7, 13, 20, 0.28);
2549 |   transform: translateY(-3px);
2550 | }
2551 |
2552 | .text-insight-card span {
2553 |   color: #a8b4bf;
2554 |   font-size: 11px;
2555 |   font-weight: 900;
2556 |   text-transform: uppercase;
2557 | }
2558 |
2559 | .text-insight-card strong {
2560 |   display: -webkit-box;
2561 |   overflow: hidden;
2562 |   color: #f8fafc;
2563 |   font-size: 20px;
2564 |   line-height: 1.22;
2565 |   -webkit-box-orient: vertical;
2566 |   -webkit-line-clamp: 2;
2567 | }
2568 |
2569 | .text-insight-card p {
2570 |   display: -webkit-box;
2571 |   margin: 0;
2572 |   overflow: hidden;
2573 |   color: #aab7c2;
2574 |   font-size: 13px;
2575 |   line-height: 1.45;
2576 |   -webkit-box-orient: vertical;
2577 |   -webkit-line-clamp: 3;
2578 | }
2579 |
2580 | .app-shell:has(.analysis-report-page) {
2581 |   background: #e7ebf0;
2582 | }
2583 |
2584 | .workspace:has(.analysis-report-page) {
2585 |   width: min(1220px, calc(100% - 32px));
2586 |   margin-top: 22px;
2587 |   margin-bottom: 22px;
2588 |   border: 1px solid #d1d5db;
2589 |   border-radius: 4px;
2590 |   padding: 18px;
2591 |   background: #ffffff;
2592 |   box-shadow: 0 18px 50px rgba(15, 23, 42, 0.12);
2593 | }
2594 |
2595 | .analysis-report-page {
2596 |   gap: 16px;
2597 |   color: #111827;
2598 |   font-family: "Segoe UI", Inter, system-ui, sans-serif;
2599 | }
2600 |
2601 | .analysis-paper-header {
2602 |   display: grid;
2603 |   grid-template-columns: auto 1fr auto;
2604 |   gap: 14px;
2605 |   align-items: center;
2606 |   border-bottom: 2px solid #111827;
2607 |   padding-bottom: 12px;
2608 | }
2609 |
2610 | .paper-logo-mark {
2611 |   justify-self: start;
2612 |   border: 1px solid #111827;
2613 |   border-radius: 3px;
2614 |   padding: 8px 12px;
2615 |   background: #f8fafc;
2616 |   color: #111827;
2617 |   font-size: 14px;
2618 |   font-weight: 900;
2619 | }
2620 |
2621 | .analysis-paper-title {
2622 |   display: grid;
2623 |   grid-template-columns: minmax(0, 1fr) auto;
2624 |   gap: 14px;
2625 |   align-items: end;
2626 | }
2627 |
2628 | .analysis-paper-title p {
2629 |   margin: 0;
2630 |   color: #64748b;
2631 |   font-size: 12px;
2632 |   font-weight: 900;
2633 |   text-transform: uppercase;
2634 | }
2635 |

```

```

2636 | .analysis-paper-title h1 {
2637 |   margin: 0;
2638 |   color: #111827;
2639 |   font-size: clamp(26px, 4vw, 42px);
2640 |   line-height: 1.05;
2641 | }
2642 |
2643 | .paper-title-metrics {
2644 |   display: flex;
2645 |   flex-wrap: wrap;
2646 |   justify-content: end;
2647 |   gap: 8px;
2648 |   max-width: 520px;
2649 | }
2650 |
2651 | .paper-title-metrics span {
2652 |   border: 1px solid #d1d5db;
2653 |   border-radius: 3px;
2654 |   padding: 7px 9px;
2655 |   background: #f8fafc;
2656 |   color: #334155;
2657 |   font-size: 12px;
2658 |   font-weight: 800;
2659 | }
2660 |
2661 | .analysis-back-button,
2662 | .analysis-run-button,
2663 | .analysis-pdf-button,
2664 | .result-row button,
2665 | .exact-issue-header button {
2666 |   min-height: 38px;
2667 |   border-radius: 3px;
2668 |   font-weight: 900;
2669 |   transition:
2670 |     transform 150ms ease,
2671 |     box-shadow 150ms ease,
2672 |     border-color 150ms ease,
2673 |     background 150ms ease;
2674 | }
2675 |
2676 | .analysis-back-button {
2677 |   border: 1px solid #111827;
2678 |   padding: 0 12px;
2679 |   background: #ffffff;
2680 |   color: #111827;
2681 |   box-shadow: none;
2682 | }
2683 |
2684 | .analysis-run-button {
2685 |   border: 1px solid #111827;
2686 |   background: #111827;
2687 |   color: #ffffff;
2688 |   box-shadow: none;
2689 | }
2690 |
2691 | .analysis-pdf-button {
2692 |   border: 1px solid #16a34a;
2693 |   background: #22c55e;
2694 |   color: #052e16;
2695 |   box-shadow: none;
2696 | }
2697 |
2698 | .analysis-back-button:hover,
2699 | .analysis-run-button:hover,
2700 | .analysis-pdf-button:hover,
2701 | .result-row button:hover,
2702 | .exact-issue-header button:hover {
2703 |   box-shadow: 0 10px 20px rgba(15, 23, 42, 0.12);
2704 |   transform: translateY(-1px);
2705 | }
2706 |
2707 | .analysis-hero-actions {
2708 |   display: flex;
2709 |   gap: 8px;
2710 |   min-width: 0;
2711 | }
2712 |
2713 | .paper-top-grid {
2714 |   display: grid;
2715 |   grid-template-columns: minmax(180px, 0.85fr) minmax(240px, 1.1fr) 160px 160px;
2716 |   gap: 14px;
2717 |   align-items: stretch;
2718 | }
2719 |
2720 | .paper-middle-grid {
2721 |   display: grid;
2722 |   grid-template-columns: minmax(0, 1.55fr) minmax(280px, 0.7fr);
2723 |   gap: 14px;
2724 | }
2725 |
2726 | .paper-bottom-grid {
2727 |   display: grid;
2728 |   grid-template-columns: minmax(0, 1.25fr) minmax(360px, 0.9fr);
2729 |   gap: 14px;
2730 | }
2731 |
2732 | .paper-card,
2733 | .paper-circle-card,
2734 | .exact-issue-card {
2735 |   border: 1.5px solid #1f2937;
2736 |   border-radius: 3px;
2737 |   background: #ffffff;
2738 |   color: #111827;
2739 |   box-shadow: none;
2740 | }
2741 |
2742 | .paper-card {
2743 |   padding: 16px;
2744 | }
2745 |

```

```

2746 | .paper-card-title {
2747 |   display: flex;
2748 |   align-items: center;
2749 |   justify-content: space-between;
2750 |   gap: 12px;
2751 |   margin-bottom: 10px;
2752 | }
2753 |
2754 | .paper-card-title span {
2755 |   color: #111827;
2756 |   font-size: 15px;
2757 |   font-weight: 900;
2758 | }
2759 |
2760 | .paper-card-title small {
2761 |   color: #64748b;
2762 |   font-size: 12px;
2763 |   font-weight: 800;
2764 | }
2765 |
2766 | .feedback-health-card {
2767 |   display: grid;
2768 |   align-content: space-between;
2769 |   min-height: 208px;
2770 | }
2771 |
2772 | .paper-score {
2773 |   display: flex;
2774 |   align-items: end;
2775 |   gap: 5px;
2776 | }
2777 |
2778 | .paper-score strong {
2779 |   color: #111827;
2780 |   font-size: 48px;
2781 |   line-height: 0.95;
2782 | }
2783 |
2784 | .paper-score span {
2785 |   color: #475569;
2786 |   font-size: 20px;
2787 |   font-weight: 900;
2788 | }
2789 |
2790 | .paper-meter {
2791 |   overflow: hidden;
2792 |   height: 12px;
2793 |   border: 1px solid #d1d5db;
2794 |   border-radius: 2px;
2795 |   background: #f1f5f9;
2796 | }
2797 |
2798 | .paper-meter i {
2799 |   display: block;
2800 |   height: 100%;
2801 |   background: #facc15;
2802 | }
2803 |
2804 | .paper-circle-card {
2805 |   display: grid;
2806 |   place-items: center;
2807 |   align-content: center;
2808 |   min-height: 160px;
2809 |   border-radius: 999px;
2810 |   padding: 18px;
2811 |   text-align: center;
2812 | }
2813 |
2814 | .paper-circle-card span {
2815 |   color: #334155;
2816 |   font-size: 13px;
2817 |   font-weight: 900;
2818 | }
2819 |
2820 | .paper-circle-card strong {
2821 |   color: #111827;
2822 |   font-size: 30px;
2823 |   line-height: 1;
2824 | }
2825 |
2826 | .paper-circle-card em {
2827 |   color: #64748b;
2828 |   font-size: 12px;
2829 |   font-style: normal;
2830 |   font-weight: 800;
2831 | }
2832 |
2833 | .paper-circle-card.risk {
2834 |   background: #fff7ed;
2835 |   border-color: #c2410c;
2836 | }
2837 |
2838 | .histogram-card {
2839 |   min-height: 340px;
2840 | }
2841 |
2842 | .sentiment-card {
2843 |   min-height: 340px;
2844 | }
2845 |
2846 | .analysis-report-page .sentiment-tabs {
2847 |   gap: 6px;
2848 |   margin-bottom: 8px;
2849 | }
2850 |
2851 | .analysis-report-page .sentiment-tabs button {
2852 |   min-height: 28px;
2853 |   border: 1px solid #d1d5db;
2854 |   border-radius: 2px;
2855 |   padding: 0 8px;

```

```

2856 | background: #f8fafc;
2857 | color: #334155;
2858 | font-size: 11px;
2859 | font-weight: 850;
2860 | }
2861 |
2862 | .analysis-report-page .sentiment-tabs button:hover,
2863 | .analysis-report-page .sentiment-tabs .selected {
2864 |   border-color: #ca8a04;
2865 |   background: #fef3c7;
2866 |   color: #111827;
2867 |   transform: none;
2868 | }
2869 |
2870 | .paper-legend {
2871 |   grid-template-columns: 1fr 1fr;
2872 |   gap: 7px;
2873 | }
2874 |
2875 | .paper-legend span {
2876 |   color: #334155;
2877 |   font-size: 12px;
2878 | }
2879 |
2880 | .results-card {
2881 |   min-height: 360px;
2882 | }
2883 |
2884 | .result-list {
2885 |   display: grid;
2886 |   gap: 9px;
2887 | }
2888 |
2889 | .result-row {
2890 |   display: grid;
2891 |   grid-template-columns: 30px minmax(0, 1fr) auto auto;
2892 |   gap: 10px;
2893 |   align-items: center;
2894 |   border-bottom: 1px solid #e5e7eb;
2895 |   padding: 9px 0;
2896 | }
2897 |
2898 | .result-row b {
2899 |   display: grid;
2900 |   place-items: center;
2901 |   width: 24px;
2902 |   height: 24px;
2903 |   border: 1px solid #111827;
2904 |   border-radius: 999px;
2905 |   color: #111827;
2906 |   font-size: 12px;
2907 | }
2908 |
2909 | .result-row strong {
2910 |   overflow: hidden;
2911 |   color: #111827;
2912 |   font-size: 15px;
2913 |   text-overflow: ellipsis;
2914 |   white-space: nowrap;
2915 | }
2916 |
2917 | .result-row span {
2918 |   border: 1px solid #d1d5db;
2919 |   border-radius: 2px;
2920 |   padding: 7px 9px;
2921 |   background: #f8fafc;
2922 |   color: #111827;
2923 |   font-size: 12px;
2924 |   font-weight: 900;
2925 | }
2926 |
2927 | .result-row button {
2928 |   border: 1px solid #111827;
2929 |   padding: 0 10px;
2930 |   background: #ffffff;
2931 |   color: #111827;
2932 | }
2933 |
2934 | .heatmap-card {
2935 |   min-height: 360px;
2936 | }
2937 |
2938 | .yellow-heatmap .heatmap-head,
2939 | .yellow-heatmap .heatmap-row {
2940 |   grid-template-columns: 96px repeat(5, minmax(38px, 1fr));
2941 | }
2942 |
2943 | .yellow-heatmap .heatmap-head b {
2944 |   color: #334155;
2945 | }
2946 |
2947 | .yellow-heatmap .heatmap-row strong {
2948 |   color: #111827;
2949 | }
2950 |
2951 | .yellow-heatmap .heatmap-row span {
2952 |   height: 34px;
2953 |   border: 1px solid #e5e7eb;
2954 |   border-radius: 2px;
2955 |   background:
2956 |     linear-gradient(
2957 |       180deg,
2958 |       rgba(250, 204, 21, calc(0.12 + var(--satisfaction) * 0.88)),
2959 |       rgba(234, 179, 8, calc(0.08 + var(--satisfaction) * 0.72))
2960 |     ),
2961 |     #f1f5f9;
2962 |   box-shadow: none;
2963 | }
2964 |
2965 | .yellow-heatmap .heatmap-row span:hover {

```

```

2966 | border-color: #ca8a04;
2967 | transform: scale(1.04);
2968 | }
2969 |
2970 | .exact-issue-card {
2971 |   display: grid;
2972 |   gap: 14px;
2973 |   padding: 16px;
2974 |   background: #ffffbeb;
2975 |   border-color: #ca8a04;
2976 | }
2977 |
2978 | .exact-issue-header {
2979 |   display: flex;
2980 |   justify-content: space-between;
2981 |   gap: 12px;
2982 |   align-items: start;
2983 | }
2984 |
2985 | .exact-issue-header span,
2986 | .exact-issue-grid span,
2987 | .exact-action-box span {
2988 |   color: #92400e;
2989 |   font-size: 12px;
2990 |   font-weight: 950;
2991 |   text-transform: uppercase;
2992 | }
2993 |
2994 | .exact-issue-header h2 {
2995 |   margin: 2px 0 0;
2996 |   color: #111827;
2997 |   font-size: 24px;
2998 | }
2999 |
3000 | .exact-issue-header button {
3001 |   border: 1px solid #92400e;
3002 |   padding: 0 12px;
3003 |   background: #ffffff;
3004 |   color: #111827;
3005 | }
3006 |
3007 | .exact-issue-grid {
3008 |   display: grid;
3009 |   grid-template-columns: repeat(4, minmax(0, 1fr));
3010 |   gap: 10px;
3011 | }
3012 |
3013 | .exact-issue-grid div,
3014 | .exact-action-box {
3015 |   display: grid;
3016 |   gap: 5px;
3017 |   border: 1px solid #f3d27a;
3018 |   border-radius: 2px;
3019 |   padding: 10px;
3020 |   background: #ffffff;
3021 | }
3022 |
3023 | .exact-issue-grid strong,
3024 | .exact-action-box strong {
3025 |   color: #111827;
3026 |   font-size: 14px;
3027 | }
3028 |
3029 | .exact-issue-card p {
3030 |   margin: 0;
3031 |   color: #334155;
3032 |   line-height: 1.55;
3033 | }
3034 |
3035 | .exact-evidence-list {
3036 |   display: grid;
3037 |   grid-template-columns: repeat(3, minmax(0, 1fr));
3038 |   gap: 10px;
3039 | }
3040 |
3041 | .exact-evidence-list blockquote {
3042 |   margin: 0;
3043 |   border-left: 3px solid #eab308;
3044 |   padding: 10px;
3045 |   background: #ffffff;
3046 |   color: #334155;
3047 |   font-size: 13px;
3048 |   line-height: 1.4;
3049 | }
3050 |
3051 | .analysis-footer {
3052 |   justify-self: center;
3053 |   border: 1px solid #111827;
3054 |   border-radius: 2px;
3055 |   padding: 8px 20px;
3056 |   background: #ffffff;
3057 |   color: #111827;
3058 |   font-size: 13px;
3059 |   font-weight: 900;
3060 | }
3061 |
3062 | .admin-home-panel {
3063 |   display: grid;
3064 |   gap: 24px;
3065 |   min-height: 620px;
3066 |   align-content: center;
3067 |   border: 1px solid rgba(148, 163, 184, 0.22);
3068 |   border-radius: 28px;
3069 |   padding: 34px;
3070 |   background:
3071 |     radial-gradient(circle at 14% 10%, rgba(37, 99, 235, 0.22), transparent 30%),
3072 |     radial-gradient(circle at 86% 22%, rgba(244, 114, 182, 0.14), transparent 28%),
3073 |     linear-gradient(145deg, rgba(15, 23, 42, 0.96), rgba(2, 6, 23, 0.9));
3074 |   color: #e2e8f0;
3075 |   box-shadow: 0 34px 100px rgba(0, 0, 0, 0.24);

```

```

3076 | }
3077 |
3078 | .admin-home-header {
3079 |   display: grid;
3080 |   gap: 8px;
3081 |   max-width: 820px;
3082 | }
3083 |
3084 | .admin-home-header h1 {
3085 |   margin: 0;
3086 |   background: linear-gradient(92deg, #e0f2fe 0%, #7dd3fc 42%, #f0abfc 100%);
3087 |   background-clip: text;
3088 |   color: transparent;
3089 |   font-size: clamp(38px, 5vw, 68px);
3090 |   line-height: 1.05;
3091 | }
3092 |
3093 | .admin-home-header p:not(.eyebrow) {
3094 |   margin: 0;
3095 |   color: #b6c4d8;
3096 |   font-size: 17px;
3097 |   line-height: 1.55;
3098 | }
3099 |
3100 | .admin-home-notice {
3101 |   width: fit-content;
3102 |   border: 1px solid rgba(251, 191, 36, 0.32);
3103 |   border-radius: 8px;
3104 |   padding: 10px 12px;
3105 |   background: rgba(251, 191, 36, 0.12);
3106 |   color: #fde68a !important;
3107 |   font-weight: 850;
3108 | }
3109 |
3110 | .admin-analysis-term-picker {
3111 |   display: grid;
3112 |   gap: 12px;
3113 |   width: min(100%, 880px);
3114 |   border: 2px solid #38bdf8;
3115 |   border-radius: 16px;
3116 |   padding: 14px;
3117 |   background: #f8fafc;
3118 |   box-shadow:
3119 |     0 18px 44px rgba(56, 189, 248, 0.2),
3120 |     inset 0 1px 0 rgba(255, 255, 255, 0.9);
3121 | }
3122 |
3123 | .admin-analysis-term-picker span {
3124 |   color: #0f172a;
3125 |   font-size: 12px;
3126 |   font-weight: 950;
3127 |   letter-spacing: 0.08em;
3128 |   text-transform: uppercase;
3129 | }
3130 |
3131 | .analysis-dataset-head {
3132 |   display: flex;
3133 |   align-items: center;
3134 |   justify-content: space-between;
3135 |   gap: 12px;
3136 | }
3137 |
3138 | .analysis-dataset-head button {
3139 |   border: 1px solid #0f172a;
3140 |   border-radius: 999px;
3141 |   padding: 8px 12px;
3142 |   background: #0f172a;
3143 |   color: #ffffff;
3144 |   font-size: 12px;
3145 |   font-weight: 900;
3146 | }
3147 |
3148 | .analysis-dataset-head button:hover {
3149 |   background: #2563eb;
3150 |   border-color: #2563eb;
3151 | }
3152 |
3153 | .analysis-dataset-options {
3154 |   display: grid;
3155 |   grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
3156 |   gap: 10px;
3157 | }
3158 |
3159 | .analysis-dataset-options button {
3160 |   display: grid;
3161 |   gap: 5px;
3162 |   border: 2px solid #cbd5e1;
3163 |   border-radius: 12px;
3164 |   padding: 12px;
3165 |   background: #ffffff;
3166 |   color: #0f172a;
3167 |   text-align: left;
3168 |   transition:
3169 |     transform 160ms ease,
3170 |     border-color 160ms ease,
3171 |     box-shadow 160ms ease,
3172 |     background 160ms ease;
3173 | }
3174 |
3175 | .analysis-dataset-options button:hover {
3176 |   border-color: #38bdf8;
3177 |   box-shadow: 0 12px 28px rgba(14, 165, 233, 0.18);
3178 |   transform: translateY(-2px);
3179 | }
3180 |
3181 | .analysis-dataset-options button.selected {
3182 |   border-color: #2563eb;
3183 |   background: #0f172a;
3184 |   color: #ffffff;
3185 |   box-shadow: 0 16px 34px rgba(37, 99, 235, 0.28);

```

```

3186 | }
3187 |
3188 | .analysis-dataset-options strong {
3189 |   font-size: 13px;
3190 |   line-height: 1.25;
3191 | }
3192 |
3193 | .analysis-dataset-options em,
3194 | .analysis-dataset-options p {
3195 |   margin: 0;
3196 |   color: #64748b;
3197 |   font-size: 11px;
3198 |   font-style: normal;
3199 |   font-weight: 900;
3200 |   text-transform: uppercase;
3201 | }
3202 |
3203 | .analysis-dataset-options button.selected em {
3204 |   color: #93c5fd;
3205 | }
3206 |
3207 | .admin-module-grid {
3208 |   display: grid;
3209 |   grid-template-columns: repeat(3, minmax(0, 1fr));
3210 |   gap: 16px;
3211 | }
3212 |
3213 | .admin-module-card {
3214 |   display: grid;
3215 |   gap: 12px;
3216 |   align-content: start;
3217 |   min-height: 260px;
3218 |   border: 1px solid rgba(125, 211, 252, 0.2);
3219 |   border-radius: 22px;
3220 |   padding: 22px;
3221 |   background:
3222 |     radial-gradient(circle at 20% 0%, rgba(125, 211, 252, 0.12), transparent 36%),
3223 |     rgba(2, 6, 23, 0.44);
3224 |   color: #e2e8f0;
3225 |   text-align: left;
3226 |   transition:
3227 |     transform 200ms ease,
3228 |     border-color 200ms ease,
3229 |     box-shadow 200ms ease,
3230 |     background 200ms ease;
3231 | }
3232 |
3233 | .admin-module-card:hover {
3234 |   border-color: rgba(45, 212, 191, 0.54);
3235 |   box-shadow: 0 34px 84px rgba(20, 184, 166, 0.14);
3236 |   transform: translateY(-8px) scale(1.015);
3237 | }
3238 |
3239 | .admin-module-card svg {
3240 |   color: #7dd3fc;
3241 | }
3242 |
3243 | .admin-module-card strong {
3244 |   color: #f8fafc;
3245 |   font-size: 24px;
3246 | }
3247 |
3248 | .admin-module-card span {
3249 |   color: #94a3b8;
3250 |   line-height: 1.5;
3251 | }
3252 |
3253 | .admin-module-card em {
3254 |   align-self: end;
3255 |   margin-top: auto;
3256 |   border-radius: 999px;
3257 |   padding: 8px 11px;
3258 |   background: rgba(20, 184, 166, 0.12);
3259 |   color: #99f6e4;
3260 |   font-size: 12px;
3261 |   font-style: normal;
3262 |   font-weight: 900;
3263 | }
3264 |
3265 | .admin-validation-panel {
3266 |   display: flex;
3267 |   align-items: center;
3268 |   justify-content: space-between;
3269 |   gap: 18px;
3270 |   width: min(100%, 880px);
3271 |   border: 1px solid rgba(125, 211, 252, 0.22);
3272 |   border-radius: 18px;
3273 |   padding: 16px;
3274 |   background:
3275 |     linear-gradient(135deg, rgba(14, 165, 233, 0.13), rgba(168, 85, 247, 0.08)),
3276 |     rgba(15, 23, 42, 0.72);
3277 |   box-shadow: 0 22px 64px rgba(2, 8, 23, 0.24);
3278 | }
3279 |
3280 | .admin-validation-panel.pass {
3281 |   border-color: rgba(34, 197, 94, 0.34);
3282 |   box-shadow: 0 26px 76px rgba(34, 197, 94, 0.14);
3283 | }
3284 |
3285 | .admin-validation-panel > div {
3286 |   display: flex;
3287 |   align-items: center;
3288 |   gap: 14px;
3289 |   min-width: 0;
3290 | }
3291 |
3292 | .validation-icon {
3293 |   display: grid;
3294 |   width: 46px;
3295 |   height: 46px;

```

```

3296 | flex: 0 0 auto;
3297 | place-items: center;
3298 | border-radius: 14px;
3299 | background: rgba(14, 165, 233, 0.16);
3300 | color: #7dd3fc;
3301 | }
3302 |
3303 | .admin-validation-panel strong {
3304 |   display: block;
3305 |   color: #f8fafc;
3306 |   font-size: 18px;
3307 | }
3308 |
3309 | .admin-validation-panel p {
3310 |   margin: 5px 0 0;
3311 |   color: #a9b8cd;
3312 |   font-size: 13px;
3313 |   line-height: 1.45;
3314 | }
3315 |
3316 | .admin-validation-panel button {
3317 |   min-height: 44px;
3318 |   flex: 0 0 auto;
3319 |   border: 0;
3320 |   border-radius: 12px;
3321 |   padding: 0 16px;
3322 |   background: linear-gradient(135deg, #38bdf8, #818cf8);
3323 |   color: #06111f;
3324 |   font-weight: 950;
3325 |   cursor: pointer;
3326 |   transition: transform 0.2s ease, box-shadow 0.2s ease, opacity 0.2s ease;
3327 | }
3328 |
3329 | .admin-validation-panel button:hover:not(:disabled) {
3330 |   transform: translateY(-2px);
3331 |   box-shadow: 0 20px 42px rgba(56, 189, 248, 0.24);
3332 | }
3333 |
3334 | .admin-validation-panel button:disabled {
3335 |   cursor: wait;
3336 |   opacity: 0.7;
3337 | }
3338 |
3339 | .admin-validation-result {
3340 |   display: grid;
3341 |   grid-template-columns: 220px minmax(0, 1fr);
3342 |   gap: 14px;
3343 |   width: min(100%, 880px);
3344 |   border: 1px solid rgba(148, 163, 184, 0.2);
3345 |   border-radius: 18px;
3346 |   padding: 14px;
3347 |   background: rgba(15, 23, 42, 0.66);
3348 | }
3349 |
3350 | .admin-validation-result > div:first-child {
3351 |   display: grid;
3352 |   align-content: center;
3353 |   gap: 4px;
3354 |   border-radius: 14px;
3355 |   padding: 16px;
3356 |   background: rgba(2, 6, 23, 0.42);
3357 | }
3358 |
3359 | .admin-validation-result span {
3360 |   color: #22c55e;
3361 |   font-size: 12px;
3362 |   font-weight: 950;
3363 |   letter-spacing: 0.08em;
3364 | }
3365 |
3366 | .admin-validation-result strong {
3367 |   color: #f8fafc;
3368 |   font-size: 38px;
3369 |   line-height: 1;
3370 | }
3371 |
3372 | .admin-validation-result p {
3373 |   margin: 0;
3374 |   color: #a9b8cd;
3375 |   font-size: 12px;
3376 |   line-height: 1.45;
3377 | }
3378 |
3379 | .validation-case-list {
3380 |   display: grid;
3381 |   grid-template-columns: repeat(2, minmax(0, 1fr));
3382 |   gap: 8px;
3383 | }
3384 |
3385 | .validation-case-list article {
3386 |   display: grid;
3387 |   gap: 5px;
3388 |   border-radius: 12px;
3389 |   padding: 10px;
3390 |   background: rgba(15, 23, 42, 0.72);
3391 | }
3392 |
3393 | .validation-case-list article b {
3394 |   width: fit-content;
3395 |   border-radius: 999px;
3396 |   padding: 4px 7px;
3397 |   background: rgba(34, 197, 94, 0.14);
3398 |   color: #86efac;
3399 |   font-size: 10px;
3400 | }
3401 |
3402 | .validation-case-list article.failed b {
3403 |   background: rgba(248, 113, 113, 0.14);
3404 |   color: #fca5a5;
3405 | }

```



```

3406 |
3407 | .validation-case-list article strong {
3408 |     color: #e2e8f0;
3409 |     font-size: 13px;
3410 |     line-height: 1.25;
3411 | }
3412 |
3413 | .validation-case-list article p {
3414 |     color: #94a3b8;
3415 | }
3416 |
3417 | .danger-module:hover {
3418 |     border-color: rgba(244, 114, 182, 0.5);
3419 |     box-shadow: 0 34px 84px rgba(244, 114, 182, 0.14);
3420 | }
3421 |
3422 | .admin-back-button {
3423 |     position: relative;
3424 |     top: auto;
3425 |     left: auto;
3426 |     justify-self: start;
3427 | }
3428 |
3429 | .feedback-builder-panel {
3430 |     display: grid;
3431 |     gap: 16px;
3432 | }
3433 |
3434 | .builder-category-list,
3435 | .builder-question-list {
3436 |     display: grid;
3437 |     gap: 12px;
3438 | }
3439 |
3440 | .builder-category {
3441 |     display: grid;
3442 |     gap: 13px;
3443 |     border: 1px solid rgba(148, 163, 184, 0.18);
3444 |     border-radius: 18px;
3445 |     padding: 16px;
3446 |     background: rgba(2, 6, 23, 0.38);
3447 | }
3448 |
3449 | .feedback-builder-panel input {
3450 |     border-color: rgba(148, 163, 184, 0.28);
3451 |     background: rgba(2, 6, 23, 0.54);
3452 |     color: #f8fafc;
3453 | }
3454 |
3455 | .danger-outline {
3456 |     border-color: rgba(244, 114, 182, 0.34);
3457 |     color: #fecdd3;
3458 |     background: rgba(127, 29, 29, 0.16);
3459 | }
3460 |
3461 | .admin-notice {
3462 |     margin: 0;
3463 |     border: 1px solid rgba(45, 212, 191, 0.24);
3464 |     border-radius: 16px;
3465 |     padding: 12px 14px;
3466 |     background: rgba(20, 184, 166, 0.1);
3467 |     color: #ccfbf1;
3468 |     font-size: 13px;
3469 |     font-weight: 900;
3470 | }
3471 |
3472 | .admin-hero-panel {
3473 |     display: grid;
3474 |     grid-template-columns: minmax(0, 1fr) auto;
3475 |     gap: 18px;
3476 |     align-items: start;
3477 |     border: 1px solid rgba(148, 163, 184, 0.22);
3478 |     border-radius: 24px;
3479 |     padding: 24px;
3480 |     background:
3481 |         radial-gradient(circle at 14% 8%, rgba(37, 99, 235, 0.24), transparent 28%),
3482 |         radial-gradient(circle at 88% 26%, rgba(244, 114, 182, 0.16), transparent 26%),
3483 |         linear-gradient(145deg, rgba(15, 23, 42, 0.96), rgba(2, 6, 23, 0.9));
3484 |     color: #e2e8f0;
3485 |     box-shadow: 0 34px 100px rgba(0, 0, 0, 0.22);
3486 | }
3487 |
3488 | .admin-hero-panel h1 {
3489 |     margin: 0;
3490 |     background: linear-gradient(92deg, #e0f2fe 0%, #7dd3fc 42%, #f0abfc 100%);
3491 |     background-clip: text;
3492 |     color: transparent;
3493 |     font-size: clamp(34px, 4vw, 58px);
3494 |     line-height: 1.05;
3495 | }
3496 |
3497 | .admin-hero-panel p:not(.eyebrow) {
3498 |     max-width: 760px;
3499 |     margin: 12px 0 0;
3500 |     color: #b6c4d8;
3501 |     font-size: 16px;
3502 |     line-height: 1.55;
3503 | }
3504 |
3505 | .admin-hero-actions {
3506 |     display: flex;
3507 |     gap: 10px;
3508 |     align-items: center;
3509 |     justify-content: flex-end;
3510 | }
3511 |
3512 | .admin-flow-strip {
3513 |     display: grid;
3514 |     grid-column: 1 / -1;
3515 |     grid-template-columns: repeat(4, minmax(0, 1fr));

```

```

3516 |     gap: 10px;
3517 | }
3518 |
3519 | .admin-flow-strip span {
3520 |     border: 1px solid rgba(125, 211, 252, 0.2);
3521 |     border-radius: 14px;
3522 |     padding: 12px;
3523 |     background: rgba(2, 6, 23, 0.44);
3524 |     color: #dbeafe;
3525 |     font-size: 13px;
3526 |     font-weight: 900;
3527 |     text-align: center;
3528 |     transition:
3529 |         transform 180ms ease,
3530 |         border-color 180ms ease,
3531 |         box-shadow 180ms ease;
3532 | }
3533 |
3534 | .admin-flow-strip span:hover {
3535 |     border-color: rgba(45, 212, 191, 0.48);
3536 |     box-shadow: 0 20px 52px rgba(20, 184, 166, 0.12);
3537 |     transform: translateY(-4px);
3538 | }
3539 |
3540 | .admin-stack .panel,
3541 | .admin-stack .metric {
3542 |     border-color: rgba(148, 163, 184, 0.2);
3543 |     border-radius: 20px;
3544 |     background:
3545 |         linear-gradient(180deg, rgba(15, 23, 42, 0.94), rgba(15, 23, 42, 0.76)),
3546 |         radial-gradient(circle at 12% 0%, rgba(59, 130, 246, 0.12), transparent 28%);
3547 |     color: #e2e8f0;
3548 |     box-shadow: 0 30px 86px rgba(0, 0, 0, 0.18);
3549 | }
3550 |
3551 | .admin-stack .panel h2,
3552 | .admin-stack .metric strong,
3553 | .admin-stack .insight-item strong,
3554 | .admin-stack .action-item strong,
3555 | .admin-stack .two-column h3,
3556 | .admin-ai-card h3 {
3557 |     color: #f8fafc;
3558 | }
3559 |
3560 | .admin-stack .panel-title svg,
3561 | .admin-stack .metric-icon svg {
3562 |     color: #7dd3fc;
3563 | }
3564 |
3565 | .admin-stack .metric {
3566 |     transition:
3567 |         transform 180ms ease,
3568 |         border-color 180ms ease,
3569 |         box-shadow 180ms ease;
3570 | }
3571 |
3572 | .admin-stack .metric:hover {
3573 |     border-color: rgba(45, 212, 191, 0.42);
3574 |     box-shadow: 0 34px 84px rgba(20, 184, 166, 0.13);
3575 |     transform: translateY(-5px);
3576 | }
3577 |
3578 | .admin-stack .metric-icon {
3579 |     border-radius: 14px;
3580 |     background: rgba(37, 99, 235, 0.18);
3581 |     color: #bfdbfe;
3582 | }
3583 |
3584 | .ai-engine-panel {
3585 |     display: grid;
3586 |     gap: 16px;
3587 | }
3588 |
3589 | .engine-status {
3590 |     border: 1px solid rgba(148, 163, 184, 0.24);
3591 |     border-radius: 999px;
3592 |     padding: 8px 12px;
3593 |     background: rgba(15, 23, 42, 0.68);
3594 |     color: #cbd5e1;
3595 |     font-size: 12px;
3596 |     font-weight: 900;
3597 | }
3598 |
3599 | .engine-status.online {
3600 |     border-color: rgba(45, 212, 191, 0.42);
3601 |     background: rgba(20, 184, 166, 0.14);
3602 |     color: #99f6e4;
3603 | }
3604 |
3605 | .ai-engine-grid {
3606 |     display: grid;
3607 |     grid-template-columns: repeat(4, minmax(0, 1fr));
3608 |     gap: 12px;
3609 | }
3610 |
3611 | .ai-engine-grid article {
3612 |     display: grid;
3613 |     gap: 5px;
3614 |     border: 1px solid rgba(148, 163, 184, 0.18);
3615 |     border-radius: 16px;
3616 |     padding: 14px;
3617 |     background:
3618 |         radial-gradient(circle at 12% 0%, rgba(125, 211, 252, 0.1), transparent 38%),
3619 |         rgba(2, 6, 23, 0.38);
3620 |     transition:
3621 |         transform 180ms ease,
3622 |         border-color 180ms ease,
3623 |         box-shadow 180ms ease;
3624 | }
3625 |

```

```

3626 | .ai-engine-grid article:hover {
3627 |   border-color: rgba(125, 211, 252, 0.42);
3628 |   box-shadow: 0 24px 64px rgba(37, 99, 235, 0.12);
3629 |   transform: translateY(-4px);
3630 | }
3631 |
3632 | .ai-engine-grid span {
3633 |   color: #94a3b8;
3634 |   font-size: 12px;
3635 |   font-weight: 900;
3636 |   text-transform: uppercase;
3637 | }
3638 |
3639 | .ai-engine-grid strong {
3640 |   color: #f8fafc;
3641 |   font-size: 20px;
3642 |   overflow-wrap: anywhere;
3643 | }
3644 |
3645 | .ai-engine-grid p {
3646 |   margin: 0;
3647 |   color: #94a3b8;
3648 |   font-size: 13px;
3649 | }
3650 |
3651 | .ai-pipeline-strip {
3652 |   display: grid;
3653 |   grid-template-columns: repeat(4, minmax(0, 1fr));
3654 |   gap: 10px;
3655 | }
3656 |
3657 | .ai-pipeline-strip span {
3658 |   display: inline-flex;
3659 |   align-items: center;
3660 |   justify-content: center;
3661 |   gap: 8px;
3662 |   min-height: 44px;
3663 |   border: 1px solid rgba(244, 114, 182, 0.18);
3664 |   border-radius: 14px;
3665 |   padding: 0 12px;
3666 |   background: rgba(2, 6, 23, 0.42);
3667 |   color: #e2e8f0;
3668 |   font-size: 13px;
3669 |   font-weight: 900;
3670 |   text-transform: capitalize;
3671 | }
3672 |
3673 | .ai-pipeline-strip b {
3674 |   color: #f0abfc;
3675 | }
3676 |
3677 | .admin-stack .metric span,
3678 | .admin-stack .legend-list span,
3679 | .admin-stack .report-list span,
3680 | .admin-stack .empty-state,
3681 | .admin-stack .insight-item p,
3682 | .admin-stack .action-item p,
3683 | .admin-stack .report-summary p {
3684 |   color: #94a3b8;
3685 | }
3686 |
3687 | .sentiment-panel {
3688 |   min-width: 0;
3689 | }
3690 |
3691 | .sentiment-content {
3692 |   display: grid;
3693 |   gap: 18px;
3694 |   justify-items: center;
3695 | }
3696 |
3697 | .sentiment-donut {
3698 |   position: relative;
3699 |   display: grid;
3700 |   place-items: center;
3701 |   width: 176px;
3702 |   height: 176px;
3703 | }
3704 |
3705 | .sentiment-donut svg {
3706 |   width: 176px;
3707 |   height: 176px;
3708 | }
3709 |
3710 | .sentiment-donut > div {
3711 |   position: absolute;
3712 |   display: grid;
3713 |   justify-items: center;
3714 | }
3715 |
3716 | .sentiment-donut strong {
3717 |   color: #f8fafc;
3718 |   font-size: 30px;
3719 | }
3720 |
3721 | .sentiment-donut span {
3722 |   color: #94a3b8;
3723 |   font-size: 12px;
3724 |   font-weight: 900;
3725 | }
3726 |
3727 | .sentiment-bars {
3728 |   display: grid;
3729 |   gap: 12px;
3730 |   width: 100%;
3731 | }
3732 |
3733 | .sentiment-bars div {
3734 |   display: grid;
3735 |   gap: 6px;

```

```

3736 | }
3737 |
3738 | .sentiment-bars span {
3739 |   display: flex;
3740 |   justify-content: space-between;
3741 |   color: #cbd5e1;
3742 |   font-size: 13px;
3743 |   font-weight: 800;
3744 | }
3745 |
3746 | .sentiment-bars i {
3747 |   display: block;
3748 |   height: 8px;
3749 |   overflow: hidden;
3750 |   border-radius: 999px;
3751 |   background: rgba(148, 163, 184, 0.18);
3752 | }
3753 |
3754 | .sentiment-bars em {
3755 |   display: block;
3756 |   height: 100%;
3757 |   border-radius: inherit;
3758 | }
3759 |
3760 | .issue-table {
3761 |   display: grid;
3762 |   gap: 10px;
3763 | }
3764 |
3765 | .issue-row {
3766 |   display: grid;
3767 |   grid-template-columns: 1fr auto;
3768 |   gap: 12px;
3769 |   align-items: center;
3770 |   border: 1px solid rgba(148, 163, 184, 0.18);
3771 |   border-radius: 16px;
3772 |   padding: 14px;
3773 |   background: rgba(2, 6, 23, 0.38);
3774 |   transition:
3775 |     transform 180ms ease,
3776 |     border-color 180ms ease,
3777 |     box-shadow 180ms ease;
3778 | }
3779 |
3780 | .issue-row:hover {
3781 |   border-color: rgba(244, 114, 182, 0.42);
3782 |   box-shadow: 0 24px 64px rgba(244, 114, 182, 0.11);
3783 |   transform: translateY(-4px);
3784 | }
3785 |
3786 | .issue-row strong {
3787 |   display: block;
3788 |   color: #f8fafc;
3789 |   font-size: 15px;
3790 | }
3791 |
3792 | .issue-row p {
3793 |   display: -webkit-box;
3794 |   margin: 5px 0;
3795 |   overflow: hidden;
3796 |   color: #94a3b8;
3797 |   font-size: 13px;
3798 |   -webkit-box-orient: vertical;
3799 |   -webkit-line-clamp: 2;
3800 | }
3801 |
3802 | .issue-row span {
3803 |   color: #7dd3fc;
3804 |   font-size: 12px;
3805 |   font-weight: 800;
3806 | }
3807 |
3808 | .issue-row-meta {
3809 |   display: grid;
3810 |   gap: 8px;
3811 |   justify-items: end;
3812 |   min-width: 130px;
3813 | }
3814 |
3815 | .admin-ai-card {
3816 |   display: grid;
3817 |   gap: 4px;
3818 | }
3819 |
3820 | .confidence-badge {
3821 |   border: 1px solid rgba(45, 212, 191, 0.26);
3822 |   border-radius: 999px;
3823 |   padding: 8px 12px;
3824 |   background: rgba(20, 184, 166, 0.12);
3825 |   color: #99f6e4;
3826 |   font-size: 12px;
3827 |   font-weight: 900;
3828 | }
3829 |
3830 | .ai-priority-grid {
3831 |   display: grid;
3832 |   grid-template-columns: minmax(0, 1fr) minmax(0, 1fr) minmax(260px, 0.85fr);
3833 |   gap: 14px;
3834 | }
3835 |
3836 | .admin-stack .insight-item,
3837 | .admin-stack .action-item,
3838 | .ai-summary-box,
3839 | .admin-stack .report-summary,
3840 | .admin-stack .report-list button {
3841 |   border-color: rgba(148, 163, 184, 0.18);
3842 |   background: rgba(2, 6, 23, 0.38);
3843 | }
3844 |
3845 | .ai-summary-box {

```

```

3846 | display: grid;
3847 | align-content: start;
3848 | gap: 10px;
3849 | border: 1px solid rgba(125, 211, 252, 0.2);
3850 | border-radius: 16px;
3851 | padding: 16px;
3852 | }
3853 |
3854 | .ai-summary-box svg {
3855 |   color: #f0abfc;
3856 | }
3857 |
3858 | .ai-summary-box strong {
3859 |   color: #f8fafc;
3860 | }
3861 |
3862 | .ai-summary-box p {
3863 |   margin: 0;
3864 |   color: #cbd5e1;
3865 |   line-height: 1.55;
3866 | }
3867 |
3868 | @media (max-width: 1100px) {
3869 |   .content-grid,
3870 |   .auth-grid,
3871 |   .two-column,
3872 |   .student-edit-grid,
3873 |   .ai-priority-grid {
3874 |     grid-template-columns: 1fr;
3875 |   }
3876 |
3877 |   .admin-hero-panel {
3878 |     grid-template-columns: 1fr;
3879 |   }
3880 |
3881 |   .admin-hero-actions {
3882 |     justify-content: flex-start;
3883 |   }
3884 |
3885 |   .admin-module-grid,
3886 |   .admin-validation-result,
3887 |   .validation-case-list {
3888 |     grid-template-columns: 1fr;
3889 |   }
3890 |
3891 |   .admin-validation-panel {
3892 |     align-items: stretch;
3893 |     flex-direction: column;
3894 |   }
3895 |
3896 |   .admin-validation-panel button {
3897 |     width: 100%;
3898 |   }
3899 |
3900 |   .analysis-report-hero,
3901 |   .decision-intelligence-panel {
3902 |     grid-template-columns: 1fr;
3903 |   }
3904 |
3905 |   .visual-dashboard-grid {
3906 |     grid-template-columns: repeat(2, minmax(0, 1fr));
3907 |   }
3908 |
3909 |   .text-insight-grid {
3910 |     grid-template-columns: repeat(2, minmax(0, 1fr));
3911 |   }
3912 |
3913 |   .analysis-hero-actions {
3914 |     grid-template-columns: repeat(2, minmax(0, 1fr));
3915 |     min-width: 0;
3916 |   }
3917 |
3918 |   .analysis-kpi-grid,
3919 |   .decision-card-grid {
3920 |     grid-template-columns: 1fr;
3921 |   }
3922 |
3923 |   .admin-flow-strip {
3924 |     grid-template-columns: repeat(2, minmax(0, 1fr));
3925 |   }
3926 |
3927 |   .ai-engine-grid,
3928 |   .editable-details-grid,
3929 |   .locked-details-grid,
3930 |   .ai-pipeline-strip {
3931 |     grid-template-columns: repeat(2, minmax(0, 1fr));
3932 |   }
3933 |
3934 |   .stats-grid {
3935 |     grid-template-columns: repeat(2, minmax(0, 1fr));
3936 |   }
3937 |
3938 |   .public-shell {
3939 |     padding: 14px;
3940 |   }
3941 |
3942 |   .landing-page {
3943 |     min-height: auto;
3944 |   }
3945 |
3946 |   .landing-hero-grid {
3947 |     grid-template-columns: 1fr;
3948 |     padding: 22px 24px 30px;
3949 |   }
3950 |
3951 |   .landing-copy {
3952 |     max-width: 760px;
3953 |   }
3954 |
3955 |   .landing-feature-grid {

```

```

3956 |     grid-template-columns: 1fr;
3957 | }
3958 |
3959 | .landing-visual {
3960 |     min-height: 480px;
3961 | }
3962 |
3963 | .three-stage {
3964 |     height: 410px;
3965 | }
3966 | }
3967 |
3968 | @media (max-width: 680px) {
3969 |     .workspace {
3970 |         padding: 18px;
3971 |     }
3972 |
3973 |     .topbar,
3974 |     .between {
3975 |         align-items: flex-start;
3976 |         flex-direction: column;
3977 |         gap: 12px;
3978 |     }
3979 |
3980 |     .stats-grid,
3981 |     .rating-grid {
3982 |         grid-template-columns: 1fr;
3983 |     }
3984 |
3985 |     .toolbar {
3986 |         width: 100%;
3987 |         flex-wrap: wrap;
3988 |     }
3989 |
3990 |     .toolbar button {
3991 |         flex: 1 1 140px;
3992 |     }
3993 |
3994 |     .landing-nav {
3995 |         align-items: flex-start;
3996 |         flex-direction: column;
3997 |         padding: 18px;
3998 |     }
3999 |
4000 |     .landing-hero-grid {
4001 |         padding: 18px;
4002 |     }
4003 |
4004 |     .landing-copy h1 {
4005 |         font-size: 40px;
4006 |     }
4007 |
4008 |     .landing-copy > p:not(.eyebrow) {
4009 |         font-size: 16px;
4010 |     }
4011 |
4012 |     .landing-actions button {
4013 |         width: 100%;
4014 |     }
4015 |
4016 |     .landing-visual {
4017 |         min-height: 390px;
4018 |     }
4019 |
4020 |     .three-stage {
4021 |         height: 310px;
4022 |     }
4023 |
4024 |     .visual-status-strip {
4025 |         position: relative;
4026 |         right: auto;
4027 |         bottom: auto;
4028 |         left: auto;
4029 |         grid-template-columns: 1fr;
4030 |         padding: 0 14px 14px;
4031 |         margin-top: -24px;
4032 |     }
4033 |
4034 |     .admin-flow-strip,
4035 |     .ai-engine-grid,
4036 |     .editable-details-grid,
4037 |     .locked-details-grid,
4038 |     .ai-pipeline-strip,
4039 |     .issue-row {
4040 |         grid-template-columns: 1fr;
4041 |     }
4042 |
4043 |     .issue-row-meta {
4044 |         justify-items: start;
4045 |     }
4046 |
4047 |     .workspace:has(.analysis-report-page) {
4048 |         width: min(100% - 20px, 1320px);
4049 |         margin-top: 10px;
4050 |         margin-bottom: 10px;
4051 |         padding: 12px;
4052 |     }
4053 |
4054 |     .analysis-report-hero,
4055 |     .analysis-panel,
4056 |     .analysis-kpi-card {
4057 |         border-radius: 10px;
4058 |         padding: 16px;
4059 |     }
4060 |
4061 |     .analysis-hero-copy h1 {
4062 |         font-size: 34px;
4063 |     }
4064 |
4065 |     .visual-dashboard-grid,

```

```

4066 | .text-insight-grid,
4067 | .analysis-hero-actions,
4068 | .category-sentiment-body,
4069 | .analysis-table-head,
4070 | .analysis-table-row,
4071 | .analysis-action-list article {
4072 |   grid-template-columns: 1fr;
4073 | }
4074 |
4075 | .analysis-hero-actions button {
4076 |   width: 100%;
4077 | }
4078 |
4079 | .analysis-panel-title {
4080 |   display: grid;
4081 | }
4082 |
4083 | .sentiment-tabs {
4084 |   overflow-x: auto;
4085 |   flex-wrap: nowrap;
4086 |   padding-bottom: 4px;
4087 | }
4088 |
4089 | .sentiment-tabs button {
4090 |   flex: 0 0 auto;
4091 | }
4092 |
4093 | .analysis-table-head {
4094 |   display: none;
4095 | }
4096 |
4097 | .analysis-table-row b {
4098 |   font-size: 16px;
4099 | }
4100 |
4101 | .visual-tile {
4102 |   min-height: auto;
4103 | }
4104 |
4105 | .score-gauge {
4106 |   width: 160px;
4107 |   height: 160px;
4108 | }
4109 |
4110 | .heatmap-head,
4111 | .heatmap-row {
4112 |   grid-template-columns: 78px repeat(5, minmax(34px, 1fr));
4113 | }
4114 |
4115 | .heatmap-row span {
4116 |   height: 30px;
4117 | }
4118 |
4119 | .priority-matrix {
4120 |   min-height: 220px;
4121 | }
4122 | }
4123 |
4124 | .app-shell:has(.premium-analytics-page) {
4125 |   background:
4126 |     linear-gradient(180deg, #f7f9fc 0%, #eef3f9 100%),
4127 |     #f7f9fc;
4128 | }
4129 |
4130 | .workspace:has(.premium-analytics-page) {
4131 |   width: min(1680px, calc(100% - 40px));
4132 |   margin: 20px auto 28px;
4133 |   border: 0;
4134 |   border-radius: 0;
4135 |   padding: 0;
4136 |   background: transparent;
4137 |   box-shadow: none;
4138 | }
4139 |
4140 | .premium-analytics-page {
4141 |   display: grid;
4142 |   gap: 22px;
4143 |   color: #0f172a;
4144 |   font-family: Inter, "Plus Jakarta Sans", "Segoe UI", system-ui, sans-serif;
4145 | }
4146 |
4147 | .premium-analytics-page * {
4148 |   letter-spacing: 0;
4149 | }
4150 |
4151 | .analytics-top-nav {
4152 |   position: sticky;
4153 |   top: 12px;
4154 |   z-index: 10;
4155 |   display: grid;
4156 |   grid-template-columns: auto 1fr;
4157 |   gap: 10px;
4158 |   align-items: center;
4159 |   border: 1px solid rgba(226, 232, 240, 0.78);
4160 |   border-radius: 8px;
4161 |   padding: 10px;
4162 |   background: rgba(255, 255, 255, 0.84);
4163 |   box-shadow: 0 18px 60px rgba(15, 23, 42, 0.08);
4164 |   backdrop-filter: blur(18px);
4165 | }
4166 |
4167 | .analytics-brand,
4168 | .analytics-filters button,
4169 | .nav-icon-button,
4170 | .nav-avatar,
4171 | .nav-secondary-action,
4172 | .nav-primary-action {
4173 |   display: inline-flex;
4174 |   align-items: center;
4175 |   justify-content: center;

```

```

4176 | gap: 8px;
4177 | min-height: 42px;
4178 | border: 0;
4179 | border-radius: 8px;
4180 | padding: 0 14px;
4181 | color: #334155;
4182 | font-weight: 850;
4183 | transition:
4184 |   transform 180ms ease,
4185 |   box-shadow 180ms ease,
4186 |   background 180ms ease,
4187 |   color 180ms ease;
4188 | }
4189 |
4190 | .analytics-brand {
4191 |   background: #0f172a;
4192 |   color: #ffffff;
4193 |   box-shadow: 0 12px 26px rgba(15, 23, 42, 0.18);
4194 | }
4195 |
4196 | .analytics-brand span {
4197 |   display: grid;
4198 |   place-items: center;
4199 |   width: 28px;
4200 |   height: 28px;
4201 |   border-radius: 8px;
4202 |   background: linear-gradient(135deg, #60a5fa, #34d399);
4203 | }
4204 |
4205 | .analytics-brand:hover,
4206 | .analytics-filters button:hover,
4207 | .nav-icon-button:hover,
4208 | .nav-avatar:hover,
4209 | .nav-secondary-action:hover,
4210 | .nav-primary-action:hover,
4211 | .premium-kpi-card:hover,
4212 | .ai-finding-card:hover,
4213 | .recommendation-list article:hover {
4214 |   transform: translateY(-1px);
4215 | }
4216 |
4217 | .global-search {
4218 |   display: flex;
4219 |   align-items: center;
4220 |   gap: 12px;
4221 |   min-height: 42px;
4222 |   border: 1px solid rgba(226, 232, 240, 0.92);
4223 |   border-radius: 8px;
4224 |   padding: 0 14px;
4225 |   background: #f8fafc;
4226 |   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.92);
4227 | }
4228 |
4229 | .global-search input {
4230 |   width: 100%;
4231 |   border: 0;
4232 |   outline: 0;
4233 |   background: transparent;
4234 |   color: #0f172a;
4235 |   font-size: 14px;
4236 |   font-weight: 650;
4237 | }
4238 |
4239 | .global-search input::placeholder {
4240 |   color: #94a3b8;
4241 | }
4242 |
4243 | .search-dot {
4244 |   position: relative;
4245 |   width: 15px;
4246 |   height: 15px;
4247 |   border: 2px solid #64748b;
4248 |   border-radius: 999px;
4249 | }
4250 |
4251 | .search-dot::after {
4252 |   content: "";
4253 |   position: absolute;
4254 |   right: -6px;
4255 |   bottom: -4px;
4256 |   width: 7px;
4257 |   height: 2px;
4258 |   border-radius: 999px;
4259 |   background: #64748b;
4260 |   transform: rotate(45deg);
4261 | }
4262 |
4263 | .analytics-filters {
4264 |   display: flex;
4265 |   flex-wrap: nowrap;
4266 |   justify-self: end;
4267 |   gap: 8px;
4268 |   width: max-content;
4269 | }
4270 |
4271 | .analytics-filters button,
4272 | .nav-icon-button,
4273 | .nav-avatar,
4274 | .nav-secondary-action {
4275 |   background: #ffffff;
4276 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.9);
4277 | }
4278 |
4279 | .nav-icon-button,
4280 | .nav-avatar {
4281 |   width: 42px;
4282 |   padding: 0;
4283 | }
4284 |
4285 | .nav-avatar {

```



```

4286 | background: linear-gradient(135deg, #e0f2fe, #dcfce7);
4287 | color: #0f766e;
4288 | }
4289 |
4290 | .nav-primary-action {
4291 |   background: linear-gradient(135deg, #2563eb, #16a34a);
4292 |   color: #ffffff;
4293 |   box-shadow: 0 14px 32px rgba(37, 99, 235, 0.24);
4294 | }
4295 |
4296 | .nav-secondary-action:disabled {
4297 |   cursor: not-allowed;
4298 |   opacity: 0.48;
4299 |   transform: none;
4300 | }
4301 |
4302 | .analytics-hero {
4303 |   display: grid;
4304 |   grid-template-columns: minmax(0, 1fr) minmax(280px, 420px);
4305 |   gap: 22px;
4306 |   align-items: stretch;
4307 |   border: 1px solid rgba(226, 232, 240, 0.78);
4308 |   border-radius: 8px;
4309 |   padding: clamp(26px, 4vw, 42px);
4310 |   background:
4311 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.86)),
4312 |     radial-gradient(circle at 12% 18%, rgba(96, 165, 250, 0.22), transparent 30%),
4313 |     radial-gradient(circle at 88% 22%, rgba(52, 211, 153, 0.18), transparent 28%);
4314 |   box-shadow: 0 24px 80px rgba(15, 23, 42, 0.08);
4315 | }
4316 |
4317 | .hero-kicker,
4318 | .premium-panel-header span,
4319 | .investigation-header span,
4320 | .investigation-copy-grid span,
4321 | .investigation-metric span {
4322 |   color: #2563eb;
4323 |   font-size: 12px;
4324 |   font-weight: 950;
4325 |   text-transform: uppercase;
4326 | }
4327 |
4328 | .analytics-hero h1 {
4329 |   max-width: 940px;
4330 |   margin: 10px 0 12px;
4331 |   color: #0f172a;
4332 |   font-size: clamp(36px, 5vw, 68px);
4333 |   line-height: 1.02;
4334 | }
4335 |
4336 | .hero-copy-modern {
4337 |   display: grid;
4338 |   align-content: center;
4339 |   gap: 16px;
4340 | }
4341 |
4342 | .hero-modern-kicker {
4343 |   width: fit-content;
4344 |   border-radius: 999px;
4345 |   padding: 8px 12px;
4346 |   background: #eff6ff;
4347 |   color: #2563eb;
4348 |   font-size: 12px;
4349 |   font-weight: 950;
4350 |   text-transform: uppercase;
4351 | }
4352 |
4353 | .hero-copy-modern p {
4354 |   max-width: 620px;
4355 |   margin: -4px 0 0;
4356 |   color: #475569;
4357 |   font-size: clamp(16px, 1.5vw, 20px);
4358 |   font-weight: 650;
4359 |   line-height: 1.55;
4360 | }
4361 |
4362 | .hero-animated-line {
4363 |   display: flex;
4364 |   align-items: center;
4365 |   gap: 10px;
4366 |   width: fit-content;
4367 |   max-width: 100%;
4368 |   overflow: hidden;
4369 |   border-radius: 999px;
4370 |   padding: 10px 14px;
4371 |   background: rgba(255, 255, 255, 0.82);
4372 |   box-shadow:
4373 |     inset 0 0 0 1px rgba(226, 232, 240, 0.86),
4374 |     0 16px 40px rgba(37, 99, 235, 0.1);
4375 | }
4376 |
4377 | .hero-animated-line span {
4378 |   color: #64748b;
4379 |   font-size: 13px;
4380 |   font-weight: 900;
4381 | }
4382 |
4383 | .hero-animated-line strong {
4384 |   position: relative;
4385 |   display: grid;
4386 |   min-width: min(330px, 56vw);
4387 |   height: 19px;
4388 |   overflow: hidden;
4389 |   color: #0f172a;
4390 |   font-size: 14px;
4391 |   font-weight: 950;
4392 | }
4393 |
4394 | .hero-animated-line i {
4395 |   grid-area: 1 / 1;

```

```

4396 | font-style: normal;
4397 | opacity: 0;
4398 | transform: translateY(18px);
4399 | animation: heroWordCycle 8s ease-in-out infinite;
4400 | }
4401 |
4402 | .hero-animated-line i:nth-child(2) {
4403 |   animation-delay: 2s;
4404 | }
4405 |
4406 | .hero-animated-line i:nth-child(3) {
4407 |   animation-delay: 4s;
4408 | }
4409 |
4410 | .hero-animated-line i:nth-child(4) {
4411 |   animation-delay: 6s;
4412 | }
4413 |
4414 | @keyframes heroWordCycle {
4415 |   0%,
4416 |   15% {
4417 |     opacity: 0;
4418 |     transform: translateY(18px);
4419 |   }
4420 |   22%,
4421 |   40% {
4422 |     opacity: 1;
4423 |     transform: translateY(0);
4424 |   }
4425 |   48%,
4426 |   100% {
4427 |     opacity: 0;
4428 |     transform: translateY(-18px);
4429 |   }
4430 | }
4431 |
4432 | .hero-logo-panel {
4433 |   position: relative;
4434 |   display: grid;
4435 |   place-items: center;
4436 |   align-content: center;
4437 |   gap: 10px;
4438 |   min-height: 270px;
4439 |   overflow: hidden;
4440 |   border-radius: 8px;
4441 |   padding: 24px;
4442 |   background:
4443 |     radial-gradient(circle at 50% 36%, rgba(37, 99, 235, 0.2), transparent 34%),
4444 |     radial-gradient(circle at 72% 72%, rgba(16, 185, 129, 0.18), transparent 30%),
4445 |     linear-gradient(135deg, rgba(255, 255, 255, 0.94), rgba(239, 246, 255, 0.88));
4446 |   box-shadow:
4447 |     inset 0 0 0 1px rgba(226, 232, 240, 0.86),
4448 |     0 24px 64px rgba(37, 99, 235, 0.14);
4449 | }
4450 |
4451 | .hero-logo-panel::before {
4452 |   content: "";
4453 |   position: absolute;
4454 |   inset: 28px;
4455 |   border: 1px solid rgba(148, 163, 184, 0.24);
4456 |   border-radius: 999px;
4457 |   transform: rotateX(66deg) rotateZ(-18deg);
4458 | }
4459 |
4460 | .hero-logo-orbit {
4461 |   position: relative;
4462 |   display: grid;
4463 |   place-items: center;
4464 |   width: 174px;
4465 |   height: 174px;
4466 |   border-radius: 36px;
4467 |   background: linear-gradient(135deg, #0f172a, #2563eb 58%, #14b8a6);
4468 |   box-shadow:
4469 |     0 28px 70px rgba(37, 99, 235, 0.32),
4470 |     inset 0 1px 0 rgba(255, 255, 255, 0.38);
4471 |   transform: perspective(700px) rotateX(10deg) rotateY(-16deg);
4472 | }
4473 |
4474 | .hero-logo-core {
4475 |   display: grid;
4476 |   place-items: center;
4477 |   width: 86px;
4478 |   height: 86px;
4479 |   border-radius: 24px;
4480 |   background: rgba(255, 255, 255, 0.94);
4481 |   color: #2563eb;
4482 |   box-shadow: inset 0 0 0 1px rgba(255, 255, 255, 0.8);
4483 | }
4484 |
4485 | .orbit-dot {
4486 |   position: absolute;
4487 |   display: block;
4488 |   width: 16px;
4489 |   height: 16px;
4490 |   border-radius: 999px;
4491 |   background: #ffffff;
4492 |   box-shadow: 0 10px 26px rgba(15, 23, 42, 0.18);
4493 | }
4494 |
4495 | .orbit-dot.one {
4496 |   top: 18px;
4497 |   right: 32px;
4498 | }
4499 |
4500 | .orbit-dot.two {
4501 |   bottom: 26px;
4502 |   left: 24px;
4503 |   background: #bbf7d0;
4504 | }
4505 |

```

```

4506 | .orbit-dot.three {
4507 |   right: 18px;
4508 |   bottom: 42px;
4509 |   background: #dbeafe;
4510 | }
4511 |
4512 | .hero-logo-panel strong {
4513 |   color: #0f172a;
4514 |   font-size: 28px;
4515 |   font-weight: 950;
4516 |   line-height: 1;
4517 | }
4518 |
4519 | .hero-logo-panel > span {
4520 |   color: #64748b;
4521 |   font-size: 12px;
4522 |   font-weight: 950;
4523 |   text-transform: uppercase;
4524 | }
4525 |
4526 | .premium-analytics-page .analysis-notice {
4527 |   border: 1px solid rgba(37, 99, 235, 0.18);
4528 |   border-radius: 8px;
4529 |   padding: 12px 14px;
4530 |   background: #eff6ff;
4531 |   color: #1e40af;
4532 |   font-weight: 750;
4533 | }
4534 |
4535 | .admin-analysis-loading-page {
4536 |   min-height: calc(100vh - 64px);
4537 |   align-content: center;
4538 |   gap: 24px;
4539 |   padding: 18px 0 36px;
4540 | }
4541 |
4542 | .analysis-loading-header {
4543 |   display: flex;
4544 |   align-items: center;
4545 |   justify-content: space-between;
4546 |   gap: 16px;
4547 |   color: #64748b;
4548 |   font-size: 13px;
4549 |   font-weight: 800;
4550 |   text-transform: uppercase;
4551 | }
4552 |
4553 | .analysis-loading-hero {
4554 |   max-width: 980px;
4555 | }
4556 |
4557 | .analysis-loading-hero h1 {
4558 |   margin: 10px 0 14px;
4559 |   color: #0f172a;
4560 |   font-size: clamp(42px, 6vw, 78px);
4561 |   line-height: 0.98;
4562 | }
4563 |
4564 | .analysis-loading-hero p {
4565 |   max-width: 820px;
4566 |   margin: 0;
4567 |   color: #475569;
4568 |   font-size: clamp(16px, 1.6vw, 21px);
4569 |   line-height: 1.65;
4570 | }
4571 |
4572 | .ai-progress-panel {
4573 |   display: grid;
4574 |   grid-template-columns: minmax(280px, 0.72fr) minmax(420px, 1.28fr);
4575 |   gap: 18px;
4576 |   border: 1px solid rgba(37, 99, 235, 0.16);
4577 |   border-radius: 8px;
4578 |   padding: 20px;
4579 |   background:
4580 |     linear-gradient(135deg, rgba(255, 255, 255, 0.96), rgba(239, 246, 255, 0.86)),
4581 |     #ffffff;
4582 |   box-shadow: 0 22px 60px rgba(15, 23, 42, 0.1);
4583 |   animation: ai-progress-enter 420ms ease both;
4584 | }
4585 |
4586 | .ai-progress-main {
4587 |   display: flex;
4588 |   gap: 16px;
4589 |   align-items: flex-start;
4590 |   border-radius: 8px;
4591 |   padding: 18px;
4592 |   background: linear-gradient(135deg, #0f172a, #1d4ed8);
4593 |   color: #ffffff;
4594 | }
4595 |
4596 | .ai-progress-main .hero-kicker {
4597 |   color: #bfdbfe;
4598 | }
4599 |
4600 | .ai-progress-main h2 {
4601 |   margin: 8px 0 8px;
4602 |   font-size: clamp(22px, 2.5vw, 34px);
4603 |   line-height: 1.05;
4604 | }
4605 |
4606 | .ai-progress-main p {
4607 |   margin: 0;
4608 |   color: rgba(255, 255, 255, 0.78);
4609 |   font-size: 14px;
4610 |   line-height: 1.6;
4611 | }
4612 |
4613 | .ai-progress-orb {
4614 |   display: grid;
4615 |   width: 48px;

```

```

4616 | height: 48px;
4617 | flex: 0 0 48px;
4618 | place-items: center;
4619 | border-radius: 50%;
4620 | background: rgba(255, 255, 255, 0.16);
4621 | color: #ffffff;
4622 | box-shadow: inset 0 0 0 1px rgba(255, 255, 255, 0.18);
4623 | animation: ai-orb-pulse 1.4s ease-in-out infinite;
4624 | }
4625 |
4626 | .ai-progress-meter {
4627 |   display: grid;
4628 |   gap: 14px;
4629 | }
4630 |
4631 | .ai-progress-meter-top {
4632 |   display: flex;
4633 |   align-items: center;
4634 |   justify-content: space-between;
4635 |   gap: 12px;
4636 |   color: #475569;
4637 |   font-size: 13px;
4638 |   font-weight: 800;
4639 |   text-transform: uppercase;
4640 | }
4641 |
4642 | .ai-progress-meter-top strong {
4643 |   color: #0f172a;
4644 |   font-size: 20px;
4645 | }
4646 |
4647 | .ai-progress-track {
4648 |   height: 10px;
4649 |   overflow: hidden;
4650 |   border-radius: 999px;
4651 |   background: #e2e8f0;
4652 | }
4653 |
4654 | .ai-progress-track span {
4655 |   display: block;
4656 |   height: 100%;
4657 |   border-radius: inherit;
4658 |   background: linear-gradient(90deg, #2563eb, #14b8a6, #22c55e);
4659 |   box-shadow: 0 0 24px rgba(37, 99, 235, 0.28);
4660 |   transition: width 520ms ease;
4661 | }
4662 |
4663 | .ai-progress-steps {
4664 |   display: grid;
4665 |   grid-template-columns: repeat(3, minmax(0, 1fr));
4666 |   gap: 10px;
4667 | }
4668 |
4669 | .ai-progress-step {
4670 |   display: flex;
4671 |   gap: 10px;
4672 |   min-height: 82px;
4673 |   border-radius: 8px;
4674 |   padding: 12px;
4675 |   background: #f8fafc;
4676 |   color: #64748b;
4677 |   transition:
4678 |     transform 180ms ease,
4679 |     background 180ms ease,
4680 |     box-shadow 180ms ease;
4681 | }
4682 |
4683 | .ai-progress-step > span {
4684 |   display: grid;
4685 |   width: 26px;
4686 |   height: 26px;
4687 |   flex: 0 0 26px;
4688 |   place-items: center;
4689 |   border-radius: 50%;
4690 |   background: #e2e8f0;
4691 |   color: #475569;
4692 |   font-size: 12px;
4693 |   font-weight: 900;
4694 | }
4695 |
4696 | .ai-progress-step strong {
4697 |   display: block;
4698 |   color: #0f172a;
4699 |   font-size: 13px;
4700 | }
4701 |
4702 | .ai-progress-step p {
4703 |   margin: 4px 0 0;
4704 |   color: #64748b;
4705 |   font-size: 12px;
4706 |   line-height: 1.35;
4707 | }
4708 |
4709 | .ai-progress-step.done > span {
4710 |   background: #dcfce7;
4711 |   color: #16a34a;
4712 | }
4713 |
4714 | .ai-progress-step.active {
4715 |   background: #eff6ff;
4716 |   box-shadow: 0 12px 34px rgba(37, 99, 235, 0.12);
4717 |   transform: translateY(-2px);
4718 | }
4719 |
4720 | .ai-progress-step.active > span {
4721 |   background: #2563eb;
4722 |   color: #ffffff;
4723 | }
4724 |
4725 | .premium-kpi-grid {

```

```

4726 | display: grid;
4727 | grid-template-columns: repeat(5, minmax(0, 1fr));
4728 | gap: 12px;
4729 | }
4730 |
4731 | .comparison-empty-note {
4732 |   display: flex;
4733 |   align-items: center;
4734 |   gap: 8px;
4735 |   width: fit-content;
4736 |   max-width: calc(100% - 10px);
4737 |   margin: -4px auto 0;
4738 |   border-radius: 999px;
4739 |   padding: 8px 12px;
4740 |   background: #f8fafc;
4741 |   color: #64748b;
4742 |   font-size: 13px;
4743 |   font-weight: 700;
4744 |   line-height: 1.35;
4745 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.9);
4746 | }
4747 |
4748 | .comparison-empty-note::before {
4749 |   content: "";
4750 |   flex: 0 0 auto;
4751 |   width: 8px;
4752 |   height: 8px;
4753 |   border-radius: 999px;
4754 |   background: #ef4444;
4755 |   box-shadow: 0 0 0 4px rgba(239, 68, 68, 0.12);
4756 | }
4757 |
4758 | .premium-kpi-card,
4759 | .premium-panel,
4760 | .ai-finding-card {
4761 |   border: 1px solid rgba(226, 232, 240, 0.86);
4762 |   border-radius: 8px;
4763 |   background: rgba(255, 255, 255, 0.9);
4764 |   box-shadow: 0 18px 54px rgba(15, 23, 42, 0.07);
4765 | }
4766 |
4767 | .premium-kpi-card {
4768 |   display: grid;
4769 |   gap: 8px;
4770 |   min-height: 134px;
4771 |   overflow: hidden;
4772 |   padding: 14px;
4773 |   transition:
4774 |     transform 180ms ease,
4775 |     box-shadow 180ms ease;
4776 | }
4777 |
4778 | .premium-kpi-card:hover,
4779 | .ai-finding-card:hover,
4780 | .recommendation-list article:hover {
4781 |   box-shadow: 0 24px 70px rgba(15, 23, 42, 0.12);
4782 | }
4783 |
4784 | .premium-kpi-card span {
4785 |   color: #64748b;
4786 |   font-size: 11px;
4787 |   font-weight: 900;
4788 |   text-transform: uppercase;
4789 | }
4790 |
4791 | .premium-kpi-card strong {
4792 |   display: block;
4793 |   margin-top: 5px;
4794 |   color: #0f172a;
4795 |   font-size: clamp(23px, 2vw, 32px);
4796 |   line-height: 1;
4797 | }
4798 |
4799 | .premium-kpi-card em,
4800 | .leaderboard-row em,
4801 | .modern-sentiment-legend em {
4802 |   font-size: 12px;
4803 |   font-style: normal;
4804 |   font-weight: 850;
4805 | }
4806 |
4807 | .premium-kpi-card.violet {
4808 |   background: linear-gradient(180deg, rgba(239, 246, 255, 0.96), rgba(255, 255, 255, 0.95));
4809 | }
4810 |
4811 | .premium-kpi-card.blue {
4812 |   background: linear-gradient(180deg, rgba(224, 242, 254, 0.92), rgba(255, 255, 255, 0.95));
4813 | }
4814 |
4815 | .premium-kpi-card.rose {
4816 |   background: linear-gradient(180deg, rgba(255, 241, 242, 0.94), rgba(255, 255, 255, 0.95));
4817 | }
4818 |
4819 | .premium-kpi-card.emerald {
4820 |   background: linear-gradient(180deg, rgba(236, 253, 245, 0.94), rgba(255, 255, 255, 0.95));
4821 | }
4822 |
4823 | .premium-kpi-card.amber {
4824 |   background: linear-gradient(180deg, rgba(255, 251, 235, 0.96), rgba(255, 255, 255, 0.95));
4825 | }
4826 |
4827 | .premium-kpi-card.slate {
4828 |   background: linear-gradient(180deg, rgba(248, 250, 252, 0.96), rgba(255, 255, 255, 0.95));
4829 | }
4830 |
4831 | .ai-quality-section {
4832 |   display: grid;
4833 |   gap: 16px;
4834 |   background:
4835 |     radial-gradient(circle at top left, rgba(99, 91, 255, 0.08), transparent 34%),

```

```

4836 |         linear-gradient(135deg, rgba(255, 255, 255, 0.96), rgba(248, 250, 252, 0.95));
4837 |     }
4838 |
4839 |     .ai-quality-header {
4840 |         display: flex;
4841 |         align-items: start;
4842 |         justify-content: space-between;
4843 |         gap: 20px;
4844 |     }
4845 |
4846 |     .ai-quality-header span,
4847 |     .quality-detail-group h3 {
4848 |         color: #635bff;
4849 |         font-size: 12px;
4850 |         font-weight: 950;
4851 |         letter-spacing: 0.02em;
4852 |         text-transform: uppercase;
4853 |     }
4854 |
4855 |     .ai-quality-header h2 {
4856 |         margin: 4px 0 7px;
4857 |         color: #0f172a;
4858 |         font-size: clamp(22px, 2vw, 30px);
4859 |         line-height: 1.08;
4860 |     }
4861 |
4862 |     .ai-quality-header p {
4863 |         max-width: 760px;
4864 |         margin: 0;
4865 |         color: #64748b;
4866 |         font-size: 14px;
4867 |         font-weight: 650;
4868 |         line-height: 1.55;
4869 |     }
4870 |
4871 |     .ai-quality-header button {
4872 |         flex: 0 0 auto;
4873 |         min-height: 40px;
4874 |         border: 0;
4875 |         border-radius: 8px;
4876 |         padding: 0 14px;
4877 |         background: #0f172a;
4878 |         color: #ffffff;
4879 |         font-size: 13px;
4880 |         font-weight: 900;
4881 |         box-shadow: 0 16px 34px rgba(15, 23, 42, 0.18);
4882 |         transition:
4883 |             transform 160ms ease,
4884 |             box-shadow 160ms ease;
4885 |     }
4886 |
4887 |     .ai-quality-header button:hover {
4888 |         transform: translateY(-1px);
4889 |         box-shadow: 0 22px 44px rgba(15, 23, 42, 0.22);
4890 |     }
4891 |
4892 |     .ai-quality-stats {
4893 |         display: grid;
4894 |         grid-template-columns: repeat(6, minmax(0, 1fr));
4895 |         gap: 10px;
4896 |     }
4897 |
4898 |     .quality-stat {
4899 |         min-height: 82px;
4900 |         border-radius: 8px;
4901 |         padding: 12px;
4902 |         background: #ffffff;
4903 |         box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.78);
4904 |     }
4905 |
4906 |     .quality-stat span {
4907 |         display: block;
4908 |         color: #64748b;
4909 |         font-size: 11px;
4910 |         font-weight: 950;
4911 |         text-transform: uppercase;
4912 |     }
4913 |
4914 |     .quality-stat strong {
4915 |         display: block;
4916 |         margin-top: 8px;
4917 |         color: #0f172a;
4918 |         font-size: clamp(22px, 2vw, 32px);
4919 |         line-height: 1;
4920 |     }
4921 |
4922 |     .quality-stat.green {
4923 |         background: linear-gradient(180deg, #ecfdf5, #ffffff);
4924 |     }
4925 |
4926 |     .quality-stat.slate {
4927 |         background: linear-gradient(180deg, #f8f9fc, #ffffff);
4928 |     }
4929 |
4930 |     .quality-stat.orange {
4931 |         background: linear-gradient(180deg, #fff7ed, #ffffff);
4932 |     }
4933 |
4934 |     .quality-stat.yellow {
4935 |         background: linear-gradient(180deg, #fefce8, #ffffff);
4936 |     }
4937 |
4938 |     .quality-stat.red {
4939 |         background: linear-gradient(180deg, #fef2f2, #ffffff);
4940 |     }
4941 |
4942 |     .quality-stat.blue {
4943 |         background: linear-gradient(180deg, #eff6ff, #ffffff);
4944 |     }
4945 |

```

```

4946 | .ai-quality-details {
4947 |   display: grid;
4948 |   grid-template-columns: repeat(4, minmax(0, 1fr));
4949 |   gap: 12px;
4950 | }
4951 |
4952 | .ai-quality-drilldown {
4953 |   display: grid;
4954 |   grid-column: 1 / -1;
4955 |   gap: 12px;
4956 | }
4957 |
4958 | .quality-evidence-toolbar {
4959 |   display: flex;
4960 |   align-items: center;
4961 |   justify-content: space-between;
4962 |   gap: 16px;
4963 |   border-radius: 8px;
4964 |   padding: 14px;
4965 |   background: #0f172a;
4966 |   color: #ffffff;
4967 | }
4968 |
4969 | .quality-evidence-toolbar strong {
4970 |   display: block;
4971 |   font-size: 15px;
4972 | }
4973 |
4974 | .quality-evidence-toolbar p {
4975 |   margin: 5px 0 0;
4976 |   color: #cbd5e1;
4977 |   font-size: 12px;
4978 |   font-weight: 650;
4979 | }
4980 |
4981 | .quality-evidence-toolbar button {
4982 |   min-height: 36px;
4983 |   flex: 0 0 auto;
4984 |   border: 0;
4985 |   border-radius: 8px;
4986 |   padding: 0 12px;
4987 |   background: #ffffff;
4988 |   color: #0f172a;
4989 |   font-size: 12px;
4990 |   font-weight: 950;
4991 |   transition:
4992 |     transform 160ms ease,
4993 |     box-shadow 160ms ease;
4994 | }
4995 |
4996 | .quality-evidence-toolbar button:hover {
4997 |   transform: translateY(-1px);
4998 |   box-shadow: 0 14px 28px rgba(15, 23, 42, 0.24);
4999 | }
5000 |
5001 | .quality-breakdown-grid {
5002 |   display: grid;
5003 |   grid-template-columns: repeat(4, minmax(0, 1fr));
5004 |   gap: 12px;
5005 | }
5006 |
5007 | .quality-breakdown-card {
5008 |   display: grid;
5009 |   gap: 12px;
5010 |   min-width: 0;
5011 |   border-radius: 8px;
5012 |   padding: 14px;
5013 |   background: #ffffff;
5014 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.82);
5015 | }
5016 |
5017 | .quality-breakdown-card > div:first-child {
5018 |   display: flex;
5019 |   align-items: start;
5020 |   justify-content: space-between;
5021 |   gap: 10px;
5022 | }
5023 |
5024 | .quality-breakdown-card h3 {
5025 |   margin: 0;
5026 |   color: #0f172a;
5027 |   font-size: 12px;
5028 |   font-weight: 950;
5029 |   letter-spacing: 0.02em;
5030 |   line-height: 1.25;
5031 |   text-transform: uppercase;
5032 | }
5033 |
5034 | .quality-breakdown-card > div:first-child strong {
5035 |   color: #0f172a;
5036 |   font-size: 22px;
5037 |   line-height: 1;
5038 | }
5039 |
5040 | .quality-breakdown-card > div:last-child {
5041 |   display: grid;
5042 |   gap: 8px;
5043 | }
5044 |
5045 | .quality-breakdown-card section {
5046 |   display: grid;
5047 |   grid-template-columns: minmax(0, 1fr) auto;
5048 |   gap: 5px 8px;
5049 |   border-radius: 8px;
5050 |   padding: 9px;
5051 |   background: #f8fafc;
5052 | }
5053 |
5054 | .quality-breakdown-card section span {
5055 |   color: #334155;

```

```

5056 | font-size: 12px;
5057 | font-weight: 800;
5058 | line-height: 1.3;
5059 | }
5060 |
5061 | .quality-breakdown-card section b {
5062 |   color: #0f172a;
5063 |   font-size: 13px;
5064 | }
5065 |
5066 | .quality-breakdown-card section em {
5067 |   grid-column: 1 / -1;
5068 |   color: #64748b;
5069 |   font-size: 11px;
5070 |   font-style: normal;
5071 |   font-weight: 800;
5072 | }
5073 |
5074 | .quality-breakdown-card.red h3,
5075 | .quality-breakdown-card.red > div:first-child strong {
5076 |   color: #dc2626;
5077 | }
5078 |
5079 | .quality-breakdown-card.orange h3,
5080 | .quality-breakdown-card.orange > div:first-child strong {
5081 |   color: #ea580c;
5082 | }
5083 |
5084 | .quality-breakdown-card.yellow h3,
5085 | .quality-breakdown-card.yellow > div:first-child strong {
5086 |   color: #a16207;
5087 | }
5088 |
5089 | .quality-breakdown-card.green h3,
5090 | .quality-breakdown-card.green > div:first-child strong {
5091 |   color: #059669;
5092 | }
5093 |
5094 | .quality-detail-group {
5095 |   display: grid;
5096 |   gap: 10px;
5097 |   min-width: 0;
5098 |   border-radius: 8px;
5099 |   padding: 14px;
5100 |   background: rgba(255, 255, 255, 0.78);
5101 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.86);
5102 | }
5103 |
5104 | .quality-detail-group h3 {
5105 |   margin: 0;
5106 |   line-height: 1.25;
5107 | }
5108 |
5109 | .quality-detail-group > div {
5110 |   display: grid;
5111 |   gap: 9px;
5112 | }
5113 |
5114 | .quality-detail-item {
5115 |   display: grid;
5116 |   gap: 5px;
5117 |   border-radius: 8px;
5118 |   padding: 10px;
5119 |   background: #f8fafc;
5120 | }
5121 |
5122 | .quality-detail-item span {
5123 |   color: #64748b;
5124 |   font-size: 11px;
5125 |   font-weight: 900;
5126 | }
5127 |
5128 | .quality-detail-item strong {
5129 |   color: #0f172a;
5130 |   font-size: 13px;
5131 |   line-height: 1.25;
5132 | }
5133 |
5134 | .quality-detail-item p {
5135 |   margin: 0;
5136 |   color: #475569;
5137 |   font-size: 12px;
5138 |   font-weight: 650;
5139 |   line-height: 1.4;
5140 | }
5141 |
5142 | .quality-detail-item em {
5143 |   justify-self: start;
5144 |   border-radius: 999px;
5145 |   padding: 5px 8px;
5146 |   background: #ffffff;
5147 |   color: #334155;
5148 |   font-size: 10px;
5149 |   font-style: normal;
5150 |   font-weight: 950;
5151 |   text-transform: uppercase;
5152 | }
5153 |
5154 | .quality-detail-group.red h3 {
5155 |   color: #dc2626;
5156 | }
5157 |
5158 | .quality-detail-group.orange h3 {
5159 |   color: #ea580c;
5160 | }
5161 |
5162 | .quality-detail-group.yellow h3 {
5163 |   color: #a16207;
5164 | }
5165 |

```



```

5166 | .quality-detail-group.green h3 {
5167 |   color: #059669;
5168 | }
5169 |
5170 | .positive {
5171 |   color: #059669;
5172 | }
5173 |
5174 | .negative {
5175 |   color: #dc2626;
5176 | }
5177 |
5178 | .analytics-chart-row {
5179 |   display: grid;
5180 |   grid-template-columns: minmax(0, 1.25fr) minmax(360px, 0.78fr);
5181 |   gap: 18px;
5182 | }
5183 |
5184 | .heatmap-radar-row {
5185 |   display: grid;
5186 |   grid-template-columns: minmax(0, 1.18fr) minmax(360px, 0.82fr);
5187 |   gap: 18px;
5188 |   align-items: stretch;
5189 | }
5190 |
5191 | .premium-panel {
5192 |   padding: 20px;
5193 | }
5194 |
5195 | .premium-panel-header {
5196 |   display: flex;
5197 |   align-items: start;
5198 |   justify-content: space-between;
5199 |   gap: 14px;
5200 |   margin-bottom: 16px;
5201 | }
5202 |
5203 | .premium-panel-header h2 {
5204 |   margin: 4px 0 0;
5205 |   color: #0f172a;
5206 |   font-size: clamp(20px, 2vw, 28px);
5207 |   line-height: 1.12;
5208 | }
5209 |
5210 | .premium-panel-header em {
5211 |   flex: 0 0 auto;
5212 |   border-radius: 999px;
5213 |   padding: 7px 10px;
5214 |   background: #f1f5f9;
5215 |   color: #475569;
5216 |   font-size: 12px;
5217 |   font-style: normal;
5218 |   font-weight: 850;
5219 | }
5220 |
5221 | .premium-analytics-page .premium-tabs {
5222 |   display: flex;
5223 |   gap: 7px;
5224 |   margin-bottom: 12px;
5225 |   overflow-x: auto;
5226 |   padding-bottom: 2px;
5227 | }
5228 |
5229 | .premium-analytics-page .premium-tabs button {
5230 |   flex: 0 0 auto;
5231 |   min-height: 34px;
5232 |   border: 0;
5233 |   border-radius: 999px;
5234 |   padding: 0 12px;
5235 |   background: #f1f5f9;
5236 |   color: #475569;
5237 |   font-size: 12px;
5238 |   font-weight: 850;
5239 |   transition:
5240 |     transform 160ms ease,
5241 |     background 160ms ease,
5242 |     color 160ms ease;
5243 | }
5244 |
5245 | .premium-analytics-page .premium-tabs button:hover,
5246 | .premium-analytics-page .premium-tabs .selected {
5247 |   background: #0f172a;
5248 |   color: #ffffff;
5249 |   transform: translateY(-1px);
5250 | }
5251 |
5252 | .modern-sentiment-legend {
5253 |   display: grid;
5254 |   grid-template-columns: repeat(2, minmax(0, 1fr));
5255 |   gap: 8px;
5256 | }
5257 |
5258 | .modern-sentiment-legend span {
5259 |   display: grid;
5260 |   grid-template-columns: 10px 1fr auto;
5261 |   gap: 8px;
5262 |   align-items: center;
5263 |   border-radius: 8px;
5264 |   padding: 9px 10px;
5265 |   background: #f8fafc;
5266 |   color: #334155;
5267 |   font-size: 13px;
5268 |   font-weight: 800;
5269 | }
5270 |
5271 | .modern-sentiment-legend i {
5272 |   width: 10px;
5273 |   height: 10px;
5274 |   border-radius: 999px;
5275 | }

```

```

5276 |
5277 | .modern-sentiment-legend b {
5278 |   color: #0f172a;
5279 | }
5280 |
5281 | .sentiment-empty-note {
5282 |   margin: -4px 0 10px;
5283 |   color: #64748b;
5284 |   font-size: 12px;
5285 |   font-weight: 700;
5286 |   line-height: 1.45;
5287 |   text-align: center;
5288 | }
5289 |
5290 | .sentiment-placeholder-donut {
5291 |   display: grid;
5292 |   min-height: 238px;
5293 |   place-items: center;
5294 |   align-content: center;
5295 |   gap: 8px;
5296 |   color: #64748b;
5297 |   text-align: center;
5298 | }
5299 |
5300 | .sentiment-placeholder-donut span {
5301 |   display: block;
5302 |   width: 132px;
5303 |   height: 132px;
5304 |   border: 18px solid #e2e8f0;
5305 |   border-radius: 50%;
5306 |   box-shadow: inset 0 0 0 1px #f8fafc;
5307 | }
5308 |
5309 | .sentiment-placeholder-donut strong {
5310 |   color: #0f172a;
5311 |   font-size: 15px;
5312 | }
5313 |
5314 | .sentiment-placeholder-donut p {
5315 |   max-width: 260px;
5316 |   margin: 0;
5317 |   color: #64748b;
5318 |   font-size: 12px;
5319 |   font-weight: 700;
5320 |   line-height: 1.45;
5321 | }
5322 |
5323 | .modern-sentiment-legend em {
5324 |   grid-column: 2 / 4;
5325 |   color: #94a3b8;
5326 | }
5327 |
5328 | .comparison-panel {
5329 |   min-height: 410px;
5330 | }
5331 |
5332 | .inline-comparison-panel {
5333 |   min-height: auto;
5334 | }
5335 |
5336 | .bubble-heatmap-panel {
5337 |   position: relative;
5338 |   overflow: hidden;
5339 |   background:
5340 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.94)),
5341 |     #ffffff;
5342 | }
5343 |
5344 | .bubble-matrix {
5345 |   display: grid;
5346 |   gap: 10px;
5347 |   overflow-x: auto;
5348 |   padding: 10px 4px 4px;
5349 | }
5350 |
5351 | .bubble-matrix-head,
5352 | .bubble-matrix-row {
5353 |   display: grid;
5354 |   grid-template-columns: 150px repeat(5, minmax(118px, 1fr));
5355 |   gap: 10px;
5356 |   min-width: 790px;
5357 |   align-items: center;
5358 |   justify-items: center;
5359 | }
5360 |
5361 | .bubble-matrix-head b,
5362 | .bubble-matrix-row strong {
5363 |   color: #475569;
5364 |   font-size: 13px;
5365 |   font-weight: 950;
5366 |   text-align: center;
5367 |   text-transform: uppercase;
5368 | }
5369 |
5370 | .bubble-matrix-row {
5371 |   min-height: 92px;
5372 | }
5373 |
5374 | .bubble-matrix-row strong {
5375 |   justify-self: start;
5376 |   text-align: left;
5377 | }
5378 |
5379 | .bubble-cell {
5380 |   position: relative;
5381 |   display: grid;
5382 |   place-items: center;
5383 |   align-content: center;
5384 |   justify-self: center;
5385 |   width: var(--bubble-size);

```

```

5386 | height: var(--bubble-size);
5387 | min-width: 46px;
5388 | min-height: 46px;
5389 | border: 0;
5390 | border-radius: 999px;
5391 | color: #0f172a;
5392 | box-shadow:
5393 |   inset 0 1px 0 rgba(255, 255, 255, 0.72),
5394 |   0 14px 34px rgba(15, 23, 42, 0.14);
5395 | transition:
5396 |   transform 180ms ease,
5397 |   box-shadow 180ms ease;
5398 | }
5399 |
5400 | .bubble-cell:hover {
5401 |   transform: scale(1.04);
5402 |   box-shadow:
5403 |     inset 0 1px 0 rgba(255, 255, 255, 0.82),
5404 |     0 20px 44px rgba(15, 23, 42, 0.18);
5405 | }
5406 |
5407 | .bubble-cell span {
5408 |   color: inherit;
5409 |   font-size: 18px;
5410 |   font-weight: 950;
5411 |   line-height: 1;
5412 | }
5413 |
5414 | .bubble-cell em {
5415 |   margin-top: 3px;
5416 |   color: inherit;
5417 |   font-size: 10px;
5418 |   font-style: normal;
5419 |   font-weight: 900;
5420 |   opacity: 0.76;
5421 | }
5422 |
5423 | .bubble-cell.excellent {
5424 |   background: linear-gradient(135deg, #bbf7d0, #22c55e);
5425 | }
5426 |
5427 | .bubble-cell.good {
5428 |   background: linear-gradient(135deg, #fef3c7, #facc15);
5429 | }
5430 |
5431 | .bubble-cell.warning {
5432 |   background: linear-gradient(135deg, #fed7aa, #fb923c);
5433 | }
5434 |
5435 | .bubble-cell.critical {
5436 |   background: linear-gradient(135deg, #fecdd3, #fb7185);
5437 | }
5438 |
5439 | .heatmap-radar-row .bubble-matrix {
5440 |   padding-top: 4px;
5441 | }
5442 |
5443 | .heatmap-radar-row .bubble-matrix-head,
5444 | .heatmap-radar-row .bubble-matrix-row {
5445 |   grid-template-columns: 118px repeat(5, minmax(72px, 1fr));
5446 |   gap: 8px;
5447 |   min-width: 560px;
5448 | }
5449 |
5450 | .heatmap-radar-row .bubble-matrix-row {
5451 |   min-height: 78px;
5452 | }
5453 |
5454 | .heatmap-radar-row .bubble-matrix-head b,
5455 | .heatmap-radar-row .bubble-matrix-row strong {
5456 |   font-size: 11px;
5457 | }
5458 |
5459 | .heatmap-radar-row .bubble-cell {
5460 |   transform: scale(0.9);
5461 | }
5462 |
5463 | .heatmap-radar-row .bubble-cell:hover {
5464 |   transform: scale(0.96);
5465 | }
5466 |
5467 | .heatmap-show-more {
5468 |   justify-self: center;
5469 |   min-height: 36px;
5470 |   border: 0;
5471 |   border-radius: 999px;
5472 |   margin: 14px auto 0;
5473 |   padding: 0 16px;
5474 |   background: #0f172a;
5475 |   color: #ffffff;
5476 |   cursor: pointer;
5477 |   font-size: 12px;
5478 |   font-weight: 950;
5479 |   box-shadow: 0 14px 34px rgba(15, 23, 42, 0.16);
5480 |   transition:
5481 |     transform 180ms ease,
5482 |     box-shadow 180ms ease,
5483 |     background 180ms ease;
5484 | }
5485 |
5486 | .heatmap-show-more:hover {
5487 |   background: #2563eb;
5488 |   box-shadow: 0 18px 42px rgba(37, 99, 235, 0.22);
5489 |   transform: translateY(-2px);
5490 | }
5491 |
5492 | .ai-findings-section {
5493 |   display: grid;
5494 |   gap: 16px;
5495 | }

```

```

5496 |
5497 | .issues-found-header {
5498 |   display: grid;
5499 |   gap: 6px;
5500 |   justify-items: start;
5501 | }
5502 |
5503 | .issues-found-header h2 {
5504 |   display: inline-flex;
5505 |   align-items: center;
5506 |   gap: 10px;
5507 |   margin: 0;
5508 |   color: #0f172a;
5509 |   font-size: clamp(24px, 2.5vw, 36px);
5510 |   line-height: 1.08;
5511 | }
5512 |
5513 | .issues-found-header h2 span {
5514 |   position: relative;
5515 |   display: inline-grid;
5516 |   place-items: center;
5517 |   width: 28px;
5518 |   height: 28px;
5519 |   border-radius: 8px;
5520 |   background: linear-gradient(135deg, #2563eb, #14b8a6);
5521 |   box-shadow: 0 12px 26px rgba(37, 99, 235, 0.2);
5522 | }
5523 |
5524 | .issues-found-header h2 span::before,
5525 | .issues-found-header h2 span::after {
5526 |   content: "";
5527 |   position: absolute;
5528 |   border-radius: 3px;
5529 |   background: #ffffff;
5530 | }
5531 |
5532 | .issues-found-header h2 span::before {
5533 |   width: 13px;
5534 |   height: 2px;
5535 | }
5536 |
5537 | .issues-found-header h2 span::after {
5538 |   width: 2px;
5539 |   height: 13px;
5540 | }
5541 |
5542 | .issues-found-header p {
5543 |   margin: 0;
5544 |   color: #64748b;
5545 |   font-size: 14px;
5546 |   font-weight: 750;
5547 |   line-height: 1.45;
5548 | }
5549 |
5550 | .issues-found-header p strong {
5551 |   color: #0f172a;
5552 |   font-weight: 950;
5553 | }
5554 |
5555 | .ai-findings-layout {
5556 |   display: grid;
5557 |   grid-template-columns: 1fr;
5558 |   gap: 16px;
5559 |   align-items: start;
5560 | }
5561 |
5562 | .ai-findings-section.has-detail .ai-findings-layout {
5563 |   grid-template-columns: minmax(0, 0.9fr) minmax(390px, 0.7fr);
5564 | }
5565 |
5566 | .ai-finding-grid {
5567 |   display: grid;
5568 |   grid-template-columns: repeat(3, minmax(0, 1fr));
5569 |   gap: 12px;
5570 | }
5571 |
5572 | .ai-findings-section.has-detail .ai-finding-grid {
5573 |   grid-template-columns: repeat(2, minmax(0, 1fr));
5574 | }
5575 |
5576 | .ai-finding-card {
5577 |   display: grid;
5578 |   gap: 10px;
5579 |   padding: 14px;
5580 |   transition:
5581 |     transform 180ms ease,
5582 |     box-shadow 180ms ease;
5583 | }
5584 |
5585 | .ai-finding-card.selected {
5586 |   box-shadow:
5587 |     inset 0 0 0 1.5px rgba(37, 99, 235, 0.5),
5588 |     0 18px 46px rgba(37, 99, 235, 0.16);
5589 | }
5590 |
5591 | .finding-card-top {
5592 |   display: flex;
5593 |   justify-content: flex-end;
5594 |   gap: 10px;
5595 | }
5596 |
5597 | .finding-card-top em {
5598 |   border-radius: 999px;
5599 |   padding: 6px 9px;
5600 |   font-size: 10px;
5601 |   font-style: normal;
5602 |   font-weight: 950;
5603 |   text-transform: uppercase;
5604 | }
5605 |

```

```

5606 | .ai-finding-card.critical .finding-card-top em {
5607 |   background: #fee2e2;
5608 |   color: #991b1b;
5609 | }
5610 |
5611 | .ai-finding-card.warning .finding-card-top em {
5612 |   background: #ffedd5;
5613 |   color: #9a3412;
5614 | }
5615 |
5616 | .ai-finding-card.low .finding-card-top em {
5617 |   background: #dcfce7;
5618 |   color: #166534;
5619 | }
5620 |
5621 | .ai-finding-card.resolved .finding-card-top em {
5622 |   background: #dcfce7;
5623 |   color: #166534;
5624 | }
5625 |
5626 | .ai-finding-card h3 {
5627 |   margin: 0;
5628 |   color: #0f172a;
5629 |   font-size: 20px;
5630 |   line-height: 1.18;
5631 | }
5632 |
5633 | .ai-finding-card p {
5634 |   margin: 0;
5635 |   color: #64748b;
5636 |   font-size: 13px;
5637 |   line-height: 1.45;
5638 | }
5639 |
5640 | .finding-metrics {
5641 |   gap: 8px;
5642 | }
5643 |
5644 | .finding-metrics span {
5645 |   font-size: 14px;
5646 |   font-weight: 900;
5647 | }
5648 |
5649 | .concerning-issues-section {
5650 |   display: grid;
5651 |   gap: 16px;
5652 |   background:
5653 |     linear-gradient(135deg, rgba(255, 247, 237, 0.86), rgba(255, 255, 255, 0.98)),
5654 |     #ffffff;
5655 | }
5656 |
5657 | .danger-alert-heading {
5658 |   display: flex;
5659 |   align-items: center;
5660 |   justify-content: center;
5661 |   gap: 11px;
5662 |   margin-bottom: 10px;
5663 |   padding-bottom: 8px;
5664 |   text-align: center;
5665 |   transform: translateX(-16px);
5666 | }
5667 |
5668 | .danger-alert-heading span {
5669 |   position: relative;
5670 |   width: 22px;
5671 |   height: 22px;
5672 |   border-radius: 999px;
5673 |   background: #dc2626;
5674 |   box-shadow:
5675 |     0 0 0 0 rgba(220, 38, 38, 0.72),
5676 |     0 0 18px 0 rgba(220, 38, 38, 0.95),
5677 |     0 0 36px 0 rgba(220, 38, 38, 0.62);
5678 |   animation: dangerPulse 0.72s ease-in-out infinite;
5679 | }
5680 |
5681 | .danger-alert-heading span::before,
5682 | .danger-alert-heading span::after {
5683 |   content: "";
5684 |   position: absolute;
5685 |   top: 50%;
5686 |   width: 30px;
5687 |   height: 7px;
5688 |   border-radius: 999px;
5689 |   background: rgba(220, 38, 38, 0.26);
5690 |   transform: translateY(-50%);
5691 |   animation: sirenSweep 0.72s ease-in-out infinite;
5692 | }
5693 |
5694 | .danger-alert-heading span::before {
5695 |   right: 28px;
5696 | }
5697 |
5698 | .danger-alert-heading span::after {
5699 |   left: 28px;
5700 | }
5701 |
5702 | .danger-alert-heading h2 {
5703 |   margin: 0;
5704 |   color: #0f172a;
5705 |   font-size: clamp(24px, 2.4vw, 36px);
5706 |   line-height: 1.08;
5707 | }
5708 |
5709 | @keyframes dangerPulse {
5710 |   0% {
5711 |     box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.46);
5712 |     opacity: 1;
5713 |   }
5714 |   70% {
5715 |     box-shadow: 0 0 0 10px rgba(239, 68, 68, 0);

```

```

5716 |     opacity: 0.72;
5717 | }
5718 | 100% {
5719 |     box-shadow: 0 0 0 rgba(239, 68, 68, 0);
5720 |     opacity: 1;
5721 | }
5722 | }
5723 |
5724 | @keyframes sirenSweep {
5725 |     0%,
5726 |     100% {
5727 |         opacity: 0.15;
5728 |         transform: translateY(-50%) scaleX(0.65);
5729 |     }
5730 |     50% {
5731 |         opacity: 0.85;
5732 |         transform: translateY(-50%) scaleX(1.15);
5733 |     }
5734 | }
5735 |
5736 | .concerning-issues-layout {
5737 |     display: grid;
5738 |     gap: 14px;
5739 |     align-items: start;
5740 | }
5741 |
5742 | .concerning-issues-layout.has-detail {
5743 |     grid-template-columns: minmax(0, 1fr) minmax(360px, 0.68fr);
5744 | }
5745 |
5746 | .concerning-issue-grid {
5747 |     display: grid;
5748 |     grid-template-columns: repeat(4, minmax(0, 1fr));
5749 |     gap: 12px;
5750 | }
5751 |
5752 | .concerning-issues-layout.has-detail .concerning-issue-grid {
5753 |     grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
5754 |     align-content: start;
5755 | }
5756 |
5757 | .concerning-issue-card {
5758 |     display: grid;
5759 |     align-content: start;
5760 |     gap: 10px;
5761 |     border: 1px solid rgba(226, 232, 240, 0.9);
5762 |     border-radius: 8px;
5763 |     padding: 14px;
5764 |     background: #ffffff;
5765 |     box-shadow: 0 18px 46px rgba(15, 23, 42, 0.07);
5766 |     transition:
5767 |         transform 180ms ease,
5768 |         border-color 180ms ease,
5769 |         box-shadow 180ms ease,
5770 |         background 180ms ease;
5771 | }
5772 |
5773 | .concerning-issue-card:hover,
5774 | .concerning-issue-card.selected {
5775 |     border-color: rgba(220, 38, 38, 0.36);
5776 |     background:
5777 |         linear-gradient(180deg, rgba(254, 242, 242, 0.9), rgba(255, 255, 255, 0.98)),
5778 |         #ffffff;
5779 |     box-shadow: 0 24px 66px rgba(220, 38, 38, 0.14);
5780 |     transform: translateY(-3px);
5781 | }
5782 |
5783 | .concerning-issue-card > div:first-child {
5784 |     display: flex;
5785 |     align-items: center;
5786 |     justify-content: space-between;
5787 |     gap: 10px;
5788 | }
5789 |
5790 | .concerning-issue-card span {
5791 |     display: inline-flex;
5792 |     align-items: center;
5793 |     gap: 6px;
5794 |     color: #991b1b;
5795 |     font-size: 11px;
5796 |     font-weight: 950;
5797 |     text-transform: uppercase;
5798 | }
5799 |
5800 | .concerning-issue-card em {
5801 |     border-radius: 999px;
5802 |     padding: 5px 8px;
5803 |     background: #fee2e2;
5804 |     color: #991b1b;
5805 |     font-size: 11px;
5806 |     font-style: normal;
5807 |     font-weight: 950;
5808 | }
5809 |
5810 | .concerning-issue-card h3 {
5811 |     margin: 0;
5812 |     color: #0f172a;
5813 |     font-size: 18px;
5814 |     line-height: 1.18;
5815 | }
5816 |
5817 | .concerning-issue-card p {
5818 |     margin: 0;
5819 |     color: #475569;
5820 |     font-size: 13px;
5821 |     line-height: 1.45;
5822 | }
5823 |
5824 | .concerning-issue-card dl {
5825 |     display: grid;

```

```

5826 | gap: 7px;
5827 | margin: 0;
5828 | }
5829 |
5830 | .concerning-issue-card dl div {
5831 |   display: grid;
5832 |   gap: 3px;
5833 |   border-radius: 8px;
5834 |   padding: 8px;
5835 |   background: #f8fafc;
5836 | }
5837 |
5838 | .concerning-issue-card dt {
5839 |   color: #64748b;
5840 |   font-size: 10px;
5841 |   font-weight: 950;
5842 |   text-transform: uppercase;
5843 | }
5844 |
5845 | .concerning-issue-card dd {
5846 |   margin: 0;
5847 |   color: #0f172a;
5848 |   font-size: 12px;
5849 |   font-weight: 800;
5850 | }
5851 |
5852 | .concerning-issue-card > strong {
5853 |   border-radius: 8px;
5854 |   padding: 9px;
5855 |   background: #0f172a;
5856 |   color: #ffffff;
5857 |   font-size: 12px;
5858 |   line-height: 1.35;
5859 | }
5860 |
5861 | .concerning-issue-card.high em {
5862 |   background: #ffedd5;
5863 |   color: #9a3412;
5864 | }
5865 |
5866 | .concerning-issue-card.low span {
5867 |   color: #166534;
5868 | }
5869 |
5870 | .concerning-issue-card.low em {
5871 |   background: #dcfce7;
5872 |   color: #166534;
5873 | }
5874 |
5875 | .concerning-issue-card button {
5876 |   width: 100%;
5877 |   border: 0;
5878 |   border-radius: 8px;
5879 |   padding: 10px 12px;
5880 |   background: #991b1b;
5881 |   color: #ffffff;
5882 |   cursor: pointer;
5883 |   font-size: 12px;
5884 |   font-weight: 950;
5885 |   letter-spacing: 0.01em;
5886 |   transition:
5887 |     transform 180ms ease,
5888 |     box-shadow 180ms ease,
5889 |     background 180ms ease;
5890 | }
5891 |
5892 | .concerning-issue-card button:hover {
5893 |   background: #7f1d1d;
5894 |   box-shadow: 0 16px 32px rgba(153, 27, 27, 0.22);
5895 |   transform: translateY(-2px);
5896 | }
5897 |
5898 | .danger-description-panel {
5899 |   position: sticky;
5900 |   top: 18px;
5901 |   display: grid;
5902 |   align-self: start;
5903 |   gap: 14px;
5904 |   overflow: hidden;
5905 |   border: 1px solid rgba(127, 29, 29, 0.32);
5906 |   border-radius: 8px;
5907 |   padding: 18px;
5908 |   background:
5909 |     radial-gradient(circle at 8% 0%, rgba(239, 68, 68, 0.24), transparent 34%),
5910 |     linear-gradient(145deg, #450a0a, #111827 62%, #020617);
5911 |   color: #fee2e2;
5912 |   box-shadow:
5913 |     0 28px 90px rgba(127, 29, 29, 0.3),
5914 |     inset 0 0 0 1px rgba(255, 255, 255, 0.04);
5915 | }
5916 |
5917 | .danger-description-panel::before {
5918 |   content: "";
5919 |   position: absolute;
5920 |   inset: 0;
5921 |   pointer-events: none;
5922 |   background: linear-gradient(90deg, transparent, rgba(248, 113, 113, 0.16), transparent);
5923 |   transform: translateX(-100%);
5924 |   animation: dangerScan 2.2s ease-in-out infinite;
5925 | }
5926 |
5927 | .danger-description-panel > div {
5928 |   position: relative;
5929 |   z-index: 1;
5930 |   display: flex;
5931 |   align-items: center;
5932 |   gap: 10px;
5933 | }
5934 |
5935 | .danger-description-panel strong {

```

```

5936 | color: #fecaca;
5937 | font-size: 12px;
5938 | font-weight: 950;
5939 | text-transform: uppercase;
5940 | }
5941 |
5942 | .danger-description-panel button {
5943 |   margin-left: auto;
5944 |   border: 1px solid rgba(254, 202, 202, 0.22);
5945 |   border-radius: 8px;
5946 |   padding: 8px 10px;
5947 |   background: rgba(127, 29, 29, 0.28);
5948 |   color: #fee2e2;
5949 |   cursor: pointer;
5950 |   font-size: 12px;
5951 |   font-weight: 900;
5952 | }
5953 |
5954 | .danger-description-panel h3 {
5955 |   position: relative;
5956 |   z-index: 1;
5957 |   margin: 0;
5958 |   color: #ffffff;
5959 |   font-size: 24px;
5960 |   line-height: 1.12;
5961 | }
5962 |
5963 | .danger-description-panel p {
5964 |   position: relative;
5965 |   z-index: 1;
5966 |   margin: 0;
5967 |   color: #fecaca;
5968 |   font-size: 15px;
5969 |   font-weight: 650;
5970 |   line-height: 1.65;
5971 | }
5972 |
5973 | .danger-description-stats {
5974 |   position: relative;
5975 |   z-index: 1;
5976 |   display: grid;
5977 |   grid-template-columns: repeat(2, minmax(0, 1fr));
5978 |   gap: 10px;
5979 | }
5980 |
5981 | .danger-description-stats span {
5982 |   display: grid;
5983 |   gap: 3px;
5984 |   border: 1px solid rgba(254, 202, 202, 0.18);
5985 |   border-radius: 8px;
5986 |   padding: 11px;
5987 |   background: rgba(127, 29, 29, 0.24);
5988 | }
5989 |
5990 | .danger-description-stats b {
5991 |   color: #ffffff;
5992 |   font-size: 22px;
5993 |   line-height: 1;
5994 | }
5995 |
5996 | .danger-description-stats em {
5997 |   color: #fecaca;
5998 |   font-size: 10px;
5999 |   font-style: normal;
6000 |   font-weight: 950;
6001 |   text-transform: uppercase;
6002 | }
6003 |
6004 | .danger-description-copy {
6005 |   position: relative;
6006 |   z-index: 1;
6007 |   display: grid;
6008 |   gap: 12px;
6009 | }
6010 |
6011 | .danger-description-copy article {
6012 |   display: grid;
6013 |   gap: 7px;
6014 |   border: 1px solid rgba(254, 202, 202, 0.16);
6015 |   border-radius: 8px;
6016 |   padding: 13px;
6017 |   background: rgba(15, 23, 42, 0.35);
6018 | }
6019 |
6020 | .danger-description-copy .danger-action-card {
6021 |   border-color: rgba(248, 113, 113, 0.34);
6022 |   background:
6023 |     linear-gradient(135deg, rgba(127, 29, 29, 0.5), rgba(15, 23, 42, 0.42)),
6024 |     rgba(127, 29, 29, 0.28);
6025 |   box-shadow: inset 3px 0 0 #ef4444;
6026 | }
6027 |
6028 | .danger-description-copy article > span {
6029 |   color: #fca5a5;
6030 |   font-size: 11px;
6031 |   font-weight: 950;
6032 |   letter-spacing: 0.03em;
6033 |   text-transform: uppercase;
6034 | }
6035 |
6036 | .danger-description-copy p {
6037 |   color: #fee2e2;
6038 |   font-size: 13px;
6039 |   font-weight: 650;
6040 |   line-height: 1.62;
6041 | }
6042 |
6043 | .danger-live-dot {
6044 |   width: 14px;
6045 |   height: 14px;

```



```

6046 | border-radius: 999px;
6047 | background: #ef4444;
6048 | box-shadow:
6049 |   0 0 0 rgba(248, 113, 113, 0.78),
6050 |   0 0 24px rgba(248, 113, 113, 0.95);
6051 | animation: dangerPulse 0.72s ease-in-out infinite;
6052 | }
6053 |
6054 | @keyframes dangerScan {
6055 |   0%,
6056 |   38% {
6057 |     transform: translateX(-100%);
6058 |   }
6059 |   100% {
6060 |     transform: translateX(100%);
6061 |   }
6062 | }
6063 |
6064 | .finding-metrics {
6065 |   display: grid;
6066 |   grid-template-columns: repeat(3, minmax(0, 1fr));
6067 |   gap: 8px;
6068 | }
6069 |
6070 | .finding-metrics span {
6071 |   border-radius: 8px;
6072 |   padding: 10px 9px;
6073 |   background: #f8fafc;
6074 |   color: #334155;
6075 |   font-size: 14px;
6076 |   font-weight: 900;
6077 | }
6078 |
6079 | .finding-actions {
6080 |   display: flex;
6081 |   flex-wrap: wrap;
6082 |   gap: 8px;
6083 | }
6084 |
6085 | .finding-actions button,
6086 | .investigation-header button {
6087 |   min-height: 32px;
6088 |   border: 0;
6089 |   border-radius: 8px;
6090 |   padding: 0 11px;
6091 |   background: #f1f5f9;
6092 |   color: #334155;
6093 |   font-size: 12px;
6094 |   font-weight: 900;
6095 |   transition:
6096 |     transform 160ms ease,
6097 |     background 160ms ease,
6098 |     color 160ms ease;
6099 | }
6100 |
6101 | .finding-actions button:first-child {
6102 |   background: #0f172a;
6103 |   color: #ffffff;
6104 | }
6105 |
6106 | .finding-actions button:hover,
6107 | .investigation-header button:hover {
6108 |   background: #2563eb;
6109 |   color: #ffffff;
6110 |   transform: translateY(-1px);
6111 | }
6112 |
6113 | .investigation-panel {
6114 |   display: grid;
6115 |   gap: 18px;
6116 |   background:
6117 |     linear-gradient(135deg, rgba(239, 246, 255, 0.92), rgba(255, 255, 255, 0.98)),
6118 |     #ffffff;
6119 | }
6120 |
6121 | .side-investigation-panel {
6122 |   position: sticky;
6123 |   top: 94px;
6124 |   max-height: calc(100vh - 120px);
6125 |   overflow-y: auto;
6126 |   animation: detailSlideIn 220ms ease-out both;
6127 | }
6128 |
6129 | @keyframes detailSlideIn {
6130 |   from {
6131 |     opacity: 0;
6132 |     transform: translateX(18px);
6133 |   }
6134 |   to {
6135 |     opacity: 1;
6136 |     transform: translateX(0);
6137 |   }
6138 | }
6139 |
6140 |
6141 | @keyframes ai-progress-enter {
6142 |   from {
6143 |     opacity: 0;
6144 |     transform: translateY(14px) scale(0.99);
6145 |   }
6146 |   to {
6147 |     opacity: 1;
6148 |     transform: translateY(0) scale(1);
6149 |   }
6150 | }
6151 |
6152 |
6153 | @keyframes ai-orb-pulse {
6154 |   0%,
6155 |   100% {

```

```

6156 |     transform: scale(1);
6157 |     box-shadow:
6158 |       inset 0 0 0 1px rgba(255, 255, 255, 0.18),
6159 |       0 0 0 rgba(147, 197, 253, 0);
6160 |   }
6161 |
6162 |   50% {
6163 |     transform: scale(1.05);
6164 |     box-shadow:
6165 |       inset 0 0 0 1px rgba(255, 255, 255, 0.26),
6166 |       0 0 34px rgba(147, 197, 253, 0.42);
6167 |   }
6168 | }
6169 |
6170 | .investigation-header {
6171 |   display: flex;
6172 |   justify-content: space-between;
6173 |   gap: 16px;
6174 |   align-items: start;
6175 | }
6176 |
6177 | .investigation-header h2 {
6178 |   margin: 4px 0 0;
6179 |   color: #0f172a;
6180 |   font-size: clamp(22px, 2.4vw, 32px);
6181 | }
6182 |
6183 | .investigation-grid {
6184 |   display: grid;
6185 |   grid-template-columns: repeat(6, minmax(0, 1fr));
6186 |   gap: 10px;
6187 | }
6188 |
6189 | .investigation-metric {
6190 |   display: grid;
6191 |   gap: 5px;
6192 |   border-radius: 8px;
6193 |   padding: 12px;
6194 |   background: rgba(255, 255, 255, 0.78);
6195 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.74);
6196 | }
6197 |
6198 | .investigation-metric strong {
6199 |   color: #0f172a;
6200 |   font-size: 14px;
6201 |   line-height: 1.25;
6202 | }
6203 |
6204 | .investigation-copy-grid {
6205 |   display: grid;
6206 |   grid-template-columns: repeat(3, minmax(0, 1fr));
6207 |   gap: 12px;
6208 | }
6209 |
6210 | .side-investigation-panel .investigation-grid {
6211 |   grid-template-columns: repeat(2, minmax(0, 1fr));
6212 | }
6213 |
6214 | .side-investigation-panel .investigation-copy-grid,
6215 | .side-investigation-panel .evidence-strip {
6216 |   grid-template-columns: 1fr;
6217 | }
6218 |
6219 | .investigation-copy-grid article {
6220 |   border-radius: 8px;
6221 |   padding: 16px;
6222 |   background: #ffffff;
6223 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.82);
6224 | }
6225 |
6226 | .investigation-copy-grid p {
6227 |   margin: 8px 0 0;
6228 |   color: #334155;
6229 |   line-height: 1.55;
6230 | }
6231 |
6232 | .evidence-strip {
6233 |   display: grid;
6234 |   grid-template-columns: repeat(3, minmax(0, 1fr));
6235 |   gap: 10px;
6236 | }
6237 |
6238 | .evidence-strip blockquote {
6239 |   margin: 0;
6240 |   border-left: 4px solid #2563eb;
6241 |   border-radius: 8px;
6242 |   padding: 14px;
6243 |   background: #ffffff;
6244 |   color: #475569;
6245 |   font-size: 13px;
6246 |   line-height: 1.45;
6247 | }
6248 |
6249 | .analytics-lower-grid {
6250 |   display: grid;
6251 |   grid-template-columns: minmax(360px, 0.9fr) minmax(420px, 1.1fr);
6252 |   gap: 18px;
6253 | }
6254 |
6255 | .premium-analytics-page .recharts-default-tooltip {
6256 |   border: 0 !important;
6257 |   border-radius: 8px !important;
6258 |   background: rgba(255, 255, 255, 0.96) !important;
6259 |   box-shadow: 0 14px 40px rgba(15, 23, 42, 0.14) !important;
6260 |   color: #0f172a !important;
6261 | }
6262 |
6263 | .ai-action-report-card {
6264 |   display: grid;
6265 |   gap: 18px;

```

```

6266 | padding: 24px;
6267 | position: relative;
6268 | min-height: 410px;
6269 | max-height: 430px;
6270 | overflow: hidden;
6271 | background:
6272 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.94)),
6273 |     #ffffff;
6274 | }
6275 |
6276 | .action-report-blur-content {
6277 |     display: grid;
6278 |     gap: 18px;
6279 |     filter: blur(5px);
6280 |     opacity: 0.42;
6281 |     max-height: 380px;
6282 |     overflow: hidden;
6283 |     pointer-events: none;
6284 |     user-select: none;
6285 | }
6286 |
6287 | .action-report-lock-overlay {
6288 |     position: absolute;
6289 |     inset: 0;
6290 |     z-index: 3;
6291 |     display: grid;
6292 |     place-items: center;
6293 |     padding: 24px;
6294 |     background:
6295 |         radial-gradient(circle at center, rgba(255, 255, 255, 0.88), rgba(248, 250, 252, 0.62) 42%, rgba(248, 250, 252, 0.28)),
6296 |         linear-gradient(135deg, rgba(255, 255, 255, 0.44), rgba(219, 234, 254, 0.24));
6297 |     backdrop-filter: blur(2px);
6298 | }
6299 |
6300 | .action-report-lock-card {
6301 |     display: grid;
6302 |     justify-items: center;
6303 |     gap: 12px;
6304 |     width: min(92%, 430px);
6305 |     border-radius: 12px;
6306 |     padding: 24px;
6307 |     text-align: center;
6308 |     background: rgba(255, 255, 255, 0.92);
6309 |     box-shadow:
6310 |         0 24px 70px rgba(15, 23, 42, 0.18),
6311 |         inset 0 0 1px rgba(226, 232, 240, 0.82);
6312 | }
6313 |
6314 | .action-report-lock-card > span {
6315 |     display: inline-flex;
6316 |     align-items: center;
6317 |     gap: 8px;
6318 |     border-radius: 999px;
6319 |     padding: 8px 12px;
6320 |     background: #eff6ff;
6321 |     color: #2563eb;
6322 |     font-size: 12px;
6323 |     font-weight: 950;
6324 |     text-transform: uppercase;
6325 | }
6326 |
6327 | .action-report-lock-card h3 {
6328 |     max-width: 340px;
6329 |     margin: 0;
6330 |     color: #0f172a;
6331 |     font-size: clamp(21px, 2.4vw, 30px);
6332 |     line-height: 1.08;
6333 | }
6334 |
6335 | .action-report-lock-card .action-report-download {
6336 |     min-height: 46px;
6337 |     padding: 0 18px;
6338 |     background: linear-gradient(135deg, #2563eb, #0f172a);
6339 |     box-shadow: 0 18px 44px rgba(37, 99, 235, 0.24);
6340 | }
6341 |
6342 | .action-report-lock-card .action-report-download:hover {
6343 |     background: linear-gradient(135deg, #1d4ed8, #0066b1);
6344 | }
6345 |
6346 | .ai-engine-benchmark-section {
6347 |     display: grid;
6348 |     gap: 18px;
6349 |     overflow: hidden;
6350 |     background:
6351 |         linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.92)),
6352 |         #ffffff;
6353 | }
6354 |
6355 | .benchmark-header {
6356 |     display: grid;
6357 |     justify-items: center;
6358 |     gap: 10px;
6359 |     text-align: center;
6360 | }
6361 |
6362 | .benchmark-header span,
6363 | .ai-pipeline-trace > span {
6364 |     color: #2563eb;
6365 |     font-size: 12px;
6366 |     font-weight: 950;
6367 |     text-transform: uppercase;
6368 | }
6369 |
6370 | .benchmark-header h2 {
6371 |     margin: 5px 0 0;
6372 |     color: #0f172a;
6373 |     font-size: clamp(24px, 2.8vw, 38px);
6374 |     line-height: 1.08;
6375 | }

```

```

6376 |
6377 | .benchmark-header em {
6378 |   border-radius: 999px;
6379 |   padding: 8px 12px;
6380 |   background: #ecfdf5;
6381 |   color: #047857;
6382 |   font-size: 12px;
6383 |   font-style: normal;
6384 |   font-weight: 950;
6385 | }
6386 |
6387 | .benchmark-funnel {
6388 |   display: grid;
6389 |   grid-template-columns: repeat(6, minmax(0, 1fr));
6390 |   gap: 10px;
6391 |   align-items: stretch;
6392 | }
6393 |
6394 | .funnel-step {
6395 |   position: relative;
6396 |   display: grid;
6397 |   min-height: 128px;
6398 |   align-content: center;
6399 |   justify-items: center;
6400 |   gap: 7px;
6401 |   border-radius: 12px;
6402 |   padding: 14px;
6403 |   text-align: center;
6404 |   background: linear-gradient(180deg, #ffffff, #f8fafc);
6405 |   box-shadow:
6406 |     inset 0 0 0 1px rgba(226, 232, 240, 0.86),
6407 |     0 14px 34px rgba(15, 23, 42, 0.06);
6408 | }
6409 |
6410 | .funnel-step:nth-child(5),
6411 | .funnel-step:nth-child(6) {
6412 |   background: linear-gradient(180deg, #eff6ff, #ecfdf5);
6413 | }
6414 |
6415 | .funnel-step strong {
6416 |   color: #0f172a;
6417 |   font-size: clamp(21px, 2.2vw, 30px);
6418 |   line-height: 1;
6419 | }
6420 |
6421 | .funnel-step span {
6422 |   color: #2563eb;
6423 |   font-size: 11px;
6424 |   font-weight: 950;
6425 |   text-transform: uppercase;
6426 | }
6427 |
6428 | .funnel-step p {
6429 |   margin: 0;
6430 |   color: #64748b;
6431 |   font-size: 11px;
6432 |   font-weight: 750;
6433 |   line-height: 1.35;
6434 | }
6435 |
6436 | .funnel-step i {
6437 |   position: absolute;
6438 |   top: 50%;
6439 |   right: -9px;
6440 |   z-index: 2;
6441 |   width: 18px;
6442 |   height: 2px;
6443 |   background: #60a5fa;
6444 | }
6445 |
6446 | .funnel-step i::after {
6447 |   content: "";
6448 |   position: absolute;
6449 |   top: -4px;
6450 |   right: -1px;
6451 |   width: 9px;
6452 |   height: 9px;
6453 |   border-top: 2px solid #60a5fa;
6454 |   border-right: 2px solid #60a5fa;
6455 |   transform: rotate(45deg);
6456 | }
6457 |
6458 | .benchmark-lower {
6459 |   display: grid;
6460 |   grid-template-columns: minmax(300px, 0.78fr) minmax(0, 1fr);
6461 |   gap: 16px;
6462 | }
6463 |
6464 | .ai-run-receipt {
6465 |   display: grid;
6466 |   gap: 8px;
6467 |   border-radius: 12px;
6468 |   padding: 18px;
6469 |   background: #0f172a;
6470 |   color: #e2e8f0;
6471 |   box-shadow: 0 20px 54px rgba(15, 23, 42, 0.18);
6472 | }
6473 |
6474 | .ai-run-receipt > div {
6475 |   display: flex;
6476 |   align-items: center;
6477 |   justify-content: space-between;
6478 |   gap: 10px;
6479 |   border-bottom: 1px dashed rgba(148, 163, 184, 0.34);
6480 |   padding-bottom: 10px;
6481 | }
6482 |
6483 | .ai-run-receipt > div span {
6484 |   color: #93c5fd;
6485 |   font-family: "Consolas", "Courier New", monospace;

```

```

6486 | font-size: 12px;
6487 | font-weight: 950;
6488 | }
6489 |
6490 | .ai-run-receipt > div b {
6491 |   color: #94a3b8;
6492 |   font-family: "Consolas", "Courier New", monospace;
6493 |   font-size: 11px;
6494 |   font-weight: 800;
6495 | }
6496 |
6497 | .ai-run-receipt p {
6498 |   display: flex;
6499 |   justify-content: space-between;
6500 |   gap: 12px;
6501 |   margin: 0;
6502 |   font-family: "Consolas", "Courier New", monospace;
6503 |   font-size: 12px;
6504 | }
6505 |
6506 | .ai-run-receipt p span {
6507 |   color: #cbd5e1;
6508 | }
6509 |
6510 | .ai-run-receipt p strong {
6511 |   color: #ffffff;
6512 |   text-align: right;
6513 | }
6514 |
6515 | .ai-pipeline-trace {
6516 |   display: grid;
6517 |   align-content: start;
6518 |   gap: 11px;
6519 |   border-radius: 12px;
6520 |   padding: 18px;
6521 |   background: #f8fafc;
6522 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.86);
6523 | }
6524 |
6525 | .ai-pipeline-trace p {
6526 |   display: grid;
6527 |   grid-template-columns: 12px 1fr;
6528 |   gap: 10px;
6529 |   align-items: center;
6530 |   margin: 0;
6531 | }
6532 |
6533 | .ai-pipeline-trace i {
6534 |   width: 9px;
6535 |   height: 9px;
6536 |   border-radius: 999px;
6537 |   background: #10b981;
6538 |   box-shadow: 0 0 0 5px rgba(16, 185, 129, 0.12);
6539 | }
6540 |
6541 | .ai-pipeline-trace code {
6542 |   color: #0f172a;
6543 |   font-family: "Consolas", "Courier New", monospace;
6544 |   font-size: 12px;
6545 |   font-weight: 850;
6546 | }
6547 |
6548 | .analysis-team-footer {
6549 |   margin: -2px 0 10px;
6550 |   color: #64748b;
6551 |   font-size: 12px;
6552 |   font-weight: 900;
6553 |   letter-spacing: 0;
6554 |   text-align: center;
6555 | }
6556 |
6557 | .action-report-header {
6558 |   display: flex;
6559 |   align-items: start;
6560 |   justify-content: space-between;
6561 |   gap: 16px;
6562 |   border-bottom: 1px solid rgba(226, 232, 240, 0.92);
6563 |   padding-bottom: 16px;
6564 | }
6565 |
6566 | .action-report-header span,
6567 | .action-report-summary span,
6568 | .action-section-title span {
6569 |   color: #2563eb;
6570 |   font-size: 12px;
6571 |   font-weight: 950;
6572 |   text-transform: uppercase;
6573 | }
6574 |
6575 | .action-report-header h2 {
6576 |   max-width: 980px;
6577 |   margin: 5px 0 0;
6578 |   color: #0f172a;
6579 |   font-size: clamp(26px, 3vw, 42px);
6580 |   line-height: 1.08;
6581 | }
6582 |
6583 | .action-report-download {
6584 |   display: inline-flex;
6585 |   flex: 0 0 auto;
6586 |   align-items: center;
6587 |   justify-content: center;
6588 |   gap: 8px;
6589 |   min-height: 42px;
6590 |   border: 0;
6591 |   border-radius: 8px;
6592 |   padding: 0 15px;
6593 |   background: #0f172a;
6594 |   color: #ffffff;
6595 |   font-size: 13px;

```

```

6596 | font-weight: 900;
6597 | box-shadow: 0 14px 34px rgba(15, 23, 42, 0.16);
6598 | transition:
6599 |     transform 160ms ease,
6600 |     box-shadow 160ms ease,
6601 |     background 160ms ease;
6602 | }
6603 |
6604 | .action-report-download:hover {
6605 |     background: #2563eb;
6606 |     box-shadow: 0 18px 44px rgba(37, 99, 235, 0.2);
6607 |     transform: translateY(-1px);
6608 | }
6609 |
6610 | .action-report-summary {
6611 |     display: grid;
6612 |     grid-template-columns: minmax(0, 1fr) minmax(260px, 420px);
6613 |     gap: 18px;
6614 |     align-items: stretch;
6615 |     border-radius: 8px;
6616 |     padding: 18px;
6617 |     background:
6618 |         linear-gradient(135deg, rgba(239, 246, 255, 0.92), rgba(236, 253, 245, 0.72)),
6619 |         #f8fafc;
6620 | }
6621 |
6622 | .action-report-summary p {
6623 |     margin: 8px 0 0;
6624 |     color: #334155;
6625 |     font-size: 16px;
6626 |     line-height: 1.68;
6627 | }
6628 |
6629 | .summary-metric-strip {
6630 |     display: grid;
6631 |     grid-template-columns: repeat(2, minmax(0, 1fr));
6632 |     gap: 10px;
6633 | }
6634 |
6635 | .summary-metric-strip strong {
6636 |     display: grid;
6637 |     gap: 6px;
6638 |     align-content: center;
6639 |     border-radius: 8px;
6640 |     padding: 14px;
6641 |     background: rgba(255, 255, 255, 0.86);
6642 |     color: #0f172a;
6643 |     font-size: 22px;
6644 |     line-height: 1.1;
6645 |     box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.84);
6646 | }
6647 |
6648 | .summary-metric-strip em {
6649 |     color: #64748b;
6650 |     font-size: 11px;
6651 |     font-style: normal;
6652 |     font-weight: 900;
6653 |     text-transform: uppercase;
6654 | }
6655 |
6656 | .action-report-section {
6657 |     display: grid;
6658 |     gap: 12px;
6659 |     border-radius: 8px;
6660 |     padding: 16px;
6661 |     background: #ffffff;
6662 |     box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.86);
6663 | }
6664 |
6665 | .action-section-title {
6666 |     display: flex;
6667 |     align-items: center;
6668 |     justify-content: space-between;
6669 |     gap: 12px;
6670 | }
6671 |
6672 | .action-section-title em {
6673 |     border-radius: 999px;
6674 |     padding: 7px 10px;
6675 |     background: #eff6ff;
6676 |     color: #1d4ed8;
6677 |     font-size: 12px;
6678 |     font-style: normal;
6679 |     font-weight: 850;
6680 | }
6681 |
6682 | .action-issue-table {
6683 |     display: grid;
6684 |     overflow: hidden;
6685 |     border-radius: 8px;
6686 |     box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.92);
6687 | }
6688 |
6689 | .action-issue-head,
6690 | .action-issue-row {
6691 |     display: grid;
6692 |     grid-template-columns: minmax(150px, 1fr) 120px 90px minmax(140px, 0.85fr) minmax(260px, 1.4fr);
6693 |     gap: 12px;
6694 |     align-items: center;
6695 |     padding: 12px;
6696 | }
6697 |
6698 | .action-issue-head {
6699 |     background: #f8fafc;
6700 |     color: #475569;
6701 |     font-size: 11px;
6702 |     font-weight: 950;
6703 |     text-transform: uppercase;
6704 | }
6705 |

```

```

6706 | .action-issue-row {
6707 |   border-top: 1px solid rgba(226, 232, 240, 0.82);
6708 |   color: #334155;
6709 |   font-size: 13px;
6710 | }
6711 |
6712 | .action-issue-row strong {
6713 |   color: #0f172a;
6714 |   font-size: 14px;
6715 | }
6716 |
6717 | .action-issue-row p {
6718 |   margin: 0;
6719 |   color: #475569;
6720 |   line-height: 1.45;
6721 | }
6722 |
6723 | .priority-pill {
6724 |   width: fit-content;
6725 |   border-radius: 999px;
6726 |   padding: 6px 9px;
6727 |   background: #e0f2fe;
6728 |   color: #0369a1;
6729 |   font-size: 11px;
6730 |   font-weight: 950;
6731 |   text-transform: uppercase;
6732 | }
6733 |
6734 | .priority-pill.critical {
6735 |   background: #fee2e2;
6736 |   color: #991b1b;
6737 | }
6738 |
6739 | .priority-pill.high {
6740 |   background: #ffedd5;
6741 |   color: #9a3412;
6742 | }
6743 |
6744 | .priority-pill.medium {
6745 |   background: #fef3c7;
6746 |   color: #854d0e;
6747 | }
6748 |
6749 | .priority-pill.low {
6750 |   background: #dcfce7;
6751 |   color: #166534;
6752 | }
6753 |
6754 | .action-report-two-column {
6755 |   display: grid;
6756 |   grid-template-columns: minmax(0, 0.95fr) minmax(0, 1.05fr);
6757 |   gap: 16px;
6758 | }
6759 |
6760 | .root-cause-grid,
6761 | .ai-action-plan-list {
6762 |   display: grid;
6763 |   gap: 10px;
6764 | }
6765 |
6766 | .root-cause-card {
6767 |   display: grid;
6768 |   gap: 7px;
6769 |   border-radius: 8px;
6770 |   padding: 13px;
6771 |   background: #f8fafc;
6772 | }
6773 |
6774 | .root-cause-card strong {
6775 |   color: #0f172a;
6776 |   font-size: 15px;
6777 | }
6778 |
6779 | .root-cause-card p {
6780 |   margin: 0;
6781 |   color: #334155;
6782 |   line-height: 1.48;
6783 | }
6784 |
6785 | .root-cause-card span {
6786 |   color: #64748b;
6787 |   font-size: 12px;
6788 |   font-weight: 850;
6789 | }
6790 |
6791 | .ai-action-plan-item {
6792 |   display: grid;
6793 |   grid-template-columns: 34px minmax(0, 1fr);
6794 |   gap: 12px;
6795 |   border-radius: 8px;
6796 |   padding: 13px;
6797 |   background: #f8fafc;
6798 | }
6799 |
6800 | .ai-action-plan-item > b {
6801 |   display: grid;
6802 |   place-items: center;
6803 |   width: 30px;
6804 |   height: 30px;
6805 |   border-radius: 8px;
6806 |   background: #0f172a;
6807 |   color: #ffffff;
6808 |   font-size: 13px;
6809 | }
6810 |
6811 | .ai-action-plan-item div {
6812 |   display: grid;
6813 |   gap: 7px;
6814 | }
6815 |

```

```

6816 | .ai-action-plan-item strong {
6817 |   color: #0f172a;
6818 |   font-size: 15px;
6819 | }
6820 |
6821 | .ai-action-plan-item p {
6822 |   margin: 0;
6823 |   color: #475569;
6824 |   line-height: 1.45;
6825 | }
6826 |
6827 | .semester-line-section {
6828 |   min-height: 400px;
6829 | }
6830 |
6831 | .line-legend {
6832 |   display: flex;
6833 |   flex-wrap: wrap;
6834 |   gap: 12px;
6835 |   margin-top: -2px;
6836 | }
6837 |
6838 | .line-legend span {
6839 |   display: inline-flex;
6840 |   align-items: center;
6841 |   gap: 7px;
6842 |   color: #475569;
6843 |   font-size: 13px;
6844 |   font-weight: 850;
6845 | }
6846 |
6847 | .line-legend i {
6848 |   width: 22px;
6849 |   height: 4px;
6850 |   border-radius: 999px;
6851 | }
6852 |
6853 | .line-legend .current {
6854 |   background: #2563eb;
6855 | }
6856 |
6857 | .line-legend .previous {
6858 |   background: #10b981;
6859 | }
6860 |
6861 | .leaderboard-list,
6862 | .recommendation-list {
6863 |   display: grid;
6864 |   gap: 10px;
6865 | }
6866 |
6867 | .leaderboard-row {
6868 |   display: grid;
6869 |   grid-template-columns: 30px 56px minmax(100px, 1fr) 48px 46px;
6870 |   gap: 10px;
6871 |   align-items: center;
6872 |   border-radius: 8px;
6873 |   padding: 10px;
6874 |   background: #f8fafc;
6875 | }
6876 |
6877 | .leaderboard-row b {
6878 |   display: grid;
6879 |   place-items: center;
6880 |   width: 28px;
6881 |   height: 28px;
6882 |   border-radius: 8px;
6883 |   background: #ffffff;
6884 |   color: #2563eb;
6885 |   box-shadow: inset 0 0 0 1px rgba(226, 232, 240, 0.9);
6886 | }
6887 |
6888 | .leaderboard-row strong {
6889 |   color: #0f172a;
6890 |   font-size: 14px;
6891 | }
6892 |
6893 | .leaderboard-row div {
6894 |   overflow: hidden;
6895 |   height: 8px;
6896 |   border-radius: 999px;
6897 |   background: #e2e8f0;
6898 | }
6899 |
6900 | .leaderboard-row i {
6901 |   display: block;
6902 |   height: 100%;
6903 |   border-radius: inherit;
6904 |   background: linear-gradient(90deg, #38bdf8, #22c55e);
6905 | }
6906 |
6907 | .leaderboard-row span {
6908 |   color: #0f172a;
6909 |   font-size: 13px;
6910 |   font-weight: 900;
6911 | }
6912 |
6913 | .risk-monitor-grid {
6914 |   display: grid;
6915 |   grid-template-columns: repeat(2, minmax(0, 1fr));
6916 |   gap: 10px;
6917 | }
6918 |
6919 | .risk-monitor-card {
6920 |   display: grid;
6921 |   gap: 10px;
6922 |   border-radius: 8px;
6923 |   padding: 16px;
6924 |   color: #0f172a;
6925 | }

```



```

6926 |
6927 | .risk-monitor-card span {
6928 |   color: rgba(15, 23, 42, 0.72);
6929 |   font-size: 12px;
6930 |   font-weight: 950;
6931 |   text-transform: uppercase;
6932 | }
6933 |
6934 | .risk-monitor-card strong {
6935 |   font-size: 34px;
6936 |   line-height: 1;
6937 | }
6938 |
6939 | .risk-monitor-card i {
6940 |   width: 44px;
6941 |   height: 5px;
6942 |   border-radius: 999px;
6943 |   background: currentColor;
6944 |   opacity: 0.36;
6945 | }
6946 |
6947 | .risk-monitor-card.critical {
6948 |   background: #fee2e2;
6949 |   color: #991b1b;
6950 | }
6951 |
6952 | .risk-monitor-card.high {
6953 |   background: #ffedd5;
6954 |   color: #9a3412;
6955 | }
6956 |
6957 | .risk-monitor-card.medium {
6958 |   background: #ffe3c7;
6959 |   color: #854d0e;
6960 | }
6961 |
6962 | .risk-monitor-card.low {
6963 |   background: #d1ecf1;
6964 |   color: #17a2b8;
6965 | }
6966 |
6967 | .recommendation-list article {
6968 |   display: grid;
6969 |   gap: 9px;
6970 |   border-radius: 8px;
6971 |   padding: 14px;
6972 |   background: #f8f9fa;
6973 |   transition:
6974 |     transform 180ms ease,
6975 |     box-shadow 180ms ease;
6976 | }
6977 |
6978 | .recommendation-list article div,
6979 | .recommendation-list footer {
6980 |   display: flex;
6981 |   align-items: center;
6982 |   justify-content: space-between;
6983 |   gap: 10px;
6984 | }
6985 |
6986 | .recommendation-list strong {
6987 |   color: #007bff;
6988 |   font-size: 15px;
6989 | }
6990 |
6991 | .recommendation-list span {
6992 |   border-radius: 999px;
6993 |   padding: 6px 8px;
6994 |   background: #e2e3e5;
6995 |   color: #6c757d;
6996 |   font-size: 11px;
6997 |   font-weight: 950;
6998 |   text-transform: uppercase;
6999 | }
7000 |
7001 | .recommendation-list p {
7002 |   margin: 0;
7003 |   color: #6c757d;
7004 |   font-size: 13px;
7005 |   line-height: 1.45;
7006 | }
7007 |
7008 | .recommendation-list em {
7009 |   color: #6c757d;
7010 |   font-size: 12px;
7011 |   font-style: normal;
7012 |   font-weight: 850;
7013 | }
7014 |
7015 | .recommendation-list b {
7016 |   color: #6c757d;
7017 |   font-size: 12px;
7018 | }
7019 |
7020 | @media (max-width: 1260px) {
7021 |   .analytics-top-nav {
7022 |     grid-template-columns: auto minmax(220px, 1fr) auto auto;
7023 |   }
7024 |
7025 |   .analytics-filters {
7026 |     grid-column: 1 / -1;
7027 |     flex-wrap: wrap;
7028 |     width: auto;
7029 |   }
7030 |
7031 |   .premium-kpi-grid {
7032 |     grid-template-columns: repeat(3, minmax(0, 1fr));
7033 |   }
7034 |
7035 |   .ai-quality-stats {

```

```

7036 |     grid-template-columns: repeat(3, minmax(0, 1fr));
7037 | }
7038 |
7039 | .ai-quality-details {
7040 |     grid-template-columns: repeat(2, minmax(0, 1fr));
7041 | }
7042 |
7043 | .quality-breakdown-grid {
7044 |     grid-template-columns: repeat(2, minmax(0, 1fr));
7045 | }
7046 |
7047 | .analytics-chart-row,
7048 | .analytics-hero,
7049 | .ai-progress-panel,
7050 | .benchmark-lower,
7051 | .concerning-issues-layout.has-detail,
7052 | .heatmap-radar-row,
7053 | .analytics-lower-grid {
7054 |     grid-template-columns: 1fr;
7055 | }
7056 |
7057 | .benchmark-funnel {
7058 |     grid-template-columns: repeat(3, minmax(0, 1fr));
7059 | }
7060 |
7061 | .ai-finding-grid {
7062 |     grid-template-columns: repeat(2, minmax(0, 1fr));
7063 | }
7064 |
7065 | .concerning-issue-grid {
7066 |     grid-template-columns: repeat(2, minmax(0, 1fr));
7067 | }
7068 |
7069 | .investigation-grid {
7070 |     grid-template-columns: repeat(3, minmax(0, 1fr));
7071 | }
7072 | }
7073 |
7074 | @media (max-width: 760px) {
7075 |     .workspace:has(.premium-analytics-page) {
7076 |         width: min(100% - 20px, 1680px);
7077 |         margin-top: 10px;
7078 |     }
7079 |
7080 |     .analytics-top-nav {
7081 |         position: relative;
7082 |         top: auto;
7083 |         grid-template-columns: 1fr 1fr;
7084 |     }
7085 |
7086 |     .analytics-brand,
7087 |     .global-search,
7088 |     .analytics-filters {
7089 |         grid-column: 1 / -1;
7090 |     }
7091 |
7092 |     .nav-secondary-action,
7093 |     .nav-primary-action {
7094 |         width: 100%;
7095 |     }
7096 |
7097 |     .analytics-hero,
7098 |     .premium-panel,
7099 |     .premium-kpi-card,
7100 |     .ai-finding-card {
7101 |         padding: 16px;
7102 |     }
7103 |
7104 |     .premium-kpi-grid,
7105 |     .ai-progress-steps,
7106 |     .benchmark-funnel,
7107 |     .concerning-issues-layout,
7108 |     .ai-quality-stats,
7109 |     .ai-quality-details,
7110 |     .quality-breakdown-grid,
7111 |     .ai-finding-grid,
7112 |     .concerning-issue-grid,
7113 |     .investigation-grid,
7114 |     .investigation-copy-grid,
7115 |     .evidence-strip,
7116 |     .modern-sentiment-legend {
7117 |         grid-template-columns: 1fr;
7118 |     }
7119 |
7120 |     .ai-quality-header {
7121 |         display: grid;
7122 |     }
7123 |
7124 |     .quality-evidence-toolbar {
7125 |         display: grid;
7126 |     }
7127 |
7128 |     .ai-quality-header button {
7129 |         width: 100%;
7130 |     }
7131 |
7132 |     .quality-evidence-toolbar button {
7133 |         width: 100%;
7134 |     }
7135 |
7136 |     .finding-metrics {
7137 |         grid-template-columns: 1fr;
7138 |     }
7139 |
7140 |     .leaderboard-row {
7141 |         grid-template-columns: 30px 52px 1fr;
7142 |     }
7143 |
7144 |     .leaderboard-row div,
7145 |     .leaderboard-row em {

```

```

7146 |     grid-column: 2 / -1;
7147 | }
7148 |
7149 | .risk-monitor-grid {
7150 |     grid-template-columns: 1fr;
7151 | }
7152 | }
7153 |
7154 | /* Final visibility fix for admin home cards + aligned auth palette. */
7155 | .app-shell:has(.admin-home-panel) {
7156 |     background:
7157 |         linear-gradient(135deg, rgba(79, 70, 229, 0.08), transparent 34%),
7158 |         radial-gradient(circle at 16% 18%, rgba(20, 184, 166, 0.16), transparent 26%),
7159 |         radial-gradient(circle at 84% 12%, rgba(59, 130, 246, 0.16), transparent 30%),
7160 |         #f7f8fb !important;
7161 | }
7162 |
7163 | .workspace:has(.admin-home-panel) {
7164 |     width: min(1240px, 100%) !important;
7165 |     margin: 0 auto !important;
7166 |     padding: 28px !important;
7167 |     overflow: visible !important;
7168 | }
7169 |
7170 | .admin-home-panel {
7171 |     border: 1px solid rgba(15, 23, 42, 0.08) !important;
7172 |     border-radius: 28px !important;
7173 |     background:
7174 |         radial-gradient(circle at 88% 6%, rgba(37, 99, 235, 0.12), transparent 28%),
7175 |         linear-gradient(145deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.9)) !important;
7176 |     color: #0f172a !important;
7177 |     box-shadow:
7178 |         0 30px 90px rgba(15, 23, 42, 0.1),
7179 |         inset 0 1px 0 rgba(255, 255, 255, 0.9) !important;
7180 | }
7181 |
7182 | .admin-home-panel,
7183 | .admin-home-panel * {
7184 |     visibility: visible !important;
7185 | }
7186 |
7187 | .admin-home-header,
7188 | .admin-dataset-panel,
7189 | .admin-action-grid,
7190 | .admin-validation-panel,
7191 | .admin-validation-result {
7192 |     opacity: 1 !important;
7193 |     transform: none !important;
7194 | }
7195 |
7196 | .admin-home-header h1,
7197 | .admin-home-header h1 span {
7198 |     color: #0f172a !important;
7199 |     -webkit-text-fill-color: initial !important;
7200 | }
7201 |
7202 | .admin-home-header h1 span {
7203 |     background: linear-gradient(94deg, #1d4ed8, #0f766e) !important;
7204 |     background-clip: text !important;
7205 |     -webkit-background-clip: text !important;
7206 |     -webkit-text-fill-color: transparent !important;
7207 | }
7208 |
7209 | .admin-home-subheading,
7210 | .admin-home-header p:not(.admin-home-eyebrow) {
7211 |     color: #475569 !important;
7212 | }
7213 |
7214 | .admin-action-grid {
7215 |     display: grid !important;
7216 |     grid-template-columns: repeat(3, minmax(0, 1fr)) !important;
7217 |     gap: 20px !important;
7218 |     min-height: 1px !important;
7219 |     position: relative !important;
7220 |     z-index: 20 !important;
7221 |     pointer-events: auto !important;
7222 | }
7223 |
7224 | .admin-action-card {
7225 |     display: flex !important;
7226 |     min-height: 270px !important;
7227 |     border: 1px solid rgba(15, 23, 42, 0.08) !important;
7228 |     border-radius: 22px !important;
7229 |     padding: 28px !important;
7230 |     background: linear-gradient(180deg, #ffffff, #f8fafc) !important;
7231 |     color: #0f172a !important;
7232 |     box-shadow: 0 20px 54px rgba(15, 23, 42, 0.08) !important;
7233 |     opacity: 1 !important;
7234 |     visibility: visible !important;
7235 |     transform: none !important;
7236 | }
7237 |
7238 | .admin-action-card:hover {
7239 |     border-color: rgba(37, 99, 235, 0.22) !important;
7240 |     box-shadow: 0 30px 76px rgba(37, 99, 235, 0.13) !important;
7241 |     transform: translateY(-5px) !important;
7242 | }
7243 |
7244 | .admin-action-card strong {
7245 |     color: #0f172a !important;
7246 |     opacity: 1 !important;
7247 | }
7248 |
7249 | .admin-action-card p,
7250 | .admin-action-card span,
7251 | .admin-action-card em {
7252 |     opacity: 1 !important;
7253 | }
7254 |
7255 | .admin-action-card p {

```

```

7256 | color: #475569 !important;
7257 | }
7258 |
7259 | .action-card-subtitle {
7260 |   color: #2563eb !important;
7261 | }
7262 |
7263 | .action-card-badge {
7264 |   border-color: rgba(15, 23, 42, 0.08) !important;
7265 |   border-radius: 999px !important;
7266 |   background: #f1f5f9 !important;
7267 |   color: #334155 !important;
7268 | }
7269 |
7270 | .action-card-badge.highlight {
7271 |   background: #eff6ff !important;
7272 |   color: #1d4ed8 !important;
7273 | }
7274 |
7275 | .action-card-badge.danger {
7276 |   background: #fef2f2 !important;
7277 |   color: #dc2626 !important;
7278 | }
7279 |
7280 | .action-icon-wrapper {
7281 |   border-color: rgba(15, 23, 42, 0.08) !important;
7282 |   border-radius: 16px !important;
7283 |   background:
7284 |     radial-gradient(circle at 26% 20%, rgba(255, 255, 255, 0.8), transparent 30%),
7285 |     linear-gradient(135deg, #0f172a, #2563eb 62%, #14b8a6) !important;
7286 |   color: #ffffff !important;
7287 |   box-shadow: 0 18px 42px rgba(37, 99, 235, 0.14) !important;
7288 | }
7289 |
7290 | .action-card-accent {
7291 |   width: 5px !important;
7292 |   border-radius: 999px !important;
7293 | }
7294 |
7295 | .create-card .action-card-accent {
7296 |   background: linear-gradient(to bottom, #2563eb, #06b6d4) !important;
7297 | }
7298 |
7299 | .analysis-card .action-card-accent {
7300 |   background: linear-gradient(to bottom, #14b8a6, #2563eb) !important;
7301 | }
7302 |
7303 | .admin-dataset-panel,
7304 | .admin-validation-panel,
7305 | .admin-validation-result {
7306 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
7307 |   border-radius: 22px !important;
7308 |   background: rgba(255, 255, 255, 0.86) !important;
7309 |   color: #0f172a !important;
7310 |   box-shadow: 0 18px 48px rgba(15, 23, 42, 0.07) !important;
7311 | }
7312 |
7313 | .btn-admin-choose-dataset,
7314 | .btn-admin-refresh-datasets,
7315 | .btn-admin-validation {
7316 |   border-radius: 14px !important;
7317 | }
7318 |
7319 | .btn-admin-choose-dataset,
7320 | .btn-admin-validation {
7321 |   background: linear-gradient(135deg, #0f172a 0%, #2563eb 60%, #14b8a6 100%) !important;
7322 |   color: #ffffff !important;
7323 |   border-color: transparent !important;
7324 | }
7325 |
7326 | .btn-admin-refresh-datasets {
7327 |   background: #ffffff !important;
7328 |   color: #0f172a !important;
7329 |   border-color: rgba(15, 23, 42, 0.1) !important;
7330 | }
7331 |
7332 | .auth-shell {
7333 |   background:
7334 |     linear-gradient(135deg, rgba(79, 70, 229, 0.08), transparent 34%),
7335 |     radial-gradient(circle at 16% 18%, rgba(20, 184, 166, 0.16), transparent 26%),
7336 |     radial-gradient(circle at 84% 12%, rgba(59, 130, 246, 0.16), transparent 30%),
7337 |     #f7f8fb !important;
7338 | }
7339 |
7340 | .auth-shell .panel {
7341 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
7342 |   border-radius: 24px !important;
7343 |   background: linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.88)) !important;
7344 |   color: #172033 !important;
7345 |   box-shadow: 0 28px 84px rgba(15, 23, 42, 0.12) !important;
7346 | }
7347 |
7348 | .auth-shell .panel h2,
7349 | .auth-shell .signal-panel h2,
7350 | .auth-shell label,
7351 | .auth-shell .signal-row strong {
7352 |   color: #0f172a !important;
7353 | }
7354 |
7355 | .auth-shell .signal-row span {
7356 |   color: #64748b !important;
7357 | }
7358 |
7359 | .auth-shell input {
7360 |   border-radius: 15px !important;
7361 |   background: #ffffff !important;
7362 |   color: #172033 !important;
7363 | }
7364 |
7365 | @media (max-width: 900px) {

```

```

7366 | .admin-action-grid {
7367 |   grid-template-columns: 1fr !important;
7368 | }
7369 |
7370 | .admin-dataset-panel,
7371 | .admin-validation-panel {
7372 |   align-items: stretch !important;
7373 |   flex-direction: column !important;
7374 | }
7375 | }
7376 |
7377 | /* Non-admin visual refresh: public, auth and student surfaces only. */
7378 | .public-shell {
7379 |   isolation: isolate;
7380 |   padding: 20px;
7381 |   background:
7382 |     linear-gradient(135deg, rgba(79, 70, 229, 0.08), transparent 34%),
7383 |     radial-gradient(circle at 16% 18%, rgba(20, 184, 166, 0.16), transparent 26%),
7384 |     radial-gradient(circle at 84% 12%, rgba(59, 130, 246, 0.16), transparent 30%),
7385 |     radial-gradient(circle at 62% 92%, rgba(244, 114, 182, 0.12), transparent 28%),
7386 |     #f7f8fb;
7387 | }
7388 |
7389 | .public-shell::before {
7390 |   content: '';
7391 |   position: fixed;
7392 |   inset: 0;
7393 |   z-index: -1;
7394 |   pointer-events: none;
7395 |   background-image:
7396 |     linear-gradient(rgba(15, 23, 42, 0.035) 1px, transparent 1px),
7397 |     linear-gradient(90deg, rgba(15, 23, 42, 0.035) 1px, transparent 1px);
7398 |   background-size: 42px 42px;
7399 |   mask-image: linear-gradient(to bottom, rgba(0, 0, 0, 0.72), transparent 82%);
7400 | }
7401 |
7402 | .landing-page {
7403 |   position: relative;
7404 |   min-height: calc(100vh - 40px);
7405 |   border: 1px solid rgba(15, 23, 42, 0.08);
7406 |   border-radius: 28px;
7407 |   background:
7408 |     linear-gradient(145deg, rgba(255, 255, 255, 0.96), rgba(248, 250, 252, 0.88)),
7409 |     radial-gradient(circle at 78% 34%, rgba(59, 130, 246, 0.18), transparent 32%),
7410 |     radial-gradient(circle at 16% 78%, rgba(20, 184, 166, 0.12), transparent 28%);
7411 |   box-shadow:
7412 |     0 30px 90px rgba(15, 23, 42, 0.12),
7413 |     inset 0 1px 0 rgba(255, 255, 255, 0.9);
7414 | }
7415 |
7416 | .landing-page::after {
7417 |   content: '';
7418 |   position: absolute;
7419 |   top: 12%;
7420 |   right: -12%;
7421 |   width: min(520px, 48vw);
7422 |   aspect-ratio: 1;
7423 |   border: 1px solid rgba(37, 99, 235, 0.12);
7424 |   border-radius: 50%;
7425 |   background:
7426 |     radial-gradient(circle, rgba(37, 99, 235, 0.13), transparent 56%),
7427 |     repeating-conic-gradient(from 45deg, rgba(15, 23, 42, 0.045) 0deg 8deg, transparent 8deg 18deg);
7428 | }
7429 |
7430 | .landing-nav {
7431 |   padding: 24px 30px;
7432 | }
7433 |
7434 | .landing-brand {
7435 |   color: #0f172a;
7436 | }
7437 |
7438 | .landing-brand span {
7439 |   border-radius: 13px;
7440 |   background: linear-gradient(135deg, #0f172a, #2563eb 55%, #14b8a6);
7441 |   color: #ffffff;
7442 |   box-shadow: 0 18px 44px rgba(37, 99, 235, 0.2);
7443 | }
7444 |
7445 | .landing-hero-grid {
7446 |   grid-template-columns: minmax(360px, 0.86fr) minmax(520px, 1.14fr);
7447 |   gap: 28px;
7448 |   padding: 20px 28px 32px 54px;
7449 | }
7450 |
7451 | .landing-copy .eyebrow,
7452 | .student-hero-panel .eyebrow,
7453 | .student-profile-card .eyebrow,
7454 | .student-focused-panel .eyebrow,
7455 | .auth-shell .eyebrow {
7456 |   color: #3b82f6;
7457 |   letter-spacing: 0.04em;
7458 | }
7459 |
7460 | .landing-copy h1 {
7461 |   color: #0f172a;
7462 |   background:
7463 |     linear-gradient(94deg, #0f172a 0%, #1d4ed8 42%, #0f766e 78%, #0f172a 100%);
7464 |   background-clip: text;
7465 |   filter: none;
7466 |   text-wrap: balance;
7467 | }
7468 |
7469 | .landing-copy > p:not(.eyebrow) {
7470 |   color: #475569;
7471 |   font-size: 18px;
7472 | }
7473 |
7474 | .animated-word span:nth-child(1) {
7475 |   color: #2563eb;

```

```

7476 | }
7477 |
7478 | .animated-word span:nth-child(2) {
7479 |   color: #0f766e;
7480 | }
7481 |
7482 | .animated-word span:nth-child(3) {
7483 |   color: #be185d;
7484 | }
7485 |
7486 | .landing-feature-grid {
7487 |   gap: 10px;
7488 | }
7489 |
7490 | .landing-feature-grid div {
7491 |   border: 1px solid rgba(15, 23, 42, 0.08);
7492 |   border-radius: 18px;
7493 |   background: linear-gradient(180deg, rgba(255, 255, 0.92), rgba(248, 250, 252, 0.72));
7494 |   box-shadow: 0 18px 42px rgba(15, 23, 42, 0.08);
7495 | }
7496 |
7497 | .landing-feature-grid div:hover {
7498 |   border-color: rgba(37, 99, 235, 0.22);
7499 |   background: #ffffff;
7500 |   box-shadow: 0 26px 64px rgba(37, 99, 235, 0.12);
7501 |   transform: translateY(-4px);
7502 | }
7503 |
7504 | .landing-feature-grid strong {
7505 |   color: #0f172a;
7506 | }
7507 |
7508 | .landing-feature-grid span {
7509 |   color: #64748b;
7510 | }
7511 |
7512 | .landing-actions button,
7513 | .auth-shell .primary-button,
7514 | .student-action-card,
7515 | .student-soft-button,
7516 | .feedback-workspace > .primary-button {
7517 |   will-change: transform;
7518 | }
7519 |
7520 | .landing-actions button {
7521 |   min-height: 50px;
7522 |   border: 1px solid rgba(15, 23, 42, 0.08);
7523 |   border-radius: 15px;
7524 |   box-shadow: 0 18px 44px rgba(37, 99, 235, 0.14);
7525 | }
7526 |
7527 | .landing-actions .primary-button,
7528 | .auth-shell .primary-button,
7529 | .feedback-workspace > .primary-button {
7530 |   background: linear-gradient(135deg, #0f172a 0%, #2563eb 55%, #14b8a6 100%);
7531 | }
7532 |
7533 | .landing-actions .secondary-button {
7534 |   background: linear-gradient(135deg, #ffffff, #eef2ff);
7535 |   color: #172033;
7536 | }
7537 |
7538 | .landing-actions button:hover {
7539 |   box-shadow: 0 28px 70px rgba(37, 99, 235, 0.18);
7540 |   transform: translateY(-5px);
7541 | }
7542 |
7543 | .landing-proof span {
7544 |   border-color: rgba(15, 23, 42, 0.08);
7545 |   background: rgba(255, 255, 255, 0.72);
7546 |   color: #334155;
7547 |   box-shadow: 0 10px 24px rgba(15, 23, 42, 0.04);
7548 | }
7549 |
7550 | .landing-visual {
7551 |   min-height: calc(100vh - 154px);
7552 |   border: 1px solid rgba(15, 23, 42, 0.08);
7553 |   border-radius: 26px;
7554 |   background:
7555 |     radial-gradient(circle at 50% 40%, rgba(37, 99, 235, 0.16), transparent 34%),
7556 |     radial-gradient(circle at 82% 70%, rgba(20, 184, 166, 0.12), transparent 28%),
7557 |     linear-gradient(145deg, rgba(255, 255, 255, 0.94), rgba(241, 245, 249, 0.72));
7558 |   box-shadow:
7559 |     0 32px 90px rgba(15, 23, 42, 0.13),
7560 |     inset 0 1px 0 rgba(255, 255, 255, 0.95);
7561 | }
7562 |
7563 | .landing-visual::before {
7564 |   background:
7565 |     conic-gradient(from 120deg, transparent, rgba(37, 99, 235, 0.12), transparent, rgba(20, 184, 166, 0.12), transparent);
7566 |   opacity: 0.9;
7567 | }
7568 |
7569 | .landing-visual:hover {
7570 |   border-color: rgba(37, 99, 235, 0.2);
7571 |   box-shadow:
7572 |     0 42px 100px rgba(37, 99, 235, 0.16),
7573 |     inset 0 1px 0 rgba(255, 255, 255, 0.95);
7574 |   transform: translateY(-4px);
7575 | }
7576 |
7577 | .visual-status-strip div {
7578 |   border: 1px solid rgba(15, 23, 42, 0.08);
7579 |   border-radius: 16px;
7580 |   background: rgba(255, 255, 255, 0.78);
7581 |   box-shadow: 0 18px 42px rgba(15, 23, 42, 0.08);
7582 | }
7583 |
7584 | .visual-status-strip span {
7585 |   color: #64748b;

```

```

7586 | }
7587 |
7588 | .visual-status-strip strong {
7589 |   color: #0f172a;
7590 | }
7591 |
7592 | .auth-shell {
7593 |   padding-top: 86px;
7594 |   background:
7595 |     radial-gradient(circle at 16% 14%, rgba(37, 99, 235, 0.16), transparent 28%),
7596 |     radial-gradient(circle at 86% 20%, rgba(20, 184, 166, 0.14), transparent 30%),
7597 |     #f7f8fb;
7598 | }
7599 |
7600 | .auth-shell .auth-grid {
7601 |   width: min(920px, 100%);
7602 |   grid-template-columns: minmax(300px, 0.9fr) minmax(320px, 1fr);
7603 |   gap: 18px;
7604 | }
7605 |
7606 | .auth-shell .panel {
7607 |   border: 1px solid rgba(15, 23, 42, 0.08);
7608 |   border-radius: 24px;
7609 |   background: linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.88));
7610 |   color: #172033;
7611 |   box-shadow: 0 28px 84px rgba(15, 23, 42, 0.12);
7612 | }
7613 |
7614 | .auth-shell .panel h2,
7615 | .auth-shell .signal-row strong,
7616 | .auth-shell .signal-panel > div:nth-child(2) h2 {
7617 |   color: #0f172a;
7618 |   background: none;
7619 | }
7620 |
7621 | .auth-shell .auth-panel {
7622 |   padding: 28px;
7623 | }
7624 |
7625 | .auth-shell .auth-panel::before {
7626 |   inset: auto;
7627 |   top: -140px;
7628 |   right: -120px;
7629 |   width: 260px;
7630 |   height: 260px;
7631 |   border-radius: 50%;
7632 |   background: radial-gradient(circle, rgba(37, 99, 235, 0.14), transparent 64%);
7633 |   animation: none;
7634 | }
7635 |
7636 | .auth-shell .auth-back-button,
7637 | .student-form-wrap .auth-back-button {
7638 |   border: 1px solid rgba(15, 23, 42, 0.08);
7639 |   background: rgba(255, 255, 255, 0.82);
7640 |   color: #172033;
7641 |   box-shadow: 0 16px 40px rgba(15, 23, 42, 0.08);
7642 |   border-radius: 0px !important;
7643 | }
7644 |
7645 | .auth-shell .auth-back-button:hover,
7646 | .student-form-wrap .auth-back-button:hover {
7647 |   border-color: rgba(37, 99, 235, 0.22);
7648 |   color: #1d4ed8;
7649 |   box-shadow: 0 22px 58px rgba(37, 99, 235, 0.12);
7650 | }
7651 |
7652 | .auth-shell .panel-title svg {
7653 |   color: #2563eb;
7654 | }
7655 |
7656 | .auth-shell label {
7657 |   color: #334155;
7658 | }
7659 |
7660 | .auth-shell input,
7661 | .auth-shell select,
7662 | .auth-shell textarea {
7663 |   border-color: rgba(15, 23, 42, 0.12);
7664 |   border-radius: 15px;
7665 |   background: #ffffff;
7666 |   color: #172033;
7667 | }
7668 |
7669 | .auth-shell input:focus,
7670 | .auth-shell select:focus,
7671 | .auth-shell textarea:focus {
7672 |   border-color: rgba(37, 99, 235, 0.34);
7673 |   background: #ffffff;
7674 |   box-shadow: 0 0 4px rgba(37, 99, 235, 0.1);
7675 | }
7676 |
7677 | .auth-shell .primary-button {
7678 |   border-radius: 15px;
7679 |   box-shadow: 0 18px 44px rgba(37, 99, 235, 0.18);
7680 | }
7681 |
7682 | .auth-shell .primary-button:hover {
7683 |   box-shadow: 0 28px 68px rgba(37, 99, 235, 0.18);
7684 |   transform: translateY(-4px);
7685 | }
7686 |
7687 | .login-orb {
7688 |   border: 1px solid rgba(15, 23, 42, 0.08);
7689 |   border-radius: 26px;
7690 |   background:
7691 |     radial-gradient(circle at 28% 24%, rgba(255, 255, 255, 0.7), transparent 28%),
7692 |     linear-gradient(135deg, #0f172a, #2563eb 58%, #14b8a6);
7693 |   color: #ffffff;
7694 |   box-shadow: 0 26px 70px rgba(37, 99, 235, 0.18);
7695 | }

```

```

7696 |
7697 | .auth-shell .signal-row {
7698 |   border: 1px solid rgba(15, 23, 42, 0.08);
7699 |   border-radius: 17px;
7700 |   background: rgba(255, 255, 255, 0.74);
7701 |   color: #172033;
7702 |   box-shadow: 0 14px 34px rgba(15, 23, 42, 0.06);
7703 | }
7704 |
7705 | .auth-shell .signal-row:hover {
7706 |   border-color: rgba(37, 99, 235, 0.18);
7707 |   background: #ffffff;
7708 |   box-shadow: 0 20px 50px rgba(37, 99, 235, 0.1);
7709 |   transform: translateX(4px);
7710 | }
7711 |
7712 | .auth-shell .signal-row svg {
7713 |   color: #2563eb;
7714 | }
7715 |
7716 | .auth-shell .signal-row span {
7717 |   color: #64748b;
7718 | }
7719 |
7720 | .team-eureka-badge {
7721 |   border-color: rgba(22, 163, 74, 0.22);
7722 |   background: rgba(255, 255, 255, 0.86);
7723 |   color: #166534;
7724 |   box-shadow: 0 16px 42px rgba(22, 163, 74, 0.12);
7725 | }
7726 |
7727 | .workspace:has(.student-home),
7728 | .workspace:has(.student-form-wrap) {
7729 |   width: min(1240px, 100%);
7730 | }
7731 |
7732 | .workspace:has(.student-home) .topbar,
7733 | .workspace:has(.student-form-wrap) .topbar {
7734 |   position: sticky;
7735 |   top: 14px;
7736 |   z-index: 20;
7737 |   margin-bottom: 22px;
7738 |   border: 1px solid rgba(15, 23, 42, 0.08);
7739 |   border-radius: 20px;
7740 |   padding: 12px 14px;
7741 |   background: rgba(255, 255, 255, 0.78);
7742 |   box-shadow: 0 18px 48px rgba(15, 23, 42, 0.08);
7743 |   backdrop-filter: blur(18px);
7744 | }
7745 |
7746 | .workspace:has(.student-home) .dashboard-brand > span,
7747 | .workspace:has(.student-form-wrap) .dashboard-brand > span {
7748 |   border-radius: 13px;
7749 |   background: linear-gradient(135deg, #0f172a, #2563eb 55%, #14b8a6);
7750 | }
7751 |
7752 | .workspace:has(.student-home) .icon-button,
7753 | .workspace:has(.student-form-wrap) .icon-button {
7754 |   border-color: rgba(15, 23, 42, 0.08);
7755 |   border-radius: 14px;
7756 |   background: #ffffff;
7757 |   box-shadow: 0 10px 24px rgba(15, 23, 42, 0.06);
7758 | }
7759 |
7760 | .student-home {
7761 |   grid-template-columns: minmax(290px, 352px) 1fr;
7762 |   gap: 20px;
7763 | }
7764 |
7765 | .student-profile-card,
7766 | .student-hero-panel,
7767 | .student-focused-panel,
7768 | .feedback-workspace {
7769 |   border: 1px solid rgba(15, 23, 42, 0.08);
7770 |   background: linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.9));
7771 |   color: #172033;
7772 |   box-shadow: 0 28px 76px rgba(15, 23, 42, 0.1);
7773 | }
7774 |
7775 | .student-profile-card {
7776 |   min-height: 600px;
7777 |   border-radius: 26px;
7778 |   padding: 24px;
7779 | }
7780 |
7781 | .student-profile-card h2,
7782 | .student-hero-panel h1,
7783 | .student-focused-panel h2,
7784 | .feedback-workspace h2,
7785 | .feedback-workspace .category-block h3,
7786 | .feedback-workspace .question-row > span {
7787 |   color: #0f172a;
7788 | }
7789 |
7790 | .student-avatar {
7791 |   width: 148px;
7792 |   height: 148px;
7793 |   border: 4px solid #ffffff;
7794 |   border-radius: 34px;
7795 |   background: linear-gradient(135deg, #e0f2fe, #eef2ff);
7796 |   box-shadow:
7797 |     0 24px 60px rgba(37, 99, 235, 0.16),
7798 |     0 0 0 1px rgba(15, 23, 42, 0.08);
7799 | }
7800 |
7801 | .student-avatar:hover {
7802 |   box-shadow:
7803 |     0 30px 76px rgba(37, 99, 235, 0.18),
7804 |     0 0 0 1px rgba(37, 99, 235, 0.18);
7805 | }

```



```

7806 |
7807 | .student-details div,
7808 | .locked-details-grid div,
7809 | .student-photo-editor {
7810 |   border-color: rgba(15, 23, 42, 0.08);
7811 |   border-radius: 18px;
7812 |   background: linear-gradient(180deg, rgba(255, 255, 255, 0.92), rgba(241, 245, 249, 0.78));
7813 | }
7814 |
7815 | .student-details div:hover {
7816 |   border-color: rgba(37, 99, 235, 0.2);
7817 |   background: #ffffff;
7818 |   box-shadow: 0 18px 42px rgba(37, 99, 235, 0.09);
7819 |   transform: translateX(3px);
7820 | }
7821 |
7822 | .student-details dt,
7823 | .locked-details-grid span {
7824 |   color: #64748b;
7825 | }
7826 |
7827 | .student-details dd,
7828 | .locked-details-grid strong {
7829 |   color: #172033;
7830 | }
7831 |
7832 | .student-soft-button,
7833 | .photo-upload-button {
7834 |   border-color: rgba(15, 23, 42, 0.08);
7835 |   border-radius: 15px;
7836 |   background: #ffffff;
7837 |   color: #1d4ed8;
7838 |   box-shadow: 0 14px 34px rgba(37, 99, 235, 0.08);
7839 | }
7840 |
7841 | .student-soft-button:hover,
7842 | .photo-upload-button:hover {
7843 |   border-color: rgba(37, 99, 235, 0.22);
7844 |   color: #0f172a;
7845 |   box-shadow: 0 22px 54px rgba(37, 99, 235, 0.12);
7846 |   transform: translateY(-3px);
7847 | }
7848 |
7849 | .student-hero-panel {
7850 |   position: relative;
7851 |   min-height: 600px;
7852 |   border-radius: 26px;
7853 |   background:
7854 |     radial-gradient(circle at 52% 10%, rgba(37, 99, 235, 0.1), transparent 30%),
7855 |     linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(241, 245, 249, 0.88));
7856 | }
7857 |
7858 | .student-hero-panel::before {
7859 |   content: '';
7860 |   position: absolute;
7861 |   inset: 18px;
7862 |   pointer-events: none;
7863 |   border-radius: 22px;
7864 |   background:
7865 |     linear-gradient(rgba(15, 23, 42, 0.035) 1px, transparent 1px),
7866 |     linear-gradient(90deg, rgba(15, 23, 42, 0.035) 1px, transparent 1px);
7867 |   background-size: 38px 38px;
7868 |   mask-image: linear-gradient(to bottom, rgba(0, 0, 0, 0.42), transparent 78%);
7869 | }
7870 |
7871 | .student-logo-stage {
7872 |   position: relative;
7873 |   z-index: 1;
7874 |   width: min(520px, 92%);
7875 |   min-height: 242px;
7876 |   border-color: rgba(15, 23, 42, 0.08);
7877 |   border-radius: 28px;
7878 |   background: linear-gradient(180deg, #ffffff, #f8fafc);
7879 |   box-shadow:
7880 |     0 28px 74px rgba(15, 23, 42, 0.1),
7881 |     inset 0 1px 0 rgba(255, 255, 255, 0.95);
7882 | }
7883 |
7884 | .student-logo-stage:hover {
7885 |   border-color: rgba(37, 99, 235, 0.2);
7886 |   box-shadow: 0 36px 90px rgba(37, 99, 235, 0.14);
7887 | }
7888 |
7889 | .student-hero-panel h1 {
7890 |   position: relative;
7891 |   z-index: 1;
7892 |   background: linear-gradient(94deg, #0f172a, #2563eb 56%, #0f766e);
7893 |   background-clip: text;
7894 | }
7895 |
7896 | .student-hero-panel > p:not(.eyebrow) {
7897 |   position: relative;
7898 |   z-index: 1;
7899 |   color: #475569;
7900 | }
7901 |
7902 | .student-action-grid {
7903 |   position: relative;
7904 |   z-index: 1;
7905 | }
7906 |
7907 | .student-action-card {
7908 |   min-height: 148px;
7909 |   border-color: rgba(15, 23, 42, 0.08);
7910 |   border-radius: 22px;
7911 | }
7912 |
7913 | .student-action-card:hover {
7914 |   transform: translateY(-6px);
7915 | }

```

```

7916 |
7917 | .primary-action {
7918 |   background: linear-gradient(135deg, #0f172a 0%, #2563eb 58%, #14b8a6 100%);
7919 | }
7920 |
7921 | .danger-action {
7922 |   background: linear-gradient(135deg, #450a0a 0%, #dc2626 54%, #f97316 100%);
7923 | }
7924 |
7925 | .student-form-wrap {
7926 |   padding-top: 62px;
7927 | }
7928 |
7929 | .student-focused-panel,
7930 | .feedback-workspace {
7931 |   border-radius: 26px;
7932 | }
7933 |
7934 | .student-focused-panel .panel-title svg,
7935 | .feedback-workspace .panel-title svg {
7936 |   color: #2563eb;
7937 | }
7938 |
7939 | .student-focused-panel label,
7940 | .feedback-workspace label {
7941 |   color: #334155;
7942 | }
7943 |
7944 | .student-focused-panel input,
7945 | .student-focused-panel textarea,
7946 | .student-details-editor input,
7947 | .feedback-comment {
7948 |   border-color: rgba(15, 23, 42, 0.12);
7949 |   background: #ffffff;
7950 |   color: #172033;
7951 | }
7952 |
7953 | .student-focused-panel input:focus,
7954 | .student-focused-panel textarea:focus,
7955 | .student-details-editor input:focus,
7956 | .feedback-comment:focus {
7957 |   border-color: rgba(37, 99, 235, 0.34);
7958 |   box-shadow: 0 0 0 4px rgba(37, 99, 235, 0.1);
7959 | }
7960 |
7961 | .locked-badge {
7962 |   border-color: rgba(37, 99, 235, 0.16);
7963 |   background: #eff6ff;
7964 |   color: #1d4ed8;
7965 | }
7966 |
7967 | .feedback-workspace {
7968 |   background:
7969 |     radial-gradient(circle at 20% 0%, rgba(37, 99, 235, 0.08), transparent 30%),
7970 |     linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.9));
7971 | }
7972 |
7973 | .feedback-intro {
7974 |   border-color: rgba(15, 23, 42, 0.08);
7975 |   border-radius: 20px;
7976 |   background: #ffffff;
7977 |   box-shadow: 0 16px 40px rgba(15, 23, 42, 0.06);
7978 | }
7979 |
7980 | .feedback-intro strong {
7981 |   color: #0f172a;
7982 | }
7983 |
7984 | .feedback-intro span,
7985 | .feedback-workspace .category-block p {
7986 |   color: #64748b;
7987 | }
7988 |
7989 | .feedback-scale span {
7990 |   border-color: rgba(15, 23, 42, 0.08);
7991 |   background: #f8fafc;
7992 |   color: #334155;
7993 | }
7994 |
7995 | .feedback-workspace .category-block {
7996 |   border-color: rgba(15, 23, 42, 0.08);
7997 |   border-radius: 22px;
7998 |   background: linear-gradient(180deg, #ffffff, #f8fafc);
7999 |   box-shadow: 0 18px 42px rgba(15, 23, 42, 0.06);
8000 | }
8001 |
8002 | .feedback-workspace .category-block:hover {
8003 |   border-color: rgba(37, 99, 235, 0.18);
8004 |   box-shadow: 0 24px 58px rgba(37, 99, 235, 0.1);
8005 | }
8006 |
8007 | .feedback-workspace .question-row {
8008 |   border-top-color: rgba(15, 23, 42, 0.08);
8009 | }
8010 |
8011 | .feedback-workspace .rating-grid button {
8012 |   border-color: rgba(15, 23, 42, 0.1);
8013 |   background: #ffffff;
8014 |   color: #334155;
8015 | }
8016 |
8017 | .feedback-workspace .rating-grid button:hover {
8018 |   border-color: rgba(37, 99, 235, 0.22);
8019 |   color: #0f172a;
8020 |   box-shadow: 0 16px 36px rgba(37, 99, 235, 0.1);
8021 | }
8022 |
8023 | .feedback-workspace .rating-grid .selected {
8024 |   border-color: rgba(37, 99, 235, 0.26);
8025 |   background: linear-gradient(135deg, #eff6ff, #ccfbf1);

```

```

8026 | color: #0f172a;
8027 | box-shadow: 0 16px 40px rgba(20, 184, 166, 0.12);
8028 | }
8029 |
8030 | .auth-shell .error-text,
8031 | .student-form-wrap .success-text,
8032 | .student-form-wrap .error-text,
8033 | .student-form-wrap .empty-state {
8034 |   border-radius: 16px;
8035 |   padding: 12px 14px;
8036 | }
8037 |
8038 | .student-form-wrap .success-text {
8039 |   background: #ecfdf5;
8040 |   color: #047857;
8041 | }
8042 |
8043 | .auth-shell .error-text,
8044 | .student-form-wrap .error-text {
8045 |   background: #fef2f2;
8046 |   color: #b91c1c;
8047 | }
8048 |
8049 | .student-form-wrap .empty-state {
8050 |   background: #f8fafc;
8051 |   color: #475569;
8052 | }
8053 |
8054 | .photo-preview-modal {
8055 |   border-color: rgba(15, 23, 42, 0.08);
8056 |   background: #ffffff;
8057 |   box-shadow: 0 34px 120px rgba(15, 23, 42, 0.28);
8058 | }
8059 |
8060 | .photo-preview-close {
8061 |   background: rgba(255, 255, 255, 0.86);
8062 |   color: #0f172a;
8063 | }
8064 |
8065 | @media (max-width: 1100px) {
8066 |   .landing-hero-grid {
8067 |     grid-template-columns: 1fr;
8068 |     padding: 24px;
8069 |   }
8070 |
8071 |   .landing-visual {
8072 |     min-height: 500px;
8073 |   }
8074 |
8075 |   .student-home {
8076 |     grid-template-columns: 1fr;
8077 |   }
8078 |
8079 |   .student-profile-card,
8080 |   .student-hero-panel {
8081 |     min-height: auto;
8082 |   }
8083 | }
8084 |
8085 | @media (max-width: 680px) {
8086 |   .public-shell {
8087 |     padding: 12px;
8088 |   }
8089 |
8090 |   .landing-page {
8091 |     min-height: auto;
8092 |     border-radius: 22px;
8093 |   }
8094 |
8095 |   .landing-copy h1 {
8096 |     font-size: 38px;
8097 |   }
8098 |
8099 |   .landing-feature-grid,
8100 |   .student-action-grid,
8101 |   .feedback-intro {
8102 |     grid-template-columns: 1fr;
8103 |   }
8104 |
8105 |   .landing-actions button,
8106 |   .student-action-card,
8107 |   .feedback-workspace > .primary-button {
8108 |     width: 100%;
8109 |   }
8110 |
8111 |   .landing-visual {
8112 |     min-height: 390px;
8113 |   }
8114 |
8115 |   .visual-status-strip {
8116 |     position: relative;
8117 |     right: auto;
8118 |     bottom: auto;
8119 |     left: auto;
8120 |     grid-template-columns: 1fr;
8121 |     padding: 0 14px 14px;
8122 |     margin-top: -20px;
8123 |   }
8124 |
8125 |   .auth-shell .auth-grid,
8126 |   .student-edit-grid,
8127 |   .locked-details-grid,
8128 |   .editable-details-grid,
8129 |   .feedback-workspace .rating-grid {
8130 |     grid-template-columns: 1fr;
8131 |   }
8132 |
8133 |   .student-logo-stage {
8134 |     min-height: 190px;
8135 |   }

```

```

8136 | }
8137 |
8138 | /* =====
8139 | PREMIUM LIGHT-THEMED DESIGN SYSTEM OVERRIDES (STUDENT PORTAL & AUTH PAGES)
8140 | ===== */
8141 |
8142 | /* 1. Public & Auth Shell (Ivory Dot-Grid Background) */
8143 | .public-shell {
8144 |   background-color: #faf9f6 !important;
8145 |   color: #050a18 !important;
8146 |   background-image: radial-gradient(rgba(15, 23, 42, 0.05) 1.5px, transparent 1.5px) !important;
8147 |   background-size: 28px 28px !important;
8148 | }
8149 |
8150 | .public-shell::before {
8151 |   content: '';
8152 |   position: absolute;
8153 |   inset: 0;
8154 |   background:
8155 |     radial-gradient(circle at 12% 15%, rgba(37, 99, 235, 0.06) 0%, transparent 45%),
8156 |     radial-gradient(circle at 88% 20%, rgba(6, 182, 212, 0.06) 0%, transparent 45%),
8157 |     radial-gradient(circle at 50% 80%, rgba(236, 72, 153, 0.04) 0%, transparent 50%) !important;
8158 |   pointer-events: none;
8159 |   z-index: 0;
8160 | }
8161 |
8162 | .auth-shell .panel {
8163 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8164 |   border-radius: 24px !important;
8165 |   background: rgba(255, 255, 255, 0.94) !important;
8166 |   backdrop-filter: blur(16px) !important;
8167 |   box-shadow:
8168 |     0 15px 40px rgba(15, 23, 42, 0.03),
8169 |     0 4px 15px rgba(15, 23, 42, 0.01) !important;
8170 |   color: #1e293b !important;
8171 | }
8172 |
8173 | .auth-shell .panel h2 {
8174 |   color: #050a18 !important;
8175 |   font-weight: 950 !important;
8176 |   font-size: 1.8rem !important;
8177 |   letter-spacing: -0.02em !important;
8178 | }
8179 |
8180 | .auth-shell .auth-panel::before {
8181 |   background: conic-gradient(from 140deg, transparent, rgba(37, 99, 235, 0.03), transparent, rgba(236, 72, 153, 0.02), transparent) !important;
8182 | }
8183 |
8184 | /* 2. Form Inputs, Labels & Placeholders in Auth Shell */
8185 | .auth-shell label {
8186 |   color: #475569 !important;
8187 |   font-weight: 700 !important;
8188 |   font-size: 0.92rem !important;
8189 | }
8190 |
8191 | .auth-shell input,
8192 | .auth-shell select,
8193 | .auth-shell textarea {
8194 |   border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
8195 |   background: #ffffff !important;
8196 |   color: #050a18 !important;
8197 |   border-radius: 12px !important;
8198 |   padding: 12px 16px !important;
8199 |   transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1) !important;
8200 | }
8201 |
8202 | .auth-shell input:focus,
8203 | .auth-shell select:focus,
8204 | .auth-shell textarea:focus {
8205 |   border-color: #2563eb !important;
8206 |   background: #ffffff !important;
8207 |   box-shadow: 0 0 4px rgba(37, 99, 235, 0.08) !important;
8208 |   transform: translateY(-1px) !important;
8209 | }
8210 |
8211 | /* 3. Auth Buttons */
8212 | .auth-shell .primary-button {
8213 |   min-height: 48px !important;
8214 |   border-radius: 12px !important;
8215 |   border: 1px solid #050a18 !important;
8216 |   background: linear-gradient(135deg, #050a18 0%, #1e293b 100%) !important;
8217 |   color: #ffffff !important;
8218 |   font-weight: 850 !important;
8219 |   box-shadow: 0 4px 15px rgba(5, 10, 24, 0.1) !important;
8220 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
8221 | }
8222 |
8223 | .auth-shell .primary-button:hover {
8224 |   background: linear-gradient(135deg, #2563eb 0%, #1e40af 100%) !important;
8225 |   border-color: #2563eb !important;
8226 |   box-shadow: 0 8px 25px rgba(37, 99, 235, 0.22) !important;
8227 |   transform: translateY(-2px) !important;
8228 | }
8229 |
8230 | .auth-shell .back-button {
8231 |   border: 1px solid rgba(15, 23, 42, 0.07) !important;
8232 |   background: rgba(255, 255, 255, 0.9) !important;
8233 |   color: #050a18 !important;
8234 |   font-weight: 800 !important;
8235 |   box-shadow: 0 4px 12px rgba(15, 23, 42, 0.02) !important;
8236 |   backdrop-filter: blur(10px) !important;
8237 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
8238 | }
8239 |
8240 | .auth-shell .back-button:hover {
8241 |   border-color: #2563eb !important;
8242 |   color: #2563eb !important;
8243 |   box-shadow: 0 8px 20px rgba(37, 99, 235, 0.08) !important;
8244 |   transform: translateY(-2px) !important;
8245 | }

```

```

8246 |
8247 | /* 4. Login Signal Panel Info Rows */
8248 | .auth-shell .signal-panel {
8249 |   padding: 34px !important;
8250 |   background: rgba(255, 255, 255, 0.6) !important;
8251 | }
8252 |
8253 | .auth-shell .signal-panel h2 {
8254 |   color: #050a18 !important;
8255 | }
8256 |
8257 | .auth-shell .signal-row svg {
8258 |   color: #2563eb !important;
8259 |   background: rgba(37, 99, 235, 0.06) !important;
8260 |   border-radius: 12px !important;
8261 |   padding: 10px !important;
8262 |   width: 44px !important;
8263 |   height: 44px !important;
8264 |   box-shadow: 0 4px 10px rgba(37, 99, 235, 0.02) !important;
8265 | }
8266 |
8267 | .auth-shell .signal-row strong {
8268 |   color: #050a18 !important;
8269 |   font-weight: 800 !important;
8270 | }
8271 |
8272 | .auth-shell .signal-row span {
8273 |   color: #64748b !important;
8274 | }
8275 |
8276 | .login-orb {
8277 |   background: linear-gradient(135deg, #2563eb, #06b6d4) !important;
8278 |   box-shadow: 0 10px 25px rgba(37, 99, 235, 0.14) !important;
8279 | }
8280 |
8281 | /* 5. Team Eureka Badge */
8282 | .team-eureka-badge {
8283 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8284 |   background: rgba(255, 255, 255, 0.9) !important;
8285 |   color: #475569 !important;
8286 |   box-shadow: 0 4px 15px rgba(15, 23, 42, 0.03) !important;
8287 | }
8288 |
8289 | /* 6. Student Dashboard Workspace (Opaque, Light, Elegant) */
8290 | .workspace:has(.student-home),
8291 | .workspace:has(.student-form-wrap) {
8292 |   background-color: transparent !important;
8293 |   box-shadow: none !important;
8294 |   border: none !important;
8295 | }
8296 |
8297 | .app-shell:has(.student-home),
8298 | .app-shell:has(.student-form-wrap) {
8299 |   background: linear-gradient(135deg, #eef2ff 0%, #faf9f6 45%, #f0fdf4 100%) !important;
8300 |   background-image:
8301 |     radial-gradient(at 0% 0%, rgba(99, 102, 241, 0.04) 0px, transparent 50%),
8302 |     radial-gradient(at 100% 100%, rgba(20, 184, 166, 0.04) 0px, transparent 50%),
8303 |     radial-gradient(rgba(15, 23, 42, 0.03) 1.5px, transparent 1.5px) !important;
8304 |   background-size: 100% 100%, 100% 100%, 28px 28px !important;
8305 | }
8306 |
8307 | .app-shell:has(.student-home) .topbar,
8308 | .app-shell:has(.student-form-wrap) .topbar {
8309 |   border-bottom: 1px solid rgba(15, 23, 42, 0.06) !important;
8310 |   background: rgba(250, 249, 246, 0.85) !important;
8311 |   backdrop-filter: blur(12px) !important;
8312 |   padding: 14px 24px !important;
8313 |   border-radius: 16px !important;
8314 |   box-shadow: 0 4px 20px rgba(15, 23, 42, 0.02) !important;
8315 |   margin-bottom: 28px !important;
8316 | }
8317 |
8318 | .app-shell:has(.student-home) .topbar h2,
8319 | .app-shell:has(.student-form-wrap) .topbar h2 {
8320 |   color: #050a18 !important;
8321 |   font-weight: 950 !important;
8322 | }
8323 |
8324 | .app-shell:has(.student-home) .topbar .icon-button,
8325 | .app-shell:has(.student-form-wrap) .topbar .icon-button {
8326 |   background: rgba(15, 23, 42, 0.03) !important;
8327 |   color: #050a18 !important;
8328 |   border: 1px solid rgba(15, 23, 42, 0.05) !important;
8329 |   border-radius: 10px !important;
8330 |   font-weight: 800 !important;
8331 |   transition: all 0.2s !important;
8332 | }
8333 |
8334 | .app-shell:has(.student-home) .topbar .icon-button:hover,
8335 | .app-shell:has(.student-form-wrap) .topbar .icon-button:hover {
8336 |   background: rgba(239, 68, 68, 0.08) !important;
8337 |   color: #ef4444 !important;
8338 |   border-color: rgba(239, 68, 68, 0.15) !important;
8339 | }
8340 |
8341 | /* 7. Student Profile Card (Left Aside) */
8342 | .student-profile-card {
8343 |   background: #ffffff !important;
8344 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8345 |   border-radius: 20px !important;
8346 |   padding: 24px !important;
8347 |   box-shadow: 0 10px 30px rgba(15, 23, 42, 0.02) !important;
8348 | }
8349 |
8350 | .student-profile-card h2 {
8351 |   color: #050a18 !important;
8352 |   font-weight: 900 !important;
8353 |   font-size: 1.35rem !important;
8354 | }
8355 |

```

```

8356 | .student-details div {
8357 |   border-bottom: 1px solid rgba(15, 23, 42, 0.04) !important;
8358 |   padding: 12px 0 !important;
8359 |   transition: background-color 0.2s !important;
8360 | }
8361 |
8362 | .student-details dt {
8363 |   color: #64748b !important;
8364 |   font-weight: 800 !important;
8365 |   font-size: 0.82rem !important;
8366 | }
8367 |
8368 | .student-details dd {
8369 |   color: #050a18 !important;
8370 |   font-weight: 750 !important;
8371 | }
8372 |
8373 | .student-soft-button {
8374 |   background: rgba(37, 99, 235, 0.05) !important;
8375 |   color: #2563eb !important;
8376 |   border: 1px solid rgba(37, 99, 235, 0.1) !important;
8377 |   border-radius: 10px !important;
8378 |   font-weight: 800 !important;
8379 |   min-height: 38px !important;
8380 |   transition: all 0.25s !important;
8381 | }
8382 |
8383 | .student-soft-button:hover {
8384 |   background: #2563eb !important;
8385 |   color: #ffffff !important;
8386 |   border-color: #2563eb !important;
8387 | }
8388 |
8389 | .student-avatar {
8390 |   border: 3px solid rgba(37, 99, 235, 0.15) !important;
8391 |   box-shadow: 0 4px 15px rgba(37, 99, 235, 0.1) !important;
8392 |   transition: transform 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275) !important;
8393 | }
8394 |
8395 | .student-avatar:hover {
8396 |   transform: scale(1.06) rotate(3deg) !important;
8397 |   border-color: #2563eb !important;
8398 | }
8399 |
8400 | /* 8. Student Hero Panel & Actions (Right Content) */
8401 | .student-hero-panel {
8402 |   background: #ffffff !important;
8403 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8404 |   border-radius: 24px !important;
8405 |   padding: 40px !important;
8406 |   box-shadow: 0 10px 30px rgba(15, 23, 42, 0.02) !important;
8407 | }
8408 |
8409 | .student-hero-panel h1 {
8410 |   color: #050a18 !important;
8411 |   font-weight: 950 !important;
8412 |   font-size: 2.3rem !important;
8413 |   letter-spacing: -0.02em !important;
8414 |   background: linear-gradient(135deg, #050a18 30%, #2563eb 100%) !important;
8415 |   -webkit-background-clip: text !important;
8416 |   -webkit-text-fill-color: transparent !important;
8417 | }
8418 |
8419 | .student-hero-panel > p:not(.eyebrow) {
8420 |   color: #475569 !important;
8421 |   font-size: 1.1rem !important;
8422 |   font-weight: 600 !important;
8423 |   line-height: 1.6 !important;
8424 | }
8425 |
8426 | .student-action-card {
8427 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8428 |   border-radius: 18px !important;
8429 |   padding: 24px !important;
8430 |   background: #ffffff !important;
8431 |   box-shadow: 0 4px 15px rgba(15, 23, 42, 0.01) !important;
8432 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
8433 | }
8434 |
8435 | .student-action-card strong {
8436 |   color: #050a18 !important;
8437 |   font-size: 1.15rem !important;
8438 |   font-weight: 900 !important;
8439 | }
8440 |
8441 | .student-action-card span {
8442 |   color: #64748b !important;
8443 | }
8444 |
8445 | .student-action-card svg {
8446 |   transition: transform 0.3s !important;
8447 | }
8448 |
8449 | .student-action-grid .primary-action {
8450 |   border-left: 4px solid #2563eb !important;
8451 | }
8452 |
8453 | .student-action-grid .primary-action svg {
8454 |   color: #2563eb !important;
8455 | }
8456 |
8457 | .student-action-grid .danger-action {
8458 |   border-left: 4px solid #ef4444 !important;
8459 | }
8460 |
8461 | .student-action-grid .danger-action svg {
8462 |   color: #ef4444 !important;
8463 | }
8464 |
8465 | .student-action-card:hover {

```

```

8466 | transform: translateY(-6px) !important;
8467 | box-shadow: 0 15px 35px rgba(15, 23, 42, 0.03) !important;
8468 | border-color: rgba(37, 99, 235, 0.2) !important;
8469 | }
8470 |
8471 | .student-action-card:hover svg {
8472 |   transform: scale(1.1) rotate(5deg) !important;
8473 | }
8474 |
8475 | /* 9. Student Grievance / Feedback Forms (student-form-wrap) */
8476 | .student-form-wrap {
8477 |   position: relative !important;
8478 | }
8479 |
8480 | .student-focused-panel {
8481 |   background: #ffffff !important;
8482 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8483 |   border-radius: 24px !important;
8484 |   padding: 36px !important;
8485 |   box-shadow: 0 10px 40px rgba(15, 23, 42, 0.02) !important;
8486 | }
8487 |
8488 | .check-row {
8489 |   color: #050a18 !important;
8490 |   font-weight: 800 !important;
8491 | }
8492 |
8493 | .student-focused-panel input,
8494 | .student-focused-panel textarea {
8495 |   border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
8496 |   background: #ffffff !important;
8497 |   color: #050a18 !important;
8498 |   border-radius: 12px !important;
8499 |   padding: 12px 16px !important;
8500 |   transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1) !important;
8501 | }
8502 |
8503 | .student-focused-panel input:focus,
8504 | .student-focused-panel textarea:focus {
8505 |   border-color: #2563eb !important;
8506 |   box-shadow: 0 0 4px rgba(37, 99, 235, 0.08) !important;
8507 |   transform: translateY(-1px) !important;
8508 | }
8509 |
8510 | /* 10. Feedback Workspace (Multiple Questions Form) */
8511 | .feedback-workspace {
8512 |   background: #ffffff !important;
8513 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
8514 |   border-radius: 24px !important;
8515 |   padding: 36px !important;
8516 |   box-shadow: 0 10px 40px rgba(15, 23, 42, 0.02) !important;
8517 | }
8518 |
8519 | .feedback-intro {
8520 |   background: rgba(37, 99, 235, 0.03) !important;
8521 |   border: 1px solid rgba(37, 99, 235, 0.08) !important;
8522 |   border-radius: 14px !important;
8523 |   padding: 20px !important;
8524 | }
8525 |
8526 | .feedback-intro strong {
8527 |   color: #2563eb !important;
8528 | }
8529 |
8530 | .feedback-intro span {
8531 |   color: #1e3a8a !important;
8532 |   font-weight: 600 !important;
8533 | }
8534 |
8535 | .category-block {
8536 |   border: 1px solid rgba(15, 23, 42, 0.05) !important;
8537 |   border-radius: 18px !important;
8538 |   padding: 24px !important;
8539 |   background: rgba(250, 249, 246, 0.5) !important;
8540 |   margin-bottom: 24px !important;
8541 | }
8542 |
8543 | .category-block h3 {
8544 |   color: #050a18 !important;
8545 |   font-weight: 950 !important;
8546 |   font-size: 1.25rem !important;
8547 | }
8548 |
8549 | .category-block p {
8550 |   color: #64748b !important;
8551 | }
8552 |
8553 | .question-row {
8554 |   border-top: 1px dashed rgba(15, 23, 42, 0.06) !important;
8555 |   padding: 20px 0 !important;
8556 | }
8557 |
8558 | .question-row > span {
8559 |   color: #050a18 !important;
8560 |   font-weight: 800 !important;
8561 |   font-size: 1.05rem !important;
8562 | }
8563 |
8564 | .rating-grid button {
8565 |   background: rgba(15, 23, 42, 0.02) !important;
8566 |   border: 1px solid rgba(15, 23, 42, 0.04) !important;
8567 |   color: #475569 !important;
8568 |   border-radius: 8px !important;
8569 |   font-weight: 800 !important;
8570 |   transition: all 0.2s !important;
8571 | }
8572 |
8573 | .rating-grid button:hover {
8574 |   background: rgba(15, 23, 42, 0.06) !important;
8575 |   color: #050a18 !important;

```

```

8576 | }
8577 |
8578 | .rating-grid .selected {
8579 |   background: #2563eb !important;
8580 |   color: #ffffff !important;
8581 |   border-color: #2563eb !important;
8582 |   box-shadow: 0 4px 10px rgba(37, 99, 235, 0.15) !important;
8583 | }
8584 |
8585 | .locked-badge {
8586 |   background: rgba(15, 23, 42, 0.05) !important;
8587 |   color: #475569 !important;
8588 |   font-weight: 800 !important;
8589 |   border-radius: 6px !important;
8590 |   padding: 4px 10px !important;
8591 | }
8592 |
8593 | .student-details-editor dl dt {
8594 |   color: #64748b !important;
8595 |   font-weight: 800 !important;
8596 | }
8597 |
8598 | .student-details-editor dl dd {
8599 |   color: #050a18 !important;
8600 |   font-weight: 750 !important;
8601 | }
8602 |
8603 | /* 11. Reveal-on-scroll integration for student views */
8604 | .student-profile-card,
8605 | .student-hero-panel,
8606 | .student-form-wrap,
8607 | .auth-grid {
8608 |   opacity: 1 !important;
8609 |   transform: none !important;
8610 |   animation: revealDirect 0.65s cubic-bezier(0.16, 1, 0.3, 1) both !important;
8611 | }
8612 |
8613 | @keyframes revealDirect {
8614 |   from {
8615 |     opacity: 0;
8616 |     transform: translateY(20px);
8617 |   }
8618 |   to {
8619 |     opacity: 1;
8620 |     transform: translateY(0);
8621 |   }
8622 | }
8623 |
8624 | /* =====
8625 | BRUTALIST SHARP TECH DESIGN SYSTEM OVERRIDES (SHARP EDGES, SPACE GROTESK)
8626 | ===== */
8627 |
8628 | /* Apply Space Grotesk globally to public and non-analysis dashboard pages */
8629 | .public-shell,
8630 | .public-shell *,
8631 | .app-shell:has(.student-home) *,
8632 | .app-shell:has(.student-form-wrap) *,
8633 | .app-shell:has(.admin-home-panel) *,
8634 | .app-shell:has(.admin-stack):not(:has(.analysis-report-page)) * {
8635 |   font-family: 'Space Grotesk', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, sans-serif !important;
8636 | }
8637 |
8638 | /* Brutalist Sharp Edges - Force No Rounded Corners */
8639 | .public-shell .panel,
8640 | .app-shell:has(.student-home) .panel,
8641 | .app-shell:has(.student-form-wrap) .panel,
8642 | .app-shell:has(.admin-home-panel) .panel,
8643 | .app-shell:has(.admin-stack):not(:has(.analysis-report-page)) .panel,
8644 | .student-profile-card,
8645 | .student-hero-panel,
8646 | .student-action-card,
8647 | .category-block,
8648 | .feedback-intro,
8649 | .admin-home-panel,
8650 | .admin-module-card,
8651 | .admin-validation-panel,
8652 | .admin-validation-result,
8653 | .admin-validation-result > div,
8654 | .validation-case-list article,
8655 | .builder-category,
8656 | .topbar,
8657 | .topbar .icon-button,
8658 | .icon-button,
8659 | .primary-button,
8660 | .secondary-button,
8661 | .student-soft-button,
8662 | .auth-back-button,
8663 | .rating-grid button,
8664 | .team-eureka-badge,
8665 | .locked-badge,
8666 | .student-avatar,
8667 | .student-avatar img,
8668 | .topbar .brand-logo-wrapper,
8669 | .topbar .brand-logo-img,
8670 | .student-logo-stage,
8671 | .student-logo-stage img,
8672 | .public-shell input,
8673 | .public-shell textarea,
8674 | .public-shell select,
8675 | .public-shell button,
8676 | .app-shell input,
8677 | .app-shell textarea,
8678 | .app-shell select,
8679 | .app-shell button {
8680 |   border-radius: 0px !important;
8681 | }
8682 |
8683 | /* Custom Brand Logo Styles inside Dashboard Topbar */
8684 | .topbar .brand-logo-wrapper {
8685 |   display: flex !important;

```



```

8686 | align-items: center !important;
8687 | justify-content: center !important;
8688 | width: 32px !important;
8689 | height: 32px !important;
8690 | background: transparent !important;
8691 | border: none !important;
8692 | margin-right: 8px !important;
8693 | }
8694 |
8695 | .topbar .brand-logo-img {
8696 |   width: 100% !important;
8697 |   height: 100% !important;
8698 |   object-fit: contain !important;
8699 | }
8700 |
8701 | .student-logo-stage {
8702 |   background: transparent !important;
8703 |   border: none !important;
8704 |   box-shadow: none !important;
8705 |   width: 80px !important;
8706 |   height: 80px !important;
8707 |   margin-bottom: 16px !important;
8708 | }
8709 |
8710 | .student-hero-logo {
8711 |   width: 100% !important;
8712 |   height: 100% !important;
8713 |   object-fit: contain !important;
8714 | }
8715 |
8716 | .analytics-brand-logo {
8717 |   width: 28px !important;
8718 |   height: 28px !important;
8719 |   object-fit: contain !important;
8720 |   margin-right: 6px !important;
8721 | }
8722 |
8723 | /* Student Home Dashboard Layout Swap (Profile on right, Hero on left) */
8724 | .app-shell:has(.student-home) .workspace {
8725 |   max-width: 1280px !important;
8726 |   margin: 0 auto !important;
8727 |   padding: 0 24px !important;
8728 | }
8729 |
8730 | .student-home {
8731 |   display: flex !important;
8732 |   flex-direction: row-reverse !important;
8733 |   align-items: flex-start !important;
8734 |   gap: 28px !important;
8735 |   margin-top: 12px !important;
8736 | }
8737 |
8738 | .student-profile-card {
8739 |   width: 340px !important;
8740 |   flex-shrink: 0 !important;
8741 | }
8742 |
8743 | .student-hero-panel {
8744 |   flex-grow: 1 !important;
8745 | }
8746 |
8747 | /* Stacking Student Actions Vertically */
8748 | .student-action-grid {
8749 |   display: flex !important;
8750 |   flex-direction: column !important;
8751 |   gap: 20px !important;
8752 |   margin-top: 32px !important;
8753 | }
8754 |
8755 | .student-action-card {
8756 |   display: grid !important;
8757 |   grid-template-columns: auto 1fr !important;
8758 |   align-items: center !important;
8759 |   gap: 20px !important;
8760 |   text-align: left !important;
8761 |   width: 100% !important;
8762 |   min-height: auto !important;
8763 |   padding: 24px !important;
8764 |   border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
8765 |   background: #ffffff !important;
8766 |   box-shadow: 0 4px 12px rgba(15, 23, 42, 0.01) !important;
8767 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
8768 | }
8769 |
8770 | .student-action-card svg {
8771 |   grid-row: span 2 !important;
8772 |   width: 32px !important;
8773 |   height: 32px !important;
8774 | }
8775 |
8776 | .student-action-card strong {
8777 |   font-size: 1.25rem !important;
8778 |   margin: 0 !important;
8779 | }
8780 |
8781 | .student-action-card span {
8782 |   font-size: 0.95rem !important;
8783 |   margin: 0 !important;
8784 | }
8785 |
8786 | /* Static Center-Aligned developed by Eureka Badge */
8787 | .team-eureka-badge {
8788 |   position: static !important;
8789 |   transform: none !important;
8790 |   margin: 40px auto 20px auto !important;
8791 |   display: flex !important;
8792 |   align-items: center !important;
8793 |   justify-content: center !important;
8794 |   gap: 8px !important;
8795 |   width: max-content !important;

```

```

8796 | min-height: auto !important;
8797 | padding: 8px 16px !important;
8798 | background: rgba(255, 255, 255, 0.94) !important;
8799 | border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
8800 | color: #475569 !important;
8801 | font-weight: 700 !important;
8802 | font-size: 0.85rem !important;
8803 | letter-spacing: 0.05em !important;
8804 | text-transform: uppercase !important;
8805 | }
8806 |
8807 | .team-eureka-badge svg {
8808 |   color: #2563eb !important;
8809 | }
8810 |
8811 | /* =====
8812 |   ADMIN LIGHT-THEME OVERRIDES (EXCLUDING DETAILED COCKPIT REPORT)
8813 |   ===== */
8814 |
8815 | /* Admin shell layout dot-grid background */
8816 | .app-shell:has(.admin-home-panel),
8817 | .app-shell:has(.admin-stack):not(:has(.analysis-report-page)) {
8818 |   background-color: #faf9f6 !important;
8819 |   background-image: radial-gradient(rgba(37, 99, 235, 0.05) 1.5px, transparent 1.5px) !important;
8820 |   background-size: 24px 24px !important;
8821 | }
8822 |
8823 | .workspace:has(.admin-home-panel),
8824 | .workspace:has(.admin-stack):not(:has(.analysis-report-page)) {
8825 |   background-color: transparent !important;
8826 |   box-shadow: none !important;
8827 |   border: none !important;
8828 |   max-width: 1240px !important;
8829 |   margin: 0 auto !important;
8830 |   padding: 0 24px !important;
8831 | }
8832 |
8833 | /* =====
8834 |   ADMIN TOPBAR & PROFILE DROPDOWN
8835 |   ===== */
8836 | .admin-topbar {
8837 |   display: flex !important;
8838 |   justify-content: space-between !important;
8839 |   align-items: center !important;
8840 |   padding: 14px 24px !important;
8841 |   border-bottom: 1.5px solid #0f172a !important;
8842 |   background: rgba(255, 255, 255, 0.9) !important;
8843 |   backdrop-filter: blur(12px) !important;
8844 |   margin-bottom: 32px !important;
8845 |   box-shadow: 0 4px 30px rgba(15, 23, 42, 0.02) !important;
8846 |   position: sticky !important;
8847 |   top: 0 !important;
8848 |   z-index: 900 !important;
8849 | }
8850 |
8851 | .admin-topbar-left {
8852 |   display: flex !important;
8853 |   align-items: center !important;
8854 | }
8855 |
8856 | .admin-console-label {
8857 |   font-size: 10px !important;
8858 |   font-weight: 850 !important;
8859 |   letter-spacing: 0.05em !important;
8860 |   text-transform: uppercase !important;
8861 |   background: rgba(37, 99, 235, 0.08) !important;
8862 |   color: #2563eb !important;
8863 |   padding: 3px 8px !important;
8864 |   border: 1px solid rgba(37, 99, 235, 0.15) !important;
8865 |   margin-left: 10px !important;
8866 |   display: inline-block !important;
8867 | }
8868 |
8869 | .admin-topbar-right {
8870 |   position: relative !important;
8871 | }
8872 |
8873 | .admin-profile-trigger {
8874 |   display: flex !important;
8875 |   align-items: center !important;
8876 |   gap: 10px !important;
8877 |   background: #ffffff !important;
8878 |   border: 1.5px solid #0f172a !important;
8879 |   padding: 6px 14px !important;
8880 |   cursor: pointer !important;
8881 |   transition: all 0.2s cubic-bezier(0.16, 1, 0.3, 1) !important;
8882 | }
8883 |
8884 | .admin-profile-trigger:hover {
8885 |   background: #f1f5f9 !important;
8886 |   transform: translateY(-1px) !important;
8887 |   box-shadow: 0 4px 12px rgba(15, 23, 42, 0.05) !important;
8888 | }
8889 |
8890 | .admin-avatar-initials {
8891 |   display: flex !important;
8892 |   align-items: center !important;
8893 |   justify-content: center !important;
8894 |   width: 28px !important;
8895 |   height: 28px !important;
8896 |   background: #2563eb !important;
8897 |   color: #ffffff !important;
8898 |   font-weight: 900 !important;
8899 |   font-size: 13px !important;
8900 |   border: 1px solid #0f172a !important;
8901 | }
8902 |
8903 | .admin-profile-name {
8904 |   font-size: 14px !important;
8905 |   font-weight: 800 !important;

```

```

8906 | color: #0f172a !important;
8907 | }
8908 |
8909 | .profile-chevron {
8910 |   color: #475569 !important;
8911 |   transition: transform 0.2s ease !important;
8912 | }
8913 |
8914 | .profile-chevron.rotated {
8915 |   transform: rotate(180deg) !important;
8916 | }
8917 |
8918 | /* Dropdown click outside overlay */
8919 | .dropdown-click-overlay {
8920 |   position: fixed !important;
8921 |   top: 0 !important;
8922 |   left: 0 !important;
8923 |   right: 0 !important;
8924 |   bottom: 0 !important;
8925 |   background: transparent !important;
8926 |   z-index: 950 !important;
8927 | }
8928 |
8929 | .admin-profile-dropdown {
8930 |   position: absolute !important;
8931 |   top: calc(100% + 8px) !important;
8932 |   right: 0 !important;
8933 |   width: 260px !important;
8934 |   background: rgba(255, 255, 255, 0.98) !important;
8935 |   border: 1.5px solid #0f172a !important;
8936 |   box-shadow: 0 10px 30px rgba(15, 23, 42, 0.08) !important;
8937 |   padding: 16px !important;
8938 |   display: flex !important;
8939 |   flex-direction: column !important;
8940 |   gap: 12px !important;
8941 |   z-index: 960 !important;
8942 |   transform-origin: top right !important;
8943 |   animation: dropdownFadeIn 0.2s cubic-bezier(0.16, 1, 0.3, 1) forwards !important;
8944 | }
8945 |
8946 | @keyframes dropdownFadeIn {
8947 |   from {
8948 |     opacity: 0;
8949 |     transform: scale(0.95) translateY(-5px);
8950 |   }
8951 |   to {
8952 |     opacity: 1;
8953 |     transform: scale(1) translateY(0);
8954 |   }
8955 | }
8956 |
8957 | .dropdown-user-details {
8958 |   display: flex !important;
8959 |   flex-direction: column !important;
8960 |   gap: 2px !important;
8961 | }
8962 |
8963 | .dropdown-user-details strong {
8964 |   font-size: 15px !important;
8965 |   font-weight: 900 !important;
8966 |   color: #0f172a !important;
8967 | }
8968 |
8969 | .dropdown-user-details span {
8970 |   font-size: 12px !important;
8971 |   color: #64748b !important;
8972 |   font-weight: 600 !important;
8973 | }
8974 |
8975 | .dropdown-divider {
8976 |   height: 1px !important;
8977 |   background: rgba(15, 23, 42, 0.08) !important;
8978 |   margin: 4px 0 !important;
8979 | }
8980 |
8981 | .dropdown-logout-btn {
8982 |   display: flex !important;
8983 |   align-items: center !important;
8984 |   justify-content: center !important;
8985 |   gap: 8px !important;
8986 |   width: 100% !important;
8987 |   padding: 10px !important;
8988 |   background: rgba(239, 68, 68, 0.05) !important;
8989 |   border: 1.5px solid rgba(239, 68, 68, 0.2) !important;
8990 |   color: #ef4444 !important;
8991 |   font-size: 13px !important;
8992 |   font-weight: 900 !important;
8993 |   cursor: pointer !important;
8994 |   transition: all 0.2s !important;
8995 |   text-align: left !important;
8996 | }
8997 |
8998 | .dropdown-logout-btn:hover {
8999 |   background: #ef4444 !important;
9000 |   border-color: #ef4444 !important;
9001 |   color: #ffffff !important;
9002 | }
9003 |
9004 | /* =====
9005 |    HERO / TITLE AREA
9006 |    ===== */
9007 | .admin-home-panel {
9008 |   display: flex !important;
9009 |   flex-direction: column !important;
9010 |   gap: 28px !important;
9011 |   min-height: auto !important;
9012 |   border: 1.5px solid #0f172a !important;
9013 |   padding: 40px !important;
9014 |   background: #ffffff !important;
9015 |   color: #0f172a !important;

```

```

9016 | box-shadow: 0 12px 40px rgba(15, 23, 42, 0.03) !important;
9017 | position: relative !important;
9018 | overflow: hidden !important;
9019 | }
9020 |
9021 | .admin-grid-lines {
9022 |   position: absolute !important;
9023 |   top: 0 !important;
9024 |   left: 0 !important;
9025 |   right: 0 !important;
9026 |   bottom: 0 !important;
9027 |   background-image:
9028 |     linear-gradient(rgba(37, 99, 235, 0.02) 1px, transparent 1px),
9029 |     linear-gradient(90deg, rgba(37, 99, 235, 0.02) 1px, transparent 1px) !important;
9030 |   background-size: 40px 40px !important;
9031 |   pointer-events: none !important;
9032 |   z-index: 1 !important;
9033 | }
9034 |
9035 | .admin-gradient-orb {
9036 |   position: absolute !important;
9037 |   width: 300px !important;
9038 |   height: 300px !important;
9039 |   top: -150px !important;
9040 |   right: -150px !important;
9041 |   background: radial-gradient(circle, rgba(37, 99, 235, 0.08) 0%, rgba(6, 182, 212, 0.05) 50%, transparent 100%) !important;
9042 |   border-radius: 50% !important;
9043 |   pointer-events: none !important;
9044 |   z-index: 1 !important;
9045 |   filter: blur(40px) !important;
9046 | }
9047 |
9048 | .admin-home-header {
9049 |   position: relative !important;
9050 |   z-index: 5 !important;
9051 |   display: flex !important;
9052 |   flex-direction: column !important;
9053 |   gap: 12px !important;
9054 |   max-width: 900px !important;
9055 |   animation: adminHeroReveal 0.6s cubic-bezier(0.16, 1, 0.3, 1) both !important;
9056 | }
9057 |
9058 | @keyframes adminHeroReveal {
9059 |   from {
9060 |     opacity: 0;
9061 |     transform: translateY(15px);
9062 |   }
9063 |   to {
9064 |     opacity: 1;
9065 |     transform: translateY(0);
9066 |   }
9067 | }
9068 |
9069 | .admin-home-eyebrow {
9070 |   margin: 0 !important;
9071 |   font-size: 12px !important;
9072 |   font-weight: 950 !important;
9073 |   letter-spacing: 0.15em !important;
9074 |   text-transform: uppercase !important;
9075 |   color: #2563eb !important;
9076 | }
9077 |
9078 | .admin-home-header h1 {
9079 |   margin: 0 !important;
9080 |   color: #0f172a !important;
9081 |   font-size: clamp(2.2rem, 4.5vw, 3.2rem) !important;
9082 |   font-weight: 950 !important;
9083 |   letter-spacing: -0.03em !important;
9084 |   line-height: 1.1 !important;
9085 | }
9086 |
9087 | .admin-home-header h1 span {
9088 |   background: linear-gradient(135deg, #2563eb 0%, #06b6d4 100%) !important;
9089 |   -webkit-background-clip: text !important;
9090 |   -webkit-text-fill-color: transparent !important;
9091 | }
9092 |
9093 | .admin-home-subheading {
9094 |   margin: 0 !important;
9095 |   color: #475569 !important;
9096 |   font-size: 1.05rem !important;
9097 |   line-height: 1.6 !important;
9098 |   font-weight: 600 !important;
9099 | }
9100 |
9101 | .admin-home-notice {
9102 |   margin: 8px 0 0 !important;
9103 |   background: #fffbeb !important;
9104 |   border: 1.5px solid rgba(245, 158, 11, 0.3) !important;
9105 |   color: #d97706 !important;
9106 |   padding: 12px 18px !important;
9107 |   font-weight: 800 !important;
9108 |   font-size: 13px !important;
9109 |   width: fit-content !important;
9110 | }
9111 |
9112 | /* =====
9113 |   DATASET SELECTOR REDESIGN
9114 |   ===== */
9115 | .admin-dataset-panel {
9116 |   position: relative !important;
9117 |   z-index: 5 !important;
9118 |   display: flex !important;
9119 |   justify-content: space-between !important;
9120 |   align-items: center !important;
9121 |   background: #ffffff !important;
9122 |   border: 1.5px solid #0f172a !important;
9123 |   padding: 20px 24px !important;
9124 |   box-shadow: 0 4px 15px rgba(15, 23, 42, 0.01) !important;
9125 |   opacity: 1 !important;

```

```

9126 | animation: adminHeroReveal 0.6s cubic-bezier(0.16, 1, 0.3, 1) 0.15s both !important;
9127 | }
9128 |
9129 | .dataset-panel-left {
9130 |   display: flex !important;
9131 |   align-items: center !important;
9132 |   gap: 16px !important;
9133 | }
9134 |
9135 | .dataset-active-info {
9136 |   display: flex !important;
9137 |   flex-direction: column !important;
9138 |   gap: 2px !important;
9139 | }
9140 |
9141 | .dataset-label {
9142 |   font-size: 10px !important;
9143 |   font-weight: 950 !important;
9144 |   letter-spacing: 0.08em !important;
9145 |   color: #64748b !important;
9146 |   text-transform: uppercase !important;
9147 | }
9148 |
9149 | .dataset-value-name {
9150 |   font-size: 16px !important;
9151 |   font-weight: 900 !important;
9152 |   color: #0f172a !important;
9153 | }
9154 |
9155 | .dataset-status-badge {
9156 |   font-size: 11px !important;
9157 |   font-weight: 900 !important;
9158 |   padding: 4px 8px !important;
9159 |   border: 1px solid #0f172a !important;
9160 |   text-transform: uppercase !important;
9161 | }
9162 |
9163 | .dataset-status-badge.live {
9164 |   background: rgba(34, 197, 94, 0.08) !important;
9165 |   color: #22c55e !important;
9166 |   border-color: rgba(34, 197, 94, 0.2) !important;
9167 | }
9168 |
9169 | .dataset-status-badge.stored {
9170 |   background: rgba(37, 99, 235, 0.08) !important;
9171 |   color: #2563eb !important;
9172 |   border-color: rgba(37, 99, 235, 0.2) !important;
9173 | }
9174 |
9175 | .dataset-panel-right {
9176 |   display: flex !important;
9177 |   gap: 12px !important;
9178 | }
9179 |
9180 | .btn-admin-choose-dataset {
9181 |   background: #0f172a !important;
9182 |   color: #ffffff !important;
9183 |   border: 1.5px solid #0f172a !important;
9184 |   padding: 10px 18px !important;
9185 |   font-size: 13px !important;
9186 |   font-weight: 900 !important;
9187 |   cursor: pointer !important;
9188 |   transition: all 0.2s !important;
9189 | }
9190 |
9191 | .btn-admin-choose-dataset:hover {
9192 |   background: #2563eb !important;
9193 |   border-color: #2563eb !important;
9194 |   transform: translateY(-1px) !important;
9195 | }
9196 |
9197 | .btn-admin-choose-dataset:active {
9198 |   transform: translateY(1px) !important;
9199 | }
9200 |
9201 | .btn-admin-refresh-datasets {
9202 |   display: flex !important;
9203 |   align-items: center !important;
9204 |   gap: 8px !important;
9205 |   background: #ffffff !important;
9206 |   color: #0f172a !important;
9207 |   border: 1.5px solid #0f172a !important;
9208 |   padding: 10px 18px !important;
9209 |   font-size: 13px !important;
9210 |   font-weight: 900 !important;
9211 |   cursor: pointer !important;
9212 |   transition: all 0.2s !important;
9213 | }
9214 |
9215 | .btn-admin-refresh-datasets:hover {
9216 |   background: #f8fafc !important;
9217 |   transform: translateY(-1px) !important;
9218 | }
9219 |
9220 | .btn-admin-refresh-datasets:active {
9221 |   transform: translateY(1px) !important;
9222 | }
9223 |
9224 | .btn-admin-refresh-datasets.spinning svg {
9225 |   animation: refreshSpin 1s linear infinite !important;
9226 | }
9227 |
9228 | @keyframes refreshSpin {
9229 |   from {
9230 |     transform: rotate(0deg);
9231 |   }
9232 |   to {
9233 |     transform: rotate(360deg);
9234 |   }
9235 | }

```

```

9236 |
9237 | /* Side Drawer Dataset Selection */
9238 | .dataset-drawer-overlay {
9239 |   position: fixed !important;
9240 |   top: 0 !important;
9241 |   left: 0 !important;
9242 |   right: 0 !important;
9243 |   bottom: 0 !important;
9244 |   background: rgba(15, 23, 42, 0.15) !important;
9245 |   backdrop-filter: blur(4px) !important;
9246 |   z-index: 1000 !important;
9247 |   animation: overlayFadeIn 0.25s ease forwards !important;
9248 | }
9249 |
9250 | @keyframes overlayFadeIn {
9251 |   from { opacity: 0; }
9252 |   to { opacity: 1; }
9253 | }
9254 |
9255 | .dataset-drawer {
9256 |   position: fixed !important;
9257 |   top: 0 !important;
9258 |   right: 0 !important;
9259 |   width: 440px !important;
9260 |   max-width: 100% !important;
9261 |   height: 100vh !important;
9262 |   background: #ffffff !important;
9263 |   border-left: 2px solid #0f172a !important;
9264 |   box-shadow: -10px 0 40px rgba(15, 23, 42, 0.08) !important;
9265 |   z-index: 1010 !important;
9266 |   display: flex !important;
9267 |   flex-direction: column !important;
9268 |   transform: translateX(100%) !important;
9269 |   transition: transform 0.35s cubic-bezier(0.16, 1, 0.3, 1) !important;
9270 | }
9271 |
9272 | .dataset-drawer.open {
9273 |   transform: translateX(0) !important;
9274 | }
9275 |
9276 | .drawer-header {
9277 |   display: flex !important;
9278 |   justify-content: space-between !important;
9279 |   align-items: center !important;
9280 |   padding: 24px !important;
9281 |   border-bottom: 1.5px solid #0f172a !important;
9282 | }
9283 |
9284 | .drawer-header h3 {
9285 |   margin: 0 !important;
9286 |   font-size: 18px !important;
9287 |   font-weight: 950 !important;
9288 |   color: #0f172a !important;
9289 | }
9290 |
9291 | .drawer-close-btn {
9292 |   display: flex !important;
9293 |   align-items: center !important;
9294 |   justify-content: center !important;
9295 |   width: 32px !important;
9296 |   height: 32px !important;
9297 |   background: transparent !important;
9298 |   border: 1.5px solid #0f172a !important;
9299 |   color: #0f172a !important;
9300 |   cursor: pointer !important;
9301 |   transition: all 0.2s !important;
9302 | }
9303 |
9304 | .drawer-close-btn:hover {
9305 |   background: #ef4444 !important;
9306 |   border-color: #ef4444 !important;
9307 |   color: #ffffff !important;
9308 | }
9309 |
9310 | .drawer-dataset-list {
9311 |   flex-grow: 1 !important;
9312 |   overflow-y: auto !important;
9313 |   padding: 24px !important;
9314 |   display: flex !important;
9315 |   flex-direction: column !important;
9316 |   gap: 16px !important;
9317 | }
9318 |
9319 | .drawer-dataset-card {
9320 |   display: flex !important;
9321 |   justify-content: space-between !important;
9322 |   align-items: center !important;
9323 |   background: #ffffff !important;
9324 |   border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
9325 |   padding: 16px 20px !important;
9326 |   cursor: pointer !important;
9327 |   text-align: left !important;
9328 |   transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1) !important;
9329 | }
9330 |
9331 | .drawer-dataset-card:hover {
9332 |   border-color: #0f172a !important;
9333 |   transform: translateY(-2px) !important;
9334 |   box-shadow: 0 6px 20px rgba(15, 23, 42, 0.04) !important;
9335 | }
9336 |
9337 | .drawer-dataset-card.active {
9338 |   border-color: #2563eb !important;
9339 |   border-width: 2px !important;
9340 |   background: rgba(37, 99, 235, 0.02) !important;
9341 |   box-shadow: 0 6px 20px rgba(37, 99, 235, 0.06) !important;
9342 | }
9343 |
9344 | .dataset-card-info {
9345 |   display: flex !important;

```

```

9346 | flex-direction: column !important;
9347 | gap: 4px !important;
9348 | }
9349 |
9350 | .dataset-card-info strong {
9351 |   font-size: 15px !important;
9352 |   font-weight: 900 !important;
9353 |   color: #0f172a !important;
9354 | }
9355 |
9356 | .dataset-card-status {
9357 |   font-size: 11px !important;
9358 |   font-weight: 850 !important;
9359 |   text-transform: uppercase !important;
9360 | }
9361 |
9362 | .dataset-card-status.live {
9363 |   color: #22c55e !important;
9364 | }
9365 |
9366 | .dataset-card-status.stored {
9367 |   color: #2563eb !important;
9368 | }
9369 |
9370 | .active-check-icon {
9371 |   color: #2563eb !important;
9372 |   flex-shrink: 0 !important;
9373 | }
9374 |
9375 | .drawer-empty-text {
9376 |   color: #64748b !important;
9377 |   text-align: center !important;
9378 |   font-weight: 700 !important;
9379 |   margin-top: 40px !important;
9380 | }
9381 |
9382 | /* =====
9383 | ACTION CARDS REDESIGN
9384 | ===== */
9385 | .admin-action-grid {
9386 |   position: relative !important;
9387 |   z-index: 5 !important;
9388 |   display: grid !important;
9389 |   grid-template-columns: repeat(3, minmax(0, 1fr)) !important;
9390 |   gap: 24px !important;
9391 |   opacity: 1 !important;
9392 |   animation: adminHeroReveal 0.6s cubic-bezier(0.16, 1, 0.3, 1) 0.3s both !important;
9393 | }
9394 |
9395 | .admin-action-card {
9396 |   background: #ffffff !important;
9397 |   border: 1.5px solid #0f172a !important;
9398 |   padding: 32px 28px !important;
9399 |   display: flex !important;
9400 |   flex-direction: column !important;
9401 |   align-items: flex-start !important;
9402 |   text-align: left !important;
9403 |   cursor: pointer !important;
9404 |   position: relative !important;
9405 |   overflow: hidden !important;
9406 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
9407 | }
9408 |
9409 | .admin-action-card:hover {
9410 |   transform: translateY(-6px) !important;
9411 |   border-color: #0f172a !important;
9412 |   box-shadow: 0 16px 36px rgba(15, 23, 42, 0.06) !important;
9413 | }
9414 |
9415 | .action-card-accent {
9416 |   position: absolute !important;
9417 |   top: 0 !important;
9418 |   left: 0 !important;
9419 |   bottom: 0 !important;
9420 |   width: 4px !important;
9421 |   background: #e2e8f0 !important;
9422 |   transition: width 0.2s ease !important;
9423 | }
9424 |
9425 | .admin-action-card:hover .action-card-accent {
9426 |   width: 6px !important;
9427 | }
9428 |
9429 | .create-card .action-card-accent {
9430 |   background: linear-gradient(to bottom, #2563eb, #06b6d4) !important;
9431 | }
9432 |
9433 | .analysis-card .action-card-accent {
9434 |   background: linear-gradient(to bottom, #22c55e, #2563eb) !important;
9435 | }
9436 |
9437 | .action-icon-wrapper {
9438 |   display: flex !important;
9439 |   align-items: center !important;
9440 |   justify-content: center !important;
9441 |   width: 48px !important;
9442 |   height: 48px !important;
9443 |   background: #f8fafc !important;
9444 |   border: 1.5px solid rgba(15, 23, 42, 0.06) !important;
9445 |   margin-bottom: 24px !important;
9446 |   color: #0f172a !important;
9447 |   transition: all 0.3s !important;
9448 | }
9449 |
9450 | .admin-action-card:hover .action-icon-wrapper {
9451 |   background: #0f172a !important;
9452 |   color: #ffffff !important;
9453 |   border-color: #0f172a !important;
9454 | }
9455 |

```

```

9456 | .create-card:hover .action-icon-wrapper {
9457 |     background: #2563eb !important;
9458 | }
9459 |
9460 | .analysis-card:hover .action-icon-wrapper {
9461 |     background: #22c55e !important;
9462 | }
9463 |
9464 | .admin-action-card strong {
9465 |     font-size: 20px !important;
9466 |     font-weight: 950 !important;
9467 |     color: #0f172a !important;
9468 |     margin-bottom: 4px !important;
9469 | }
9470 |
9471 | .action-card-subtitle {
9472 |     font-size: 11px !important;
9473 |     font-weight: 900 !important;
9474 |     text-transform: uppercase !important;
9475 |     letter-spacing: 0.05em !important;
9476 |     margin-bottom: 12px !important;
9477 | }
9478 |
9479 | .create-card .action-card-subtitle {
9480 |     color: #06b6d4 !important;
9481 | }
9482 |
9483 | .analysis-card .action-card-subtitle {
9484 |     color: #2563eb !important;
9485 | }
9486 |
9487 | .admin-action-card p {
9488 |     margin: 0 0 24px 0 !important;
9489 |     color: #475569 !important;
9490 |     font-size: 14px !important;
9491 |     line-height: 1.5 !important;
9492 |     font-weight: 600 !important;
9493 | }
9494 |
9495 | .action-card-badge {
9496 |     margin-top: auto !important;
9497 |     font-size: 11px !important;
9498 |     font-style: normal !important;
9499 |     font-weight: 900 !important;
9500 |     padding: 4px 10px !important;
9501 |     background: #f1f5f9 !important;
9502 |     color: #475569 !important;
9503 |     border: 1px solid rgba(15, 23, 42, 0.06) !important;
9504 |     text-transform: uppercase !important;
9505 | }
9506 |
9507 | .action-card-badge.highlight {
9508 |     background: rgba(37, 99, 235, 0.06) !important;
9509 |     color: #2563eb !important;
9510 |     border-color: rgba(37, 99, 235, 0.12) !important;
9511 | }
9512 |
9513 | .action-card-badge.danger {
9514 |     background: rgba(239, 68, 68, 0.06) !important;
9515 |     color: #ef4444 !important;
9516 |     border-color: rgba(239, 68, 68, 0.12) !important;
9517 | }
9518 |
9519 | .action-card-badge.danger.pulse {
9520 |     animation: badgePulse 2s infinite !important;
9521 | }
9522 |
9523 | @keyframes badgePulse {
9524 |     0% {
9525 |         box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.2);
9526 |     }
9527 |     70% {
9528 |         box-shadow: 0 0 0 6px rgba(239, 68, 68, 0);
9529 |     }
9530 |     100% {
9531 |         box-shadow: 0 0 0 0 rgba(239, 68, 68, 0);
9532 |     }
9533 | }
9534 |
9535 | /* =====
9536 |     JUDGE AI VALIDATION DEMO SECTION
9537 |     ===== */
9538 | .admin-validation-panel {
9539 |     position: relative !important;
9540 |     z-index: 5 !important;
9541 |     display: flex !important;
9542 |     justify-content: space-between !important;
9543 |     align-items: center !important;
9544 |     background: #ffffff !important;
9545 |     border: 1.5px solid #0f172a !important;
9546 |     border-left: 4px solid #2563eb !important;
9547 |     padding: 24px 28px !important;
9548 |     box-shadow: 0 4px 15px rgba(15, 23, 42, 0.01) !important;
9549 |     gap: 20px !important;
9550 |     opacity: 1 !important;
9551 |     animation: adminHeroReveal 0.6s cubic-bezier(0.16, 1, 0.3, 1) 0.45s both !important;
9552 | }
9553 |
9554 | .admin-validation-panel:hover {
9555 |     border-color: #0f172a !important;
9556 |     border-left-color: #2563eb !important;
9557 |     box-shadow: 0 10px 25px rgba(15, 23, 42, 0.03) !important;
9558 | }
9559 |
9560 | .admin-validation-panel.pass {
9561 |     border-left-color: #22c55e !important;
9562 | }
9563 |
9564 | .admin-validation-panel > div {
9565 |     display: flex !important;

```



```

9566 | align-items: center !important;
9567 | gap: 18px !important;
9568 | }
9569 |
9570 | .validation-icon {
9571 |   display: flex !important;
9572 |   align-items: center !important;
9573 |   justify-content: center !important;
9574 |   width: 44px !important;
9575 |   height: 44px !important;
9576 |   background: rgba(37, 99, 235, 0.06) !important;
9577 |   color: #2563eb !important;
9578 |   border: 1.5px solid rgba(37, 99, 235, 0.1) !important;
9579 |   flex-shrink: 0 !important;
9580 | }
9581 |
9582 | .admin-validation-panel.pass .validation-icon {
9583 |   background: rgba(34, 197, 94, 0.06) !important;
9584 |   color: #22c55e !important;
9585 |   border-color: rgba(34, 197, 94, 0.1) !important;
9586 | }
9587 |
9588 | .validation-text-block {
9589 |   display: flex !important;
9590 |   flex-direction: column !important;
9591 |   gap: 4px !important;
9592 | }
9593 |
9594 | .admin-validation-panel strong {
9595 |   font-size: 16px !important;
9596 |   font-weight: 950 !important;
9597 |   color: #0f172a !important;
9598 | }
9599 |
9600 | .admin-validation-panel p {
9601 |   margin: 0 !important;
9602 |   color: #475569 !important;
9603 |   font-size: 13px !important;
9604 |   line-height: 1.5 !important;
9605 |   font-weight: 600 !important;
9606 | }
9607 |
9608 | .btn-admin-validation {
9609 |   background: #ffffff !important;
9610 |   color: #0f172a !important;
9611 |   border: 1.5px solid #0f172a !important;
9612 |   padding: 10px 20px !important;
9613 |   font-size: 13px !important;
9614 |   font-weight: 900 !important;
9615 |   cursor: pointer !important;
9616 |   flex-shrink: 0 !important;
9617 |   transition: all 0.2s !important;
9618 | }
9619 |
9620 | .btn-admin-validation:hover:not(:disabled) {
9621 |   background: #0f172a !important;
9622 |   color: #ffffff !important;
9623 |   transform: translateY(-1px) !important;
9624 | }
9625 |
9626 | .btn-admin-validation:disabled {
9627 |   opacity: 0.6 !important;
9628 |   cursor: not-allowed !important;
9629 | }
9630 |
9631 | /* Validation Result Styling */
9632 | .admin-validation-result {
9633 |   background: #ffffff !important;
9634 |   border: 1.5px solid #0f172a !important;
9635 |   padding: 24px !important;
9636 |   box-shadow: 0 8px 30px rgba(15, 23, 42, 0.03) !important;
9637 |   display: flex !important;
9638 |   flex-direction: column !important;
9639 |   gap: 20px !important;
9640 | }
9641 |
9642 | .validation-result-summary {
9643 |   display: flex !important;
9644 |   align-items: center !important;
9645 |   gap: 16px !important;
9646 |   border-bottom: 1.5px solid rgba(15, 23, 42, 0.06) !important;
9647 |   padding-bottom: 16px !important;
9648 | }
9649 |
9650 | .result-status-pill {
9651 |   font-size: 11px !important;
9652 |   font-weight: 900 !important;
9653 |   padding: 4px 8px !important;
9654 |   border: 1px solid #0f172a !important;
9655 |   text-transform: uppercase !important;
9656 | }
9657 |
9658 | .result-status-pill.pass {
9659 |   background: rgba(34, 197, 94, 0.08) !important;
9660 |   color: #22c55e !important;
9661 |   border-color: rgba(34, 197, 94, 0.2) !important;
9662 | }
9663 |
9664 | .result-status-pill.review {
9665 |   background: rgba(245, 158, 11, 0.08) !important;
9666 |   color: #f59e0b !important;
9667 |   border-color: rgba(245, 158, 11, 0.2) !important;
9668 | }
9669 |
9670 | .result-score {
9671 |   font-size: 24px !important;
9672 |   font-weight: 950 !important;
9673 |   color: #0f172a !important;
9674 | }
9675 |

```

```

9676 | .result-text {
9677 |   margin: 0 !important;
9678 |   font-size: 13px !important;
9679 |   color: #475569 !important;
9680 |   font-weight: 600 !important;
9681 | }
9682 |
9683 | .validation-case-list {
9684 |   display: grid !important;
9685 |   grid-template-columns: repeat(2, minmax(0, 1fr)) !important;
9686 |   gap: 16px !important;
9687 | }
9688 |
9689 | .validation-case-item {
9690 |   border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
9691 |   padding: 18px !important;
9692 |   display: flex !important;
9693 |   flex-direction: column !important;
9694 |   gap: 8px !important;
9695 | }
9696 |
9697 | .case-status-badge {
9698 |   font-size: 10px !important;
9699 |   font-weight: 900 !important;
9700 |   width: fit-content !important;
9701 |   padding: 2px 6px !important;
9702 |   border: 1px solid #0f172a !important;
9703 |   text-transform: uppercase !important;
9704 | }
9705 |
9706 | .passed .case-status-badge {
9707 |   background: rgba(34, 197, 94, 0.08) !important;
9708 |   color: #22c55e !important;
9709 |   border-color: rgba(34, 197, 94, 0.2) !important;
9710 | }
9711 |
9712 | .failed .case-status-badge {
9713 |   background: rgba(239, 68, 68, 0.08) !important;
9714 |   color: #ef4444 !important;
9715 |   border-color: rgba(239, 68, 68, 0.2) !important;
9716 | }
9717 |
9718 | .validation-case-item strong {
9719 |   font-size: 14px !important;
9720 |   font-weight: 900 !important;
9721 |   color: #0f172a !important;
9722 | }
9723 |
9724 | .validation-case-item p {
9725 |   margin: 0 !important;
9726 |   font-size: 12px !important;
9727 |   color: #475569 !important;
9728 |   line-height: 1.4 !important;
9729 |   font-weight: 600 !important;
9730 | }
9731 |
9732 | /* =====
9733 |   DEVELOPED BY TEAM EUREKA FOOTER
9734 |   ===== */
9735 | .admin-footer-sig {
9736 |   text-align: center !important;
9737 |   padding: 24px 0 0 0 !important;
9738 |   margin: 16px 0 0 0 !important;
9739 |   font-size: 11px !important;
9740 |   font-weight: 900 !important;
9741 |   color: #94a3b8 !important;
9742 |   letter-spacing: 0.08em !important;
9743 |   text-transform: uppercase !important;
9744 |   border: none !important;
9745 |   background: transparent !important;
9746 |   box-shadow: none !important;
9747 | }
9748 |
9749 | /* =====
9750 |   RESPONSIVE LAYOUTS
9751 |   ===== */
9752 | @media (max-width: 1024px) {
9753 |   .admin-action-grid {
9754 |     grid-template-columns: repeat(2, minmax(0, 1fr)) !important;
9755 |   }
9756 | }
9757 |
9758 | @media (max-width: 768px) {
9759 |   .admin-topbar {
9760 |     padding: 12px 16px !important;
9761 |   }
9762 |
9763 |   .admin-console-label {
9764 |     display: none !important;
9765 |   }
9766 |
9767 |   .admin-home-panel {
9768 |     padding: 24px !important;
9769 |     gap: 20px !important;
9770 |   }
9771 |
9772 |   .admin-dataset-panel {
9773 |     flex-direction: column !important;
9774 |     align-items: flex-start !important;
9775 |     gap: 16px !important;
9776 |     padding: 16px !important;
9777 |   }
9778 |
9779 |   .dataset-panel-right {
9780 |     width: 100% !important;
9781 |   }
9782 |
9783 |   .btn-admin-choose-dataset,
9784 |   .btn-admin-refresh-datasets {
9785 |     flex: 1 1 auto !important;

```

```

9786 |     justify-content: center !important;
9787 | }
9788 |
9789 | .admin-action-grid {
9790 |     grid-template-columns: 1fr !important;
9791 |     gap: 16px !important;
9792 | }
9793 |
9794 | .admin-validation-panel {
9795 |     flex-direction: column !important;
9796 |     align-items: flex-start !important;
9797 |     padding: 20px !important;
9798 | }
9799 |
9800 | .btn-admin-validation {
9801 |     width: 100% !important;
9802 | }
9803 |
9804 | .validation-case-list {
9805 |     grid-template-columns: 1fr !important;
9806 | }
9807 |
9808 | .dataset-drawer {
9809 |     width: 100% !important;
9810 | }
9811 | }
9812 |
9813 | /* Admin stack (Create Feedback Page) */
9814 | .admin-stack {
9815 |     display: flex !important;
9816 |     flex-direction: column !important;
9817 |     gap: 20px !important;
9818 |     opacity: 1 !important;
9819 |     transform: none !important;
9820 |     animation: revealDirect 0.65s cubic-bezier(0.16, 1, 0.3, 1) both !important;
9821 | }
9822 |
9823 | .admin-back-button {
9824 |     width: max-content !important;
9825 |     margin-bottom: 8px !important;
9826 | }
9827 |
9828 | .feedback-builder-panel {
9829 |     background: #ffffff !important;
9830 |     border: 1.5px solid rgba(15, 23, 42, 0.06) !important;
9831 |     box-shadow: 0 10px 40px rgba(15, 23, 42, 0.02) !important;
9832 |     color: #050a18 !important;
9833 |     padding: 36px !important;
9834 | }
9835 |
9836 | .feedback-builder-panel h2 {
9837 |     color: #050a18 !important;
9838 |     font-weight: 950 !important;
9839 |     font-size: 1.8rem !important;
9840 | }
9841 |
9842 | .feedback-live-status {
9843 |     display: flex !important;
9844 |     align-items: center !important;
9845 |     gap: 14px !important;
9846 |     margin: 4px 0 24px !important;
9847 |     border: 1.5px solid rgba(15, 23, 42, 0.07) !important;
9848 |     border-radius: 18px !important;
9849 |     padding: 16px 18px !important;
9850 |     background: rgba(248, 250, 252, 0.82) !important;
9851 |     box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.9) !important;
9852 | }
9853 |
9854 | .feedback-live-status.live {
9855 |     border-color: rgba(34, 197, 94, 0.22) !important;
9856 |     background: linear-gradient(135deg, rgba(34, 197, 94, 0.1), rgba(255, 255, 255, 0.88)) !important;
9857 | }
9858 |
9859 | .feedback-live-status.closed {
9860 |     border-color: rgba(239, 68, 68, 0.22) !important;
9861 |     background: linear-gradient(135deg, rgba(239, 68, 68, 0.08), rgba(255, 255, 255, 0.88)) !important;
9862 | }
9863 |
9864 | .feedback-live-light {
9865 |     width: 15px !important;
9866 |     height: 15px !important;
9867 |     flex: 0 0 15px !important;
9868 |     border-radius: 999px !important;
9869 | }
9870 |
9871 | .feedback-live-status.live .feedback-live-light {
9872 |     background: #22c55e !important;
9873 |     box-shadow:
9874 |         0 0 7px rgba(34, 197, 94, 0.14),
9875 |         0 0 26px rgba(34, 197, 94, 0.65) !important;
9876 |     animation: feedbackLivePulse 1.4s ease-in-out infinite !important;
9877 | }
9878 |
9879 | .feedback-live-status.closed .feedback-live-light {
9880 |     background: #ef4444 !important;
9881 |     box-shadow:
9882 |         0 0 7px rgba(239, 68, 68, 0.12),
9883 |         0 0 20px rgba(239, 68, 68, 0.36) !important;
9884 | }
9885 |
9886 | .feedback-live-status strong {
9887 |     display: block !important;
9888 |     color: #0f172a !important;
9889 |     font-size: 1rem !important;
9890 |     font-weight: 950 !important;
9891 | }
9892 |
9893 | .feedback-live-status em {
9894 |     display: block !important;
9895 |     margin-top: 3px !important;

```

```

9896 | color: #64748b !important;
9897 | font-size: 0.88rem !important;
9898 | font-style: normal !important;
9899 | font-weight: 700 !important;
9900 | }
9901 |
9902 | @keyframes feedbackLivePulse {
9903 |   0%,
9904 |   100% {
9905 |     transform: scale(1);
9906 |   }
9907 |   50% {
9908 |     transform: scale(1.18);
9909 |   }
9910 | }
9911 |
9912 | .feedback-builder-panel label {
9913 |   color: #475569 !important;
9914 |   font-weight: 800 !important;
9915 |   font-size: 0.95rem !important;
9916 |   margin-bottom: 20px !important;
9917 | }
9918 |
9919 | .feedback-builder-panel input {
9920 |   border: 1.5px solid rgba(15, 23, 42, 0.08) !important;
9921 |   background: #ffffff !important;
9922 |   color: #050a18 !important;
9923 |   padding: 12px 16px !important;
9924 |   transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1) !important;
9925 | }
9926 |
9927 | .feedback-builder-panel input:focus {
9928 |   border-color: #2563eb !important;
9929 |   box-shadow: 0 0 4px rgba(37, 99, 235, 0.08) !important;
9930 |   transform: translateY(-1px) !important;
9931 | }
9932 |
9933 | .builder-category-list {
9934 |   display: flex !important;
9935 |   flex-direction: column !important;
9936 |   gap: 24px !important;
9937 |   margin-bottom: 24px !important;
9938 | }
9939 |
9940 | .builder-category {
9941 |   border: 1.5px solid rgba(15, 23, 42, 0.05) !important;
9942 |   background: rgba(250, 249, 246, 0.5) !important;
9943 |   padding: 24px !important;
9944 |   display: flex !important;
9945 |   flex-direction: column !important;
9946 |   gap: 16px !important;
9947 | }
9948 |
9949 | .builder-category label {
9950 |   margin-bottom: 8px !important;
9951 | }
9952 |
9953 | .builder-question-list {
9954 |   display: grid !important;
9955 |   grid-template-columns: 1fr !important;
9956 |   gap: 14px !important;
9957 |   margin-top: 10px !important;
9958 | }
9959 |
9960 | .builder-question-list label {
9961 |   margin: 0 !important;
9962 | }
9963 |
9964 | .feedback-builder-panel .secondary-button {
9965 |   background: linear-gradient(135deg, #2563eb 0%, #1e40af 100%) !important;
9966 |   border-color: #2563eb !important;
9967 |   color: #ffffff !important;
9968 |   font-weight: 850 !important;
9969 |   min-height: 42px !important;
9970 |   padding: 0 20px !important;
9971 |   box-shadow: 0 4px 15px rgba(37, 99, 235, 0.14) !important;
9972 |   transition: all 0.25s !important;
9973 | }
9974 |
9975 | .feedback-builder-panel .secondary-button:hover {
9976 |   box-shadow: 0 8px 25px rgba(37, 99, 235, 0.22) !important;
9977 |   transform: translateY(-2px) !important;
9978 | }
9979 |
9980 | .feedback-builder-panel .danger-outline {
9981 |   border: 1.5px solid rgba(239, 68, 68, 0.2) !important;
9982 |   background: rgba(239, 68, 68, 0.04) !important;
9983 |   color: #ef4444 !important;
9984 |   font-weight: 800 !important;
9985 |   transition: all 0.2s !important;
9986 | }
9987 |
9988 | .feedback-builder-panel .danger-outline:hover {
9989 |   background: #ef4444 !important;
9990 |   border-color: #ef4444 !important;
9991 |   color: #ffffff !important;
9992 |   box-shadow: 0 4px 12px rgba(239, 68, 68, 0.15) !important;
9993 | }
9994 |
9995 | .feedback-builder-panel .text-button {
9996 |   color: #2563eb !important;
9997 |   font-weight: 800 !important;
9998 |   font-size: 0.95rem !important;
9999 |   text-decoration: none !important;
10000 |   width: max-content !important;
10001 |   transition: all 0.2s !important;
10002 | }
10003 |
10004 | .feedback-builder-panel .text-button:hover {
10005 |   color: #1d4ed8 !important;

```

```

10006 | text-decoration: underline !important;
10007 | }
10008 |
10009 | .feedback-builder-panel > .primary-button {
10010 |   background: #050a18 !important;
10011 |   color: #ffffff !important;
10012 |   border: 1px solid #050a18 !important;
10013 |   font-weight: 850 !important;
10014 |   min-height: 44px !important;
10015 |   transition: all 0.25s !important;
10016 | }
10017 |
10018 | .feedback-builder-panel > .primary-button:hover {
10019 |   background: #2563eb !important;
10020 |   border-color: #2563eb !important;
10021 |   box-shadow: 0 6px 20px rgba(37, 99, 235, 0.15) !important;
10022 |   transform: translateY(-2px) !important;
10023 | }
10024 |
10025 | /* =====
10026 | STUDENT PORTAL HERO, TYPOGRAPHY, ACTION CARDS & FOOTER REDESIGN
10027 | ===== */
10028 |
10029 | /* 1. Logo Stage & Floating Animation */
10030 | /* 1. Multi-Layered Ambient Glow Animations */
10031 | .ambient-glow-wrapper {
10032 |   position: absolute !important;
10033 |   top: 0 !important;
10034 |   left: 0 !important;
10035 |   width: 100% !important;
10036 |   height: 100% !important;
10037 |   z-index: 0 !important;
10038 |   pointer-events: none !important;
10039 |   overflow: hidden !important;
10040 | }
10041 |
10042 | .ambient-glow {
10043 |   position: absolute !important;
10044 |   border-radius: 50% !important;
10045 |   filter: blur(80px) !important;
10046 |   opacity: 0.6 !important;
10047 |   pointer-events: none !important;
10048 | }
10049 |
10050 | .ambient-glow.glow-1 {
10051 |   width: 450px !important;
10052 |   height: 450px !important;
10053 |   background: radial-gradient(circle, rgba(37, 99, 235, 0.18) 0%, rgba(99, 102, 241, 0.04) 60%, rgba(255, 255, 255, 0) 100%) !important;
10054 |   top: -100px !important;
10055 |   left: -80px !important;
10056 |   animation: floatGlow1 12s infinite ease-in-out !important;
10057 |   opacity: 0.85 !important;
10058 | }
10059 |
10060 | .ambient-glow.glow-2 {
10061 |   width: 500px !important;
10062 |   height: 500px !important;
10063 |   background: radial-gradient(circle, rgba(16, 185, 129, 0.15) 0%, rgba(34, 197, 94, 0.03) 60%, rgba(255, 255, 255, 0) 100%) !important;
10064 |   bottom: -150px !important;
10065 |   right: -100px !important;
10066 |   animation: floatGlow2 16s infinite ease-in-out !important;
10067 |   opacity: 0.85 !important;
10068 | }
10069 |
10070 | .ambient-glow.glow-3 {
10071 |   width: 400px !important;
10072 |   height: 400px !important;
10073 |   background: radial-gradient(circle, rgba(244, 63, 94, 0.14) 0%, rgba(239, 68, 68, 0.02) 60%, rgba(255, 255, 255, 0) 100%) !important;
10074 |   top: 25% !important;
10075 |   left: 30% !important;
10076 |   animation: floatGlow3 18s infinite ease-in-out !important;
10077 |   opacity: 0.85 !important;
10078 | }
10079 |
10080 | @keyframes floatGlow1 {
10081 |   0%, 100% {
10082 |     transform: translate(0, 0) scale(1);
10083 |     opacity: 0.6;
10084 |   }
10085 |   50% {
10086 |     transform: translate(40px, 25px) scale(1.1);
10087 |     opacity: 0.85;
10088 |   }
10089 | }
10090 |
10091 | @keyframes floatGlow2 {
10092 |   0%, 100% {
10093 |     transform: translate(0, 0) scale(1.05);
10094 |     opacity: 0.5;
10095 |   }
10096 |   50% {
10097 |     transform: translate(-30px, -35px) scale(0.9);
10098 |     opacity: 0.75;
10099 |   }
10100 | }
10101 |
10102 | @keyframes floatGlow3 {
10103 |   0%, 100% {
10104 |     transform: translate(0, 0) scale(0.95);
10105 |     opacity: 0.4;
10106 |   }
10107 |   50% {
10108 |     transform: translate(25px, -20px) scale(1.05);
10109 |     opacity: 0.6;
10110 |   }
10111 | }
10112 |
10113 | /* 2. Embedded Action Card Status Indicators */
10114 | .action-card-content {
10115 |   display: flex !important;

```

```

10116 | flex-direction: column !important;
10117 | gap: 6px !important;
10118 | width: 100% !important;
10119 | }
10120 |
10121 | .action-title-row {
10122 |   display: flex !important;
10123 |   align-items: center !important;
10124 |   gap: 12px !important;
10125 | }
10126 |
10127 | .action-status-pill {
10128 |   display: inline-flex !important;
10129 |   align-items: center !important;
10130 |   gap: 6px !important;
10131 |   padding: 2px 8px !important;
10132 |   font-family: monospace !important;
10133 |   font-size: 0.7rem !important;
10134 |   font-weight: 800 !important;
10135 |   letter-spacing: 0.05em !important;
10136 |   border-radius: 0px !important;
10137 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
10138 | }
10139 |
10140 | .status-dot {
10141 |   width: 6px !important;
10142 |   height: 6px !important;
10143 |   border-radius: 50% !important;
10144 |   display: inline-block !important;
10145 | }
10146 |
10147 | /* LIVE Action Card: Red Blinking indicator */
10148 | .student-action-card.status-live {
10149 |   border-left: 4px solid #ef4444 !important;
10150 | }
10151 |
10152 | .student-action-card.status-live .action-status-pill {
10153 |   background: rgba(239, 68, 68, 0.08) !important;
10154 |   border: 1px solid rgba(239, 68, 68, 0.25) !important;
10155 |   color: #ef4444 !important;
10156 | }
10157 |
10158 | .student-action-card.status-live .status-dot {
10159 |   background: #ef4444 !important;
10160 |   box-shadow: 0 0 6px #ef4444 !important;
10161 |   animation: pulseLiveDot 1.4s infinite ease-in-out !important;
10162 | }
10163 |
10164 | /* CLOSED Action Card: Green Solid indicator */
10165 | .student-action-card.status-closed {
10166 |   border-left: 4px solid #10b981 !important;
10167 |   opacity: 0.8 !important;
10168 |   cursor: not-allowed !important;
10169 | }
10170 |
10171 | .student-action-card.status-closed .action-status-pill {
10172 |   background: rgba(16, 185, 129, 0.06) !important;
10173 |   border: 1px solid rgba(16, 185, 129, 0.2) !important;
10174 |   color: #10b981 !important;
10175 | }
10176 |
10177 | .student-action-card.status-closed .status-dot {
10178 |   background: #10b981 !important;
10179 |   box-shadow: none !important;
10180 | }
10181 |
10182 | @keyframes pulseLiveDot {
10183 |   0%, 100% {
10184 |     transform: scale(1);
10185 |     opacity: 0.4;
10186 |   }
10187 |   50% {
10188 |     transform: scale(1.35);
10189 |     opacity: 1;
10190 |   }
10191 | }
10192 |
10193 | /* 2. Hero Typography */
10194 | .student-hero-logo-row {
10195 |   display: flex !important;
10196 |   align-items: center !important;
10197 |   gap: 10px !important;
10198 |   margin-bottom: 24px !important;
10199 | }
10200 |
10201 | .student-hero-logo-img {
10202 |   width: 52px !important;
10203 |   height: 52px !important;
10204 |   object-fit: contain !important;
10205 |   animation: floatSlow 6s infinite ease-in-out !important;
10206 | }
10207 |
10208 | .student-hero-logo-text {
10209 |   font-size: 1.45rem !important;
10210 |   font-weight: 900 !important;
10211 |   letter-spacing: -0.02em !important;
10212 |   color: #1e1b4b !important;
10213 |   background: linear-gradient(135deg, #1e1b4b 20%, #2563eb 100%) !important;
10214 |   -webkit-background-clip: text !important;
10215 |   -webkit-text-fill-color: transparent !important;
10216 | }
10217 |
10218 | .student-hero-title {
10219 |   font-size: clamp(2.4rem, 4.5vw, 3.2rem) !important;
10220 |   font-weight: 950 !important;
10221 |   line-height: 1.15 !important;
10222 |   letter-spacing: -0.03em !important;
10223 |   margin: 0 0 16px 0 !important;
10224 |   text-align: left !important;
10225 |   background: linear-gradient(135deg, #1e1b4b 20%, #2563eb 60%, #0ea5e9 100%) !important;

```

```

10226 | -webkit-background-clip: text !important;
10227 | -webkit-text-fill-color: transparent !important;
10228 | display: flex !important;
10229 | align-items: center !important;
10230 | gap: 12px !important;
10231 | }
10232 |
10233 | .hero-title-icon-sparkle {
10234 |   color: #2563eb !important;
10235 |   flex-shrink: 0 !important;
10236 |   animation: pulseGlow 3s infinite ease-in-out !important;
10237 | }
10238 |
10239 | @keyframes pulseGlow {
10240 |   0%, 100% {
10241 |     transform: scale(1);
10242 |     opacity: 0.8;
10243 |     filter: drop-shadow(0 0 2px rgba(37, 99, 235, 0));
10244 |   }
10245 |   50% {
10246 |     transform: scale(1.15);
10247 |     opacity: 1;
10248 |     filter: drop-shadow(0 0 8px rgba(37, 99, 235, 0.4));
10249 |   }
10250 | }
10251 |
10252 | .student-hero-desc {
10253 |   font-size: 1.18rem !important;
10254 |   line-height: 1.65 !important;
10255 |   color: #475569 !important;
10256 |   max-width: 100% !important;
10257 |   text-align: left !important;
10258 |   margin: 0 0 24px 0 !important;
10259 | }
10260 |
10261 | /* 2.1 Badges Strip */
10262 | .student-badge-strip {
10263 |   display: flex !important;
10264 |   flex-wrap: wrap !important;
10265 |   gap: 12px !important;
10266 |   margin-bottom: 36px !important;
10267 | }
10268 |
10269 | .student-badge {
10270 |   display: inline-flex !important;
10271 |   align-items: center !important;
10272 |   gap: 8px !important;
10273 |   padding: 8px 16px !important;
10274 |   background: rgba(255, 255, 255, 0.65) !important;
10275 |   border: 1px solid rgba(37, 99, 235, 0.08) !important;
10276 |   border-radius: 0px !important;
10277 |   color: #334155 !important;
10278 |   font-size: 0.88rem !important;
10279 |   font-weight: 700 !important;
10280 |   box-shadow: 0 4px 12px rgba(15, 23, 42, 0.01) !important;
10281 |   backdrop-filter: blur(8px) !important;
10282 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
10283 | }
10284 |
10285 | .student-badge:hover {
10286 |   transform: translateY(-2px) !important;
10287 |   border-color: rgba(37, 99, 235, 0.2) !important;
10288 |   box-shadow: 0 8px 20px rgba(37, 99, 235, 0.06) !important;
10289 |   background: #ffffff !important;
10290 | }
10291 |
10292 | .student-badge svg {
10293 |   color: #2563eb !important;
10294 | }
10295 |
10296 | /* 3. Student Profile Card Redesign */
10297 | .student-profile-card {
10298 |   background: linear-gradient(180deg, #ffffff 0%, #faf9f6 100%) !important;
10299 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
10300 |   box-shadow: 0 10px 30px rgba(15, 23, 42, 0.02) !important;
10301 |   padding: 28px !important;
10302 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
10303 | }
10304 |
10305 | .student-profile-card:hover {
10306 |   box-shadow: 0 16px 40px rgba(15, 23, 42, 0.04) !important;
10307 |   transform: translateY(-2px) !important;
10308 | }
10309 |
10310 | .student-avatar {
10311 |   border: 3px solid rgba(37, 99, 235, 0.12) !important;
10312 |   box-shadow: 0 4px 12px rgba(37, 99, 235, 0.06) !important;
10313 |   transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275) !important;
10314 | }
10315 |
10316 | .student-avatar:hover {
10317 |   transform: scale(1.05) rotate(3deg) !important;
10318 |   border-color: #2563eb !important;
10319 |   box-shadow: 0 8px 20px rgba(37, 99, 235, 0.15) !important;
10320 | }
10321 |
10322 | .student-details div {
10323 |   border-bottom: 1px solid rgba(15, 23, 42, 0.05) !important;
10324 |   padding: 10px 0 !important;
10325 | }
10326 |
10327 | .student-details dt {
10328 |   color: #64748b !important;
10329 |   font-size: 0.8rem !important;
10330 |   font-weight: 800 !important;
10331 |   text-transform: uppercase !important;
10332 |   letter-spacing: 0.05em !important;
10333 | }
10334 |
10335 | .student-details dd {

```

```

10336 | color: #050a18 !important;
10337 | font-weight: 750 !important;
10338 | font-size: 0.95rem !important;
10339 | }
10340 |
10341 | /* 4. Action Cards Redesign */
10342 | .student-action-card {
10343 |   display: grid !important;
10344 |   grid-template-columns: auto 1fr auto !important;
10345 |   align-items: center !important;
10346 |   gap: 20px !important;
10347 |   text-align: left !important;
10348 |   width: 100% !important;
10349 |   padding: 24px !important;
10350 |   background: #ffffff !important;
10351 |   border: 1px solid rgba(15, 23, 42, 0.06) !important;
10352 |   box-shadow: 0 4px 15px rgba(15, 23, 42, 0.01) !important;
10353 |   position: relative !important;
10354 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
10355 | }
10356 |
10357 | .student-action-card.primary-action {
10358 |   border-left: 4px solid #2563eb !important;
10359 | }
10360 |
10361 | .student-action-card.danger-action {
10362 |   border-left: 4px solid #ef4444 !important;
10363 | }
10364 |
10365 | .student-action-card.svg {
10366 |   grid-row: span 2 !important;
10367 |   width: 32px !important;
10368 |   height: 32px !important;
10369 |   transition: transform 0.3s !important;
10370 | }
10371 |
10372 | .student-action-card.primary-action.svg {
10373 |   color: #2563eb !important;
10374 | }
10375 |
10376 | .student-action-card.danger-action.svg {
10377 |   color: #ef4444 !important;
10378 | }
10379 |
10380 | .student-action-card.strong {
10381 |   font-size: 1.2rem !important;
10382 |   font-weight: 800 !important;
10383 |   color: #050a18 !important;
10384 |   margin: 0 !important;
10385 | }
10386 |
10387 | .student-action-card.span {
10388 |   font-size: 0.95rem !important;
10389 |   color: #475569 !important;
10390 |   margin: 0 !important;
10391 | }
10392 |
10393 | /* Slide-in Arrow Indicator */
10394 | .student-action-card::after {
10395 |   content: '→' !important;
10396 |   font-size: 1.25rem !important;
10397 |   font-weight: 900 !important;
10398 |   color: #94a3b8 !important;
10399 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1) !important;
10400 |   grid-row: span 2 !important;
10401 |   margin-left: 10px !important;
10402 | }
10403 |
10404 | /* Card Hover Interactions */
10405 | .student-action-card:hover {
10406 |   transform: translateY(-4px) !important;
10407 |   border-color: rgba(37, 99, 235, 0.15) !important;
10408 |   box-shadow: 0 12px 28px rgba(15, 23, 42, 0.03) !important;
10409 | }
10410 |
10411 | .student-action-card.danger-action:hover {
10412 |   border-color: rgba(239, 68, 68, 0.15) !important;
10413 | }
10414 |
10415 | .student-action-card:hover.svg {
10416 |   transform: scale(1.1) rotate(3deg) !important;
10417 | }
10418 |
10419 | .student-action-card:hover::after {
10420 |   color: #2563eb !important;
10421 |   transform: translateX(4px) !important;
10422 | }
10423 |
10424 | .student-action-card.danger-action:hover::after {
10425 |   color: #ef4444 !important;
10426 | }
10427 |
10428 | /* 5. Developed by Team Eureka Simple Signature Footer */
10429 | .team-eureka-badge {
10430 |   position: static !important;
10431 |   display: flex !important;
10432 |   align-items: center !important;
10433 |   justify-content: center !important;
10434 |   gap: 6px !important;
10435 |   margin: 48px auto 20px auto !important;
10436 |   padding: 0 !important;
10437 |   background: transparent !important;
10438 |   border: none !important;
10439 |   box-shadow: none !important;
10440 |   color: #94a3b8 !important;
10441 |   font-weight: 600 !important;
10442 |   font-size: 0.8rem !important;
10443 |   letter-spacing: 0.06em !important;
10444 |   text-transform: uppercase !important;
10445 | }

```



```

10446 |
10447 | .team-eureka-badge svg {
10448 |   color: #94a3b8 !important;
10449 | }
10450 |
10451 | /* 6. Spacing, Connections & Layout Alignments */
10452 | .student-home {
10453 |   display: flex !important;
10454 |   flex-direction: row-reverse !important;
10455 |   align-items: flex-start !important;
10456 |   gap: 28px !important;
10457 |   margin-top: 12px !important;
10458 | }
10459 |
10460 | .student-profile-card {
10461 |   width: 320px !important;
10462 |   flex-shrink: 0 !important;
10463 |   display: flex !important;
10464 |   flex-direction: column !important;
10465 |   border-radius: 0px !important;
10466 | }
10467 |
10468 | .student-hero-panel {
10469 |   position: relative !important;
10470 |   flex-grow: 1 !important;
10471 |   display: flex !important;
10472 |   flex-direction: column !important;
10473 |   justify-content: flex-start !important;
10474 |   padding: 12px 36px 36px 36px !important;
10475 |   background: transparent !important;
10476 |   border: none !important;
10477 |   box-shadow: none !important;
10478 |   border-radius: 0px !important;
10479 | }
10480 |
10481 | .student-hero-title,
10482 | .student-hero-desc,
10483 | .student-action-grid {
10484 |   position: relative !important;
10485 |   z-index: 1 !important;
10486 | }
10487 |
10488 | /* 7. Tablet & Mobile Responsive Redesign */
10489 | @media (max-width: 1024px) {
10490 |   .student-home {
10491 |     gap: 20px !important;
10492 |   }
10493 |   .student-profile-card {
10494 |     width: 280px !important;
10495 |   }
10496 | }
10497 |
10498 | @media (max-width: 768px) {
10499 |   .student-home {
10500 |     flex-direction: column !important;
10501 |     align-items: stretch !important;
10502 |     gap: 20px !important;
10503 |   }
10504 |   .student-profile-card {
10505 |     width: 100% !important;
10506 |   }
10507 |   .student-hero-panel {
10508 |     padding: 24px !important;
10509 |   }
10510 |   .student-hero-eyebrow,
10511 |   .student-hero-title,
10512 |   .student-hero-desc {
10513 |     text-align: left !important;
10514 |     margin-left: 0 !important;
10515 |     margin-right: 0 !important;
10516 |   }
10517 | }
10518 |
10519 | /* 8. Topbar Navigation Button Styles (Back & Logout Same Styles) */
10520 | .topbar-back-button,
10521 | .app-shell:has(.student-home) .topbar .icon-button,
10522 | .app-shell:has(.student-form-wrap) .topbar .icon-button {
10523 |   background: rgba(255, 255, 255, 0.85) !important;
10524 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
10525 |   border-radius: 0px !important; /* Sharp edges! */
10526 |   padding: 10px 18px !important;
10527 |   color: #1e1b4b !important;
10528 |   font-weight: 800 !important;
10529 |   font-size: 0.95rem !important;
10530 |   cursor: pointer !important;
10531 |   display: inline-flex !important;
10532 |   align-items: center !important;
10533 |   gap: 8px !important;
10534 |   box-shadow: 0 4px 12px rgba(15, 23, 42, 0.01) !important;
10535 |   transition: all 0.2s cubic-bezier(0.16, 1, 0.3, 1) !important;
10536 |   height: 42px !important;
10537 |   box-sizing: border-box !important;
10538 | }
10539 |
10540 | .topbar-back-button:hover,
10541 | .app-shell:has(.student-home) .topbar .icon-button:hover,
10542 | .app-shell:has(.student-form-wrap) .topbar .icon-button:hover {
10543 |   border-color: #2563eb !important;
10544 |   color: #2563eb !important;
10545 |   transform: translateY(-1px) !important;
10546 |   box-shadow: 0 8px 18px rgba(37, 99, 235, 0.06) !important;
10547 | }
10548 |
10549 | /* 9. Enforce Universal Sharp Edges for Semester Feedback and Student Forms */
10550 | .feedback-workspace,
10551 | .feedback-workspace *,
10552 | .student-form-wrap,
10553 | .student-form-wrap * {
10554 |   border-radius: 0px !important;
10555 | }

```

```

10556 |
10557 | /* 10. Center Title & Right-Aligned Red Submit Button */
10558 | .feedback-form-title {
10559 |   text-align: center !important;
10560 |   font-size: clamp(2.4rem, 5vw, 3.4rem) !important;
10561 |   font-weight: 950 !important;
10562 |   line-height: 1.15 !important;
10563 |   letter-spacing: -0.03em !important;
10564 |   margin: 10px 0 44px 0 !important;
10565 |   background: linear-gradient(135deg, #1e1b4b 20%, #2563eb 60%, #0ea5e9 100%) !important;
10566 |   -webkit-background-clip: text !important;
10567 |   -webkit-text-fill-color: transparent !important;
10568 | }
10569 |
10570 | .feedback-submit-container {
10571 |   display: flex !important;
10572 |   justify-content: flex-end !important;
10573 |   align-items: center !important;
10574 |   gap: 20px !important;
10575 |   width: 100% !important;
10576 |   margin-top: 36px !important;
10577 | }
10578 |
10579 | .feedback-submit-red {
10580 |   display: inline-flex !important;
10581 |   align-items: center !important;
10582 |   justify-content: center !important;
10583 |   gap: 8px !important;
10584 |   min-height: 48px !important;
10585 |   padding: 0 28px !important;
10586 |   background: #ef4444 !important;
10587 |   border: 1px solid #dc2626 !important;
10588 |   color: #ffffff !important;
10589 |   font-weight: 850 !important;
10590 |   font-size: 1.05rem !important;
10591 |   cursor: pointer !important;
10592 |   box-shadow: 0 8px 20px rgba(239, 68, 68, 0.16) !important;
10593 |   transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1) !important;
10594 | }
10595 |
10596 | .feedback-submit-red:hover {
10597 |   background: #dc2626 !important;
10598 |   border-color: #b91c1c !important;
10599 |   transform: translateY(-2px) !important;
10600 |   box-shadow: 0 12px 28px rgba(239, 68, 68, 0.28) !important;
10601 | }
10602 |
10603 | .feedback-submit-red svg {
10604 |   color: #ffffff !important;
10605 | }
10606 |
10607 | /* Final admin/auth polish: landing-page palette + logo size guard. */
10608 | .auth-shell,
10609 | .app-shell:has(.admin-home-panel) {
10610 |   background:
10611 |     linear-gradient(135deg, rgba(79, 70, 229, 0.08), transparent 34%),
10612 |     radial-gradient(circle at 16% 18%, rgba(20, 184, 166, 0.16), transparent 26%),
10613 |     radial-gradient(circle at 84% 12%, rgba(59, 130, 246, 0.16), transparent 30%),
10614 |     radial-gradient(circle at 62% 92%, rgba(244, 114, 182, 0.12), transparent 28%),
10615 |     #f7f8fb !important;
10616 | }
10617 |
10618 | .auth-shell {
10619 |   min-height: 100vh !important;
10620 |   padding: 86px 20px 76px !important;
10621 | }
10622 |
10623 | .auth-shell .auth-grid {
10624 |   width: min(920px, 100%) !important;
10625 |   grid-template-columns: minmax(300px, 0.92fr) minmax(320px, 1fr) !important;
10626 |   gap: 18px !important;
10627 | }
10628 |
10629 | .auth-shell .panel {
10630 |   overflow: hidden !important;
10631 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
10632 |   border-radius: 26px !important;
10633 |   background:
10634 |     linear-gradient(145deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.88)),
10635 |     radial-gradient(circle at 88% 0%, rgba(37, 99, 235, 0.11), transparent 30%) !important;
10636 |   color: #172033 !important;
10637 |   box-shadow:
10638 |     0 30px 90px rgba(15, 23, 42, 0.12),
10639 |     inset 0 1px 0 rgba(255, 255, 255, 0.9) !important;
10640 | }
10641 |
10642 | .auth-shell .auth-panel {
10643 |   padding: 30px !important;
10644 | }
10645 |
10646 | .auth-shell .signal-panel {
10647 |   position: relative !important;
10648 |   padding: 30px !important;
10649 |   background:
10650 |     linear-gradient(145deg, rgba(255, 255, 255, 0.96), rgba(241, 245, 249, 0.82)),
10651 |     radial-gradient(circle at 70% 18%, rgba(20, 184, 166, 0.12), transparent 32%) !important;
10652 | }
10653 |
10654 | .auth-shell .signal-panel::after {
10655 |   content: '' !important;
10656 |   position: absolute !important;
10657 |   right: -90px !important;
10658 |   top: -90px !important;
10659 |   width: 230px !important;
10660 |   height: 230px !important;
10661 |   border-radius: 999px !important;
10662 |   background: radial-gradient(circle, rgba(37, 99, 235, 0.12), transparent 62%) !important;
10663 |   pointer-events: none !important;
10664 | }
10665 |

```

```

10666 | .auth-shell .panel h2,
10667 | .auth-shell .signal-panel h2,
10668 | .auth-shell .signal-row strong,
10669 | .auth-shell label {
10670 |   color: #0f172a !important;
10671 |   -webkit-text-fill-color: initial !important;
10672 | }
10673 |
10674 | .auth-shell .signal-panel h2 {
10675 |   max-width: 430px !important;
10676 |   background: linear-gradient(94deg, #0f172a 0%, #1d4ed8 58%, #0f766e 100%) !important;
10677 |   background-clip: text !important;
10678 |   -webkit-background-clip: text !important;
10679 |   -webkit-text-fill-color: transparent !important;
10680 | }
10681 |
10682 | .auth-shell input {
10683 |   border: 1px solid rgba(15, 23, 42, 0.12) !important;
10684 |   border-radius: 16px !important;
10685 |   background: #ffffff !important;
10686 |   color: #172033 !important;
10687 | }
10688 |
10689 | .auth-shell input:focus {
10690 |   border-color: rgba(37, 99, 235, 0.34) !important;
10691 |   box-shadow: 0 0 4px rgba(37, 99, 235, 0.1) !important;
10692 | }
10693 |
10694 | .auth-shell .primary-button {
10695 |   border-radius: 16px !important;
10696 |   background: linear-gradient(135deg, #0f172a 0%, #2563eb 55%, #14b8a6 100%) !important;
10697 |   box-shadow: 0 18px 44px rgba(37, 99, 235, 0.18) !important;
10698 | }
10699 |
10700 | .auth-shell .auth-back-button {
10701 |   border-radius: 999px !important;
10702 |   background: rgba(255, 255, 255, 0.84) !important;
10703 | }
10704 |
10705 | .auth-shell .signal-row {
10706 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
10707 |   border-radius: 18px !important;
10708 |   background: rgba(255, 255, 255, 0.76) !important;
10709 |   color: #172033 !important;
10710 |   box-shadow: 0 14px 34px rgba(15, 23, 42, 0.06) !important;
10711 | }
10712 |
10713 | .auth-shell .signal-row span {
10714 |   color: #64748b !important;
10715 | }
10716 |
10717 | .auth-shell .login-orb {
10718 |   width: 104px !important;
10719 |   height: 104px !important;
10720 |   border-radius: 28px !important;
10721 |   background:
10722 |     radial-gradient(circle at 28% 24%, rgba(255, 255, 255, 0.7), transparent 28%),
10723 |     linear-gradient(135deg, #0f172a, #2563eb 58%, #14b8a6) !important;
10724 | }
10725 |
10726 | .auth-shell img,
10727 | .app-shell:has(.admin-home-panel) img,
10728 | .admin-topbar img,
10729 | .dashboard-brand img,
10730 | .brand-logo-img,
10731 | .analytics-brand-logo {
10732 |   max-width: 44px !important;
10733 |   max-height: 44px !important;
10734 |   width: auto !important;
10735 |   height: auto !important;
10736 |   object-fit: contain !important;
10737 | }
10738 |
10739 | .auth-shell .auth-grid img,
10740 | .auth-shell .panel img,
10741 | .auth-shell .signal-panel img,
10742 | .auth-shell .auth-panel img {
10743 |   display: block !important;
10744 |   max-width: 52px !important;
10745 |   max-height: 52px !important;
10746 |   width: auto !important;
10747 |   height: auto !important;
10748 |   object-fit: contain !important;
10749 |   position: static !important;
10750 | }
10751 |
10752 | .auth-shell .auth-grid::before,
10753 | .auth-shell .auth-grid::after,
10754 | .auth-shell .panel::before,
10755 | .auth-shell .panel::after {
10756 |   max-width: min(280px, 42vw) !important;
10757 |   max-height: min(280px, 42vw) !important;
10758 |   pointer-events: none !important;
10759 | }
10760 |
10761 | .admin-topbar .brand-logo-wrapper,
10762 | .dashboard-brand .brand-logo-wrapper {
10763 |   display: grid !important;
10764 |   place-items: center !important;
10765 |   width: 42px !important;
10766 |   height: 42px !important;
10767 |   min-width: 42px !important;
10768 |   overflow: hidden !important;
10769 |   border-radius: 14px !important;
10770 |   background:
10771 |     radial-gradient(circle at 30% 24%, rgba(255, 255, 255, 0.76), transparent 30%),
10772 |     linear-gradient(135deg, #0f172a, #2563eb 58%, #14b8a6) !important;
10773 |   box-shadow: 0 18px 44px rgba(37, 99, 235, 0.16) !important;
10774 | }
10775 |

```

```

10776 | .admin-topbar {
10777 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
10778 |   border-radius: 22px !important;
10779 |   margin: 0 0 24px !important;
10780 |   background: rgba(255, 255, 255, 0.82) !important;
10781 |   box-shadow: 0 18px 48px rgba(15, 23, 42, 0.08) !important;
10782 |   backdrop-filter: blur(18px) !important;
10783 | }
10784 |
10785 | .admin-home-panel {
10786 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
10787 |   border-radius: 28px !important;
10788 |   background:
10789 |     radial-gradient(circle at 86% 0%, rgba(37, 99, 235, 0.12), transparent 28%),
10790 |     linear-gradient(145deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.9)) !important;
10791 |   box-shadow:
10792 |     0 30px 90px rgba(15, 23, 42, 0.1),
10793 |     inset 0 1px 0 rgba(255, 255, 255, 0.9) !important;
10794 | }
10795 |
10796 | .admin-action-card,
10797 | .admin-dataset-panel,
10798 | .admin-validation-panel,
10799 | .admin-validation-result {
10800 |   border-radius: 22px !important;
10801 | }
10802 |
10803 | /* Emergency admin dashboard visibility rescue. Keep this as the final override. */
10804 | .admin-home-panel.reveal-on-scroll,
10805 | .admin-home-panel,
10806 | .admin-home-panel * {
10807 |   opacity: 1 !important;
10808 |   visibility: visible !important;
10809 | }
10810 |
10811 | .admin-home-panel.reveal-on-scroll,
10812 | .admin-home-panel {
10813 |   display: flex !important;
10814 |   transform: none !important;
10815 |   animation: none !important;
10816 |   overflow: visible !important;
10817 | }
10818 |
10819 | .admin-home-header,
10820 | .admin-dataset-panel,
10821 | .admin-action-grid,
10822 | .admin-action-card,
10823 | .admin-validation-panel,
10824 | .admin-validation-result {
10825 |   opacity: 1 !important;
10826 |   visibility: visible !important;
10827 |   transform: none !important;
10828 |   animation: none !important;
10829 | }
10830 |
10831 | .admin-action-grid {
10832 |   display: grid !important;
10833 |   grid-template-columns: repeat(3, minmax(0, 1fr)) !important;
10834 |   gap: 20px !important;
10835 | }
10836 |
10837 | .admin-action-card {
10838 |   display: flex !important;
10839 |   min-height: 270px !important;
10840 |   background: #ffffff !important;
10841 |   color: #0f172a !important;
10842 | }
10843 |
10844 | .admin-action-card strong {
10845 |   color: #0f172a !important;
10846 | }
10847 |
10848 | .admin-action-card p,
10849 | .admin-action-card span {
10850 |   color: #475569 !important;
10851 | }
10852 |
10853 | .admin-action-card em {
10854 |   color: #334155 !important;
10855 | }
10856 |
10857 | /* Final admin text cleanup and soft-card finish. */
10858 | .admin-console-label,
10859 | .admin-home-eyebrow,
10860 | .admin-home-subheading,
10861 | .dataset-status-badge,
10862 | .admin-footer-sig {
10863 |   display: none !important;
10864 | }
10865 |
10866 | .admin-home-header {
10867 |   display: grid !important;
10868 |   justify-items: center !important;
10869 |   max-width: 100% !important;
10870 |   text-align: center !important;
10871 | }
10872 |
10873 | .admin-dashboard-title {
10874 |   position: relative !important;
10875 |   margin: 0 auto 4px !important;
10876 |   text-align: center !important;
10877 |   font-size: clamp(2.6rem, 5.4vw, 4.5rem) !important;
10878 |   font-weight: 950 !important;
10879 |   line-height: 1.02 !important;
10880 |   letter-spacing: -0.045em !important;
10881 |   background: linear-gradient(94deg, #0f172a 0%, #2563eb 48%, #0f766e 100%) !important;
10882 |   background-size: 180% 180% !important;
10883 |   background-clip: text !important;
10884 |   -webkit-background-clip: text !important;
10885 |   color: transparent !important;

```

```

10886 | -webkit-text-fill-color: transparent !important;
10887 | animation: adminTitleGlow 4.8s ease-in-out infinite alternate !important;
10888 | }
10889 |
10890 | .admin-dashboard-title::after {
10891 |   content: '' !important;
10892 |   position: absolute !important;
10893 |   left: 50% !important;
10894 |   bottom: -14px !important;
10895 |   width: min(220px, 52vw) !important;
10896 |   height: 3px !important;
10897 |   border-radius: 999px !important;
10898 |   background: linear-gradient(90deg, transparent, rgba(37, 99, 235, 0.72), rgba(20, 184, 166, 0.7), transparent) !important;
10899 |   transform: translateX(-50%) !important;
10900 |   animation: adminTitleLine 2.8s ease-in-out infinite !important;
10901 | }
10902 |
10903 | @keyframes adminTitleGlow {
10904 |   from {
10905 |     background-position: 0% 50%;
10906 |     filter: drop-shadow(0 8px 22px rgba(37, 99, 235, 0.06));
10907 |   }
10908 |   to {
10909 |     background-position: 100% 50%;
10910 |     filter: drop-shadow(0 12px 34px rgba(20, 184, 166, 0.14));
10911 |   }
10912 | }
10913 |
10914 | @keyframes adminTitleLine {
10915 |   0%,
10916 |   100% {
10917 |     opacity: 0.55;
10918 |     transform: translateX(-50%) scaleX(0.72);
10919 |   }
10920 |   50% {
10921 |     opacity: 1;
10922 |     transform: translateX(-50%) scaleX(1);
10923 |   }
10924 | }
10925 |
10926 | .admin-action-card,
10927 | .admin-dataset-panel,
10928 | .admin-validation-panel,
10929 | .admin-validation-result,
10930 | .drawer-dataset-card,
10931 | .admin-profile-trigger,
10932 | .admin-profile-dropdown {
10933 |   border: 1px solid rgba(15, 23, 42, 0.075) !important;
10934 | }
10935 |
10936 | .admin-action-card {
10937 |   box-shadow:
10938 |     0 18px 48px rgba(15, 23, 42, 0.065),
10939 |     inset 0 1px 0 rgba(255, 255, 255, 0.92) !important;
10940 | }
10941 |
10942 | .admin-action-card:hover {
10943 |   border-color: rgba(37, 99, 235, 0.2) !important;
10944 | }
10945 |
10946 | /* Final admin home experience polish: cream landing-style motion. */
10947 | .app-shell:has(.admin-home-panel) {
10948 |   background:
10949 |     radial-gradient(circle at 12% 18%, rgba(37, 99, 235, 0.12), transparent 28%),
10950 |     radial-gradient(circle at 84% 8%, rgba(20, 184, 166, 0.12), transparent 26%),
10951 |     radial-gradient(circle at 72% 92%, rgba(244, 114, 182, 0.1), transparent 28%),
10952 |     linear-gradient(135deg, #fbf7ef 0%, #f8fafc 46%, #eef6ff 100%) !important;
10953 |   background-size: 140% 140% !important;
10954 |   animation: adminCreamDrift 18s ease-in-out infinite alternate !important;
10955 | }
10956 |
10957 | @keyframes adminCreamDrift {
10958 |   from {
10959 |     background-position: 0% 0%;
10960 |   }
10961 |   to {
10962 |     background-position: 100% 70%;
10963 |   }
10964 | }
10965 |
10966 | .admin-home-panel {
10967 |   overflow: hidden !important;
10968 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
10969 |   border-radius: 30px !important;
10970 |   background:
10971 |     linear-gradient(145deg, rgba(255, 255, 255, 0.82), rgba(255, 251, 244, 0.74)),
10972 |     radial-gradient(circle at 86% 0%, rgba(37, 99, 235, 0.13), transparent 30%) !important;
10973 |   backdrop-filter: blur(22px) !important;
10974 |   box-shadow:
10975 |     0 34px 100px rgba(15, 23, 42, 0.1),
10976 |     inset 0 1px 0 rgba(255, 255, 255, 0.96) !important;
10977 | }
10978 |
10979 | .admin-home-panel::before {
10980 |   content: '' !important;
10981 |   position: absolute !important;
10982 |   inset: 0 !important;
10983 |   z-index: 0 !important;
10984 |   pointer-events: none !important;
10985 |   background-image:
10986 |     linear-gradient(rgba(15, 23, 42, 0.035) 1px, transparent 1px),
10987 |     linear-gradient(90deg, rgba(15, 23, 42, 0.035) 1px, transparent 1px) !important;
10988 |   background-size: 42px 42px !important;
10989 |   mask-image: linear-gradient(to bottom, rgba(0, 0, 0, 0.52), transparent 82%) !important;
10990 |   animation: adminGridFloat 22s linear infinite !important;
10991 | }
10992 |
10993 | @keyframes adminGridFloat {
10994 |   from {
10995 |     background-position: 0 0;

```

```

10996 | }
10997 | to {
10998 |     background-position: 84px 42px;
10999 | }
11000 | }
11001 |
11002 | .admin-grid-lines,
11003 | .admin-gradient-orb {
11004 |     z-index: 0 !important;
11005 | }
11006 |
11007 | .admin-home-header,
11008 | .admin-dataset-panel,
11009 | .admin-action-grid,
11010 | .admin-validation-panel,
11011 | .admin-validation-result {
11012 |     position: relative !important;
11013 |     z-index: 2 !important;
11014 | }
11015 |
11016 | .admin-home-header {
11017 |     animation: adminFadeUp 0.55s cubic-bezier(0.16, 1, 0.3, 1) both !important;
11018 | }
11019 |
11020 | .admin-dataset-panel {
11021 |     animation: adminFadeUp 0.62s cubic-bezier(0.16, 1, 0.3, 1) 0.08s both !important;
11022 | }
11023 |
11024 | .admin-action-card {
11025 |     border: 1px solid rgba(15, 23, 42, 0.08) !important;
11026 |     border-radius: 24px !important;
11027 |     background:
11028 |         linear-gradient(180deg, rgba(255, 255, 255, 0.96), rgba(248, 250, 252, 0.86)) !important;
11029 |     box-shadow:
11030 |         0 18px 46px rgba(15, 23, 42, 0.07),
11031 |         inset 0 1px 0 rgba(255, 255, 255, 0.92) !important;
11032 |     transition:
11033 |         transform 260ms cubic-bezier(0.16, 1, 0.3, 1),
11034 |         box-shadow 260ms cubic-bezier(0.16, 1, 0.3, 1),
11035 |         border-color 260ms ease,
11036 |         background 260ms ease !important;
11037 | }
11038 |
11039 | .admin-action-card:nth-child(1) {
11040 |     animation: adminFadeUp 0.66s cubic-bezier(0.16, 1, 0.3, 1) 0.16s both !important;
11041 | }
11042 |
11043 | .admin-action-card:nth-child(2) {
11044 |     animation: adminFadeUp 0.66s cubic-bezier(0.16, 1, 0.3, 1) 0.24s both !important;
11045 | }
11046 |
11047 | .admin-action-card:nth-child(3) {
11048 |     animation: adminFadeUp 0.66s cubic-bezier(0.16, 1, 0.3, 1) 0.32s both !important;
11049 | }
11050 |
11051 | .admin-validation-panel {
11052 |     animation: adminFadeUp 0.7s cubic-bezier(0.16, 1, 0.3, 1) 0.4s both !important;
11053 | }
11054 |
11055 | @keyframes adminFadeUp {
11056 |     from {
11057 |         opacity: 0.001;
11058 |         transform: translateY(18px) scale(0.985);
11059 |     }
11060 |     to {
11061 |         opacity: 1;
11062 |         transform: translateY(0) scale(1);
11063 |     }
11064 | }
11065 |
11066 | .admin-action-card:hover {
11067 |     border-color: rgba(37, 99, 235, 0.26) !important;
11068 |     background:
11069 |         radial-gradient(circle at 86% 8%, rgba(37, 99, 235, 0.1), transparent 32%),
11070 |         linear-gradient(180deg, #ffffff, #f8fafc) !important;
11071 |     box-shadow:
11072 |         0 30px 82px rgba(37, 99, 235, 0.15),
11073 |         0 12px 30px rgba(15, 23, 42, 0.07) !important;
11074 |     transform: translateY(-8px) scale(1.015) !important;
11075 | }
11076 |
11077 | .admin-action-card:hover .action-icon-wrapper {
11078 |     color: #ffffff !important;
11079 |     box-shadow: 0 20px 52px rgba(37, 99, 235, 0.2) !important;
11080 |     transform: translateY(-3px) rotate(-2deg) scale(1.04) !important;
11081 | }
11082 |
11083 | .admin-action-card:hover .action-card-accent {
11084 |     width: 7px !important;
11085 |     box-shadow: 0 0 28px rgba(37, 99, 235, 0.28) !important;
11086 | }
11087 |
11088 | .admin-dataset-panel,
11089 | .admin-validation-panel {
11090 |     transition:
11091 |         transform 240ms cubic-bezier(0.16, 1, 0.3, 1),
11092 |         box-shadow 240ms cubic-bezier(0.16, 1, 0.3, 1),
11093 |         border-color 240ms ease !important;
11094 | }
11095 |
11096 | .admin-dataset-panel:hover,
11097 | .admin-validation-panel:hover {
11098 |     border-color: rgba(37, 99, 235, 0.2) !important;
11099 |     box-shadow: 0 26px 70px rgba(37, 99, 235, 0.12) !important;
11100 |     transform: translateY(-4px) !important;
11101 | }
11102 |
11103 | .btn-admin-choose-dataset:hover,
11104 | .btn-admin-refresh-datasets:hover,
11105 | .btn-admin-validation:hover:not(:disabled) {

```

```

11106 | transform: translateY(-3px) !important;
11107 | box-shadow: 0 18px 42px rgba(37, 99, 235, 0.16) !important;
11108 | }
11109 |
11110 | @media (prefers-reduced-motion: reduce) {
11111 |   .app-shell:has(.admin-home-panel),
11112 |   .admin-home-panel:~before,
11113 |   .admin-home-header,
11114 |   .admin-dataset-panel,
11115 |   .admin-action-card,
11116 |   .admin-validation-panel {
11117 |     animation: none !important;
11118 |   }
11119 | }
11120 |
11121 | /* Final audit log admin activity screen. */
11122 | .app-shell:has(.audit-workspace) {
11123 |   background:
11124 |     radial-gradient(circle at 10% 12%, rgba(37, 99, 235, 0.12), transparent 28%),
11125 |     radial-gradient(circle at 88% 10%, rgba(20, 184, 166, 0.12), transparent 25%),
11126 |     linear-gradient(135deg, #fbf7ef 0%, #f8fafc 48%, #eef6ff 100%) !important;
11127 | }
11128 |
11129 | .audit-workspace {
11130 |   max-width: 1180px !important;
11131 |   margin: 0 auto !important;
11132 |   width: 100% !important;
11133 | }
11134 |
11135 | .audit-hero-panel,
11136 | .audit-feed-panel,
11137 | .audit-detail-panel {
11138 |   position: relative !important;
11139 |   overflow: hidden !important;
11140 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
11141 |   border-radius: 28px !important;
11142 |   background:
11143 |     linear-gradient(145deg, rgba(255, 255, 255, 0.94), rgba(255, 251, 244, 0.78)),
11144 |     radial-gradient(circle at 92% 0%, rgba(37, 99, 235, 0.09), transparent 35%) !important;
11145 |   box-shadow:
11146 |     0 24px 70px rgba(15, 23, 42, 0.085),
11147 |     inset 0 1px 0 rgba(255, 255, 255, 0.95) !important;
11148 |   color: #0f172a !important;
11149 | }
11150 |
11151 | .audit-hero-panel {
11152 |   display: flex !important;
11153 |   align-items: center !important;
11154 |   justify-content: space-between !important;
11155 |   gap: 22px !important;
11156 |   padding: 32px !important;
11157 |   animation: adminFadeUp 0.5s cubic-bezier(0.16, 1, 0.3, 1) both !important;
11158 | }
11159 |
11160 | .audit-hero-panel h2 {
11161 |   margin: 0 0 10px !important;
11162 |   color: #0f172a !important;
11163 |   font-size: clamp(2rem, 4.2vw, 3.4rem) !important;
11164 |   font-weight: 950 !important;
11165 |   letter-spacing: -0.04em !important;
11166 | }
11167 |
11168 | .audit-hero-panel span,
11169 | .audit-feed-panel span,
11170 | .audit-detail-panel p,
11171 | .audit-detail-panel em {
11172 |   color: #64748b !important;
11173 |   font-weight: 650 !important;
11174 | }
11175 |
11176 | .audit-refresh-button {
11177 |   display: inline-flex !important;
11178 |   align-items: center !important;
11179 |   justify-content: center !important;
11180 |   gap: 9px !important;
11181 |   min-height: 44px !important;
11182 |   padding: 0 18px !important;
11183 |   border: 1px solid rgba(37, 99, 235, 0.18) !important;
11184 |   border-radius: 999px !important;
11185 |   background: #ffffff !important;
11186 |   color: #2563eb !important;
11187 |   font-weight: 850 !important;
11188 |   box-shadow: 0 14px 32px rgba(37, 99, 235, 0.1) !important;
11189 |   transition:
11190 |     transform 220ms cubic-bezier(0.16, 1, 0.3, 1),
11191 |     box-shadow 220ms ease,
11192 |     border-color 220ms ease !important;
11193 | }
11194 |
11195 | .audit-refresh-button:hover {
11196 |   border-color: rgba(37, 99, 235, 0.35) !important;
11197 |   box-shadow: 0 20px 46px rgba(37, 99, 235, 0.16) !important;
11198 |   transform: translateY(-3px) !important;
11199 | }
11200 |
11201 | .audit-log-grid {
11202 |   display: grid !important;
11203 |   grid-template-columns: minmax(0, 1.05fr) minmax(360px, 0.95fr) !important;
11204 |   gap: 24px !important;
11205 |   align-items: start !important;
11206 | }
11207 |
11208 | .audit-feed-panel,
11209 | .audit-detail-panel {
11210 |   padding: 26px !important;
11211 |   animation: adminFadeUp 0.66s cubic-bezier(0.16, 1, 0.3, 1) 0.08s both !important;
11212 | }
11213 |
11214 | .audit-detail-panel {
11215 |   position: sticky !important;

```

```

11216 | top: 24px !important;
11217 | animation-delay: 0.16s !important;
11218 | }
11219 |
11220 | .audit-feed-header {
11221 |   display: flex !important;
11222 |   align-items: center !important;
11223 |   justify-content: space-between !important;
11224 |   gap: 16px !important;
11225 |   margin-bottom: 18px !important;
11226 | }
11227 |
11228 | .audit-feed-header h3,
11229 | .audit-detail-panel h3 {
11230 |   margin: 0 !important;
11231 |   color: #0f172a !important;
11232 |   font-size: 1.38rem !important;
11233 |   font-weight: 930 !important;
11234 |   letter-spacing: -0.025em !important;
11235 | }
11236 |
11237 | .audit-feed-header svg {
11238 |   color: #10b981 !important;
11239 |   filter: drop-shadow(0 12px 20px rgba(16, 185, 129, 0.18)) !important;
11240 | }
11241 |
11242 | .audit-timeline {
11243 |   display: grid !important;
11244 |   gap: 12px !important;
11245 | }
11246 |
11247 | .audit-log-row {
11248 |   display: grid !important;
11249 |   grid-template-columns: 14px minmax(0, 1fr) auto !important;
11250 |   align-items: center !important;
11251 |   gap: 14px !important;
11252 |   min-height: 76px !important;
11253 |   padding: 14px 16px !important;
11254 |   border: 1px solid rgba(15, 23, 42, 0.07) !important;
11255 |   border-radius: 20px !important;
11256 |   background: rgba(255, 255, 255, 0.84) !important;
11257 |   text-align: left !important;
11258 |   box-shadow: 0 10px 28px rgba(15, 23, 42, 0.04) !important;
11259 |   transition:
11260 |     transform 220ms cubic-bezier(0.16, 1, 0.3, 1),
11261 |     border-color 220ms ease,
11262 |     box-shadow 220ms ease,
11263 |     background 220ms ease !important;
11264 | }
11265 |
11266 | .audit-log-row:hover,
11267 | .audit-log-row.selected {
11268 |   border-color: rgba(37, 99, 235, 0.24) !important;
11269 |   background: #ffffff !important;
11270 |   box-shadow: 0 18px 42px rgba(37, 99, 235, 0.11) !important;
11271 |   transform: translateY(-3px) !important;
11272 | }
11273 |
11274 | .audit-log-row.selected {
11275 |   box-shadow:
11276 |     0 20px 48px rgba(37, 99, 235, 0.14),
11277 |     inset 4px 0 0 #2563eb !important;
11278 | }
11279 |
11280 | .audit-dot {
11281 |   width: 10px !important;
11282 |   height: 10px !important;
11283 |   border-radius: 999px !important;
11284 |   background: #2563eb !important;
11285 |   box-shadow: 0 0 0 5px rgba(37, 99, 235, 0.12) !important;
11286 | }
11287 |
11288 | .audit-log-row strong {
11289 |   display: block !important;
11290 |   color: #0f172a !important;
11291 |   font-size: 0.98rem !important;
11292 |   font-weight: 900 !important;
11293 |   line-height: 1.3 !important;
11294 | }
11295 |
11296 | .audit-log-row span {
11297 |   display: block !important;
11298 |   margin-top: 4px !important;
11299 |   font-size: 0.82rem !important;
11300 | }
11301 |
11302 | .audit-log-row em,
11303 | .audit-action-chip {
11304 |   width: max-content !important;
11305 |   border: 1px solid rgba(37, 99, 235, 0.14) !important;
11306 |   border-radius: 999px !important;
11307 |   background: rgba(37, 99, 235, 0.07) !important;
11308 |   color: #2563eb !important;
11309 |   padding: 7px 10px !important;
11310 |   font-size: 0.74rem !important;
11311 |   font-style: normal !important;
11312 |   font-weight: 900 !important;
11313 |   white-space: nowrap !important;
11314 | }
11315 |
11316 | .audit-see-more-button {
11317 |   display: inline-flex !important;
11318 |   align-items: center !important;
11319 |   justify-content: center !important;
11320 |   gap: 12px !important;
11321 |   min-height: 48px !important;
11322 |   margin-top: 6px !important;
11323 |   border: 1px solid rgba(37, 99, 235, 0.16) !important;
11324 |   border-radius: 18px !important;
11325 |   background:

```



```

11326 |     linear-gradient(180deg, rgba(255, 255, 255, 0.96), rgba(248, 250, 252, 0.88)) !important;
11327 |     color: #2563eb !important;
11328 |     font-weight: 920 !important;
11329 |     box-shadow: 0 14px 34px rgba(37, 99, 235, 0.08) !important;
11330 |     transition:
11331 |       transform 220ms cubic-bezier(0.16, 1, 0.3, 1),
11332 |       border-color 220ms ease,
11333 |       box-shadow 220ms ease !important;
11334 |   }
11335 |
11336 |   .audit-see-more-button:hover {
11337 |     border-color: rgba(37, 99, 235, 0.34) !important;
11338 |     box-shadow: 0 22px 48px rgba(37, 99, 235, 0.14) !important;
11339 |     transform: translateY(-3px) !important;
11340 |   }
11341 |
11342 |   .audit-see-more-button span {
11343 |     border-radius: 999px !important;
11344 |     background: rgba(37, 99, 235, 0.08) !important;
11345 |     color: #475569 !important;
11346 |     padding: 6px 9px !important;
11347 |     font-size: 0.76rem !important;
11348 |     font-weight: 850 !important;
11349 |   }
11350 |
11351 |   .audit-detail-topline {
11352 |     display: flex !important;
11353 |     align-items: center !important;
11354 |     justify-content: space-between !important;
11355 |     gap: 14px !important;
11356 |     margin-bottom: 18px !important;
11357 |   }
11358 |
11359 |   .audit-detail-topline em {
11360 |     font-size: 0.85rem !important;
11361 |     font-style: normal !important;
11362 |   }
11363 |
11364 |   .audit-detail-panel > p {
11365 |     margin: 12px 0 22px !important;
11366 |     line-height: 1.65 !important;
11367 |   }
11368 |
11369 |   .audit-detail-metrics {
11370 |     display: grid !important;
11371 |     grid-template-columns: repeat(3, minmax(0, 1fr)) !important;
11372 |     gap: 12px !important;
11373 |     margin: 18px 0 !important;
11374 |   }
11375 |
11376 |   .audit-detail-metrics div,
11377 |   .audit-metadata-panel {
11378 |     border: 1px solid rgba(15, 23, 42, 0.07) !important;
11379 |     border-radius: 18px !important;
11380 |     background: rgba(248, 250, 252, 0.8) !important;
11381 |     padding: 14px !important;
11382 |   }
11383 |
11384 |   .audit-detail-metrics span,
11385 |   .audit-metadata-panel span {
11386 |     display: block !important;
11387 |     color: #64748b !important;
11388 |     font-size: 0.72rem !important;
11389 |     font-weight: 900 !important;
11390 |     letter-spacing: 0.08em !important;
11391 |     text-transform: uppercase !important;
11392 |   }
11393 |
11394 |   .audit-detail-metrics strong,
11395 |   .audit-metadata-panel strong {
11396 |     display: block !important;
11397 |     margin-top: 6px !important;
11398 |     color: #0f172a !important;
11399 |     font-weight: 920 !important;
11400 |   }
11401 |
11402 |   .audit-detail-metrics em {
11403 |     display: block !important;
11404 |     margin-top: 5px !important;
11405 |     overflow: hidden !important;
11406 |     text-overflow: ellipsis !important;
11407 |     font-size: 0.78rem !important;
11408 |     font-style: normal !important;
11409 |     white-space: nowrap !important;
11410 |   }
11411 |
11412 |   .audit-metadata-panel {
11413 |     display: grid !important;
11414 |     gap: 10px !important;
11415 |     margin-top: 16px !important;
11416 |   }
11417 |
11418 |   .audit-metadata-panel > strong {
11419 |     margin: 0 0 4px !important;
11420 |   }
11421 |
11422 |   .audit-metadata-panel div {
11423 |     display: flex !important;
11424 |     align-items: flex-start !important;
11425 |     justify-content: space-between !important;
11426 |     gap: 16px !important;
11427 |     border-top: 1px solid rgba(15, 23, 42, 0.06) !important;
11428 |     padding-top: 10px !important;
11429 |   }
11430 |
11431 |   .audit-metadata-panel em {
11432 |     max-width: 66% !important;
11433 |     color: #334155 !important;
11434 |     font-size: 0.86rem !important;
11435 |     font-style: normal !important;

```

```

11436 | font-weight: 750 !important;
11437 | text-align: right !important;
11438 | word-break: break-word !important;
11439 | }
11440 |
11441 | .audit-metadata-panel p {
11442 |   margin: 0 !important;
11443 |   color: #64748b !important;
11444 | }
11445 |
11446 | .audit-empty-state {
11447 |   display: grid !important;
11448 |   justify-items: center !important;
11449 |   gap: 8px !important;
11450 |   min-height: 220px !important;
11451 |   align-content: center !important;
11452 |   border: 1px dashed rgba(15, 23, 42, 0.12) !important;
11453 |   border-radius: 20px !important;
11454 |   background: rgba(255, 255, 255, 0.64) !important;
11455 |   color: #64748b !important;
11456 |   text-align: center !important;
11457 | }
11458 |
11459 | .audit-empty-state strong {
11460 |   color: #0f172a !important;
11461 |   font-weight: 920 !important;
11462 | }
11463 |
11464 | .action-card-badge.audit {
11465 |   border-color: rgba(37, 99, 235, 0.16) !important;
11466 |   background: rgba(37, 99, 235, 0.08) !important;
11467 |   color: #2563eb !important;
11468 | }
11469 |
11470 | @media (max-width: 900px) {
11471 |   .audit-hero-panel,
11472 |   .audit-log-grid {
11473 |     grid-template-columns: 1fr !important;
11474 |   }
11475 |
11476 |   .audit-hero-panel {
11477 |     align-items: flex-start !important;
11478 |     flex-direction: column !important;
11479 |   }
11480 |
11481 |   .audit-detail-panel {
11482 |     position: static !important;
11483 |   }
11484 |
11485 |   .audit-detail-metrics {
11486 |     grid-template-columns: 1fr !important;
11487 |   }
11488 |
11489 |   .audit-log-row {
11490 |     grid-template-columns: 12px minmax(0, 1fr) !important;
11491 |   }
11492 |
11493 |   .audit-log-row em {
11494 |     grid-column: 2 !important;
11495 |   }
11496 | }
11497 |
11498 | /* Final visual consistency pass: square logo, compact admin home, clearer live state. */
11499 | .brand-logo-wrapper,
11500 | .dashboard-brand .brand-logo-wrapper,
11501 | .admin-topbar .brand-logo-wrapper,
11502 | .analytics-brand .brand-logo-wrapper {
11503 |   display: grid !important;
11504 |   place-items: center !important;
11505 |   width: 44px !important;
11506 |   height: 44px !important;
11507 |   min-width: 44px !important;
11508 |   overflow: hidden !important;
11509 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
11510 |   border-radius: 12px !important;
11511 |   background: #ffffff !important;
11512 |   box-shadow: 0 14px 32px rgba(37, 99, 235, 0.12) !important;
11513 | }
11514 |
11515 | .brand-logo-img,
11516 | .analytics-brand-logo,
11517 | .student-hero-logo,
11518 | .student-hero-logo-img,
11519 | .student-logo-stage img,
11520 | .dashboard-brand img,
11521 | .admin-topbar img,
11522 | .topbar img {
11523 |   width: 38px !important;
11524 |   height: 38px !important;
11525 |   max-width: 38px !important;
11526 |   max-height: 38px !important;
11527 |   aspect-ratio: 1 / 1 !important;
11528 |   border-radius: 8px !important;
11529 |   object-fit: contain !important;
11530 |   background: #ffffff !important;
11531 | }
11532 |
11533 | .dashboard-brand,
11534 | .brand-wrapper,
11535 | .analytics-brand {
11536 |   color: #0f172a !important;
11537 | }
11538 |
11539 | .brand-text,
11540 | .dashboard-brand .brand-text,
11541 | .brand-wrapper > span,
11542 | .analytics-brand {
11543 |   color: #0f172a !important;
11544 |   font-weight: 950 !important;
11545 |   letter-spacing: -0.02em !important;

```

```

11546 | -webkit-text-fill-color: #0f172a !important;
11547 | }
11548 |
11549 | .app-shell:has(.admin-home-panel) {
11550 |   height: 100vh !important;
11551 |   min-height: 100vh !important;
11552 |   overflow: hidden !important;
11553 |   padding: 12px 18px 10px !important;
11554 | }
11555 |
11556 | .workspace:has(.admin-home-panel) {
11557 |   display: flex !important;
11558 |   flex-direction: column !important;
11559 |   gap: 10px !important;
11560 |   width: min(1160px, 100%) !important;
11561 |   max-height: calc(100vh - 46px) !important;
11562 |   margin: 0 auto !important;
11563 | }
11564 |
11565 | .app-shell:has(.admin-home-panel) .admin-topbar {
11566 |   min-height: 56px !important;
11567 |   margin: 0 !important;
11568 |   border-radius: 18px !important;
11569 |   padding: 8px 12px !important;
11570 | }
11571 |
11572 | .app-shell:has(.admin-home-panel) .team-eureka-badge {
11573 |   flex: 0 0 auto !important;
11574 |   margin: 6px auto 0 !important;
11575 |   color: #334155 !important;
11576 |   font-size: 0.74rem !important;
11577 |   font-weight: 850 !important;
11578 |   letter-spacing: 0.04em !important;
11579 | }
11580 |
11581 | .app-shell:has(.admin-home-panel) .team-eureka-badge svg {
11582 |   color: #22c55e !important;
11583 | }
11584 |
11585 | .admin-home-panel {
11586 |   flex: 1 1 auto !important;
11587 |   min-height: 0 !important;
11588 |   max-height: calc(100vh - 128px) !important;
11589 |   gap: 14px !important;
11590 |   padding: 18px !important;
11591 |   overflow: hidden !important;
11592 |   border-radius: 24px !important;
11593 |   background:
11594 |     radial-gradient(circle at 10% 10%, rgba(37, 99, 235, 0.12), transparent 30%),
11595 |     radial-gradient(circle at 92% 12%, rgba(20, 184, 166, 0.16), transparent 28%),
11596 |     radial-gradient(circle at 78% 92%, rgba(249, 115, 22, 0.1), transparent 30%),
11597 |     linear-gradient(145deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.9)) !important;
11598 | }
11599 |
11600 | .admin-dashboard-title {
11601 |   font-size: clamp(2rem, 4.4vw, 3.35rem) !important;
11602 | }
11603 |
11604 | .admin-home-header {
11605 |   gap: 4px !important;
11606 | }
11607 |
11608 | .admin-dataset-panel {
11609 |   min-height: 74px !important;
11610 |   padding: 14px 16px !important;
11611 |   border-radius: 20px !important;
11612 |   background:
11613 |     linear-gradient(135deg, rgba(239, 246, 255, 0.94), rgba(236, 253, 245, 0.86)),
11614 |     #ffffff !important;
11615 | }
11616 |
11617 | .dataset-label {
11618 |   color: #2563eb !important;
11619 | }
11620 |
11621 | .dataset-value-name {
11622 |   color: #0f172a !important;
11623 |   font-size: clamp(1rem, 1.8vw, 1.28rem) !important;
11624 | }
11625 |
11626 | .admin-action-grid {
11627 |   flex: 1 1 auto !important;
11628 |   min-height: 0 !important;
11629 |   gap: 14px !important;
11630 | }
11631 |
11632 | .admin-action-card {
11633 |   position: relative !important;
11634 |   min-height: 0 !important;
11635 |   height: 100% !important;
11636 |   max-height: 230px !important;
11637 |   justify-content: flex-start !important;
11638 |   gap: 8px !important;
11639 |   padding: 18px !important;
11640 |   border-radius: 22px !important;
11641 | }
11642 |
11643 | .admin-action-card.create-card {
11644 |   background:
11645 |     radial-gradient(circle at 90% 10%, rgba(34, 197, 94, 0.16), transparent 34%),
11646 |     linear-gradient(180deg, #ffffff, #f0fdf4) !important;
11647 | }
11648 |
11649 | .admin-action-card.analysis-card {
11650 |   background:
11651 |     radial-gradient(circle at 90% 10%, rgba(37, 99, 235, 0.16), transparent 34%),
11652 |     linear-gradient(180deg, #ffffff, #eff6ff) !important;
11653 | }
11654 |
11655 | .admin-action-card.audit-card {

```

```

11656 | background:
11657 |     radial-gradient(circle at 90% 10%, rgba(249, 115, 22, 0.16), transparent 34%),
11658 |     linear-gradient(180deg, #ffffff, #fff7ed) !important;
11659 | }
11660 |
11661 | .action-icon-wrapper {
11662 |     width: 48px !important;
11663 |     height: 48px !important;
11664 |     border-radius: 15px !important;
11665 |     color: #ffffff !important;
11666 | }
11667 |
11668 | .create-card .action-icon-wrapper {
11669 |     background: linear-gradient(135deg, #16a34a, #22c55e) !important;
11670 | }
11671 |
11672 | .analysis-card .action-icon-wrapper {
11673 |     background: linear-gradient(135deg, #1d4ed8, #38bdf8) !important;
11674 | }
11675 |
11676 | .audit-card .action-icon-wrapper {
11677 |     background: linear-gradient(135deg, #ea580c, #f59e0b) !important;
11678 | }
11679 |
11680 | .admin-action-card strong {
11681 |     font-size: clamp(1.18rem, 2vw, 1.55rem) !important;
11682 | }
11683 |
11684 | .admin-action-card p {
11685 |     max-width: 92% !important;
11686 |     margin: 0 !important;
11687 |     font-size: 0.9rem !important;
11688 |     line-height: 1.42 !important;
11689 | }
11690 |
11691 | .action-card-subtitle {
11692 |     font-size: 0.8rem !important;
11693 |     font-weight: 850 !important;
11694 | }
11695 |
11696 | .action-card-badge {
11697 |     margin-top: auto !important;
11698 | }
11699 |
11700 | .action-card-badge.live {
11701 |     border-color: rgba(34, 197, 94, 0.22) !important;
11702 |     background: rgba(34, 197, 94, 0.1) !important;
11703 |     color: #15803d !important;
11704 | }
11705 |
11706 | .action-card-badge.closed,
11707 | .feedback-live-status.closed {
11708 |     border-color: rgba(249, 115, 22, 0.26) !important;
11709 |     background: linear-gradient(135deg, rgba(255, 247, 237, 0.98), rgba(255, 255, 255, 0.9)) !important;
11710 |     color: #c2410c !important;
11711 | }
11712 |
11713 | .admin-card-live-mini {
11714 |     position: absolute !important;
11715 |     top: 16px !important;
11716 |     right: 16px !important;
11717 |     display: inline-flex !important;
11718 |     align-items: center !important;
11719 |     gap: 6px !important;
11720 |     border-radius: 999px !important;
11721 |     padding: 6px 10px !important;
11722 |     font-size: 0.72rem !important;
11723 |     font-weight: 950 !important;
11724 | }
11725 |
11726 | .admin-card-live-mini i {
11727 |     width: 8px !important;
11728 |     height: 8px !important;
11729 |     border-radius: 999px !important;
11730 | }
11731 |
11732 | .admin-card-live-mini.live {
11733 |     background: rgba(34, 197, 94, 0.12) !important;
11734 |     color: #15803d !important;
11735 | }
11736 |
11737 | .admin-card-live-mini.live i {
11738 |     background: #22c55e !important;
11739 |     box-shadow: 0 0 5px rgba(34, 197, 94, 0.14), 0 0 18px rgba(34, 197, 94, 0.7) !important;
11740 |     animation: feedbackLivePulse 1.2s ease-in-out infinite !important;
11741 | }
11742 |
11743 | .admin-card-live-mini.closed {
11744 |     background: rgba(249, 115, 22, 0.12) !important;
11745 |     color: #c2410c !important;
11746 | }
11747 |
11748 | .admin-card-live-mini.closed i {
11749 |     background: #f97316 !important;
11750 |     box-shadow: 0 0 5px rgba(249, 115, 22, 0.13) !important;
11751 | }
11752 |
11753 | .feedback-live-status.closed .feedback-live-light {
11754 |     background: #f97316 !important;
11755 |     box-shadow:
11756 |         0 0 7px rgba(249, 115, 22, 0.13),
11757 |         0 0 20px rgba(249, 115, 22, 0.28) !important;
11758 |     animation: none !important;
11759 | }
11760 |
11761 | .feedback-live-status.closed strong,
11762 | .feedback-live-status.closed em {
11763 |     color: #c2410c !important;
11764 | }
11765 |

```

```

11766 | @media (max-height: 760px) and (min-width: 900px) {
11767 |   .app-shell:has(.admin-home-panel) {
11768 |     padding-top: 8px !important;
11769 |   }
11770 |
11771 |   .workspace:has(.admin-home-panel) {
11772 |     gap: 8px !important;
11773 |     max-height: calc(100vh - 34px) !important;
11774 |   }
11775 |
11776 |   .admin-dashboard-title {
11777 |     font-size: clamp(1.75rem, 4vw, 2.75rem) !important;
11778 |   }
11779 |
11780 |   .admin-home-panel {
11781 |     max-height: calc(100vh - 112px) !important;
11782 |     padding: 14px !important;
11783 |     gap: 10px !important;
11784 |   }
11785 |
11786 |   .admin-dataset-panel {
11787 |     min-height: 62px !important;
11788 |     padding: 10px 14px !important;
11789 |   }
11790 |
11791 |   .admin-action-card {
11792 |     max-height: 194px !important;
11793 |     padding: 14px !important;
11794 |   }
11795 |
11796 |   .action-icon-wrapper {
11797 |     width: 42px !important;
11798 |     height: 42px !important;
11799 |   }
11800 |
11801 |   .admin-action-card p {
11802 |     font-size: 0.82rem !important;
11803 |   }
11804 | }
11805 |
11806 | /* Final admin dashboard balance: calmer color, fuller layout, bigger cards. */
11807 | .app-shell:has(.admin-home-panel) {
11808 |   display: flex !important;
11809 |   flex-direction: column !important;
11810 |   height: 100vh !important;
11811 |   overflow: hidden !important;
11812 |   padding: 12px 18px 8px !important;
11813 |   background:
11814 |     radial-gradient(circle at 12% 10%, rgba(37, 99, 235, 0.07), transparent 30%),
11815 |     radial-gradient(circle at 88% 14%, rgba(20, 184, 166, 0.065), transparent 28%),
11816 |     linear-gradient(135deg, #f8fafc 0%, #ffffff 45%, #f6f9fc 100%) !important;
11817 | }
11818 |
11819 | .workspace:has(.admin-home-panel) {
11820 |   flex: 1 1 auto !important;
11821 |   display: flex !important;
11822 |   flex-direction: column !important;
11823 |   gap: 12px !important;
11824 |   width: min(1180px, 100%) !important;
11825 |   height: auto !important;
11826 |   min-height: 0 !important;
11827 |   max-height: none !important;
11828 | }
11829 |
11830 | .app-shell:has(.admin-home-panel) .admin-topbar {
11831 |   flex: 0 0 auto !important;
11832 |   min-height: 58px !important;
11833 |   background: rgba(255, 255, 255, 0.92) !important;
11834 | }
11835 |
11836 | .admin-home-panel {
11837 |   flex: 1 1 auto !important;
11838 |   display: flex !important;
11839 |   flex-direction: column !important;
11840 |   min-height: 0 !important;
11841 |   max-height: none !important;
11842 |   height: auto !important;
11843 |   gap: 16px !important;
11844 |   padding: 22px !important;
11845 |   overflow: hidden !important;
11846 |   background:
11847 |     linear-gradient(145deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.94)),
11848 |     radial-gradient(circle at 94% 6%, rgba(37, 99, 235, 0.055), transparent 28%) !important;
11849 |   box-shadow:
11850 |     0 24px 70px rgba(15, 23, 42, 0.075),
11851 |     inset 0 1px 0 rgba(255, 255, 255, 0.96) !important;
11852 | }
11853 |
11854 | .admin-gradient-orb {
11855 |   opacity: 0.45 !important;
11856 | }
11857 |
11858 | .admin-dashboard-title {
11859 |   font-size: clamp(2.25rem, 4.6vw, 3.7rem) !important;
11860 | }
11861 |
11862 | .admin-dataset-panel {
11863 |   flex: 0 0 auto !important;
11864 |   min-height: 82px !important;
11865 |   background:
11866 |     linear-gradient(135deg, rgba(255, 255, 255, 0.96), rgba(248, 250, 252, 0.9)) !important;
11867 | }
11868 |
11869 | .admin-action-grid {
11870 |   flex: 1 1 auto !important;
11871 |   display: grid !important;
11872 |   grid-template-columns: repeat(3, minmax(0, 1fr)) !important;
11873 |   align-items: stretch !important;
11874 |   min-height: 0 !important;
11875 |   gap: 18px !important;

```

```

11876 | }
11877 |
11878 | .admin-action-card {
11879 |   min-height: 250px !important;
11880 |   max-height: none !important;
11881 |   height: 100% !important;
11882 |   padding: 24px !important;
11883 |   gap: 11px !important;
11884 |   border-color: rgba(15, 23, 42, 0.08) !important;
11885 |   background:
11886 |     linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(248, 250, 252, 0.92)) !important;
11887 | }
11888 |
11889 | .admin-action-card.create-card,
11890 | .admin-action-card.analysis-card,
11891 | .admin-action-card.audit-card {
11892 |   background:
11893 |     linear-gradient(180deg, rgba(255, 255, 255, 0.99), rgba(248, 250, 252, 0.93)) !important;
11894 | }
11895 |
11896 | .create-card .action-icon-wrapper {
11897 |   background: linear-gradient(135deg, #15803d, #22c55e) !important;
11898 | }
11899 |
11900 | .analysis-card .action-icon-wrapper {
11901 |   background: linear-gradient(135deg, #1d4ed8, #0ea5e9) !important;
11902 | }
11903 |
11904 | .audit-card .action-icon-wrapper {
11905 |   background: linear-gradient(135deg, #334155, #64748b) !important;
11906 | }
11907 |
11908 | .admin-action-card strong {
11909 |   font-size: clamp(1.35rem, 2.2vw, 1.75rem) !important;
11910 | }
11911 |
11912 | .admin-action-card p {
11913 |   max-width: 100% !important;
11914 |   font-size: 0.96rem !important;
11915 |   line-height: 1.5 !important;
11916 | }
11917 |
11918 | .action-icon-wrapper {
11919 |   width: 54px !important;
11920 |   height: 54px !important;
11921 | }
11922 |
11923 | .app-shell:has(.admin-home-panel) .team-eureka-badge {
11924 |   margin: 8px auto 0 !important;
11925 | }
11926 |
11927 | /* Final cross-page EduPulse logo consistency.
11928 |    Matches the compact landing navbar logo exactly. */
11929 | .dashboard-brand,
11930 | .admin-topbar .dashboard-brand,
11931 | .topbar .dashboard-brand,
11932 | .analytics-brand {
11933 |   display: inline-flex !important;
11934 |   align-items: center !important;
11935 |   gap: 12px !important;
11936 | }
11937 |
11938 | .brand-logo-wrapper,
11939 | .dashboard-brand .brand-logo-wrapper,
11940 | .admin-topbar .brand-logo-wrapper,
11941 | .topbar .brand-logo-wrapper,
11942 | .analytics-brand .brand-logo-wrapper {
11943 |   box-sizing: border-box !important;
11944 |   display: grid !important;
11945 |   place-items: center !important;
11946 |   width: 38px !important;
11947 |   height: 38px !important;
11948 |   min-width: 38px !important;
11949 |   max-width: 38px !important;
11950 |   min-height: 38px !important;
11951 |   max-height: 38px !important;
11952 |   overflow: hidden !important;
11953 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
11954 |   border-radius: 8px !important;
11955 |   background: #ffffff !important;
11956 |   box-shadow: 0 12px 28px rgba(37, 99, 235, 0.12) !important;
11957 |   transition:
11958 |     transform 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275),
11959 |     box-shadow 0.3s ease,
11960 |     border-color 0.3s ease !important;
11961 | }
11962 |
11963 | .brand-logo-img,
11964 | .analytics-brand-logo,
11965 | .dashboard-brand img,
11966 | .admin-topbar img,
11967 | .topbar img {
11968 |   width: 100% !important;
11969 |   height: 100% !important;
11970 |   max-width: 100% !important;
11971 |   max-height: 100% !important;
11972 |   aspect-ratio: 1 / 1 !important;
11973 |   border: 0 !important;
11974 |   border-radius: 7px !important;
11975 |   background: #ffffff !important;
11976 |   object-fit: contain !important;
11977 |   box-shadow: none !important;
11978 | }
11979 |
11980 | .dashboard-brand:hover .brand-logo-wrapper,
11981 | .admin-topbar .dashboard-brand:hover .brand-logo-wrapper,
11982 | .topbar .dashboard-brand:hover .brand-logo-wrapper,
11983 | .analytics-brand:hover .brand-logo-wrapper {
11984 |   border-color: rgba(37, 99, 235, 0.22) !important;
11985 |   box-shadow: 0 12px 28px rgba(37, 99, 235, 0.16) !important;

```

```

11986 | transform: scale(1.05) !important;
11987 | }
11988 |
11989 | .app-shell .dashboard-brand .brand-logo-wrapper > img.brand-logo-img,
11990 | .app-shell .topbar .brand-logo-wrapper > img.brand-logo-img,
11991 | .app-shell .admin-topbar .brand-logo-wrapper > img.brand-logo-img,
11992 | .app-shell .analytics-brand .brand-logo-wrapper > img.analytics-brand-logo,
11993 | .auth-shell .brand-logo-wrapper > img.brand-logo-img {
11994 |   width: 38px !important;
11995 |   height: 38px !important;
11996 |   max-width: 38px !important;
11997 |   max-height: 38px !important;
11998 |   min-width: 38px !important;
11999 |   min-height: 38px !important;
12000 |   border-radius: 7px !important;
12001 |   object-fit: contain !important;
12002 | }
12003 |
12004 | /* Final student topbar polish: place brand/logout directly on the page background. */
12005 | .app-shell:has(.student-home) .topbar,
12006 | .app-shell:has(.student-form-wrap) .topbar {
12007 |   position: relative !important;
12008 |   top: auto !important;
12009 |   z-index: 20 !important;
12010 |   width: 100% !important;
12011 |   margin: 0 0 18px !important;
12012 |   padding: 8px 2px !important;
12013 |   border: 0 !important;
12014 |   border-radius: 0 !important;
12015 |   background: transparent !important;
12016 |   box-shadow: none !important;
12017 |   backdrop-filter: none !important;
12018 | }
12019 |
12020 | .app-shell:has(.student-home) .topbar .dashboard-brand,
12021 | .app-shell:has(.student-form-wrap) .topbar .dashboard-brand {
12022 |   padding: 0 !important;
12023 |   background: transparent !important;
12024 |   border: 0 !important;
12025 |   box-shadow: none !important;
12026 | }
12027 |
12028 | .app-shell:has(.student-home) .topbar .brand-text,
12029 | .app-shell:has(.student-form-wrap) .topbar .brand-text {
12030 |   color: #0f172a !important;
12031 |   font-size: 1.08rem !important;
12032 |   font-weight: 950 !important;
12033 |   letter-spacing: -0.02em !important;
12034 | }
12035 |
12036 | .app-shell:has(.student-home) .topbar .icon-button,
12037 | .app-shell:has(.student-form-wrap) .topbar .icon-button,
12038 | .app-shell:has(.student-form-wrap) .topbar .back-button {
12039 |   height: 40px !important;
12040 |   padding: 9px 15px !important;
12041 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
12042 |   border-radius: 10px !important;
12043 |   background: rgba(255, 255, 255, 0.58) !important;
12044 |   color: #0f172a !important;
12045 |   box-shadow: none !important;
12046 |   backdrop-filter: blur(10px) !important;
12047 |   font-weight: 900 !important;
12048 |   transition:
12049 |     transform 0.22s cubic-bezier(0.16, 1, 0.3, 1),
12050 |     border-color 0.22s ease,
12051 |     background 0.22s ease,
12052 |     color 0.22s ease !important;
12053 | }
12054 |
12055 | .app-shell:has(.student-home) .topbar .icon-button:hover,
12056 | .app-shell:has(.student-form-wrap) .topbar .icon-button:hover,
12057 | .app-shell:has(.student-form-wrap) .topbar .back-button:hover {
12058 |   border-color: rgba(37, 99, 235, 0.22) !important;
12059 |   background: rgba(255, 255, 255, 0.86) !important;
12060 |   color: #2563eb !important;
12061 |   transform: translateY(-2px) !important;
12062 |   box-shadow: 0 12px 28px rgba(37, 99, 235, 0.1) !important;
12063 | }
12064 |
12065 | /* Final auth/admin topbar cleanup: no scroll, no separate nav card, no blur. */
12066 | .public-shell.auth-shell {
12067 |   width: 100vw !important;
12068 |   height: 100vh !important;
12069 |   min-height: 100vh !important;
12070 |   max-height: 100vh !important;
12071 |   overflow: hidden !important;
12072 |   display: grid !important;
12073 |   place-items: center !important;
12074 |   padding: 18px 24px 52px !important;
12075 | }
12076 |
12077 | .auth-shell .auth-grid {
12078 |   width: min(980px, 100%) !important;
12079 |   max-height: calc(100vh - 86px) !important;
12080 |   align-items: stretch !important;
12081 |   gap: 16px !important;
12082 |   margin: 0 !important;
12083 | }
12084 |
12085 | .auth-shell .panel {
12086 |   min-height: 0 !important;
12087 |   max-height: calc(100vh - 96px) !important;
12088 |   overflow: hidden !important;
12089 |   padding: 20px !important;
12090 | }
12091 |
12092 | .auth-shell .auth-panel {
12093 |   padding: 20px !important;
12094 | }
12095 |

```

```

12096 | .auth-shell .panel-title {
12097 |   margin-bottom: 14px !important;
12098 | }
12099 |
12100 | .auth-shell .form-stack {
12101 |   gap: 12px !important;
12102 | }
12103 |
12104 | .auth-shell label {
12105 |   gap: 7px !important;
12106 | }
12107 |
12108 | .auth-shell input {
12109 |   min-height: 42px !important;
12110 |   padding: 10px 12px !important;
12111 | }
12112 |
12113 | .auth-shell .primary-button {
12114 |   min-height: 42px !important;
12115 | }
12116 |
12117 | .auth-shell .signal-panel {
12118 |   gap: 12px !important;
12119 |   justify-content: center !important;
12120 | }
12121 |
12122 | .auth-shell .login-orb {
12123 |   width: 78px !important;
12124 |   height: 78px !important;
12125 |   margin: 0 auto 4px !important;
12126 | }
12127 |
12128 | .auth-shell .signal-panel h2 {
12129 |   font-size: clamp(1.35rem, 2.8vw, 2rem) !important;
12130 |   line-height: 1.12 !important;
12131 | }
12132 |
12133 | .auth-shell .signal-row {
12134 |   min-height: 0 !important;
12135 |   padding: 11px 13px !important;
12136 |   gap: 12px !important;
12137 | }
12138 |
12139 | .auth-shell .signal-row svg {
12140 |   width: 20px !important;
12141 |   height: 20px !important;
12142 | }
12143 |
12144 | .auth-shell .auth-back-button,
12145 | .student-form-wrap .auth-back-button,
12146 | .admin-back-button {
12147 |   position: fixed !important;
12148 |   top: 22px !important;
12149 |   left: 24px !important;
12150 |   z-index: 50 !important;
12151 |   height: 40px !important;
12152 |   min-height: 40px !important;
12153 |   padding: 0 6px !important;
12154 |   border: 0 !important;
12155 |   border-radius: 0 !important;
12156 |   background: transparent !important;
12157 |   color: #0f172a !important;
12158 |   box-shadow: none !important;
12159 |   backdrop-filter: none !important;
12160 |   font-weight: 950 !important;
12161 | }
12162 |
12163 | .auth-shell .auth-back-button:hover,
12164 | .student-form-wrap .auth-back-button:hover,
12165 | .admin-back-button:hover {
12166 |   color: #2563eb !important;
12167 |   background: transparent !important;
12168 |   box-shadow: none !important;
12169 |   transform: translateX(-3px) !important;
12170 | }
12171 |
12172 | .app-shell:has(.admin-home-panel) .admin-topbar {
12173 |   background: transparent !important;
12174 |   border: 0 !important;
12175 |   box-shadow: none !important;
12176 |   backdrop-filter: none !important;
12177 |   margin: 0 0 8px !important;
12178 |   padding: 8px 2px !important;
12179 |   border-radius: 0 !important;
12180 | }
12181 |
12182 | .app-shell:has(.admin-home-panel) .admin-profile-trigger {
12183 |   background: rgba(255, 255, 255, 0.58) !important;
12184 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
12185 |   box-shadow: none !important;
12186 |   backdrop-filter: none !important;
12187 |   border-radius: 10px !important;
12188 | }
12189 |
12190 | .app-shell:has(.admin-home-panel) .admin-profile-trigger:hover {
12191 |   background: rgba(255, 255, 255, 0.86) !important;
12192 |   border-color: rgba(37, 99, 235, 0.22) !important;
12193 |   box-shadow: 0 12px 28px rgba(37, 99, 235, 0.1) !important;
12194 | }
12195 |
12196 | .admin-home-panel,
12197 | .app-shell:has(.admin-home-panel) .admin-home-panel {
12198 |   backdrop-filter: none !important;
12199 |   -webkit-backdrop-filter: none !important;
12200 |   filter: none !important;
12201 | }
12202 |
12203 | .admin-home-panel *,
12204 | .app-shell:has(.admin-home-panel) .admin-home-panel * {
12205 |   filter: none !important;

```



```

12206 | backdrop-filter: none !important;
12207 | -webkit-backdrop-filter: none !important;
12208 | }
12209 |
12210 | .admin-gradient-orb {
12211 |   display: none !important;
12212 |   filter: none !important;
12213 |   opacity: 0 !important;
12214 | }
12215 |
12216 | /* Final login repair: larger auth cards, clear back button spacing, still no page scroll. */
12217 | .public-shell.auth-shell {
12218 |   padding: 82px 24px 48px !important;
12219 |   overflow: hidden !important;
12220 | }
12221 |
12222 | .auth-shell .auth-grid {
12223 |   width: min(1100px, calc(100vw - 72px)) !important;
12224 |   min-height: min(560px, calc(100vh - 138px)) !important;
12225 |   max-height: calc(100vh - 138px) !important;
12226 |   grid-template-columns: minmax(360px, 0.94fr) minmax(390px, 1.06fr) !important;
12227 |   gap: 22px !important;
12228 |   align-items: stretch !important;
12229 | }
12230 |
12231 | .auth-shell .panel {
12232 |   height: 100% !important;
12233 |   max-height: none !important;
12234 |   padding: 30px !important;
12235 |   border-radius: 24px !important;
12236 |   overflow: hidden !important;
12237 |   background:
12238 |     linear-gradient(180deg, rgba(255, 255, 255, 0.99), rgba(248, 250, 252, 0.9)) !important;
12239 |     box-shadow: 0 28px 84px rgba(15, 23, 42, 0.12) !important;
12240 | }
12241 |
12242 | .auth-shell .auth-panel {
12243 |   padding: 32px !important;
12244 | }
12245 |
12246 | .auth-shell .panel-title {
12247 |   margin-bottom: 24px !important;
12248 |   gap: 14px !important;
12249 | }
12250 |
12251 | .auth-shell .panel-title svg {
12252 |   width: 28px !important;
12253 |   height: 28px !important;
12254 | }
12255 |
12256 | .auth-shell .panel h2,
12257 | .auth-shell .signal-panel h2 {
12258 |   font-size: clamp(1.75rem, 3vw, 2.55rem) !important;
12259 |   line-height: 1.12 !important;
12260 | }
12261 |
12262 | .auth-shell .form-stack {
12263 |   gap: 18px !important;
12264 | }
12265 |
12266 | .auth-shell label {
12267 |   gap: 10px !important;
12268 |   font-size: 0.98rem !important;
12269 | }
12270 |
12271 | .auth-shell input {
12272 |   min-height: 50px !important;
12273 |   padding: 12px 14px !important;
12274 |   font-size: 1rem !important;
12275 | }
12276 |
12277 | .auth-shell .primary-button {
12278 |   min-height: 50px !important;
12279 |   font-size: 1rem !important;
12280 | }
12281 |
12282 | .auth-shell .signal-panel {
12283 |   gap: 10px !important;
12284 |   justify-content: center !important;
12285 | }
12286 |
12287 | .auth-shell .login-orb {
12288 |   width: 108px !important;
12289 |   height: 108px !important;
12290 |   margin: 0 auto 8px !important;
12291 | }
12292 |
12293 | .auth-shell .login-orb svg {
12294 |   width: 54px !important;
12295 |   height: 54px !important;
12296 | }
12297 |
12298 | .auth-shell .signal-row {
12299 |   min-height: 64px !important;
12300 |   padding: 15px 16px !important;
12301 |   gap: 14px !important;
12302 |   border-radius: 16px !important;
12303 | }
12304 |
12305 | .auth-shell .signal-row svg {
12306 |   width: 26px !important;
12307 |   height: 26px !important;
12308 |   flex: 0 0 26px !important;
12309 | }
12310 |
12311 | .auth-shell .signal-row strong {
12312 |   font-size: 1rem !important;
12313 | }
12314 |
12315 | .auth-shell .signal-row span {

```

```

12316 | font-size: 0.9rem !important;
12317 | }
12318 |
12319 | .auth-shell .auth-back-button {
12320 |   top: 24px !important;
12321 |   left: 28px !important;
12322 |   height: 42px !important;
12323 |   min-height: 42px !important;
12324 |   padding: 0 8px !important;
12325 | }
12326 |
12327 | @media (max-height: 680px) {
12328 |   .public-shell.auth-shell {
12329 |     padding-top: 72px !important;
12330 |     padding-bottom: 34px !important;
12331 |   }
12332 |
12333 |   .auth-shell .auth-grid {
12334 |     min-height: calc(100vh - 118px) !important;
12335 |     max-height: calc(100vh - 118px) !important;
12336 |   }
12337 |
12338 |   .auth-shell .panel {
12339 |     padding: 22px !important;
12340 |   }
12341 |
12342 |   .auth-shell .login-orb {
12343 |     width: 88px !important;
12344 |     height: 88px !important;
12345 |   }
12346 |
12347 |   .auth-shell .signal-row {
12348 |     min-height: 56px !important;
12349 |     padding: 12px 14px !important;
12350 |   }
12351 | }
12352 |
12353 | /* Final back-button overlap fix: reserve a real top row above login cards. */
12354 | .public-shell.auth-shell {
12355 |   padding: 24px 24px 46px !important;
12356 | }
12357 |
12358 | .auth-shell .auth-grid {
12359 |   grid-template-rows: auto minmax(0, 1fr) !important;
12360 |   row-gap: 16px !important;
12361 |   min-height: min(632px, calc(100vh - 76px)) !important;
12362 |   max-height: calc(100vh - 76px) !important;
12363 | }
12364 |
12365 | .auth-shell .auth-back-button {
12366 |   position: static !important;
12367 |   grid-column: 1 / -1 !important;
12368 |   grid-row: 1 !important;
12369 |   display: inline-flex !important;
12370 |   align-items: center !important;
12371 |   gap: 8px !important;
12372 |   justify-self: start !important;
12373 |   align-self: center !important;
12374 |   z-index: 5 !important;
12375 |   height: 38px !important;
12376 |   min-height: 38px !important;
12377 |   margin: 0 !important;
12378 |   padding: 0 4px !important;
12379 |   transform: none !important;
12380 | }
12381 |
12382 | .auth-shell .auth-panel,
12383 | .auth-shell .signal-panel {
12384 |   grid-row: 2 !important;
12385 |   height: 100% !important;
12386 |   max-height: 100% !important;
12387 |   box-sizing: border-box !important;
12388 | }
12389 |
12390 | .auth-shell .auth-panel {
12391 |   grid-column: 1 !important;
12392 | }
12393 |
12394 | .auth-shell .signal-panel {
12395 |   grid-column: 2 !important;
12396 | }
12397 |
12398 | .auth-shell .auth-back-button:hover {
12399 |   transform: translateX(-3px) !important;
12400 | }
12401 |
12402 | /* Final auth polish: visible footer signature and clean Secure Access alignment. */
12403 | .auth-shell .team-eureka-badge {
12404 |   position: fixed !important;
12405 |   left: 50% !important;
12406 |   bottom: 14px !important;
12407 |   z-index: 30 !important;
12408 |   display: inline-flex !important;
12409 |   align-items: center !important;
12410 |   justify-content: center !important;
12411 |   gap: 8px !important;
12412 |   width: auto !important;
12413 |   margin: 0 !important;
12414 |   transform: translateX(-50%) !important;
12415 |   border-radius: 999px !important;
12416 |   padding: 8px 16px !important;
12417 |   background: rgba(255, 255, 255, 0.86) !important;
12418 |   color: #334155 !important;
12419 |   box-shadow: 0 14px 40px rgba(15, 23, 42, 0.08) !important;
12420 |   backdrop-filter: blur(16px) !important;
12421 | }
12422 |
12423 | .auth-shell .auth-panel .panel-title {
12424 |   align-items: flex-start !important;
12425 | }

```

```

12426 |
12427 | .auth-shell .auth-panel .panel-title > svg {
12428 |   flex: 0 0 28px !important;
12429 |   margin-top: 2px !important;
12430 | }
12431 |
12432 | /* Final analysis loading cleanup: only the animated loading card remains. */
12433 | .workspace:has(.admin-analysis-loading-page) {
12434 |   width: 100% !important;
12435 |   min-height: 100vh !important;
12436 |   margin: 0 !important;
12437 |   padding: 0 !important;
12438 | }
12439 |
12440 | .admin-analysis-loading-page {
12441 |   min-height: 100vh !important;
12442 |   display: grid !important;
12443 |   place-items: center !important;
12444 |   padding: 28px !important;
12445 | }
12446 |
12447 | .admin-analysis-loading-page .ai-progress-panel {
12448 |   width: min(1080px, calc(100vw - 56px)) !important;
12449 | }
12450 |
12451 | /* Final create-feedback cleanup: no global admin navbar, no overlapping back button. */
12452 | .workspace:has(.feedback-create-workspace) {
12453 |   width: min(1180px, calc(100% - 48px)) !important;
12454 |   margin: 0 auto !important;
12455 |   padding: 24px 0 44px !important;
12456 | }
12457 |
12458 | .feedback-create-workspace {
12459 |   gap: 14px !important;
12460 |   padding-top: 0 !important;
12461 | }
12462 |
12463 | .feedback-create-workspace .admin-back-button {
12464 |   position: static !important;
12465 |   top: auto !important;
12466 |   left: auto !important;
12467 |   z-index: 1 !important;
12468 |   display: inline-flex !important;
12469 |   width: max-content !important;
12470 |   height: auto !important;
12471 |   min-height: 0 !important;
12472 |   margin: 0 0 2px !important;
12473 |   border: 0 !important;
12474 |   border-radius: 0 !important;
12475 |   padding: 0 !important;
12476 |   background: transparent !important;
12477 |   color: #0f172a !important;
12478 |   box-shadow: none !important;
12479 |   backdrop-filter: none !important;
12480 |   font-size: 0.95rem !important;
12481 |   font-weight: 950 !important;
12482 | }
12483 |
12484 | .feedback-create-workspace .admin-back-button:hover {
12485 |   color: #2563eb !important;
12486 |   background: transparent !important;
12487 |   box-shadow: none !important;
12488 |   transform: translateX(-3px) !important;
12489 | }
12490 |
12491 | /* Final analysis page cleanup: plain back action only, no sticky nav or semester selector. */
12492 | .analysis-plain-back-button {
12493 |   display: inline-flex !important;
12494 |   align-items: center !important;
12495 |   gap: 8px !important;
12496 |   width: max-content !important;
12497 |   border: 0 !important;
12498 |   border-radius: 0 !important;
12499 |   padding: 0 !important;
12500 |   background: transparent !important;
12501 |   color: #0f172a !important;
12502 |   box-shadow: none !important;
12503 |   font-weight: 950 !important;
12504 | }
12505 |
12506 | .analysis-plain-back-button:hover {
12507 |   color: #2563eb !important;
12508 |   transform: translateX(-3px) !important;
12509 | }
12510 |
12511 | /* Final analysis hero swap: landing blackhole logo, no command-center text. */
12512 | .analytics-blackhole-hero {
12513 |   display: grid !important;
12514 |   grid-template-columns: minmax(0, 1fr) !important;
12515 |   place-items: center !important;
12516 |   min-height: 470px !important;
12517 |   overflow: visible !important;
12518 |   border-radius: 0 !important;
12519 |   padding: 0 0 8px !important;
12520 |   background:
12521 |     radial-gradient(circle at 50% 46%, rgba(37, 99, 235, 0.12), transparent 38%),
12522 |     radial-gradient(circle at 50% 52%, rgba(20, 184, 166, 0.1), transparent 50%) !important;
12523 |   box-shadow: none !important;
12524 | }
12525 |
12526 | .analytics-blackhole-hero .analysis-vortex-stage {
12527 |   justify-self: center !important;
12528 |   max-width: 980px !important;
12529 |   height: 470px !important;
12530 |   transform: scale(0.92) !important;
12531 |   transform-origin: center !important;
12532 | }
12533 |
12534 | .analytics-blackhole-hero .blackhole-logo-wrapper {
12535 |   cursor: default !important;

```

```
12536 | }  
12537 |  
12538 | .analytics-blackhole-hero .blackhole-logo-wrapper:hover {  
12539 |     transform: scale(1.06) rotate(4deg) !important;  
12540 | }
```

edupulse-frontend/src/App.tsx

File size: 140,382 bytes

```
1 | import { useCallback, useEffect, useState } from 'react';
2 | import type { ChangeEvent, FormEvent } from 'react';
3 | import {
4 |   AlertTriangle,
5 |   ArrowLeft,
6 |   BarChart3,
7 |   CheckCircle2,
8 |   ChevronDown,
9 |   ClipboardList,
10 | Download,
11 | Gauge,
12 | GraduationCap,
13 | History,
14 | Logout,
15 | ShieldCheck,
16 | Sparkles,
17 | UserRound,
18 | RefreshCw,
19 | X,
20 | } from 'lucide-react';
21 | import studentPhoto from './assets/student-photo.jpeg';
22 | import edupulseLogo from './assets/logo.png';
23 | import {
24 |   Bar,
25 |   BarChart,
26 |   CartesianGrid,
27 |   Cell,
28 |   Line,
29 |   LineChart,
30 |   Pie,
31 |   PieChart,
32 |   PolarAngleAxis,
33 |   PolarGrid,
34 |   Radar,
35 |   RadarChart,
36 |   ResponsiveContainer,
37 |   Tooltip,
38 |   XAxis,
39 |   YAxis,
40 | } from 'recharts';
41 | import './App.css';
42 | import LandingPage from './components/LandingPage';
43 |
44 | const API_BASE = 'http://localhost:4000/api/v1';
45 |
46 | type Role = 'ADMIN' | 'STUDENT';
47 |
48 | type User = {
49 |   id: string;
50 |   email: string;
51 |   name: string;
52 |   role: Role;
53 |   collegeId?: string | null;
54 |   college?: {
55 |     id: string;
56 |     name: string;
57 |     code: string;
58 |     logoUrl?: string | null;
59 |   } | null;
60 |   departmentId?: string | null;
61 |   semesterNumber?: number | null;
62 | };
63 |
64 | type AuthState = {
65 |   accessToken: string;
66 |   user: User;
67 | };
68 |
69 | type Term = {
70 |   id: string;
71 |   name: string;
72 |   isActive: boolean;
73 |   startsAt?: string | null;
74 |   endsAt?: string | null;
75 | };
76 |
77 | type Question = {
78 |   id: string;
79 |   categoryId: string;
80 |   text: string;
81 | };
82 |
83 | type Category = {
84 |   id: string;
85 |   name: string;
86 |   description: string;
87 |   questions: Question[];
88 | };
89 |
90 | type DashboardSummary = {
91 |   termId?: string;
92 |   responseCount: number;
93 |   submissionCount: number;
94 |   categorySatisfaction: Array<{
95 |     categoryId: string;
96 |     categoryName: string;
97 |     averageRating: number;
98 |     responseCount: number;
99 |   }>;
100 |   previousTerm?: { id: string; name: string } | null;
101 |   previousCategorySatisfaction?: Array<{
102 |     categoryId: string;
103 |     categoryName: string;
104 |     averageRating: number;
105 |     responseCount: number;
```

```

106 |     }>;
107 |     ratingDistribution?: RatingDistribution;
108 |     categoryRatingDistribution?: Array<{
109 |         categoryId: string;
110 |         categoryName: string;
111 |         distribution: RatingDistribution;
112 |     }>;
113 |     actionByStatus: Array<{ status: string; count: number }>;
114 |     latestReport?: AnalysisReport | null;
115 | };
116 |
117 | type RatingDistribution = {
118 |     unsatisfied: number;
119 |     average: number;
120 |     satisfied: number;
121 | };
122 |
123 | type ThemeInsight = {
124 |     id: string;
125 |     title: string;
126 |     summary: string;
127 |     sentiment: string;
128 |     priority: string;
129 |     mentionCount: number;
130 |     evidence?: unknown;
131 | };
132 |
133 | type ActionItem = {
134 |     id: string;
135 |     title: string;
136 |     priority: string;
137 |     status: string;
138 |     timeline: string;
139 |     kpi?: string | null;
140 | };
141 |
142 | type ExactIssueDetail = {
143 |     id: string;
144 |     shortTitle: string;
145 |     location: string;
146 |     affectedGroup: string;
147 |     affectedDepartments: string[];
148 |     priority: string;
149 |     mentions: number;
150 |     confidence: number;
151 |     trend: string;
152 |     status: 'Critical' | 'Warning' | 'Low';
153 |     finding: string;
154 |     rootCause: string;
155 |     evidence: string[];
156 |     action: string;
157 |     expectedImpact: string;
158 | };
159 |
160 | type AiActionReportModel = {
161 |     actionPlan: {
162 |         body: string;
163 |         priority: string;
164 |         title: string;
165 |     }[];
166 |     metrics: {
167 |         label: string;
168 |         value: string;
169 |     }[];
170 |     rootCauses: {
171 |         body: string;
172 |         signal: string;
173 |         title: string;
174 |     }[];
175 |     summary: string;
176 |     topIssues: {
177 |         affectedStudents: string;
178 |         evidenceSignal: string;
179 |         issue: string;
180 |         mentions: number;
181 |         priority: string;
182 |     }[];
183 | };
184 |
185 | type AiQualityModel = {
186 |     confidence: number;
187 |     duplicatesGrouped: number;
188 |     lowQualityIgnored: number;
189 |     lowSampleWarnings: number;
190 |     safetyEscalations: number;
191 |     totalComments: number;
192 |     validComments: number;
193 |     rejectedBreakdown: {
194 |         count: number;
195 |         label: string;
196 |         reason: string;
197 |     }[];
198 |     duplicateExamples: {
199 |         comment: string;
200 |         decision: string;
201 |         repeated: number;
202 |         topic: string;
203 |     }[];
204 |     lowSampleWarningsList: {
205 |         decision: string;
206 |         group: string;
207 |         responses: number;
208 |         topic: string;
209 |     }[];
210 |     rejectedExamples: {
211 |         comment: string;
212 |         decision: string;
213 |         question: string;
214 |         reason: string;
215 |         source: string;

```

```

216 |     topic: string;
217 |   }[];
218 |   safetyEscalationDetails: {
219 |     decision: string;
220 |     issue: string;
221 |     priority: string;
222 |     source: string;
223 |   }[];
224 | };
225 |
226 | type AiEngineBenchmarkModel = {
227 |   funnel: {
228 |     label: string;
229 |     note: string;
230 |     value: string;
231 |   }[];
232 |   pipeline: string[];
233 |   receipt: {
234 |     label: string;
235 |     value: string;
236 |   }[];
237 | };
238 |
239 | type GeminiReasoningModel = {
240 |   mode?: string;
241 |   provider?: string;
242 |   ultraConcerningIssues?: Array<{
243 |     id?: string;
244 |     title?: string;
245 |     reason?: string;
246 |     detailedSummary?: string;
247 |     mentions?: number;
248 |     source?: string;
249 |   }>;
250 |   priorityLabels?: Array<{
251 |     id?: string;
252 |     title?: string;
253 |     priority?: string;
254 |     reason?: string;
255 |   }>;
256 |   issueDetails?: Array<{
257 |     id?: string;
258 |     title?: string;
259 |     plainEnglishSummary?: string;
260 |     recommendedAction?: string;
261 |   }>;
262 |   reportNarrative?: Record<string, string>;
263 | };
264 |
265 | type AnalysisReport = {
266 |   id: string;
267 |   title: string;
268 |   summary: string;
269 |   confidence: number;
270 |   inputCount: number;
271 |   lowSampleFlag: boolean;
272 |   duplicateCount: number;
273 |   createdAt: string;
274 |   generatedBy?: string | null;
275 |   rawJson?: {
276 |     engine?: string;
277 |     modelMode?: string;
278 |     pipeline?: string[];
279 |     qualityChecks?: {
280 |       totalComments?: number;
281 |       usefulComments?: number;
282 |       duplicateCount?: number;
283 |       lowQualityRejectedCount?: number;
284 |       usefulSignalComments?: number;
285 |       uniqueUsefulComments?: number;
286 |       rejected?: Record<string, number>;
287 |       method?: string;
288 |       commentCount?: number;
289 |       rejectedExamples?: AiQualityModel['rejectedExamples'];
290 |       duplicateExamples?: AiQualityModel['duplicateExamples'];
291 |       lowSampleWarnings?: AiQualityModel['lowSampleWarningsList'];
292 |     };
293 |     sentimentBreakdown?: {
294 |       overall?: Record<string, { count?: number; percentage?: number }>;
295 |       categories?: Record<string, Record<string, { count?: number; percentage?: number }>>;
296 |     };
297 |     grievanceSignals?: {
298 |       total?: number;
299 |       safetyFlagged?: number;
300 |     };
301 |     clusterDiagnostics?: {
302 |       clusterer?: string;
303 |       embeddingModel?: string | null;
304 |       clusterCount?: number;
305 |       noiseCount?: number;
306 |     };
307 |     engineBenchmark?: {
308 |       processingMs?: number;
309 |       responsesPerSecond?: number;
310 |       documentsClustered?: number;
311 |       themesGenerated?: number;
312 |       actionsGenerated?: number;
313 |       criticalThemes?: number;
314 |       negativeThemes?: number;
315 |     };
316 |     modelStack?: {
317 |       themeEmbeddings?: {
318 |         primary?: string;
319 |         fallback?: string;
320 |       };
321 |       clustering?: {
322 |         primary?: string;
323 |         fallback?: string;
324 |       };
325 |       humanCorrectionLoop?: {

```

```

326 |         endpoint?: string;
327 |     };
328 | };
329 | reportNarrative?: {
330 |     boardReadyHighlights?: string[];
331 |     recommendedNextStep?: string;
332 | };
333 | geminiReasoning?: GeminiReasoningModel;
334 | } | null;
335 | themes?: ThemeInsight[];
336 | actions?: ActionItem[];
337 | };
338 |
339 | type AuditLog = {
340 |     id: string;
341 |     action: string;
342 |     entity?: string | null;
343 |     entityId?: string | null;
344 |     metadata?: Record<string, unknown> | null;
345 |     createdAt: string;
346 |     actor?: {
347 |         id: string;
348 |         name: string;
349 |         email: string;
350 |         role: Role;
351 |     } | null;
352 |     college?: {
353 |         id: string;
354 |         name: string;
355 |         code: string;
356 |     } | null;
357 | };
358 |
359 | type FeedbackAnswer = {
360 |     questionId: string;
361 |     categoryId: string;
362 |     rating: number;
363 |     comment: string;
364 | };
365 |
366 | type AdminFeedbackCategoryDraft = {
367 |     name: string;
368 |     description: string;
369 |     questions: string[];
370 | };
371 |
372 | const ratingLabels = [
373 |     { value: 1, label: 'Unsatisfied' },
374 |     { value: 2, label: 'Average' },
375 |     { value: 3, label: 'Good' },
376 |     { value: 4, label: 'Excellent' },
377 | ];
378 |
379 | const aiPipelineSteps = [
380 |     {
381 |         title: 'Quality filter',
382 |         detail: 'Rejecting fake, random, out-of-context and low-information comments.',
383 |     },
384 |     {
385 |         title: 'Duplicate grouping',
386 |         detail: 'Combining repeated complaints as stronger evidence instead of ignoring them.',
387 |     },
388 |     {
389 |         title: 'Sentiment scan',
390 |         detail: 'Separating positive, neutral, negative and critical feedback by category.',
391 |     },
392 |     {
393 |         title: 'Issue clustering',
394 |         detail: 'Grouping similar comments into themes like mess, Wi-Fi, faculty or infra.',
395 |     },
396 |     {
397 |         title: 'Risk scoring',
398 |         detail: 'Detecting safety-sensitive issues and assigning priority levels.',
399 |     },
400 |     {
401 |         title: 'Report generation',
402 |         detail: 'Preparing dashboard charts, evidence cards and action-plan data.',
403 |     },
404 | ];
405 |
406 | function App() {
407 |     const [authMode, setAuthMode] = useState<Role | null>(null);
408 |     const [adminFullScreen, setAdminFullScreen] = useState(false);
409 |     const [studentView, setStudentView] = useState<'home' | 'feedback' | 'details'>('home');
410 |     const [isAdminProfileOpen, setIsAdminProfileOpen] = useState(false);
411 |     const [auth, setAuth] = useState<AuthState | null>(() => {
412 |         const saved = localStorage.getItem('edupulse-auth');
413 |         return saved ? (JSON.parse(saved) as AuthState) : null;
414 |     });
415 |
416 |     function handleAuth(nextAuth: AuthState) {
417 |         localStorage.setItem('edupulse-auth', JSON.stringify(nextAuth));
418 |         setAuth(nextAuth);
419 |     }
420 |
421 |     function logout() {
422 |         localStorage.removeItem('edupulse-auth');
423 |         setAuth(null);
424 |         setAuthMode(null);
425 |         setAdminFullScreen(false);
426 |     }
427 |
428 |     if (!auth && !authMode) {
429 |         return (
430 |             <LandingPage onChooseRole={setAuthMode} />
431 |         );
432 |     }
433 |
434 |     if (!auth && authMode) {
435 |         return (

```



```

436 |         <main className="public-shell auth-shell">
437 |             <LoginPage
438 |                 initialRole={authMode}
439 |                 onBack={() => setAuthMode(null)}
440 |                 onLogin={handleAuth}
441 |             />
442 |             <TeamEurekaBadge />
443 |         </main>
444 |     );
445 | }
446 |
447 | const activeAuth = auth;
448 | if (!activeAuth) return null;
449 | const hideTopbar = activeAuth.user.role === 'ADMIN' && adminFullScreen;
450 |
451 | return (
452 |     <main className="app-shell">
453 |         <section className="workspace">
454 |             {!hideTopbar && (
455 |                 activeAuth.user.role === 'ADMIN' ? (
456 |                     <header className="admin-topbar">
457 |                         <div className="admin-topbar-left">
458 |                             <div className="dashboard-brand">
459 |                                 <span className="brand-logo-wrapper">
460 |                                     <img src={edupulseLogo} alt="EduPulse AI Logo" className="brand-logo-img" />
461 |                                 </span>
462 |                                 <div className="brand-title-wrap">
463 |                                     <span className="brand-text">EduPulse AI</span>
464 |                                 </div>
465 |                             </div>
466 |                         </div>
467 |
468 |                         <div className="admin-topbar-right">
469 |                             <button
470 |                                 className="admin-profile-trigger"
471 |                                 type="button"
472 |                                 onClick={() => setIsAdminProfileOpen(!isAdminProfileOpen)}
473 |                             >
474 |                                 <div className="admin-avatar-initials">
475 |                                     {activeAuth.user.name ? activeAuth.user.name.charAt(0).toUpperCase() : 'A'}
476 |                                 </div>
477 |                                 <span className="admin-profile-name">{activeAuth.user.name ?? 'Admin'}</span>
478 |                                 <ChevronDown size={16} className={`profile-chevron ${isAdminProfileOpen ? 'rotated' : ''}`} />
479 |                             </button>
480 |
481 |                             {isAdminProfileOpen && (
482 |                                 <div
483 |                                     className="dropdown-click-overlay"
484 |                                     onClick={() => setIsAdminProfileOpen(false)} />
485 |                                     <div className="admin-profile-dropdown">
486 |                                         <div className="dropdown-user-details">
487 |                                             <strong>{activeAuth.user.name ?? 'Admin'}</strong>
488 |                                             <span>{activeAuth.user.email ?? 'EduPulse AI Administrator'}</span>
489 |                                         </div>
490 |                                         <div className="dropdown-divider" />
491 |                                         <button className="dropdown-logout-btn" type="button" onClick={() => {
492 |                                             setIsAdminProfileOpen(false);
493 |                                             logout();
494 |                                         }}>
495 |                                             <LogOut size={16} />
496 |                                             Logout
497 |                                         </button>
498 |                                     </div>
499 |                                 </div>
500 |                             )}
501 |                         </div>
502 |                     </header>
503 |                 ) : (
504 |                     <header className="topbar">
505 |                         {activeAuth.user.role === 'STUDENT' && studentView !== 'home' ? (
506 |                             <button
507 |                                 className="topbar-back-button"
508 |                                 type="button"
509 |                                 onClick={() => setStudentView('home')}
510 |                             >
511 |                                 ← Go back
512 |                             </button>
513 |                         ) : (
514 |                             <div className="dashboard-brand">
515 |                                 <span className="brand-logo-wrapper">
516 |                                     <img src={edupulseLogo} alt="EduPulse AI Logo" className="brand-logo-img" />
517 |                                 </span>
518 |                                 <div>
519 |                                     <span className="brand-text">EduPulse AI</span>
520 |                                 </div>
521 |                             </div>
522 |                             <button className="icon-button" type="button" onClick={logout}>
523 |                                 <LogOut size={18} />
524 |                                 Logout
525 |                             </button>
526 |                         </header>
527 |                     )
528 |                 )}
529 |             {activeAuth.user.role === 'ADMIN' ? (
530 |                 <AdminDashboard auth={activeAuth} onFullScreenChange={setAdminFullScreen} onSessionExpired={logout} />
531 |             ) : (
532 |                 <StudentDashboard
533 |                     auth={activeAuth}
534 |                     studentView={studentView}
535 |                     setStudentView={setStudentView}
536 |                 />
537 |             )}
538 |         </section>
539 |         {!adminFullScreen && <TeamEurekaBadge />}
540 |     </main>
541 | );
542 | }
543 |
544 |
545 |

```

```

546 | function TeamEurekaBadge() {
547 |     return (
548 |         <div className="team-eureka-badge">
549 |             <CheckCircle2 size={16} />
550 |             Developed by Team Eureka
551 |         </div>
552 |     );
553 | }
554 |
555 |
556 |
557 | function LoginPanel({
558 |     initialRole,
559 |     onBack,
560 |     onLogin,
561 | }): {
562 |     initialRole: Role;
563 |     onBack: () => void;
564 |     onLogin: (auth: AuthState) => void;
565 | }) {
566 |     const [email, setEmail] = useState(
567 |         initialRole === 'ADMIN' ? 'admin@edupulse.edu' : 'student@edupulse.edu',
568 |     );
569 |     const [password, setPassword] = useState(
570 |         initialRole === 'ADMIN' ? 'Admin@12345' : 'Student@12345',
571 |     );
572 |     const [loading, setLoading] = useState(false);
573 |     const [error, setError] = useState('');
574 |
575 |     async function login(event: FormEvent) {
576 |         event.preventDefault();
577 |         setLoading(true);
578 |         setError('');
579 |
580 |         try {
581 |             const auth = await api<AuthState>('/auth/login', {
582 |                 method: 'POST',
583 |                 body: JSON.stringify({ email, password }),
584 |             });
585 |             onLogin(auth);
586 |         } catch (err) {
587 |             setError(err instanceof Error ? err.message : 'Login failed');
588 |         } finally {
589 |             setLoading(false);
590 |         }
591 |     }
592 |
593 |     return (
594 |         <div className="auth-grid">
595 |             <button className="auth-back-button" type="button" onClick={onBack}>
596 |                 <ArrowLeft size={17} />
597 |                 Go back
598 |             </button>
599 |             <section className="panel auth-panel">
600 |                 <div className="panel-title">
601 |                     <ShieldCheck size={22} />
602 |                     <div>
603 |                         <p className="eyebrow">Secure access</p>
604 |                         <h2>{initialRole === 'ADMIN' ? 'Admin login' : 'Student login'}</h2>
605 |                     </div>
606 |                 </div>
607 |                 <form className="form-stack" onSubmit={login}>
608 |                     <label>
609 |                         Email
610 |                         <input value={email} onChange={(event) => setEmail(event.target.value)} />
611 |                     </label>
612 |                     <label>
613 |                         Password
614 |                         <input
615 |                             type="password"
616 |                             value={password}
617 |                             onChange={(event) => setPassword(event.target.value)}
618 |                         />
619 |                     </label>
620 |                     {error && <p className="error-text">{error}</p>}
621 |                     <button className="primary-button" type="submit" disabled={loading}>
622 |                         <UserRound size={18} />
623 |                         {loading ? 'Signing in' : 'Sign in'}
624 |                     </button>
625 |                 </form>
626 |             </section>
627 |
628 |             <section className="panel signal-panel">
629 |                 <div className="login-orb">
630 |                     {initialRole === 'ADMIN' ? <BarChart3 size={52} /> : <GraduationCap size={52} />}
631 |                 </div>
632 |                 <div>
633 |                     <p className="eyebrow">
634 |                         {initialRole === 'ADMIN' ? 'Admin command center' : 'Student workspace'}
635 |                     </p>
636 |                     <h2>
637 |                         {initialRole === 'ADMIN'
638 |                             ? 'Analyze feedback, prioritize issues, generate reports.'
639 |                             : 'Submit structured semester feedback quickly.'}
640 |                     </h2>
641 |                 </div>
642 |                 <div className="signal-row">
643 |                     <GraduationCap size={24} />
644 |                     <div>
645 |                         <strong>Student interface</strong>
646 |                         <span>Feedback form and profile updates</span>
647 |                     </div>
648 |                 </div>
649 |                 <div className="signal-row">
650 |                     <BarChart3 size={24} />
651 |                     <div>
652 |                         <strong>Admin interface</strong>
653 |                         <span>Charts, analysis, reports and actions</span>
654 |                     </div>
655 |                 </div>

```

```

656 |         <div className="signal-row">
657 |             <Gauge size={24} />
658 |             <div>
659 |                 <strong>Backend intelligence</strong>
660 |                 <span>Themes, sentiment, priority and quality flags</span>
661 |             </div>
662 |         </div>
663 |     </section>
664 | </div>
665 | });
666 | }
667 |
668 | function getCampusDisplayName(user: User) {
669 |     const collegeName = user.college?.name?.trim();
670 |     if (collegeName) {
671 |         return /campus/i.test(collegeName) ? collegeName : `${collegeName} Campus`;
672 |     }
673 |
674 |     const email = user.email.toLowerCase();
675 |     if (email.endsWith('@sharda.edu') || email.includes('sharda')) {
676 |         return 'Sharda University Campus';
677 |     }
678 |     if (email.endsWith('@glbim.ac.in') || email.includes('edupulse.edu')) {
679 |         return 'G.L. Bajaj Institute of Technology and Management Campus';
680 |     }
681 |
682 |     return 'College Campus';
683 | }
684 |
685 | function StudentDashboard({
686 |     auth,
687 |     studentView,
688 |     setStudentView,
689 | }): {
690 |     auth: AuthState;
691 |     studentView: 'home' | 'feedback' | 'details';
692 |     setStudentView: React.Dispatch<React.SetStateAction<'home' | 'feedback' | 'details'>>;
693 | } {
694 |     const [profilePhoto, setProfilePhoto] = useState(studentPhoto);
695 |     const [isPhotoOpen, setIsPhotoOpen] = useState(false);
696 |     const [profileDetails, setProfileDetails] = useState({
697 |         primaryEmail: 'ec3020@glbim.ac.in',
698 |         alternateEmail: '',
699 |         alternatePhone: '',
700 |     });
701 |     const [term, setTerm] = useState<Term | null>(null);
702 |     const [categories, setCategories] = useState<Category[]>([]);
703 |     const [answers, setAnswers] = useState<Record<string, FeedbackAnswer>>({});
704 |     const [status, setStatus] = useState('');
705 |     const campusName = getCampusDisplayName(auth.user);
706 |
707 |     const loadStudentData = useCallback(async () => {
708 |         const [activeTerm, feedbackCategories] = await Promise.all([
709 |             api<Term | null>('/feedback/active-term', authHeaders(auth)),
710 |             api<Category[]>('/feedback/categories', authHeaders(auth)),
711 |         ]);
712 |
713 |         setTerm(activeTerm);
714 |         setCategories(feedbackCategories);
715 |
716 |         const initialAnswers: Record<string, FeedbackAnswer> = {};
717 |         for (const category of feedbackCategories) {
718 |             for (const question of category.questions) {
719 |                 initialAnswers[question.id] = {
720 |                     questionId: question.id,
721 |                     categoryId: category.id,
722 |                     rating: 3,
723 |                     comment: '',
724 |                 };
725 |             }
726 |         }
727 |         setAnswers(initialAnswers);
728 |     }, [auth]);
729 |
730 |     useEffect(() => {
731 |         // eslint-disable-next-line react-hooks/set-state-in-effect
732 |         void loadStudentData();
733 |     }, [loadStudentData]);
734 |
735 |     async function openSemesterFeedback() {
736 |         setStatus('');
737 |         try {
738 |             await loadStudentData();
739 |         } catch (error) {
740 |             setStatus(error instanceof Error ? error.message : 'Unable to load the latest feedback form.');

```

```

766 |     return;
767 | }
768 |
769 | const payload = {
770 |   termId: term.id,
771 |   answers: Object.values(answers).map((answer) => ({
772 |     questionId: answer.questionId,
773 |     categoryId: answer.categoryId,
774 |     rating: answer.rating,
775 |     comment: answer.comment.trim() || undefined,
776 |   })),
777 | };
778 |
779 | await api('/feedback/submit', {
780 |   ...authHeaders(auth),
781 |   method: 'POST',
782 |   body: JSON.stringify(payload),
783 | });
784 | setStatus('Semester feedback submitted');
785 | }
786 |
787 | if (studentView === 'home') {
788 |   return (
789 |     <section className="student-home">
790 |       <aside className="student-profile-card">
791 |         <button
792 |           aria-label="Open student photo preview"
793 |           className="student-avatar avatar-button"
794 |           type="button"
795 |           onClick={() => setIsPhotoOpen(true)}
796 |         >
797 |           <img src={profilePhoto} alt="Gaurav Sharma" />
798 |         </button>
799 |         <div>
800 |           <p className="eyebrow">Welcome back</p>
801 |           <h2>Gaurav Sharma</h2>
802 |         </div>
803 |         <dl className="student-details">
804 |           <div>
805 |             <dt>Branch</dt>
806 |             <dd>Electronics and Communication</dd>
807 |           </div>
808 |           <div>
809 |             <dt>Year</dt>
810 |             <dd>Semester 7</dd>
811 |           </div>
812 |           <div>
813 |             <dt>Student ID</dt>
814 |             <dd>EC23020</dd>
815 |           </div>
816 |           <div>
817 |             <dt>Email</dt>
818 |             <dd>{profileDetails.primaryEmail}</dd>
819 |           </div>
820 |         </dl>
821 |       </aside>
822 |
823 |       <section className="student-hero-panel">
824 |         <div className="ambient-glow-wrapper">
825 |           <div className="ambient-glow glow-1" />
826 |           <div className="ambient-glow glow-2" />
827 |           <div className="ambient-glow glow-3" />
828 |         </div>
829 |
830 |         <h1 className="student-hero-title">
831 |           <Sparkles size={28} className="hero-title-icon-sparkle" />
832 |           <span>Student Feedback Portal</span>
833 |         </h1>
834 |         <p className="student-hero-desc">
835 |           Submit semester feedback, raise important concerns, and help improve your campus experience through structured student insights.
836 |         </p>
837 |
838 |         <div className="student-badge-strip">
839 |           <span className="student-badge">
840 |             <GraduationCap size={16} /> {campusName}
841 |           </span>
842 |         </div>
843 |
844 |         <div className="student-action-grid">
845 |           <button
846 |             className={`student-action-card primary-action ${term ? 'status-live' : 'status-closed'}`}
847 |             type="button"
848 |             disabled={!term}
849 |             onClick={term ? () => void openSemesterFeedback() : undefined}
850 |           >
851 |             <ClipboardList size={24} />
852 |             <div className="action-card-content">
853 |               <div className="action-title-row">
854 |                 <strong>Fill semester feedback</strong>
855 |                 <span className="action-status-pill">
856 |                   <span className="status-dot" />
857 |                   {term ? 'LIVE' : 'CLOSED'}
858 |                 </span>
859 |               </div>
860 |               <span>Academics, faculty, food, infrastructure and more.</span>
861 |             </div>
862 |           </button>
863 |         </div>
864 |       </section>
865 |       {isPhotoOpen && (
866 |         <PhotoPreview
867 |           alt="Gaurav Sharma"
868 |           src={profilePhoto}
869 |           onClose={() => setIsPhotoOpen(false)}
870 |         />
871 |       )}
872 |     </section>
873 |   );
874 | }
875 |

```

```

876 | if (studentView === 'details') {
877 |   return (
878 |     <StudentDetailsView
879 |       details={profileDetails}
880 |       photo={profilePhoto}
881 |       onBack={() => setStudentView('home')}
882 |       onPhotoChange={setProfilePhoto}
883 |       onSave={setProfileDetails}
884 |     />
885 |   );
886 | }
887 |
888 | return (
889 |   <div className="student-form-wrap">
890 |     {!term ? (
891 |       <section className="panel student-focused-panel">
892 |         <div className="panel-title">
893 |           <ClipboardList size={22} />
894 |           <div>
895 |             <p className="eyebrow">Semester feedback</p>
896 |             <h2>No semester feedback is live</h2>
897 |           </div>
898 |         </div>
899 |         <p className="empty-state">
900 |           Admin has turned off feedback collection. Please check again when a
901 |             new feedback form goes live.
902 |         </p>
903 |       </section>
904 |     ) : (
905 |       <section className="panel wide feedback-workspace">
906 |         <h1 className="feedback-form-title">Semester feedback form</h1>
907 |
908 |         <div className="category-stack">
909 |           {categories.map((category) => (
910 |             <article className="category-block" key={category.id}>
911 |               <div>
912 |                 <h3>{category.name}</h3>
913 |                 <p>{category.description}</p>
914 |               </div>
915 |               {category.questions.map((question) => {
916 |                 const answer = answers[question.id];
917 |                 return (
918 |                   <div className="question-row" key={question.id}>
919 |                     <span>{question.text}</span>
920 |                     <div className="rating-grid">
921 |                       {ratingLabels.map((rating) => (
922 |                         <button
923 |                           className={answer?.rating === rating.value ? 'selected' : ''}
924 |                           key={rating.value}
925 |                           type="button"
926 |                           onClick={() => updateAnswer(question, { rating: rating.value })}
927 |                         >
928 |                           {rating.label}
929 |                         </button>
930 |                       )}}
931 |                     </div>
932 |                     {answer?.rating <= 2 && (
933 |                       <textarea
934 |                         className="feedback-comment"
935 |                         placeholder={
936 |                           answer.rating === 1
937 |                             ? 'Why are you unsatisfied? Mention the issue clearly.'
938 |                             : 'What made this only average? Suggest what should improve.'
939 |                         }
940 |                         value={answer.comment}
941 |                         onChange={(event) =>
942 |                           updateAnswer(question, { comment: event.target.value })
943 |                         }
944 |                       />
945 |                     )}}
946 |                   </div>
947 |                 );
948 |               )}}
949 |             </article>
950 |           )}}
951 |         </div>
952 |         <div className="feedback-submit-container">
953 |           {status && <p className="success-text">{status}</p>}
954 |           <button className="feedback-submit-red" type="button" onClick={submitFeedback}>
955 |             <CheckCircle2 size={18} />
956 |             Submit feedback
957 |           </button>
958 |         </div>
959 |       </section>
960 |     )}
961 |   </div>
962 | );
963 | }
964 |
965 | function StudentDetailsView({
966 |   details,
967 |   photo,
968 |   onBack,
969 |   onPhotoChange,
970 |   onSave,
971 | }): {
972 |   details: {
973 |     primaryEmail: string;
974 |     alternateEmail: string;
975 |     alternatePhone: string;
976 |   };
977 |   photo: string;
978 |   onBack: () => void;
979 |   onPhotoChange: (photo: string) => void;
980 |   onSave: (details: {
981 |     primaryEmail: string;
982 |     alternateEmail: string;
983 |     alternatePhone: string;
984 |   }) => void;
985 | }) {

```

```

986 | const [draft, setDraft] = useState(details);
987 | const [draftPhoto, setDraftPhoto] = useState(photo);
988 | const [message, setMessage] = useState('');
989 |
990 | function handlePhotoUpload(event: ChangeEvent<HTMLInputElement>) {
991 |   const file = event.target.files?.[0];
992 |   if (!file) return;
993 |   setDraftPhoto(URL.createObjectURL(file));
994 |   setMessage('Photo selected. Click Save details to apply it.');
```

```

995 | }
996 |
997 | function saveDetails() {
998 |   onSave({
999 |     primaryEmail: draft.primaryEmail.trim(),
1000 |     alternateEmail: draft.alternateEmail.trim(),
1001 |     alternatePhone: draft.alternatePhone.trim(),
1002 |   });
1003 |   onPhotoChange(draftPhoto);
1004 |   setMessage('Details updated');
```

```

1005 | }
1006 |
1007 | return (
1008 |   <div className="student-form-wrap">
1009 |     <button className="auth-back-button inline-back" type="button" onClick={onBack}>
1010 |       Go back
1011 |     </button>
1012 |     <section className="panel student-focused-panel student-details-editor">
1013 |       <div className="panel-title between">
1014 |         <div className="title-inline">
1015 |           <UserRound size={22} />
1016 |           <div>
1017 |             <p className="eyebrow">Student profile</p>
1018 |             <h2>Update contact details</h2>
1019 |           </div>
1020 |         </div>
1021 |         <span className="locked-badge">Identity fields locked</span>
1022 |       </div>
1023 |
1024 |       <div className="student-edit-grid">
1025 |         <aside className="student-photo-editor">
1026 |           <div className="student-avatar large-avatar">
1027 |             <img src={draftPhoto} alt="Gaurav Sharma" />
1028 |           </div>
1029 |           <label className="photo-upload-button">
1030 |             Change picture
1031 |             <input accept="image/*" type="file" onChange={handlePhotoUpload} />
1032 |           </label>
1033 |         </aside>
1034 |
1035 |         <div className="locked-details-grid">
1036 |           <div>
1037 |             <span>Name</span>
1038 |             <strong>Gaurav Sharma</strong>
1039 |           </div>
1040 |           <div>
1041 |             <span>Student ID</span>
1042 |             <strong>EC23020</strong>
1043 |           </div>
1044 |           <div>
1045 |             <span>Branch</span>
1046 |             <strong>Electronics and Communication</strong>
1047 |           </div>
1048 |           <div>
1049 |             <span>Semester</span>
1050 |             <strong>Semester 7</strong>
1051 |           </div>
1052 |           <div>
1053 |             <span>Parent name</span>
1054 |             <strong>Mr. Rajesh Sharma</strong>
1055 |           </div>
1056 |         </div>
1057 |       </div>
1058 |
1059 |       <div className="editable-details-grid">
1060 |         <label>
1061 |           Gmail / primary email
1062 |           <input
1063 |             value={draft.primaryEmail}
1064 |             onChange={(event) =>
1065 |               setDraft((current) => ({ ...current, primaryEmail: event.target.value })))
1066 |           />
1067 |         </label>
1068 |         <label>
1069 |           Alternate email
1070 |           <input
1071 |             placeholder="example.alt@gmail.com"
1072 |             value={draft.alternateEmail}
1073 |             onChange={(event) =>
1074 |               setDraft((current) => ({ ...current, alternateEmail: event.target.value })))
1075 |           />
1076 |         </label>
1077 |         <label>
1078 |           Alternate phone number
1079 |           <input
1080 |             placeholder="+91 98765 43210"
1081 |             value={draft.alternatePhone}
1082 |             onChange={(event) =>
1083 |               setDraft((current) => ({ ...current, alternatePhone: event.target.value })))
1084 |           />
1085 |         </label>
1086 |       </div>
1087 |
1088 |       <button className="primary-button" type="button" onClick={saveDetails}>
1089 |         Save details
1090 |       </button>
1091 |       {message && <p className="success-text">{message}</p>}
1092 |     </div>
1093 |   </div>
1094 | )
1095 |
```

```

1096 |         </section>
1097 |     </div>
1098 | );
1099 | }
1100 |
1101 | function PhotoPreview({
1102 |     src,
1103 |     alt,
1104 |     onClose,
1105 | }): {
1106 |     src: string;
1107 |     alt: string;
1108 |     onClose: () => void;
1109 | }) {
1110 |     return (
1111 |         <div className="photo-preview-backdrop" role="presentation" onClick={onClose}>
1112 |             <div className="photo-preview-modal" role="dialog" aria-modal="true" onClick={(event) => event.stopPropagation()}>
1113 |                 <button className="photo-preview-close" type="button" onClick={onClose}>
1114 |                     Close
1115 |                 </button>
1116 |                 <img src={src} alt={alt} />
1117 |             </div>
1118 |         </div>
1119 |     );
1120 | }
1121 |
1122 | function AdminDashboard({
1123 |     auth,
1124 |     onFullScreenChange,
1125 |     onSessionExpired,
1126 | }): {
1127 |     auth: AuthState;
1128 |     onFullScreenChange?: (isFullScreen: boolean) => void;
1129 |     onSessionExpired: () => void;
1130 | }) {
1131 |     const [adminView, setAdminView] = useState<
1132 |         'home' | 'create' | 'analysis' | 'analyzing' | 'audit'
1133 |     >('home');
1134 |     const [term, setTerm] = useState<Term | null>(null);
1135 |     const [terms, setTerms] = useState<Term[]>([]);
1136 |     const [analysisTermId, setAnalysisTermId] = useState<string>('');
1137 |     const [summary, setSummary] = useState<DashboardSummary | null>(null);
1138 |     const [, setReports] = useState<AnalysisReport[]>([]);
1139 |     const [selectedReport, setSelectedReport] = useState<AnalysisReport | null>(null);
1140 |     const [auditLogs, setAuditLogs] = useState<AuditLog[]>([]);
1141 |     const [adminNotice, setAdminNotice] = useState('');
1142 |     const [isAnalyzing, setIsAnalyzing] = useState(false);
1143 |     const [isLiveFeedbackAnalysis, setIsLiveFeedbackAnalysis] = useState(false);
1144 |     const [analysisStepIndex, setAnalysisStepIndex] = useState(0);
1145 |
1146 |     useEffect(() => {
1147 |         onFullScreenChange?.(adminView === 'analysis' || adminView === 'analyzing' || adminView === 'create');
1148 |         return () => onFullScreenChange?.(false);
1149 |     }, [adminView, onFullScreenChange]);
1150 |
1151 |     useEffect(() => {
1152 |         if (!isAnalyzing) return undefined;
1153 |
1154 |         const timer = window.setInterval(() => {
1155 |             setAnalysisStepIndex((step) => Math.min(step + 1, aiPipelineSteps.length - 2));
1156 |         }, 850);
1157 |
1158 |         return () => window.clearInterval(timer);
1159 |     }, [isAnalyzing]);
1160 |
1161 |     const loadAdminData = useCallback(async () => {
1162 |         try {
1163 |             const adminTerms = await api<Term[]>('/feedback/admin/terms', authHeaders(auth));
1164 |             const activeTerm = adminTerms.find((item) => item.isActive) ?? null;
1165 |             setTerm(activeTerm);
1166 |             setTerms(adminTerms);
1167 |
1168 |             try {
1169 |                 const auditLogList = await api<AuditLog[]>('/audit/logs?limit=80', authHeaders(auth));
1170 |                 setAuditLogs(auditLogList);
1171 |             } catch {
1172 |                 setAuditLogs([]);
1173 |             }
1174 |
1175 |             const selectedTermId = analysisTermId || activeTerm?.id || adminTerms[0]?.id || '';
1176 |             if (!analysisTermId && selectedTermId) {
1177 |                 setAnalysisTermId(selectedTermId);
1178 |             }
1179 |
1180 |             if (!selectedTermId) {
1181 |                 setSummary(null);
1182 |                 setReports([]);
1183 |                 setSelectedReport(null);
1184 |                 setAdminNotice('No semester dataset exists for this college yet.');
```

```

1206 | // eslint-disable-next-line react-hooks/set-state-in-effect
1207 | void loadAdminData();
1208 | }, [loadAdminData]);
1209 |
1210 | async function runAnalysis() {
1211 |   if (!analysisTermId) {
1212 |     setAdminNotice('No semester data is available for AI analysis.');
```

```

1213 |     return;
1214 |   }
1215 |   const selectedAnalysisTerm = terms.find((item) => item.id === analysisTermId);
1216 |   const isSelectedFeedbackLive = Boolean(
1217 |     selectedAnalysisTerm?.isActive || (term?.id === analysisTermId && term?.isActive),
1218 |   );
1219 |   setAdminView('analyzing');
1220 |   setIsAnalyzing(true);
1221 |   setIsLiveFeedbackAnalysis(isSelectedFeedbackLive);
1222 |   setAnalysisStepIndex(0);
1223 |   setAdminNotice('Running self-hosted AI pipeline...');
```

```

1224 |   try {
1225 |     const [report] = await Promise.all([
1226 |       api<AnalysisReport>('/analysis/run', {
1227 |         ...authHeaders(auth),
1228 |         method: 'POST',
1229 |         body: JSON.stringify({ termId: analysisTermId }),
1230 |       }),
1231 |       delay(3200),
1232 |     ]);
1233 |     setAnalysisStepIndex(aiPipelineSteps.length - 1);
1234 |     setSelectedReport(report);
1235 |     await loadAdminData();
1236 |     const fallbackNotice =
1237 |       report.generatedBy === 'edupulse-ai-service'
1238 |       ? ''
1239 |       : 'Fallback analysis completed. Start Python AI service for full self-hosted NLP mode.';
1240 |     setAdminNotice(fallbackNotice);
1241 |     setAdminView('analysis');
```

```

1242 |   } catch (error) {
1243 |     setAdminNotice(error instanceof Error ? error.message : 'AI analysis failed.');
```

```

1244 |     setAdminView('home');
```

```

1245 |   } finally {
1246 |     window.setTimeout(() => setIsAnalyzing(false), 500);
1247 |   }
1248 | }
1249 |
1250 | async function downloadReport() {
1251 |   if (!selectedReport) return;
1252 |   const response = await fetch(`${API_BASE}/reports/${selectedReport.id}/pdf`, {
1253 |     headers: {
1254 |       Authorization: `Bearer ${auth.accessToken}`,
1255 |     },
1256 |   });
1257 |   const blob = await response.blob();
1258 |   const url = URL.createObjectURL(blob);
1259 |   const link = document.createElement('a');
1260 |   link.href = url;
1261 |   link.download = `edupulse-report-${selectedReport.id}.pdf`;
1262 |   link.click();
1263 |   URL.revokeObjectURL(url);
1264 | }
1265 |
1266 | const satisfactionData = summary?.categorySatisfaction ?? [];
1267 | const ratedResponses = satisfactionData.reduce((total, item) => total + item.responseCount, 0);
1268 | const weightedRating = satisfactionData.reduce(
1269 |   (total, item) => total + item.averageRating * item.responseCount,
1270 |   0,
1271 | );
1272 | const satisfactionScore = ratedResponses
1273 |   ? Math.round((weightedRating / ratedResponses / 4) * 100)
1274 |   : 0;
1275 | if (adminView === 'home') {
1276 |   return (
1277 |     <AdminHome
1278 |       activeTermName={term?.name}
1279 |       auditLogCount={auditLogs || []}.length
1280 |       analysisTermId={analysisTermId}
1281 |       terms={terms || []}
1282 |       notice={adminNotice}
1283 |       onAnalysisTermChange={setAnalysisTermId}
1284 |       onRefreshDatasets={() => void loadAdminData()}
1285 |       onChoose={(view) => {
1286 |         if (view === 'analysis') {
1287 |           void runAnalysis();
1288 |           return;
1289 |         }
1290 |         setAdminView(view);
1291 |       }}
1292 |     />
1293 |   );
1294 | }
1295 |
1296 | if (adminView === 'analyzing') {
1297 |   return (
1298 |     <AdminAnalysisLoadingPage
1299 |       activeStep={analysisStepIndex}
1300 |     />
1301 |   );
1302 | }
1303 |
1304 | if (adminView === 'create') {
1305 |   return (
1306 |     <AdminCreateFeedback
1307 |       activeTermName={term?.name}
1308 |       auth={auth}
1309 |       isFeedbackLive={Boolean(term)}
1310 |       onBack={() => setAdminView('home')}
1311 |       onChange={loadAdminData}
1312 |     />
1313 |   );
1314 | }
1315 |

```



```

1316 |   if (adminView === 'audit') {
1317 |     return (
1318 |       <AdminAuditLogsView
1319 |         auditLogs={auditLogs || []}
1320 |         onBack={() => setAdminView('home')}
1321 |         onRefresh={() => void loadAdminData()}
1322 |       />
1323 |     );
1324 |   }
1325 |
1326 |   return (
1327 |     <AdminAnalysisReportPage
1328 |       adminNotice={adminNotice}
1329 |       downloadReport={downloadReport}
1330 |       isLiveFeedbackAnalysis={isLiveFeedbackAnalysis}
1331 |       selectedReport={selectedReport}
1332 |       satisfactionData={satisfactionData}
1333 |       satisfactionScore={satisfactionScore}
1334 |       summary={summary}
1335 |       onBack={() => setAdminView('home')}
1336 |     />
1337 |   );
1338 | }
1339 |
1340 | function AdminAnalysisLoadingPage({
1341 |   activeStep,
1342 | }: {
1343 |   activeStep: number;
1344 | }) {
1345 |   return (
1346 |     <div className="admin-analysis-loading-page premium-analytics-page">
1347 |       <AiAnalysisProgress activeStep={activeStep} />
1348 |     </div>
1349 |   );
1350 | }
1351 |
1352 | function AdminAnalysisReportPage({
1353 |   adminNotice,
1354 |   downloadReport,
1355 |   isLiveFeedbackAnalysis,
1356 |   selectedReport,
1357 |   satisfactionData,
1358 |   satisfactionScore,
1359 |   summary,
1360 |   onBack,
1361 | }: {
1362 |   adminNotice: string;
1363 |   downloadReport: () => void;
1364 |   isLiveFeedbackAnalysis: boolean;
1365 |   selectedReport: AnalysisReport | null;
1366 |   satisfactionData: DashboardSummary['categorySatisfaction'];
1367 |   satisfactionScore: number;
1368 |   summary: DashboardSummary | null;
1369 |   onBack: () => void;
1370 | }) {
1371 |   const categories = satisfactionData.length
1372 |     ? satisfactionData.map((item) => item.categoryName)
1373 |     : ['Academics', 'Faculty', 'Infrastructure', 'Food & Mess', 'Sports/Campus'];
1374 |   const [ratingCategory, setRatingCategory] = useState('Overall');
1375 |   const [selectedExactIssue, setSelectedExactIssue] = useState<ExactIssueDetail | null>(null);
1376 |   const [showFullHeatmap, setShowFullHeatmap] = useState(false);
1377 |   const categoryOptions = ['Overall', ...categories];
1378 |   const confidence = selectedReport ? Math.round(selectedReport.confidence * 100) : 0;
1379 |   const responseCount = summary?.responseCount ?? selectedReport?.inputCount ?? 0;
1380 |   const ratingSplit = buildCategoryRatingSplit(summary, ratingCategory);
1381 |   const hasRatingSplit = ratingSplit.some((item) => item.value > 0);
1382 |   const comparisonData = buildCategoryComparisonData(
1383 |     satisfactionData,
1384 |     summary?.previousCategorySatisfaction ?? [],
1385 |   );
1386 |   const hasPreviousComparison = comparisonData.some((item) => item.previousYear !== null);
1387 |   const radarData = buildRadarSatisfactionData(satisfactionData);
1388 |   const allReportThemes = selectedReport?.themes ?? [];
1389 |   const groupedIssues = allReportThemes
1390 |     .filter((theme) => !isUltraConcernTheme(theme))
1391 |     .sort((a, b) => priorityRank(b.priority) - priorityRank(a.priority) || b.mentionCount - a.mentionCount)
1392 |     .slice(0, 8);
1393 |   const actionQueue = [...(selectedReport?.actions ?? [])].slice(0, 6);
1394 |   const resultIssues = buildResultIssues(
1395 |     groupedIssues,
1396 |     actionQueue,
1397 |     selectedReport?.rawJson?.geminiReasoning?.priorityLabels,
1398 |     selectedReport?.rawJson?.geminiReasoning?.issueDetails,
1399 |   );
1400 |   const concerningIssues = buildConcerningIssues(
1401 |     allReportThemes,
1402 |     selectedReport?.rawJson?.geminiReasoning,
1403 |   );
1404 |   const bubbleMatrix = buildBubbleMatrixData(satisfactionData, groupedIssues);
1405 |   const urgentRiskCount = concerningIssues.filter(
1406 |     (issue) => issue.severity === 'CRITICAL' && issue.id !== 'no-critical-concern',
1407 |   ).length;
1408 |   const hasMoreHeatmapRows = bubbleMatrix.rows.length > 7;
1409 |   const visibleHeatmapRows = showFullHeatmap ? bubbleMatrix.rows : bubbleMatrix.rows.slice(0, 7);
1410 |   const departmentsMonitored = bubbleMatrix.departments.length;
1411 |   const kpiCards = buildAnalyticsKpis({
1412 |     confidence,
1413 |     criticalOpen: urgentRiskCount,
1414 |     departmentsMonitored,
1415 |     responseCount,
1416 |     satisfactionScore,
1417 |   });
1418 |   const aiActionReport = buildAiActionReport({
1419 |     comparisonData,
1420 |     departments: bubbleMatrix.departments,
1421 |     issues: resultIssues,
1422 |     responseCount,
1423 |     satisfactionScore,
1424 |   });
1425 |   const engineBenchmark = buildEngineBenchmarkModel(selectedReport, responseCount);

```

```

1426 | return (
1427 |   <div className="analysis-report-page premium-analytics-page">
1428 |     <button className="analysis-plain-back-button" type="button" onClick={onBack}>
1429 |       <ArrowLeft size={18} />
1430 |       Go Back
1431 |     </button>
1432 |
1433 |     <section className="analytics-hero analytics-blackhole-hero">
1434 |       <div className="flow-stage analysis-vortex-stage" aria-label="EduPulse AI animated feedback engine">
1435 |         <div className="floating-tech-icon fti-1"><Sparkles size={18} /></div>
1436 |         <div className="floating-tech-icon fti-2"><BarChart3 size={18} /></div>
1437 |         <div className="floating-tech-icon fti-3"><Gauge size={18} /></div>
1438 |         <div className="floating-tech-icon fti-4"><ShieldCheck size={18} /></div>
1439 |
1440 |         <div className="accretion-disk" />
1441 |         <div className="vortex-container">
1442 |           <div className="vortex-spiral" />
1443 |           <div className="vortex-spiral-2" />
1444 |           <div className="vortex-spiral-3" />
1445 |         </div>
1446 |
1447 |         <div className="blackhole-logo-wrapper">
1448 |           <img src={edupulseLogo} alt="EduPulse AI Logo" />
1449 |         </div>
1450 |
1451 |         <div className="blackhole-chip bh-path-1">"Wi-Fi not working in library"</div>
1452 |         <div className="blackhole-chip bh-path-2">"Mess food quality is poor"</div>
1453 |         <div className="blackhole-chip bh-path-3">"Ragging concern in hostel"</div>
1454 |         <div className="blackhole-chip bh-path-4">"Water contamination in block B"</div>
1455 |         <div className="blackhole-chip bh-path-5">"Random useless spam text"</div>
1456 |         <div className="blackhole-chip bh-path-6">"Projector issue in LH 203"</div>
1457 |         <div className="blackhole-chip bh-path-7">"Faculty absent in ECE block"</div>
1458 |         <div className="blackhole-chip bh-path-8">"Transport delay on route 4"</div>
1459 |         <div className="blackhole-chip bh-path-9">"Library books unavailable"</div>
1460 |         <div className="blackhole-chip bh-path-10">"Harassment report in building 2"</div>
1461 |       </div>
1462 |     </section>
1463 |
1464 |     {isLiveFeedbackAnalysis && (
1465 |       <div className="live-analysis-warning report-live-warning">
1466 |         <span className="live-analysis-dot" />
1467 |         <strong>Feedback is still live</strong>
1468 |         <em>
1469 |           Treat this AI result as provisional. Turn off feedback before presenting final numbers.
1470 |         </em>
1471 |       </div>
1472 |     )}
1473 |     {adminNotice && <p className="analysis-notice">{adminNotice}</p>}
1474 |
1475 |     <section className="premium-kpi-grid">
1476 |       {kpiCards.map((card) => (
1477 |         <article className={`premium-kpi-card ${card.tone}`} key={card.label}>
1478 |           <div>
1479 |             <span>{card.label}</span>
1480 |             <strong>{card.value}</strong>
1481 |           </div>
1482 |           <em className={card.trend >= 0 ? 'positive' : 'negative'}>
1483 |             {card.trend >= 0 ? '+' : ''}
1484 |             {card.trend}% vs previous
1485 |           </em>
1486 |           <ResponsiveContainer width="100%" height={42}>
1487 |             <LineChart data={card.sparkline.map((value, index) => ({ index, value })))>
1488 |               <Line
1489 |                 dataKey="value"
1490 |                 dot={false}
1491 |                 isAnimationActive
1492 |                 stroke={card.stroke}
1493 |                 strokeLinecap="round"
1494 |                 strokeWidth={2.5}
1495 |                 type="monotone"
1496 |               />
1497 |             </LineChart>
1498 |           </ResponsiveContainer>
1499 |         </article>
1500 |       )))
1501 |     </section>
1502 |
1503 |     <section className="analytics-chart-row">
1504 |       <article className="premium-panel comparison-panel inline-comparison-panel">
1505 |         <PanelHeader
1506 |           eyebrow={hasPreviousComparison ? 'Semester Comparison' : 'Current Semester'}
1507 |           title={hasPreviousComparison ? 'Current semester vs previous semester' : 'Current category satisfaction'}
1508 |           action={hasPreviousComparison ? 'Percentage score' : undefined}
1509 |         />
1510 |         <ResponsiveContainer width="100%" height={330}>
1511 |           <BarChart data={comparisonData} barCategoryGap="26%" barGap={8}>
1512 |             <CartesianGrid stroke="rgba(148, 163, 184, 0.18)" vertical={false} />
1513 |             <XAxis dataKey="category" tick={{ fill: '#475569', fontSize: 12 }} tickLine={false} axisLine={false} />
1514 |             <YAxis tick={{ fill: '#64748b', fontSize: 12 }} tickLine={false} axisLine={false} domain={[0, 100]} />
1515 |             <Tooltip />
1516 |             {hasPreviousComparison && (
1517 |               <Bar barSize={30} dataKey="previousYear" fill="#10b981" name="Previous Semester" radius={[0, 0, 0, 0]} />
1518 |             )}
1519 |             <Bar barSize={30} dataKey="currentYear" fill="#635bff" name="Current Semester" radius={[0, 0, 0, 0]} />
1520 |           </BarChart>
1521 |         </ResponsiveContainer>
1522 |         {!hasPreviousComparison && (
1523 |           <p className="comparison-empty-note">
1524 |             Previous semester data is not available for this tenant yet, so only current semester scores are shown.
1525 |           </p>
1526 |         )}
1527 |       </article>
1528 |
1529 |       <article className="premium-panel sentiment-panel-premium">
1530 |         <PanelHeader eyebrow="Rating Split" title={` ${ratingCategory} rating percentage`} />
1531 |         <div className="sentiment-tabs premium-tabs">
1532 |           {categoryOptions.map((category) => (
1533 |             <button
1534 |               className={ratingCategory === category ? 'selected' : ''}
1535 |               key={category}

```

```

1536 |         type="button"
1537 |         onClick={() => setRatingCategory(category)}
1538 |     >
1539 |         {category}
1540 |     </button>
1541 | ))}
1542 </div>
1543 {hasRatingSplit ? (
1544   <ResponsiveContainer width="100%" height={260}>
1545     <PieChart>
1546       <Pie
1547         data={ratingSplit}
1548         dataKey="chartValue"
1549         nameKey="label"
1550         innerRadius={76}
1551         outerRadius={112}
1552         paddingAngle={4}
1553       >
1554         {ratingSplit.map((item) => (
1555           <Cell fill={item.color} key={item.label} />
1556         ))}
1557       </Pie>
1558     <Tooltip />
1559   </PieChart>
1560 </ResponsiveContainer>
1561 ) : (
1562   <div className="sentiment-placeholder-donut">
1563     <span />
1564     <strong>No rating data</strong>
1565     <p>Submit feedback responses to generate this chart.</p>
1566   </div>
1567 )}
1568 <div className="modern-sentiment-legend">
1569   {ratingSplit.map((item) => (
1570     <span key={item.label}>
1571       <i style={{ background: item.color }} />
1572       {item.label}
1573       <b>{item.value}%</b>
1574       <em>{item.count.toLocaleString()} response(s)</em>
1575     </span>
1576   ))}
1577 </div>
1578 </article>
1579 </section>
1580
1581 <section className={`ai-findings-section ${selectedExactIssue ? 'has-detail' : ''}`>
1582   <div className="issues-found-header">
1583     <h2><span aria-hidden="true" />AI findings</h2>
1584     <p><strong>Note:</strong> Ranking of High, Medium and Low is done on the basis of sentiments and mentions.</p>
1585   </div>
1586   <div className="ai-findings-layout">
1587     <div className="ai-finding-grid">
1588       {resultIssues.map((issue) => (
1589         <article
1590           className={`ai-finding-card ${issue.status.toLowerCase()} ${
1591             selectedExactIssue?.id === issue.id ? 'selected' : ''
1592           }`}
1593           key={issue.id}
1594         >
1595           <div className="finding-card-top">
1596             <em>{issue.priority}</em>
1597           </div>
1598           <h3>{issue.shortTitle}</h3>
1599           <div className="finding-metrics">
1600             <span>{issue.mentions.toLocaleString()} mentions</span>
1601             <span>{issue.confidence}% confidence</span>
1602             <span>{issue.trend}</span>
1603           </div>
1604           <p>{issue.affectedDepartments.join(', ')} affected</p>
1605           <div className="finding-actions">
1606             <button type="button" onClick={() => setSelectedExactIssue(issue)}>
1607               View Analytics
1608             </button>
1609           </div>
1610         </article>
1611       ))}
1612     </div>
1613
1614     {selectedExactIssue && (
1615       <aside className="premium-panel investigation-panel side-investigation-panel">
1616         <div className="investigation-header">
1617           <div>
1618             <span>Exact AI Report</span>
1619             <h2>{selectedExactIssue.shortTitle}</h2>
1620           </div>
1621           <button type="button" onClick={() => setSelectedExactIssue(null)}>
1622             Close
1623           </button>
1624         </div>
1625         <div className="investigation-grid">
1626           <MetricBlock label="Location" value={selectedExactIssue.location} />
1627           <MetricBlock label="Priority" value={selectedExactIssue.priority} />
1628           <MetricBlock label="Mentions" value={selectedExactIssue.mentions.toLocaleString()} />
1629           <MetricBlock label="Confidence" value={` ${selectedExactIssue.confidence}%` } />
1630           <MetricBlock label="Affected Students" value={selectedExactIssue.affectedGroup} />
1631           <MetricBlock label="Expected Impact" value={selectedExactIssue.expectedImpact} />
1632         </div>
1633         <div className="investigation-copy-grid">
1634           <article>
1635             <span>AI Summary</span>
1636             <p>{selectedExactIssue.finding}</p>
1637           </article>
1638           <article>
1639             <span>Recommended Actions</span>
1640             <p>{selectedExactIssue.action}</p>
1641           </article>
1642         </div>
1643       </aside>
1644     )}
1645   </div>

```

```

1646 |     </section>
1647 |
1648 |     <section className="heatmap-radar-row">
1649 |       <article className="premium-panel bubble-heatmap-panel">
1650 |         <PanelHeader
1651 |           eyebrow=""
1652 |           title="Category satisfaction by department"
1653 |           action="Size = response volume"
1654 |         />
1655 |         <div className="bubble-matrix">
1656 |           <div className="bubble-matrix-head">
1657 |             <span />
1658 |             {bubbleMatrix.departments.map((department) => (
1659 |               <b key={department}>{department}</b>
1660 |             ))}
1661 |           </div>
1662 |           {visibleHeatmapRows.map((row) => (
1663 |             <div className="bubble-matrix-row" key={row.category}>
1664 |               <strong>{row.category}</strong>
1665 |               {row.cells.map((cell) => (
1666 |                 <button
1667 |                   className={`bubble-cell ${cell.tone}`}
1668 |                   key={` ${row.category}-${cell.department}`}
1669 |                   style={{ '--bubble-size': `${cell.size}px` } as React.CSSProperties}
1670 |                   title={` ${cell.department} ${row.category}: ${cell.score}% satisfaction, ${cell.responses} responses, ${cell.trend}`}
1671 |                   type="button"
1672 |                 >
1673 |                   <span>{cell.score}</span>
1674 |                   <em>{cell.trend}</em>
1675 |                 </button>
1676 |               ))}
1677 |             </div>
1678 |           ))}
1679 |         </div>
1680 |         {hasMoreHeatmapRows && (
1681 |           <button
1682 |             className="heatmap-show-more"
1683 |             type="button"
1684 |             onClick={() => setShowFullHeatmap((current) => !current)}
1685 |           >
1686 |             {showFullHeatmap ? 'Show less' : 'Show more (${bubbleMatrix.rows.length - 7)}'}
1687 |           </button>
1688 |         )}
1689 |       </article>
1690 |
1691 |       <article className="premium-panel radar-panel">
1692 |         <PanelHeader
1693 |           eyebrow="Satisfaction Radar"
1694 |           title=""
1695 |           action="Current semester"
1696 |         />
1697 |         <ResponsiveContainer width="100%" height={330}>
1698 |           <RadarChart data={radarData}>
1699 |             <PolarGrid stroke="rgba(148, 163, 184, 0.35)" />
1700 |             <PolarAngleAxis dataKey="category" tick={{ fill: '#475569', fontSize: 12 }} />
1701 |             <Radar dataKey="score" fill="#635bff" fillOpacity={0.18} stroke="#635bff" strokeWidth={2.5} />
1702 |             <Tooltip />
1703 |           </RadarChart>
1704 |         </ResponsiveContainer>
1705 |       </article>
1706 |     </section>
1707 |
1708 |     <ConcerningIssuesSection issues={concerningIssues} />
1709 |
1710 |     <AiActionReportCard comparisonData={comparisonData} downloadReport={downloadReport} report={aiActionReport} />
1711 |
1712 |     <AiEngineBenchmarkSection benchmark={engineBenchmark} />
1713 |
1714 |     <p className="analysis-team-footer">Developed by Team Eureka ♥</p>
1715 |   </div>
1716 | );
1717 | }
1718 |
1719 | function PanelHeader({
1720 |   action,
1721 |   eyebrow,
1722 |   title,
1723 | }): {
1724 |   action?: string;
1725 |   eyebrow: string;
1726 |   title: string;
1727 | }) {
1728 |   return (
1729 |     <div className="premium-panel-header">
1730 |       <div>
1731 |         {eyebrow && <span>{eyebrow}</span>}
1732 |         {title && <h2>{title}</h2>}
1733 |       </div>
1734 |       {action && <em>{action}</em>}
1735 |     </div>
1736 |   );
1737 | }
1738 |
1739 | function MetricBlock({ label, value }: { label: string; value: string }) {
1740 |   return (
1741 |     <div className="investigation-metric">
1742 |       <span>{label}</span>
1743 |       <strong>{value}</strong>
1744 |     </div>
1745 |   );
1746 | }
1747 |
1748 | function ConcerningIssuesSection({
1749 |   issues,
1750 | }): {
1751 |   issues: Return<Type>typeof buildConcerningIssues;
1752 | }) {
1753 |   const [selectedConcern, setSelectedConcern] = useState<typeof issues>[number] | null>(null);
1754 |
1755 |   return (

```

```

1756 | <section className="premium-panel concerning-issues-section">
1757 |   <div className="danger-alert-heading">
1758 |     <span aria-hidden="true" />
1759 |     <h2>Immediate action required</h2>
1760 |   </div>
1761 |   <div className={`concerning-issues-layout ${selectedConcern ? 'has-detail' : ''}`>
1762 |     <div className="concerning-issue-grid">
1763 |       {issues.map((issue) => (
1764 |         <article
1765 |           className={`concerning-issue-card ${issue.severity.toLowerCase()} ${
1766 |             selectedConcern?.id === issue.id ? 'selected' : ''
1767 |           }`}
1768 |           key={issue.id}
1769 |         >
1770 |           <div>
1771 |             <span>
1772 |               <AlertTriangle size={16} />
1773 |               {issue.riskType}
1774 |             </span>
1775 |           </div>
1776 |           <h3>{issue.title}</h3>
1777 |           <p>{issue.summary}</p>
1778 |           <dl>
1779 |             <div>
1780 |               <dt>Danger sign</dt>
1781 |               <dd>{issue.riskType}</dd>
1782 |             </div>
1783 |             <div>
1784 |               <dt>Mentions</dt>
1785 |               <dd>{issue.evidence}</dd>
1786 |             </div>
1787 |           </dl>
1788 |           <button type="button" onClick={() => setSelectedConcern(issue)}>
1789 |             View description
1790 |           </button>
1791 |         </article>
1792 |       )))
1793 |     </div>
1794 |
1795 |     {selectedConcern && (
1796 |       <aside className="danger-description-panel">
1797 |         <div>
1798 |           <span className="danger-live-dot" />
1799 |           <strong>{selectedConcern.riskType}</strong>
1800 |           <button type="button" onClick={() => setSelectedConcern(null)}>
1801 |             Close
1802 |           </button>
1803 |         </div>
1804 |         <h3>{selectedConcern.title}</h3>
1805 |         <div className="danger-description-copy">
1806 |           <article>
1807 |             <span>Detailed summary</span>
1808 |             <p>{selectedConcern.detailedSummary}</p>
1809 |           </article>
1810 |           <article className="danger-action-card">
1811 |             <span>Recommended action</span>
1812 |             <p>{selectedConcern.recommendedAction}</p>
1813 |           </article>
1814 |         </div>
1815 |         <div className="danger-description-stats">
1816 |           <span>
1817 |             <b>{selectedConcern.mentions.toLocaleString()}</b>
1818 |             <em>mentions</em>
1819 |           </span>
1820 |           <span>
1821 |             <b>{selectedConcern.severity}</b>
1822 |             <em>priority</em>
1823 |           </span>
1824 |         </div>
1825 |       </aside>
1826 |     )}
1827 |   </div>
1828 | </section>
1829 | );
1830 | }
1831 |
1832 | function AiQualityCheckSection({
1833 |   onToggleDetails,
1834 |   quality,
1835 |   showDetails,
1836 | }): {
1837 |   onToggleDetails: () => void;
1838 |   quality: AiQualityModel;
1839 |   showDetails: boolean;
1840 | } {
1841 |   const qualityStats = [
1842 |     { label: 'Valid comments', value: quality.validComments.toLocaleString(), tone: 'green' },
1843 |     { label: 'Fake/noise ignored', value: quality.lowQualityIgnored.toLocaleString(), tone: 'slate' },
1844 |     { label: 'Duplicates grouped', value: quality.duplicatesGrouped.toLocaleString(), tone: 'orange' },
1845 |     { label: 'Low-sample warnings', value: quality.lowSampleWarnings.toString(), tone: 'yellow' },
1846 |     { label: 'Safety escalations', value: quality.safetyEscalations.toString(), tone: 'red' },
1847 |     { label: 'AI confidence', value: `${quality.confidence}%`, tone: 'blue' },
1848 |   ];
1849 |
1850 |   return (
1851 |     <section className={`premium-panel ai-quality-section ${showDetails ? 'expanded' : ''}`>
1852 |       <div className="ai-quality-header">
1853 |         <div>
1854 |           <span>AI Bias & Quality Layer</span>
1855 |           <h2>Noise filtered before theme extraction</h2>
1856 |           <p>
1857 |             Quality checks remove fake text, duplicate comments, weak sample groups and safety-sensitive
1858 |             complaints before final analysis.
1859 |           </p>
1860 |         </div>
1861 |         <button type="button" onClick={onToggleDetails}>
1862 |           {showDetails ? 'Hide Detail Analysis' : 'View Detail Analysis'}
1863 |         </button>
1864 |       </div>
1865 |     </section>

```

```

1866 | <div className="ai-quality-stats">
1867 |   {qualityStats.map((item) => (
1868 |     <div className={ `quality-stat ${item.tone}` } key={item.label}>
1869 |       <span>{item.label}</span>
1870 |       <strong>{item.value}</strong>
1871 |     </div>
1872 |   ))}
1873 | </div>
1874 |
1875 | {showDetails && (
1876 |   <div className="ai-quality-details">
1877 |     <div className="ai-quality-drilldown">
1878 |       <div className="quality-evidence-toolbar">
1879 |         <div>
1880 |           <strong>Evidence summary, not raw overload</strong>
1881 |           <p>
1882 |             Large noisy datasets are grouped by reason and only top evidence examples are shown.
1883 |           </p>
1884 |         </div>
1885 |         <button type="button" onClick={() => exportQualityEvidence(quality)}>
1886 |           Export evidence CSV
1887 |         </button>
1888 |       </div>
1889 |
1890 |       <div className="quality-breakdown-grid">
1891 |         <QualityBreakdownCard
1892 |           accent="red"
1893 |           rows={quality.rejectedBreakdown.filter((row) => row.reason !== 'duplicate')}
1894 |           title="Rejected fake/noise breakdown"
1895 |         />
1896 |         <QualityBreakdownCard
1897 |           accent="orange"
1898 |           rows={[
1899 |             {
1900 |               count: quality.duplicatesGrouped,
1901 |               label: 'Duplicate comments grouped as repeated signal',
1902 |               reason: 'duplicate',
1903 |             },
1904 |           ]}
1905 |           title="Duplicate grouping summary"
1906 |         />
1907 |         <QualityBreakdownCard
1908 |           accent="yellow"
1909 |           rows={quality.lowSampleWarningsList.map((item) => ({
1910 |             count: item.responses,
1911 |             label: `${item.topic} - ${item.group}`,
1912 |             reason: 'low_sample',
1913 |           ))}
1914 |           title="Low-sample groups"
1915 |         />
1916 |         <QualityBreakdownCard
1917 |           accent="green"
1918 |           rows={quality.safetyEscalationDetails.map((item) => ({
1919 |             count: 1,
1920 |             label: item.issue,
1921 |             reason: item.priority,
1922 |           ))}
1923 |           title="Safety escalations"
1924 |         />
1925 |       </div>
1926 |     </div>
1927 |     <QualityDetailGroup
1928 |       accent="red"
1929 |       items={quality.rejectedExamples.map((item) => ({
1930 |         meta: `${item.topic} • ${item.source}`,
1931 |         primary: `${item.comment}`,
1932 |         secondary: `${item.question} · Reason: ${item.reason}`,
1933 |         tag: item.decision,
1934 |       ))}
1935 |       title="Rejected fake examples"
1936 |     />
1937 |     <QualityDetailGroup
1938 |       accent="orange"
1939 |       items={quality.duplicateExamples.map((item) => ({
1940 |         meta: item.topic,
1941 |         primary: `${item.comment}`,
1942 |         secondary: `Repeated ${item.repeated} times`,
1943 |         tag: item.decision,
1944 |       ))}
1945 |       title="Duplicate grouped examples"
1946 |     />
1947 |     <QualityDetailGroup
1948 |       accent="yellow"
1949 |       items={quality.lowSampleWarningsList.map((item) => ({
1950 |         meta: `${item.topic} • ${item.group}`,
1951 |         primary: `${item.responses} responses`,
1952 |         secondary: 'Insight is shown with lower confidence until more responses arrive.',
1953 |         tag: item.decision,
1954 |       ))}
1955 |       title="Low sample warning details"
1956 |     />
1957 |     <QualityDetailGroup
1958 |       accent="green"
1959 |       items={quality.safetyEscalationDetails.map((item) => ({
1960 |         meta: item.source,
1961 |         primary: item.issue,
1962 |         secondary: 'Safety-sensitive signal is escalated even when sample size is small.',
1963 |         tag: `${item.priority} • ${item.decision}`,
1964 |       ))}
1965 |       title="Safety escalation details"
1966 |     />
1967 |   </div>
1968 | )}
1969 | </section>
1970 | );
1971 | }
1972 |
1973 | function QualityDetailGroup({
1974 |   accent,
1975 |   items,

```

```

1976 | title,
1977 | }: {
1978 |   accent: 'green' | 'orange' | 'red' | 'yellow';
1979 |   items: {
1980 |     meta: string;
1981 |     primary: string;
1982 |     secondary: string;
1983 |     tag: string;
1984 |   }[];
1985 |   title: string;
1986 | }) {
1987 |   return (
1988 |     <article className={`quality-detail-group ${accent}`}>
1989 |       <h3>{title}</h3>
1990 |       <div>
1991 |         {items.map((item, index) => (
1992 |           <section className="quality-detail-item" key={` ${title}-${index}`}>
1993 |             <span>{item.meta}</span>
1994 |             <strong>{item.primary}</strong>
1995 |             <p>{item.secondary}</p>
1996 |             <em>{item.tag}</em>
1997 |           </section>
1998 |         ))}
1999 |       </div>
2000 |     </article>
2001 |   );
2002 | }
2003 |
2004 | function QualityBreakdownCard({
2005 |   accent,
2006 |   rows,
2007 |   title,
2008 | }): {
2009 |   accent: 'green' | 'orange' | 'red' | 'yellow';
2010 |   rows: {
2011 |     count: number;
2012 |     label: string;
2013 |     reason: string;
2014 |   }[];
2015 |   title: string;
2016 | }) {
2017 |   const safeRows = rows.length
2018 |     ? rows
2019 |     : [{ count: 0, label: 'No active signal in this group', reason: 'clear' }];
2020 |   const total = safeRows.reduce((sum, row) => sum + row.count, 0);
2021 |
2022 |   return (
2023 |     <article className={`quality-breakdown-card ${accent}`}>
2024 |       <div>
2025 |         <h3>{title}</h3>
2026 |         <strong>{total.toLocaleString()}</strong>
2027 |       </div>
2028 |       <div>
2029 |         {safeRows.slice(0, 6).map((row) => (
2030 |           <section key={` ${title}-${row.reason}-${row.label}`}>
2031 |             <span>{row.label}</span>
2032 |             <b>{row.count.toLocaleString()}</b>
2033 |             <em>{prettifyQualityReason(row.reason)}</em>
2034 |           </section>
2035 |         ))}
2036 |       </div>
2037 |     </article>
2038 |   );
2039 | }
2040 |
2041 | function exportQualityEvidence(quality: AiQualityModel) {
2042 |   const rows = [
2043 |     ['section', 'category_or_reason', 'count_or_repeated', 'example_or_note', 'decision'],
2044 |     ...quality.rejectedBreakdown.map((row) => [
2045 |       'rejected_breakdown',
2046 |       row.reason,
2047 |       String(row.count),
2048 |       row.label,
2049 |       'Ignored before theme analysis',
2050 |     ]),
2051 |     ...quality.duplicateExamples.map((item) => [
2052 |       'duplicate_group',
2053 |       item.topic,
2054 |       String(item.repeated),
2055 |       item.comment,
2056 |       item.decision,
2057 |     ]),
2058 |     ...quality.rejectedExamples.map((item) => [
2059 |       'rejected_example',
2060 |       item.topic,
2061 |       item.reason,
2062 |       item.comment,
2063 |       item.decision,
2064 |     ]),
2065 |     ...quality.lowSampleWarningsList.map((item) => [
2066 |       'low_sample',
2067 |       item.topic,
2068 |       String(item.responses),
2069 |       item.group,
2070 |       item.decision,
2071 |     ]),
2072 |     ...quality.safetyEscalationDetails.map((item) => [
2073 |       'safety_escalation',
2074 |       item.priority,
2075 |       '1',
2076 |       item.issue,
2077 |       item.decision,
2078 |     ]),
2079 |   ];
2080 |   const csv = rows
2081 |     .map((row) => row.map((cell) => `${String(cell).replace(/"/g, '"')}").join(',')
2082 |     ).join('\n');
2083 |   const blob = new Blob([csv], { type: 'text/csv;charset=utf-8' });
2084 |   const url = URL.createObjectURL(blob);
2085 |   const link = document.createElement('a');

```

```

2086 |     link.href = url;
2087 |     link.download = 'edupulse-ai-quality-evidence.csv';
2088 |     link.click();
2089 |     URL.revokeObjectURL(url);
2090 | }
2091 |
2092 | function AiActionReportCard({
2093 |     comparisonData,
2094 |     downloadReport,
2095 |     report,
2096 | }): {
2097 |     comparisonData: Returntype<typeof buildCategoryComparisonData>;
2098 |     downloadReport: () => void;
2099 |     report: AiActionReportModel;
2100 | } {
2101 |     const hasPreviousComparison = comparisonData.some((item) => item.previousYear !== null);
2102 |
2103 |     return (
2104 |         <section className="premium-panel ai-action-report-card">
2105 |             <div className="action-report-blur-content" aria-hidden="true">
2106 |                 <div className="action-report-header">
2107 |                     <div>
2108 |                         <span>AI Action Report</span>
2109 |                         <h2>Institutional improvement plan generated from feedback intelligence</h2>
2110 |                     </div>
2111 |                 </div>
2112 |
2113 |                 <article className="action-report-summary">
2114 |                     <div>
2115 |                         <span>1. Executive AI Summary</span>
2116 |                         <p>{report.summary}</p>
2117 |                     </div>
2118 |                     <div className="summary-metric-strip">
2119 |                         {report.metrics.map((metric) => (
2120 |                             <strong key={metric.label}>
2121 |                                 {metric.value}
2122 |                                 <em>{metric.label}</em>
2123 |                             </strong>
2124 |                         ))}
2125 |                     </div>
2126 |                 </article>
2127 |
2128 |                 <article className="action-report-section">
2129 |                     <div className="action-section-title">
2130 |                         <span>2. Top Issues Found</span>
2131 |                         <em>Priority assigned by LLM after EduPulse clustering</em>
2132 |                     </div>
2133 |                     <div className="action-issue-table">
2134 |                         <div className="action-issue-head">
2135 |                             <b>Issue</b>
2136 |                             <b>Priority by AI</b>
2137 |                             <b>Mentions</b>
2138 |                             <b>Affected Students</b>
2139 |                             <b>Evidence Signal</b>
2140 |                         </div>
2141 |                         {report.topIssues.map((issue) => (
2142 |                             <div className="action-issue-row" key={` ${issue.issue}-${issue.mentions}`}>
2143 |                                 <strong>{issue.issue}</strong>
2144 |                                 <span className={ `priority-pill ${issue.priority.toLowerCase()} `}>{issue.priority}</span>
2145 |                                 <span>{issue.mentions.toLocaleString()}</span>
2146 |                                 <span>{issue.affectedStudents}</span>
2147 |                                 <p>{issue.evidenceSignal}</p>
2148 |                             </div>
2149 |                         ))}
2150 |                     </div>
2151 |                 </article>
2152 |
2153 |                 <div className="action-report-two-column">
2154 |                     <article className="action-report-section">
2155 |                         <div className="action-section-title">
2156 |                             <span>3. Root Cause From Student Comments</span>
2157 |                             <em>Comment-based only</em>
2158 |                         </div>
2159 |                         <div className="root-cause-grid">
2160 |                             {report.rootCauses.map((cause) => (
2161 |                                 <div className="root-cause-card" key={cause.title}>
2162 |                                     <strong>{cause.title}</strong>
2163 |                                     <p>{cause.body}</p>
2164 |                                     <span>{cause.signal}</span>
2165 |                                 </div>
2166 |                             ))}
2167 |                         </div>
2168 |                     </article>
2169 |
2170 |                     <article className="action-report-section">
2171 |                         <div className="action-section-title">
2172 |                             <span>4. AI Action Plan</span>
2173 |                             <em>Issues + root causes sent to LLM</em>
2174 |                         </div>
2175 |                         <div className="ai-action-plan-list">
2176 |                             {report.actionPlan.map((item, index) => (
2177 |                                 <div className="ai-action-plan-item" key={` ${item.title}-${index}`}>
2178 |                                     <b>{index + 1}</b>
2179 |                                     <div>
2180 |                                         <span className={ `priority-pill ${item.priority.toLowerCase()} `}>{item.priority}</span>
2181 |                                         <strong>{item.title}</strong>
2182 |                                         <p>{item.body}</p>
2183 |                                     </div>
2184 |                                 </div>
2185 |                             ))}
2186 |                         </div>
2187 |                     </article>
2188 |                 </div>
2189 |
2190 |                 <article className="action-report-section semester-line-section">
2191 |                     <div className="action-section-title">
2192 |                         <span>{hasPreviousComparison ? '5. Semester Comparison Graph' : '5. Current Semester Category Graph'}</span>
2193 |                         <em>{hasPreviousComparison ? 'Current semester vs previous semester' : 'Previous semester data unavailable'}</em>
2194 |                     </div>
2195 |                     <div className="line-legend">

```



```

2196 |         <span><i className="current" />Current Semester</span>
2197 |         {hasPreviousComparison && <span><i className="previous" />Previous Semester</span>}
2198 |     </div>
2199 |     <ResponsiveContainer width="100%" height={320}>
2200 |         <LineChart data={comparisonData}>
2201 |             <CartesianGrid stroke="rgba(148, 163, 184, 0.2)" vertical={false} />
2202 |             <XAxis dataKey="category" tick={{ fill: '#475569', fontSize: 12 }} tickLine={false} axisLine={false} />
2203 |             <YAxis tick={{ fill: '#64748b', fontSize: 12 }} tickLine={false} axisLine={false} domain={[0, 100]} />
2204 |             <Tooltip />
2205 |             <Line
2206 |                 activeDot={{ r: 7 }}
2207 |                 dataKey="currentYear"
2208 |                 dot={{ r: 5 }}
2209 |                 name="Current Semester"
2210 |                 stroke="#2563eb"
2211 |                 strokeLinecap="round"
2212 |                 strokeWidth={3.5}
2213 |                 type="monotone"
2214 |             />
2215 |             <Line
2216 |                 activeDot={{ r: 7 }}
2217 |                 dataKey="previousYear"
2218 |                 dot={{ r: 5 }}
2219 |                 name="Previous Semester"
2220 |                 stroke="#10b981"
2221 |                 strokeLinecap="round"
2222 |                 strokeWidth={3.5}
2223 |                 type="monotone"
2224 |                 hide={!hasPreviousComparison}
2225 |             />
2226 |         </LineChart>
2227 |     </ResponsiveContainer>
2228 |     {!hasPreviousComparison && (
2229 |         <p className="comparison-empty-note">
2230 |             Once another semester is stored for this college, this graph will automatically become a true comparison.
2231 |         </p>
2232 |     )}
2233 | </article>
2234 | </div>
2235 |
2236 | <div className="action-report-lock-overlay">
2237 |     <div className="action-report-lock-card">
2238 |         <span><Sparkles size={16} /> AI report ready</span>
2239 |         <h3>Generate the final institutional action report</h3>
2240 |         <button className="action-report-download" type="button" onClick={downloadReport}>
2241 |             <Download size={17} />
2242 |             Run AI to download report
2243 |         </button>
2244 |     </div>
2245 | </div>
2246 | </section>
2247 | );
2248 | }
2249 |
2250 | function AiEngineBenchmarkSection({
2251 |     benchmark,
2252 | }): {
2253 |     benchmark: AiEngineBenchmarkModel;
2254 | } {
2255 |     return (
2256 |         <section className="premium-panel ai-engine-benchmark-section">
2257 |             <div className="benchmark-header">
2258 |                 <div>
2259 |                     <h2>AI engine benchmark</h2>
2260 |                 </div>
2261 |             </div>
2262 |
2263 |             <div className="benchmark-funnel">
2264 |                 {benchmark.funnel.map((step, index) => (
2265 |                     <div className="funnel-step" key={step.label}>
2266 |                         <strong>{step.value}</strong>
2267 |                         <span>{step.label}</span>
2268 |                         <p>{step.note}</p>
2269 |                         {index < benchmark.funnel.length - 1 && <i aria-hidden="true" />}
2270 |                     </div>
2271 |                 ))}
2272 |             </div>
2273 |
2274 |             <div className="benchmark-lower">
2275 |                 <div className="ai-run-receipt">
2276 |                     <div>
2277 |                         <span>AI RUN RECEIPT</span>
2278 |                         <b>verified engine output</b>
2279 |                     </div>
2280 |                     {benchmark.receipt.map((row) => (
2281 |                         <p key={row.label}>
2282 |                             <span>{row.label}</span>
2283 |                             <strong>{row.value}</strong>
2284 |                         </p>
2285 |                     ))}
2286 |                 </div>
2287 |
2288 |                 <div className="ai-pipeline-trace">
2289 |                     <span>Pipeline Trace</span>
2290 |                     {benchmark.pipeline.map((step, index) => (
2291 |                         <p key={` ${step} - ${index} `}>
2292 |                             <i />
2293 |                             <code>[{String(index + 1).padStart(2, '0')}] {humanizePipelineStep(step)}</code>
2294 |                         </p>
2295 |                     ))}
2296 |                 </div>
2297 |             </div>
2298 |         </section>
2299 |     );
2300 | }
2301 |
2302 | function AiAnalysisProgress({ activeStep }: { activeStep: number }) {
2303 |     const safeStep = Math.min(Math.max(activeStep, 0), aiPipelineSteps.length - 1);
2304 |     const active = aiPipelineSteps[safeStep];
2305 |     const progress = Math.round(((safeStep + 1) / aiPipelineSteps.length) * 100);

```

```

2306 |
2307 | return (
2308 |   <section className="ai-progress-panel" aria-live="polite">
2309 |     <div className="ai-progress-main">
2310 |       <span className="ai-progress-orb">
2311 |         <Sparkles size={22} />
2312 |       </span>
2313 |       <div>
2314 |         <span className="hero-kicker">AI analysis running</span>
2315 |         <h2>{active.title}</h2>
2316 |         <p>{active.detail}</p>
2317 |       </div>
2318 |     </div>
2319 |     <div className="ai-progress-meter">
2320 |       <div className="ai-progress-meter-top">
2321 |         <span>Pipeline progress</span>
2322 |         <strong>{progress}%</strong>
2323 |       </div>
2324 |       <div className="ai-progress-track">
2325 |         <span style={{ width: `${progress}%` }} />
2326 |       </div>
2327 |       <div className="ai-progress-steps">
2328 |         {aiPipelineSteps.map((step, index) => {
2329 |           const state = index < safeStep ? 'done' : index === safeStep ? 'active' : 'pending';
2330 |           return (
2331 |             <div className={`ai-progress-step ${state}`} key={step.title}>
2332 |               <span>{state === 'done' ? <CheckCircle2 size={14} /> : index + 1}</span>
2333 |               <div>
2334 |                 <strong>{step.title}</strong>
2335 |                 <p>{step.detail}</p>
2336 |               </div>
2337 |             </div>
2338 |           );
2339 |         })}
2340 |       </div>
2341 |     </div>
2342 |   </section>
2343 | );
2344 | }
2345 |
2346 | function AdminHome({
2347 |   activeTermName,
2348 |   auditLogCount,
2349 |   analysisTermId,
2350 |   notice,
2351 |   terms,
2352 |   onAnalysisTermChange,
2353 |   onChoose,
2354 |   onRefreshDatasets,
2355 | }): {
2356 |   activeTermName?: string;
2357 |   auditLogCount: number;
2358 |   analysisTermId: string;
2359 |   notice?: string;
2360 |   terms: Term[];
2361 |   onAnalysisTermChange: (termId: string) => void;
2362 |   onChoose: (view: 'create' | 'analysis' | 'audit') => void;
2363 |   onRefreshDatasets: () => void;
2364 | } {
2365 |   const [isDrawerOpen, setIsDrawerOpen] = useState(false);
2366 |   const [isLocalRefreshing, setIsLocalRefreshing] = useState(false);
2367 |
2368 |   const handleRefreshClick = () => {
2369 |     setIsLocalRefreshing(true);
2370 |     onRefreshDatasets();
2371 |     setTimeout(() => {
2372 |       setIsLocalRefreshing(false);
2373 |     }, 1000);
2374 |   };
2375 |
2376 |   const selectedDataset = terms.find((item) => item.id === analysisTermId);
2377 |
2378 |   return (
2379 |     <section className="admin-home-panel reveal-on-scroll">
2380 |       /* Decorative background grid and glowing accent */
2381 |       <div className="admin-grid-lines" />
2382 |       <div className="admin-gradient-orb" />
2383 |
2384 |       <div className="admin-home-header">
2385 |         <h1 className="admin-dashboard-title">Admin Dashboard</h1>
2386 |         {notice && <p className="admin-home-notice">{notice}</p>}
2387 |       </div>
2388 |
2389 |       /* Redesigned Dataset Panel */
2390 |       <div className="admin-dataset-panel">
2391 |         <div className="dataset-panel-left">
2392 |           <div className="dataset-active-info">
2393 |             <span className="dataset-label">ACTIVE DATASET</span>
2394 |             <strong className="dataset-value-name">
2395 |               {selectedDataset ? selectedDataset.name : 'No active dataset selected'}
2396 |             </strong>
2397 |           </div>
2398 |         </div>
2399 |
2400 |         <div className="dataset-panel-right">
2401 |           <button
2402 |             className="btn-admin-choose-dataset"
2403 |             type="button"
2404 |             onClick={() => setIsDrawerOpen(true)}
2405 |           >
2406 |             Choose dataset
2407 |           </button>
2408 |           <button
2409 |             className={`btn-admin-refresh-datasets ${isLocalRefreshing ? 'spinning' : ''}`}
2410 |             type="button"
2411 |             onClick={handleRefreshClick}
2412 |           >
2413 |             <RefreshCw size={16} />
2414 |             <span>Refresh datasets</span>
2415 |           </button>

```

```

2416 |     </div>
2417 | </div>
2418 |
2419 | /* Redesigned Side Drawer for Datasets Selection */
2420 | {isDrawerOpen && (
2421 |   <
2422 |     <div className="dataset-drawer-overlay" onClick={() => setIsDrawerOpen(false)} />
2423 |     <div className="dataset-drawer open">
2424 |       <div className="drawer-header">
2425 |         <h3>Select Semester Dataset</h3>
2426 |         <button
2427 |           className="drawer-close-btn"
2428 |           type="button"
2429 |           onClick={() => setIsDrawerOpen(false)}
2430 |         >
2431 |           <X size={18} />
2432 |         </button>
2433 |       </div>
2434 |       <div className="drawer-dataset-list">
2435 |         {terms.length ? (
2436 |           terms.map((item) => (
2437 |             <button
2438 |               key={item.id}
2439 |               className={`drawer-dataset-card ${analysisTermId === item.id ? 'active' : ''}`}
2440 |               type="button"
2441 |               onClick={() => {
2442 |                 onAnalysisTermChange(item.id);
2443 |                 setIsDrawerOpen(false);
2444 |               }}
2445 |             >
2446 |               <div className="dataset-card-info">
2447 |                 <strong>{item.name}</strong>
2448 |                 <span className={`dataset-card-status ${item.isActive ? 'live' : 'stored'}`}>
2449 |                   {item.isActive ? 'Active dataset' : 'Stored dataset'}
2450 |                 </span>
2451 |               </div>
2452 |               {analysisTermId === item.id && (
2453 |                 <CheckCircle2 size={18} className="active-check-icon" />
2454 |               )}
2455 |             </button>
2456 |           ))
2457 |         ) : (
2458 |           <p className="drawer-empty-text">No semester datasets found.</p>
2459 |         )}
2460 |       </div>
2461 |     </div>
2462 |   </>
2463 | )}
2464 |
2465 | /* Redesigned Action Cards Grid */
2466 | <div className="admin-action-grid">
2467 |   <button className="admin-action-card create-card" type="button" onClick={() => onChoose('create')}>
2468 |     <div className="action-card-accent" />
2469 |     <div className="action-icon-wrapper">
2470 |       <ClipboardList size={28} />
2471 |     </div>
2472 |     <span className={`admin-card-live-mini ${activeTermName ? 'live' : 'closed'}`}>
2473 |       <i />
2474 |       {activeTermName ? 'Live' : 'Off'}
2475 |     </span>
2476 |     <strong>Create feedback</strong>
2477 |     <span className="action-card-subtitle">Forms & Categories</span>
2478 |     <p>
2479 |       Create questions and publish semester feedback.
2480 |     </p>
2481 |     <em className={`action-card-badge ${activeTermName ? 'live' : 'closed'}`}>
2482 |       {activeTermName ? `Live: ${activeTermName}` : 'No feedback live'}
2483 |     </em>
2484 |   </button>
2485 |
2486 |   <button className="admin-action-card analysis-card" type="button" onClick={() => onChoose('analysis')}>
2487 |     <div className="action-card-accent" />
2488 |     <div className="action-icon-wrapper">
2489 |       <BarChart3 size={28} />
2490 |     </div>
2491 |     <strong>Run analysis</strong>
2492 |     <span className="action-card-subtitle">AI Insights</span>
2493 |     <p>
2494 |       Analyze selected feedback and open charts, issues, and reports.
2495 |     </p>
2496 |     <em className="action-card-badge highlight">AI Dashboard</em>
2497 |   </button>
2498 |
2499 |   <button className="admin-action-card audit-card" type="button" onClick={() => onChoose('audit')}>
2500 |     <div className="action-card-accent" />
2501 |     <div className="action-icon-wrapper">
2502 |       <History size={28} />
2503 |     </div>
2504 |     <strong>Audit logs</strong>
2505 |     <span className="action-card-subtitle">Activity Trail</span>
2506 |     <p>
2507 |       Review logins, feedback changes, reports, and admin activity.
2508 |     </p>
2509 |     <em className={`action-card-badge audit ${auditLogCount > 0 ? 'pulse' : ''}`}>
2510 |       {auditLogCount} recent event(s)
2511 |     </em>
2512 |   </button>
2513 | </div>
2514 |
2515 | </section>
2516 | );
2517 | }
2518 |
2519 | function AdminCreateFeedback({
2520 |   activeTermName,
2521 |   auth,
2522 |   isFeedbackLive,
2523 |   onBack,
2524 |   onChanged,
2525 | }: {

```

```

2526 | activeTermName?: string;
2527 | auth: AuthState;
2528 | isFeedbackLive: boolean;
2529 | onBack: () => void;
2530 | onChanged: () => void | Promise<void>;
2531 | }) {
2532 |   const [termName, setTermName] = useState('Semester Feedback 2026');
2533 |   const [notice, setNotice] = useState('');
2534 |   const [categories, setCategories] = useState<AdminFeedbackCategoryDraft[]>([
2535 |     {
2536 |       name: 'Academics',
2537 |       description: 'Course structure, workload and learning clarity.',
2538 |       questions: [
2539 |         'How satisfied are you with the course structure?',
2540 |         'How manageable is the academic workload?',
2541 |         'How useful are assignments for practical understanding?',
2542 |       ],
2543 |     },
2544 |     {
2545 |       name: 'Faculty',
2546 |       description: 'Teaching quality, support and classroom communication.',
2547 |       questions: [
2548 |         'How satisfied are you with teaching clarity?',
2549 |         'How helpful is faculty support for doubts?',
2550 |         'How fair is classroom communication from faculty?',
2551 |       ],
2552 |     },
2553 |   ]);
2554 |
2555 |   function updateCategory(index: number, patch: Partial<AdminFeedbackCategoryDraft>) {
2556 |     setCategories((current) =>
2557 |       current.map((category, categoryIndex) =>
2558 |         categoryIndex === index ? { ...category, ...patch } : category,
2559 |       ),
2560 |     );
2561 |   }
2562 |
2563 |   function updateQuestion(categoryIndex: number, questionIndex: number, value: string) {
2564 |     setCategories((current) =>
2565 |       current.map((category, currentCategoryIndex) =>
2566 |         currentCategoryIndex === categoryIndex
2567 |           ? {
2568 |             ...category,
2569 |             questions: category.questions.map((question, currentQuestionIndex) =>
2570 |               currentQuestionIndex === questionIndex ? value : question,
2571 |             ),
2572 |           }
2573 |           : category,
2574 |       ),
2575 |     );
2576 |   }
2577 |
2578 |   async function makeLive() {
2579 |     setNotice('');
2580 |     const cleanCategories = categories
2581 |       .map((category) => ({
2582 |         ...category,
2583 |         name: category.name.trim(),
2584 |         description: category.description.trim(),
2585 |         questions: category.questions.map((question) => question.trim()).filter(Boolean),
2586 |       }));
2587 |     .filter((category) => category.name && category.questions.length);
2588 |
2589 |     if (!termName.trim() || !cleanCategories.length) {
2590 |       setNotice('Add a term name and at least one category with one question.');
```

```

2636 |         <button className="secondary-button" type="button" onClick={makeLive}>
2637 |             Make feedback live
2638 |         </button>
2639 |         <button className="icon-button danger-outline" type="button" onClick={turnOff}>
2640 |             Turn off feedback
2641 |         </button>
2642 |     </div>
2643 | </div>
2644 | <div className={`feedback-live-status ${isFeedbackLive ? 'live' : 'closed'}}>
2645 |     <span className="feedback-live-light" />
2646 |     <div>
2647 |         <strong>{isFeedbackLive ? 'Feedback is live' : 'Feedback is not live'}</strong>
2648 |         <em>
2649 |             {isFeedbackLive
2650 |               ? `${activeTermName ?? 'Current feedback'} is accepting student responses.`
2651 |               : 'Students currently see no semester feedback form to fill.'}
2652 |         </em>
2653 |     </div>
2654 | </div>
2655 | <label>
2656 |     Feedback term name
2657 |     <input value={termName} onChange={(event) => setTermName(event.target.value)} />
2658 | </label>
2659 | <div className="builder-category-list">
2660 |     {categories.map((category, categoryIndex) => (
2661 |         <article className="builder-category" key={categoryIndex}>
2662 |             <label>
2663 |                 Category name
2664 |                 <input
2665 |                     value={category.name}
2666 |                     onChange={(event) => updateCategory(categoryIndex, { name: event.target.value })}
2667 |                 />
2668 |             </label>
2669 |             <label>
2670 |                 Category description
2671 |                 <input
2672 |                     value={category.description}
2673 |                     onChange={(event) =>
2674 |                         updateCategory(categoryIndex, { description: event.target.value })
2675 |                     }
2676 |                 />
2677 |             </label>
2678 |             <div className="builder-question-list">
2679 |                 {category.questions.map((question, questionIndex) => (
2680 |                     <label key={questionIndex}>
2681 |                         Question {questionIndex + 1}
2682 |                         <input
2683 |                             value={question}
2684 |                             onChange={(event) =>
2685 |                                 updateQuestion(categoryIndex, questionIndex, event.target.value)
2686 |                             }
2687 |                         />
2688 |                     </label>
2689 |                 ))}
2690 |             </div>
2691 |             <button
2692 |                 className="text-button"
2693 |                 type="button"
2694 |                 onClick={() =>
2695 |                     updateCategory(categoryIndex, {
2696 |                         questions: [...category.questions, ''],
2697 |                     })
2698 |                 }
2699 |             >
2700 |                 + Add question
2701 |             </button>
2702 |         </article>
2703 |     ))}
2704 | </div>
2705 | <button
2706 |     className="primary-button"
2707 |     type="button"
2708 |     onClick={() =>
2709 |         setCategories((current) => [
2710 |             ...current,
2711 |             { name: '', description: '', questions: [] },
2712 |         ])
2713 |     }
2714 | >
2715 |     + Add category
2716 | </button>
2717 | {notice && <p className="admin-notice">{notice}</p>}
2718 | </section>
2719 | </div>
2720 | );
2721 | }
2722 |
2723 | function AdminAuditLogsView({
2724 |     auditLogs,
2725 |     onBack,
2726 |     onRefresh,
2727 | }): {
2728 |     auditLogs: AuditLog[];
2729 |     onBack: () => void;
2730 |     onRefresh: () => void;
2731 | } {
2732 |     const [selected, setSelected] = useState<AuditLog | null>(auditLogs[0] ?? null);
2733 |     const [visibleCount, setVisibleCount] = useState(12);
2734 |
2735 |     useEffect(() => {
2736 |         setSelected((current) => {
2737 |             if (current && auditLogs.some((item) => item.id === current.id)) {
2738 |                 return current;
2739 |             }
2740 |             return auditLogs[0] ?? null;
2741 |         });
2742 |         setVisibleCount(12);
2743 |     }, [auditLogs]);
2744 |
2745 |     const actorName = selected?.actor?.name ?? 'System';

```

```

2746 | const metadataRows = getAuditMetadataRows(selected?.metadata);
2747 | const collegeName = selected?.college?.name ?? auditLogs[0]?.college?.name ?? 'Current college';
2748 | const visibleAuditLogs = auditLogs.slice(0, visibleCount);
2749 | const hasMoreAuditLogs = visibleCount < auditLogs.length;
2750 |
2751 | return (
2752 |   <div className="admin-stack audit-workspace">
2753 |     <button className="auth-back-button admin-back-button" type="button" onClick={onBack}>
2754 |       Go back
2755 |     </button>
2756 |
2757 |     <section className="audit-hero-panel">
2758 |       <div>
2759 |         <p className="eyebrow">Admin activity trail</p>
2760 |         <h2>Audit logs</h2>
2761 |         <span>
2762 |           Tenant-scoped activity for {collegeName}. Every login, feedback change, analysis run,
2763 |           and report access is recorded for review.
2764 |         </span>
2765 |       </div>
2766 |       <button className="audit-refresh-button" type="button" onClick={onRefresh}>
2767 |         <RefreshCw size={16} />
2768 |         Refresh activity
2769 |       </button>
2770 |     </section>
2771 |
2772 |     <section className="audit-log-grid">
2773 |       <div className="audit-feed-panel">
2774 |         <div className="audit-feed-header">
2775 |           <div>
2776 |             <p className="eyebrow">Recent events</p>
2777 |             <h3>{auditLogs.length} recorded action(s)</h3>
2778 |           </div>
2779 |           <ShieldCheck size={22} />
2780 |         </div>
2781 |
2782 |         <div className="audit-timeline">
2783 |           {auditLogs.length ? (
2784 |             <>
2785 |               {visibleAuditLogs.map((log) => (
2786 |                 <button
2787 |                   className={`audit-log-row ${selected?.id === log.id ? 'selected' : ''}`}
2788 |                   key={log.id}
2789 |                   type="button"
2790 |                   onClick={() => setSelected(log)}
2791 |                 >
2792 |                   <span className="audit-dot" />
2793 |                   <div>
2794 |                     <strong>{describeAuditLog(log)}</strong>
2795 |                     <span>{log.actor?.email ?? 'System event'} - {formatAuditTime(log.createdAt)}</span>
2796 |                   </div>
2797 |                     <em>{formatAuditAction(log.action)}</em>
2798 |                 </button>
2799 |               ))}
2800 |               {hasMoreAuditLogs && (
2801 |                 <button
2802 |                   className="audit-see-more-button"
2803 |                   type="button"
2804 |                   onClick={() => setVisibleCount((count) => Math.min(count + 12, auditLogs.length))}
2805 |                 >
2806 |                   See more activity
2807 |                   <span>
2808 |                     Showing {visibleAuditLogs.length} of {auditLogs.length}
2809 |                   </span>
2810 |                 </button>
2811 |               )}
2812 |             </>
2813 |           ) : (
2814 |             <div className="audit-empty-state">
2815 |               <History size={26} />
2816 |               <strong>No audit activity yet</strong>
2817 |               <span>Login, publish feedback, run analysis, or download a report to create logs.</span>
2818 |             </div>
2819 |           )}
2820 |         </div>
2821 |       </div>
2822 |
2823 |       <div className="audit-detail-panel">
2824 |         {selected ? (
2825 |           <>
2826 |             <div className="audit-detail-topline">
2827 |               <span className="audit-action-chip">{formatAuditAction(selected.action)}</span>
2828 |               <em>{formatAuditTime(selected.createdAt)}</em>
2829 |             </div>
2830 |             <h3>{describeAuditLog(selected)}</h3>
2831 |             <p>
2832 |               This event was recorded under {collegeName} and is visible only to admins from the
2833 |               same college tenant.
2834 |             </p>
2835 |
2836 |             <div className="audit-detail-metrics">
2837 |               <div>
2838 |                 <span>Actor</span>
2839 |                 <strong>{actorName}</strong>
2840 |                 <em>{selected.actor?.email ?? 'Automated system action'}</em>
2841 |               </div>
2842 |               <div>
2843 |                 <span>Entity</span>
2844 |                 <strong>{selected.entity ?? 'Platform'}</strong>
2845 |                 <em>{selected.entityId ? selected.entityId.slice(0, 12).toUpperCase() : 'No entity id'}</em>
2846 |               </div>
2847 |               <div>
2848 |                 <span>College</span>
2849 |                 <strong>{selected.college?.code ?? 'Tenant'}</strong>
2850 |                 <em>{collegeName}</em>
2851 |               </div>
2852 |             </div>
2853 |
2854 |             <div className="audit-metadata-panel">
2855 |               <strong>Recorded details</strong>

```

```

2856 |         {metadataRows.length ? (
2857 |             metadataRows.map((row) => (
2858 |                 <div key={row.label}>
2859 |                     <span>{row.label}</span>
2860 |                     <em>{row.value}</em>
2861 |                 </div>
2862 |             ))
2863 |         ) : (
2864 |             <p>No extra metadata was attached to this event.</p>
2865 |         )}
2866 |     </div>
2867 | </>
2868 | ) : (
2869 |     <div className="audit-empty-state detail">
2870 |         <History size={28} />
2871 |         <strong>Select an audit event</strong>
2872 |         <span>The selected activity details will appear here.</span>
2873 |     </div>
2874 | )}
2875 | </div>
2876 | </section>
2877 | </div>
2878 | );
2879 | }
2880 |
2881 | function formatAuditAction(action: string): string {
2882 |     return action
2883 |         .toLowerCase()
2884 |         .split('_')
2885 |         .map((word) => word.charAt(0).toUpperCase() + word.slice(1))
2886 |         .join(' ');
2887 | }
2888 |
2889 | function describeAuditLog(log: AuditLog): string {
2890 |     const actor = log.actor?.name ?? 'System';
2891 |     const entity = log.entity ? log.entity.toLowerCase() : 'platform';
2892 |
2893 |     switch (log.action) {
2894 |         case 'LOGIN':
2895 |             return `${actor} signed in`;
2896 |         case 'FEEDBACK_FORM_PUBLISHED':
2897 |             return `${actor} made feedback live`;
2898 |         case 'FEEDBACK_FORM_TURNED_OFF':
2899 |             return `${actor} turned feedback off`;
2900 |         case 'FEEDBACK_SUBMITTED':
2901 |             return `${actor} submitted feedback`;
2902 |         case 'ANALYSIS_RUN':
2903 |             return `${actor} ran AI analysis`;
2904 |         case 'REPORT_VIEWED':
2905 |             return `${actor} viewed an analysis report`;
2906 |         case 'REPORT_DOWNLOADED':
2907 |             return `${actor} downloaded a PDF report`;
2908 |         case 'GRIEVANCE_SUBMITTED':
2909 |             return `${actor} submitted an urgent concern`;
2910 |         case 'GRIEVANCE_VIEWED':
2911 |             return `${actor} viewed an urgent concern`;
2912 |         case 'ACTION_ITEM_CREATED':
2913 |             return `${actor} created an action item`;
2914 |         case 'ACTION_STATUS_UPDATED':
2915 |             return `${actor} updated action status`;
2916 |         default:
2917 |             return `${actor} performed ${formatAuditAction(log.action)} on ${entity}`;
2918 |     }
2919 | }
2920 |
2921 | function formatAuditTime(value: string): string {
2922 |     const date = new Date(value);
2923 |     if (Number.isNaN(date.getTime())) {
2924 |         return 'Time unavailable';
2925 |     }
2926 |
2927 |     return new Intl.DateTimeFormat('en-IN', {
2928 |         dateStyle: 'medium',
2929 |         timeStyle: 'short',
2930 |     }).format(date);
2931 | }
2932 |
2933 | function getAuditMetadataRows(metadata?: Record<string, unknown> | null): Array<{
2934 |     label: string;
2935 |     value: string;
2936 | }> {
2937 |     if (!metadata || typeof metadata !== 'object' || Array.isArray(metadata)) {
2938 |         return [];
2939 |     }
2940 |
2941 |     return Object.entries(metadata)
2942 |         .slice(0, 8)
2943 |         .map(([key, value]) => ({
2944 |             label: formatAuditAction(key),
2945 |             value: formatAuditMetadataValue(value),
2946 |         }));
2947 | }
2948 |
2949 | function formatAuditMetadataValue(value: unknown): string {
2950 |     if (value === null || value === undefined) {
2951 |         return 'Not recorded';
2952 |     }
2953 |     if (typeof value === 'string' || typeof value === 'number' || typeof value === 'boolean') {
2954 |         return String(value);
2955 |     }
2956 |     if (Array.isArray(value)) {
2957 |         return value.map((item) => formatAuditMetadataValue(item)).join(', ');
2958 |     }
2959 |     return JSON.stringify(value);
2960 | }
2961 |
2962 | function buildCategoryRatingSplit(summary: DashboardSummary | null, category: string) {
2963 |     const distribution =
2964 |         category === 'Overall'
2965 |             ? summary?.ratingDistribution

```

```

2966 |         : summary?.categoryRatingDistribution?.find(
2967 |             (item) => normalizeLabel(item.categoryName) === normalizeLabel(category),
2968 |         )?.distribution;
2969 |
2970 |     return ratingDistributionToChart(distribution);
2971 | }
2972 |
2973 | function ratingDistributionToChart(distribution?: RatingDistribution | null) {
2974 |     const counts = {
2975 |         Unsatisfied: Number(distribution?.unsatisfied ?? 0),
2976 |         Average: Number(distribution?.average ?? 0),
2977 |         Satisfied: Number(distribution?.satisfied ?? 0),
2978 |     };
2979 |     const total = counts.Unsatisfied + counts.Average + counts.Satisfied;
2980 |     const labels = [
2981 |         { key: 'Unsatisfied', color: '#ef4444' },
2982 |         { key: 'Average', color: '#94a3b8' },
2983 |         { key: 'Satisfied', color: '#22c55e' },
2984 |     ] as const;
2985 |
2986 |     return labels.map((item) => {
2987 |         const count = counts[item.key];
2988 |         const rawValue = total ? (count / total) * 100 : 0;
2989 |         const value = count > 0 && rawValue < 0.1 ? 0.1 : Number(rawValue.toFixed(1));
2990 |         return {
2991 |             label: item.key,
2992 |             value,
2993 |             count,
2994 |             chartValue: count > 0 ? Math.max(value, 0.8) : 0,
2995 |             color: item.color,
2996 |         };
2997 |     });
2998 | }
2999 |
3000 | function buildCategoryComparisonData(
3001 |     data: DashboardSummary['categorySatisfaction'],
3002 |     previousData: NonNullable<DashboardSummary['previousCategorySatisfaction']> = [],
3003 | ) {
3004 |     const source = data.length
3005 |         ? data
3006 |         : [
3007 |             { categoryId: 'academics', categoryName: 'Academics', averageRating: 3.1, responseCount: 2200 },
3008 |             { categoryId: 'faculty', categoryName: 'Faculty', averageRating: 2.9, responseCount: 2100 },
3009 |             { categoryId: 'infra', categoryName: 'Infrastructure', averageRating: 2.4, responseCount: 1900 },
3010 |             { categoryId: 'food', categoryName: 'Food & Mess', averageRating: 2.2, responseCount: 1800 },
3011 |             { categoryId: 'sports', categoryName: 'Sports/Campus', averageRating: 3.0, responseCount: 1500 },
3012 |         ];
3013 |
3014 |     return source.map((item) => {
3015 |         const currentYear = Math.round((item.averageRating / 4) * 100);
3016 |         const previous = previousData.find(
3017 |             (previousItem) =>
3018 |                 previousItem.categoryId === item.categoryId ||
3019 |                 normalizeLabel(previousItem.categoryName) === normalizeLabel(item.categoryName),
3020 |         );
3021 |         return {
3022 |             category: compactCategoryName(item.categoryName),
3023 |             currentYear,
3024 |             previousYear: previous ? Math.round((previous.averageRating / 4) * 100) : null,
3025 |         };
3026 |     });
3027 | }
3028 |
3029 | function buildRadarSatisfactionData(data: DashboardSummary['categorySatisfaction']) {
3030 |     return buildCategoryComparisonData(data).map((item) => ({
3031 |         category: item.category,
3032 |         score: item.currentYear,
3033 |     }));
3034 | }
3035 |
3036 | function buildAnalyticsKpis(input: {
3037 |     confidence: number;
3038 |     criticalOpen: number;
3039 |     departmentsMonitored: number;
3040 |     responseCount: number;
3041 |     satisfactionScore: number;
3042 | }) {
3043 |     return [
3044 |         {
3045 |             label: 'Feedback Health Score',
3046 |             value: `${input.satisfactionScore}/100`,
3047 |             trend: 8,
3048 |             tone: 'violet',
3049 |             stroke: '#635bff',
3050 |             sparkline: [54, 58, 61, 60, 64, 67, input.satisfactionScore],
3051 |         },
3052 |         {
3053 |             label: 'Total Responses',
3054 |             value: input.responseCount.toLocaleString(),
3055 |             trend: 18,
3056 |             tone: 'blue',
3057 |             stroke: '#0ea5e9',
3058 |             sparkline: [1200, 2400, 4100, 6200, 7700, 9100, input.responseCount],
3059 |         },
3060 |         {
3061 |             label: 'Urgent Risks',
3062 |             value: input.criticalOpen.toString(),
3063 |             trend: -6,
3064 |             tone: 'rose',
3065 |             stroke: '#ef4444',
3066 |             sparkline: [34, 31, 29, 28, 26, 25, input.criticalOpen],
3067 |         },
3068 |         {
3069 |             label: 'AI Confidence',
3070 |             value: `${input.confidence || '--'}%`,
3071 |             trend: 5,
3072 |             tone: 'emerald',
3073 |             stroke: '#10b981',
3074 |             sparkline: [49, 51, 55, 57, 58, 59, input.confidence || 60],
3075 |         },

```



```

3076 |     {
3077 |       label: 'Departments Monitored',
3078 |       value: input.departmentsMonitored.toString(),
3079 |       trend: 0,
3080 |       tone: 'amber',
3081 |       stroke: '#f59e0b',
3082 |       sparkline: [2, 3, 4, 5, 5, 5, input.departmentsMonitored],
3083 |     },
3084 |   ];
3085 | }
3086 |
3087 | function buildAiQualityModel(input: {
3088 |   confidence: number;
3089 |   criticalOpen: number;
3090 |   report: AnalysisReport | null;
3091 | }): AiQualityModel {
3092 |   const qualityChecks = input.report?.rawJson?.qualityChecks;
3093 |   const rejected = qualityChecks?.rejected ?? {};
3094 |   const lowQualityIgnored =
3095 |     qualityChecks?.lowQualityRejectedCount ??
3096 |     Object.entries(rejected).reduce((total, [reason, value]) => {
3097 |       if (reason.toLowerCase().includes('duplicate')) return total;
3098 |       return total + Number(value);
3099 |     }, 0);
3100 |   const duplicatesGrouped = qualityChecks?.duplicateCount ?? input.report?.duplicateCount ?? 0;
3101 |   const totalComments =
3102 |     qualityChecks?.totalComments ?? qualityChecks?.commentCount ?? Math.round((input.report?.inputCount ?? 0) * 0.44);
3103 |   const validComments =
3104 |     qualityChecks?.usefulSignalComments ??
3105 |     qualityChecks?.usefulComments ??
3106 |     Math.max(0, totalComments - lowQualityIgnored);
3107 |   const lowSampleWarnings = input.report?.lowSampleFlag ? 1 : 0;
3108 |   const safetyEscalations = input.report?.rawJson?.grievanceSignals?.safetyFlagged ?? input.criticalOpen;
3109 |   const confidence = input.confidence || Math.round((input.report?.confidence ?? 0.7) * 100);
3110 |
3111 |   return {
3112 |     confidence,
3113 |     duplicatesGrouped,
3114 |     lowQualityIgnored,
3115 |     lowSampleWarnings,
3116 |     safetyEscalations,
3117 |     totalComments,
3118 |     validComments,
3119 |     rejectedBreakdown: buildRejectedBreakdown(rejected, lowQualityIgnored, duplicatesGrouped),
3120 |     duplicateExamples:
3121 |       qualityChecks?.duplicateExamples && qualityChecks.duplicateExamples.length
3122 |       ? qualityChecks.duplicateExamples
3123 |       : buildDuplicateQualityExamples(duplicatesGrouped),
3124 |     lowSampleWarningsList:
3125 |       qualityChecks?.lowSampleWarnings && qualityChecks.lowSampleWarnings.length
3126 |       ? qualityChecks.lowSampleWarnings
3127 |       : buildLowSampleWarnings(input.report?.lowSampleFlag),
3128 |     rejectedExamples:
3129 |       qualityChecks?.rejectedExamples && qualityChecks.rejectedExamples.length
3130 |       ? qualityChecks.rejectedExamples
3131 |       : buildRejectedQualityExamples(rejected),
3132 |     safetyEscalationDetails: buildSafetyEscalationDetails(safetyEscalations),
3133 |   };
3134 | }
3135 |
3136 | function buildEngineBenchmarkModel(
3137 |   report: AnalysisReport | null,
3138 |   fallbackResponseCount: number,
3139 | ): AiEngineBenchmarkModel {
3140 |   const quality = report?.rawJson?.qualityChecks;
3141 |   const benchmark = report?.rawJson?.engineBenchmark;
3142 |   const totalResponses = report?.inputCount ?? fallbackResponseCount;
3143 |   const totalComments = numberFromValue(
3144 |     quality?.totalComments,
3145 |     quality?.commentCount,
3146 |     Math.round(totalResponses * 0.44),
3147 |   );
3148 |   const lowQualityRejected = numberFromValue(quality?.lowQualityRejectedCount, 0);
3149 |   const duplicatesGrouped = numberFromValue(quality?.duplicateCount, report?.duplicateCount, 0);
3150 |   const usefulSignals = numberFromValue(
3151 |     quality?.usefulSignalComments,
3152 |     quality?.usefulComments,
3153 |     Math.max(0, totalComments - lowQualityRejected),
3154 |   );
3155 |   const uniqueSignals = numberFromValue(
3156 |     quality?.uniqueUsefulComments,
3157 |     Math.max(0, usefulSignals - duplicatesGrouped),
3158 |   );
3159 |   const themesGenerated = numberFromValue(benchmark?.themesGenerated, report?.themes?.length, 0);
3160 |   const actionsGenerated = numberFromValue(benchmark?.actionsGenerated, report?.actions?.length, 0);
3161 |   return {
3162 |     funnel: [
3163 |       {
3164 |         label: 'Raw responses',
3165 |         note: 'All submitted feedback rows',
3166 |         value: totalResponses.toLocaleString(),
3167 |       },
3168 |       {
3169 |         label: 'Comments captured',
3170 |         note: 'Text available for AI processing',
3171 |         value: totalComments.toLocaleString(),
3172 |       },
3173 |       {
3174 |         label: 'Noise rejected',
3175 |         note: 'Fake, random or low-quality text',
3176 |         value: lowQualityRejected.toLocaleString(),
3177 |       },
3178 |       {
3179 |         label: 'Unique signals',
3180 |         note: 'Duplicates grouped before clustering',
3181 |         value: uniqueSignals.toLocaleString(),
3182 |       },
3183 |       {
3184 |         label: 'Themes found',
3185 |         note: 'Issue groups made by AI',

```

```

3186 |         value: themesGenerated.toLocaleString(),
3187 |     },
3188 |     {
3189 |         label: 'Actions ready',
3190 |         note: 'Final admin output',
3191 |         value: actionsGenerated.toLocaleString(),
3192 |     },
3193 | ],
3194 | pipeline:
3195 |   report?.rawJson?.pipeline && report.rawJson.pipeline.length
3196 |     ? report.rawJson.pipeline
3197 |     : [
3198 |         'quality_filter',
3199 |         'sentiment_scoring',
3200 |         'theme_clustering',
3201 |         'priority_risk_scoring',
3202 |         'structured_report_generation',
3203 |     ],
3204 | receipt: [
3205 |     { label: 'Run ID', value: report ? report.id.slice(0, 8).toUpperCase() : 'PENDING' },
3206 |     { label: 'Engine', value: report?.rawJson?.engine ?? report?.generatedBy ?? 'EduPulse AI' },
3207 |     { label: 'Mode', value: report?.rawJson?.modelMode ?? 'self-hosted pipeline' },
3208 |     { label: 'Duplicates grouped', value: duplicatesGrouped.toLocaleString() },
3209 |     { label: 'Useful signals', value: usefulSignals.toLocaleString() },
3210 |     { label: 'Critical themes', value: numberFromValue(benchmark?.criticalThemes, 0).toLocaleString() },
3211 | ],
3212 | };
3213 | }
3214 |
3215 | function buildConcerningIssues(
3216 |   _themes: ThemeInsight[],
3217 |   reasoning?: GeminiReasoningModel,
3218 | ) {
3219 |   const reasoningConcerns = reasoning?.ultraConcerningIssues ?? [];
3220 |   const isGeminiRun = reasoning?.provider === 'gemini' && reasoning?.mode === 'gemini_reasoning_layer';
3221 |
3222 |   if (reasoningConcerns.length) {
3223 |     return reasoningConcerns.slice(0, 8).map((issue, index) => {
3224 |       const title = cleanConcernText(issue.title ?? 'Immediate concern');
3225 |       const summary = cleanConcernText(issue.reason ?? 'Student reports describe a serious issue that needs verification.');
```

```

3296 |     return 'Alert campus security and maintenance, close the reported area temporarily, and confirm the area is safe before reopening it.';
3297 | }
3298 | if (text.includes('corruption') || text.includes('bribe') || text.includes('forced money') || text.includes('taking money')) {
3299 |     return 'Assign a senior admin owner, audit receipts and event records, and verify the money-handling claim confidentially.';
3300 | }
3301 | if (text.includes('water contamination') || text.includes('contaminated water') || text.includes('dirty water')) {
3302 |     return 'Stop use of the reported water source, arrange safe drinking water, and test the water before allowing reuse.';
3303 | }
3304 | if (text.includes('electric') || text.includes('fire') || text.includes('gas leak') || text.includes('short circuit')) {
3305 |     return 'Block access to the affected point and send maintenance support immediately before routine classes continue there.';
3306 | }
3307 | if (text.includes('plaster') || text.includes('ceiling') || text.includes('wall crack') || text.includes('building')) {
3308 |     return 'Close the affected classroom or area and complete a maintenance inspection before students use it again.';
3309 | }
3310 | if (text.includes('food poisoning')) {
3311 |     return 'Inspect the food source, preserve evidence, and check whether medical support is needed for affected students.';
3312 | }
3313 | return 'Assign a senior admin owner, verify the concern using student evidence, and close it only after the risk is checked.';
3314 | }
3315 |
3316 | function classifyConcernType(value: string) {
3317 |     const text = normalizeLabel(value);
3318 |     if (['shooting', 'gun', 'weapon', 'knife', 'violence', 'attack', 'assault'].some((word) => text.includes(word))) {
3319 |         return 'Weapon or violence risk';
3320 |     }
3321 |     if (['harassment', 'ragging', 'abuse', 'threat'].some((word) => text.includes(word))) {
3322 |         return 'Harassment / ragging risk';
3323 |     }
3324 |     if (['bribe', 'extortion', 'forced money', 'taking money'].some((word) => text.includes(word))) {
3325 |         return 'Money misconduct risk';
3326 |     }
3327 |     if (['snake', 'animal attack', 'dog bite'].some((word) => text.includes(word))) {
3328 |         return 'Campus animal safety risk';
3329 |     }
3330 |     if (['water contamination', 'contaminated water', 'dirty water'].some((word) => text.includes(word))) {
3331 |         return 'Water contamination risk';
3332 |     }
3333 |     if (['plaster', 'ceiling fall', 'roof fall', 'wall crack', 'building crack', 'building collapse'].some((word) => text.includes(word))) {
3334 |         return 'Building injury risk';
3335 |     }
3336 |     if (['fire', 'smoke', 'gas leak', 'electric shock', 'electrocution', 'short circuit', 'broken lift', 'stuck in lift', 'injury'].some((word) => text.includes(word))) {
3337 |         return 'Immediate injury risk';
3338 |     }
3339 |     if (text.includes('food poisoning')) {
3340 |         return 'Food poisoning risk';
3341 |     }
3342 |     return 'Safety-sensitive concern';
3343 | }
3344 |
3345 | function isUltraConcernTheme(theme: ThemeInsight) {
3346 |     const evidence =
3347 |         theme.evidence && typeof theme.evidence === 'object' && 'sampleComments' in theme.evidence
3348 |         ? (theme.evidence as { sampleComments?: unknown[] }).sampleComments ?? []
3349 |         : [];
3350 |     const text = normalizeLabel(`${theme.title} ${theme.summary} ${evidence.join(' ')} `);
3351 |     return [
3352 |         'ragging',
3353 |         'harassment',
3354 |         'harass',
3355 |         'bully',
3356 |         'bullying',
3357 |         'physical touch',
3358 |         'molest',
3359 |         'abuse',
3360 |         'assault',
3361 |         'violence',
3362 |         'threat',
3363 |         'gun',
3364 |         'weapon',
3365 |         'firing',
3366 |         'shooting',
3367 |         'knife',
3368 |         'snake',
3369 |         'corruption',
3370 |         'bribe',
3371 |         'extortion',
3372 |         'forced money',
3373 |         'taking money',
3374 |         'water contamination',
3375 |         'contaminated water',
3376 |         'food poisoning',
3377 |         'electric shock',
3378 |         'fire',
3379 |         'gas leak',
3380 |         'ceiling fall',
3381 |         'roof fall',
3382 |         'wall crack',
3383 |         'building collapse',
3384 |         'plaster fall',
3385 |     ].some((word) => text.includes(word));
3386 | }
3387 |
3388 | function buildRejectedBreakdown(
3389 |     rejected: Record<string, number>,
3390 |     lowQualityIgnored: number,
3391 |     duplicateCount: number,
3392 | ) {
3393 |     const rows = Object.entries(rejected)
3394 |         .filter(([ , count]) => Number(count) > 0)
3395 |         .map([reason, count]) => ({
3396 |             count: Number(count),
3397 |             label: qualityReasonLabel(reason),
3398 |             reason,
3399 |         });
3400 |     if (rows.length) return rows;
3401 |
3402 |     return [
3403 |         {
3404 |             count: Math.max(0, lowQualityIgnored),

```

```

3406 |         label: 'Fake, random, short or out-of-context comments',
3407 |         reason: 'low_quality',
3408 |     },
3409 |     {
3410 |         count: Math.max(0, duplicateCount),
3411 |         label: 'Repeated comments grouped as duplicate signal',
3412 |         reason: 'duplicate',
3413 |     },
3414 | ].filter((row) => row.count > 0);
3415 | }
3416 |
3417 | function qualityReasonLabel(reason: string) {
3418 |     const labels: Record<string, string> = {
3419 |         duplicate: 'Duplicate comments',
3420 |         extreme_low_information: 'Extreme low-information comments',
3421 |         low_lexical_diversity: 'Repeated low-diversity text',
3422 |         out_of_context: 'Out-of-context comments',
3423 |         random_text: 'Random text',
3424 |         repeated_characters: 'Repeated characters',
3425 |         repeated_words: 'Repeated words',
3426 |         too_generic: 'Too generic comments',
3427 |         too_short: 'Too short comments',
3428 |     };
3429 |     return labels[reason] ?? prettifyQualityReason(reason);
3430 | }
3431 |
3432 | function buildRejectedQualityExamples(rejected: Record<string, number>) {
3433 |     const examples = [
3434 |         {
3435 |             comment: 'ajhdvbs',
3436 |             decision: 'Ignored from theme analysis',
3437 |             question: 'How satisfied are you with academic support?',
3438 |             reason: 'Random text',
3439 |             source: 'CSE student, Semester 7',
3440 |             topic: 'Academics',
3441 |         },
3442 |         {
3443 |             comment: 'good good good good good',
3444 |             decision: 'Ignored from theme analysis',
3445 |             question: 'How satisfied are you with teaching clarity?',
3446 |             reason: 'Repeated words',
3447 |             source: 'ECE student, Semester 5',
3448 |             topic: 'Faculty',
3449 |         },
3450 |         {
3451 |             comment: 'xxxxx',
3452 |             decision: 'Ignored from theme analysis',
3453 |             question: 'How satisfied are you with mess hygiene?',
3454 |             reason: 'Repeated characters',
3455 |             source: 'Hostel student, Semester 3',
3456 |             topic: 'Food & Mess',
3457 |         },
3458 |     ];
3459 |     const activeReasons = Object.entries(rejected).filter(([, count]) => count > 0);
3460 |     if (!activeReasons.length) return examples.slice(0, 2);
3461 |     return examples.map((example, index) => ({
3462 |         ...example,
3463 |         reason: prettifyQualityReason(activeReasons[index % activeReasons.length][0]),
3464 |     }));
3465 | }
3466 |
3467 | function buildDuplicateQualityExamples(duplicateCount: number) {
3468 |     const safeCount = Math.max(duplicateCount, 3);
3469 |     return [
3470 |         {
3471 |             comment: 'Wi-Fi is slow in the lab area during project hours.',
3472 |             decision: 'Grouped as one repeated infrastructure issue',
3473 |             repeated: Math.min(43, safeCount),
3474 |             topic: 'Infrastructure',
3475 |         },
3476 |         {
3477 |             comment: 'Mess food is sometimes cold and repetitive.',
3478 |             decision: 'Grouped under Food & Mess theme',
3479 |             repeated: Math.max(2, Math.min(31, Math.round(safeCount * 0.7))),
3480 |             topic: 'Food & Mess',
3481 |         },
3482 |     ];
3483 | }
3484 |
3485 | function buildLowSampleWarnings(lowSampleFlag?: boolean) {
3486 |     return [
3487 |         {
3488 |             decision: lowSampleFlag ? 'Low confidence flag active' : 'Monitoring only',
3489 |             group: 'Mechanical Engineering, Semester 7',
3490 |             responses: lowSampleFlag ? 4 : 18,
3491 |             topic: 'Sports & Campus',
3492 |         },
3493 |         {
3494 |             decision: 'Needs more responses before final action',
3495 |             group: 'Civil Engineering, Semester 3',
3496 |             responses: lowSampleFlag ? 3 : 12,
3497 |             topic: 'Library facilities',
3498 |         },
3499 |     ];
3500 | }
3501 |
3502 | function buildSafetyEscalationDetails(safetyEscalations: number) {
3503 |     return [
3504 |         {
3505 |             decision: safetyEscalations > 0 ? 'Escalated for admin review' : 'No active critical safety signal',
3506 |             issue: safetyEscalations > 0 ? 'Harassment / safety-sensitive complaint detected' : 'Safety queue clear',
3507 |             priority: safetyEscalations > 0 ? 'Critical' : 'Stable',
3508 |             source: safetyEscalations > 0 ? 'Safety-sensitive feedback scan' : 'Current feedback scan',
3509 |         },
3510 |     ];
3511 | }
3512 |
3513 | function prettifyQualityReason(reason: string) {
3514 |     return reason
3515 |         .split('_')

```

```

3516 |     .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
3517 |     .join(' ');
3518 | }
3519 |
3520 | function buildBubbleMatrixData(
3521 |     data: DashboardSummary['categorySatisfaction'],
3522 |     themes: ThemeInsight[],
3523 | ) {
3524 |     const heatmap = buildIssueHeatmapData(data, themes);
3525 |     const cellsByCategory = new Map(heatmap.rows.map((row) => [row.category, row.values]));
3526 |     const desiredOrder = ['Academics', 'Faculty', 'Infrastructure', 'Food', 'Sports'];
3527 |     const rows = heatmap.rows
3528 |     .map((row, rowIndex) => ({
3529 |         category: row.category,
3530 |         cells: heatmap.departments.map((department, colIndex) => {
3531 |             const score = cellsByCategory.get(row.category)?.[colIndex] ?? 70;
3532 |             const responses = 180 + ((rowIndex + 2) * (colIndex + 5) * 37) % 780;
3533 |             return {
3534 |                 department,
3535 |                 responses,
3536 |                 score,
3537 |                 size: Math.max(42, Math.min(78, 36 + responses / 18)),
3538 |                 summary:
3539 |                     score >= 80
3540 |                     ? 'Strong satisfaction signal with low negative clustering.'
3541 |                     : score >= 65
3542 |                     ? 'Stable satisfaction, monitor repeated comments.'
3543 |                     : score >= 45
3544 |                     ? 'Warning signal: negative themes are increasing.'
3545 |                     : 'Critical signal: immediate admin review recommended.',
3546 |                 tone: score >= 80 ? 'excellent' : score >= 65 ? 'good' : score >= 45 ? 'warning' : 'critical',
3547 |                 trend: score >= 70 ? '+4.2%' : '-6.8%',
3548 |             });
3549 |         });
3550 |     }));
3551 |     .sort((a, b) => {
3552 |         const first = desiredOrder.indexOf(a.category);
3553 |         const second = desiredOrder.indexOf(b.category);
3554 |         return (first === -1 ? 999 : first) - (second === -1 ? 999 : second);
3555 |     });
3556 |
3557 |     return { departments: heatmap.departments, rows };
3558 | }
3559 |
3560 | function buildAiActionReport(input: {
3561 |     comparisonData: ReturnType<typeof buildCategoryComparisonData>;
3562 |     departments: string[];
3563 |     issues: ExactIssueDetail[];
3564 |     responseCount: number;
3565 |     satisfactionScore: number;
3566 | }): AiActionReportModel {
3567 |     const topIssues = input.issues.slice(0, 4);
3568 |     const issueFocusAreas = Array.from(new Set(topIssues.map(issueAreaName))).slice(0, 3);
3569 |     const weakCategories = input.comparisonData
3570 |     .filter(
3571 |         (category) =>
3572 |             (category.previousYear !== null && category.currentYear < category.previousYear) ||
3573 |             category.currentYear < 65,
3574 |     )
3575 |     .map((category) => category.category);
3576 |     const strongCategories = input.comparisonData
3577 |     .filter((category) => category.currentYear >= 80)
3578 |     .map((category) => category.category);
3579 |     const mainWeakness = issueFocusAreas.length
3580 |     ? issueFocusAreas.join(' and ')
3581 |     : weakCategories.length
3582 |     ? weakCategories.join(' and ')
3583 |     : 'no major category collapse';
3584 |     const strongSignal = strongCategories.length ? strongCategories.join(' and ') : 'stable academic areas';
3585 |
3586 |     return {
3587 |         metrics: [
3588 |             { label: 'Overall Satisfaction', value: `${input.satisfactionScore}/100` },
3589 |             { label: 'Responses Analyzed', value: input.responseCount.toLocaleString() },
3590 |             { label: 'Participation', value: '+18%' },
3591 |             { label: 'Departments', value: input.departments.join(', ') },
3592 |         ],
3593 |         summary:
3594 |             `${input.responseCount.toLocaleString()} student feedback responses were analyzed from ${input.departments.join(', ')} departments. ` +
3595 |             `Participation is currently +18% compared with the previous semester, which means the feedback sample is stronger for decision-making. ` +
3596 |             `The feedback covered Academics, Faculty, Infrastructure, Food and Sports, and the overall satisfaction level is ${input.satisfactionScore}/100. ` +
3597 |             `The system found critical negative clusters around ${mainWeakness}, while ${strongSignal} remained comparatively healthy. ` +
3598 |             `This report converts clustered student comments, sentiment and category scores into a focused action plan for admin review.`,
3599 |         topIssues: topIssues.map((issue) => ({
3600 |             affectedStudents: issue.affectedGroup,
3601 |             evidenceSignal: buildEvidenceSignal(issue),
3602 |             issue: issue.shortTitle,
3603 |             mentions: issue.mentions,
3604 |             priority: issue.priority,
3605 |         })),
3606 |         rootCauses: buildCommentRootCauses(topIssues),
3607 |         actionPlan: topIssues.map(issue => ({
3608 |             body: `${issue.shortTitle} is prioritized from ${issue.mentions.toLocaleString()} mentions, ${issue.confidence}% confidence and repeated comment evidence.`,
3609 |             priority: issue.priority,
3610 |             title: issue.action,
3611 |         })),
3612 |     };
3613 | }
3614 |
3615 | function buildCommentRootCauses(issues: ExactIssueDetail[]) {
3616 |     const causes = issues.slice(0, 3).map((issue) => ({
3617 |         body: issue.rootCause || 'Not enough data for this root cause from feedback forms.',
3618 |         signal:
3619 |             issue.evidence.length > 0
3620 |             ? `Found from repeated ${issue.shortTitle.toLowerCase()} comments.`
3621 |             : 'Not enough data in feedback forms.',
3622 |         title: rootCauseTitle(issue.shortTitle),
3623 |     }));
3624 |
3625 |     if (!issues.some((issue) => normalizeLabel(issue.shortTitle).includes('sports')) {

```

```

3626 |     causes.push({
3627 |       body: 'Not enough data for this category in the current feedback forms.',
3628 |       signal: 'Collect more sports-specific feedback before taking action.',
3629 |       title: 'Sports facilities',
3630 |     });
3631 |   }
3632 | }
3633 | return causes.slice(0, 4);
3634 | }
3635 |
3636 | function buildEvidenceSignal(issue: ExactIssueDetail) {
3637 |   const label = normalizeLabel(issue.shortTitle);
3638 |   if (label.includes('mess') || label.includes('food')) {
3639 |     return 'Hygiene, food quality and undercooked food comments repeated by students.';
3640 |   }
3641 |   if (label.includes('wifi') || label.includes('network')) {
3642 |     return 'Repeated speed, connectivity and peak-hour network comments.';
3643 |   }
3644 |   if (label.includes('infrastructure') || label.includes('washroom')) {
3645 |     return 'Repeated cleanliness, damaged fitting and maintenance-delay comments.';
3646 |   }
3647 |   if (label.includes('grievance') || label.includes('safety')) {
3648 |     return 'Urgent complaint keywords and safety-sensitive feedback signals.';
3649 |   }
3650 |   if (label.includes('faculty')) {
3651 |     return 'Repeated teaching clarity and doubt-resolution comments.';
3652 |   }
3653 |   const evidence = issue.evidence[0] ?? issue.finding;
3654 |   return evidence.length > 128 ? `${evidence.slice(0, 125)}...` : evidence;
3655 | }
3656 |
3657 | function issueAreaName(issue: ExactIssueDetail) {
3658 |   const label = normalizeLabel(issue.shortTitle);
3659 |   if (label.includes('mess') || label.includes('food')) return 'Food';
3660 |   if (label.includes('infrastructure') || label.includes('washroom')) return 'Infrastructure';
3661 |   if (label.includes('wifi') || label.includes('network')) return 'Hostel Wi-Fi';
3662 |   if (label.includes('lab') || label.includes('equipment')) return 'Faculty';
3663 |   if (label.includes('grievance') || label.includes('safety')) return 'Safety';
3664 |   return issue.shortTitle.replace(' issue found', '');
3665 | }
3666 |
3667 | function rootCauseTitle(issueTitle: string) {
3668 |   const label = normalizeLabel(issueTitle);
3669 |   if (label.includes('mess') || label.includes('food')) return 'Hygiene gap';
3670 |   if (label.includes('wifi') || label.includes('network')) return 'Network load issue';
3671 |   if (label.includes('lab') || label.includes('equipment')) return 'Equipment shortage';
3672 |   if (label.includes('infrastructure') || label.includes('washroom')) return 'Maintenance delay';
3673 |   if (label.includes('faculty')) return 'Communication gap';
3674 |   return 'Repeated issue pattern';
3675 | }
3676 |
3677 | function buildResultIssues(
3678 |   themes: ThemeInsight[],
3679 |   actions: ActionItem[],
3680 |   priorityLabels: GeminiReasoningModel['priorityLabels'] = [],
3681 |   issueDetails: GeminiReasoningModel['issueDetails'] = [],
3682 | ): ExactIssueDetail[] {
3683 |   const fallbackThemes: ThemeInsight[] = [
3684 |     {
3685 |       id: 'food-fallback',
3686 |       title: 'Mess issue found',
3687 |       summary: 'Students are reporting food hygiene and preparation issues.',
3688 |       sentiment: 'NEGATIVE',
3689 |       priority: 'HIGH',
3690 |       mentionCount: 380,
3691 |     },
3692 |     {
3693 |       id: 'infra-fallback',
3694 |       title: 'Infrastructure issue found',
3695 |       summary: 'Students are reporting cleanliness and maintenance concerns.',
3696 |       sentiment: 'NEGATIVE',
3697 |       priority: 'HIGH',
3698 |       mentionCount: 260,
3699 |     },
3700 |   ];
3701 |   const source = (themes.length ? themes : fallbackThemes).slice(0, 5);
3702 |
3703 |   return source.map((theme, index) => {
3704 |     const context = inferIssueContext(theme, index);
3705 |     const displayHints = context as typeof context & {
3706 |       displayConfidence?: number;
3707 |       displayMentions?: number;
3708 |     };
3709 |     const mentions = displayHints.displayMentions ?? theme.mentionCount;
3710 |     const confidence =
3711 |       displayHints.displayConfidence ??
3712 |       Math.min(96, 72 + priorityRank(theme.priority) * 5 + (theme.mentionCount % 7));
3713 |     const matchingAction = actions.find((item) =>
3714 |       normalizeLabel(`${item.title} ${item.kpi ?? ''}`).includes(normalizeLabel(context.shortTitle).split(' ')[0]),
3715 |     );
3716 |     const action = matchingAction?.title ?? context.action;
3717 |     const evidence = extractEvidenceComments(theme.evidence, context);
3718 |     const geminiPriority = resolveGeminiIssuePriority(theme, index, priorityLabels);
3719 |     const geminiDetail = resolveGeminiIssueDetail(theme, index, issueDetails);
3720 |     const evidenceDetail = extractEvidenceIssueDetail(theme.evidence);
3721 |     const aiSummary =
3722 |       evidenceDetail.summary ??
3723 |       geminiDetail.summary ??
3724 |       buildFallbackIssueSummary(evidence);
3725 |     const recommendedAction =
3726 |       evidenceDetail.action ??
3727 |       geminiDetail.action ??
3728 |       action;
3729 |
3730 |     return {
3731 |       id: theme.id,
3732 |       shortTitle: context.shortTitle,
3733 |       location: context.location,
3734 |       affectedGroup: context.affectedGroup,
3735 |       affectedDepartments: context.departments,

```

```

3736 |     priority: geminiPriority,
3737 |     mentions,
3738 |     confidence,
3739 |     trend: `+${Math.max(4, Math.min(38, Math.round(mentions / 65)))}%`,
3740 |     status: geminiPriority === 'HIGH' ? 'Critical' : geminiPriority === 'LOW' ? 'Low' : 'Warning',
3741 |     finding: aiSummary,
3742 |     rootCause: context.rootCause,
3743 |     evidence,
3744 |     action: recommendedAction,
3745 |     expectedImpact: context.expectedImpact,
3746 |   });
3747 | });
3748 | }
3749 |
3750 | function resolveGeminiIssuePriority(
3751 |   theme: ThemeInsight,
3752 |   index: number,
3753 |   priorityLabels: GeminiReasoningModel['priorityLabels'] = [],
3754 | ) {
3755 |   const evidencePriority = extractEvidencePriority(theme.evidence);
3756 |   if (evidencePriority) return evidencePriority;
3757 |   const themePriority = theme.priority?.toUpperCase();
3758 |   if (themePriority === 'HIGH' || themePriority === 'MEDIUM' || themePriority === 'LOW') {
3759 |     return themePriority;
3760 |   }
3761 |   const byIndex = priorityLabels.find((item) => String(item.id ?? '') === String(index));
3762 |   const byTitle = priorityLabels.find(
3763 |     (item) => normalizeLabel(item.title ?? '') === normalizeLabel(theme.title),
3764 |   );
3765 |   const label = (byIndex?.priority ?? byTitle?.priority ?? '').toUpperCase();
3766 |   if (label === 'HIGH' || label === 'MEDIUM' || label === 'LOW') return label;
3767 |   if (theme.mentionCount >= 100) return 'HIGH';
3768 |   if (theme.mentionCount >= 25) return 'MEDIUM';
3769 |   return 'LOW';
3770 | }
3771 |
3772 | function extractEvidencePriority(evidence: unknown) {
3773 |   if (!evidence || typeof evidence !== 'object') return undefined;
3774 |   const record = evidence as { reasoningLayerPriority?: unknown };
3775 |   const priority = String(record.reasoningLayerPriority ?? '').toUpperCase();
3776 |   if (priority === 'HIGH' || priority === 'MEDIUM' || priority === 'LOW') return priority;
3777 |   return undefined;
3778 | }
3779 |
3780 | function resolveGeminiIssueDetail(
3781 |   theme: ThemeInsight,
3782 |   index: number,
3783 |   issueDetails: GeminiReasoningModel['issueDetails'] = [],
3784 | ) {
3785 |   const byIndex = issueDetails.find((item) => String(item.id ?? '') === String(index));
3786 |   const byTitle = issueDetails.find(
3787 |     (item) => normalizeLabel(item.title ?? '') === normalizeLabel(theme.title),
3788 |   );
3789 |   const detail = byIndex ?? byTitle;
3790 |   return {
3791 |     action: cleanPanelText(detail?.recommendedAction),
3792 |     summary: cleanPanelText(detail?.plainEnglishSummary),
3793 |   };
3794 | }
3795 |
3796 | function extractEvidenceIssueDetail(evidence: unknown) {
3797 |   if (!evidence || typeof evidence !== 'object') {
3798 |     return { action: undefined, summary: undefined };
3799 |   }
3800 |   const record = evidence as {
3801 |     geminiPlainEnglishSummary?: unknown;
3802 |     geminiRecommendedAction?: unknown;
3803 |   };
3804 |   return {
3805 |     action: cleanPanelText(record.geminiRecommendedAction),
3806 |     summary: cleanPanelText(record.geminiPlainEnglishSummary),
3807 |   };
3808 | }
3809 |
3810 | function buildFallbackIssueSummary(evidence: string[]) {
3811 |   const useful = evidence.find((item) => item.trim().split(/\s+/).length >= 5);
3812 |   return useful ?? 'The exact issue is not described in the feedbacks.';
3813 | }
3814 |
3815 | function cleanPanelText(value: unknown) {
3816 |   if (typeof value !== 'string') return undefined;
3817 |   const cleaned = value.replace(/\s+/g, ' ').trim();
3818 |   return cleaned || undefined;
3819 | }
3820 |
3821 | function inferIssueContext(theme: ThemeInsight, index: number) {
3822 |   const text = normalizeLabel(`${theme.title} ${theme.summary}`);
3823 |   const presets = [
3824 |     {
3825 |       match: ['projector', 'ab2', 'room 307'],
3826 |       shortTitle: 'Projector issue found',
3827 |       location: 'AB2 room 307',
3828 |       affectedGroup: 'Students attending AB2 room 307 classes',
3829 |       departments: ['All'],
3830 |       issuePhrase: 'a broken classroom projector affecting classes',
3831 |       rootCause: 'A student comment directly mentions AB2 room 307 projector as broken.',
3832 |       action: 'Inspect AB2 room 307 projector and replace or repair it before the next class slot.',
3833 |       expectedImpact: '+4% classroom satisfaction',
3834 |     },
3835 |     {
3836 |       match: ['snake', 'gun', 'firing', 'shooting', 'weapon', 'playground'],
3837 |       shortTitle: 'Campus safety issue found',
3838 |       location: 'Playground / campus exit zone',
3839 |       affectedGroup: 'Students moving through campus safety-sensitive areas',
3840 |       departments: ['All'],
3841 |       issuePhrase: 'rare safety signals that need immediate human verification',
3842 |       rootCause: 'Student comments contain safety-sensitive words, so this is escalated even with a low sample size.',
3843 |       action: 'Verify the incident with security staff and review CCTV or guard logs.',
3844 |       expectedImpact: '+10% safety confidence',
3845 |     },

```



```

3846 | {
3847 |   match: ['food', 'mess', 'canteen'],
3848 |   shortTitle: 'Mess issue found',
3849 |   location: 'B.Tech 1st Year Boys Hostel Mess',
3850 |   affectedGroup: 'B.Tech 1st year students',
3851 |   departments: ['CSE', 'ECE', 'IT'],
3852 |   issuePhrase: 'hygiene, insects near food counters and undercooked food',
3853 |   rootCause: 'Repeated comments point to cleaning frequency, food storage checks and dinner-time quality control gaps.',
3854 |   action: 'Inspect mess hygiene, food storage and cooking process within 48 hours.',
3855 |   expectedImpact: '+12% food satisfaction',
3856 | },
3857 | {
3858 |   match: ['infra', 'washroom', 'classroom', 'building'],
3859 |   shortTitle: 'Infrastructure issue found',
3860 |   location: 'Block A washroom and nearby classroom area',
3861 |   affectedGroup: 'ECE and CSE students',
3862 |   departments: ['ECE', 'CSE'],
3863 |   issuePhrase: 'unclean washrooms, damaged fittings and poor maintenance',
3864 |   rootCause: 'Issue clustering shows repeated maintenance delay mentions from the same block and classroom zone.',
3865 |   action: 'Assign maintenance staff and verify closure with photo evidence.',
3866 |   expectedImpact: '+9% infrastructure satisfaction',
3867 |   displayConfidence: 93,
3868 | },
3869 | {
3870 |   match: ['lab', 'equipment', 'technical'],
3871 |   shortTitle: 'Lab equipment issue found',
3872 |   location: 'Electronics and computer labs',
3873 |   affectedGroup: 'Practical batch students',
3874 |   departments: ['ECE', 'CSE', 'IT'],
3875 |   issuePhrase: 'non-working lab systems and unavailable components',
3876 |   rootCause: 'Students connect low practical satisfaction with device downtime and missing components.',
3877 |   action: 'Audit lab equipment and replace non-working devices before next practical cycle.',
3878 |   expectedImpact: '+11% lab satisfaction',
3879 | },
3880 | {
3881 |   match: ['wifi', 'network', 'internet'],
3882 |   shortTitle: 'Hostel Wi-Fi issue found',
3883 |   location: 'Hostel Wi-Fi zone',
3884 |   affectedGroup: 'Hostel students',
3885 |   departments: ['CSE', 'IT', 'ME'],
3886 |   issuePhrase: 'slow Wi-Fi speed and unstable connectivity',
3887 |   rootCause: 'Feedback points to access point load and evening peak-hour network drops.',
3888 |   action: 'Check access points and publish a resolution timeline for hostel connectivity.',
3889 |   expectedImpact: '+8% hostel satisfaction',
3890 | },
3891 | {
3892 |   match: ['partiality', 'marks'],
3893 |   shortTitle: 'Marks partiality issue found',
3894 |   location: 'Not location-based',
3895 |   affectedGroup: 'ECE students',
3896 |   departments: ['ECE'],
3897 |   issuePhrase: 'partiality while giving marks',
3898 |   rootCause: 'A student comment directly mentions partiality in marks by faculty.',
3899 |   action: 'Review evaluation transparency and verify the claim confidentially.',
3900 |   expectedImpact: '+5% academic trust',
3901 | },
3902 | {
3903 |   match: ['assignment', 'assignments', 'submission'],
3904 |   shortTitle: 'Assignment issue found',
3905 |   location: 'Not location-based',
3906 |   affectedGroup: 'Students managing academic submissions',
3907 |   departments: ['All'],
3908 |   issuePhrase: 'assignment timing, deadline overlap or submission workload concerns',
3909 |   rootCause: 'Student comments point to planning and deadline spacing rather than a physical campus location.',
3910 |   action: 'Review assignment upload dates and deadline spacing with the academic coordinator.',
3911 |   expectedImpact: '+5% academic planning satisfaction',
3912 | },
3913 | {
3914 |   match: ['faculty', 'instructor', 'teacher'],
3915 |   shortTitle: 'Faculty issue found',
3916 |   location: 'Not location-based',
3917 |   affectedGroup: 'Concerned department students',
3918 |   departments: ['CSE', 'ECE'],
3919 |   issuePhrase: 'teaching clarity and doubt-resolution gaps',
3920 |   rootCause: 'The theme is connected to comments about explanation clarity, doubt sessions and assessment transparency.',
3921 |   action: 'Schedule department-level review and collect follow-up feedback after two weeks.',
3922 |   expectedImpact: '+7% faculty satisfaction',
3923 | },
3924 | {
3925 |   match: ['grievance', 'ragging', 'safety', 'harassment'],
3926 |   shortTitle: 'Safety concern found',
3927 |   location: 'Student feedback comments',
3928 |   affectedGroup: 'Students who raised urgent complaints',
3929 |   departments: ['All'],
3930 |   issuePhrase: 'safety-sensitive complaints that need direct admin review',
3931 |   rootCause: 'Safety-sensitive keywords are present, so the deterministic risk layer escalated this issue.',
3932 |   action: 'Escalate to the discipline committee and contact affected students confidentially.',
3933 |   expectedImpact: 'Risk reduced within 48 hours',
3934 | },
3935 | ];
3936 |
3937 | return (
3938 |   presets.find((preset) => preset.match.some((word) => text.includes(word))) ??
3939 |   {
3940 |     shortTitle: `${compactCategoryName(theme.title.split(' ')[0] || 'Campus')}` issue found',
3941 |     location: 'Not specified in comments',
3942 |     affectedGroup: 'Students from repeated feedback cluster',
3943 |     departments: ['CSE', 'ECE', 'IT'].slice(0, 1 + (index % 3)),
3944 |     issuePhrase: theme.summary.toLowerCase(),
3945 |     rootCause: 'Repeated phrases, rating drops and category context point to the same issue pattern.',
3946 |     action: 'Assign an owner, inspect the exact area and track closure in the next report.',
3947 |     expectedImpact: '+6% satisfaction after closure',
3948 |   }
3949 | );
3950 | }
3951 |
3952 | function extractEvidenceComments(evidence: unknown, context: ReturnType<typeof inferIssueContext>) {
3953 |   const comments: string[] = [];
3954 |   if (evidence && typeof evidence === 'object') {
3955 |     const record = evidence as { sampleComments?: unknown; examples?: unknown };

```



```

3956 |     if (Array.isArray(record.sampleComments)) {
3957 |         comments.push(...record.sampleComments.filter((item): item is string => typeof item === 'string'));
3958 |     }
3959 |     if (Array.isArray(record.examples)) {
3960 |         comments.push(
3961 |             ...record.examples
3962 |             .map(item => {
3963 |                 if (typeof item === 'string') return item;
3964 |                 if (item && typeof item === 'object' && 'title' in item) return String((item as { title: unknown }).title);
3965 |                 return '';
3966 |             })
3967 |             .filter(Boolean),
3968 |         );
3969 |     }
3970 | }
3971 |
3972 | return (comments.length ? comments : [
3973 |     `${context.location} is being mentioned repeatedly in low-satisfaction comments.`,
3974 |     `Students are connecting this issue with ${context.issuePhrase}.`,
3975 |     `The affected group is mainly ${context.affectedGroup}.`,
3976 | ]).slice(0, 4);
3977 | }
3978 |
3979 | function buildIssueHeatmapData(
3980 |     data: DashboardSummary['categorySatisfaction'],
3981 |     themes: ThemeInsight[],
3982 | ) {
3983 |     const departments = ['CSE', 'ECE', 'IT', 'ME', 'CE'];
3984 |     const categories = (data.length ? data : [
3985 |         { categoryId: 'academics', categoryName: 'Academics', averageRating: 3.1, responseCount: 2200 },
3986 |         { categoryId: 'faculty', categoryName: 'Faculty', averageRating: 2.9, responseCount: 2100 },
3987 |         { categoryId: 'infra', categoryName: 'Infrastructure', averageRating: 2.4, responseCount: 1900 },
3988 |         { categoryId: 'food', categoryName: 'Food & Mess', averageRating: 2.2, responseCount: 1800 },
3989 |         { categoryId: 'sports', categoryName: 'Sports/Campus', averageRating: 3.0, responseCount: 1500 },
3990 |     ]);
3991 |
3992 |     const rows = categories.map((category, rowIndex) => {
3993 |         const issuePenalty = themes
3994 |             .filter((theme) => normalizeLabel(`${theme.title} ${theme.summary}`).includes(normalizeLabel(category.categoryName).split(' ')[0]))
3995 |             .reduce((total, theme) => total + Math.min(28, theme.mentionCount / 18), 0);
3996 |         const baseSatisfaction = Math.round((category.averageRating / 4) * 100);
3997 |
3998 |         return {
3999 |             category: compactCategoryName(category.categoryName),
4000 |             values: departments.map((_, colIndex) => {
4001 |                 const variation = (((rowIndex + 2) * (colIndex + 3) * 7) % 22) - 10;
4002 |                 return Math.max(12, Math.min(98, Math.round(baseSatisfaction - issuePenalty + variation)));
4003 |             }),
4004 |         };
4005 |     });
4006 |
4007 |     return { departments, rows };
4008 | }
4009 |
4010 | function compactCategoryName(name: string) {
4011 |     return name
4012 |         .replace('Food & Mess', 'Food')
4013 |         .replace('Sports & Campus', 'Sports')
4014 |         .replace('Sports/Campus', 'Sports');
4015 | }
4016 |
4017 | function normalizeLabel(value: string) {
4018 |     return value.toLowerCase().replace(/&/g, 'and').replace(/^[a-z0-9]+/g, ' ').trim();
4019 | }
4020 |
4021 | function priorityRank(priority: string) {
4022 |     return { LOW: 1, MEDIUM: 2, HIGH: 3, CRITICAL: 4 }[priority] ?? 0;
4023 | }
4024 |
4025 | function numberFromValue(...values: unknown[]) {
4026 |     for (const value of values) {
4027 |         const numeric = Number(value);
4028 |         if (Number.isFinite(numeric)) return Math.max(0, Math.round(numeric));
4029 |     }
4030 |     return 0;
4031 | }
4032 |
4033 | function humanizePipelineStep(step: string) {
4034 |     return step
4035 |         .replace(/_/g, ' ')
4036 |         .replace(/^\b\w/g, (letter) => letter.toUpperCase());
4037 | }
4038 |
4039 | void AiQualityCheckSection;
4040 | void buildAiQualityModel;
4041 |
4042 | function authHeaders(auth: AuthState): RequestInit {
4043 |     return {
4044 |         headers: {
4045 |             Authorization: `Bearer ${auth.accessToken}`,
4046 |         },
4047 |     };
4048 | }
4049 |
4050 | async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
4051 |     const response = await fetch(`${API_BASE}${path}`, {
4052 |         ...init,
4053 |         headers: {
4054 |             'Content-Type': 'application/json',
4055 |             ...(init.headers ?? {}),
4056 |         },
4057 |     });
4058 |
4059 |     if (!response.ok) {
4060 |         const errorBody = await response.json().catch(() => null);
4061 |         throw new Error(errorBody?.message ?? `Request failed: ${response.status}`);
4062 |     }
4063 |
4064 |     const text = await response.text();
4065 | }

```

```
4066 |   if (!text) {  
4067 |     return undefined as T;  
4068 |   }  
4069 |  
4070 |   return JSON.parse(text) as T;  
4071 | }  
4072 |  
4073 | function delay(ms: number) {  
4074 |   return new Promise((resolve) => window.setTimeout(resolve, ms));  
4075 | }  
4076 |  
4077 | export default App;
```

edupulse-frontend/src/components/LandingPage.css

File size: 48,414 bytes

```
1 | /*
2 |   EduPulse AI - Premium SaaS Landing Page Styles
3 |   Handcrafted with premium light-theme gradients, hovering animations, and soft glows.
4 | */
5 |
6 | :root {
7 |   --ivory-bg: #faf9f6;
8 |   --navy-text: #1e1b4b; /* Smooth dark Indigo-Navy instead of absolute black */
9 |   --slate-text: #475569; /* Smooth Slate-grey instead of dark charcoal */
10 |
11 |   /* Brand colors & glows */
12 |   --electric-blue: #1e40af;
13 |   --bright-blue: #2563eb;
14 |   --cyan-accent: #06b6d4;
15 |   --fresh-green: #10b981;
16 |   --alert-yellow: #eab308;
17 |   --glow-pink: #ec4899;
18 |
19 |   --soft-border: rgba(15, 23, 42, 0.07);
20 |   --hover-border: rgba(59, 130, 246, 0.3);
21 |   --card-shadow: 0 4px 24px rgba(15, 23, 42, 0.03), 0 10px 40px rgba(15, 23, 42, 0.04);
22 |   --hover-shadow: 0 25px 50px rgba(37, 99, 235, 0.12), 0 1px 3px rgba(0,0,0,0.01);
23 | }
24 |
25 | @keyframes cardGlow {
26 |   0%, 100% {
27 |     box-shadow: 0 4px 24px rgba(15, 23, 42, 0.02), 0 10px 40px rgba(15, 23, 42, 0.03);
28 |   }
29 |   50% {
30 |     box-shadow: 0 12px 36px rgba(37, 99, 235, 0.08), 0 16px 56px rgba(37, 99, 235, 0.04);
31 |   }
32 | }
33 |
34 | .landing-shell {
35 |   background-color: var(--ivory-bg);
36 |   color: var(--navy-text);
37 |   font-family: 'Inter', system-ui, -apple-system, sans-serif;
38 |   overflow-x: hidden;
39 |   position: relative;
40 |   min-height: 100vh;
41 |
42 |   /* Textured dot pattern to cluster empty space */
43 |   background-image:
44 |     radial-gradient(rgba(15, 23, 42, 0.05) 1.5px, transparent 1.5px);
45 |   background-size: 28px 28px;
46 | }
47 |
48 | /* Background glows */
49 | .landing-shell:before {
50 |   content: '';
51 |   position: absolute;
52 |   top: 0;
53 |   left: 0;
54 |   right: 0;
55 |   height: 1400px;
56 |   background:
57 |     radial-gradient(circle at 10% 12%, rgba(59, 130, 246, 0.08) 0%, transparent 45%),
58 |     radial-gradient(circle at 90% 25%, rgba(6, 182, 212, 0.08) 0%, transparent 45%),
59 |     radial-gradient(circle at 50% 50%, rgba(236, 72, 153, 0.05) 0%, transparent 50%),
60 |     radial-gradient(circle at 30% 80%, rgba(245, 158, 11, 0.04) 0%, transparent 45%);
61 |   pointer-events: none;
62 |   z-index: 0;
63 | }
64 |
65 | /* --- Section 1: Sticky Navigation --- */
66 | .nav-header {
67 |   position: sticky;
68 |   top: 0;
69 |   z-index: 100;
70 |   border-bottom: 1px solid var(--soft-border);
71 |   background: rgba(250, 249, 246, 0.85);
72 |   backdrop-filter: blur(16px);
73 |   padding: 12px 48px;
74 |   display: flex;
75 |   align-items: center;
76 |   justify-content: space-between;
77 | }
78 |
79 | .brand-wrapper {
80 |   display: flex;
81 |   align-items: center;
82 |   gap: 12px;
83 |   font-size: 1.35rem;
84 |   font-weight: 900;
85 |   color: var(--navy-text);
86 |   cursor: pointer;
87 |   transition: transform 0.2s;
88 | }
89 |
90 | .brand-wrapper:hover {
91 |   transform: scale(1.02);
92 | }
93 |
94 | .brand-logo-img {
95 |   width: 38px;
96 |   height: 38px;
97 |   object-fit: contain;
98 |   border-radius: 50%;
99 |   border: 1.5px solid rgba(59, 130, 246, 0.22);
100 |   box-shadow: 0 3px 8px rgba(59, 130, 246, 0.12);
101 |   background: #ffffff;
102 |   transition: transform 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275);
103 | }
104 |
105 | .brand-wrapper:hover .brand-logo-img {
```

```

106 |     transform: rotate(15deg) scale(1.08);
107 | }
108 |
109 | .nav-links {
110 |     display: flex;
111 |     align-items: center;
112 |     gap: 36px;
113 | }
114 |
115 | .nav-item {
116 |     color: var(--slate-text);
117 |     font-weight: 750; /* bold link */
118 |     text-decoration: none;
119 |     font-size: 0.95rem;
120 |     transition: color 0.2s, transform 0.2s;
121 |     cursor: pointer;
122 |     position: relative;
123 | }
124 |
125 | .nav-item::after {
126 |     content: '';
127 |     position: absolute;
128 |     width: 100%;
129 |     transform: scaleX(0);
130 |     height: 2px;
131 |     bottom: -4px;
132 |     left: 0;
133 |     background-color: var(--bright-blue);
134 |     transform-origin: bottom right;
135 |     transition: transform 0.25s ease-out;
136 | }
137 |
138 | .nav-item:hover::after {
139 |     transform: scaleX(1);
140 |     transform-origin: bottom left;
141 | }
142 |
143 | .nav-item:hover {
144 |     color: var(--navy-text);
145 |     transform: translateY(-1px);
146 | }
147 |
148 | /* Premium Buttons */
149 | /* Premium Buttons */
150 | .btn-primary {
151 |     background: linear-gradient(135deg, var(--navy-text) 0%, #1e293b 100%);
152 |     color: #fff !important;
153 |     border: 1px solid var(--navy-text);
154 |     padding: 12px 24px;
155 |     border-radius: 0px;
156 |     font-weight: 800;
157 |     font-size: 0.95rem;
158 |     cursor: pointer;
159 |     box-shadow: 0 4px 15px rgba(11, 19, 43, 0.1);
160 |     transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);
161 | }
162 |
163 | .btn-primary:hover {
164 |     background: linear-gradient(135deg, var(--bright-blue) 0%, var(--electric-blue) 100%);
165 |     border-color: var(--bright-blue);
166 |     box-shadow: 0 8px 25px rgba(59, 130, 246, 0.3);
167 |     transform: translateY(-2px);
168 | }
169 |
170 | .btn-secondary {
171 |     background: #fff;
172 |     color: var(--navy-text);
173 |     border: 1px solid var(--soft-border);
174 |     padding: 12px 24px;
175 |     border-radius: 0px;
176 |     font-weight: 800;
177 |     font-size: 0.95rem;
178 |     cursor: pointer;
179 |     box-shadow: 0 2px 8px rgba(0,0,0,0.02);
180 |     transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);
181 | }
182 |
183 | .btn-secondary:hover {
184 |     background: rgba(15, 23, 42, 0.02);
185 |     border-color: var(--navy-text);
186 |     box-shadow: 0 6px 18px rgba(0,0,0,0.06);
187 |     transform: translateY(-2px);
188 | }
189 |
190 | /* --- Section 2: Hero Section --- */
191 | .hero-panel {
192 |     max-width: 1280px;
193 |     margin: 0 auto;
194 |     padding: 40px 48px 60px 48px;
195 |     display: flex !important;
196 |     flex-direction: column !important;
197 |     gap: 82px !important;
198 |     align-items: center !important;
199 |     position: relative;
200 |     z-index: 1;
201 | }
202 |
203 | .hero-info {
204 |     display: flex !important;
205 |     flex-direction: column !important;
206 |     align-items: center !important;
207 |     text-align: center !important;
208 |     gap: 24px !important;
209 |     max-width: 850px !important;
210 |     margin: 0 auto !important;
211 | }
212 |
213 | .hero-tag {
214 |     align-self: center !important;
215 |     background: linear-gradient(135deg, rgba(59, 130, 246, 0.1), rgba(6, 182, 212, 0.1));

```

```

216 | color: var(--bright-blue);
217 | border: 1px solid rgba(59, 130, 246, 0.15);
218 | padding: 6px 18px;
219 | border-radius: 0px !important;
220 | font-size: 0.85rem;
221 | font-weight: 900;
222 | letter-spacing: 0.06em;
223 | text-transform: uppercase;
224 | }
225 |
226 | .hero-info h1 {
227 |   font-size: clamp(2.4rem, 5vw, 3.6rem) !important;
228 |   line-height: 1.1 !important;
229 |   font-weight: 950;
230 |   letter-spacing: -0.03em;
231 |   text-align: center !important;
232 |   background: linear-gradient(110deg, #111827 0%, #1d4ed8 30%, #06b6d4 48%, #ec4899 64%, #111827 88%);
233 |   background-size: 240% 100%;
234 |   background-clip: text;
235 |   -webkit-background-clip: text;
236 |   color: transparent;
237 |   margin: 0;
238 |   animation: heroTitleCharge 5.8s ease-in-out infinite;
239 |   filter: drop-shadow(0 14px 28px rgba(37, 99, 235, 0.12));
240 | }
241 |
242 | .hero-info p {
243 |   font-size: 1.25rem !important;
244 |   line-height: 1.65;
245 |   font-weight: 600;
246 |   color: var(--slate-text);
247 |   margin: 0 auto !important;
248 |   max-width: 720px !important;
249 |   text-align: center !important;
250 | }
251 |
252 | .hero-actions {
253 |   display: flex;
254 |   gap: 18px;
255 |   align-items: center;
256 |   margin-top: 8px;
257 |   justify-content: center !important;
258 | }
259 |
260 | .hero-trust-strip {
261 |   display: flex;
262 |   gap: 32px;
263 |   margin-top: 16px;
264 |   border-top: 1px solid var(--soft-border);
265 |   padding-top: 28px;
266 |   justify-content: center !important;
267 |   width: 100% !important;
268 | }
269 |
270 | .trust-item {
271 |   display: flex;
272 |   align-items: center;
273 |   gap: 10px;
274 |   font-size: 0.95rem;
275 |   font-weight: 800;
276 |   color: var(--slate-text);
277 | }
278 |
279 | .trust-item svg {
280 |   color: var(--fresh-green);
281 |   filter: drop-shadow(0 2px 4px rgba(16, 185, 129, 0.2));
282 | }
283 |
284 | /* --- Hero Black Hole Vortex Animation --- */
285 | .flow-stage {
286 |   width: 100% !important;
287 |   max-width: 850px !important;
288 |   height: 600px !important;
289 |   position: relative;
290 |   display: flex;
291 |   align-items: center;
292 |   justify-content: center;
293 |   background: transparent !important;
294 |   border: none !important;
295 |   box-shadow: none !important;
296 |   border-radius: 0px !important;
297 |   overflow: visible !important;
298 |   margin: 0 auto !important;
299 | }
300 |
301 | /* Vortex spiral background layers */
302 | .vortex-container {
303 |   position: absolute;
304 |   width: 700px !important;
305 |   height: 700px !important;
306 |   border-radius: 50%;
307 |   display: flex;
308 |   align-items: center;
309 |   justify-content: center;
310 |   pointer-events: none;
311 | }
312 |
313 | .vortex-spiral {
314 |   position: absolute;
315 |   width: 100%;
316 |   height: 100%;
317 |   border-radius: 50%;
318 |   border: 2px dashed rgba(59, 130, 246, 0.25);
319 |   animation: rotateClockwise 16s linear infinite;
320 | }
321 |
322 | .vortex-spiral-2 {
323 |   position: absolute;
324 |   width: 82%;
325 |   height: 82%;

```

```

326 |     border-radius: 50%;
327 |     border: 1.5px dashed rgba(6, 182, 212, 0.3);
328 |     animation: rotateCounterClockwise 12s linear infinite;
329 | }
330 |
331 | .vortex-spiral-3 {
332 |     position: absolute;
333 |     width: 64%;
334 |     height: 64%;
335 |     border-radius: 50%;
336 |     border: 2.5px dotted rgba(236, 72, 153, 0.25);
337 |     animation: rotateClockwise 8s linear infinite;
338 | }
339 |
340 | /* Accretion Disk behind Logo */
341 | .accretion-disk {
342 |     position: absolute;
343 |     width: 520px !important;
344 |     height: 520px !important;
345 |     border-radius: 50%;
346 |     background: radial-gradient(circle, rgba(59, 130, 246, 0.35) 0%, rgba(6, 182, 212, 0.18) 45%, rgba(236, 72, 153, 0.06) 65%, transparent 85%);
347 |     pointer-events: none;
348 |     animation: rotateClockwise 22s linear infinite;
349 |     z-index: 4;
350 | }
351 |
352 | /* Central Logo black hole style (SUPER BIG) */
353 | .blackhole-logo-wrapper {
354 |     width: 320px !important;
355 |     height: 320px !important;
356 |     border-radius: 50%;
357 |     background: #fff;
358 |     border: 4px solid rgba(59, 130, 246, 0.3);
359 |     box-shadow:
360 |         0 20px 60px rgba(15, 23, 42, 0.25),
361 |         0 0 110px rgba(59, 130, 246, 0.55),
362 |         0 0 160px rgba(6, 182, 212, 0.35),
363 |         inset 0 0 45px rgba(59, 130, 246, 0.35) !important;
364 |     display: flex;
365 |     align-items: center;
366 |     justify-content: center;
367 |     position: relative;
368 |     z-index: 10;
369 |     cursor: pointer;
370 |     overflow: hidden;
371 |     transition: transform 0.4s cubic-bezier(0.175, 0.885, 0.32, 1.275), box-shadow 0.4s;
372 |     animation: logoFloat 4s ease-in-out infinite, logoPowerPulse 2.6s ease-in-out infinite;
373 | }
374 |
375 | .blackhole-logo-wrapper::before,
376 | .blackhole-logo-wrapper::after {
377 |     content: '';
378 |     position: absolute;
379 |     inset: -12px;
380 |     border-radius: inherit;
381 |     pointer-events: none;
382 | }
383 |
384 | .blackhole-logo-wrapper::before {
385 |     border: 2px solid rgba(6, 182, 212, 0.36);
386 |     border-top-color: rgba(236, 72, 153, 0.72);
387 |     border-right-color: rgba(37, 99, 235, 0.66);
388 |     animation: rotateClockwise 7s linear infinite;
389 | }
390 |
391 | .blackhole-logo-wrapper::after {
392 |     inset: 18px;
393 |     border: 1px dashed rgba(15, 23, 42, 0.18);
394 |     animation: rotateCounterClockwise 10s linear infinite;
395 | }
396 |
397 | .blackhole-logo-wrapper img {
398 |     width: 260px !important;
399 |     height: 260px !important;
400 |     object-fit: contain;
401 |     border-radius: 50%;
402 |     position: relative;
403 |     z-index: 2;
404 |     transition: transform 0.4s ease;
405 | }
406 |
407 | .blackhole-logo-wrapper:hover {
408 |     transform: scale(1.1) rotate(6deg);
409 |     box-shadow:
410 |         0 25px 70px rgba(15, 23, 42, 0.25),
411 |         0 0 130px rgba(6, 182, 212, 0.65),
412 |         0 0 190px rgba(236, 72, 153, 0.45) !important;
413 | }
414 |
415 | .blackhole-logo-wrapper:hover img {
416 |     transform: scale(1.05) rotate(-6deg);
417 | }
418 |
419 | @keyframes heroTitleCharge {
420 |     0%,
421 |     100% {
422 |         background-position: 0% 50%;
423 |         filter: drop-shadow(0 14px 28px rgba(37, 99, 235, 0.1));
424 |     }
425 |
426 |     45% {
427 |         background-position: 100% 50%;
428 |         filter:
429 |             drop-shadow(0 18px 34px rgba(37, 99, 235, 0.2))
430 |             drop-shadow(0 0 14px rgba(6, 182, 212, 0.22));
431 |     }
432 |
433 |     52% {
434 |         background-position: 84% 50%;
435 |         filter:

```

```

436 |         drop-shadow(0 20px 38px rgba(236, 72, 153, 0.16))
437 |         drop-shadow(0 0 16px rgba(6, 182, 212, 0.24));
438 |     }
439 | }
440 |
441 | @keyframes logoFloat {
442 |     0%, 100% {
443 |         transform: translateY(0px) rotate(0deg);
444 |     }
445 |     50% {
446 |         transform: translateY(-12px) rotate(2deg);
447 |     }
448 | }
449 |
450 | @keyframes logoPowerPulse {
451 |     0%,
452 |     100% {
453 |         box-shadow:
454 |             0 20px 60px rgba(15, 23, 42, 0.25),
455 |             0 0 110px rgba(59, 130, 246, 0.55),
456 |             0 0 160px rgba(6, 182, 212, 0.35),
457 |             inset 0 0 45px rgba(59, 130, 246, 0.35);
458 |     }
459 |
460 |     50% {
461 |         box-shadow:
462 |             0 26px 74px rgba(15, 23, 42, 0.22),
463 |             0 0 138px rgba(37, 99, 235, 0.72),
464 |             0 0 205px rgba(236, 72, 153, 0.42),
465 |             inset 0 0 58px rgba(6, 182, 212, 0.38);
466 |     }
467 | }
468 |
469 | @keyframes chipLightSweep {
470 |     0%,
471 |     36% {
472 |         transform: translateX(-120%);
473 |         opacity: 0;
474 |     }
475 |
476 |     48% {
477 |         opacity: 0.75;
478 |     }
479 |
480 |     64%,
481 |     100% {
482 |         transform: translateX(120%);
483 |         opacity: 0;
484 |     }
485 | }
486 |
487 | /* Feedback particles spiraling into black hole */
488 | .blackhole-chip {
489 |     position: absolute;
490 |     background:
491 |         linear-gradient(90deg, rgba(255, 255, 255, 0.98), rgba(239, 246, 255, 0.92)),
492 |         radial-gradient(circle at 0% 50%, rgba(6, 182, 212, 0.22), transparent 42%);
493 |     border: 1.5px solid rgba(37, 99, 235, 0.2);
494 |     box-shadow:
495 |         0 10px 26px rgba(15, 23, 42, 0.08),
496 |         0 0 26px rgba(37, 99, 235, 0.12);
497 |     padding: 10px 16px;
498 |     border-radius: 4px !important;
499 |     font-size: 0.78rem;
500 |     font-weight: 950;
501 |     color: #0f172a;
502 |     letter-spacing: 0.02em;
503 |     white-space: nowrap;
504 |     pointer-events: none;
505 |     opacity: 0;
506 |     z-index: 5;
507 |     text-transform: uppercase;
508 |     clip-path: polygon(8px 0, 100% 0, calc(100% - 8px) 100%, 0 100%);
509 |     filter: saturate(1.12);
510 | }
511 |
512 | .blackhole-chip::after {
513 |     content: '';
514 |     position: absolute;
515 |     inset: 0;
516 |     background: linear-gradient(90deg, transparent, rgba(255, 255, 255, 0.9), transparent);
517 |     transform: translateX(-110%);
518 |     animation: chipLightSweep 2.8s ease-in-out infinite;
519 |     pointer-events: none;
520 | }
521 |
522 | /* Spiral Gravitational Pull Keyframes (10 paths) */
523 | .bh-path-1 { animation: suctionPath1 7.2s infinite linear; }
524 | .bh-path-2 { animation: suctionPath2 8.6s infinite linear; animation-delay: 1.2s; }
525 | .bh-path-3 { animation: suctionPath3 7.8s infinite linear; animation-delay: 2.5s; }
526 | .bh-path-4 { animation: suctionPath4 8.2s infinite linear; animation-delay: 3.8s; }
527 | .bh-path-5 { animation: suctionPath5 9s infinite linear; animation-delay: 0.6s; }
528 | .bh-path-6 { animation: suctionPath6 6.8s infinite linear; animation-delay: 1.8s; }
529 | .bh-path-7 { animation: suctionPath1 8.2s infinite linear; animation-delay: 3.2s; }
530 | .bh-path-8 { animation: suctionPath2 8.8s infinite linear; animation-delay: 4.8s; }
531 | .bh-path-9 { animation: suctionPath3 7.4s infinite linear; animation-delay: 5.6s; }
532 | .bh-path-10 { animation: suctionPath4 8.4s infinite linear; animation-delay: 0.2s; }
533 |
534 | /* Custom colored indicator dots inside particles */
535 | .blackhole-chip::before {
536 |     content: '';
537 |     display: inline-block;
538 |     width: 6px;
539 |     height: 6px;
540 |     border-radius: 50%;
541 |     margin-right: 8px;
542 |     vertical-align: middle;
543 | }
544 |
545 | .bh-path-1::before { background: var(--bright-blue); }

```

```

546 | .bh-path-2::before { background: var(--fresh-green); }
547 | .bh-path-3::before { background: var(--glow-pink); }
548 | .bh-path-4::before { background: var(--alert-yellow); }
549 | .bh-path-5::before { background: var(--cyan-accent); }
550 | .bh-path-6::before { background: var(--navy-text); }
551 | .bh-path-7::before { background: var(--bright-blue); }
552 | .bh-path-8::before { background: var(--glow-pink); }
553 | .bh-path-9::before { background: var(--fresh-green); }
554 | .bh-path-10::before { background: var(--cyan-accent); }
555 |
556 | @keyframes suctionPath1 {
557 |   0% { transform: translate(-340px, -320px) rotate(0deg) scale(1.15); opacity: 0; }
558 |   12% { opacity: 1; }
559 |   82% { opacity: 0.95; }
560 |   100% { transform: translate(0px, 0px) rotate(540deg) scale(0.08); opacity: 0; }
561 | }
562 |
563 | @keyframes suctionPath2 {
564 |   0% { transform: translate(350px, -300px) rotate(0deg) scale(1.15); opacity: 0; }
565 |   12% { opacity: 1; }
566 |   82% { opacity: 0.95; }
567 |   100% { transform: translate(0px, 0px) rotate(-540deg) scale(0.08); opacity: 0; }
568 | }
569 |
570 | @keyframes suctionPath3 {
571 |   0% { transform: translate(-360px, 280px) rotate(0deg) scale(1.15); opacity: 0; }
572 |   12% { opacity: 1; }
573 |   82% { opacity: 0.95; }
574 |   100% { transform: translate(0px, 0px) rotate(360deg) scale(0.08); opacity: 0; }
575 | }
576 |
577 | @keyframes suctionPath4 {
578 |   0% { transform: translate(340px, 320px) rotate(0deg) scale(1.15); opacity: 0; }
579 |   12% { opacity: 1; }
580 |   82% { opacity: 0.95; }
581 |   100% { transform: translate(0px, 0px) rotate(-360deg) scale(0.08); opacity: 0; }
582 | }
583 |
584 | @keyframes suctionPath5 {
585 |   0% { transform: translate(-420px, -25px) rotate(0deg) scale(1.15); opacity: 0; }
586 |   12% { opacity: 1; }
587 |   82% { opacity: 0.95; }
588 |   100% { transform: translate(0px, 0px) rotate(480deg) scale(0.08); opacity: 0; }
589 | }
590 |
591 | @keyframes suctionPath6 {
592 |   0% { transform: translate(420px, 25px) rotate(0deg) scale(1.15); opacity: 0; }
593 |   12% { opacity: 1; }
594 |   82% { opacity: 0.95; }
595 |   100% { transform: translate(0px, 0px) rotate(-480deg) scale(0.08); opacity: 0; }
596 | }
597 |
598 |
599 | /* --- Section 3: Storytelling Section (PREMIUM LIGHT-THEMED HUD - NO CARDS) --- */
600 | .story-wrapper {
601 |   background: linear-gradient(to bottom, #fcfbfa, #faf9f6);
602 |   border-top: 1px solid var(--soft-border);
603 |   border-bottom: 1px solid var(--soft-border);
604 |   padding: 50px 48px;
605 |   color: var(--navy-text);
606 |   position: relative;
607 | }
608 |
609 | .story-container {
610 |   max-width: 1320px;
611 |   margin: 0 auto;
612 | }
613 |
614 | .story-header {
615 |   text-align: center;
616 |   margin-bottom: 45px;
617 | }
618 |
619 | .story-wrapper .story-header h2 {
620 |   font-size: 3rem;
621 |   font-weight: 950;
622 |   margin-bottom: 18px;
623 |   text-align: center;
624 |   color: var(--navy-text);
625 |   text-transform: uppercase;
626 |   letter-spacing: 0.05em;
627 |   background: linear-gradient(135deg, var(--navy-text) 50%, var(--bright-blue) 100%);
628 |   background-clip: text;
629 |   -webkit-background-clip: text;
630 |   -webkit-text-fill-color: transparent;
631 | }
632 |
633 | .story-wrapper .story-header p {
634 |   font-size: 1.35rem; /* simple, large text */
635 |   font-weight: 700;
636 |   text-align: center;
637 |   color: var(--slate-text);
638 |   max-width: 650px;
639 |   margin: 0 auto;
640 |   line-height: 1.6;
641 | }
642 |
643 | /* Light HUD Grid Layout */
644 | .hud-lifecycle-grid {
645 |   display: grid;
646 |   grid-template-columns: minmax(560px, 1.32fr) minmax(390px, 0.78fr);
647 |   gap: 42px;
648 |   margin-top: 60px;
649 |   align-items: center;
650 | }
651 |
652 | /* Left side: HUD Step List (Borderless, clean lines) */
653 | .hud-step-stack {
654 |   display: flex;
655 |   flex-direction: column;

```



```

656 | gap: 16px;
657 | margin-left: -34px;
658 | max-width: 720px;
659 | }
660 |
661 | .hud-step-item {
662 |   background:
663 |     linear-gradient(135deg, rgba(255, 255, 255, 0.97), rgba(239, 246, 255, 0.8)),
664 |     radial-gradient(circle at 0% 50%, rgba(37, 99, 235, 0.12), transparent 38%);
665 |   border: 1px solid rgba(37, 99, 235, 0.16);
666 |   border-left: 6px solid rgba(59, 130, 246, 0.4); /* Tech style left bar */
667 |   border-radius: 10px;
668 |   padding: 20px 30px;
669 |   text-align: left;
670 |   cursor: pointer;
671 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);
672 |   display: flex;
673 |   flex-direction: column;
674 |   gap: 9px;
675 |   user-select: none;
676 |   box-shadow:
677 |     0 16px 34px rgba(15, 23, 42, 0.055),
678 |     inset 0 1px 0 rgba(255, 255, 255, 0.94);
679 |   position: relative;
680 |   overflow: hidden;
681 | }
682 |
683 | .hud-step-item.is-selected-step {
684 |   border-left-color: var(--bright-blue);
685 |   border-color: rgba(37, 99, 235, 0.34);
686 |   background:
687 |     linear-gradient(135deg, rgba(239, 246, 255, 0.98), rgba(224, 242, 254, 0.88)),
688 |     radial-gradient(circle at 8% 50%, rgba(37, 99, 235, 0.18), transparent 42%);
689 |   box-shadow:
690 |     0 24px 54px rgba(37, 99, 235, 0.14),
691 |     inset 1px 0 0 var(--bright-blue),
692 |     inset 0 1px 0 rgba(255, 255, 255, 0.95);
693 |   transform: translateX(16px) scale(1.015);
694 | }
695 |
696 | .hud-step-item:nth-child(2) {
697 |   background:
698 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(240, 253, 250, 0.78)),
699 |     radial-gradient(circle at 0% 50%, rgba(20, 184, 166, 0.13), transparent 38%);
700 |   border-left-color: rgba(20, 184, 166, 0.48);
701 | }
702 |
703 | .hud-step-item:nth-child(3) {
704 |   background:
705 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(253, 242, 248, 0.78)),
706 |     radial-gradient(circle at 0% 50%, rgba(236, 72, 153, 0.12), transparent 38%);
707 |   border-left-color: rgba(236, 72, 153, 0.42);
708 | }
709 |
710 | .hud-step-item:nth-child(4) {
711 |   background:
712 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(255, 247, 237, 0.82)),
713 |     radial-gradient(circle at 0% 50%, rgba(245, 158, 11, 0.16), transparent 38%);
714 |   border-left-color: rgba(245, 158, 11, 0.56);
715 | }
716 |
717 | .hud-step-item:nth-child(5) {
718 |   background:
719 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(240, 249, 255, 0.8)),
720 |     radial-gradient(circle at 0% 50%, rgba(6, 182, 212, 0.13), transparent 38%);
721 |   border-left-color: rgba(6, 182, 212, 0.48);
722 | }
723 |
724 | .hud-step-item:nth-child(6) {
725 |   background:
726 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(240, 253, 244, 0.8)),
727 |     radial-gradient(circle at 0% 50%, rgba(16, 185, 129, 0.14), transparent 38%);
728 |   border-left-color: rgba(16, 185, 129, 0.5);
729 | }
730 |
731 | .hud-step-item:nth-child(7) {
732 |   background:
733 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(245, 243, 255, 0.78)),
734 |     radial-gradient(circle at 0% 50%, rgba(124, 58, 237, 0.13), transparent 38%);
735 |   border-left-color: rgba(124, 58, 237, 0.48);
736 | }
737 |
738 | .hud-step-item:nth-child(8) {
739 |   background:
740 |     linear-gradient(135deg, rgba(255, 255, 255, 0.98), rgba(236, 253, 245, 0.78)),
741 |     radial-gradient(circle at 0% 50%, rgba(5, 150, 105, 0.13), transparent 38%);
742 |   border-left-color: rgba(5, 150, 105, 0.48);
743 | }
744 |
745 | .hud-step-header-row {
746 |   display: flex;
747 |   align-items: center;
748 |   gap: 12px;
749 |   margin-top: 4px;
750 | }
751 |
752 | .hud-step-icon-wrapper {
753 |   background: rgba(15, 23, 42, 0.03);
754 |   color: #64748b;
755 |   width: 34px;
756 |   height: 34px;
757 |   border-radius: 8px;
758 |   display: flex;
759 |   align-items: center;
760 |   justify-content: center;
761 |   border: 1px solid rgba(15, 23, 42, 0.05);
762 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);
763 | }
764 |
765 | .hud-step-item.is-selected-step .hud-step-icon-wrapper {

```

```

766 | background: rgba(37, 99, 235, 0.1);
767 | color: var(--bright-blue);
768 | border-color: rgba(37, 99, 235, 0.2);
769 | transform: scale(1.05);
770 | }
771 |
772 | .hud-step-meta {
773 |   display: flex;
774 |   align-items: center;
775 |   gap: 12px;
776 |   font-size: 0.95rem;
777 |   font-weight: 900;
778 |   letter-spacing: 0.1em;
779 |   text-transform: uppercase;
780 |   color: #64748b;
781 |   transition: color 0.3s;
782 | }
783 |
784 | .hud-step-item.is-selected-step .hud-step-meta {
785 |   color: var(--bright-blue);
786 | }
787 |
788 | .hud-step-tech-bracket {
789 |   color: var(--bright-blue);
790 |   font-family: monospace;
791 | }
792 |
793 | .hud-step-item h3 {
794 |   margin: 0;
795 |   font-size: 1.58rem; /* Larger and simpler step titles */
796 |   font-weight: 950;
797 |   color: #263247;
798 |   transition: color 0.3s;
799 |   letter-spacing: -0.01em;
800 | }
801 |
802 | .hud-step-item.is-selected-step h3 {
803 |   color: var(--navy-text);
804 | }
805 |
806 | /* Right side: Frosted Light Glass Radar Scope Display */
807 | .hud-radar-display {
808 |   display: flex;
809 |   flex-direction: column;
810 |   align-items: center;
811 |   justify-content: center;
812 |   background: rgba(255, 255, 255, 0.94);
813 |   border: 2px solid rgba(37, 99, 235, 0.18);
814 |   border-radius: 24px;
815 |   padding: 40px;
816 |   height: 480px;
817 |   position: relative;
818 |   overflow: hidden;
819 |   box-shadow:
820 |     0 15px 40px rgba(0, 0, 0, 0.03),
821 |     inset 0 0 25px rgba(37, 99, 235, 0.03);
822 | }
823 |
824 | .hud-radar-scope {
825 |   width: 240px;
826 |   height: 240px;
827 |   border-radius: 50%;
828 |   border: 2px solid rgba(37, 99, 235, 0.25); /* More visible rings */
829 |   position: relative;
830 |   display: flex;
831 |   align-items: center;
832 |   justify-content: center;
833 |   margin-bottom: 24px;
834 | }
835 |
836 | .hud-radar-ring-1 {
837 |   position: absolute;
838 |   width: 82%;
839 |   height: 82%;
840 |   border-radius: 50%;
841 |   border: 1.5px dashed rgba(6, 182, 212, 0.35);
842 |   animation: rotateCounterClockwise 18s linear infinite;
843 | }
844 |
845 | .hud-radar-ring-2 {
846 |   position: absolute;
847 |   width: 60%;
848 |   height: 60%;
849 |   border-radius: 50%;
850 |   border: 2px dotted rgba(236, 72, 153, 0.3);
851 |   animation: rotateClockwise 12s linear infinite;
852 | }
853 |
854 | .hud-radar-sweep {
855 |   position: absolute;
856 |   width: 50%;
857 |   height: 50%;
858 |   background: linear-gradient(135deg, rgba(37, 99, 235, 0.28) 0%, transparent 60%);
859 |   transform-origin: bottom right;
860 |   top: 0;
861 |   left: 0;
862 |   animation: rotateClockwise 6s linear infinite;
863 | }
864 |
865 | .hud-radar-center-icon {
866 |   position: absolute;
867 |   color: var(--bright-blue);
868 |   opacity: 0.85;
869 |   filter: drop-shadow(0 0 12px rgba(37, 99, 235, 0.4));
870 |   animation: radarPulse 2.5s ease-in-out infinite;
871 |   display: flex;
872 |   align-items: center;
873 |   justify-content: center;
874 |   z-index: 5;
875 | }

```

```

876 |
877 | @keyframes radarPulse {
878 |   0%, 100% {
879 |     transform: scale(1);
880 |     opacity: 0.85;
881 |   }
882 |   50% {
883 |     transform: scale(1.12);
884 |     opacity: 1;
885 |     filter: drop-shadow(0 0 24px rgba(37, 99, 235, 0.65));
886 |   }
887 | }
888 |
889 | .hud-radar-crosshair {
890 |   position: absolute;
891 |   color: var(--bright-blue);
892 |   font-family: monospace;
893 |   font-size: 1.25rem;
894 |   font-weight: 800;
895 | }
896 |
897 | .ch-top-left { top: 14px; left: 14px; }
898 | .ch-top-right { top: 14px; right: 14px; }
899 | .ch-bottom-left { bottom: 14px; left: 14px; }
900 | .ch-bottom-right { bottom: 14px; right: 14px; }
901 |
902 | .hud-active-step-details {
903 |   text-align: center;
904 |   width: 100%;
905 |   z-index: 10;
906 | }
907 |
908 | .hud-active-step-details h4 {
909 |   margin: 0 0 10px 0;
910 |   font-size: 1.45rem; /* Big simple heading text */
911 |   font-weight: 950;
912 |   letter-spacing: 0.05em;
913 |   text-transform: uppercase;
914 |   color: var(--navy-text);
915 | }
916 |
917 | .hud-active-step-details p {
918 |   margin: 0 0 18px 0;
919 |   font-size: 1.05rem; /* Big readable description */
920 |   font-weight: 600;
921 |   line-height: 1.6;
922 |   color: var(--slate-text);
923 |   height: 64px;
924 | }
925 |
926 | .hud-tech-feed {
927 |   display: flex;
928 |   justify-content: center;
929 |   gap: 24px;
930 |   font-family: monospace;
931 |   font-size: 0.78rem;
932 |   font-weight: 800;
933 |   color: var(--bright-blue);
934 |   border-top: 1px dashed rgba(37, 99, 235, 0.2);
935 |   padding-top: 14px;
936 | }
937 |
938 |
939 | /* --- Section 4: Team Section (FORCED SINGLE ROW) --- */
940 | .team-container {
941 |   max-width: 1640px;
942 |   margin: 0 auto;
943 |   padding: 56px 18px;
944 | }
945 |
946 | .team-heading-block {
947 |   text-align: center;
948 |   margin-bottom: 80px;
949 | }
950 |
951 | .team-heading-block h2 {
952 |   font-size: 3rem;
953 |   font-weight: 950;
954 |   margin-bottom: 18px;
955 |   color: var(--navy-text);
956 | }
957 |
958 | .team-heading-block p {
959 |   color: var(--slate-text);
960 |   font-size: 1.25rem;
961 |   font-weight: 600;
962 |   max-width: 680px;
963 |   margin: 0 auto;
964 |   line-height: 1.6;
965 | }
966 |
967 | .team-grid {
968 |   display: grid;
969 |   grid-template-columns: repeat(auto-fit, minmax(286px, 1fr));
970 |   gap: 18px;
971 |   justify-content: center;
972 | }
973 |
974 | /* Force all 5 team cards in a single row on desktop */
975 | @media (min-width: 992px) {
976 |   .team-grid {
977 |     grid-template-columns: repeat(5, minmax(270px, 1fr)) !important;
978 |     gap: 16px;
979 |   }
980 | }
981 |
982 | .team-member-card {
983 |   background:
984 |     linear-gradient(180deg, rgba(255, 255, 255, 0.98), rgba(250, 249, 246, 0.9)),
985 |     radial-gradient(circle at 50% 0%, rgba(37, 99, 235, 0.08), transparent 58%);

```

```

986 | border: 1px solid var(--soft-border);
987 | border-radius: 10px !important;
988 | padding: 26px 22px !important;
989 | aspect-ratio: 1 / 1;
990 | min-height: 318px;
991 | min-width: 0;
992 | display: flex;
993 | flex-direction: column;
994 | align-items: center;
995 | text-align: center;
996 | justify-content: center;
997 | box-shadow: var(--card-shadow);
998 | transition: all 0.4s cubic-bezier(0.16, 1, 0.3, 1);
999 | position: relative;
1000 | overflow: hidden;
1001 | border-top: 4px solid var(--soft-border) !important;
1002 | animation: cardGlow 6s infinite ease-in-out !important;
1003 | }
1004 |
1005 | .team-member-card:nth-child(1) { border-top-color: var(--bright-blue); }
1006 | .team-member-card:nth-child(2) { border-top-color: var(--cyan-accent); }
1007 | .team-member-card:nth-child(3) { border-top-color: var(--fresh-green); }
1008 | .team-member-card:nth-child(4) { border-top-color: var(--glow-pink); }
1009 | .team-member-card:nth-child(5) { border-top-color: var(--alert-yellow); }
1010 |
1011 | .team-member-card:hover {
1012 |   transform: translateY(-8px) scale(1.02);
1013 |   box-shadow: var(--hover-shadow);
1014 |   border-color: var(--hover-border);
1015 | }
1016 |
1017 | .team-member-card::before {
1018 |   content: '';
1019 |   position: absolute;
1020 |   top: 0;
1021 |   left: 0;
1022 |   width: 100%;
1023 |   height: 100%;
1024 |   background: radial-gradient(circle at 50% 0%, rgba(59, 130, 246, 0.03), transparent 70%);
1025 |   opacity: 0;
1026 |   transition: opacity 0.3s;
1027 | }
1028 |
1029 | .team-member-card:hover::before {
1030 |   opacity: 1;
1031 | }
1032 |
1033 | .avatar-initials {
1034 |   width: 68px;
1035 |   height: 68px;
1036 |   border-radius: 12px !important;
1037 |   background: linear-gradient(135deg, rgba(59, 130, 246, 0.08), rgba(6, 182, 212, 0.08));
1038 |   color: var(--bright-blue);
1039 |   font-size: 1.55rem;
1040 |   font-weight: 950;
1041 |   display: flex;
1042 |   align-items: center;
1043 |   justify-content: center;
1044 |   margin-bottom: 16px;
1045 |   border: 2px solid rgba(59, 130, 246, 0.12);
1046 |   box-shadow: 0 4px 10px rgba(59, 130, 246, 0.05);
1047 |   transition: all 0.3s ease;
1048 | }
1049 |
1050 | .team-member-card:hover .avatar-initials {
1051 |   transform: scale(1.1);
1052 |   color: #ffff;
1053 |   background: linear-gradient(135deg, var(--bright-blue), var(--cyan-accent));
1054 |   border-color: transparent;
1055 |   box-shadow: 0 8px 20px rgba(59, 130, 246, 0.2);
1056 | }
1057 |
1058 | .team-member-card h3 {
1059 |   font-family: 'Plus Jakarta Sans', 'Inter', system-ui, sans-serif;
1060 |   font-size: 1.34rem;
1061 |   font-weight: 950;
1062 |   margin: 0 0 8px 0;
1063 |   color: var(--navy-text);
1064 |   transition: color 0.2s;
1065 |   line-height: 1.08;
1066 |   letter-spacing: -0.025em;
1067 | }
1068 |
1069 | .team-member-card:hover h3 {
1070 |   color: var(--bright-blue);
1071 | }
1072 |
1073 | .member-role {
1074 |   font-family: 'Plus Jakarta Sans', 'Inter', system-ui, sans-serif;
1075 |   font-size: 1rem;
1076 |   font-weight: 950;
1077 |   color: var(--bright-blue);
1078 |   margin-bottom: 8px;
1079 |   line-height: 1.2;
1080 | }
1081 |
1082 | .member-edu {
1083 |   font-size: 0.88rem;
1084 |   color: #334155;
1085 |   font-weight: 850;
1086 |   margin-bottom: 10px;
1087 |   line-height: 1.25;
1088 | }
1089 |
1090 | .member-desc {
1091 |   font-size: 0.9rem;
1092 |   line-height: 1.42;
1093 |   color: #475569;
1094 |   font-weight: 650;
1095 |   margin: 0;

```

```

1096 |     z-index: 1;
1097 |     overflow: visible;
1098 | }
1099 |
1100 |
1101 | /* --- Section 5: AI Pipeline (3D CSS DICE Animation) --- */
1102 | .pipeline-section {
1103 |     background: linear-gradient(to bottom, #faf9f6, #f4f3f0);
1104 |     border-top: 1px solid var(--soft-border);
1105 |     border-bottom: 1px solid var(--soft-border);
1106 |     padding: 50px 48px;
1107 | }
1108 |
1109 | .pipeline-wrapper {
1110 |     max-width: 1240px;
1111 |     margin: 0 auto;
1112 | }
1113 |
1114 | .pipeline-wrapper .team-heading-block h2 {
1115 |     font-size: 3rem;
1116 |     font-weight: 950;
1117 |     color: var(--navy-text);
1118 | }
1119 |
1120 | .pipeline-wrapper .team-heading-block p {
1121 |     font-size: 1.25rem;
1122 |     font-weight: 600;
1123 |     color: var(--slate-text);
1124 | }
1125 |
1126 | .pipeline-grid-layout {
1127 |     display: grid;
1128 |     grid-template-columns: 1fr 1fr;
1129 |     gap: 50px;
1130 |     align-items: center;
1131 |     margin-top: 40px;
1132 | }
1133 |
1134 | @media (max-width: 991px) {
1135 |     .pipeline-grid-layout {
1136 |         grid-template-columns: 1fr;
1137 |         gap: 80px;
1138 |     }
1139 | }
1140 |
1141 | /* 3D Dice Stage */
1142 | .dice-stage {
1143 |     height: 480px !important;
1144 |     display: flex;
1145 |     align-items: center;
1146 |     justify-content: center;
1147 |     perspective: 1200px;
1148 |     position: relative;
1149 |     margin: 30px 0 !important;
1150 | }
1151 |
1152 | /* 3D Cube Container - Clickable Dice */
1153 | .pipeline-cube {
1154 |     width: 320px !important;
1155 |     height: 320px !important;
1156 |     position: relative;
1157 |     transform-style: preserve-3d;
1158 |     transition: transform 0.8s cubic-bezier(0.175, 0.885, 0.32, 1.15); /* Snappy feedback feel with spring effect */
1159 |     cursor: pointer;
1160 | }
1161 |
1162 | .pipeline-cube:hover .cube-face.is-active-face {
1163 |     box-shadow: 0 25px 60px rgba(59, 130, 246, 0.35), inset 0 0 25px rgba(59, 130, 246, 0.1) !important;
1164 | }
1165 |
1166 | /* Base face styling */
1167 | .cube-face {
1168 |     position: absolute;
1169 |     width: 320px !important;
1170 |     height: 320px !important;
1171 |     border-radius: 0px !important;
1172 |     display: flex;
1173 |     flex-direction: column;
1174 |     align-items: center;
1175 |     justify-content: center;
1176 |     padding: 32px !important;
1177 |     text-align: center;
1178 |     background: #ffffff; /* Solid white background for perfect contrast */
1179 |     backface-visibility: hidden; /* Hide the back of faces to prevent overlap text ghosting */
1180 |     transition: opacity 0.5s, border-color 0.5s, box-shadow 0.5s, background-color 0.5s;
1181 |     user-select: none;
1182 |     box-shadow: 0 8px 30px rgba(15, 23, 42, 0.04);
1183 |     opacity: 0.45; /* Fade out inactive faces for clean aesthetic focus */
1184 | }
1185 |
1186 | .cube-face h5 {
1187 |     margin: 0 0 12px 0 !important;
1188 |     font-size: 1.6rem !important; /* Big, readable heading */
1189 |     font-weight: 950;
1190 |     color: var(--navy-text);
1191 |     letter-spacing: -0.01em;
1192 | }
1193 |
1194 | .cube-face p {
1195 |     margin: 0;
1196 |     font-size: 1.05rem !important; /* Clean, simple size */
1197 |     line-height: 1.55;
1198 |     color: var(--slate-text);
1199 |     font-weight: 700; /* Medium-bold for high visibility */
1200 | }
1201 |
1202 | .cube-face-icon {
1203 |     margin-bottom: 24px !important;
1204 |     width: 64px !important;
1205 |     height: 64px !important;

```

```

1206 | border-radius: 0px !important;
1207 | display: flex;
1208 | align-items: center;
1209 | justify-content: center;
1210 | }
1211 |
1212 | /* Face placement in 3D Space (320px cube size => 160px translateZ) */
1213 | .face-front { transform: rotateY(0deg) translateZ(160px) !important; border: 2.5px solid rgba(59, 130, 246, 0.2) !important; }
1214 | .face-back { transform: rotateY(180deg) translateZ(160px) !important; border: 2.5px solid rgba(236, 72, 153, 0.2) !important; }
1215 | .face-right { transform: rotateY(90deg) translateZ(160px) !important; border: 2.5px solid rgba(6, 182, 212, 0.2) !important; }
1216 | .face-left { transform: rotateY(-90deg) translateZ(160px) !important; border: 2.5px solid rgba(245, 158, 11, 0.2) !important; }
1217 | .face-top { transform: rotateX(90deg) translateZ(160px) !important; border: 2.5px solid rgba(16, 185, 129, 0.2) !important; }
1218 | .face-bottom { transform: rotateX(-90deg) translateZ(160px) !important; border: 2.5px solid rgba(30, 80, 200, 0.2) !important; }
1219 |
1220 | /* Active Face gets full opacity and rich premium shadows */
1221 | .cube-face.is-active-face {
1222 |   opacity: 1;
1223 |   background: #ffffff;
1224 |   border-width: 4px !important;
1225 | }
1226 |
1227 | .face-front.is-active-face {
1228 |   border-color: var(--bright-blue) !important;
1229 |   box-shadow: 0 18px 45px rgba(59, 130, 246, 0.22), inset 0 0 20px rgba(59, 130, 246, 0.05) !important;
1230 | }
1231 | .face-back.is-active-face {
1232 |   border-color: var(--glow-pink) !important;
1233 |   box-shadow: 0 18px 45px rgba(236, 72, 153, 0.22), inset 0 0 20px rgba(236, 72, 153, 0.05) !important;
1234 | }
1235 | .face-right.is-active-face {
1236 |   border-color: var(--cyan-accent) !important;
1237 |   box-shadow: 0 18px 45px rgba(6, 182, 212, 0.22), inset 0 0 20px rgba(6, 182, 212, 0.05) !important;
1238 | }
1239 | .face-left.is-active-face {
1240 |   border-color: var(--alert-yellow) !important;
1241 |   box-shadow: 0 18px 45px rgba(245, 158, 11, 0.22), inset 0 0 20px rgba(245, 158, 11, 0.05) !important;
1242 | }
1243 | .face-top.is-active-face {
1244 |   border-color: var(--fresh-green) !important;
1245 |   box-shadow: 0 18px 45px rgba(16, 185, 129, 0.22), inset 0 0 20px rgba(16, 185, 129, 0.05) !important;
1246 | }
1247 | .face-bottom.is-active-face {
1248 |   border-color: var(--electric-blue) !important;
1249 |   box-shadow: 0 18px 45px rgba(30, 80, 200, 0.22), inset 0 0 20px rgba(30, 80, 200, 0.05) !important;
1250 | }
1251 |
1252 | /* Description panel next to dice */
1253 | .pipeline-desc-panel {
1254 |   display: flex;
1255 |   flex-direction: column;
1256 |   gap: 20px;
1257 | }
1258 |
1259 | .pipeline-step-card {
1260 |   background: #fff;
1261 |   border: 1px solid var(--soft-border);
1262 |   border-radius: 0px !important;
1263 |   padding: 44px 38px !important;
1264 |   box-shadow: var(--card-shadow);
1265 |   transition: all 0.4s cubic-bezier(0.16, 1, 0.3, 1);
1266 |   border-left: 5px solid var(--bright-blue) !important;
1267 |   animation: cardGlow 6s infinite ease-in-out !important;
1268 | }
1269 |
1270 | .pipeline-step-card.glow-card {
1271 |   border-color: var(--hover-border);
1272 |   box-shadow: 0 16px 40px rgba(59, 130, 246, 0.06);
1273 | }
1274 |
1275 | .pipeline-header-block {
1276 |   display: flex;
1277 |   align-items: center;
1278 |   justify-content: space-between;
1279 |   margin-bottom: 18px !important;
1280 | }
1281 |
1282 | .pipeline-header-block h3 {
1283 |   margin: 0;
1284 |   font-size: 1.65rem !important;
1285 |   font-weight: 900;
1286 |   color: var(--navy-text);
1287 | }
1288 |
1289 | .pipeline-step-indicator {
1290 |   font-size: 0.95rem !important;
1291 |   background: linear-gradient(135deg, rgba(59, 130, 246, 0.08), rgba(6, 182, 212, 0.08));
1292 |   color: var(--bright-blue);
1293 |   border: 1px solid rgba(59, 130, 246, 0.1);
1294 |   padding: 6px 14px !important;
1295 |   border-radius: 0px !important;
1296 |   font-weight: 900;
1297 | }
1298 |
1299 | .pipeline-desc-panel p.step-details {
1300 |   font-size: 1.25rem !important;
1301 |   line-height: 1.7 !important;
1302 |   color: var(--slate-text);
1303 |   font-weight: 500;
1304 |   margin: 0;
1305 | }
1306 |
1307 | /* Dots progress indicator */
1308 | .pipeline-dots-progress {
1309 |   display: flex;
1310 |   gap: 8px;
1311 |   justify-content: center;
1312 |   margin-top: 10px;
1313 | }
1314 |
1315 | .progress-dot {

```

```

1316 | width: 10px;
1317 | height: 10px;
1318 | border-radius: 50%;
1319 | background: rgba(15, 23, 42, 0.08);
1320 | cursor: pointer;
1321 | transition: all 0.3s;
1322 | }
1323 |
1324 | .progress-dot.active {
1325 |   background: var(--bright-blue);
1326 |   width: 24px;
1327 |   border-radius: 10px;
1328 |   box-shadow: 0 0 10px rgba(59, 130, 246, 0.3);
1329 | }
1330 |
1331 |
1332 | /* --- Section 6: SaaS Business Value --- */
1333 | .business-value-wrapper {
1334 |   max-width: 1280px;
1335 |   margin: 0 auto;
1336 |   padding: 50px 48px;
1337 | }
1338 |
1339 | .business-heading-center {
1340 |   text-align: center;
1341 |   margin-bottom: 80px;
1342 | }
1343 |
1344 | .business-heading-center h2 {
1345 |   font-size: 3rem;
1346 |   font-weight: 950;
1347 |   margin-bottom: 18px;
1348 |   color: var(--navy-text);
1349 | }
1350 |
1351 | .business-heading-center p {
1352 |   color: var(--slate-text);
1353 |   font-size: 1.25rem;
1354 |   font-weight: 600;
1355 |   max-width: 650px;
1356 |   margin: 0 auto;
1357 |   line-height: 1.6;
1358 | }
1359 |
1360 | .value-cards-layout {
1361 |   display: grid;
1362 |   grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
1363 |   gap: 30px;
1364 | }
1365 |
1366 | .value-panel-box {
1367 |   background: #fff;
1368 |   border: 1px solid var(--soft-border);
1369 |   border-radius: 0px !important;
1370 |   padding: 46px 38px !important;
1371 |   display: flex;
1372 |   flex-direction: column;
1373 |   gap: 20px;
1374 |   box-shadow: var(--card-shadow);
1375 |   transition: all 0.4s cubic-bezier(0.16, 1, 0.3, 1);
1376 |   position: relative;
1377 |   animation: cardGlow 6s infinite ease-in-out !important;
1378 | }
1379 |
1380 | .value-panel-box::before {
1381 |   content: '';
1382 |   position: absolute;
1383 |   inset: 0;
1384 |   border-radius: 0px !important;
1385 |   background: radial-gradient(circle at 100% 100%, rgba(59, 130, 246, 0.02), transparent 60%);
1386 |   opacity: 0;
1387 |   transition: opacity 0.3s;
1388 | }
1389 |
1390 | .value-panel-box:hover {
1391 |   transform: translateY(-6px);
1392 |   border-color: var(--hover-border);
1393 |   box-shadow: var(--hover-shadow);
1394 | }
1395 |
1396 | .value-panel-box:hover::before {
1397 |   opacity: 1;
1398 | }
1399 |
1400 | .value-icon-box {
1401 |   width: 56px !important;
1402 |   height: 56px !important;
1403 |   background: linear-gradient(135deg, rgba(59, 130, 246, 0.08), rgba(6, 182, 212, 0.08));
1404 |   border: 1px solid rgba(59, 130, 246, 0.1);
1405 |   border-radius: 0px !important;
1406 |   display: flex;
1407 |   align-items: center;
1408 |   justify-content: center;
1409 |   color: var(--bright-blue);
1410 |   transition: all 0.3s;
1411 | }
1412 |
1413 | .value-panel-box:hover .value-icon-box {
1414 |   transform: scale(1.1) rotate(5deg);
1415 |   color: #fff;
1416 |   background: linear-gradient(135deg, var(--bright-blue), var(--cyan-accent));
1417 |   border-color: transparent;
1418 |   box-shadow: 0 4px 15px rgba(59, 130, 246, 0.18);
1419 | }
1420 |
1421 | .value-panel-box h3 {
1422 |   font-size: 1.6rem !important;
1423 |   font-weight: 900;
1424 |   margin: 0;
1425 |   color: var(--navy-text);

```

```

1426 | }
1427 |
1428 | .value-panel-box p {
1429 |   font-size: 1.15rem !important;
1430 |   line-height: 1.7 !important;
1431 |   color: var(--slate-text);
1432 |   font-weight: 500;
1433 |   margin: 0;
1434 |   z-index: 1;
1435 | }
1436 |
1437 |
1438 | /* --- Section 7: Final CTA --- */
1439 | .final-cta-wrapper {
1440 |   background: #fff;
1441 |   border-top: 1px solid var(--soft-border);
1442 |   padding: 50px 48px;
1443 |   text-align: center;
1444 | }
1445 |
1446 | .final-cta-card {
1447 |   max-width: 800px;
1448 |   margin: 0 auto;
1449 |   display: flex;
1450 |   flex-direction: column;
1451 |   align-items: center;
1452 |   gap: 30px;
1453 | }
1454 |
1455 | .final-cta-card p.cta-paragraph {
1456 |   font-size: 1.3rem;
1457 |   line-height: 1.75;
1458 |   font-weight: 600;
1459 |   color: var(--slate-text);
1460 |   margin: 0;
1461 | }
1462 |
1463 | .cta-buttons-block {
1464 |   display: flex;
1465 |   gap: 10px;
1466 |   justify-content: center;
1467 | }
1468 |
1469 |
1470 | /* --- Section 8: Footer --- */
1471 | .footer-strip {
1472 |   border-top: 1px solid var(--soft-border);
1473 |   background: #faf9f6;
1474 |   padding: 24px 48px;
1475 |   text-align: center;
1476 | }
1477 |
1478 | .footer-content {
1479 |   max-width: 600px;
1480 |   margin: 0 auto;
1481 |   display: flex;
1482 |   flex-direction: column;
1483 |   gap: 10px;
1484 | }
1485 |
1486 | .footer-content h4 {
1487 |   font-size: 1.3rem;
1488 |   font-weight: 950;
1489 |   margin: 0;
1490 |   color: var(--navy-text);
1491 | }
1492 |
1493 | .footer-content p {
1494 |   font-size: 0.95rem;
1495 |   font-weight: 600;
1496 |   color: var(--slate-text);
1497 |   margin: 0;
1498 | }
1499 |
1500 |
1501 | /* --- Unified Role Selector Modal --- */
1502 | .role-modal-overlay {
1503 |   position: fixed;
1504 |   inset: 0;
1505 |   background: rgba(11, 19, 43, 0.35);
1506 |   backdrop-filter: blur(10px);
1507 |   z-index: 200;
1508 |   display: flex;
1509 |   align-items: center;
1510 |   justify-content: center;
1511 |   padding: 24px;
1512 | }
1513 |
1514 | .role-modal-box {
1515 |   background: #fff;
1516 |   border: 1px solid var(--soft-border);
1517 |   border-radius: 0px !important;
1518 |   width: 100%;
1519 |   max-width: 500px;
1520 |   padding: 40px;
1521 |   box-shadow: 0 30px 70px rgba(11, 19, 43, 0.18);
1522 |   display: flex;
1523 |   flex-direction: column;
1524 |   gap: 28px;
1525 |   position: relative;
1526 | }
1527 |
1528 | .role-modal-close {
1529 |   position: absolute;
1530 |   top: 24px;
1531 |   right: 24px;
1532 |   background: none;
1533 |   border: none;
1534 |   font-size: 1.35rem;
1535 |   cursor: pointer;

```



```

1536 |   color: var(--slate-text);
1537 |   transition: color 0.2s, transform 0.2s;
1538 |   display: flex;
1539 |   align-items: center;
1540 |   justify-content: center;
1541 | }
1542 |
1543 | .role-modal-close:hover {
1544 |   color: var(--navy-text);
1545 |   transform: rotate(90deg);
1546 | }
1547 |
1548 | .role-modal-box h2 {
1549 |   font-size: 1.7rem;
1550 |   font-weight: 950;
1551 |   margin: 0;
1552 |   text-align: center;
1553 |   color: var(--navy-text);
1554 | }
1555 |
1556 | .role-modal-desc {
1557 |   font-size: 1.05rem;
1558 |   color: var(--slate-text);
1559 |   text-align: center;
1560 |   margin: -14px 0 10px 0;
1561 | }
1562 |
1563 | .role-options-stack {
1564 |   display: flex;
1565 |   flex-direction: column;
1566 |   gap: 10px;
1567 | }
1568 |
1569 | .role-option-button {
1570 |   border: 1.5px solid var(--soft-border);
1571 |   border-radius: 0px !important;
1572 |   padding: 22px;
1573 |   background: #fff;
1574 |   display: flex;
1575 |   align-items: center;
1576 |   gap: 20px;
1577 |   cursor: pointer;
1578 |   text-align: left;
1579 |   transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);
1580 | }
1581 |
1582 | .role-option-button:hover {
1583 |   transform: translateY(-3px);
1584 |   border-color: var(--bright-blue);
1585 |   box-shadow: 0 10px 24px rgba(59, 130, 246, 0.08);
1586 | }
1587 |
1588 | .role-option-icon {
1589 |   width: 46px;
1590 |   height: 46px;
1591 |   background: rgba(59, 130, 246, 0.08);
1592 |   color: var(--bright-blue);
1593 |   border-radius: 0px !important;
1594 |   display: flex;
1595 |   align-items: center;
1596 |   justify-content: center;
1597 | }
1598 |
1599 | .role-option-button:last-child .role-option-icon {
1600 |   background: rgba(6, 182, 212, 0.08);
1601 |   color: var(--cyan-accent);
1602 | }
1603 |
1604 | .role-option-text {
1605 |   display: flex;
1606 |   flex-direction: column;
1607 |   gap: 4px;
1608 | }
1609 |
1610 | .role-option-title {
1611 |   font-weight: 900;
1612 |   color: var(--navy-text);
1613 |   font-size: 1.1rem;
1614 | }
1615 |
1616 | .role-option-hint {
1617 |   font-size: 0.85rem;
1618 |   color: var(--slate-text);
1619 | }
1620 |
1621 |
1622 | /* --- Responsive Adjustments --- */
1623 | @media (max-width: 991px) {
1624 |   .hero-panel {
1625 |     grid-template-columns: 1fr;
1626 |     gap: 50px;
1627 |     padding-top: 50px;
1628 |   }
1629 |   .flow-stage {
1630 |     height: 400px;
1631 |   }
1632 |   .nav-links {
1633 |     display: none;
1634 |   }
1635 |   .nav-header {
1636 |     padding: 18px 24px;
1637 |   }
1638 |   .team-grid {
1639 |     grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)) !important;
1640 |   }
1641 |   .hud-lifecycle-grid {
1642 |     grid-template-columns: 1fr;
1643 |     gap: 40px;
1644 |   }
1645 | }

```

```

1646 |
1647 | @media (max-width: 600px) {
1648 |   .hero-actions {
1649 |     flex-direction: column;
1650 |     align-items: stretch;
1651 |   }
1652 |   .cta-buttons-block {
1653 |     flex-direction: column;
1654 |     align-items: stretch;
1655 |   }
1656 |   .hero-trust-strip {
1657 |     flex-direction: column;
1658 |     gap: 14px;
1659 |   }
1660 | }
1661 |
1662 | /* --- High-Tech Decorative Floating Icons --- */
1663 | .floating-tech-icon {
1664 |   position: absolute;
1665 |   color: var(--bright-blue);
1666 |   opacity: 0.18;
1667 |   pointer-events: none;
1668 |   z-index: 2;
1669 |   transition: opacity 0.3s;
1670 | }
1671 |
1672 | .fti-1 { top: 12%; left: 12%; animation: floatSlow 7s infinite ease-in-out; }
1673 | .fti-2 { top: 16%; right: 12%; animation: floatSlow 9s infinite ease-in-out 1s; color: var(--glow-pink); }
1674 | .fti-3 { bottom: 16%; left: 16%; animation: floatSlow 8s infinite ease-in-out 2s; color: var(--cyan-accent); }
1675 | .fti-4 { bottom: 12%; right: 16%; animation: floatSlow 10s infinite ease-in-out 3s; color: var(--fresh-green); }
1676 |
1677 | @keyframes floatSlow {
1678 |   0%, 100% {
1679 |     transform: translateY(0px) rotate(0deg) scale(1);
1680 |   }
1681 |   50% {
1682 |     transform: translateY(-22px) rotate(20deg) scale(1.1);
1683 |     opacity: 0.35;
1684 |     filter: drop-shadow(0 0 8px currentColor);
1685 |   }
1686 | }
1687 |
1688 | /* --- HUD Active Terminal Logs --- */
1689 | .hud-terminal-log {
1690 |   background: rgba(15, 23, 42, 0.03);
1691 |   border: 1px solid rgba(37, 99, 235, 0.08);
1692 |   border-radius: 12px;
1693 |   padding: 14px;
1694 |   margin-top: 18px;
1695 |   font-family: monospace;
1696 |   font-size: 0.75rem;
1697 |   text-align: left;
1698 |   display: flex;
1699 |   flex-direction: column;
1700 |   gap: 6px;
1701 |   color: var(--slate-text);
1702 |   box-shadow: inset 0 2px 8px rgba(0, 0, 0, 0.02);
1703 | }
1704 |
1705 | .hud-log-line {
1706 |   color: var(--bright-blue);
1707 |   opacity: 0.9;
1708 |   white-space: nowrap;
1709 |   overflow: hidden;
1710 |   text-overflow: ellipsis;
1711 |   animation: logFadeIn 0.4s ease-out forwards;
1712 | }
1713 |
1714 | @keyframes logFadeIn {
1715 |   from {
1716 |     opacity: 0;
1717 |     transform: translateY(4px);
1718 |   }
1719 |   to {
1720 |     opacity: 0.9;
1721 |     transform: translateY(0);
1722 |   }
1723 | }
1724 |
1725 | /* --- Avatar Laser Scan Sweep --- */
1726 | .avatar-initials {
1727 |   position: relative;
1728 |   overflow: hidden;
1729 | }
1730 |
1731 | .avatar-initials::after {
1732 |   content: '';
1733 |   position: absolute;
1734 |   left: 0;
1735 |   width: 100%;
1736 |   height: 3px;
1737 |   background: linear-gradient(to right, transparent, var(--bright-blue), transparent);
1738 |   box-shadow: 0 0 8px var(--bright-blue);
1739 |   animation: scanSweep 3.5s infinite linear;
1740 |   opacity: 0.6;
1741 | }
1742 |
1743 | .team-member-card:nth-child(2) .avatar-initials::after {
1744 |   background: linear-gradient(to right, transparent, var(--cyan-accent), transparent);
1745 |   box-shadow: 0 0 8px var(--cyan-accent);
1746 | }
1747 | .team-member-card:nth-child(3) .avatar-initials::after {
1748 |   background: linear-gradient(to right, transparent, var(--fresh-green), transparent);
1749 |   box-shadow: 0 0 8px var(--fresh-green);
1750 | }
1751 | .team-member-card:nth-child(4) .avatar-initials::after {
1752 |   background: linear-gradient(to right, transparent, var(--glow-pink), transparent);
1753 |   box-shadow: 0 0 8px var(--glow-pink);
1754 | }
1755 | .team-member-card:nth-child(5) .avatar-initials::after {

```

```

1756 | background: linear-gradient(to right, transparent, var(--alert-yellow), transparent);
1757 | box-shadow: 0 0 8px var(--alert-yellow);
1758 | }
1759 |
1760 | @keyframes scanSweep {
1761 |   0% { top: -10%; }
1762 |   50% { top: 110%; }
1763 |   100% { top: -10%; }
1764 | }
1765 |
1766 | /* --- Scroll Reveal Animations --- */
1767 | .reveal-on-scroll {
1768 |   opacity: 0;
1769 |   transform: translate3d(0, 58px, 0) scale(0.975);
1770 |   filter: blur(10px);
1771 |   transition:
1772 |     opacity 0.95s cubic-bezier(0.16, 1, 0.3, 1),
1773 |     transform 0.95s cubic-bezier(0.16, 1, 0.3, 1),
1774 |     filter 0.95s cubic-bezier(0.16, 1, 0.3, 1);
1775 |   will-change: transform, opacity, filter;
1776 | }
1777 |
1778 | .reveal-on-scroll.is-visible {
1779 |   opacity: 1;
1780 |   transform: translate3d(0, 0, 0) scale(1);
1781 |   filter: blur(0);
1782 | }
1783 |
1784 | .story-header.reveal-on-scroll,
1785 | .team-heading-block.reveal-on-scroll,
1786 | .business-heading-center.reveal-on-scroll,
1787 | .final-cta-card.reveal-on-scroll,
1788 | .hero-info.reveal-on-scroll {
1789 |   transform: translate3d(0, 72px, 0) scale(0.97);
1790 | }
1791 |
1792 | .story-header.reveal-on-scroll.is-visible,
1793 | .team-heading-block.reveal-on-scroll.is-visible,
1794 | .business-heading-center.reveal-on-scroll.is-visible,
1795 | .final-cta-card.reveal-on-scroll.is-visible,
1796 | .hero-info.reveal-on-scroll.is-visible {
1797 |   transform: translate3d(0, 0, 0) scale(1);
1798 | }
1799 |
1800 | .hud-lifecycle-grid.reveal-on-scroll .hud-step-item,
1801 | .team-grid.reveal-on-scroll .team-member-card,
1802 | .pipeline-grid-layout.reveal-on-scroll > *,
1803 | .value-cards-layout.reveal-on-scroll > *,
1804 | .final-cta-card.reveal-on-scroll > *,
1805 | .hero-info.reveal-on-scroll > * {
1806 |   opacity: 0;
1807 |   transform: translate3d(0, 34px, 0);
1808 |   transition:
1809 |     opacity 0.72s cubic-bezier(0.16, 1, 0.3, 1),
1810 |     transform 0.72s cubic-bezier(0.16, 1, 0.3, 1);
1811 |   will-change: transform, opacity;
1812 | }
1813 |
1814 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item,
1815 | .team-grid.reveal-on-scroll.is-visible .team-member-card,
1816 | .pipeline-grid-layout.reveal-on-scroll.is-visible > *,
1817 | .value-cards-layout.reveal-on-scroll.is-visible > *,
1818 | .final-cta-card.reveal-on-scroll.is-visible > *,
1819 | .hero-info.reveal-on-scroll.is-visible > * {
1820 |   opacity: 1;
1821 |   transform: translate3d(0, 0, 0);
1822 | }
1823 |
1824 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(2),
1825 | .team-grid.reveal-on-scroll.is-visible .team-member-card:nth-child(2),
1826 | .value-cards-layout.reveal-on-scroll.is-visible > *:nth-child(2),
1827 | .hero-info.reveal-on-scroll.is-visible > *:nth-child(2) {
1828 |   transition-delay: 0.08s;
1829 | }
1830 |
1831 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(3),
1832 | .team-grid.reveal-on-scroll.is-visible .team-member-card:nth-child(3),
1833 | .value-cards-layout.reveal-on-scroll.is-visible > *:nth-child(3),
1834 | .hero-info.reveal-on-scroll.is-visible > *:nth-child(3) {
1835 |   transition-delay: 0.16s;
1836 | }
1837 |
1838 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(4),
1839 | .team-grid.reveal-on-scroll.is-visible .team-member-card:nth-child(4),
1840 | .value-cards-layout.reveal-on-scroll.is-visible > *:nth-child(4),
1841 | .hero-info.reveal-on-scroll.is-visible > *:nth-child(4) {
1842 |   transition-delay: 0.24s;
1843 | }
1844 |
1845 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(5),
1846 | .team-grid.reveal-on-scroll.is-visible .team-member-card:nth-child(5),
1847 | .hero-info.reveal-on-scroll.is-visible > *:nth-child(5) {
1848 |   transition-delay: 0.32s;
1849 | }
1850 |
1851 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(6) {
1852 |   transition-delay: 0.4s;
1853 | }
1854 |
1855 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(7) {
1856 |   transition-delay: 0.48s;
1857 | }
1858 |
1859 | .hud-lifecycle-grid.reveal-on-scroll.is-visible .hud-step-item:nth-child(8) {
1860 |   transition-delay: 0.56s;
1861 | }
1862 |
1863 | /* Reduced Motion Compatibility */
1864 | @media (prefers-reduced-motion: reduce) {
1865 |   .flow-stage,

```

```

1866 | .blackhole-logo-wrapper,
1867 | .vortex-spiral,
1868 | .vortex-spiral-2,
1869 | .vortex-spiral-3,
1870 | .accretion-disk,
1871 | .blackhole-chip,
1872 | .pipeline-cube,
1873 | .cube-face,
1874 | .hud-radar-sweep,
1875 | .hud-radar-ring-1,
1876 | .hud-radar-ring-2,
1877 | .story-node {
1878 |   animation: none !important;
1879 |   transition: none !important;
1880 |   transform: none !important;
1881 |   opacity: 1 !important;
1882 | }
1883 | }
1884 |
1885 | /* Final brand consistency: keep the compact navbar logo aligned. */
1886 | .brand-wrapper {
1887 |   color: var(--navy-text) !important;
1888 |   font-weight: 950 !important;
1889 |   letter-spacing: -0.02em !important;
1890 | }
1891 |
1892 | .brand-wrapper .brand-logo-img {
1893 |   width: 38px !important;
1894 |   height: 38px !important;
1895 |   max-width: 38px !important;
1896 |   max-height: 38px !important;
1897 |   aspect-ratio: 1 / 1 !important;
1898 |   border-radius: 8px !important;
1899 |   border: 1px solid rgba(15, 23, 42, 0.08) !important;
1900 |   background: #ffffff !important;
1901 |   object-fit: contain !important;
1902 |   box-shadow: 0 12px 28px rgba(37, 99, 235, 0.12) !important;
1903 | }
1904 |
1905 | .brand-wrapper:hover .brand-logo-img {
1906 |   transform: scale(1.05) !important;
1907 | }
1908 |
1909 | .blackhole-logo-wrapper {
1910 |   border-radius: 999px !important;
1911 |   overflow: hidden !important;
1912 | }
1913 |
1914 | .blackhole-logo-wrapper img {
1915 |   aspect-ratio: 1 / 1 !important;
1916 |   object-fit: contain !important;
1917 |   border-radius: 999px !important;
1918 | }

```

edupulse-frontend/src/components/LandingPage.tsx

File size: 30,890 bytes

```
1 | import { useEffect, useRef, useState } from 'react';
2 | import {
3 |   Sparkles,
4 |   CheckCircle2,
5 |   ShieldCheck,
6 |   GraduationCap,
7 |   ChevronRight,
8 |   Lock,
9 |   Users,
10 |   FileText,
11 |   Layers,
12 |   X,
13 |   ArrowRight,
14 |   Filter,
15 |   Copy,
16 |   Activity,
17 |   Tags,
18 |   AlertOctagon,
19 |   FolderTree,
20 |   TrendingUp,
21 |   Sparkle,
22 |   Gauge
23 | } from 'lucide-react';
24 |
25 | import edupulseLogo from '../assets/logo.png';
26 | import './LandingPage.css';
27 |
28 | // types
29 | type Role = 'ADMIN' | 'STUDENT';
30 |
31 | interface LandingProps {
32 |   onChooseRole: (role: Role) => void;
33 | }
34 |
35 | // Lifecycle Steps for HUD
36 | const lifecycleSteps = [
37 |   {
38 |     step: "01",
39 |     title: "Student Submission",
40 |     detail: "Students submit feedback securely along with ratings, comments, category, branch, and semester tags.",
41 |     coord: "LAT: 28.469 | LNG: 77.524",
42 |     status: "INGESTION_ACTIVE",
43 |     iconName: "GraduationCap",
44 |     logs: [
45 |       "> INGESTING FEEDBACK STREAM FROM STUDENT PORTALS",
46 |       "> SECURING META-TAGS (BRANCH, RATINGS, TERM)",
47 |       "> INGESTION SYNCHRONIZED [OK]"
48 |     ]
49 |   },
50 |   {
51 |     step: "02",
52 |     title: "Data Cleansing",
53 |     detail: "EduPulse AI automatically filters out fake, spam, repeating, and out-of-context comments.",
54 |     coord: "INTEGRITY_SCAN: 99.8%",
55 |     status: "CLEANSING_ACTIVE",
56 |     iconName: "Filter",
57 |     logs: [
58 |       "> EXECUTING HEURISTIC SPAM SHIELD V4.2",
59 |       "> FILTERING SPAMMERS AND OFF-TOPIC PHRASES",
60 |       "> INTEGRITY VERIFICATION: 99.8% TRUST RATIO"
61 |     ]
62 |   },
63 |   {
64 |     step: "03",
65 |     title: "Duplicate Grouping",
66 |     detail: "Similar complaints are automatically cataloged and counted together instead of filling the board as spam.",
67 |     coord: "DEDUPLICATION: ENABLED",
68 |     status: "GROUPING_ACTIVE",
69 |     iconName: "Copy",
70 |     logs: [
71 |       "> VECTOR CLUSTERING ON COMMENT SEGMENTS",
72 |       "> SEARCHING FOR DUPLICATED COMPLAINT PATHS",
73 |       "> CONSOLIDATED 14 REPEATED NODES [OK]"
74 |     ]
75 |   },
76 |   {
77 |     step: "04",
78 |     title: "Urgency Detection",
79 |     detail: "Serious concerns (such as ragging or harassment) are separated instantly for quick leadership attention.",
80 |     coord: "SEVERITY_LEVEL: CRITICAL",
81 |     status: "SAFETY_FILTER_ONLINE",
82 |     iconName: "AlertOctagon",
83 |     logs: [
84 |       "> DISPATCHING SAFETY THREAD SCANNER",
85 |       "> SCANNING FOR HARASSMENT OR SECURITY ISSUES",
86 |       "> CRITICAL FLAG DETECTED -> REPORT ESCALATED"
87 |     ]
88 |   },
89 |   {
90 |     step: "05",
91 |     title: "Classification Math",
92 |     detail: "Issues are grouped across exact facility, academic, transport, safety, and hygiene taxonomies.",
93 |     coord: "TAXONOMY: 6_CHANNELS",
94 |     status: "CLASSIFIER_ONLINE",
95 |     iconName: "FolderTree",
96 |     logs: [
97 |       "> RUNNING TOKENIZER & N-GRAM CORRELATOR",
98 |       "> SORTING ENTRIES TO TAXONOMY MAP",
99 |       "> CATEGORY CORRELATION ASSIGNED [SUCCESS]"
100 |   },
101 | ],
102 | {
103 |   step: "06",
104 |   title: "Priority Scoring",
105 |   detail: "Priority level is calculated using transparent math proportional to the volume of valid student complaints.",
```

```

106 |     coord: "PRIORITY_RATIO: 1.48",
107 |     status: "SCORING_ACTIVE",
108 |     iconName: "Gauge",
109 |     logs: [
110 |         "> AGGREGATING GRIEVANCE FREQUENCY RATIOS",
111 |         "> INITIATING SCORE INTEGRATOR (FACTOR: 1.48)",
112 |         "> PRIORITY STATUS CALCULATED -> HIGH"
113 |     ]
114 | },
115 | {
116 |     step: "07",
117 |     title: "Gemini Narrative",
118 |     detail: "Gemini AI structures findings into human-readable action summaries and board-ready highlights.",
119 |     coord: "MODEL_STACK: GEMINI_PRO",
120 |     status: "NARRATIVE_ONLINE",
121 |     iconName: "Sparkles",
122 |     logs: [
123 |         "> CONNECTING TO GEMINI MODEL API CLUSTER",
124 |         "> PARSING DATASETS INTO EXECUTIVE RESUMES",
125 |         "> NARRATIVE SYNTHESIS COMPLETED [OK]"
126 |     ]
127 | },
128 | {
129 |     step: "08",
130 |     title: "Admin Action Center",
131 |     detail: "Institution heads receive visual dashboards, critical alerts, and downloadable action reports.",
132 |     coord: "PORTAL_SYNC: ESTABLISHED",
133 |     status: "SYNC_ACTIVE",
134 |     iconName: "ShieldCheck",
135 |     logs: [
136 |         "> SYNCHRONIZING REALTIME ADMIN CONSOLE",
137 |         "> REFRESHING ANALYTICS SCHEMATICS & CHARTS",
138 |         "> COMPILING AUDIT-READY PDF BRIEFINGS"
139 |     ]
140 | }
141 | ];
142 |
143 | const getLifecycleIcon = (name: string, size = 24) => {
144 |     switch (name) {
145 |         case 'GraduationCap': return <GraduationCap size={size} />;
146 |         case 'Filter': return <Filter size={size} />;
147 |         case 'Copy': return <Copy size={size} />;
148 |         case 'AlertOctagon': return <AlertOctagon size={size} />;
149 |         case 'FolderTree': return <FolderTree size={size} />;
150 |         case 'Gauge': return <Gauge size={size} />;
151 |         case 'Sparkles': return <Sparkles size={size} />;
152 |         case 'ShieldCheck': return <ShieldCheck size={size} />;
153 |         default: return <Sparkles size={size} />;
154 |     }
155 | };
156 |
157 | // 3D Pipeline steps
158 | const pipelineSteps = [
159 |     {
160 |         num: "01",
161 |         title: "Raw Feedback",
162 |         detail: "Student ratings, comments, category, branch, and semester tags are collected at entry.",
163 |         face: 1, // Front
164 |         icon: <Sparkles size={24} />
165 |     },
166 |     {
167 |         num: "02",
168 |         title: "Quality Filter",
169 |         detail: "Fake, useless, random, repeated, and out-of-context comments are automatically removed.",
170 |         face: 2, // Back
171 |         icon: <Filter size={24} />
172 |     },
173 |     {
174 |         num: "03",
175 |         title: "Duplicate Grouping",
176 |         detail: "Same or similar repeated comments are counted together instead of filling the database as spam.",
177 |         face: 2, // Back
178 |         icon: <Copy size={24} />
179 |     },
180 |     {
181 |         num: "04",
182 |         title: "Sentiment & Rating",
183 |         detail: "Rating-based math classifies feedback into satisfied, average, and unsatisfied groups.",
184 |         face: 3, // Right
185 |         icon: <Activity size={24} />
186 |     },
187 |     {
188 |         num: "05",
189 |         title: "Issue Taxonomy",
190 |         detail: "The system detects exact issue types across facilities, academics, faculty, transport, safety, and hygiene.",
191 |         face: 3, // Right
192 |         icon: <Tags size={24} />
193 |     },
194 |     {
195 |         num: "06",
196 |         title: "Urgent Concern",
197 |         detail: "Serious issues like ragging, harassment, weapons, snakes, or contamination move to Immediate Action.",
198 |         face: 4, // Left
199 |         icon: <AlertOctagon size={24} />
200 |     },
201 |     {
202 |         num: "07",
203 |         title: "Normal Clustering",
204 |         detail: "Similar normal issues are grouped together (e.g. multiple instructor absences form one attendance issue).",
205 |         face: 5, // Top
206 |         icon: <FolderTree size={24} />
207 |     },
208 |     {
209 |         num: "08",
210 |         title: "Priority Scoring",
211 |         detail: "Priority is calculated: High (>10% valid reviews), Medium (5%-10%), and Low (<5%).",
212 |         face: 5, // Top
213 |         icon: <TrendingUp size={24} />
214 |     },
215 | ];

```

```

216 |     num: "09",
217 |     title: "Gemini Layer",
218 |     detail: "Gemini is used after structuring to draft human-like action wording and board highlights.",
219 |     face: 6, // Bottom
220 |     icon: <Sparkle size={24} />
221 | },
222 | {
223 |   num: "10",
224 |   title: "Dashboard & Report",
225 |   detail: "Visual administrative dashboards, live alerts, and downloadable PDF action reports are generated.",
226 |   face: 6, // Bottom
227 |   icon: <Gauge size={24} />
228 | }
229 | ];
230 |
231 | export default function LandingPage({ onChooseRole }: LandingProps) {
232 |   const [modalOpen, setModalOpen] = useState(false);
233 |   const pipelineRef = useRef<HTMLDivElement | null>(null);
234 |   const productRef = useRef<HTMLDivElement | null>(null);
235 |   const teamRef = useRef<HTMLDivElement | null>(null);
236 |
237 |   // Pipeline animation step cycling
238 |   const [activeStep, setActiveStep] = useState(0);
239 |
240 |   // HUD Interactive Lifecycle Step Selection
241 |   const [hudStep, setHudStep] = useState(0);
242 |
243 |   const rotationTimerRef = useRef<ReturnType<typeof setInterval> | null>(null);
244 |
245 |   const startAutoRotation = () => {
246 |     if (rotationTimerRef.current) {
247 |       clearInterval(rotationTimerRef.current);
248 |     }
249 |     rotationTimerRef.current = setInterval(() => {
250 |       setActiveStep((prev) => (prev + 1) % 10);
251 |     }, 3200);
252 |   };
253 |
254 |   useEffect(() => {
255 |     startAutoRotation();
256 |     return () => {
257 |       if (rotationTimerRef.current) {
258 |         clearInterval(rotationTimerRef.current);
259 |       }
260 |     };
261 |   }, []);
262 |
263 |   useEffect(() => {
264 |     const revealElements = Array.from(document.querySelectorAll<HTMLElement>('.reveal-on-scroll'));
265 |
266 |     const revealVisibleElements = () => {
267 |       const triggerPoint = window.innerHeight * 0.86;
268 |       revealElements.forEach((el) => {
269 |         if (el.classList.contains('is-visible')) return;
270 |         const rect = el.getBoundingClientRect();
271 |         if (rect.top <= triggerPoint && rect.bottom >= 0) {
272 |           el.classList.add('is-visible');
273 |         }
274 |       });
275 |     };
276 |
277 |     const observer = 'IntersectionObserver' in window
278 |       ? new IntersectionObserver(
279 |         (entries, currentObserver) => {
280 |           entries.forEach((entry) => {
281 |             if (entry.isIntersecting) {
282 |               entry.target.classList.add('is-visible');
283 |               currentObserver.unobserve(entry.target);
284 |             }
285 |           });
286 |         },
287 |         {
288 |           root: null,
289 |           rootMargin: '0px 0px -8% 0px',
290 |           threshold: 0.08,
291 |         },
292 |       )
293 |       : null;
294 |
295 |     revealElements.forEach((el) => observer?.observe(el));
296 |     revealVisibleElements();
297 |     const animationFrame = window.requestAnimationFrame(revealVisibleElements);
298 |
299 |     window.addEventListener('scroll', revealVisibleElements, { passive: true });
300 |     window.addEventListener('resize', revealVisibleElements);
301 |
302 |     return () => {
303 |       window.cancelAnimationFrame(animationFrame);
304 |       window.removeEventListener('scroll', revealVisibleElements);
305 |       window.removeEventListener('resize', revealVisibleElements);
306 |       observer?.disconnect();
307 |     };
308 |   }, []);
309 |
310 |   const handleCubeClick = () => {
311 |     setActiveStep((prev) => (prev + 1) % 10);
312 |     startAutoRotation();
313 |   };
314 |
315 |   const handleDotClick = (idx: number) => {
316 |     setActiveStep(idx);
317 |     startAutoRotation();
318 |   };
319 |
320 |   const triggerScroll = (elementRef: React.RefObject<HTMLDivElement | null>) => {
321 |     if (elementRef.current) {
322 |       elementRef.current.scrollIntoView({ behavior: 'smooth' });
323 |     }
324 |   };
325 |

```

```

326 // Get rotation degrees for the 3D dice based on current active face
327 const getDiceRotation = () => {
328   const face = pipelineSteps[activeStep].face;
329   switch(face) {
330     case 1: return 'rotateX(0deg) rotateY(0deg)';
331     case 2: return 'rotateX(0deg) rotateY(180deg)';
332     case 3: return 'rotateX(0deg) rotateY(-90deg)';
333     case 4: return 'rotateX(0deg) rotateY(90deg)';
334     case 5: return 'rotateX(-90deg) rotateY(0deg)';
335     case 6: return 'rotateX(90deg) rotateY(0deg)';
336     default: return 'rotateX(0deg) rotateY(0deg)';
337   }
338 };
339
340 return (
341   <div className="landing-shell">
342     { /* 1. STICKY NAVIGATION BAR */ }
343     <header className="nav-header">
344       <div className="brand-wrapper" onClick={() => window.scrollTo({ top: 0, behavior: 'smooth' })}>
345         <img src={eduPulseLogo} alt="EduPulse AI Logo" className="brand-logo-img" />
346         <span>EduPulse AI</span>
347       </div>
348
349       <nav className="nav-links">
350         <span className="nav-item" onClick={() => triggerScroll(productRef)}>Product</span>
351         <span className="nav-item" onClick={() => triggerScroll(pipelineRef)}>Pipeline</span>
352         <span className="nav-item" onClick={() => triggerScroll(teamRef)}>Team</span>
353         <button className="btn-primary" onClick={() => setModalOpen(true)}>
354           Get Started
355         </button>
356       </nav>
357     </header>
358
359     { /* 2. HERO SECTION (Black Hole Animation with Logo) */ }
360     <section className="hero-panel">
361       { /* Black Hole stage */ }
362       <div className="flow-stage reveal-on-scroll">
363         { /* Floating decorative high-tech symbols */ }
364         <div className="floating-tech-icon fti-1"><Sparkles size={18} /></div>
365         <div className="floating-tech-icon fti-2"><Activity size={18} /></div>
366         <div className="floating-tech-icon fti-3"><Gauge size={18} /></div>
367         <div className="floating-tech-icon fti-4"><ShieldCheck size={18} /></div>
368
369         { /* Vortex accretion glow & rotation */ }
370         <div className="accretion-disk" />
371         <div className="vortex-container">
372           <div className="vortex-spiral" />
373           <div className="vortex-spiral-2" />
374           <div className="vortex-spiral-3" />
375         </div>
376
377         { /* Central Logo Black Hole (VERY BIG) */ }
378         <div className="blackhole-logo-wrapper" onClick={() => setModalOpen(true)}>
379           <img src={eduPulseLogo} alt="EduPulse AI Logo" />
380         </div>
381
382         { /* Feedback chips swirling (10 chips) */ }
383         <div className="blackhole-chip bh-path-1">"Wi-Fi not working in library"</div>
384         <div className="blackhole-chip bh-path-2">"Mess food quality is poor"</div>
385         <div className="blackhole-chip bh-path-3">"Ragging concern in hostel"</div>
386         <div className="blackhole-chip bh-path-4">"Water contamination in block B"</div>
387         <div className="blackhole-chip bh-path-5">"Random useless spam text"</div>
388         <div className="blackhole-chip bh-path-6">"Projector issue in LH 203"</div>
389         <div className="blackhole-chip bh-path-7">"Faculty absent in ECE block"</div>
390         <div className="blackhole-chip bh-path-8">"Transport delay on Route 4"</div>
391         <div className="blackhole-chip bh-path-9">"Library books unavailable"</div>
392         <div className="blackhole-chip bh-path-10">"Harassment report in building 2"</div>
393       </div>
394
395       <div className="hero-info reveal-on-scroll">
396         <h1>Transform Campus Feedback Into Actionable Institutional Intelligence</h1>
397         <p>
398           EduPulse AI turns raw student feedback into verified insights, urgent alerts, structured issue clusters, and admin-ready action reports.
399         </p>
400
401         <div className="hero-actions">
402           <button className="btn-primary" onClick={() => setModalOpen(true)}>
403             Get Started
404           </button>
405           <button className="btn-secondary" onClick={() => triggerScroll(pipelineRef)}>
406             View AI Pipeline
407           </button>
408         </div>
409
410         <div className="hero-trust-strip">
411           <div className="trust-item">
412             <CheckCircle2 size={16} /> Quality-filtered feedback
413           </div>
414           <div className="trust-item">
415             <CheckCircle2 size={16} /> Urgent concern detection
416           </div>
417           <div className="trust-item">
418             <CheckCircle2 size={16} /> Admin-ready reports
419           </div>
420         </div>
421       </div>
422     </section>
423
424     { /* 3. STORYTELLING SECTION (FUTURISTIC HUD LIFE-CYCLE - NO CARDS) */ }
425     <section ref={productRef} className="story-wrapper">
426       <div className="story-container">
427         <div className="story-header reveal-on-scroll">
428           <h2 style={{ display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
429             <Activity size={24} style={{ marginRight: '12px', color: 'var(--bright-blue)' }} />
430             Campus Feedback Lifecycle
431           </h2>
432           <p>See how raw student reviews translate into solid, non-spammed reports for executive decision-makers.</p>
433         </div>
434
435         { /* Futuristic HUD interface */ }

```



```

436 | <div className="hud-lifecycle-grid reveal-on-scroll">
437 |
438 |   {/* Left side list of steps (borderless, bracketed lines) */}
439 |   <div className="hud-step-stack">
440 |     {lifecycleSteps.map((item, idx) => (
441 |       <div
442 |         key={idx}
443 |         className={`hud-step-item ${hudStep === idx ? 'is-selected-step' : ''}`}
444 |         onMouseEnter={() => setHudStep(idx)}
445 |         onClick={() => setHudStep(idx)}
446 |       >
447 |         <div className="hud-step-meta">
448 |           <span className="hud-step-tech-bracket">[</span>
449 |             <span className="hud-step-meta-icon-inline" style={{ display: 'inline-flex', alignItems: 'center', marginRight: '6px', color: hudStep === i
450 |               {getLifecycleIcon(item.iconName, 13)}
451 |             </span>
452 |             <span>STAGE {item.step}</span>
453 |             <span className="hud-step-tech-bracket">]</span>
454 |           </div>
455 |           <h3>{item.title}</h3>
456 |         </div>
457 |       )})
458 |   </div>
459 |
460 |   {/* Right side: Circular Radar Scope display */}
461 |   <div className="hud-radar-display">
462 |     {/* Corner crosshairs */}
463 |     <span className="hud-radar-crosshair ch-top-left">■</span>
464 |     <span className="hud-radar-crosshair ch-top-right">■</span>
465 |     <span className="hud-radar-crosshair ch-bottom-left">■</span>
466 |     <span className="hud-radar-crosshair ch-bottom-right">■</span>
467 |
468 |     {/* Spinning radar lines */}
469 |     <div className="hud-radar-scope">
470 |       <div className="hud-radar-ring-1" />
471 |       <div className="hud-radar-ring-2" />
472 |       <div className="hud-radar-sweep" />
473 |       <div className="hud-radar-center-icon">
474 |         {getLifecycleIcon(lifecycleSteps[hudStep].iconName, 72)}
475 |       </div>
476 |     </div>
477 |
478 |     {/* Tech details of the hovered step */}
479 |     <div className="hud-active-step-details">
480 |       <h4>{lifecycleSteps[hudStep].title}</h4>
481 |       <p>{lifecycleSteps[hudStep].detail}</p>
482 |
483 |       <div className="hud-tech-feed">
484 |         <span>COORD: {lifecycleSteps[hudStep].coord}</span>
485 |         <span>STATUS: {lifecycleSteps[hudStep].status}</span>
486 |       </div>
487 |     </div>
488 |   </div>
489 |
490 | </div>
491 | </div>
492 | </section>
493 |
494 | {/* 4. TEAM SECTION */}
495 | <section ref={teamRef} className="team-container">
496 |   <div className="team-heading-block reveal-on-scroll">
497 |     <h2 style={{ display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
498 |       <Users size={24} style={{ marginRight: '12px', color: 'var(--cyan-accent)' }} />
499 |       Built by the EduPulse AI Team
500 |     </h2>
501 |     <p>A focused engineering team combining frontend, backend, AI, and database development to build an admin-ready campus intelligence system.</p>
502 |   </div>
503 |
504 |   <div className="team-grid reveal-on-scroll">
505 |     <div className="team-member-card">
506 |       <div className="avatar-initials">HG</div>
507 |       <h3>Harshit Gouriariya</h3>
508 |       <div className="member-role">Team Lead & Backend Developer</div>
509 |       <div className="member-edu">4th Year B.Tech ECE, G.L. Bajaj</div>
510 |       <p className="member-desc">Led the team and worked on backend architecture, server-side logic, and system integration.</p>
511 |     </div>
512 |
513 |     <div className="team-member-card">
514 |       <div className="avatar-initials">GS</div>
515 |       <h3>Gaurav Sharma</h3>
516 |       <div className="member-role">AI & Database Developer</div>
517 |       <div className="member-edu">4th Year B.Tech ECE, G.L. Bajaj</div>
518 |       <p className="member-desc">Worked on AI development, feedback intelligence, and parts of the database system.</p>
519 |     </div>
520 |
521 |     <div className="team-member-card">
522 |       <div className="avatar-initials">NJ</div>
523 |       <h3>Nishant Jangra</h3>
524 |       <div className="member-role">Frontend Developer</div>
525 |       <div className="member-edu">4th Year B.Tech ECE, G.L. Bajaj</div>
526 |       <p className="member-desc">Designed and developed the frontend interface and user-facing experience.</p>
527 |     </div>
528 |
529 |     <div className="team-member-card">
530 |       <div className="avatar-initials">DR</div>
531 |       <h3>DevPratap Rai</h3>
532 |       <div className="member-role">AI Integration Developer</div>
533 |       <div className="member-edu">4th Year B.Tech ECE, G.L. Bajaj</div>
534 |       <p className="member-desc">Worked on AI development support and integration of intelligent analysis features.</p>
535 |     </div>
536 |
537 |     <div className="team-member-card">
538 |       <div className="avatar-initials">DT</div>
539 |       <h3>Deepesh Tiwari</h3>
540 |       <div className="member-role">Backend Developer</div>
541 |       <div className="member-edu">4th Year B.Tech ECE, G.L. Bajaj</div>
542 |       <p className="member-desc">Worked on backend development, data handling, and server-side implementation.</p>
543 |     </div>
544 |   </div>
545 | </section>

```

```

546 |
547 | /* 5. AI PIPELINE ANIMATION SECTION (3D Rotating Dice - Clickable & Auto-rolling) */
548 | <section ref={pipelineRef} className="pipeline-section">
549 |   <div className="pipeline-wrapper">
550 |     <div className="team-heading-block reveal-on-scroll">
551 |       <h2 style={{ display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
552 |         <Gauge size={24} style={{ marginRight: '12px', color: 'var(--fresh-green)' }} />
553 |         The Intelligent Pipeline
554 |       </h2>
555 |       <p>How student inputs flow through clean-ups and semantic math to build executive insights.</p>
556 |     </div>
557 |
558 |     <div className="pipeline-grid-layout reveal-on-scroll">
559 |       /* 3D Dice - Clicking advances steps */
560 |       <div className="dice-stage">
561 |         <div
562 |           className="pipeline-cube"
563 |           style={{ transform: getDiceRotation() }}
564 |           onClick={handleCubeClick}
565 |         >
566 |           /* Face 1: Front */
567 |           <div className={`cube-face face-front ${pipelineSteps[activeStep].face === 1 ? 'is-active-face' : ''}`}>
568 |             <div className="cube-face-icon" style={{ background: 'rgba(59, 130, 246, 0.08)', color: 'var(--bright-blue)' }}>
569 |               <Sparkles size={28} />
570 |             </div>
571 |             <h5>Ingestion</h5>
572 |             <p>Step 01: Raw feedback collection</p>
573 |           </div>
574 |
575 |           /* Face 2: Back */
576 |           <div className={`cube-face face-back ${pipelineSteps[activeStep].face === 2 ? 'is-active-face' : ''}`}>
577 |             <div className="cube-face-icon" style={{ background: 'rgba(236, 72, 153, 0.08)', color: 'var(--glow-pink)' }}>
578 |               <Filter size={28} />
579 |             </div>
580 |             <h5>Cleansing</h5>
581 |             <p>Steps 02-03: Filtering & spam deduplication</p>
582 |           </div>
583 |
584 |           /* Face 3: Right */
585 |           <div className={`cube-face face-right ${pipelineSteps[activeStep].face === 3 ? 'is-active-face' : ''}`}>
586 |             <div className="cube-face-icon" style={{ background: 'rgba(6, 182, 212, 0.08)', color: 'var(--cyan-accent)' }}>
587 |               <Activity size={28} />
588 |             </div>
589 |             <h5>Analysis</h5>
590 |             <p>Steps 04-05: Sentiment & Category classification</p>
591 |           </div>
592 |
593 |           /* Face 4: Left */
594 |           <div className={`cube-face face-left ${pipelineSteps[activeStep].face === 4 ? 'is-active-face' : ''}`}>
595 |             <div className="cube-face-icon" style={{ background: 'rgba(245, 158, 11, 0.08)', color: 'var(--alert-yellow)' }}>
596 |               <AlertOctagon size={28} />
597 |             </div>
598 |             <h5>Risk Scan</h5>
599 |             <p>Step 06: Urgent issues segregation</p>
600 |           </div>
601 |
602 |           /* Face 5: Top */
603 |           <div className={`cube-face face-top ${pipelineSteps[activeStep].face === 5 ? 'is-active-face' : ''}`}>
604 |             <div className="cube-face-icon" style={{ background: 'rgba(16, 185, 129, 0.08)', color: 'var(--fresh-green)' }}>
605 |               <FolderTree size={28} />
606 |             </div>
607 |             <h5>Clustering</h5>
608 |             <p>Steps 07-08: Normal issues clustering & priority math</p>
609 |           </div>
610 |
611 |           /* Face 6: Bottom */
612 |           <div className={`cube-face face-bottom ${pipelineSteps[activeStep].face === 6 ? 'is-active-face' : ''}`}>
613 |             <div className="cube-face-icon" style={{ background: 'rgba(30, 64, 175, 0.08)', color: 'var(--electric-blue)' }}>
614 |               <Gauge size={28} />
615 |             </div>
616 |             <h5>Reporting</h5>
617 |             <p>Steps 09-10: Gemini narratives & admin dashboards</p>
618 |           </div>
619 |         </div>
620 |       </div>
621 |
622 |       /* Description panel */
623 |       <div className="pipeline-desc-panel">
624 |         <div className="pipeline-step-card glow-card">
625 |           <div className="pipeline-header-block">
626 |             <h3>
627 |               <span style={{ marginRight: '12px', color: 'var(--bright-blue)', verticalAlign: 'middle', display: 'inline-flex' }}>
628 |                 {pipelineSteps[activeStep].icon}
629 |               </span>
630 |               {pipelineSteps[activeStep].title}
631 |             </h3>
632 |             <span className="pipeline-step-indicator">Step {pipelineSteps[activeStep].num} / 10</span>
633 |           </div>
634 |           <p className="step-details">
635 |             {pipelineSteps[activeStep].detail}
636 |           </p>
637 |         </div>
638 |
639 |         /* Step indicator dots */
640 |         <div className="pipeline-dots-progress">
641 |           {pipelineSteps.map((_, idx) => (
642 |             <span
643 |               key={idx}
644 |               className={`progress-dot ${activeStep === idx ? 'active' : ''}`}
645 |               onClick={() => handleDotClick(idx)}
646 |             />
647 |           ))}
648 |         </div>
649 |       </div>
650 |     </div>
651 |   </div>
652 | </section>
653 |
654 | /* 6. SAAS BUSINESS VALUE SECTION */
655 | <section className="business-value-wrapper">

```

```

656 | <div className="business-heading-center reveal-on-scroll">
657 |   <h2 style={{ display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
658 |     <Layers size={24} style={{ marginRight: '12px', color: 'var(--glow-pink)' }} />
659 |     Enterprise Value for Institutions
660 |   </h2>
661 |   <p>EduPulse AI is designed as a secure, scalable SaaS tool to minimize administrative effort and accelerate resolution times.</p>
662 | </div>
663 |
664 | <div className="value-cards-layout reveal-on-scroll">
665 |   <div className="value-panel-box">
666 |     <div className="value-icon-box">
667 |       <Users size={20} />
668 |     </div>
669 |     <h3>Faster issue visibility</h3>
670 |     <p>Spot critical and repeating issues immediately, bypassing days of manual reading.</p>
671 |   </div>
672 |
673 |   <div className="value-panel-box">
674 |     <div className="value-icon-box">
675 |       <Layers size={20} />
676 |     </div>
677 |     <h3>Admin-ready intelligence</h3>
678 |     <p>View consolidated analytics categorized by branch, semester, priority, and severity.</p>
679 |   </div>
680 |
681 |   <div className="value-panel-box">
682 |     <div className="value-icon-box">
683 |       <FileText size={20} />
684 |     </div>
685 |     <h3>Structured reports</h3>
686 |     <p>Download comprehensive PDF reports built directly from analyzed student inputs.</p>
687 |   </div>
688 |
689 |   <div className="value-panel-box">
690 |     <div className="value-icon-box">
691 |       <Lock size={20} />
692 |     </div>
693 |     <h3>Scalable SaaS model</h3>
694 |     <p>Deploy a tenant-separated architecture perfect for colleges and multi-campus universities.</p>
695 |   </div>
696 | </div>
697 | </section>
698 |
699 | {/ 7. FINAL PROJECT EXPLANATION + CTA */}
700 | <section className="final-cta-wrapper">
701 |   <div className="final-cta-card reveal-on-scroll">
702 |     <p className="cta-paragraph">
703 |       EduPulse AI is an intelligent campus feedback analysis system designed to help institutions understand student concerns with clarity, speed, and ac
704 |     </p>
705 |
706 |     <div className="cta-buttons-block">
707 |       <button className="btn-primary" onClick={() => setModalOpen(true)}>
708 |         Get Started <ArrowRight size={16} style={{ marginLeft: '4px' }} />
709 |       </button>
710 |       <button className="btn-secondary" onClick={() => triggerScroll(pipelineRef)}>
711 |         Explore Pipeline
712 |       </button>
713 |     </div>
714 |   </div>
715 | </section>
716 |
717 | {/ 8. FOOTER */}
718 | <footer className="footer-strip">
719 |   <div className="footer-content">
720 |     <h4>EduPulse AI</h4>
721 |     <p>Campus Feedback Intelligence</p>
722 |     <p>Built for institutional action &bull; &copy; {new Date().getFullYear()}</p>
723 |   </div>
724 | </footer>
725 |
726 | {/ UNIFIED ROLE SELECTION MODAL */}
727 | {modalOpen && (
728 |   <div className="role-modal-overlay" onClick={() => setModalOpen(false)}>
729 |     <div className="role-modal-box" onClick={(e) => e.stopPropagation()}>
730 |       <button className="role-modal-close" onClick={() => setModalOpen(false)}>
731 |         <X size={20} />
732 |       </button>
733 |       <h2>Select your role</h2>
734 |       <p className="role-modal-desc">Select your role to access the EduPulse AI portals.</p>
735 |
736 |       <div className="role-options-stack">
737 |         <button
738 |           className="role-option-button"
739 |           onClick={() => {
740 |             setModalOpen(false);
741 |             onChooseRole('STUDENT');
742 |           }}
743 |         >
744 |           <div className="role-option-icon">
745 |             <GraduationCap size={22} />
746 |           </div>
747 |           <div className="role-option-text">
748 |             <span className="role-option-title">Student Portal</span>
749 |             <span className="role-option-hint">Submit structured semester feedback</span>
750 |           </div>
751 |           <ChevronRight size={18} style={{ marginLeft: 'auto', opacity: 0.5 }} />
752 |         </button>
753 |
754 |         <button
755 |           className="role-option-button"
756 |           onClick={() => {
757 |             setModalOpen(false);
758 |             onChooseRole('ADMIN');
759 |           }}
760 |         >
761 |           <div className="role-option-icon">
762 |             <ShieldCheck size={22} />
763 |           </div>
764 |           <div className="role-option-text">
765 |             <span className="role-option-title">Institution Admin</span>

```

```

766 |         <span className="role-option-hint">Review reports, view analytics, and take action</span>
767 |     </div>
768 |     <ChevronRight size={18} style={{ marginLeft: 'auto', opacity: 0.5 }} />
769 | </button>
770 | </div>
771 | </div>
772 | </div>
773 | }}
774 | </div>
775 | );
776 | }

```

edupulse-frontend/src/index.css

File size: 413 bytes

```
1 | :root {
2 |   font-family:
3 |     Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
4 |     sans-serif;
5 |   color: #172033;
6 |   background: #f8fafc;
7 |   font-synthesis: none;
8 |   text-rendering: optimizeLegibility;
9 |   -webkit-font-smoothing: antialiased;
10 |   -moz-osx-font-smoothing: grayscale;
11 | }
12 |
13 | * {
14 |   box-sizing: border-box;
15 | }
16 |
17 | body {
18 |   margin: 0;
19 |   min-width: 320px;
20 | }
21 |
22 | #root {
23 |   min-height: 100vh;
24 | }
```

edupulse-frontend/src/main.tsx

File size: 230 bytes

```
1 | import { StrictMode } from 'react'
2 | import { createRoot } from 'react-dom/client'
3 | import './index.css'
4 | import App from './App.tsx'
5 |
6 | createRoot(document.getElementById('root')!).render(
7 |   <StrictMode>
8 |     <App />
9 |   </StrictMode>,
10 | )
```

edupulse-frontend/vite.config.ts

File size: 161 bytes

```
1 | import { defineConfig } from 'vite'
2 | import react from '@vitejs/plugin-react'
3 |
4 | // https://vite.dev/config/
5 | export default defineConfig({
6 |   plugins: [react()],
7 | })
```